



ArrayOS APV 10.7 CLI Handbook

Last Update: June 30, 2025

Copyright Notice

Copyright © 2025 Array Networks. All rights reserved.

This document is protected by copyright, and no part of this document may be used, copied, reproduced, distributed and compiled in any form by any means without prior written consent of Array Networks.

This Document may without further notice to be amended from time to time solely upon Array Networks' discretion. This document is provided "as is" without warranty of any kind, either express or implied, including but not limited to any kind of implied or express warranty of non-infringement or warranties of merchantability or fitness for a particular purpose.

Array Networks assumes no responsibility or liability arising from the use of products described herein, except as expressly agreed to in writing by Array Networks. The use and purchase of the products contain herein does not convey a license nor consent to assign any of Array Networks' patent, copyright, or trademark rights, or any other rights owned by Array Networks.



Warning: Modifications made to the Array Networks products, unless expressly approved by Array Networks, could void the user's authority to operate the products.

Revision History

Date	Description
2024-01-04	First version. Inherited from 10.4.3. Chapter 13 SSL Acceleration > Client Authentication ssl settings crt offline : Added the tls_flag parameter.
2024-03-18	- Updated the company logo. - Updated headers and footers
2024-03-21	Revised part of the content.
2024-04-22	Chapter 21 Application Security > DDoS Attack Defense ddos {on off} : Added new descriptions. Chapter 24 Logging > Log Customization log http custom: Added the unit of time (millisecond) to the descriptions of the time-related format specifiers. log nat custom: Updated the command's description.
2024-05-14	Chapter 25 Administrative Tools snmp traps loglevel : Added the description to explain what will happen if the default value of the log_level parameter is changed.
2024-07-18	Chapter 8 Server Load Balancing (SLB) > SLB DirectFWD slb directfwd syncache {on off} : Added a note to describe the packet and TCP option on APV. Chapter 25 Administrative Tools > SSH Access ssh cipher : Deleted chacha20-poly1305@openssh.com.
2024-10-04	Chapter 8 Server Load Balancing (SLB) > SLB DirectFWD slb directfwd {on off} [virtual_service] : Added the MSS description when this command is used with WebAgent. Chapter 25 Administrative Tools > Configuration Management Commands Added LDAP(S) to the following commands: admin aaa {on off} [priority] admin aaa authorize {on off} admin aaa method [method] admin aaa server <server_id> <host_name ip_address> <port> [secret] no admin aaa server <server_id> [certificate]
2024-10-23	Chapter 2 Basic System Operations show interface Added the descriptions for Receive errors and Driver dropped . Chapter 6 High Availability (HA) > SSF ha ssf {on off} [virtual_service] - In the virtual_service parameter's description, removed TCPS, HTTP, and HTTPS. - In Note, removed the entries related to TCPS, HTTP, and HTTPS virtual services.
2025-01-06	Chapter 2 Basic System Operations > NTP Updated the NTP section.
2025-02-10	Chapter 24 Logging Added the section Prometheus. Chapter 25 Administrative Tools > Management Access > WebUI Access webui ssl import certificate [url]: Added support for PFX certificates.
2025-03-13	The three predefined IP region tables predefined_cernet, predefined_cnc, and predefined_ct are replaced with new IP region

Date	Description
	tables: Chapter 18 Link Load Balancing (LLB) > LLB Route ipregion route: Updated the description of the ipregion_name parameter. • Chapter 25 Administrative Tools > Custom Configuration Script Added the clear directory var command. • Added Chapter 26 Cloud: Azure commands.
2025-04-09	• Chapter 2 Basic System Operations > NTP ▪ ntp key <id> <md5-key> Added special characters support to the md5-key parameter. ▪ show ntp <keys status> Added special character examples for MD5 strings. • Chapter 26 Cloud Added AWS commands.
2025-06-16	• Chapter 6 High Availability (HA) > SSF Added ha ssf filter {on off}. • Chapter 25 Administrative Tools > Management Access > WebUI Access webui ssl import certificate [url]: Both PEM and PFX only require entering the password once.
2025-06-30	Chapter 2 Basic System Operations Updated the example of the show statistics tcp command.

Contents

Chapter 1 CLI Basics	1
Command Line Symbols and Meanings	1
Regular Expressions.....	1
Login APV Appliance	2
Levels of Global Access Control	3
ShortHand	4
Chapter 2 Basic System Operations.....	5
NTP	35
Hostname	39
System.....	40
Website Classification.....	45
Bridge.....	46
SPAN Port.....	49
System Health Check.....	53
Chapter 3 Advanced System Operations.....	65
LLDP.....	85
IP Region Table.....	88
Basic IP Region Table Commands.....	88
GeoIP-based IP Region Table	90
Crontab.....	92
VXLAN.....	94
Chapter 4 Link Aggregation.....	101
Chapter 5 Clustering	104
Chapter 6 High Availability (HA).....	113
HA Unit.....	113

HA Links	114
Floating IP Group and Floating MAC	115
HA Health Check and Failover	121
Peer Unit HA Group Status Check.....	124
Configuration Synchronization	124
SSF	128
HA Consistency Check	131
HA Logging	131
General Configurations	132
Chapter 7 Single System Image (SSI)	134
Chapter 8 Server Load Balancing (SLB)	136
Real Service	136
Defining Real Service	136
General Configurations of Real Service	169
Real Service Group	171
Defining the Real Service Group and Group Method	171
Adding Real Services to the Group	211
General Real Service Group Configuration	213
Proxy IP Pool	219
HTTP/2 Group Support.....	221
Virtual Service	221
Defining the Virtual Service	221
Basic Virtual Service Configurations.....	243
Port Ranges of Virtual Services	247
Transfer Mode for Virtual Services.....	249
TCP Options for Virtual Services	251
HTTP/2 Virtual Service Support.....	253

Layer 2 SLB Local Exception IP	253
Other Configurations	255
Load Balancing Policy	256
Defining the Load Balancing Policy	258
Policy Nesting	289
Policy Order Template	290
Other Configurations	292
Health Check.....	293
Additional Health Check.....	293
Group Health Check	295
Additional Configurations for Specific Types of Health Checks	298
Layer 2 SLB Health Check	323
Passive Health Check.....	324
General Configurations of Health Check.....	330
Other SIP Commands.....	339
Basic SLB Commands	340
SLB Mode.....	340
SLB DirectFWD	343
Actively Closing TCP Connections	344
Overload Persistence.....	346
Host Node	346
Other Basic SLB Commands	348
Compatibility Check	354
SLB Statistics.....	355
Real Service Statistics.....	356
Group Statistics	356
Virtual Service Statistics	357

Load Balancing Policy Statistics	359
Health Check Statistics	362
Other Statistics	364
SLB Summary.....	364
Chapter 9 Application Security Orchestrator	367
Traffic Listener.....	367
Security Device	369
Security Service	372
Security Service Chain.....	377
Health Check.....	378
Orchestration Policy.....	383
Orchestration Rules.....	384
Network Orchestration Rules.....	384
EPolicy Integration	388
Orchestrator Statistics	388
Functions and Specifications	390
Other Orchestration Commands	390
Chapter 10 Segment Management	391
System Mode	391
Segment Mode	405
Chapter 11 HTTP Proxy.....	406
HTTP/2 Support.....	406
HTTP Rewrite	406
Header Rewrite	406
Content Rewrite	427
HTTP Security Control	434
URL Filtering	434

Access Control	447
HTTP Error Processing	449
HTTPS Forwarding Control	455
HTTP Compression.....	470
HTTP Cache.....	474
Other HTTP Commands	490
Chapter 12 DNS Cache	497
Chapter 13 SSL Acceleration	501
SSL Host Configuration.....	501
SSL Certificate Management	504
SSL Protocol Version and Cipher Suite	534
Signature Algorithm and Elliptic Curve	540
TLSv1.3 Feature Options.....	541
Client Authentication	543
Advanced Options for Client Authentication.....	561
Server Authentication.....	564
SSL Renegotiation	564
SSL Session Function Options.....	565
SSL Error Page Customization	568
SSL Tunneling.....	570
SSL General Commands	573
FIPS.....	577
Cryptographic Algorithm	578
Chapter 14 SSL Interception.....	579
SSL Interception and Certificate Settings	579
Domain List	581
URL Filtering	585

Chapter 15 Secure Application Access (SAA).....	594
AAA.....	594
SAML	600
RADIUS.....	603
LDAP	606
OAuth.....	611
Session Management	616
Web SSO	621
Chapter 16 Webagent	624
Webagent Service.....	624
Webagent DNS.....	625
Webagent Access Control	627
Chapter 17 Quality of Service (QoS).....	631
QoS Queue	631
QoS Filter Rule	633
Other QoS Commands	635
Chapter 18 Link Load Balancing (LLB).....	636
LLB Route	636
Outbound Link Load Balancing.....	647
Inbound Link Load Balancing	658
Link Health Check	660
IP Address Group Support	664
LLB Statistics.....	666
General Configurations of LLB	669
Chapter 19 Global Server Load Balancing (GSLB)	671
Basic SDNS Commands	671
SDNS Host Name	673

SDNS Listening IP Address	674
SDNS Service	675
SDNS Pool.....	677
SDNS Region.....	687
SDNS Policy	688
SDNS Monitor	690
SDNS Static and ipregion Proximity Rules	702
SDNS DPS	703
DNS Resource Record	710
SDNS Full DNS	714
SDNS IANA	714
SDNS DNSSEC	715
SDNS Data Center Synchronization	720
SDNS Link.....	727
SDNS Statistics.....	731
SDNS Configuration Backup.....	739
SDNS Configuration Loading.....	740
Chapter 20 Deep Packet Inspection (DPI)	743
Chapter 21 Application Security	751
WebWall.....	751
Access Groups	751
Access Rule.....	751
WebWall.....	756
Web Application Firewall (WAF)	757
Advanced ACL.....	763
ACL Rule	763
HTTP ACL Rule.....	768

DNS ACL Rule	772
DDoS Attack Defense	775
TCP DDoS	776
UDP DDoS.....	777
DNS DDoS.....	777
SSL DDoS.....	778
HTTP DDoS.....	779
ACL Blacklist	782
Chapter 22 Advanced IPv6 Configuration	787
DNS64 and NAT64	787
DNS46 and NAT46	789
General Settings	792
Chapter 23 ePolicy.....	794
Chapter 24 Logging	796
Basic Settings.....	796
RFC 5424 Syslog	798
Log Customization.....	800
Disabling Individual System Log	803
Local and Remote Log Host	804
Log Alert	809
SNMP Trap for System Logs	810
Application Visualization.....	811
Prometheus.....	812
Chapter 25 Administrative Tools	819
Configuration Management Commands	819
System Management.....	841
Management Access.....	843

WebUI Access	843
SSH Access	849
RESTful API Access	854
XML RPC Access	860
USB Storage Access	862
Role-based Privilege Management	863
Administrative Privilege Separation	870
WebUI Two-Factor Authentication	871
Configuration Synchronization Commands.....	872
Configuration Synchronization	872
Synchronization Rollback.....	876
Other Commands	877
SDNS Configuration Synchronization Commands	878
SNMP Commands.....	878
Troubleshooting Commands	885
Debug Commands.....	888
Remote Access Commands.....	896
Custom Configuration Script	897
Bandwidth management commands	899
Login Page Management Commands	900
Network Interface Card Management Commands	900
Chapter 26 Cloud	902
Microsoft Azure	902
Basic Settings.....	902
High Availability (HA).....	902
User-Defined Route	903
Logging.....	904

Amazon Web Services (AWS)	905
Basic Settings.....	905
High Availability (HA).....	906
Logging	907
Appendix I SNMP OID List	909
Appendix II Functions and Specifications	934

Chapter 1 CLI Basics

The APV Command Line Interface (CLI) is designed to maximize the functionality and performance of the APV appliance by allowing administrators to configure and control key functions of the APV appliance directly.

Command Line Symbols and Meanings

This CLI Handbook covers the proper use and execution of each command available to the APV appliance administrator and user alike. The commands covered in this handbook will adhere to these general conventions:

Style	Convention
Bold typeface	The body of a CLI command is in Boldface.
<i>Italic</i>	CLI parameters are in Italic.
< >	Parameters in angle brackets < > are required.
[]	Parameters in square brackets [] are optional. Subcommand such as “ no ”, “ show ” and “ clear ”.
{ x y ... }	Alternative items are grouped in braces and separated by vertical bars. One should be selected.
[x y ...]	Optional alternative items are grouped in square brackets and separated by vertical bars. One or none is selected.



Note:

- It is recommended to enclose the string-type parameter value by double quotes to make sure that the appliance can execute the command correctly.
- When administrators input a CLI, the maximum length of each line is 2048 bytes.

Regular Expressions

The APV appliance supports Perl-based regular expressions for the parameters of some command lines. Regular expression matching has two types: quick regular expression matching and full regular expression matching. The following table lists the Perl meta-characters supported by these two types of regular expressions.

Type	Supported Perl Meta-characters
Quick regular expression	Only the following three Perl meta-characters are supported: “^”: matches the beginning characters of the line. “*”: matches zero or more characters ahead of it. “\$”: matches the ending characters of the line.
Full regular expression	Compared with the quick regular expression, more Perl meta-characters are supported: “^”: matches the beginning characters of the line. “.”: matches any character.

Type	Supported Perl Meta-characters
	“+”: matches one or more characters ahead of it. “*”: matches zero or more characters ahead of it. “?”: matches zero or one character ahead of it. “\$”: matches the ending characters of the line. “[]”: matches any character in the square bracket. “ ”: matches the character ahead of or following it. “()”: matches the characters in the bracket as a group.

**Note:**

- The meaning of “*” in Perl-based regular expressions differs from “*” in wildcard expression. A single “*” must be avoided in a regular expression. The meaning of “.*” in Perl-based regular expression is the same as that of “*” in wildcard expression.
- When any of the Perl meta-characters needs to be matched, the “\” symbol should be used ahead of it for escaping. For example, to match the “*” symbol, configure “*”.
- When a command has two or more parameters supporting regular expressions, whether the appliance will use regular expression matching depends on whether the first parameter value is set to a regular expression. If the first parameter among those that support regular expressions is not set to a regular expression, other parameters’ regular expressions will not take effect.
- For a command that supports both the priority parameter and full regular expression match, the priority setting takes effect only when all configurations of this command use full regular expression match, or when all use quick regular expression and exact match. If the matched configurations contain both the one using full regular expression and the one using quick regular expression or exact match, the appliance preferentially selects the one using quick regular expression or exact match.

Login APV Appliance

After getting connected to the APV appliance successfully via SSH or Console connection, the administrator will be prompted for a login username and a password. The default/initial login username and password are “array” and “admin”.

The APV appliance provides the recovery mechanism for the “array” account to allow administrators to:

- Recover the password of the “array” account if changing the password of the “array” administrator account and forgetting the new password.
- Recover the “array” account if it is deleted mistakenly.

To recover the password of the “array” account or the entire “array” account, please perform the following steps:

1. Establish a Console connection with the APV appliance.
2. Input the command “**recovery**” in the CLI.
3. Copy the challenge string generated by the APV appliance and pasted it in an email sent to **Customer Support** to request the response string. The challenge string is the string behind “challenge:”, for example “challenge: waker Parma baker galah woke”.
4. Paste the entire response string returned by **Customer Support** behind the “response:” prompt and press “Enter”. The response string begins with “--begin--” and ends with “--end--”.

After the preceding steps are performed, if the “array” account exists, the system will reset the password of the “array” account to “admin” and the access privilege to “Config”, if the “array” account does not exist, the system will create the “array” account with password “admin” and the access privilege “Config”. After the “array” account or its password is recovered, make sure to log in with the “array” account and modify its password. If not, after a system reboot, the system can be logged in with only the account and password used before the recovery operation.

Otherwise, the system also supports accessing the appliance via WebUI, XML-RPC, RESTful API and so on.

Levels of Global Access Control

The APV appliance offers three levels or modes for global configuration and access to the ArrayOS. Each mode is designated by a unique cursor prompt. The CLI prompt consists of the host name of the APV appliance followed by either “>”, “#” or “(config)#”.

- User Mode

The first level is User Mode. Here, the user is only authorized to execute some very basic operations and non-critical functions. The User Mode prompt appears as “AN>” in the CLI.

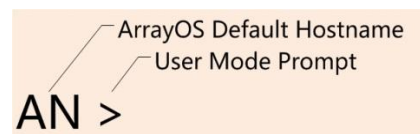


Diagram illustrating the User Mode CLI prompt. The prompt is shown as "AN >". A line points from the text "ArrayOS Default Hostname" to "AN", and another line points from the text "User Mode Prompt" to ">".

- Enable Mode

Users in this mode have access to a majority of view only commands such as “**show log config**”. Commands from both the User and Enable modes can be executed. Once accessing the Enable mode successfully, the CLI prompt changes from “AN>” to “AN#”.

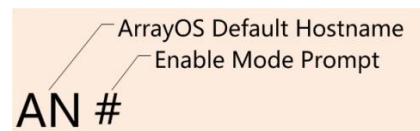
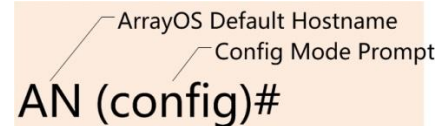


Diagram illustrating the Enable Mode CLI prompt. The prompt is shown as "AN #". A line points from the text "ArrayOS Default Hostname" to "AN", and another line points from the text "Enable Mode Prompt" to "#".

- Config Mode

The final level is Config mode. At this level, users can make changes to any part of the APV appliance configuration. By default, no two users may access the Config mode at the same time. In the Config mode, the CLI prompt will change from “AN#” to “AN(config)#”.



ArrayOS Default Hostname
Config Mode Prompt
AN (config)#

By executing the command “**config multiconfig enable**”, the system allows multiple administrator accounts to be simultaneously in the Config mode or one administrator account to simultaneously establish multiple SSH connections of the Config level authority.



Note: In the ArrayOS, users can be assigned with Enable or Config access privilege. The Enable users cannot access the Config mode.

ShortHand

The ArrayOS has been designed with Shorthand to make interaction user friendly by allowing the APV appliance to intuitively complete CLI commands based on the first letters entered. Other user shortcuts are listed below:

CLI Shortcuts	Operation
^a / _a	Move the cursor to the beginning/end of a line.
^f / _f ^b / _b	Move the cursor forward/backward one character.
Esc-f	Move the cursor forward one word.
Esc-b	Move the cursor backward one word.
^d	Delete the character under the cursor.
^k	Delete from the cursor to the end of the line.
^u	Delete the entire line.



Note: The symbol “[^]” indicates holding down the Control (Ctrl) Key while pressing the letter that appears after the symbol.

Chapter 2 Basic System Operations

This chapter covers basic system configuration commands, including the commands for configuring:

- System IP addresses, routes, and interfaces
- System tuning parameters
- Alarm mail and mail relay function
- System health check function

?

This command is used to display the first keyword for all commands available at the current access level, or the next keyword for the current command.

help

This command is used to display the first keyword for all commands available at the current access level.

enable [recovery]

The “enable” command is used to access the Enable level of the ArrayOS. After executing this command, the system will prompt the user to supply the Enable level password. The default password is null (empty). The “**enable recovery**” command is used to reset the Enable level password when required. The reset procedure is as follows:

1. Enter “**enable recovery**” at the User level prompt. A challenge string will be displayed.
2. Email the challenge string to Customer Support. A response code will be returned via email by the Customer Support personnel.
3. Copy and paste the response code in the CLI, and press “Enter”. The password of the Enable level will be reset to empty.

disable

This command is used to return from the current access level to the User level.

exit

This command is used to return from the Config level to the Enable level. Executing the command at Enable level or User level, the Administrator will exit the CLI.

quit

This command is used to exit the CLI. It can be executed at any time throughout the configuration process.

switch user <user_name> <password>

This command is used to switch from the current administrator account to another administrator account. After switch, the current user session will be closed.

user_name	This parameter specifies the username of the administrator account to switch. Its value can be the username of a system administrator account or a segment administrator account.
password	This parameter specifies the password of the administrator account to switch.

show tech

This command is used to capture and display essential system information in real time.

show system warning [module]

This command is used to display the real-time system warning information.

module	Optional. This parameter specifies the function module name. Its value must be: • system: displays system-related warning information, such as hardware issues including the CPU fan, CPU temperature, system temperature, power supply, SSL acceleration card and NIC. • slb: display SLB-related warning information, such as limit-cross issues of concurrent connections (CC) and connections per second (CPS) on virtual services and real services. • all: displays warning information about both the system and SLB modules. The default value is “all”.
--------	--

show system status

This command is used to display system status, including CPU utilization, disk utilization, memory utilization (not displayed when smaller than 1%), temperature, fan speed, power supply and interface status.

show system ipmi <type> <mode> [entryid|sensorid|entityid|sensortype]

Displays the information on the current System Event Log (SEL) or Sensor Data Records (SDR).

type	This parameter specifies the type of the information to be displayed. The value must be as follows: • sel: Displays the information on the current System Event Log. • srd: Displays the information on the current Sensor Data Records.
mode	This parameter specifies the display mode the information. When the type value is sel, the mode value must be as follows: • list: Displays all the SEL information, including ID, Date, Time, Sensor Type/Num, Description, and EventDirection. • get: Displays the detailed SEL information that is the same as the filter string specified by the parameter entryid, including SEL Record ID (System Event Log Record ID), Record Type (for example, 02 is Generic Discrete), and Timestamp (event timestamp). For more information about other fields, see the IPMI specification. When the type value is sdr, the mode value must be as follows: • list: Lists all the SDR information, including Name, Type, State, Entity, and Value. • get: Displays the detailed SDR information that is the same as the filter string specified by the parameter sensorid, including Sensor ID (the unique ID identifying a sensor), Entity ID (the entity identifier associated with a specific sensor in SDR), and Sensor Type (A sensor's type and threshold). For more information about other fields, see the IPMI specification. • type: – If the parameter sensortype is not specified, the corresponding relationship between all type names and type values in SDR will be displayed. Type values are represented by two hexadecimal digits. For example, the type name "OS Boot" corresponds to the type value "0x1f." – If the parameter sensortype is specified, the detailed SDR information that is the same as the filter string specified by sensortype will be displayed, including Name, Type, State, Entity,

and **Value**.

- **entity:**
 - If the parameter **entityid** is not specified, the corresponding relationship between the Entity ID and Entity in all SDR will be displayed. For example, if Entity ID is zero, it corresponds to the Entity “Unspecified.”
 - If the parameter **entityid** is specified, the detailed SDR information that is the same as the filter string specified by **entityid** will be displayed, including **Name**, **Type**, **State**, **Entity**, and **Value**.

entryid|sensorid|

Optional. This parameter specifies the filter string used to display related information. It needs to be configured in the following cases:

sensortype|entityid

- **entryid:** Log Entry ID. This parameter needs to be configured when the **type** value is **sel** and the **mode** value is **get**, and its value must be the same as the ID displayed by the command **show system ipmi sel list**.
- **sensorid:** Sensor ID. This parameter needs to be configured when the **type** value is **sdr** and the **mode** value is **get**, and its value must be the same as the ID displayed by the command **show system ipmi sdr list**.
- **sensortype:** Sensor Type. It’s optional to configure this parameter (see the parameter **mode**) when the **type** value is **sdr** and the **mode** value is **type**, and its value must be the same as the type value displayed by the command **show system ipmi sdr type**.
- **entityid:** Entity ID. It’s optional to configure this parameter (see the parameter **mode**) when the **type** value is **sdr** and the **mode** value is **entity**, and its value must be the same as the ID displayed by the command **show system ipmi sdr entity**.

ip address <interface_name> <ip_address> {netmask|prefix} [overlap]

This command is used to set the IP address and netmask or prefix length for the system interface, MNET interface, VLAN interface, VXLAN interface or bond interface.

interface_name

This parameter specifies the name of the interface, which can be a system interface, MNET interface, VLAN interface, VXLAN interface or Bond interface.

For the system interface, the interface name can be

	customized using the “interface name” command. The default system interface name is port1, port2, port3, port4 and so on. For the bond interface, the default bond interface name is bond1, bond2, bond3, bond4 and so on.
ip_address	This parameter specifies the IP address of the interface. Its value must be an IPv4 or IPv6 address.
netmask prefix	This parameter specifies the netmask or prefix length of the interface IP address. • “netmask” indicates the netmask of the IPv4 address. Its value must be in dotted decimal notation or an integer ranging from 0 to 32. • “prefix” indicates the prefix length of the IPv6 address. Its value must be an integer ranging from 0 to 128.
overlap	Optional. This parameter specifies the “overlap” property for the specified interface, indicating that the subnet of the specified interface overlaps with that of an interface defined previously. Each subnet can only have one interface with “overlap” property. An interface defined as “overlap” need to be used only when Non-uniform Memory Access (NUMA) SLB (single VIP, proxy reverse mode) is deployed. This parameter is not allowed for the VLAN interface. The system supports a maximum of 16 interfaces with “overlap” property.

Example:

```
AN(config)#ip address port1 209.120.10.0 255.255.255.0
AN(config)#ip address port2 209.120.10.1 255.255.255.0 overlap
```

no ip address <interface_name> [ip_type]

This command is used to delete the IP address of the specified interface.

interface_name	This parameter specifies the name of the interface.
ip_type	Optional. This parameter specifies the type of the IP address to be removed. • 4: indicates the IPv4 address. • 6: indicates the IPv6 address. • The default value is 4.

show ip address

This command is used to display system IP address along with the assigned netmask.

clear ip address

This command is used to clear all the configured IP addresses. Delete the virtual IP configured on the interfaces before deleting the system IP.

ip statistics {on|off}

This command is used to enable or disable the IP statistics function. By default, this function is disabled.

In addition to IP related statistics, this function also controls whether to collect CPS statistics of virtual services and real services, which can be viewed via the “**show statistics slb all**” command.

The status of this function does not affect the CPS restriction function of virtual service and real services, which are configured by the “**slb virtual application cps**” and “**slb real application cps**” commands respectively.

show ip statistics

This command is used to display the status of the IP statistics function.

show statistics ip [ip_address]

This command is used to display the statistics for the specific IP address. If the IP address is not specified, the statistics for all IP addresses will be displayed.

For example:

AN(config)# show statistics ip					
Host: AN					
Current Time: 06/26/12 15:00					
IP	Pkts In	Bytes In	Pkts Out	Bytes Out	Start Time
10.8.6.62	14565	1242430	48147	9178799	06/25/12 16:22
192.168.1.10	0	0	0	0	06/21/12 14:37
172.5.6.26	0	0	0	0	06/21/12 14:37
10.212.196.33	0	0	0	0	06/21/12 14:37
21da:d3:0:2f3b:2aa:ff:fe28:9c5a	0	0	0	0	06/21/12 14:37

Total:	14565	1242430	48147	9178799	

clear statistics ip [ip_address]

This command is used to clear the statistics for the specified IP address. If the IP address is not specified, the statistics for all IP addresses will be cleared.

ip dhcp {on|off} <interface_name>

This command is used to enable or disable the DHCP function for the specified system interface. After this function is enabled, the specified system interface will obtain the IP address from the DHCP server automatically. By default, this function is disabled.

interface_name	This parameter specifies the name of the system interface, which can be customized by the “interface name” command. The default system interface name is port1, port2, port3, port4, etc.
----------------	---



Note: After the DHCP function is enabled for the specified system interface, if the administrator executes the “**write memory**” command, the system will not save the static IP configuration of this system interface.

ip arp <ip_address> <mac_address>

This command is used to add a static ARP entry to the system. After system reboot, the static ARP entries still exist while the dynamic ARP entries will be cleared. A maximum of 128 static ARP entries can be added to the system.

ip_address	This parameter specifies the IP address of the remote host. Its value must be an IPv4 address.
mac_address	This parameter specifies the MAC address of the remote host.

no ip arp <ip_address>

This command is used to delete a static or dynamic ARP entry of the specified IP address.

show ip arp [ip_address]

This command is used to display the ARP entry of the specified IP address. If the “ip_address” parameter is not specified, all the static and dynamic ARP entries will be displayed.

clear ip arp

This command is used to clear all static and dynamic ARP entries.

ip host <host_name> <ip_address>

This command is used to add a DNS resource record, which will be used in DNS resolution.

host_name	This parameter specifies the host name.
-----------	---

`ip_address` This parameter specifies the IP address of the host name.

no ip host *<host_name> [ip_address]*

This command is used to delete a DNS resource record.

show ip host

This command is used to display all DNS resource records.

clear ip host

This command is used to clear all DNS resource records.

ip route default *<gateway_ip>*

This command is used to configure a default system route. Only one default route is permitted to be configured.

`gateway_ip` This parameter specifies the IP address of the default gateway. Its value must be an IPv4 or IPv6 address.



Note: If a default IPv4 route is configured using this command in the segment mode, a default IPv4 route needs to be configured using this command in the system mode.

no ip route default *<gateway_ip>*

The command is used to delete the default system route.

ip route static *<destination_ip> {netmask|prefix} <gateway_ip>*

This command is used to configure a static route. Multiple static routes are permitted.

`destination_ip` This parameter specifies the destination IP address of the static route. Its value must be an IPv4 or IPv6 address.

`netmask|prefix` This parameter specifies the netmask or prefix length of the destination IP address.

- “netmask” is used for an IPv4 address. Its value must be a dotted IP address.
- “prefix” is used for an IPv6 address. Its value must be an integer ranging from 0 to 128.

`gateway_ip` This parameter specifies the gateway IP address for the static route. Its value must be an IPv4 or IPv6 address. When an IPv6 link-local address is specified, the port number must also be specified, and the parameter value

should be in the format of
“IPv6_link-local_address%%port_number”.

no ip route static <destination_ip> {netmask|prefix} <gateway_ip>

This command is used to delete a static route.

show ip route

This command is used to display routing table information about the default route, management routes, static routes and dynamic routes.

clear ip route

This command is used to clear all the default route, static route and dynamic route.

show route match <source_ip> <source_port> <destination_ip>
<destination_port> <protocol> [interface_name] [action_table]

This command is used to display the routes or route groups that match the specified source IP, source port, destination IP, destination port and protocol type, or display the access rules that match the specified source IP, source port, destination IP, destination port, protocol type, interface name and access rule type.

source_ip	This parameter specifies the source IP address.
source_port	This parameter specifies the source port.
destination_ip	This parameter specifies the destination IP address.
destination_port	This parameter specifies the destination port.
protocol	This parameter specifies the protocol type. Its value must be: • tcp: indicates TCP protocol type. • udp: indicates UDP protocol type. • icmp: indicates ICMP protocol type. • ah: indicates the Authentication Header (AH) network authentication protocol, which is used to match the access rule. • esp: indicates the Encapsulating Security Payload (ESP) network authentication protocol, which is used to match the access rule. • any: indicates any protocol type.
interface_name	Optional. This parameter specifies the interface name. The default value is null (Use “” to represent null), which

indicates all interfaces.

action_table

Optional. This parameter specifies the access rule type. Its value must be:

- 1: indicates the permit rule type.
- 2: indicates the deny rule type.
- The default value is 1.



Note: If NAT is configured and the NAT rule is matched, executing the command “**show route match**” also displays the matched NAT configuration and the hit virtual IP address.

interface mac <interface_name> <mac_address>

This command is used to set the MAC address for the specified system interface.

interface_name

This parameter specifies the system interface name.

mac_address

This parameter specifies the MAC address of the specified system interface. Even if the system interface has been configured as VLAN, MNET, bond interface or configured for SLB virtual services, its MAC address will not be changed.

no interface mac <interface_name>

This command is used to reset the MAC address of the specified system interface to the default value.

clear interface mac

This command is used to reset the MAC addresses of all system interfaces to the default value.

interface name <interface_id> <interface_name>

This command is used to customize the interface name.

interface_id

This parameter specifies the interface ID. The number of physical interfaces supported by the APV appliance varies by models.

The default system interface name is port1, port2, port3, port4, etc.

interface_name

This parameter specifies the interface name. Its value must be a string of 1 to 31 characters enclosed in double quotes. Letters, numbers, and special characters are supported.

no interface name <interface_id>

This command is used to reset the specified interface name to the default name.

clear interface name

This command is used to reset all the interface names to the default names.

interface description <interface_id> <description>

This command is used to configure the description information of the specified interface.

interface_id	This parameter specifies the name of the interface, which can be a system interface, VLAN interface, VXLAN interface or Bond interface. The default system interface ID is port1, port2, port3, port4 and so on. And the default bond interface ID is bond1, bond2, bond3, bond4 and so on. Note: The number of physical interfaces supported by the APV appliance varies with models.
description	This parameter specifies the interface description information. Its value must be a string of 1 to 255 characters.

no interface description <interface_name>

This command is used to delete the description information of the specified interface.

interface mtu <interface_id> <mtu >

This command is used to change the Maximum Transmission Unit (MTU) for the specified interface. The default value is 1500 bytes.

interface_id	This parameter specifies the interface name. Its value must be a VLAN interface name, bond interface name or default physical interface name (such as port1, port2, port3, etc).
mtu	This parameter specifies the interface MTU value, in bytes. The maximum MTU supported by an interface will vary according to the NIC type (the maximum MTU supported by some NIC interfaces can reach 65535 bytes). If the “interface_id” parameter is set to a VLAN interface name and its parent interface is a 10G Ethernet interface, a 4-byte VLAN tag will be added into the Ethernet frame header. In this situation, the parameter value should be at

least 4 smaller than that of its parent interface.

**Note:**

1. When both a bond interface and its associated physical interface have MTU configured, the size of a packet sending from the bond interface complies with the MTU configured for the bond interface.
2. According to RFC 2460, IPv6 requires that every Linux interface on the Internet must have an MTU of 1280 octets or greater.

clear interface mtu <interface_id>

This command is used to reset the MTU of the specified interface to the default value.

interface_id	This parameter specifies the interface name. Its value must be a VLAN interface name, bond interface name, default physical interface name (such as port1, port2, port3, etc) or “all”. The value “all” means the MTUs of all interfaces will be reset to default..
--------------	---

interface speed <interface_id> <speed_option>

This command is used to set the interface speed and working mode for the specified system interface. If the interface is connected to a device (such as a router or switch with a specific speed and duplex mode) , the administrator needs to configure the interface speed and working mode to match those requirements. Use the “**show interface**” command to view the current speed setting.

interface_id	This parameter specifies the system interface name. The number of system interfaces supported by the APV appliance varies by model.
--------------	---

The default system interface name is port1, port2, port3, port4, etc.

speed_option	This parameter specifies the interface speed and working mode. Its value must be: • 10half: indicates 10 Mbps Ethernet half duplex communication. • 100half: indicates 100 Mbps Ethernet half duplex communication. • 100full: indicates 100 Mbps full duplex communication. • 1000full: indicates 1000 Mbps Ethernet full duplex communication. • auto: indicates that the interface speed and working mode is self-adaptive. The interface speed and working
--------------	---

mode for 10 Gigabit Ethernet interface can only be set to “auto”.

The default value is “auto”.

clear interface speed [interface_id]

This command is used to delete the interface speed and working mode settings for the specified interface. If the “interface_id” is not specified, the interface speed and working mode settings for all interfaces will be deleted.

interface promisc <interface_id> {enable|disable}

This command is used to enable or disable the promiscuous mode for the specified system interface. If the promiscuous mode is enabled, the specified interface will also process the packets whose destination MAC address is not local. By default, this function is disabled.

interface_id This parameter specifies the system interface name. The number of the system interfaces supported by the APV appliance varies by model.

The default system interface name is port1, port2, port3, port4, etc.

clear interface promisc

This command is used to disable the promiscuous mode for all system interfaces.

interface management <interface_id> <gateway_ip>

This command is used to set the specified system interface or bond interface as the management port and set the gateway IP address. If a system interface bears VLAN configurations, it cannot be set as a management port. The system supports only one management interface, on which one IPv4 gateway and one IPv6 gateway can be specified.

A management interface supports management traffic only, such as SSH, WebUI, SNMP and RESTful API traffic.

interface_id This parameter specifies the system interface name or bond interface name, for example, port1, bond1.

gateway_ip This parameter specifies the gateway IP address. Its value can be IPv4 or IPv6 address. The gateway IP address cannot be “0.0.0.0” and cannot be the same as an interface IP address.

no interface management <interface_id> <gateway_ip_type>

This command is used to delete the specified management port.

<code>interface_id</code>	This parameter specifies the system interface name or bond interface name, for example, <code>port1</code> , <code>bond1</code> .
<code>gateway_ip_type</code>	Optional. This parameter specifies the gateway IP type. Its value must be: • 4: deletes the IPv4 gateway. • 6: deletes the IPv6 gateway. The default value is 4.

clear interface management

This command is used to clear all management ports.

show management

This command is used to display all management ports.

interface bandwidthtimer *[interval]*

This command is used to set the statistical interval of interface bandwidth.

When this command is not configured, the default statistical interval is 300s. Administrators can view the statistical interval setting of interface bandwidth via the “**show interface**” command.

<code>interval</code>	Optional. This parameter specifies the statistical interval of interface bandwidth, in seconds. Its value must be an integer ranging from 5 to 300. The default value is 300.
-----------------------	--

no interface bandwidthtimer

This command is used to restore the statistical interval setting of interface bandwidth to default.

interface shutdown *<interface_id>*

This command is used to manually shut down the specified physical interface. After the physical interface is shut down, the following changes occur:

- The gateway status changes to Down
- The interface no longer allows the following properties to be configured:
 - MTU value, management interface, MAC address, promiscuous mode, and interface speed and operating mode

- The interface retains configuration for the following properties:
 - Interface name, interface description, bandwidth statistics interval
- When the physical interface is shut down, the VLAN interfaces under the physical interface display “no carrier”
- When all physical interfaces under the bond interface are shut down, the bond interface displays “no carrier”
- The interface status in the Segment changes to Down, and the gateway status changes to Down

If the physical interface is manually shut down, the administrator will see the displayed information of “status: down (Administratively down)” through the “**show interface**” command.

`interface_id` This parameter specifies the interface name. The default system interface name is port1, port2, port3, port4, etc.



Note: After the physical interface is manually closed, the administrator will see the displayed information of “status: down (Administratively down)”. In this case, the system will not check the interface status anymore. If the administrator continues to pull out the interface line, the interface status is still “down (Administratively down)”.

no interface shutdown <interface_id>

This command is used to cancel the administrative down state of the specified physical interface.

clear interface shutdown

This command is used to cancel the administrative down state of all physical interfaces.

show interface [interface_id]

This command is used to display the configuration and statistics information for the specified system interface, VLAN interface, VXLAN interface or bond interface. If the “interface_id” parameter is not specified, the configuration and statistics information for all interfaces will be displayed.

The following table lists the meaning of each command’s output:

Output	Meaning
Input queue size	Current occupied input.
Input queue max	Maximum items of input.
Runt	Number of frames received that have passed address filtering, that are less than the minimum size (64 bytes), and have a valid CRC.
Giant	Number of frames received that have passed address filtering, that are larger than the maximum size, and have a valid CRC.
Jabber	Number of frames received that have passed address filtering, that are

Output	Meaning
	larger than the maximum size, and have a bad CRC. It may be the result of a bad NIC or electronic interfering.
CRC	Number of packets received with alignment errors.
Flow Control	Number of unsupported flow control frames that are received.
Fragments	Number of fragment received that have passed address filtering, that are less than the minimum size, and have a bad CRC.
Frame	Number of packets received with alignment errors (the packet length is not an integer).
Receive errors	Number of bad packets received.
Driver dropped	Number of packets dropped by the network adapter due to receive buffer overflow caused by incoming traffic rate exceeding buffer capacity at high link speed.
Lengths	Number of length error events received.
No Buffers	Number of times that frames are received when there are no available buffers to store them.
Overruns	Number of missing packets. Packets are missing when the received FIFO has insufficient space to store the incoming packets. This can be caused by too few allocated buffers, or insufficient bandwidth on the PCI bus.
Carrier extension errors	Number of received packets where the carrier extension error is signaled across the internal PHY interface.
Collisions	Total number of collisions that are not late collisions as seen by the transmitter.
Late collisions	Number of collisions that occur after 64-byte time into the transmission of the packet while working in 10-100 Mb/s data rate, and after 512-byte time into the transmission of the packet while working in the 1000 Mb/s data rate.
Deferred	A deferred event occurs when the transmitter cannot immediately send a packet because the medium is busy or another device is transmitting.
Single Collisions	Number of times that a successfully transmitted packet has encountered only one collision.
Multiple Collisions	Number of times that a successfully transmitted packet has encountered more than one collision but less than 16.
Excessive collisions	Number of times that 16 or more collisions have occurred on a packet.
Packet drop (not permit)	Number of packets dropped for not matching any permit access rules.
Packet drop (deny)	Number of packets dropped for matching deny access rules.

**Note:**

- If the IP statistics function is disabled (default), the number of packets permitted or dropped by the WebWall will be 0 in the output of the command “**show interface**”.
- Please enable the IP statistics function by the “**ip statistics on**” command before viewing the traffic statistics of VLAN or bond interfaces.

system tune accel mq

This command is used to enable the multi-queue load balance acceleration function for the APV appliance. When multiple VIPs are configured, use this command to accelerate the Layer 4 SLB and LLB features. To enable this feature, the administrator should firstly enable the DirectFWD function by the “**slb directfwd on**” command.



Note: When this function is enabled, the traffic cannot be transferred between the 10G NIC and 1G NIC on the APV appliance.

no system tune accel mq

This command is used to disable the multi-queue load balance acceleration function for the APV appliance.

system tune attackfilter <filter_level>

This command is used to set the filtering level of invalid IP packets.

filter_level	This parameter specifies the filtering level of invalid IP packets. Its value must be: • 0: disables the IP packets filtering function, which means all packets are permitted to enter into the APV appliance through the Ethernet card. • 1: drops the packets when the packets match one of the following cases: - Source IP or destination IP is 0.0.0.0 - Source IP is 255.255.255.255 - Source IP is 224.x.x.x - TCP port or UDP port is 0. In this case, administrators should set up firewall on specific interface to defend against suspicious network attacks. • 2: drops the packets when the packets match one of the following cases: - Source IP or destination IP is 0.0.0.0 - Source IP is 255.255.255.255 - Source IP is 224.x.x.x - TCP port or UDP port is 0. In this case, administrators should set up firewall on specific interface to defend against suspicious network attacks. - Source IP is the local IP address (but the packets are received by Ethernet interfaces).
--------------	--

The default value is 0.

no system tune attackfilter

This command is used to restore the configuration of filtering level of invalid IP packets.

show system attackfilter

This command is used to display the statistics information of the attack packets dropped by the APV appliance.

system tune ip randomid {on|off}

This command is used to enable or disable the function of setting a random number for an IP packet. When this function is enabled, each IP packet will be assigned a random number. When this function is disabled, the IP packet ID number will be increased sequentially. By default, this function is disabled.

no system tune ip randomid

This command is used to reset the feature of setting a random number for an IP packet to the default configuration.

system tune route numaaccel {on|off}

This command is used to enable or disable the routing acceleration function in NUMA. By default, this function is disabled.



Note: This function may affect other communication traffic. For how to use this function, please contact Customer Support.

no system tune route numaaccel

This command is used to reset the routing acceleration function in NUMA to the default configuration.

system tune hwcksum {on|off}

This command is used to enable or disable checksum offloading on the network card. This function takes effect on both IPv4 and IPv6 packets. By default, this function is enabled.

no system tune hwcksum

The command is used to restore the checksum offloading on the network card to its default configuration.

system tune numa {auto|off}

The command “**system tune numa auto**” is used to enable automatic detection of the NUMA function. The execution of this command enables the NUMA function on the appliances that support it, but disables it on the appliances that do not support it. By default, this function is disabled.

The command “**system tune numa off**” is used to disable the NUMA function.

The configuration of the command “**system tune numa**” takes effect only after the system reboot. Use the command “**show system tune**” to view the NUMA configuration.



Note: If automatic detection of the NUMA function is enabled:

- In the scenario where no cross-domain bond interfaces are configured, traffic will be sent out through the system interface connected with the real service or gateway.
- In the scenario where cross-domain bond interfaces are configured:
 - Service traffic: Service traffic will be sent out through another system interface in the NUMA domain to which the ingress interface belongs.
 - Non-service traffic: IP traffic will be sent out through another system interface in the NUMA domain to which the ingress interface belongs. UDP and TCP traffic will be sent out through the other NUMA domain.

show system numa

This command is used to display the status of the NUMA function, the ports of a NUMA domain and the binding relationship between the NIC and the NUMA domain (you can use this command to determine whether the NIC is cross-domain).

By default, this function is enabled.

system tune tcpidle <idle_time>

This command is used to set the maximum idle time of TCP connection. Once the idle time ends, the TCP connection will be terminated. By default, the maximum idle time of a TCP connection is 300 seconds.

idle_time	This parameter specifies the maximum idle time of TCP connection, in seconds. Its value must be an integer ranging from 60 to 7200.
-----------	---

no system tune tcpidle

This command is used to restore the maximum idle time of a TCP connection to the default setting.

system tune tcp retransmit timeout <timeout>

This command is used to set the TCP retransmission timer. By default, the TCP retransmission timer is 1000 milliseconds. It is recommended that the default setting not be changed without contacting Customer Support.

timeout	This parameter specifies the TCP retransmission timer, in milliseconds. Its value must be an integer ranging from 100 to 4,294,967,295.
---------	---

no system tune tcp retransmit timeout

This command is used to reset the TCP retransmission timer to the default value.

system tune tcp retransmit dupacks <dupacks>

This command is used to set the number of duplicate ACK packets received before TCP fast retransmission. The default setting is 3, which means TCP will start fast retransmission after receiving 3 duplicate ACK packets. It is recommended that the default setting not be changed without contacting Customer Support.

dupacks	This parameter specifies the number of duplicate ACK packets received before TCP fast retransmission.
---------	---

no system tune tcp retransmit dupacks

This command is used to reset the number of duplicate ACK packets before TCP fast retransmission to the default value.

system tune tcp retransmit accelerate on

This command is used to enable the TCP timeout retransmission acceleration function. By default, this function is disabled.

system tune tcp retransmit accelerate on

This command is used to disable the TCP timeout retransmission acceleration function. By default, this function is disabled.

no system tune tcp retransmit accelerate

This command is used to reset the TCP timeout retransmission acceleration function to the default setting.

system tune tcp slowstart {on|off}

This command is used to enable or disable the TCP slow start function. By default, this function is enabled. It is recommended that the default setting not be changed without contacting Customer Support.

no system tune tcp slowstart

This command is used to reset the TCP slow start function to the default configuration.

system tune tcp delack count <count>

This command is used to set the number of packets received before sending the delayed ACK packets. The default value is 4, which means TCP will send the delayed ACK packet after receiving 4 packets.

count	This parameter specifies the number of packets received before sending the delayed ACK packet. Its value must be an integer ranging from 0 to 128. When the value is set to 0, the ACK packets will be sent out without delay.
-------	--

system tune tcp delack timeout <timeout>

This command is used to set the timer for sending the delayed ACK packets. The default timer is 100 milliseconds.

timeout	This parameter specifies the timer for sending the delayed ACK packets, in milliseconds. Its value must be a multiple of 10, ranging from 0 to 5000.
---------	--

no system tune tcp delack

This command is used to reset the TCP delayed ACK settings to the default.

system tune tcp syntimeout <max_timeout>

This command is used to set the timer for sending TCP SYN packets. The default timer is 60 seconds.

max_timeout	This parameter specifies the timer for sending the TCP SYN packets, in seconds. Its value must be an integer ranging from 60 to 600.
-------------	--

no system tune tcp syntimeout

This command is used to reset the timeout for sending TCP SYN packets to the default configuration.

system tune tcp zwdefend {on|off}

This command is used to enable the protective function against TCP zero window attack. After this function is enabled, the system will not refresh the timeout counter of the TCP connection when receiving zero window packets. If zero window packets are constantly sent from the client, this TCP connection will be terminated when the maximum idle time is reached. By default, this function is disabled.

system tune tcp tso {on|off}

This command is used to enable or disable the TCP Segmentation Offload (TSO) function. When the TSO function is enabled, the segmentation work of large data blocks at the TCP layer will be offloaded to the network interface card, which alleviates the data sending burden on the CPU. By default, this function is disabled.

system tune tcp pktdropopt <action_option>

This command is used to configure the system action when TCP packets are received and dropped on a closed TCP port. This function aims to slow down port scanning of vulnerable services on a system. It could also potentially mitigate DoS attacks. By default, no TCP RST packets will be returned when TCP packets are received and dropped on a closed TCP port.

action_option This parameter specifies the system action when TCP packets are received and dropped on a closed TCP port. Its value must be:

- 0: returns a TCP RST packet.
- 1: returns TCP RST packets for all TCP packets except TCP SYN packets.
- 2: returns no TCP RST packets.

no system tune tcp pktdropopt

This command is used to reset the system action when TCP packets are received and dropped on a closed TCP port to the default configuration.

system tune tcp syncache {on|off}

This command is used to enable or disable the TCP Syncache function. The Syncache function allows the APV appliance to record the TCP option settings, which helps avoid synflood attacks. By default, this function is disabled.



Note: To make the Window Scale option, Timestamp option, SACK, or MSS option take effect for the system or a specified virtual service, the TCP Syncache function must be enabled first.

system tune tcp option wscale {on|off} [shift_count]

This command is used to enable or disable the TCP Window Scale function. By default, this function is disabled.

shift_count Optional. This parameter specifies the number of bits by which the system left-shifts the window size. Its value must be an integer ranging from 0 to 14. This parameter is available only when the Window Scale function is enabled.

The default value is 3.



Note: Actually used window size = Window size in the TCP packet * 2^{shift_count}

system tune tcp option timestamp {on|off}

This command is used to enable or disable the TCP Timestamp function. By default, this function is disabled.

system tune tcp option sack {on|off}

This command is used to enable or disable the TCP Selective Acknowledgment (SACK) function. By default, this function is disabled.

system tune tcp rmem_max [size]

This command is used to modify the maximum receive buffer size for the system. By default, the maximum receive buffer size is 131,072 bytes.

size Optional. This parameter specifies the maximum receive buffer size for the system, in bytes. Its value must be an integer ranging from 65,536 to 1,073,725,440.

The default value is 131,072. If the “size” value is not specified, the maximum receive buffer size will be reset to the default value.

system tune tcp timewait [timeout]

This command is used to set the waiting time for TCP data transmission. The default value is 50 seconds. It is recommended that the default settings not be changed without contacting Customer Support.

timeout Optional. This parameter specifies the waiting time for TCP data transmission, in seconds. Its value must be an integer ranging from 1 to 50.

The default value is 50. If the “timeout” value is not specified, the waiting time for TCP data transmission will be reset to the default value.

no system tune tcp timewait

This command is used to reset the waiting time for TCP data transmission to the default value.

system tune tcp finacc {on|off}

This command is used to enable or disable the TCP FIN acceleration function.

When this function is enabled, if the system wants to close a TCP connection but there is data to be sent on this connection, it will attach the FIN flag to the last data packet to accelerate the close process.

When this function is disabled, the system will separately send the last data packet and the FIN packet. By default, this function is enabled.

show statistics tcp [global|virtual_service|all]

This command is used to display detailed information about TCP connections including the connection numbers for each kind of TCP connection state.

global|virtual_service|all Optional. This parameter specifies the type of connection number statistics to be displayed. Its value must be:

- global: Displays the global connection number statistics.
- virtual service name: Displays the connection number statistics of the specified virtual service.
- all: Displays the global connection number statistics and the connection number statistics of all virtual services.

The default value is global.

Example:

```
AN#show statistics tcp
Global Failed Connections Statistics:
    0 port allocation failures
    drop 0 synd by ours in use
    drop 0 syms by over max connection
    drop 0 syms by get mss error
    drop 0 syms by bad mss size
    drop 0 syms by eroute error
    route 3 syms to system stack
    0 syms was reset
    0 connection conflicts
    0 connection failures for connection table full
    0 connection failures for fail to open local port
    0 connection failures for fail to send syn

Global Statistics:
    LISTEN: 0
    SYN_SENT: 0
    SYN_RCVD: 0
    ESTABLISHED: 0
    CLOSE_WAIT: 0
    FIN_WAIT_1: 0
    CLOSING: 0
    LAST_ACK: 0
    FIN_WAIT_2: 0
    TIME_WAIT: 0
```

The “TIME_WAIT” value in this command is the same as that of “TCP small pcb” (USED column) in “show memory” output. The sum of all the values from “LISTEN” to “FIN_WAIT” equals to the value of “USED” column in “tcp pcb”.

show statistics reset

This command is used to display the statistics of the TCP RESET error codes.

Example:

```
AN(config)#show statistics reset
```

Total:

ID	Times	Reason
0x2159	1	Connection was closed when shunt_reset was on



Note: The language of the “Reason” column is determined by the setting of the “log language” command.

system tune tlsidle <timeout_value>

This command is used to set the timeout value of idle TLS connections. If this command is not configured, the default timeout value is 300s.

timeout_value	This parameter specifies the timeout value of idle TLS connections, in seconds. Its value must be an integer ranging from 300 to 199,999,999.
---------------	---



Note: If the timeout value set by the “slb timeout” command is larger than the timeout value set by the “system tune tlsidle” command, idle TLS connections are subject to the configuration of the former. Otherwise, they are subject to the configuration of the latter.

no system tune tlsidle <timeout_value>

This command is used to restore the timeout setting of idle TLS connections to default.

system tune sslcardtimeout <timeout>

This command is used to set the timeout value for SSL acceleration cards to process data.

When this command is not configured, the system employs the default timeout setting. The default timeout setting is 4s for Nitrox V series SSL acceleration cards and 600s for Nitrox I, NPX and Nitrox III series.

timeout	This parameter specifies the timeout value for SSL acceleration cards to process data, in seconds. Its value must be an integer ranging from 4 to 600.
---------	--



Note: The timeout value of Intel QAT card is 600s, which cannot be changed.

no system tune sslcardtimeout

This command is used to restore the timeout setting for SSL acceleration cards to default.

system tune pollactivity <activity>

This command is used to set the activity of the system main process polling, that is, the number of consecutive polling times.

The system supports two polling modes:

- Static mode: In this mode, when no message is received after the specified number of consecutive polling is reached, the polling is paused for 0.1 milliseconds and then restarted.
- Dynamic mode: In this mode, the CPU can automatically adapt to the service load and dynamically change with the service load.

When this command is not configured, the default ATCP polling activity is 64.

activity	This parameter specifies the activity of polling. Its value must be an integer ranging from 0 to 2,000,000,000. 0 indicates the system will use a dynamic mode polling algorithm. If the value is not 0, the system will use the static mode polling algorithm.
----------	---

no system tune pollactivity

This command is used to reset the system main process polling activity to the default value.

system tune dispatcher sipudp {on|off}

This command is used to enable or disable the SIP UDP dispatch algorithm.

After this dispatch algorithm is enabled, when SIP requests hit a SIP UDP virtual service, the system will calculate the persistence mappings based on the source IP, destination IP, source port and destination port of the SIP requests, and dispatch the requests to multiple CPU cores on a relatively balancing basis.

When this dispatch algorithm is disabled, the system will calculate the persistence mappings based on only the source port number and destination port number of the SIP requests.

By default, the SIP UDP dispatch algorithm is disabled.

system tune dispatcher epolicy on

This command is used to enable the ePolicy dispatch algorithm.

When the ePolicy dispatch algorithm is enabled, if requests hit the ePolicy module, the system will calculate the persistence mappings based on the source IP, destination IP, source port and destination port of the requests, and dispatch the requests to multiple CPU cores on a relatively balancing basis. When it is disabled, the system will calculate the persistence mappings based on only the source and destination ports.

By default, the ePolicy dispatch algorithm is disabled.

system tune dispatcher epolicy off

This command is used to disable the ePolicy dispatch algorithm.

system tune rxmode {interrupt|poll} [sleep_duration]

This command is used to configure the receiving mode of DPDK interfaces. The “**system tune rxmode interrupt**” command will set the interrupt-based receiving mode; the “**system tune rxmode poll [sleep_duration]**” command will set the polling-based receiving mode.

When this command is not configured, DPDK interfaces will use the polling-based receiving mode by default, and the sleep duration under light traffic is 100 microseconds.

sleep_duration	Optional. This parameter specifies the sleep duration under light traffic for the polling-based mode, in microseconds. Its value must be an integer ranging from 0 to 100. When the value 0 is set, polling will not stop.
----------------	--

The default value is 100.

system tune arp timeout <timeout>

This command is used to change the timeout value of ARP entries.

When this command is not configured, the default timeout value is 3600s.

timeout	This parameter specifies the timeout value of ARP entries. Its value must be an integer ranging from 1 to 2,147,483,647.
---------	--

system tune ndp timeout <timeout>

This command is used to change the timeout value of NDP entries.

When this command is not configured, the default timeout value is 86,400s.

timeout	This parameter specifies the timeout value of NDP entries. Its value must be an integer ranging from 1 to 2,147,483,647.
---------	--

no system tune arp timeout

This command is used to restore the timeout setting of ARP entries to default.

no system tune ndp timeout

This command is used to restore the timeout setting of NDP entries to default.

system tune icmp replylimit [*limit*]

This command is used to set the threshold for the number of ICMP replies that is allowed per second. When the number of ICMP replies received per seconds exceeds the specified threshold, the system will drop subsequent ICMP replies.

When this command is not configured, the default threshold for the number of ICMP replies that is allowed per second is 1000.

limit	Optional. This parameter specifies the threshold for the number of ICMP replies that is allowed per second. Its value must be an integer ranging from 0 to 20,000. The value 0 indicates no limitation.
-------	---

The default value is 1000.

no system tune icmp replylimit

This command is used to reset the threshold for the number of ICMP replies that is allowed per second to default.

system tune rtos {on|off}

This command is used to enable or disable the Real Time Operating System (RTOS) function. After this function is enabled, in TCPS load balancing scenarios that adopts client authentication, the response speed of the system will increase significantly.

The configuration requires a system reboot to take effect and only takes effect on physical devices. By default, this function is disabled.

show system rtos

This command is used to display the configuration and enabling status of the RTOS function.

system tune segment iprange <ip_address> {netmask|prefix}

This command is used to configure an IP subnet for the function of automatically assigning NAT address to the segment IP address. After the IP subnet is configured, the system will automatically select an IP address from the IP subnet as the NAT translation address of the segment IP address when processing the NAT translation of the segment IP.

ip_address	This parameter specifies the IP address of the IP subnet. Its value must be an IPv4 or IPv6 address.
------------	--

netmask prefix	This parameter specifies the netmask or prefix length of the IP subnet.
----------------	---

- “netmask” indicates the netmask of the IPv4 address. Its value must be in dotted decimal notation or an integer ranging from 0 to 32.
- “prefix” indicates the prefix length of the IPv6 address. Its value must be an integer ranging from 0 to 128.

no system tune segment iprange <ip_address> {netmask|prefix}

This command is used to delete the IP address segment configured the function of automatically assigning NAT address to the segment IP address.

show system tune

This command is used to display the system tuning parameters.

clear system tune

This command is used to reset the system tuning parameters to the default values.

system thread {single|auto}

This command is used to set the running thread mode of the system. After the system’s running thread mode is modified, the system must be restarted for the modification to take effect.

With “**system thread single**” configured, the system will run in single-thread mode.

With “**system thread auto**” configured, the system will run in multi-thread mode.

By default, the system runs in multi-thread mode.

show system thread

This command is used to display the running thread mode setting of the system.

ip nameserver <ip_address>

This command is used to add a DNS server. It is allowed to add 3 DNS servers at most.

ip_address This parameter specifies the IP address for the DNS server.

no ip nameserver <ip_address>

This command is used to delete a DNS server.

show ip nameserver

This command is used to display the IP addresses for the configured DNS servers.

fwd mode <fwd_mode>

This command is used to set the operation mode of port forwarding and IP address translation. The APV appliance supports two kinds of port forwarding modes: non-transparent (source IP translation required) and transparent. The default port forwarding mode is transparent.

- fwd_mode** This parameter specifies the operation mode of port forwarding and IP address translation. Its value must be:
- **nontransparent**: indicates non-transparent mode. In non-transparent mode, the system management IP will be used as the source IP.
 - **transparent**: indicates transparent mode. In transparent mode, the client IP will be used as the source IP.



Note: The Port Forwarding feature cannot support FTP. Users are recommended to use the SLB feature instead.

no fwd mode

This command is used to reset the operation mode of port forwarding and IP address translation to the default configuration.

show fwd

This command is used to display the operation mode of port forwarding and IP address translation.

system date <year> <month> <date>

This command is used to set the system date for the APV appliance. It applies to the situation when there is no NTP server available. For example, the administrator could execute the following command to set the system date to October 20, 2010:

```
AN(config)#system date 10 10 20
```

system time <hour> <minute> <second>

This command is used to set the system time for the APV appliance. It applies to the situation when there is no NTP server available. The APV appliance adopts 24-hour time format. For example, the administrator could execute the following command to set the system time to 11:33:51 PM:

```
AN(config)#system time 23 33 51
```

system timezone

This command is used to set the system time zone for the APV appliance. The default value is “GMT”. To set the system time zone, follow these steps:

1. Select the continent.

2. Select the country.
3. Select the time zone.



Note: During the time zone setup process, the administrator can enter 0 to return to the previous option. For example, entering 0 on the country list page will direct the administrators to the continent selection page.

show system timezone

This command is used to display the current time zone.

clear system timezone

This command is used to reset the system time zone to the default value.

show date

This command is used to display the current date and time of the APV appliance.

NTP

After 10.7.2.5, all NTP commands are updated. The parameters and output of the new ones will be different from that of the old ones.

NTP server keys authenticate and encrypt the communication between an NTP server and client. By using the key, unauthorized hosts can be prevented from gaining control over the NTP server, and the integrity and confidentiality of data during communication can be safeguarded. NTP server keys adopt a symmetric key encryption algorithm, which is efficient, scalable, and flexible.

ntp {on|off}

Enables or disables Network Time Protocol (NTP) and Network Time Security (NTS). When this function is enabled, the system synchronizes its system time with the NTP server. By default, this function is disabled.



Note:

- To use NTS, both TCP port 4460 and UDP port 123 need to be opened.
- If a big time gap exists between the current system time and the time on the NTP server, enabling this function might cause the system to be restarted.

ntp server <host_name|ip_address> [option] [key-id]

Sets up an NTP server. When the NTP function is enabled and an NTP server is created, the APV will act as an NTP client to synchronize its system time with the NTP server.

Before you restart APV, remember to use the **write memory** command to save your NTP server settings, or they might be gone after the restart.

host_name|ip_address A URL or an IPv4 or IPv6 address of an NTP server. Either URL or IP address need to be enclosed by double quotes.

Examples:

- `ntp server "110.75.186.247"`
- `ntp server "time.couldflare.com"`

option Optional. It enables an additional feature of the NTP server, such as iBurst and Network Time Security (NTS).

Examples:

- `ntp server "110.75.186.247" iburst`
- `ntp server "time.couldflare.com" nts`

key-id Optional. It is the index of an authentication key, which is used while the NTP server is communicating with the NTP client. The range of its value is 1–65535.

Examples:

`ntp server "time.couldflare.com" iburst 10`

no ntp server <host_name|ip_address>

Deletes an NTP server.

host_name|ip_address A URL or an IPv4 or IPv6 address of the NTP server. Either IP address or URL need to be enclosed by double quotes.

Examples:

`AN(config)#no ntp server "110.75.186.247"`

`AN(config)#no ntp server "time.couldflare.com"`

ntp key <id> <md5-key>

Defines the NTP authentication key.

id The index of an authentication key. The range of its value is 1–65535.

md5-key The value of the MD5 key. It is a case-sensitive string with a maximum length of 32 bytes. The string can contain alphanumeric and special characters.

All special characters are supported except quotation marks (“”) and white space.

Examples:

```
AN(config)#ntp key 10 test123
```

no ntp key <id>

Deletes an NTP authentication key.

id The index of an authentication key. The range of its value is 1–65535.

Examples:

```
AN(config)#no ntp key 10
```

show ntp <keys|status>

Displays NTP servers’ keys and states.

keys Displays authentication keys.

status Displays NTP servers' settings, including their time dispersion and association.



Note: You can use the **show running ntp** command to show all active configurations that contain the “ntp” string.

Examples:

```
AN(config)#show ntp keys
```

Key ID	MD5 String
--------	------------

-----	-----
1	ArrayNetworks
10	test123
111	ytrueruerej
444	Password
101	Key\$2@9*%
102	GRN&I^+?

AN(config)#show ntp status					
server 1.2.3.4					
server 4.3.2.1 key 10					
server 5.6.7.8 nts					
server 9.9.9.9 iburst					
server 172.20.0.119 iburst key 10					
Chrony Sources:					
=====					
MS Name/IP address	Stratum	Poll	Reach	LastRx	Last sample
1.2.3.4	0	10	0	-	+0ns [+0ns] +/- 0ns
4.3.2.1	0	10	0	-	+0ns [+0ns] +/- 0ns
dynamic-005-006-007-008.>	0	6	0	-	+0ns [+0ns] +/- 0ns
dns9.quad9.net	0	10	0	-	+0ns [+0ns] +/- 0ns

radius_server	3	6	377	61	+12us	[-92us]	+/-		
					5396us				
Chrony Authentication Data:									
=====									
Name/IP address	Mode	KeyID	Type	KLen	Last	Atmp	NAK	Cook	CLen
1.2.3.4	-	0	0	0	-	0	0	0	0
4.3.2.1	SK	10	1	56		0	0	0	0
dynamic -005-006 -007-008 ,>	NTS	0	0	0	-	10	0	0	0
dns9.qua d9.net	-	0	0	0	-	0	0	0	0
radius_s erver	SK	10	1	56	-	0	0	0	0

clear ntp

Deletes all NTP configurations.

Hostname

hostname <host_name>

This command is used to configure the host name of the APV appliance. The default host name is “AN”

host_name

This parameter specifies the host name of the APV appliance. Its value must be a string of 1 to 64 characters. If the parameter value contains special characters or only numbers, it must be enclosed in double quotes. Letters, numbers, and special characters are supported. “#”, “\$”, “,” and space characters are not allowed.

no hostname

This command is used to reset the host name of the APV appliance to the default value “AN”.

show hostname

This command is used to display the host name of the APV appliance.

System

system mail external {on|off}

This command is used to enable or disable the external Simple Mail Transfer Protocol (SMTP) mail server function. Before this function is enabled, the external SMTP mail server must be configured using the “system mail external server” command. When this function is enabled, the system will act as the mail client and use the external SMTP mail server to send the alert mail. When this function is disabled, the system will use the built-in local mail server to send the alert mail. Currently, the system only supports the external SMTP mail server. By default, this function is disabled.

system mail external server <server_address> [server_port] [ssl_flag]

This command is used to configure an external SMTP mail server. Only one external SMTP email server can be configured in the system.

This command is also used to modify the configuration of the existing external SMTP mail server.

server_address	This parameter specifies the name or IPv4 address of the external SMTP mail server. Its value must be IPv4 address or a string of 1 to 64 characters enclosed in double quotes. Letters, numbers, and special characters are supported.
server_port	Optional. This parameter specifies the port number of the external SMTP mail server. Its value must be an integer ranging from 0 to 65,535. The default value is 25.
ssl_flag	Optional. This parameter specifies whether the SSL or TLS protocol is used to encrypt the communication between the system and the external SMTP mail server. Its value must be: • 0: indicates that the system will not use the SSL or TLS protocol. • 1: indicates that the system will use the SSL or TLS protocol. The default value is 0.

no system mail external server

This command is used to delete the external SMTP mail server.

system mail external user <email_address> <password>

This command is used to configure the email account of the sender using the external SMTP mail server to send the alert mail. When the system uses the external SMTP mail server to send the alert mail, the external SMTP mail server will verify the email account of the sender first. The external SMTP mail server will forward the mail only when the email account of the sender is valid.

This command is also used to modify the configuration of the existing email account of the sender.

email_address	This parameter specifies the email address of the sender. Its value must be a string of 1 to 32 characters enclosed in double quotes. Letters, numbers, and special characters are supported.
password	This parameter specifies the password of the sender. Its value must be a string of 1 to 64 characters enclosed in double quotes.

no system mail external user

This command is used to delete the email account of the sender using the external SMTP mail server to send the alert mail.

system mail from <from_string>

This command is used to modify the email account of the sender using the local mail server to send the alert mail. The default email account of the alert mail sender in local mail server is “%h<alert@log.domain>”.

from_string	This parameter specifies the sender email account. Its value must be enclosed in double quotes. It supports the following escape strings: • %h: indicates the host name configured by the “hostname” command. • %q: indicates the English double quotes. For example, you have to use the string “%qan%q” to represent the string“"an"”. • %%: indicates the percent sign (%).
-------------	---

Note: In the above escape strings, the first “%” character is an escape character and it cannot be used alone.

For example:

AN(config)#system mail from "apv@abc.com"

In the above configuration, the email account of the alert mail sender is "apv@abc.com".

no system mail from

This command is used to reset the email account of the sender using the local mail server to send the alert mail to the default configuration.

system mail hostname <host_string> [option]

This command is used to configure the host name in the alert mail from the local mail server. The default host name is "%l.alert_pseudo_domain".

host_string	This parameter specifies the host name in the alert mail from the local mail server. Its value must be a fully qualified domain name (FQDN), enclosed in double quotes, such as "acb.com". It supports the following escape strings:
-------------	--

- %h: indicates the host name configured by the "hostname" command.
- %l: indicates the part of the host name before the first "." character.

Note: In the above escape strings, the first "%" character is an escape character and it cannot be used alone.

option	Optional. This parameter specifies whether to send alert messages if the host name is the same as the mail server domain name. Its value must be: • 1: sends alert messages if the host name is the same as the mail server domain name. • 0: stops sending alert messages if the host name is the same as the mail server domain name.
--------	---

The default value is 0.

no system mail hostname

This command is used to reset the host name in the alert mail from the local mail server to the default value.

show system mail

This command is used to display the current configuration of the local mail server and external SMTP mail server.

clear system mail

This command is used to restore the configuration of the local mail server and external SMTP mail server. The configured external SMTP mail server and its related sender email account will be deleted.

system mail relay {on|off}

This command is used to enable or disable the system mail relay function. After this function is enabled, alert emails sent to the specified domain will be sent to the mail relay server for forwarding. Before enabling this function, please make sure the mail relay server has been configured using the “**system mail relay server**” command. By default, this function is disabled.

system mail relay server <host_name> <relay_server>

This command is used to configure a mail relay server for the specified domain.

host_name	This parameter specifies the domain name.
relay_server	This parameter specifies the domain name or IPv4 address for the mail relay server. If this parameter is specified as an IPv4 address, it must be enclosed in double quotes. If this parameter is specified as a domain name, please make sure this domain name can be resolved by the configured DNS server.

For example,

AN(config)# system mail relay server “mailserver.com” “relay.com” AN(config)# system mail relay on

In the above configuration, when sending the alert mail to the email address including the domain name “mailserver.com”, the system forwards this alert mail to the mail relay server “relay.com” for forwarding.

no system mail relay server <host_name>

This command is used to delete the mail relay server configured for the specified domain name.

show system relay

This command is used to display the configuration and status of the system mail relay function.

clear system relay

This command is used to restore the configuration of the system mail relay function.

system interactive {on|off}

This command is used to enable or disable the CLI command interactive mode. When this function is enabled, more detailed command information will be displayed.

show system interactive

This command is used to display the setting of the current system interactive mode.

system command timeout <timeout>

This command is used to set the timeout value for the execution of the “**config file**” or “**config memory**” command. The default timeout value is 120 seconds.

timeout	This parameter specifies the timeout value, in seconds. Its value must be 0 or an integer ranging from 30 to 65,533. Value 0 means no timeout.
---------	--

show system command timeout

This command is used to display the timeout value setting for the execution of the “**config file**” or “**config memory**” command.

system command override on

This command is used to enable the command override function. When this function is enabled and a command is repeatedly executed for the same virtual service or real service, the previous command configuration will be overwritten. This function is turned on by default.



Note: Layer 2 IP virtual service does not support the command override function.

system command override off

This command is used to disable the command override function.

show system command override

This command is used to display the status of the command override function.

switch weblink <url>

This command is used to set the management URL for the switch built in the APV appliance. Administrators can use the specified URL to manage and configure the switch via the APV WebUI page.

url	This parameter specifies the URL for the switch built in the APV appliance.
-----	---

Website Classification

webclassify license <license_key>

This command is used to import a license key for the website classification function. This license allows the APV appliance to use the website classification function provided by Webroot BrightCloud Threat Intelligence Service. Webroot provides a 30-day trial license for the website classification function. After the trial license expires, the Webroot server will deny the APV appliance's access and the APV appliance will stop sending website classification queries to the Webroot server.

license_key	This parameter specifies the license key. Please contact Customer Support and provide the APV model name and serial number to obtain the license key of the website classification function.
-------------	--

show webclassify license

This command is used to display information about the website classification function license, including OEM, device ID, serial number, license type (trial license or production license) as well as the issue information and expiration time of the license.

webclassify {on|off}

This command is used to enable or disable the global website classification function. By default, this function is disabled.

webclassify cloud {on|off}

This command is used to enable or disable the online website classification lookup function. This function can be enabled only after the global website classification function is enabled ("**webclassify on**"). By default, online website classification lookup is disabled.

When this function is disabled, the system performs only local lookups. After this function is enabled, the system will connect to the Webroot server for online lookup if the local lookup fails or the locally cached entries time out.



Note: The disabling of the online website classification lookup function takes a short term of time, during which the system will still connect to the Webroot server for online lookup due to the time interval caused by asynchronous communication.

show webclassify settings

This command is used to display the configurations of the global website classification and online website classification lookup functions.

show webclassify status

This command is used to display global configurations related to website classification functions, such as the domain name of the WebRoot BrightCloud server, the license status, the enabling status of the online website classification lookup function, and the database version.

show webclassify url categories

This command is used to display all website categories that the system can look up.

show webclassify url category <website_name>

This command is used to look up the category of the specified website. If the system can obtain the category of a website from the local cache or database, it will display the result quickly. If the local cache and database has no category information for a website, the system will experience a slight delay in returning the result. The maximum delay is 20s.

website_name	This parameter specifies the domain name of the website. Its value must be a string of 1 to 256 characters, such as “google.com”.
--------------	---

Bridge

bridge name <bridge_name>

This command is used to create a bridge. The system supports a maximum of 16 bridges.

bridge_name	This parameter specifies the bridge name. Its value must be a string of 1 to 31 characters.
-------------	---

no bridge name <bridge_name>

This command is used to delete the specified bridge.

bridge member <bridge_name> <system_ifname> [broadcast_option]

This command is used to add a system interface to the specified bridge. Each bridge can have at most 32 system interfaces as its member interfaces.

bridge_name	This parameter specifies the bridge name. Its value must be a string of 1 to 31 characters.
-------------	---

system_ifname	This parameter specifies the system interface name. The default system interface name is port1, port2,
---------------	--

port3...(Adminsitrators can define the name of system interfaces usings the “interface name” command.)

broadcast_option

Optional. This parameter specifies whether the member interface can send broadcast and multicast packets. Its value must be:

- yes: allows the member interface to send broadcast and multicast packets.
- no: disallows the member interface to send broadcast and multicast packets.

The default value is “yes”.

no bridge member <bridge_name> <system_ifname>

This command is used to remove a member interface from the specified bridge.

bridge vlan <bridge_name> <system_ifname> <vlan_id>

This command is used to set a VLAN tag for the specified member interface of the specified bridge. Packets carrying the specified VLAN tag can be forwarded out of this member interface. Every member interface can have at most 4094 VLAN tags.

bridge_name

This parameter specifies the bridge name.

system_ifname

This parameter specifies the system interface name. The default system interface name is port1, port2, port3...(Adminsitrators can define the name of system interfaces usings the “interface name” command.)

vlan_id

This parameter specifies the ID of the VLAN interface. Its value must be an integer ranging from 1 to 4094.

no bridge vlan <bridge_name> <system_ifname> <vlan_ID>

This command is used to remove the VLAN interface from a member interface of the specified bridge.

clear bridge vlan <bridge_name>

This command is used to clear all VLAN interfaces of the specified bridge.

bridge apprule <bridge_name> <src_ip> <src_port> <dst_ip> <dst_port> <protocol>

This command is used to configure a filter rule for the specified bridge. Traffic hitting this filter rule will be transferred to upper protocol layers, such as HTTP, DNS, HTTPS and FTP traffic. Other traffic will be directly forwarded.

bridge_name	This parameter specifies the bridge name.
src_ip	This parameter specifies the source IP address of the traffic to be filtered. The value 0.0.0.0 indicates all IP addresses.
src_port	This parameter specifies the source port number of the traffic to be filtered. Its value must be an integer ranging from 0 to 65535. The value 0 indicates all port numbers.
dst_ip	This parameter specifies the destination IP address of the traffic to be filtered. The value 0.0.0.0 indicates all IP addresses.
dst_port	This parameter specifies the destination port number of the traffic to be filtered. Its value must be an integer ranging from 0 to 65535. The value 0 indicates all port numbers.
protocol	This parameter specifies the transport layer protocol type of the traffic to be filtered. Its value must be “tcp” or “udp”.

no bridge apprule <bridge_name> <src_ip> <src_port> <dst_ip> <dst_port>

This command is used to delete a filter rule of the specified bridge.

clear bridge apprule [bridge_name]

This command is used to clear all filter rules of the specified bridge. If the “bridge_name” parameter is not specified, the filter rules of all bridges will be cleared.

show bridge config [bridge_name]

This command is used to display all configurations related to the specified bridge. If the “bridge_name” parameter is not specified, the configurations related to all bridges will be cleared.

clear bridge config [bridge_name]

This command is used to clear all configurations related to the specified bridge. If the “bridge_name” parameter is not specified, the configurations related to all bridges will be cleared.

show bridge mactable [bridge_name|filter_string]

This command is used to display the MAC entries of the specified bridge or the MAC entries matching the specified filter string.

bridge_name|filter_string Optional. This parameter specifies the bridge name or the filter string used to match MAC entries. Bridge name and filter string cannot be specified at the same time. When this parameter is not configured, the MAC entries of all bridges will be displayed.

show bridge appflowtable

This command is used to display the application flow tables of the specified bridge.

show bridge status [bridge_name]

This command is used to display the status of the specified bridge. If the “bridge_name” parameter is not specified, the status of all bridges will be displayed.

show statistics bridge [bridge_name]

This command is used to display the statistics related to the specified bridge. If the “bridge_name” parameter is not specified, the statistics related to all bridges will be displayed.

SPAN Port

spanport filterlist name <filterlist_name>

This command is used to configure a filter list. The system supports a maximum of eight filter lists.

filterlist_name This parameter specifies the filter list name. Its value must be a string of 1 to 31 characters.

no spanport filterlist name <filterlist_name>

This command is used to delete the specified filter list.

show spanport filterlist [filterlist_name]

This command is used to display the specified filter list. If the “filterlist_name” parameter is not specified, all filter lists will be displayed.

clear spanport filterlist

This command is used to clear all filter lists and their filter rules.

spanport filterlist member <filterlist_name> <interface_name> <src_ip> <src_port> <dst_ip> <dst_port> <protocol> [direction]

This command is used to add a filter rule to the specified filter list. The system will filter out the packets to be copied based on the filter rule. A maximum of eight filter rules can be added to a filter list. It is suggested that the filter rules should not have duplicate matching conditions. If a packet hits multiple filter rules at the same time, the system will select only one of them.

filterlist_name	This parameter specifies the name of an existing filter list.
interface_name	This parameter specifies the source SPAN port. Packets passing through the source SPAN port will be copied. Its value must be a system interface name, such as “port1”, “port2”, “port3” etc.
src_ip	This parameter specifies the source IP address of packets to be copied. Its value must be an IPv4 address. The value “0.0.0.0” indicates all IP addresses.
src_port	This parameter specifies the source port number of packets to be copied. Its value must be an integer ranging from 0 to 65,535. The value 0 indicates all port numbers.
dst_ip	This parameter specifies the destination IP address of packets to be copied. Its value must be an IPv4 address. The value “0.0.0.0” indicates all IP addresses.
dst_port	This parameter specifies the destination port number of packets to be copied. Its value must be an integer ranging from 0 to 65,535. The value 0 indicates all port numbers.
protocol	This parameter specifies the transport layer protocol of packets to be copied. Its value must be “tcp”, “udp” or “all” (both TCP and UDP).
direction	Optional. This parameter specifies the direction of packets to be copied. Its value must be: • both: copies packets flowing in both the inbound and outbound directions. • inbound: copies packets flowing in only the inbound direction. • outbound: copies packets flowing in only the outbound direction. The default value is “both”.

no spanport filterlist member <filterlist_name> <interface_name> <src_ip> <src_port> <dst_ip> <dst_port> <protocol> <direction>

This command is used to delete a filter rule from the specified filter list.

spanport devicegroup name <group_name> <group_method>

This command is used to configure the security device group to which the copied packets are destined and set the method that the system uses to send the copied packets to the group. The system supports a maximum of eight security device groups.

group_name	This parameter specifies the name of the security device group. Its value must be a string of 1 to 31 characters.
group_method	This parameter specifies the method that the system uses to send the copied packets to the group. Its value must be: • lb: sends the copied packets to the security devices in the group in a load balancing manner based on the hash value of the source IP and destination IP addresses. Packets with the same hash value of source IP and destination IP addresses will be persisted to the same security device. • Dup: sends a duplicate of copied packets to each security device in the group.

no spanport devicegroup name <group_name>

This command is used to delete the specified security device group.

show spanport devicegroup [group_name]

This command is used to display the specified security device group . If the “group_name” parameter is not specified, all security device groups will be displayed.

clear spanport devicegroup

This command is used to clear all security device groups and and their members.

spanport devicegroup member <group_name> <interface_name> <mac> [sort_string]

This command is used to add a security device as a member to the specified security device group. Each group can have at most eight members.

group_name	This parameter specifies the name of an existing security device group.
interface_name	This parameter specifies the destination SPAN port. Its value must be a system interface name, such as “port1”, “port2”, “port3” etc. The destination SPAN port cannot reside on the same system interface as the source SPAN port. Besides, the destination SPAN cannot be used for other services.

mac	This parameter specifies the MAC address of the security device.
sort_string	Optional. This parameter specifies the sort string. When it is necessary to copy packets from two APVs and the copied packets are sent to two or more security devices in the load balancing manner, this parameter must be configured and the value for the members in the same security device group must be the same. The default value is “abc”.



Note: Destination SPAN ports do not support the transfer of VLAN packets to security devices. For copied packets carrying VLAN tags, a destination SPAN port will remove the VLAN tags before forwarding them.

no spanport devicegroup member *<group_name> <interface_name> <mac>*

This command is used to delete a member from the specified security device group.

spanport policy *<policy_name> <filterlist_name> <group_name>*

This command is used to configure a SPAN port policy to associate the specified filter list with the specified security device group, that is, to bind the source SPAN port to the destination SPAN port. Each filter list can be associated with only one security device group. The system supports a maximum of eight SPAN port policies. It is suggested that the two ports use the same NIC type and MTU setting.

policy_name	This parameter specifies the name of the SPAN port policy. Its value must be a string of 1 to 31 characters.
filterlist_name	This parameter specifies the name of an existing filter list.
group_name	This parameter specifies the name of an existing security device group.

no spanport policy *<policy_name>*

This command is used to delete the specified SPAN port policy.

show spanport policy *[policy_name]*

This command is used to display the specified SPAN port policy. If the “policy_name” parameter is not specified, all SPAN port policies will be displayed.

clear spanport policy

This command is used to clear all SPAN port policies.

clear spanport config

This command is used to clear all SPAN port configurations.

show statistics spanport

This command is used to display SPAN port statistics.

clear statistics spanport

This command is used to clear SPAN port statistics.

System Health Check

This section covers commands for configuring system health check. The system health check function applies to both the scenario where only one APV appliance is deployed and the scenario where two or multiple APV appliances (HA) are deployed. When the system health check function is configured, HA can use system health check as HA failover conditions.

monitor system {on|off}

This command is used to enable or disable the system health check function. By default, this function is enabled. After this function is enabled, the appliance will perform health checks based on the following default configurations:

- **monitor system cpu overheat 75 5000 3 3**
- **monitor system cpu utilization 60 5000 3 3**
- **monitor system sslcard 10000 3 3**
- **monitor system memory utilization system 90 5000 3 3**
- **monitor system memory utilization network 90 5000 3 3**
- **monitor system memory utilization swap 20 5000 3 3**
- **monitor system memory utilization connection 95 5000 3 3**
- **monitor system ntp 5000 3 3**
- **monitor system disk system 60 60**
- **monitor system disk data 80 60**

Administrators can change the default settings as required, and can also modify health check conditions by using the following commands.



Note: After administrators modify the health check conditions, the system will immediately mark the related status as “Normal.”

monitor network gateway *<unit_name>* *<gateway_ip>* *<condition_name>*
[interval] *[up_check_times]* *[down_check_times]* *[source_ip]*

This command is used to configure a gateway health check condition for the specified HA unit.

unit_name	This parameter specifies the name of an HA unit. In scenarios where HA is not deployed, set the value of this parameter to empty (“”).
gateway_ip	This parameter specifies the gateway IP address of the specified HA unit. Its value must be an IPv4 or IPv6 address.
condition_name	This parameter specifies the name of the health check condition. The value of this parameter ranges from GATEWAY_1 to GATEWAY_32.
interval	Optional. This parameter specifies the interval, in ms, at which the health check is performed. Its value must be an integer ranging from 1000 to 100,000. The default value is 1000.
up_check_times	Optional. This parameter specifies the number of consecutive times that the gateway is detected to be Up. Its value must be an integer ranging from 3 to 10. The default value is 3.
down_check_times	Optional. This parameter specifies the number of consecutive times that the gateway is detected to be Down. Its value must be an integer ranging from 3 to 1000. The default value is 15.
source_ip	Optional. This parameter specifies the source IP address used to ping the gateway. Both IPv4 and IPv6 addresses are supported. The default value is empty.

no monitor network gateway <unit_name> <gateway_ip>

This command is used to delete a gateway health check condition configured for the specified HA unit.

clear monitor network gateway

This command is used to delete all gateway health check conditions.

monitor system bond <bond_id> [threshold]

This command is used to configure the bond interface health check condition.

<code>bond_id</code>	This parameter specifies the id of an existing bond interface.
<code>threshold</code>	Optional. This parameter specifies the threshold value of the bond interface state switching to down, in percent (%). Its value must be an integer ranging from 1 to 100. In the bond interface, when the proportion of ports in the state of “down” is less than the set threshold, the state of the bond interface is “up”, otherwise, it will turn to “down”. The default value is 50.



Note: If the health check state of the bond interface is “down”, it does not mean that the bond interface is damaged. It only means that the proportion of the port in the bond interface whose state is “down” is equal to or exceeds the set threshold value.

no monitor system bond <*bond_id*>

This command is used to delete the health check condition for the specified bond interface.

clear monitor system bond all

This command is used to reset all bond interfaces’ health check conditions to default configuration.

monitor system cpu overheat [*temperature*] [*interval*] [*up_check_times*] [*down_check_times*]

This command is used to configure the CPU temperature health check condition.

<code>temperature</code>	Optional. This parameter specifies the CPU temperature threshold, in Centigrade. Its value must be an integer ranging from 1 to 100. The default value is 75.
<code>interval</code>	Optional. This parameter specifies the interval, in ms, at which the health check is performed. Its value must be an integer ranging from 5000 to 1,000,000. The default value is 5000.
<code>up_check_times</code>	Optional. This parameter specifies the number of times that the CPU temperature does not exceed the threshold. Its value must be an integer ranging from 3 to 10. The default value is 3.

down_check_times Optional. This parameter specifies the number of consecutive times that the CPU temperature exceeds the threshold. Its value must be an integer ranging from 3 to 10.

The default value is 3.

no monitor system cpu overheat

This command is used to delete the CPU temperature health check condition.

monitor system cpu utilization [*fatal_percent*] [*interval*] [*up_check_times*] [*down_check_times*]

This command is used to configure the CPU utilization health check condition.

fatal_percent Optional. This parameter specifies the threshold for the CPU utilization, in percent (%). Its value must be an integer ranging from 1 to 100.

The default value is 60.

interval Optional. This parameter specifies the interval, in ms, at which the health check is performed. Its value must be an integer ranging from 5000 to 1,000,000.

The default value is 5000.

up_check_times Optional. This parameter specifies the number of consecutive times that the CPU utilization does not exceed the threshold. Its value must be an integer ranging from 3 to 10.

The default value is 3.

down_check_times Optional. This parameter specifies the number of consecutive times that the CPU utilization exceeds the threshold. Its value must be an integer ranging from 3 to 10.

The default value is 3.

no monitor system cpu utilization

This command is used to delete the CPU utilization health check condition.

clear monitor system cpu all

This command is used to reset the CPU temperature and utilization health check conditions to default configuration.

monitor system sslcard *[interval] [up_check_times] [down_check_times]*

This command is used to configure the SSL card health check condition.

interval	Optional. This parameter specifies the interval, in ms, at which the health check is performed. Its value must be an integer ranging from 10,000 to 3,600,000. The default value is 10,000.
up_check_times	Optional. This parameter specifies the number of times that the SSL card is detected to be Up. Its value must be an integer ranging from 1 to 10. The default value is 3.
down_check_times	Optional. This parameter specifies the number of times that the SSL card is detected to be Down. Its value must be an integer ranging from 1 to 10. The default value is 3.



Note: The SSL card health check interval must be larger than the timeout value for SSL acceleration cards to process data, which can be configured by the “**system tune sslcardtimeout**” command.

no monitor system sslcard

This command is used to delete the SSL card health check condition.

monitor system ssl {ae|se} *[fatal_percent] [interval] [up_check_times] [down_check_times]*

This command is used to configure the SSL card (QAT card and Nitrox V series SSL cards) AE or SE core utilization health check condition.

By default, the appliance will only load the SSL card AE or SE core utilization health check condition for the appliance with Nitrox V series SSL cards.

fatal_percent	Optional. This parameter specifies the threshold for the SSL card AE or SE core utilization, in percent (%). Its value must be an integer ranging from 1 to 100. The default value is 60.
interval	Optional. This parameter specifies the interval, in ms, at which the health check is performed. Its value must be an integer ranging from 5000 to 1,000,000. The default value is 5000.

up_check_times Optional. This parameter specifies the number of consecutive times that the SSL card AE or SE core utilization does not exceed the threshold. Its value must be an integer ranging from 3 to 10.

The default value is 3.

down_check_times Optional. This parameter specifies the number of consecutive times that the SSL card AE or SE core utilization exceeds the threshold. Its value must be an integer ranging from 3 to 10.

The default value is 3.

no monitor system ssl {ae|se}

This command is used to delete the SSL card AE or SE core utilization health check condition.

clear monitor system ssl all

This command is used to delete both the SSL card AE and SE core utilization health check conditions.

monitor system memory utilization <memory_type> [fatal_percent] [interval] [up_check_times] [down_check_times]

This command is used to configure the memory utilization health check condition, including system memory, network memory, connection memory and SWAP memory.

memory_type This parameter specifies the type of memory utilization to be detected. Its value must be:

- system: indicate system memory utilization
- network: indicates network memory utilization
- connection: indicates network memory utilization
- swap: indicates SWAP memory utilization

fatal_percent Optional. This parameter specifies the threshold for memory utilization, in percent (%). Its value must be an integer ranging from 1 to 100.

The default value varies for different types of memory utilization.

- System memory: 90
- Network memory: 90
- Connection memory: 20

	• SWAP memory: 95
interval	Optional. This parameter specifies the interval, in ms, at which the health check is performed. Its value must be an integer ranging from 5000 to 1,000,000. The default value is 5000.
up_check_times	Optional. This parameter specifies the memory utilization does not exceed the threshold. Its value must be an integer ranging from 3 to 10. The default value is 3.
down_check_times	Optional. This parameter specifies the number of times that the memory utilization exceeds the threshold. Its value must be an integer ranging from 3 to 10. The default value is 3.

no monitor system memory utilization <memory_type>

This command is used to delete the specified type of memory utilization health check condition.

monitor system disk system [fatal_percent] [interval]

This command is used to configure a system disk space utilization health check condition. System disk space refers to the current system root partition.

fatal_percent	Optional. This parameter specifies the threshold for system disk space utilization, in percent (%). Its value must be an integer ranging from 1 to 100. The default value is 60.
interval	Optional. This parameter specifies the interval, in minutes, at which the health check is performed. Its value must be an integer ranging from 1 to 300. The default value is 60.

no monitor system disk system

This command is used to delete the system disk space utilization health check condition.

monitor system disk data [fatal_percent] [interval]

This command is used to configure a data disk space utilization health check condition. Data disk space refers to the disk partition used for service data storage.

<code>fatal_percent</code>	Optional. This parameter specifies the threshold for data disk space utilization, in percent (%). Its value must be an integer ranging from 1 to 100. The default value is 80.
<code>interval</code>	Optional. This parameter specifies the interval, in minutes, at which the health check is performed. Its value must be an integer ranging from 1 to 300. The default value is 60.

no monitor system disk data

This command is used to delete the data disk space utilization health check condition.

clear monitor system disk all

This command is used to reset the system disk space and data disk space utilization health check conditions to default configuration.

monitor system ntp [*interval*] [*up_check_times*] [*down_check_times*]

This command is used to configure the NTP health check condition.

<code>interval</code>	Optional. This parameter specifies the interval, in ms, at which the health check is performed. Its value must be an integer ranging from 5000 to 1,000,000. The default value is 5000.
<code>up_check_times</code>	Optional. This parameter specifies the number of times that the NTP service is detected to be available. Its value must be an integer ranging from 3 to 10. The default value is 3.
<code>down_check_times</code>	Optional. This parameter specifies the number of times that the NTP service is detected to be unavailable. Its value must be an integer ranging from 3 to 10. The default value is 3.

no monitor system ntp

This command is used to delete the NTP health check condition.

monitor vcondition name <*vcondition_name*> <*condition_name*> [*logic*]

This command is used to configure a health check condition group (vcondition). The vcondition can comprise multiple sub-conditions. The sub-condition can be a real health check condition or another vcondition, which further comprises sub-conditions.

vcondition_name	This parameter specifies the name of the vcondition. Its value must be a string of 1 to 128 characters.
condition_name	This parameter specifies the predefined condition name that is associated with the vcondition. The value of this parameter ranges from VCONDITION_1~VCONDITION_32.
logic	This parameter specifies the logical relationship among multiple sub-conditions of the vcondition, which can be either “AND” or “OR”. When “AND” is specified, the vcondition is “up” only if all the sub-conditions are “up”. When “OR” is specified, the vcondition is “up” if any sub-condition is “up”.

no monitor vcondition name <vcondition_name>

This command is used to delete the specified vcondition.



Note: If the command “**no monitor vcondition name**” is executed to delete a specified vcondition, the configurations related to this vcondition will also be deleted, including sub-conditions and related HA failover rules.

clear monitor vcondition all

This command is used to delete all the vconditions.

This command is used to clear all health check condition configurations, and reset the following to defaults:

- System health check function
- CPU temperature health check condition
- CPU utilization health check condition
- SSL card health check condition
- Memory utilization health check conditions, including system memory, network memory, connection memory and SWAP memory utilization
- Disk utilization health check conditions, including system disk space and data disk space utilization
- NTP health check condition

monitor vcondition member <vcondition_name> <subcondition_name>

This command is used to add a sub-condition to a specified vcondition. A vcondition can comprise a maximum of 16 sub-conditions.

vcondition_name	This parameter specifies the name of a vcondition.
subcondition_name	This parameter specifies the name of a sub-condition, which can be a real health check condition or a vcondition.

no monitor vcondition member <vcondition_name> <subcondition_name>

This command is used to delete a sub-condition from a specified vcondition.

clear monitor vcondition member <vcondition_name>

This command is used to delete all the sub-conditions from a specified vcondition.

monitor import scp <username@remote_scp_ip>

This command is used to import a self-defined health check script from a remote SCP server. To use self-defined health check scripts, please contact Customer Support.

username@remote_scp_ip	This parameter specifies the remote SCP server's user name and IP address, which must be separated by the "@" symbol. The directory and file name of the health check script must be set after the IP address followed by a colon symbol. The whole value must be enclosed in double quotes, for example, "user@10.8.3.12: /usr/ tool/ scr.tar.gz". Both IPv4 and IPv6 SCP servers are supported.
------------------------	---

monitor export scp <username@remote_scp_ip>

This command is used to export the self-defined health check script to a remote SCP server.

username@remote_scp_ip	This parameter specifies the remote SCP server's user name and IP address, which must be separated by the "@" symbol. The directory used to store the health check script must be set after the IP address followed by a colon symbol. The whole value must be enclosed in double quotes, for example, "user@10.8.3.12: /usr/save". Both IPv4 and IPv6 SCP servers are supported.
------------------------	---

monitor import tftp <remote_tftp_ip> <file_name>

This command is used to import a self-defined health check script from a remote TFTP (Trivial File Transfer Protocol) server. To use self-defined health check scripts, please contact Customer Support.

<code>remote_tftp_ip</code>	This parameter specifies the IP address of the remote TFTP server. Both IPv4 and IPv6 TFTP servers are supported.
-----------------------------	---

<code>file_name</code>	This parameter specifies the name of the script file saved on the remote TFTP server.
------------------------	---

monitor export tftp <remote_tftp_ip>

This command is used to export the self-defined health check script to a remote TFTP server.

<code>remote_tftp_ip</code>	This parameter specifies the IP address of the remote TFTP server. Both IPv4 and IPv6 TFTP servers are supported.
-----------------------------	---

show monitor config

This command is used to display the status of the system health check function and the configurations of all system health check conditions.

clear monitor all

This command is used to clear all health check condition configurations, and reset the following to defaults:

- System health check function
- CPU temperature health check condition
- CPU utilization health check condition
- SSL card health check condition
- Memory utilization health check conditions, including system memory, network memory, connection memory and SWAP memory utilization
- NTP health check condition

show statistics monitor

This command is used to display all system health check statistics.

clear statistics monitor

This command is used to clear all system health check statistics.

show statistics cpu

This command is used to display the current CPU utilization.

show statistics system

This command is used to display the current CPU utilization, average number of connections established per second, and average number of requests received per second.

clear system resource

This command is used to release the occupied system memory.



Note: This command can be used only when there is no or low network traffic; otherwise it may lead to unpredicted errors.

ip license server <ip_address> <port> <admin_name> <admin_password>
<email>

This command is used to configure the activation server. Currently, it is only used for vAPV. The vAPV must maintain regular contact with the activation server for registration and bandwidth usage reporting. If the vAPV fails to contact the activation server for seven days, the registered license key will expire.

ip_address	This parameter specifies the IP address of the activation server.
port	This parameter specifies the port number of the activation server. Its value must be an integer ranging from 1025 to 65,000.
admin_name	This parameter specifies the user name of the activation server. Its value must be a string of 1 to 16 characters enclosed in double quotes. Letters, numbers, and special characters are supported.
admin_password	This parameter specifies the password of the activation server. Its value must be a string of 1 to 80 characters enclosed in double quotes. Letters, numbers, and special characters are supported.
email	This parameter specifies an email address to receive alert mails. Its value must be enclosed in double quotes.

no ip license server

This command is used to delete the configuration of the activation server.

show ip license server

This command is used to display the configuration of the activation server.

Chapter 3 Advanced System Operations

This chapter covers the commands for Advanced System Operations.

mnet {*system_ifname*|*bond_ifname*|*vlan_ifname*} <*user_interface_name*>

This command is used to create a Multi-Netting (MNET) interface for the specified system interface, bond interface or VLAN interface. The system supports creating at most 4126 MNET interfaces.

system_ifname This parameter specifies the system interface name, which, by default, is port1, port2, port3, port4, etc. (Administrators can self-define the system interface name by using the command “**interface name**”).

bond_ifname This parameter specifies the bond interface name, which, by default, is bond1, bond2, bond3, bond4, etc. (Administrators can self-define the bond interface name by using the command “**bond name**”).

vlan_ifname This parameter specifies the VLAN interface name. (Administrators can create VLAN interfaces by using the command “**vlan**”).

user_interface_name This parameter specifies the MNET interface name, which should be a case-insensitive string with a length not greater than 31 characters. Letters, numbers, and special characters are supported.

Supported special characters: the at sign (@), exclamation mark (!), tilde (~), underscore (_), colon (:), hyphen (-), and period (.).

no mnet <*showmnet_ifname*>

This command is used to delete a specified MNET interface from the system.

mnet_ifname Specify the name of the system MNET interface which will be deleted.

[show|clear] mnet

This command is used to display or remove the configurations of all MNET interfaces.

vlan {*system_ifname*|*bond_ifname*} <*vlan_ifname*> <*vlan_tag*>

This command is used to create a Virtual Local Area Network (VLAN) interface for the specified system interface or bond interface. The system supports creating at most 4094 VLAN interfaces.

system_ifname	This parameter specifies the system interface name, which, by default, is port1, port2, port3, port4, etc. (Administrators can self-define the system interface name by using the command “interface name”).
bond_ifname	This parameter specifies the bond interface name, which, by default, is bond1, bond2, bond3, bond4, etc. (Administrators can self-define the bond interface name by using the command “bond name”).
vlan_ifname	This parameter specifies the name of the VLAN interface, which should be a case-insensitive string with a length not greater than 31 characters. Letters, numbers, and special characters are supported. Supported special characters: the at sign (@), exclamation mark (!), tilde (~), underscore (_), colon (:), hyphen (-), and period (.).
vlan_tag	This parameter specifies the ID for the VLAN interface being created, which should be any integer from 1 to 4094.



Note: Please do not configure VLAN on interface that has been configured as bond interface.

no vlan <vlan_ifname>

This command is used to delete a specified VLAN interface from the system.

vlan_ifname	Specify the name of the VLAN interface which will be deleted.
-------------	---

[show|clear] vlan

This command is used to display or remove the configurations of all VLAN interfaces.

fwd tcp <local_ip> <local_port> <remote_ip> <remote_port> [timeout]

This command allows users to assign a port on the APV appliance to a network IP/port pair. All TCP traffic to a specific local IP and port received by the APV appliance will be routed to a specified remote IP and port.

local_ip	Specify the local IP address to forward. It can be an IPv4 or IPv6 address.
----------	---

<code>local_port</code>	Specify the port to forward into the network server farm.
<code>remote_ip</code>	Specify the IP address of the server that the appliance will forward to the backend server. It can be an IPv4 or IPv6 address.
<code>remote_port</code>	Specify the destination port corresponding to the remote IP address.
<code>timeout</code>	Optional. This parameter specifies the timeout value in seconds. The default value is 300.

The maximum number of “**fwd tcp|udp**” items allowed on the APV appliance varies with system memories. Please see the following table for details.

System Memory	Maximum “fwd tcp udp” Items
<=4 GB	200
8 GB	600
>=16 GB	1200

fwd udp <local_ip> <local_port> <remote_ip> <remote_port> [timeout]

This command allows user to forward UDP packets. All UDP traffic to a specific local IP and port will be routed to a specified remote IP and port.

<code>local_ip</code>	Specify the local IP address to forward. It can be an IPv4 or IPv6 address.
<code>local_port</code>	Specify the UDP port to forward.
<code>remote_ip</code>	Specify the IP address of the server, in standard dotted format. It can be an IPv4 or IPv6 address.
<code>remote_port</code>	Specify the destination port corresponding to the IP address.
<code>timeout</code>	Optional. This parameter specifies the timeout value in seconds. The default value is 60.

The maximum number of “**fwd tcp|udp**” items allowed on the APV appliance varies with system memories. Please see the following table for details.

System Memory	Maximum “fwd tcp udp” Items
<=4 GB	200
8 GB	600
>=16 GB	1200

no fwd tcp <local_ip> <local_port>

no fwd udp <local_ip> <local_port>

These commands are used to disable the specified port-forwarding configuration.

show fwd

This command is used to display the mode of port forwarding.

clear fwd

This command is used to remove any configured port forwarding.

nat port {pool_name|vip} {source_ip|source_ip_group} {netmask|prefix}
[timeout] [gateway] [description]

This command is used to enable Network Address Translation (NAT) along with port translation. NAT converts the address of each server or device on the inside network into one IP address/IP address group or IP addresses in the pre-defined IP pool for the Internet, and vice versa. It also serves as a WebWall by keeping individual IP addresses hidden from the outside world. The appliance will check for subnet overlap or verify that the configured virtual IP exists. Data packets will be NATted if and only if:

- The source IP address is in the range of the configured “source_ip|source_ip_group” and “netmask|prefix”.
- The configured “gateway” is the same as the route gateway. If the “gateway” is set to the default value 0.0.0.0, the “VIP/IP pool” and the route gateway should be within the same network segment.

Up to 512 “**nat port**” configurations are allowed on one APV appliance.

pool_name vip	The virtual IP address or IP pool name that can be an IPv4 or IPv6 address/pool.
---------------	--

Note: If the configured VIP address is not on the same network segment as any system interface IP address, the VIP address is bound to interface port1 by default. Therefore, interface port1 must be pre-configured with an IP address.

source_ip source_ip_group	The network IP/IP address group to perform the network translation on, which can be an IPv4 or IPv6 address.
---------------------------	--

netmask prefix	The netmask or prefix length for the network performing the NAT.
----------------	--

- “netmask” is used for an IPv4 address. It can be a dotted IP address or an integer. If it is an integer, its value should range from 0 to 32.
- “prefix” is used for an IPv6 address. Its value should range from 0 to 128.

When the parameter “source_ip|source_ip_group” is

	specified as the source IP address group, this parameter is invalid.
timeout	Optional timeout setting in seconds; and it defaults to 60 seconds.
gateway	Specify the gateway IP address. The default value is 0.0.0.0. If “vip” or “pool_name” is in the same network segment as any interface on the APV appliance, the APV appliance will choose the route automatically and the “gateway” do not need to be set. If not, the “gateway” need to be set.
description	The description for this “ nat port ” configuration, which can serve as a reminder or note. The description should be enclosed in double quotes. Its maximum length is 31 characters.



Note:

- For IPv6 address, only ICMP, TCP (except FTP applications) and UDP packets can be NATted.
- L2 SLB configurations do not work with the “**nat port**” configuration. When the “**nat port**” configuration is associated with the system interface on which L2 SLB has been configured, it will not take effect.

no nat port {pool_name|vip} {source_ip|source_ip_group} {netmask|prefix}

This command is used to delete a specified “**nat port**” configuration.

clear nat port

This command is used to clear all “**nat port**” configurations.

nat portdst {pool_name|vip} {destination_ip|destination_ip_group}
{netmask|prefix} [timeout] [gateway] [description]

This command is used to enable NAT based on the destination IP address/IP address group of the packet. The source IP address in a packet will be translated when the destination IP address is in the specified network segment or in the same network segment as that of the route gateway. If the gateway is set to the default value “0.0.0.0”, the destination IP or IP pool for NAT and the route gateway should be in the same network segment.

pool_name vip	The virtual IP address or IP pool name that can be IPv4 or IPv6 address/pool.
destination_ip destination_ip_group	The destination IP or destination IP address group. Its value can be IPv4 or IPv6 address.
netmask prefix	The netmask or prefix length for the network performing

the NAT.

- `netmask` is used for an IPv4 address. It can be a dotted IP address or an integer. If it is an integer, its value should range from 0 to 32.
- `prefix` is used for an IPv6 address. Its value should range from 0 to 128.

When the parameter `destination_ip|destination_ip_group` is specified as the destination IP address group, this parameter is invalid.

timeout	Optional timeout setting of a NAT entry in seconds; and it defaults to 60 seconds.
gateway	Specify the gateway IP address. The default value is 0.0.0.0. If <code>vip</code> or <code>pool_name</code> is in the same network segment as any interface on the APV appliance, the APV appliance will choose the route automatically and the <code>gateway</code> do not need to be set. If not, the <code>gateway</code> need to be set.
description	The description for this “nat portdst” configuration, which can serve as a reminder or note. The description should be enclosed in double quotes. Its maximum length is 31 characters.



Note: For IPv6 address, only ICMP, TCP (except FTP applications) and UDP packets can be NATted.

no nat portdst *{pool_name|vip} {destination_ip|destination_ip_group} {netmask|prefix}*

This command is used to delete a specified **“nat portdst”** configuration.

clear nat portdst

This command is used to clear all **“nat portdst”** configurations.

show nat port

This command is used to display all **“nat port”** and **“nat portdst”** configurations.

nat static *<vip> <network_ip> [timeout] [gateway] [description]*

This command allows users to establish the static NAT route. This type of NAT route only translate the IP address but the port, and can process the traffic from the internal network to the public network and vice versa. Data packets will be NATted if and only if:

- The source IP address should be in the range of the configured `network_ip`.

- The configured “gateway” should be the same as the route gateway (The route gateway is configured by using the command “**ip route default**”). If the “gateway” is set to the default value 0.0.0.0, the “vip” and the route gateway should be within the same network segment.

The address translation will be performed on the IPv4 packets. However, if the first condition is met, the address translation will be performed on the IPv6 packets.

vip	A supplied virtual IP address. It can be an IPv4 or IPv6 address. Note: If the configured VIP address is not on the same network segment as any system interface IP address, the VIP address is bound to interface port1 by default. Therefore, interface port1 must be pre-configured with an IP address.
network_ip	The network IP to perform the network translation on. It can be an IPv4 or IPv6 address.
timeout	Timeout value in seconds; it defaults to 60 seconds.
gateway	Specify the gateway IP address. It can be an IPv4 or IPv6 address. For the IPv4 address, the default value is 0.0.0.0 and for the IPv6 address, the default value is ::. If “vip” or “pool_name” is in the same network segment as any interface on the APV appliance, the APV appliance will choose the route automatically and the “gateway” do not need to be set. If not, the “gateway” need to be set.
description	The description for this “ nat static ” configuration, which can serve as a reminder or note. The description should be enclosed in double quotes. Its maximum length is 31 characters.



Note: L2 SLB configurations do not work with the “**nat static**” configuration. When the “**nat static**” configuration is associated with the system interface on which L2 SLB has been configured, it will not take effect.

no nat static <vip>

This command is used to remove the specified virtual IP address from the static NAT configuration.

show nat static

This command is used to display all static NAT configurations.

clear nat static

This command is used to stop and remove the static NAT configuration.

nat itcpopt on <tcp_option_kind>

This command is used to enable the function of forwarding clients' addresses to servers via the specified TCP option in the static NAT, NATP (Network Address Port Translation) and NAT64&NAT46 scenarios. Servers can obtain clients' real addresses from the TCP option for logging or auditing. The specified TCP option will be inserted in the first SYN and ACK messages that the system sends to the servers. By default, this function is disabled.

tcp_option_kind	This parameter specifies the kind value of the TCP option. Its value must be an integer ranging from 31 to 252.
-----------------	---

nat itcpopt off

This command is used to disable the function of forwarding clients' addresses to the server via the specified TCP option in the static NAT and NATP scenarios.

show nat itcpopt

This command is used to display the enabling status of the function of forwarding clients' addresses to the server via the specified TCP option in the static NAT and NATP scenarios.

clear nat itcpopt

This command is used to restore the function of forwarding clients' addresses to the server via the specified TCP option in the static NAT and NATP scenarios to default.

nat protocol pptp [port] [direction]

This command is used to enable NAT traversal for PPTP tunnels in the inbound or outbound direction. This function is disabled by default.

port	Optional. This parameter specifies the port number of the PPTP server. The default value is 1723.
------	---

direction	Optional. This parameter specifies the direction in which NAT traversal for PPTP tunnels is enabled. The valid values of this parameter are "inbound" and "outbound". The default value is "outbound".
-----------	--

no nat protocol pptp [direction]

This command is used to disable NAT traversal for PPTP tunnels in the inbound or outbound direction.

direction	Optional. This parameter specifies the direction in which NAT traversal for PPTP tunnels is disabled. The valid values of this parameter are "inbound" and "outbound".
-----------	--

The default value is “outbound”.

clear statistics ptp

This command is used to remove all the PPTP statistics on the APV appliance.

show nat protocol ptp

This command is used to display the configurations of NAT traversal for PPTP tunnels.

nat protocol ftp port <port_number>

This command is used to configure an alternative FTP port for the NAT function of the FTP service. A maximum of 10 alternative FTP ports can be configured for the NAT function of the FTP service. If no alternative FTP port is configured for the NAT function of the FTP service, NAT can process only FTP data packets received on port “21” in FTP active mode.

port_number	This parameter specifies the FTP port number. Its value must be an integer ranging from 1 to 65390. The parameter cannot be set to “21” because the FTP port “21” is the default FTP port already supported by the system.
-------------	--

no nat protocol ftp port <port_number>

This command is used to delete the specified alternative FTP port configured for the NAT function of the FTP service.

show nat protocol ftp port

This command is used to display all alternative FTP ports configured for the NAT function of the FTP service.

clear nat protocol ftp port

This command is used to clear all alternative FTP ports configured for the NAT function of the FTP service.

nat protocol dns [method]

This command is used to enable the DNS NAT function. This function is applicable to the scenario where the resolved IPs in the DNS responses returned by internal DNS servers are the IP addresses of internal servers that cannot be publicly accessed. With the DNS NAT function enabled, when receiving DNS responses from an internal DNS server, the appliance will translate the internal resolved IP addresses in the DNS responses to the public IP addresses based on the NAT static entries (configured using the “**nat static**” command) and return the translated DNS responses to the client. When external clients use the publicly accessible IP addresses for access, the system translates the publicly accessible IP addresses back to the internal resolved IP

addresses of internal servers based on the NAT static entries. In this way, external clients can access internal servers. By default, this function is disabled.

method

Optional. This parameter specifies how to translate the resolved IP addresses in DNS responses in multi-link environment. Its value must be:

- **rr**: indicates that the resolved IP addresses in the DNS responses will be translated to the public IP addresses in the NAT static entries corresponding to multiple links in round robin manner.
- **proximity**: indicates that the resolved IP addresses in the DNS responses will be translated to the public IP address in the NAT static entry corresponding to the nearest link used to transmit the DNS response.

The default value is proximity.



Note: After the DNS NAT function is enabled in multi-link environment:

- For the **rr** method, if the link corresponding to the selected NAT static entry is available, this NAT static entry will be used for translation; otherwise, the appliance will select the next NAT static entry in round robin manner. If links corresponding to all NAT static entries are unavailable, the NAT static translation will not be performed.
- For the **proximity** method, if the nearest link is selected, the NAT static entry corresponding to this link will be used for translation. If this link corresponds to multiple NAT static entries, the first configured NAT static entry will be used for translation. If there is no NAT static entry corresponding to this link, the appliance will select the NAT static entry based on the **rr** method.

no nat protocol dns

This command is used to disable the DNS NAT function.

show nat protocol dns

This command is used to display the configuration of the DNS NAT function.

clear nat protocol dns

This command is used to clear the configuration of the DNS NAT function.

show nat dns status <network_ip>

This command is used to display the DNS NAT statistics for the specified network IP.

network_ip

This parameter specifies a network IP address. Its value must be the “network_ip” value in an existing NAT static entry configured using the “**nat static**” command.

clear nat dns status

This command is used to clear all the DNS NAT statistics.

show nat table [ip_address]

This command displays the existing network translations for incoming and outgoing traffic and the statistics of GRE tunnels. This command only supports TCP protocol and doesn't support ICMP and UDP protocols.

ip_address Specify the IP address which is needed to filter. The IP address can be the source IP address, destination IP address, or virtual IP address of the NAT entry.

show statistics nat

This command is used to display the statistics on the NAT function.

```
AN(config)#show statistics nat
nat static 10.8.1.194 10.3.188.189 60 0.0.0.0
    Outbound protocol(total connection/current connection): icmp(4/10) udp(5/1) tcp(9/2)
    IP packet(total packet in/total packet out):      933/933
    IP packet(total byte in/total byte out):          100764/100764

    Inbound  protocol(total connection/current connection): icmp(2/0) udp(6000/3338)
tcp(25/2)
    IP packet(total packet in/total packet out):      933/933
    IP packet(total byte in/total byte out):          100764/100764
nat static 2012:1081::10:8:1:194 2012:1030::10:3:188:189 60 ::
    Outbound      protol(total/current):icmp(0/0) udp(63/4) tcp(4/1)
    Inbound       protol(total/current):icmp(0/0) udp(6000/3338) tcp(3000/1499)
```



Note: The IPv6 addresses of ICMP packets can not be translated; therefore the values of ICMP statistics for IPv6 are 0.

nat pureip {on|off}

This command is used to enable or disable the NAT traversal function for the non-TCP, non-UDP and non-ICMP IP packets. This function is applicable to the protocols for which the NAT function can be implemented by only the change of the IP header. For example, the Encapsulating Security Payload (ESP) protocol in the tunnel mode. This function is used for only the IPv4 packets and the static NAT. By default, it is disabled.

**Note:**

- For IP packets of some protocols, only changing the IP header cannot implement the NAT function. Therefore, it is recommended to enable this function only when it is required.
- When the NAT function is implemented on the IP packets of the ESP protocol in the tunnel mode, the Security Parameters Index (SPI) conflict may exist. Therefore, when this function is enabled, multiple private IP

addresses cannot be translated to one VIP.

- When this function is enabled, the APV appliance will use the client's IP (transparent) as source IP address in port forward connection regardless of the port forwarding mode.

show nat pureip

This command is used to display the status of the NAT traversal function for the non-TCP, non-UDP and non-ICMP IP packets.

rip {on|off}

This command is used to enable or disable Routing Information Protocol (RIP). When this function is enabled, the system learns RIP routes. The administrator can use the “**show ip route**” command to view the learned RIP routes. The RIP function is disabled by default.

For this function to work, please use the command “**rip network**” to add a Droute for RIP.

rip version <rip_version>

This command is used to set the version of RIP.

rip_version	This parameter specifies the version of RIP, and the value must be 1 or 2.
-------------	--

rip network <ip_address> <netmask>

This command is used to add a Droute for RIP. The system learns the route on other appliances through the specified Droute.

ip_address	This parameter specifies the IP address of the Droute to be added to RIP. Its value must be an IPv4 address.
------------	--

netmask	This parameter specifies the netmask of the IP address. Its value must be a dotted IP address.
---------	--

no rip network <ip_address> <netmask>

This command is used to delete a specified Droute for RIP .

show rip status

This command is used to display the status of RIP.

show rip settings

This command is used to display the current settings of RIP.

ripng {on|off}

This command is used to enable or disable the RIP next generation (RIPng) for IPv6 networks. When this function is enabled, the system learns RIPng routes. The

administrator can use the “**show ip route**” command to view the learned RIPng routes . The RIPng function is disabled by default.

For this function to work, please use the command “**ripng interface**” to enable the RIPng function for the specified interface.

ripng interface <interface_name>

This command is used to enable the RIPng function for a specified interface.

interface_name	This parameter specifies the interface on which RIPng is to be enabled.
----------------	---

no rip interface <interface_name>

This command is used to disable the RIPng function for a specified interface.

show ripng status

This command is used to display the running status information of RIPng.

show ripng settings

This command is used to display all current RIPng settings.

ospf {on|off}

This command is used to enable or disable the OSPF function. When the OSPF function is enabled, the system will study OSPF routes. However, the studied OSPF routes will not take effect immediately. They will take effect within about 30 seconds. The OSPF function is disabled by default.

ospf network <ip_address> <netmask> <area_id>

This command is used to enable the OSPF function for the specified network and set the area ID for the OSPF network.

ip_address	This parameter specifies the network IP address. Its value must be an IPv4 address.
------------	---

netmask	This parameter specifies the netmask of the network. Its value must be a dotted IP address
---------	--

area_id	This parameter specifies the area assigned to the OSPF network. Its value must be an integer ranging from 0 to 4,294,967,296.
---------	---

no ospf network <ip_address> <netmask> <area_id>

This command is used to disable the OSPF function for the specified network and delete the area ID setting for the OSPF network.

ospf interface cost <interface_name> <cost>

This command is used to configure an OSPF interface and set the interface cost. If this command is not configured, the cost is calculated by the interface speed.

interface_name	This parameter specifies the interface name. Its value must be the name of a system interface, VLAN interface or bond interface.
cost	This parameter specifies the cost needed by the interface to run the OSPF protocol. Its value must be an integer ranging from 1 to 65,535.

no ospf interface cost <interface_name>

This command is used to delete the cost setting of the specified interface.

ospf router <router_id>

This command is used to set the OSPF router ID. If this command is not configured, the largest IP address of system interfaces will be used as the router ID.

router_id	This parameter specifies the OSPF router ID. Its value must be an IPv4 address.
-----------	---

no ospf router

This command is used to delete the OSPF router ID setting.

show ospf status

This command is used to display running status of OSPF.

show ospf settings

This command is used to display the current settings of OSPF.

ipv6 ospf {on|off}

This command is used to enable/disable the OSPFv3 function (IPv6 OSPF). The function is disabled by default.

ipv6 ospf routerid <id>

This command is used to set the OSPF router ID in dotted format IPv4 address.

ipv6 ospf drpriority <interface_name> <priority>

This command is used to set the interface priority for the OSPF DR (designated router) election.

interface_name	Specify the interface name, which can be the system
----------------	---

interface, bond interface, VLAN interface or MNET interface.

priority Specify the interface priority. Its value ranges from 0 to 255 and defaults to 1.

ipv6 ospf interface <interface_name> <area_id>

This command is used to enable the OSPFv3 on the specified interface and define an area ID for this interface.

Interface_name Specify the interface name, which can be the system interface, bond interface, VLAN interface or MNET interface.

area_id The “area_id” parameter can be specified with a number between 0 to 4294967295 and default to 0.

no ipv6 ospf interface <interface_name> <area_id>

This command is used to delete the OSPFv3 configuration on the interface with the specified area ID.

show ipv6 ospf settings

This command is used to display the OSPFv3 settings. For example:

```
AN#show ipv6 ospf settings
ipv6 ospf on
ipv6 ospf drpriority port2 2
ipv6 ospf drpriority port1 1
ipv6 ospf drpriority port4 1
ipv6 ospf routerid 1.1.1.1
ipv6 ospf interface port4 0
ipv6 ospf interface port1 2
```

show ipv6 ospf status

This command is used to display the OSPFv3 status (on/off).

ospf rhi {on|off}

This command is used to enable or disable the Route Health Injection (RHI) function. By default, the RHI function is disabled.

After the RHI function is enabled:

- When the health check result of any real service related to a virtual IP address is UP, the virtual IP address will be set as ACT (active). The system will inject the routing information of the virtual IP address to the route table. When the virtual

IP address is configured in ospf network, the virtual IP address is published to other devices on the network through OSPF. If the network segment where the ACT virtual IP is located has enabled the OSPF function (through the “ospf network” command), the route information of this virtual IP address will be published to other devices in the network by the OSPF protocol.

- When the health check results of all real services related to a virtual IP address are DOWN, the virtual IP address will be set as INACT (inactive). The system will remove the routing information of the INACT virtual IP address from the routing table and therefore the routing information will not be advertised to other devices on the network.



Note: If the virtual IP address is the only virtual IP address of a virtual cluster, the virtual cluster will be set as Active or Init as the virtual IP address is set to ACT or INACT. If not, the ACT or INACT status of the virtual IP address does not affect the status of the virtual cluster.

show ospf rhi status

This command is used to display the ACT and INACT statuses of virtual IP addresses.

show statistics rhi

This command is used to display the statistics of activated and deactivated virtual IP addresses.

clear statistics rhi

This command is used to clear the statistics of ACT and INACT virtual IP addresses.

bgp {on|off}

This command is used to enable or disable the BGP function. By default, this function is disabled. After this function is enabled, the APV appliance will work as a local BGP router.

bgp asn <as_number>

This command is used to set the Autonomous System Number (ASN) of the local BGP router.

as_number	Specify the ASN of the local BGP. Its value should be an integer ranging from 1 to 4,294,967,295.
-----------	---

no bgp asn <as_number>

This command is used to delete the BGP ASN specified for the local BGP router. After an ASN is deleted, all configurations related with the ASN including BGP neighbors and the BGP management distance value will be deleted.

bgp router <router_id>

This command is used to set the ID of the local BGP router. If this command is not configured, the local BGP router will use the largest interface IPv4 address of the APV appliance as the router ID by default.

<code>router_id</code>	Specify the ID of the local BGP router. Its value should be an IPv4 address in the dotted decimal notation.
------------------------	---

no bgp router <router_id>

This command is used to delete the specified router ID.

bgp network <ip_address> {netmask|prefix}

This command is used to add the network to be advertised to neighbors. This command is available only after the ASN of the local BGP router is configured.

<code>ip_address</code>	Specify the IP address of the network to be advertised to neighbors. Its value can be an IPv4 or IPv6 address.
<code>netmask prefix</code>	Specify the netmask or prefix length of the network to be advertised to neighbors. • “netmask” is used for an IPv4 address. The parameter can be a dotted IP address or an integer ranging from 0 to 32. • “prefix” is used for an IPv6 address. Its value should be an integer ranging from 0 to 128.

no bgp network <ip_address> {netmask|prefix}

This command is used to delete the network to be advertised to neighbors.

bgp neighbor <ip_address> <as_number>

This command is used to specify a BGP neighbor and the ASN of the BGP neighbor. This command is available only after the ASN of the local BGP router is configured.

<code>ip_address</code>	Specify the IP address of the BGP neighbor. Its value can be an IPv4 or IPv6 address. Note: If the BGP connection is established through an IPv6 Link-Local address, you need to execute the command “ bgp interface <ip_address> <interface_name> ” to specify the interface used by the local BGP router.
<code>as_number</code>	Specify the ASN of the BGP neighbor. Its value should be an integer ranging from 1 to 4,294,967,295.

no bgp neighbor <ip_address>

This command is used to delete the specified BGP neighbor.

show ip bgp neighbors [ip_address]

This command is used to display detailed information of the specified BGP neighbor. If the parameter “ip_address” is not specified, the detailed information of all BGP neighbors will be displayed.

bgp interface <ip_address> <interface_name>

This command is used to specify the interface used by the local BGP router using an IPv6 Link-Local address to establish the BGP connection. This command is available only after the BGP neighbor is specified.

ip_address	Specify the IPv6 address of the interface on the local BGP router.
interface_name	Specify the name of the interface on the local BGP router. Its value can be a system interface or a bond interface.

no bgp interface <ip_address>

This command is used to delete the configuration of the interface used by the local BGP router using an IPv6 Link-Local address to establish the BGP connection.

bgp distance <ext_distance> <int_distance> <loc_distance>

This command is used to specify the management distance of the local BGP router, which uses the management distance to select optimal routes. The lower the management distance value, the higher the route priority. This command is available only after the ASN of the local BGP router is configured.

ext_distance	Specify the management distance of external routes, which are the routes learnt through External BGP (EBGP). Its value should be an integer ranging from 1 to 255.
int_distance	Specify the management distance of internal routes, which are the routes learnt through Internal BGP (IBGP). Its value should be an integer ranging from 1 to 255.
loc_distance	Specify the management distance of local routes, which are the routes configured on the local BGP router. Its value should be an integer ranging from 1 to 255.

show ip bgp route [ip_address/mask|prefix]

This command is used to display the detailed information of the specified BGP route.

ip_address/netmask prefi	Specify the IPv4 address and netmask or IPv6 address and prefix length. If this parameter is not specified, the detailed
--------------------------	--

- x information of all BGP routes will be displayed.
- “netmask” is used for an IPv4 address. The parameter can be a dotted IP address or an integer ranging from 0 to 32.
 - “prefix” is used for an IPv6 address. The parameter value should be an integer ranging from 0 to 128.

bgp capability <ip_address> <option>

This command is used to advertise specified BGP capabilities to the specified BGP neighbor.

- ip_address Specify the IP address of the BGP neighbor. Its value can be an IPv4 or IPv6 address.
- option Specify the BGP capability to be advertised to the BGP neighbor. Its values are as follows (case sensitive):
- dynamic: Advertise the dynamic update capability. The dynamic update capability enables BGP neighbors to dynamically update BGP capabilities without resetting BGP connections.
 - both: Exchange the Outbound Route Filter (ORF) prefix list with the specified BGP neighbor.
 - receive: Receive the ORF prefix list from the specified BGP neighbor.
 - send: Send the ORF prefix list to the specified BGP neighbor.

no bgp capability <ip_address> <option>

This command is used to cancel the advertisement of the specified BGP capability previously to the specified BGP neighbor.

bgp passive <ip_address>

This command is used to configure the BGP neighbor to which the local BGP router does not actively send the Open packet.

- ip_address Specify the IP address of the BGP neighbor. Its value can be an IPv4 or IPv6 address.

bgp nexthopself <ip_address>

This command is used to set the next hop IP address of the advertised route to the IP address of the local BGP router. This command is available only after the ASN of the local BGP router is configured.

<code>ip_address</code>	This parameter specifies the IP address of the BGP neighbor. Its value can be an IPv4 or IPv6 address.
-------------------------	--

bgp def_orig <ip_address>

This command is used to advertise the default route destined for the local BGP router to the BGP neighbor.

<code>ip_address</code>	Specify the IP address of the BGP neighbor. Its value can be an IPv4 or IPv6 address.
-------------------------	---

bgp local_as <ip_address> <as_number>

This command is used to set the substitute ASN used by the local BGP router to communicate with the specified EBGp neighbor.

<code>ip_address</code>	Specify the IP address of the EBGp neighbor. Its value can be an IPv4 or IPv6 address.
-------------------------	--

<code>as_number</code>	Specify the substitute ASN used by the local BGP router. Its value should be an integer ranging from 1 to 4,294,967,295.
------------------------	--



Note: The substitute ASN cannot be the same as the ASN of the BGP neighbor.

no bgp local_as <ip_address> <as_number>

This command is used to delete the substitute ASN used by the local BGP router to communicate with the specified EBGp neighbor.

bgp shutdown <ip_address>

This command is used to terminate the communication with the specified BGP neighbor.

<code>ip_address</code>	Specify the IP address of the BGP neighbor. Its value can be an IPv4 or IPv6 address.
-------------------------	---

no bgp shutdown <ip_address>

This command is used to restore the communication with the specified BGP neighbor.

bgp redistribute <option>

This command is used to dynamically redistribute the routes of the specified type into BGP.

option	Specify the type of routes to be dynamically redistribute into BGP. Its values are as follows (case sensitive): • connected: directly connected routes • ospf: OSPF routes • rip: RIP routes • static: static routes
--------	--

no bgp redistribute <option>

This command is used to cancel the dynamical redistribution of the routes of the specified type into BGP.

show bgp settings

This command is used to display all BGP configurations.

clear bgp ip [ip_address]

This command is used to reset the connection between the local BGP router and the specified BGP neighbor. If the parameter “ip_address” is not specified, the connections between the local BGP router and all BGP neighbors will be reset. After BGP configurations such as the management distance value are changed, you need to reset the connection between the local BGP router and the specified BGP neighbor to make changes take effect.

ip_address	Specify the IP address of the BGP neighbor. Its value can be an IPv4 or IPv6 address.
------------	---

clear bgp asn <as_number>

This command is used to reset the connections between the local BGP router and all BGP neighbors in the specified Autonomous System (AS). After BGP configurations such as the management distance value are changed, you need to reset the connection between the local BGP router and all BGP neighbors in the specified AS to make changes take effect.

as_number	Specify the ASN of the BGP neighbors.
-----------	---------------------------------------

LLDP**lldp {on|off} <system_ifname>**

This command is used to enable or disable the Link Layer Discovery Protocol for the specified system interface. After LLDP is enabled on a system interface, it is able to receive and send LLDP packets. By default, LLDP is disabled on all system interfaces.

system_ifname This parameter specifies the system interface name, which can be customized by the “**interface name**” command.

The default system interface name is port1, port2, port3, port4, etc.

lldp sysname <system_name>

This command is used to set the system name of the APV appliance which is acting as an LLDP neighbor.

When this command is not configured, the hostname set by the “hostname” command will be used as the system name by default.

system_name This parameter specifies the system name. Its value must be a string of 1 to 64 characters. Letters, numbers and special characters are supported. When the parameter value contains special characters, it must be enclosed in double quotes.

lldp transmit_delay <interval>

This command is used to set the transmission interval between LLDP packets.

When this command is not configured, the default interval is 30s.

interval This parameter specifies the interval, in seconds. Its value must be an integer ranging from 1 to 100.

lldp max_neighbors <number>

This command is used to set the maximum number of neighbors allowed on each system interface with LLDP enabled.

When this command is not configured, the default value is 32.

number This parameter specifies the maximum number of neighbors. Its value must be an integer ranging from 1 to 64.

show lldp config

This command is used to display all LLDP configurations.

show lldp neighbors

This command is used to display the information of all LLDP neighbors.

clear lldp all

This command is used to restore all LLDP configurations to default.

ipv6 ndp <ipv6_address> <mac_address>

This command is used to add a static NDP entry to the system.

ipv6_address Specify the IPv6 address of a remote host.

mac_address Specify the MAC address of the remote host.

no ipv6 ndp <ipv6_address>

This command is used to remove a static or dynamic NDP entry of the specified IPv6 address.

show ipv6 ndp

This command is used to display all the static and dynamic NDP entries.

clear ipv6 ndp

This command is used to clear all the static and dynamic NDP entries.

ip pool <pool_name> <start_ip> [end_ip] [interface_name]

This command is used to create an IP pool and add an IP segment into the IP pool. This command can also be used to only add an IP segment into an IP pool. Multiple IP segments can be added into a pool. If the IP pool input does not exist, ArrayOS will create a new IP pool. The maximum number of IP pools supported on APV appliance varies with different system memories, and the maximum number of IP addresses allowed for each IP pool is 256.

pool_name This parameter specifies the name of the IP pool. Its value must be a string of 1 to 128 characters. Letters, numbers, and special characters are supported. The value must begin with a letter, and be enclosed in double quotes when it contains special characters.

start_ip Specify the starting IP address of the IP segment. It can be an IPv4 or IPv6 address.

end_ip Optional. Specify the end IP address of the IP segment. It can be an IPv4 or IPv6 address. If not assigned, only the “start_ip” will be added into the IP pool.

interface_name Optional. Specify the name of interface, which should be configured with the overlap option, such as a system, MNET, or Bond interface. This parameter need to be used only when the NUMA SLB (single VIP, reverse mode) is

deployed. The default value is empty and the pool is bound to the original interface by default. If the interface of the IP pool is an IPv6 interface, the interface address and IP pool addresses must be of the same segment.



Note:

- An IP pool only can be bound to an interface.
- Normally, the IP segment in several IP pools can overlapped. But if an IP pool is bound to the interface with “**overlap**” configured , the IP segment included in this IP pool should not added to other IP pools.
- Multiple IP pools can be configured on the APV appliance and at most 256 IP addresses can be added to an IP pool. The maximum number of IP pools varies with different system memories. Please see the table below for details.

System Memory	Maximum IP Pool Number
4GB	32
8GB	64
16GB	128
32GB	256

no ip pool <pool_name> [start_ip]

The command is used to remove an IP segment from the specified IP pool.

pool_name The name of the IP pool.

start_ip The starting IP address of the IP segment to be removed.
With the start IP configured, the IP segment that begins with this IP address will be removed.

This parameter is optional. If not specified, the specified IP pool will be removed.

clear ip pool [pool_name]

This command is used to remove the specified IP pool. If the parameter “pool_name” is not assigned, the command will remove all the IP pools.

show ip pool [pool_name]

This command is used to display configurations about the specified IP pool. If the parameter “pool_name” is not assigned, the command will show configurations about all the IP pools.

IP Region Table

Basic IP Region Table Commands

show ipregion name

This command is used to display the names of all existing IP region tables in the system.

ipregion table import <ipregion_name> <url>

This command is used to import a customized IP region table to the system from an HTTP server or FTP server.

ipregion_name	This parameter specifies the name of the IP region table to be imported. Its value must be a case-sensitive string of 1 to 24 characters. Its value must not be the name of a predefined IP region table.
---------------	---

url	This parameter specifies the HTTP or FTP URL through which to import the IP region table.
-----	---

Note: The “path” of the FTP URL (ftp://user:password@host:port/path) should be a relative path.

The customized IP region table includes the following items:

- IP subnet (in CIDR format)
- Country name (optional, up to 7 bytes)
- Brief description (optional, up to 63 bytes)

Every two items must be separated with a “Tab”.

The system will check the contents of the file instead of the file type when an IP region file is imported. To ensure that the IP region file can be imported successfully, please use the following format to define an IP region table:

IP subnet Country name Brief description

For example, define an IP region table named “test.txt” that includes the following content:

17.249.0/24 AM Armenia 2.56.204.0/22 AM Armenia
--

Use the following command to import this IP region table:

AN(config)# ipregion table import test ftp://10.3.109.1/test.txt

If the FTP server requires login username and password, administrators need to include valid username and password information in the FTP URL, such as the following example:

AN(config)# ipregion table import test ftp://username:passwd@10.3.109.1/test.txt

ipregion table export <ipregion_name> <url>

This command is used to export an IP region table to the FTP server.

ipregion_name This parameter specifies the name of the IP region table to be exported.

url This parameter specifies the FTP URL to which the IP region table will be exported.

Note: The “path” of the FTP URL (ftp://user:password@host:port/path) should be a relative path.

Example:

```
AN(config)#ipregion table export test12 ftp://10.3.109.1/test
ipregion "test12" is successfully exported
```

If the FTP server requires login username and password, administrators need to include the valid username and password information in the FTP URL, such as the following example:

```
AN(config)#ipregion table export test12 ftp://username:passwd@10.3.109.1/test
ipregion "test12" is successfully exported
```

no ipregion table <ipregion_name>

This command is used to delete the specified IP region table.

show ipregion table <ipregion_name>

This command is used to display the entries of the specified IP region table.

clear ipregion table [predefined]

This command is used to delete customized or predefined IP region tables.

predefined Optional, this parameter specifies whether to delete the predefined IP region tables. Its value must be:

- predefined: means deleting predefined IP region tables.
- empty: means deleting customized IP region tables.

The default value is empty.

GeoIP-based IP Region Table

The system supports the generation of IP region tables based on GeoIP databases. The following commands are provided for importing GeoIP databases into the system and generating country-level IP region tables. After IP region tables are generated, you can import them into the system or change related configurations by using the commands in section “Basic IP Region Table Commands”.

ipregion geoip import <region_level> <url>

This command is used to import a GeoIP (Geolocation IP) database from the specified URL address, which will be used to generate IP region tables.

After the country-level GeoIP database is imported, the system will generate the following CSV database files (“YYYYMMDD” indicates the update date of the corresponding GeoIP database):

- Country_IPv4_YYYYMMDD.csv: country-level IPv4 address database
- Country_IPv6_YYYYMMDD.csv: country-level IPv6 address database
- Country_Location_YYYYMMDD.csv: country-level region database

After the state-level GeoIP database is imported, the system will generate the following CSV database files (“YYYYMMDD” indicates the update date of the corresponding GeoIP database):

- State_IPv4_YYYYMMDD.csv: state-level IPv4 address database
- State_IPv6_YYYYMMDD.csv: state-level IPv6 address database
- State_Location_YYYYMMDD.csv: state-level region database

Note: The system supports only zipped CSV-format databases provided by MaxMind. Download address: <http://dev.maxmind.com/geoip/geoip2/geolite2/>.

region_level	This parameter specifies the region level of the GeoIP database to be imported. Its value must be: • country: imports a country-level GeoIP database. • state: imports a state-level GeoIP database.
url	This parameter specifies the URL address of the database to be imported, which must be enclosed in double quotes. Both FTP and HTTP URLs are supported.

Example:

```
AN(config)#ipregion geoip import state
“ftp://192.168.5.55/home/GeoLite2-City-CSV_20160503.zip”
```

show ipregion geoip

This command is used to display all GeoIP databases that the system has generated.

ipregion geoip convert country <country_name>

This command is used to generate the specified country's IP region table based on the country-level GeoIP database. Before this command is executed, you need to import the country-level GeoIP database using the “ipregion geoip import” command.

country_name This parameter specifies the country name. Its value must be a country name defined by the “country_name” column in the country-level GeoIP database. This parameter can also be set to “all”, indicating that all countries' IP region tables will be generated.

ipregion geoip convert state <country_name> <state_name>

This command is used to generate the IP region table of the specified state based on the imported state-level GeoIP database. Before this command is executed, you need to import the state-level GeoIP database using the “ipregion geoip import” command.

country_name This parameter specifies the country name.

Crontab

crontab {enable|disable}

This command is used to enable or disable the Crontab function. If the Crontab function is enabled, the system will scan the Crontab file at the minutes 0, 10, 20, 30, 40 and 50 of each hour and execute the CLI commands whose start time is within five minutes earlier than or later than five minutes of these minutes. For example, if the current time is 12:50, all the CLI commands whose start time is from 12:45:00 to 12:54:59 will be executed.

The Crontab file must be imported using the command “crontab import” before this function is enabled. By default, this function is disabled.

The Crontab file can contain multiple Crontab entries. Each Crontab entry consists of time information items and a CLI command. The time information items specify the start time of CLI commands.

The Crontab entry supports two formats:

- Absolute: year-month-day hour:minute command
- Periodic: day of week hour:minute command

Value ranges of parameters in the time information items are as the following table:

Time Information Item	Parameter Value Range
minute	00-59
hour	00-23

Time Information Item	Parameter Value Range
day	01-31
month	01-12
day-of-week	0 to 7: “1” to “7” indicates Monday to Sunday; “0” indicates Sunday.
	Mon, Tue, Wed, Thu, Fri, Sat or Sun.
	weekday: indicates Monday to Friday in every week.
	weekend: indicates Saturday and Sunday in every week.
	daily: indicates Monday to Sunday in every week.

These items must be separated with a “Tab” or space. For the “day-of-week” parameter, its value is case-sensitive.

For example:

- Define a Crontab file named “Eroute.txt”, which is used to set that an Eroute “p1” will be added at 08:00 and deleted at 22:00 each day from Monday to Friday. The content of the Crontab file is as follows:

```
weekday 08:00 ip eroute "p1" 1300 0.0.0.0 0.0.0.0 0 2.1.1.0 255.255.255.0 0 any 1.1.1.1 1
weekday 22:00 no ip eroute "p1"
```

- Define a Crontab file named “Eroute.txt”, which is used to set that an Eroute “p2” will be added at October 10, 2014 and deleted at October 10, 2014. The content of the Crontab file is as follows:

```
2014-10-10 08:00 ip eroute "p2" 1300 0.0.0.0 0.0.0.0 0 3.1.1.0 255.255.255.0 0 any 1.1.1.1 1
2014-10-15 22:00 no ip eroute "p2"
```



Note:

- Please make sure the existing configuration does not conflict with the configuration specified in the Crontab file; otherwise, it may lead to unpredictable error.
- The Crontab function does not support the commands “**ip address**”, “**ip route**”, “**ssh ip**”, “**bond**”, “**hostname**”, “**mnet**”, “**vlan**”, “**webwall**”, “**webui ip**”, “**cluster virtual priority**”, “**interface name**”, “**interface speed**”, “**interface mtu**”, “**interface mac**”, “**ha ssh peer**”, “**accesslist**” and “**accessgroup**”.

crontab import <url>

This command is use to import the Crontab file. Only one Crontab file can be imported in the system.

This command is also used to modify the imported Crontab file.

url This parameter specifies the HTTP or FTP URL of the Crontab file to be imported. Its value must be a string that supports 0-9, a-z, A-Z, and special characters. The value only supports the following special characters: “@”, “!”, “-”, “_”, “.”, “/”, “:”.

Note: The “path” of the FTP URL

(ftp://user:password@host:port/path) should be a relative path.

crontab export <url>

This command is used to export the current Crontab file.

url This parameter specifies the FTP URL to which the Crontab file is exported.

Note: The “path” of the FTP URL (ftp://user:password@host:port/path) should be a relative path.

show crontab file

This command is used to display the contents of the imported Crontab file.

show crontab config

This command is used to display the current Crontab configuration.

VXLAN**vxlan enable**

This command is used to enable the Virtual eXtensible Local Area Network (VXLAN) function. By default, this function is disabled.

vxlan disable

This command is used to disable the VXLAN function.

vxlan mode <p2mp|multicast>

This command is used to configure the working mode of the VXLAN function. The administrator should specify the VXLAN working mode before configuring any VXLAN tunnel. Please note that the binding relationship between the VXLAN tunnels and VXLAN interfaces should be cleared if the VXLAN working mode is modified. Otherwise, the VXLAN function will not work properly. If this command is not configured, the default working mode is p2mp.

p2mp|multicast This parameter specifies the working mode of the VXLAN function. Its value must be:

- p2mp: in this working mode, for a VXLAN, the administrator needs to create unicast tunnels for the appliance with all VXLAN Tunnel End Points (VTEPs).

- multicast: in this working mode, for a VXLAN, the administrator needs to create a multicast tunnel to this VXLAN for the appliance.

show vxlan mode

This command is used to display the working mode of the VXLAN function.

vxlan learn <on|off>

This command is used to enable or disable the VXLAN learning function. After this function is enabled, the appliance will automatically learn the mapping between the VTEP IP addresses and the MAC addresses. Please note that this function can be disabled only when all the mapping entries between the VTEP IP addresses and MAC addresses are manually configured. Otherwise, there will be a large number of flooding packets in the network. By default, this function is enabled.

on|off This parameter specifies whether to enable the VXLAN learning function. Its value must be “on” or “off”.



Note: The learned forwarding entries will expire and be deleted in 20 minutes.

show vxlan learn

This command is used to display the status of the VXLAN learning function.

vxlan port <vxlan_port>

This command is used to set the destination port number for the VXLAN tunnel. If this command is not configured, the default destination port number is 4789.

vxlan_port This parameter specifies the destination port number for the VXLAN Tunnel. Its value must be an integer ranging from 1025 to 65,000.

show vxlan port

This command is used to display the destination port number for the VXLAN tunnel.

vxlan interface <vxlan_ifname> <vni>

This command is used to create a VXLAN interface and associate it with a VXLAN network. The VXLAN interface supports setting the IP address, IP pool and WebWall. A maximum of 32,768 VXLAN interfaces can be configured.

Each created VXLAN interface will be the local VTEP of this VXLAN network.

<code>vxlان_ifname</code>	This parameter specifies the name of the VXLAN interface. Its value must be a case-insensitive string with a length not greater than 32 characters. Letters, numbers, and special characters are supported. Supported special characters: the at sign (@), exclamation mark (!), tilde (~), underscore (_), colon (:), hyphen (-), and period (.).
<code>vni</code>	This parameter specifies the VNI number used to identify a VXLAN network. Its value must be an integer ranging from 1 to 32,768.



Note:

- The VXLAN interface does not support setting MTU.
- In the HA environment, the configuration of the VXLAN interface can be synchronized to the peer HA unit.

no vxlan interface <vxlan_ifname>

This command is used to delete the specified VXLAN interface.

show vxlan interface

This command is used to display all VXLAN interfaces.

clear vxlan interface

This command is used to clear all VXLAN interfaces.

vxlan tunnel <tunnel_name> <local_ip> <remote_ip>

This command is used to create a VXLAN tunnel. The appliance supports two types of VXLAN tunnels: unicast and multicast. A maximum of 64 VXLAN unicast and multicast VXLAN tunnels can be created.

When the VXLAN working mode is p2mp, for each VXLAN network, the administrator needs to create a unicast tunnel to each VTEP, and bind them to the corresponding VXLAN interface.

When the VXLAN working mode is multicast, the administrator needs to create a multicast tunnel to this VXLAN network and bind this tunnel to the corresponding VXLAN interface.

<code>tunnel_name</code>	This parameter specifies the name of the VXLAN tunnel. Its value must be a string of 1 to 32 characters.
<code>local_ip</code>	This parameter specifies the local VTEP's IP address. Its value must be an IPv4 or IPv6 address. The value should be any IP address configured on the appliance, including

the cluster virtual IP address or HA floating IP address. When the local VTEP IP address is a cluster virtual IP address or an HA floating IP address, the configurations of tunnel can be synchronized to the peer node.

remote_ip This parameter specifies the remote VTEP's IP address. Its value must be an IPv4 or IPv6 address. For a unicast tunnel, the parameter value should be set to a unicast IP address; for multicast tunnels, the parameter value should be set to a multicast IP address.

Note: It is recommended to specify different multicast group addresses for different multicast tunnels.

no vxlan tunnel < tunnel_name >

This command is used to delete the specified VXLAN tunnel.

show vxlan tunnel [tunnel_name]

This command is used to display the configuration of the specified VXLAN tunnel. If the parameter “tunnel_name” is not specified, all configurations of VXLAN tunnels will be displayed.

clear vxlan tunnel

This command is used to clear all configurations of VXLAN tunnels.

vxlan bind < vxlan_ifname > < tunnel_name >

This command is used to bind a VXLAN interface and a VXLAN tunnel. A VXLAN interface can be bound to a maximum of 12 unicast VXLAN tunnels (p2mp working mode) or only one multicast VXLAN tunnel (multicast working mode).

vxlan_ifname This parameter specifies the name of the existing VXLAN interface.

tunnel_name This parameter specifies the name of the existing VXLAN tunnel.

no vxlan bind < vxlan_ifname > < tunnel_name >

This command is used to delete the binding between the VXLAN interface and the VXLAN tunnel.

show vxlan bind [vxlan_ifname]

This command is used to display the tunnel binding configurations of the specified VXLAN interface. If the parameter “vxlan_ifname” is not specified, the tunnel binding configurations of all VXLAN interfaces will be displayed.

clear vxlan bind [vxlan_ifname]

This command is used to clear the tunnel binding configurations of the specified VXLAN interface. If the “vxlan_ifname” parameter is not specified, the tunnel binding configurations of all VXLAN interfaces will be cleared.

vxlan associate <vxlan_ifname> <vlan_ifname>

This command is used to associate a VXLAN interface with a VLAN interface. This command only needs to be configured in the scenario where the appliance functions as a VXLAN L2 gateway. A VXLAN interface can be associated with only one VLAN interface.

vxlan_ifname	This parameter specifies the name of the VXLAN interface.
vlan_ifname	This parameter specifies the name of the VLAN interface.

no vxlan associate <vxlan_ifname> <vlan_ifname>

This command is used to delete the association between the VXLAN interface and the VLAN interface.

show vxlan associate [vxlan_ifname]

This command is used to display the association with the VLAN interface for the specified VXLAN interface. If the “vxlan_ifname” parameter is not specified, the associations with VLAN interfaces for all VXLAN interfaces will be displayed.

clear vxlan associate

This command is used to clear the associations with VLAN interfaces for all VXLAN interfaces.

vxlan forwarding remote <vni> <mac_address> <vtep_ip>

This command is used to add a static VXLAN forwarding entry to the remote VTEP for the specified VXLAN network.

vni	This parameter specifies the VNI number used to identify a VXLAN network.
mac_address	This parameter specifies the destination MAC address.
vtep_ip	This parameter specifies the remote VTEP’s IP address.



Note: The system supports configuring 2000 static VXLAN forwarding entries for each VXLAN network (including forwarding entries of the remote VTEP and L2 interface).

no vxlan forwarding remote <vni> <mac_address>

This command is used to delete a static VXLAN forwarding entry to the remote VTEP for the specified VXLAN network.

vxlan forwarding local <vni> <mac_address>

This command is used to add a static VXLAN forwarding entry to the L2 interface for the specified VXLAN network.

vni This parameter specifies the VNI number used to identify a VXLAN network.

mac_address This parameter specifies the destination MAC address.



Note: The system supports configuring 2000 static VXLAN forwarding entries for each VXLAN network (including forwarding entries of the remote VTEP and L2 interface).

no vxlan forwarding local <vni> <mac_address>

This command is used to delete a static VXLAN forwarding entry to the L2 interface for the specified VXLAN network.



Note: After the association between the VXLAN interface and the VLAN interface is deleted, the static VXLAN forwarding table of the L2 interface should be manually deleted.

show vxlan forwarding <vni> [mac_address]

This command is used to display all VXLAN forwarding entries for the specified VXLAN network.

vni This parameter specifies the VNI number used to identify a VXLAN network.

mac_address Optional. This parameter specifies the MAC address. If this parameter is specified, the VXLAN forwarding entries associated with this specified MAC address will be displayed. The default value is empty, indicating all VXLAN forwarding entries will be displayed.

clear vxlan forwarding dynamic <vni>

This command is used to clear all dynamic VXLAN forwarding entries for the specified VXLAN network.

vni This parameter specifies the VNI number used to identify a

VXLAN network.

clear vxlan forwarding static <vni>

This command is used to clear all static VXLAN forwarding entries for the specified VXLAN network.

vni	This parameter specifies the VNI number used to identify a VXLAN network.
-----	---

show vxlan all

This command is used to display all configurations of the VXLAN function.

clear vxlan all

This command is used to clear all configurations of the VXLAN function.

Chapter 4 Link Aggregation

The Link Aggregation configuration commands are designed for the users to set the vital parameters to use this new functionality.

bond name <*bond_id*> <*bond_name*>

This command is used to configure a name for the specified bond interface. The system supports at most eight bond interfaces.

bond_id	This parameter specifies the default bond interface ID (bond1, bond2, bond3, bond4 ...) for the bond interfaces on the APV appliance.
bond_name	This parameter specifies a bond interface name. The value must be a case-insensitive string with a length not greater than 32 characters. Letters, numbers, and special characters are supported. The default bond interface name is bond1, bond2, bond3, bond4, etc. Supported special characters: the at sign (@), exclamation mark (!), tilde (~), underscore (_), colon (:), hyphen (-), and period (.).

bond interface <*bond_name*> <*interface_name*> [1|0]

This command allows users to add a system interface to the specified bond interface. At most 12 system interfaces can be added to a bond interface.

The parameter “1|0” can be used to set the interface as one of the primary (1) or backup (0) interfaces in the bond. Multiple primary or backup interfaces can be set in the bond. When all the primary interfaces in the bond fail, the backup interfaces will take the place of primary interfaces to work.

bond_name	A bond interface name specified by an alphanumeric string. The default bond interface name is bond1, bond2, bond3, bond4, etc.
interface_name	A network interface name specified by an alphanumeric string. The default system interface name is port1, port2, port3, port4, etc. The interface can be set by using the command “ interface name ”.
1 0	1: Sets the interface as one of the primary interfaces in the bond. By default, 1 applies. 0: Sets the interface as one of the backup interfaces in the bond.

no bond interface <bond_name> <interface_name>

This command allows users to remove the system interface from the bond interface.

show bond [bond_name]

This command is used to display all current system bond interfaces' information. If the bond interface name is specified, the command will only show the specified interface's information.

clear bond [bond_name]

This command is used to reset the bond interface's configurations to default. If no bond interface name is specified, all the bond interfaces' configurations are removed.

bond health <bond_name> <destination_ip> [interval] [timeout]
[up_check_times] [down_check_times] [gateway_ip]

This command is used to configure and enable the health check for the bond interface. The APV appliance will send an ICMP echo request to the destination IP address through every subinterface of the bond interface, and check the corresponding ICMP echo reply. If the times of the consecutively received ICMP echo response reach the value specified by the parameter "up_check_times", the subinterface will be marked as "up"; if the times of the ICMP echo response consecutively failed to be received reach the value specified by the parameter "down_check_times", the subinterface will be marked as "down". By default, the health check is disabled.

bond_name	Specify the bond interface name. It is an alphanumeric string up to 32 characters. The default bond interface name is bond1, bond2, bond3, bond4, etc.
destination_ip	Specify the destination IP address. It can be an IPv4 or IPv6 address.
interval	Optional. Specify the interval of ICMP echo requests. The value of this parameter ranges from 1 to 65535. The default value is 5, in seconds.
timeout	Optional. Specify the time interval between the ICMP echo request and the ICMP echo response. If the corresponding ICMP echo response is not received within the specified time interval, the health check fails. The value of this parameter ranges from 1 to 65535 The default value is 3, in seconds.
up_check_times	Optional. Specify the times of the consecutively received ICMP echo response. If the times of the consecutively received ICMP echo response reach the value specified by the parameter "up_check_times", the bond interface will be marked as "up". The value of this parameter ranges from 1

to 65535. The default value is 3.

down_check_times Optional. Specify the times of the ICMP echo response consecutively failed to be received. If the times of the ICMP echo response consecutively failed to be received reach the value specified by the parameter “down_check_times”, the bond interface will be marked as “down”. The value of this parameter ranges from 1 to 65535. The default value is 3.

gateway_ip Optional. Specify the gateway IP address. It can be an IPv4 and IPv6 address. For the IPv4 address, the default value is 0.0.0.0; for the IPv6 address, the default value is ::. **Note:** If the parameter “destination_ip” is not in the same network segment as the bond interface, the parameter “gateway_ip” must be configured.

no bond health <bond_name>

This command is used to delete the health check for the specified bond interface.

Chapter 5 Clustering

This chapter covers the commands for configuring the Clustering function.

show cluster virtual status [*interface_name*]

The command is used to output the status of the cluster feature for the APV appliance (either on or off), followed by the state of each configured virtual cluster (either in incomplete, initialize, backup, or master state), and the name and link status of the interfaces specified for each virtual cluster.

If an interface name is specified, the system will only display the cluster status information about this interface.

interface_name Specify the interface name, which can be the system interface, bond interface, VLAN interface or MNET interface.

Example:

```
AN(config)#show cluster virtual status
```

```
ffo cable status: remote no power
```

```
discreet mode enabled
```

```
ifname= port1
```

```
vcid      off/on  vc state
```

```
1         on     master
```

```
ifname= port2
```

```
vcid      off/on  vc state
```

```
1         on     master
```

```
ifname= mnet1
```

```
vcid      off/on  vc state
```

```
1         on     master
```

cluster virtual {on|off} [*cluster_id*0] [*interface_name*]

This command is used to enable or disable the virtual clustering capabilities for the APV appliance. The minimum value of a virtual cluster ID is 1 and the maximum decimal value is 255. It defaults to 0, which means all clusters will be enabled. Also with this command, users must specify the appropriate interface name. If no cluster ID or interface name is supplied, all clusters will be enabled.

cluster virtual ffo {on|off}

This command is used to enable or disable the Fast Failover (FFO) feature. The default value is off.

cluster virtual ffo interface carrier loss timeout <*interface_timeout*>

This command is used to configure how long an APV appliance waits before failover (if necessary) when it detects interface carrier loss (in milliseconds). If network carrier recovers in the timeout value, no action will be taken. This timeout value ranges from 0 to 65535, in milliseconds. 0 means no wait while 65535 means no failover.

system test failover port [on/off]

This command allows you to test the status of the FFO port on the APV appliance. The APV appliance provides the FFO port via USB.



Note: Before using this command to test the FFO port, first make sure that you have disabled the Fast Failover function by executing the “**cluster virtual ffo off**” command.

Administrators can perform the following steps to test the status of the FFO port:

1. Insert one end of the USB FFO cable into the USB-type FFO port.
2. Install the USB driver on your PC to make the APV appliance known to the PC.
3. On the PC, start any terminal software and connect the USB-COM port.
4. Execute “**system test failover port on**” on the terminal software to enable the testing mode.
5. Press any keys on the keyboard to see whether the entered characters are displayed on the terminal software.

If yes: The USB-type FFO port is normal.

If no: The USB-type FFO port is abnormal.

6. Execute “**system test failover port off**” on the terminal software to disable the testing mode.

show cluster virtual config [interface_name]

This command is used to display the current virtual cluster configuration or the virtual cluster configuration of all the interfaces. If an interface name is specified, the system will only display the cluster status information about this interface.

interface_name	Specify the interface name, which can be the system interface, bond interface, VLAN interface or MNET interface. The default value is all.
----------------	--

Example:

```
AN(config)#show cluster virtual config port2
cluster virtual ifname "port2" 1
cluster virtual vip "port2" 1 10.30.0.30
cluster virtual auth "port2" 1 1 "myString"
cluster virtual interval "port2" 1 10
```

```
cluster virtual preempt "port2" 1 0
cluster virtual priority "port2" 1 200
```

show cluster virtual ffo

This command is used to display the current fast failover configurations.

cluster virtual ifname <interface_name> <cluster_id>

This command is used to define a virtual cluster ID for specific interface.

interface_name	Specify the interface name, which can be the system interface, bond interface, VLAN interface or MNET interface.
cluster_id	A virtual cluster ID where the minimum decimal value is 1 and the maximum decimal value is 255.



Note:

- As too many Virtual Cluster IDs (VCID) might cause unnecessary system overload, it is suggested not to configure too many VCIDs in the system. If many virtual IP addresses are needed, administrators can configure multiple IP addresses within one VCID, instead of configuring one VCID for each IP address.
- If only an IPv6 address has been configured for the specified interface on the local node and an IPv6 address has been configured for the specified interface on the peer node, the IPv6 link will be used to send VRRP packets between the two nodes. If both IPv4 and IPv6 addresses have been configured for the specified interfaces on both nodes, the IPv4 link will be used to send VRRP packets between the two nodes.

show cluster virtual interface

This command allows users to view declared interface names configured via the “cluster virtual ifname” command.

clear cluster virtual ifname <interface_name> <cluster_id>

This command is used to remove a virtual cluster from the specified interface.

interface_name	This parameter specifies the interface name. Its value must be the name of the system interface, bond interface, VLAN interface, MNET interface or “all” (all interfaces).
cluster_id	This parameter specifies the virtual cluster ID. Its value must be an integer ranging from 0 to 255. The value 0 means all virtual clusters.

cluster virtual vip <interface_name> <cluster_id> <vip>

This command is used to set the virtual IP address for a virtual cluster on specified interface.

interface_name	Specify the interface name, which can be the system interface, bond interface, VLAN interface or MNET interface.
cluster_id	A virtual cluster ID where the minimum decimal value is 1 and the maximum decimal value is 255. A cluster ID can have up to 255 virtual IP addresses. The same virtual IDs, located on different interfaces, are treated as different virtual IDs. All the virtual IP addresses with the same virtual ID will have the same status (master or backup).
vip	A valid virtual IP address barring reserved IP addresses such as loop back, multicast, and other commonly known specialized ranges. Each virtual IP address entered must be unique and can be IPv4 or IPv6 address.



Note: When a service is provided via multiple virtual IP addresses, all virtual IP addresses must be bound to the same VCID, regardless of whether they use the same interface. Otherwise, the service will be divided into different virtual clusters, which might have different status.

cluster virtual auth <interface_name> <cluster_id> {0|1} [password]

This command is used to enable and configure virtual cluster authentication.

interface_name	Specify the interface name, which can be the system interface, bond interface, VLAN interface or MNET interface.
cluster_id	Specify a virtual cluster ID where the minimum decimal value is 1 and the maximum decimal value is 255.
0 1	Specify the authentication type. “0” indicates that no password will be used. “1” indicates the password field in plain text will be used.
password	When the authentication type is “1”, the parameter need to be specified. The password consists of up to eight alphanumeric characters. If the password string contains only numerals, it must be enclosed in double quotes.

cluster virtual preempt <interface_name> <cluster_id> <mode>

This command is used to configure virtual cluster preemption. (Note: Exception is a cluster that has been configured with a priority 255.)

interface_name	Specify the interface name, which can be the system interface, bond interface, VLAN interface or MNET interface.
cluster_id	The assigned identification number for the virtual cluster.
mode	Specify the virtual cluster preemption. It can be set to 1 or 0 and defaults to 1. The value “1” allows preemption of a higher priority master, while the value “0” prohibits preemption of a higher priority master.

cluster virtual interval <interface_name> <cluster_id> <seconds>

This command is used to set the advertisement interval for the specified cluster.

interface_name	Specify the interface name, which can be the system interface, bond interface, VLAN interface or MNET interface.
cluster_id	Specify the assigned identification number for the virtual cluster.
seconds	Specify the advertisement interval in seconds. Its value should be an integer ranging from 3 to 255 and defaults to 5. Any state transition of the virtual cluster will take approximately 2 to 3 times of the advertisement interval.



Note: The same advertisement interval should be set to the same virtual cluster ID that is configured on the same interface name of the Master and Backup nodes. Otherwise, the state of both the two nodes will become “MAST”, which will lead to cluster failure.

cluster virtual priority <interface_name> <cluster_id> <priority> [synconfig_peer_name]

This command is used to set the virtual cluster priority. If this command is not configured, the priority of the virtual cluster is 100. Each interface supports priority definition for a maximum of 64 peers.

interface_name	Specify the interface name, which can be the system interface, bond interface, VLAN interface or MNET interface.
cluster_id	Specify the identification number for the virtual cluster.
priority	Specify the priority for the virtual cluster. The greater the value, the higher the priority. The value ranges from 1 to

255.

synconfig_peer_name Optional. Specify the name of local node or peer node. The parameter value can be “Primary” or any peer node defined by the command “**synconfig peer** <peer_name> <peer_ip>”. When the parameter value is set to “Primary”, the command applies to the local node. When the parameter value is set to the name defined by the command “**synconfig peer** <peer_name> <peer_ip>”, the command applies to the corresponding node. The default value is “Primary”.



Note: To synchronize the virtual cluster priority to the peer node, note that the virtual cluster priority of both the local node and peer node defined by the command “**synconfig peer** <peer_name> <peer_ip>” should be configured on the local node.

no cluster virtual vip <interface_name> <cluster_id> <vip>

This command is used to remove the VIP from the specified cluster ID and interface name.



Note:

- The state of the Master or Backup node will switch to “INIT” immediately after the VIP of a virtual cluster on the Master or Backup node is deleted. In the meantime, the Backup node in the cluster will take over the Master node to be the new Master.
- If the cluster virtual configurations on the Master or Backup nodes are different, the state of both the two nodes may become “MAST”, which will lead to IP address conflict. Therefore, it is strongly suggested to disable the clustering function on the Backup node before changing the configurations on the Master node, and then synchronize the Backup node’s configurations with the Backup node after changing configurations is finished on the Master node.

no cluster virtual auth <interface_name> <cluster_id>

This command is used to reset cluster authentication to default setting. That is, cluster authentication is disabled.

no cluster virtual interval <interface_name> <cluster_id>

This command is used to reset advertisement interval to default (5 seconds).

no cluster virtual preempt <interface_name> <cluster_id>

This command is used to reset cluster preemption mode to default setting. That is, the preemption mode is enabled.

no cluster virtual priority <interface_name> <cluster_id>
[synconfig_peer_name]

This command is used to reset the cluster priority to the default value (100) or delete the cluster priority information of the cluster specified by the parameter “synconfig_peer_name” from the local node.

interface_name	Specify the interface name, which can be the system interface, bond interface, VLAN interface or MNET interface.
cluster_id	Specify the assigned identification number for the virtual cluster.
synconfig_peer_name	Optional. The default value is “Primary”. Except for the default value (“Primary”), this value can be any synconfig peer defined by the command “ synconfig peer <peer_name> <peer_ip>”. When the value is set to “Primary” or the name of the synconfig peer of the local node, this command restores the cluster priority of the local node to the default value (100). When the value is set to the name of other synconfig peers, this command only deletes the cluster priority information of the cluster from the local node.

cluster virtual discreet {on|off}

This command is used to enable/disable the discreet backup mode. In this mode, the system determines whether a status transition is needed for the devices based on their status information detected through a heartbeat cable. This mode makes the status transition more reliable, and any VRRP packet loss will not result in double-master status. This mode is “off” by default.



Note: In discreet backup mode, the system utilizes the heartbeat cable to collect the status information. Please make sure the heartbeat cable between the devices is well connected and turned on by “**cluster virtual ffo on**” command firstly (Heartbeat cable and FFO cable are one cable).

show cluster virtual discreet

This command is used to display the discreet backup mode configuration.

show cluster virtual transition [interface_name]

This command is used to display the last 100 cluster state transition logs on the specified interface. If no interface name is given, it will display the last 100 cluster state transition logs on all the interfaces. Cluster states include Initial (INIT), Backup (BACK), Discreet Backup (DISCREET), FFO and Master (MAST).

interface_name Specify the interface name, which can be the system interface, bond interface, VLAN interface or MNET interface. The default value is all.

Example:

```
AN(config)#show cluster virtual transition
ifname = port1, vcid = 1
Sep 25 17:36:22 (+0000) [BACK -> MAST] Timeout.
Sep 25 17:36:17 (+0000) [DISCREET -> BACK] Receive a VRRP advertisement of priority 0.
Sep 25 17:34:58 (+0000) [BACK -> DISCREET] Entering discreet mode.
Sep 25 17:34:58 (+0000) [FFO -> BACK] FFO cable is OK. Cluster is ready to work.
Sep 25 17:34:58 (+0000) [INIT -> FFO] FFO is enabled.
Sep 25 17:34:56 (+0000) [BACK -> INIT] Stop running.
Sep 25 17:34:56 (+0000) [FFO -> BACK] FFO cable is OK. Cluster is ready to work.
Sep 25 17:34:56 (+0000) [INIT -> FFO] FFO is enabled.
```

clear cluster virtual transition [*interface_name*] [*cluster_id*]

This command is used to remove cluster state transition logs on the specified interface of either the specified virtual cluster or all virtual clusters. By default, “interface_name” parameter is set to all, which means removing cluster state transition logs on all interfaces. By default, “cluster_id” parameter is set to 0, which means removing cluster state transition logs on all virtual clusters.

show statistics cluster virtual [*interface*]

This command is used to display the virtual clustering statistics information on the specified interface. If no interface name is given, it will display the virtual clustering statistics information on all the interfaces.

Example:

```
AN(config)#show statistics cluster virtual
ifname = port1, vcid = 1
transition to master: 1
(Switch to master, gain VIPs)
quit master: 0
(Leave master, release VIPs)
VRRP loss: 0
(Possible VRRP loss - receive none VRRP advertisements from master for two intervals while in backup state, but receive a valid VRRP advertisement before timeout (three intervals))
quick transition: 2
(Received VRRP advertisements of priority 0 -used for quick transition)
inconsistency: 0
(Detect inconsistent state with other interfaces of the same VCID, drop remote VRRP packets with lower priority)
```



Note: The above contents in the brackets are explanations for output information.

clear statistics cluster virtual [*interface_name*] [*cluster_id*]

This command is used to remove the cluster statistics on the specified interface of either the specified virtual cluster or all virtual clusters. By default, “interface_name” is set to all, which means removing the cluster statistics on all interfaces. By default, “cluster_id” parameter is set to 0, which means removing the cluster statistics information on all virtual clusters.

cluster virtual arp interval [seconds]

This command is used to set the interval of masters broadcasting gratuitous ARP.

seconds	It can be 0, or any integer from 30 to 65535, in seconds. It defaults to 60 seconds. 0 means that devices only broadcast gratuitous ARP when switching to Master state.
---------	---

Chapter 6 High Availability (HA)

This chapter covers the commands for configuring the HA function.



Note: The HA and Clustering functions are mutually exclusive. Their configurations cannot coexist.

HA Unit

ha unit <unit_name> <ip_address> [port]

This command is used to add an HA unit for the HA domain. An HA domain allows at most 32 units. After adding multiple units for an HA domain, the system will establish primary link connections between each two units automatically.

unit_name This parameter specifies the name of the HA unit. Its value must be a case-sensitive string of 1 to 8 characters. The name of each unit should be unique in an HA domain.

ip_address This parameter specifies the IP address of the HA unit, which is used for primary link communication with other units. Its value must be an IPv4 or IPv6 address. The IP addresses of the units in an HA domain must be all IPv4 or all IPv6.

port Optional. This parameter specifies the port used for primary link communication with other units.

The default value is 65521.



Note:

- Before configuring the local unit, you must have configured the local unit's interface IP address. Otherwise, the local unit cannot be identified by the HA domain.
- The port of the HA unit should not be the same as that of the SDNS data center (configured using the "**sdns dc**" command). Otherwise, the status of the HA unit will become abnormal.

no ha unit <unit_name>

This command is used to delete a configured HA unit.



Note: For an HA unit that has been referenced in the "**segment ip address**" command, it is recommended to cancel the reference by using the "**no segment ip address**" command before delete it.

show ha unit

This command is used to display all configured HA units.

clear ha unit

This command is used to clear all configured HA units.

HA Links**ha link network secondary <unit_name> <link_id> <ip_address> [port]**

This command is used to configure a secondary link on an HA unit. At most 31 secondary links can be established between two HA units.

unit_name	This parameter specifies the name of the HA unit.
link_id	This parameter specifies the ID of the secondary link. Its value must be an integer ranging from 1 to 31. The ID of each secondary link between two units should be unique.
ip_address	This parameter specifies the IP address of the HA unit, which is used for secondary link communication with other units. Its value must be an IPv4 or IPv6 address.
port	Optional. This parameter specifies the port used for secondary link communication with other units. The default value is 65521.



Note: Please be noted that to establish a secondary link between two units, you need to configure a secondary link with the same ID on the two units respectively.

For example, the IP address of two HA units “u1” and “u2” are 192.168.1.1 and 192.168.1.2 respectively. To establish a secondary link between the two units, the following two commands must be executed on both units:

```
AN(config)#ha link network secondary u1 1 192.168.1.1 65521
AN(config)#ha link network secondary u2 1 192.168.1.2 65521
```

After the above configurations are finished on both units, a secondary link with ID “1” is established between “u1” and “u2”.



Note: The IP addresses of the two ends of a secondary link must be both IPv4 or both IPv6 addresses.

no ha link network secondary <unit_name> <link_id>

This command is used to delete a secondary link between two HA units.

clear ha link network secondary

This command is used to delete the configurations about all secondary links on the local unit.

ha link network {on|off}

This command is used to enable or disable the network links established between the local unit and other peer units, including the primary link and the secondary link(s). By default, the network links are enabled.

ha link ffo {on|off}

This command is used to enable or disable the FFO link for the HA function. The FFO link can only be applied to the HA domain with two units. By default, the FFO link is disabled.



Note: The FFO link must be used together with network links. If network links are disabled, the FFO link and the HA function will not take effect.

ha link ffo heartbeat {on|off}

This command is used to enable or disable the FFO link heartbeat for the HA function. This function can take effect only when the FFO link function is enabled. By default, this function is disabled.

show ha link

This command is used to display the status of the network links and FFO link, and the configurations of secondary links.

Floating IP Group and Floating MAC

ha group id <group_id>

This command is used to define a floating IP group for the local unit. A maximum of 1024 groups can be defined for each unit, including both system floating IP groups and segment floating IP groups.

group_id	This parameter specifies the ID of the floating IP group. Its value must be an integer ranging from 0 to 1023.
----------	--

no ha group id <group_id>

This command is used to delete the specified floating IP group from the local unit.

show ha group id [group_id]

This command is used to display all configurations related to the specified floating IP group on the local HA unit. If the “group_id” parameter is not specified, the configurations related to all floating IP groups will be displayed.

clear ha group id

This command is used to delete all the floating IP groups defined for the local unit.

ha group fip <group_id> <floating_ip> [interface_name] [floating_mac]

This command is used to configure a floating IP address for the specified floating IP group, associate the IP address with the specified interface and configure the floating MAC function. The total number of floating IP addresses and floating IP ranges configured for a floating IP group cannot exceed 16.

group_id This parameter specifies the ID of the floating IP group. Its value must be an integer ranging from 0 to 1023.

floating_ip This parameter specifies the floating IP address. Its value must be an IPv4 or IPv6 address.

interface_name Optional. This parameter specifies the interface to be bound to the floating IP address. Its value must be the name of a system interface, bond interface, MNET interface, VLAN interface or “auto”. The value “auto” means that an interface will be automatically selected. Note: It is not recommended to set this parameter to the interface of the primary link. When a floating MAC address needs to be set, this parameter cannot be set to the member interface of a bond interface. When a floating MAC address needs to be set, this parameter cannot be set to the member interface of a bond interface.

The default value is “auto”.

When you want to configure the “floating_mac” parameter and meanwhile use the default value for this parameter, “auto” must be specified. If the “floating_mac” parameter will not be configured, it is allowed to not set this parameter to indicate that the default value will be used.

floating_mac Optional. This parameter specifies the floating MAC address, in the format “xx:xx:xx:xx:xx:xx”. Its value cannot be the same as any of the following MAC addresses: system interface MAC address; MAC address bound to other system or bond interface in the same floating IP group; MAC address bound to other floating IP groups; MAC address of system interface added to a bridge.

The default value is empty, indicating that the floating

MAC function will not be used.

no ha group fip <group_id> <floating_ip>

This command is used to delete a floating IP address of the specified floating IP group.

clear ha group fip <group_id>

This command is used to delete all floating IP addresses of the specified floating IP group.

ha group fiprange <group_id> <start_floating_ip> <end_floating_ip> [interface_name] [floating_mac]

This command is used to configure a floating IP range for the specified floating IP group, associate the IP range with the specified interface and configure the floating MAC function. Each floating IP range contains utmost 256 IP addresses. The total number of floating IP addresses and floating IP ranges configured for a floating IP group cannot exceed 16.

group_id This parameter specifies the ID of the floating IP group. Its value must be an integer ranging from 0 to 1023.

start_floating_ip This parameter specifies the start IP address of the floating IP range. Its value must be an IPv4 or IPv6 address.

end_floating_ip This parameter specifies the end IP address of the floating IP range. Its value must be an IPv4 or IPv6 address.

interface_name Optional. This parameter specifies the interface to be bound to the floating IP address. Its value must be the name of a system interface, bond interface, MNET interface, VLAN interface or “auto”. The value “auto” means that an interface will be automatically selected. Note: It is not recommended to set this parameter to the interface of the primary link. When a floating MAC address needs to be set, this parameter cannot be set to the member interface of a bond interface.

The default value is “auto”.

When you want to configure the “floating_mac” parameter and meanwhile use the default value for this parameter, “auto” must be specified. If the “floating_mac” parameter will not be configured, it is allowed to not set this parameter to indicate that the default value will be used.

floating_mac Optional. This parameter specifies the floating MAC address, in the format “xx:xx:xx:xx:xx:xx”. Its value

cannot be the same as any of the following MAC addresses: system interface MAC address; MAC address bound to other system or bond interface in the same floating IP group; MAC address bound to other floating IP groups; MAC address of system interface added to a bridge.

The default value is empty, indicating that the floating MAC function will not be used.

The maximum number of floating IP addresses allowed on the appliance varies with system memories. Please see the following table for details.

System Memory	Maximum Floating IP Addresses
<= 8 GB	2000
16 GB	4000
24 GB	4000
>= 32 GB	8000



Note: All the IP addresses in the floating IP range, including the start IP and the end IP, cannot be those assigned to specific interfaces by the command “**ip address**”.

no ha group fiprange <group_id> <start_floating_ip> <end_floating_ip>

This command is used to delete a floating IP range of the specified floating IP group.

clear ha group fiprange <group_id>

This command is used to delete all floating IP ranges of the specified floating IP group.

ha floatmac {on|off}

This command is used to enable or disable the floating MAC function for HA. With this function enabled, the floating MAC address is switched to the interface of the unit on which the group status is “Active”. In this way, after group status switches, the clients will not be aware that the device that provides the application services has been changed, because the MAC addresses of the appliances that provide application services before and after group status switch remain unchanged.

By default, the floating MAC function is disabled. Before enabling this function, the HA function must be first disabled by executing the command “**ha off**”.

ha arp interval <interval>

This command is used to set the interval of sending ARP broadcast packets for the local unit.

interval

This parameter specifies the interval of sending ARP broadcast packets, in seconds. Its value must be set to 0 or

an integer ranging from 30 to 65535. If it is set to 0, it means the ARP broadcast packets will be sent only when the group status of the HA unit is switched to “Active”.

The default value is 30.

ha group priority <unit_name|ssi> <group_id> <priority>

This command is used to configure or change the priority of a specified floating IP group on the specified HA unit.

unit_name|ssi This parameter specifies the HA unit name. Its value must be the name of the local or peer HA unit, indicating that the priority configured for the floating IP group will be used for the HA function.

The “ssi” parameter applies only to the SSI function. For details, please see descriptions of this command in Chapter 7 Single System Image (SSI).

group_id This parameter specifies the ID of the floating IP group.

priority This parameter specifies the priority of a specified floating IP group on the specified unit. Its value must be an integer ranging from 0 to 255. The larger the value, the higher the priority.



Note: If the priority of a floating IP group is not specified on a unit or is deleted, the group will not take effect on the unit (the group status on this unit displayed by the “**show ha status**” command will always be “-”).

no ha group priority <unit_name|ssi> <group_id>

This command is used to remove the priority of a specified floating IP group on the specified HA unit, or all units in the SSI domain.

ha group preempt {on|off} <group_id>

This command is used to enable or disable the preempt mode for a specified floating IP group. With the preempt mode enabled, the status of a floating IP group on the available unit with the highest group priority will be always kept as “Active”. By default, the preempt mode is disabled for the floating IP group.

group_id This parameter specifies the ID of the floating IP group. Its value must be an integer ranging from 0 to 1024. “1024” means enabling or disabling the preempt mode for all floating IP groups.



Note: To ensure the preempt mode takes effect for the specified floating IP group, make sure that the preempt mode has been enabled on all units on which this group is enabled.

ha group {enable|disable} <group_id>

This command is used to enable or disable the specified floating IP group on the local unit. By default, all floating IP groups are disabled. If a floating IP group is enabled on multiple units, only the unit whose group status is “Active” will provide services in that group. The floating IP groups can be associated with the application services by executing the command “**ha group fip**” and “**ha group fiprange**”.

group_id	This parameter specifies the ID of the floating IP group. Its value must be an integer ranging from 0 to 1024. “1024” means enabling or disabling all the floating IP groups on the local unit.
----------	---

ha heartbeat [interval] [down_check_times]

This command is used to set the interval of sending heartbeat packets of the local unit to the peer units through the primary link, secondary link(s) and FFO link. If no heartbeat response has been received from the peer unit on any of the links for consecutive times (specified by “down_check_times”), the status of the peer unit will be marked as “Down”. Otherwise, the status of the peer unit will be marked as “Up”.

interval	Optional. This parameter specifies the interval of sending the heartbeat packets, in milliseconds (ms). The value of this parameter ranges from 100 to 10,000.
----------	--

The default value is 1000.

down_check_times	Optional. This parameter specifies the number of consecutive times that have not received heartbeat response from the peer unit. The value of this parameter ranges from 3 to 1000.
------------------	---

The default value is 15.

Once any error occurs to the peer unit, its status in a floating IP group will be switched from “Active” to “Init”. Meanwhile, the unit which is available and with the highest priority in the floating IP group will be switched from “Standby” to “Active” to start to provide services for clients in that group.



Note: Please make sure that the “**ha heartbeat**” settings are the same on HA units. Otherwise, frequent HA status switchovers may occur.

ha group switch <unit_name> <group_id>

This command is used to switch the specified floating IP group to the specified HA unit. After a switchover is performed, the state of this group will change from active to standby on the current unit, and from standby to active on the target unit. This command only takes effect on the groups with the preemption function (configured by the “ha group preempt” command) disabled.

unit_name	This parameter specifies the name of an existing HA unit.
group_id	This parameter specifies the ID of an existing floating IP group. If the value 1024 is set, all floating IP groups in the active state will be switched.

HA Health Check and Failover

HA supports three types of health checks:

- **Predefined health check:** HA has predefined network connectivity check mechanism to detect network interface faults and network disconnections between nodes. By default, when a network interface is faulty, the system will perform Group_Failover. Administrators can modify a failover action associated with the predefined health check condition.
- **System health check:** HA supports the use of the system health check as failover conditions. Administrator can define the following health check conditions for HA failover.
- **Self-defined health check script:** The system supports the import of self-defined health check scripts from an SCP or TFTP server as well as the export of the scripts for modification. Please see the “System Health Check” section of Chapter 2 for related configuration commands.

show monitor condition [*unit_name*]

This command is used to display the health status of a specified HA unit.

unit_name	This parameter specifies the name of the HA unit. If this parameter is not specified, the health status of all HA units will be displayed.
-----------	--



Note: The health check conditions in “Init” status will not be displayed by this command.

ha decision rule <condition_name> <action_name> <group_id>

This command is used to configure a failover rule for a specified floating IP group. The failover rule indicates the failover operation to be performed when the result of a specified health check changes from “Up” to “Down”. A health check condition can be used for configuring a maximum of 1024 failover rules.

condition_name	This parameter specifies the name of the health check condition. Its value must be the name of a real health check condition or a vcondition. The system supports the following values: • (Predefined) PORT_1~PORT_32: port health check conditions • GATEWAY_1~GATEWAY_32: gateway health check conditions • CPU_UTIL: CPU utilization health check condition • CPU_TEMP: CPU temperature health check condition • SSL_CARD: SSL card health check condition • SSL_AE_UTIL: SSL card AE core utilization health check condition • SSL_SE_UTIL: SSL card SE core utilization health check condition • SYSTEM_MEM_UTIL: system memory utilization health check • NETWORK_MEM_UTIL: network memory utilization health check • SWAP_MEM_UTIL: SWAP memory utilization health check • CONN_MEM_UTIL: connection memory utilization health check • SYSTEM_DISK_UTIL: system disk space utilization health check condition • DATA_DISK_UTIL: data disk space utilization health check condition • BOND_1~BOND_8: bond state health check condition • Names of customized health check condition groups (vconditions)
action_name	This parameter specifies the failover operation to be performed when the result of a specified health check is “Down”. Its value must be “Unit_Failover”, “Group_Failover” or “Reboot”.
group_id	This parameter specifies the ID of the floating IP group for which the failover rule takes effect. This parameter is available only when the parameter “action_name” is set to “Group_Failover”.

**Note:**

- For the units who provide failover support for the same floating IP group

in the HA domain, the failover rules of the floating IP group configured on the units should be exactly the same, including the name of the health check condition in failover rules.

- For the units who provide failover support for “Unit_Failover” in the HA domain, the failover rules of the “Unit_Failover” configured on the units should be exactly the same, including the name of the health check condition in failover rules.
- The system provides predefined failover rules. You can execute the command “**ha decision rule**” to modify “action_name” of these predefined rules. When the system interface bound to the floating IP address or floating IP range becomes down, the predefined failover rule for this interface will take effect in 1000~10,000 ms, which is the interval of sending the heartbeat packets to peer units.
- To achieve more flexible failover based on interface status, you can add the network port health check conditions (PORT_1~PORT_32) to vconditions and use vconditions as failover conditions. For example, by adding the two network port health check conditions to a vcondition and configuring the relationship between both conditions as “OR”, you can customize a rule to perform a Group_Failover only when both interfaces are down. After a network port health check condition is added to a vcondition, the corresponding predefined failover rule will not take effect.

no ha decision rule <condition_name> <action_name> <group_id>

This command is used to delete a failover rule of a specified floating IP group.



Note: If the parameter “condition_name” is set to a value from “PORT_1” to “PORT_32”, the system will reset “action_name” to “Group_Failover”.

clear ha decision rule

This command is used to delete the failover rules of all floating IP groups.



Note: Executing this command will not delete the predefined failover rules. However, “action_name” of the predefined rules will be reset to “Group_Failover”.

show ha decision

This command is used to display all the failover rules of all floating IP groups on the local unit, including both the predefined and customized rules.

For example:

AN(config)#show ha decision			
ID	Condition_Name	Action_Name	Group_ID
0	PORT_1	Group_Failover	-
1	PORT_2	Group_Failover	-
2	PORT_3	Group_Failover	-
3	PORT_4	Group_Failover	-

show ha failover [group_id]

This command is used to display the failover logs of the specified floating IP group. If the “group_id” parameter is not specified, the failover logs of all floating IP groups will be displayed.

Peer Unit HA Group Status Check**ha checkpeer {on|off}**

This command is used to enable or disable the peer unit HA group status check function.

After this function is enabled, when the health check result is Down on the active HA unit, it will check HA group status on the peer HA unit. If the HA group on the peer HA unit is also Down, it will stay in the active status instead of switching to Down.

After this function is disabled, when the health check result is Down on the active HA unit, it will not check the HA group status on the peer HA unit but directly switches to Down.

By default, this function is enabled.

clear ha checkpeer

This command is used to store the peer unit HA group status check function to the default setting. After this command is executed, this function will be enabled.

Configuration Synchronization**ha synconfig bootup {on|off}**

This command is used to enable or disable the bootup synconfig function. This function synchronizes configurations on the peer HA unit to the local HA unit. By default, this function is disabled.

The configuration items that will not be synchronized by this function is listed as follows:

```
ip address
ip route
ip dhcp
ssh ip
bond
hostname
mnet
vlan
access
webui ip
webwall
```

```

ip redundant
cluster virtual priority, with "synconfig_peer_name" being "primary"
llb dns local
sdns listener
sdns member local
sdns dps localdetector
interface
no interface
clear interface
interface name
interface speed
interface mtu
interface mac
interface management
ha ssf peer
user
admin mode separation on
admin mode separation off

```

**Note:**

- This function synchronizes HA unit and link configuration through the FFO link and the configurations saved to the memory by executing the “**write memory**” command through the primary network links.
- If the configuration synchronization secure mode is enabled using the command “**synconfig secure on**”, you must configure synchronization peers on both the peer and local HA units by using the “**synconfig peer**” command before enabling this function. The specified peer name must be the same as the HA unit name.
- After this function is enabled, all statistics related to SLB will be cleared from the local HA unit.
- During bootup synconfig, any addition, modification, or deletion of configurations is not allowed.
- In an environment where the HA function is enabled, administrators need to make sure the bootup synconfig function is disabled before executing any of the “**synconfig from**”, “**synconfig to**”, “**synconfig rollback local**”, “**synconfig rollback peer**”, “**config memory**”, “**config memory**” and “**config file**” commands. Otherwise, configuration abnormality might occur.

ha synconfig runtime {on|off}

This command is used to enable or disable the runtime synconfig function. Two units can synchronize configurations to each other in real time only when the runtime synconfig function is enabled on both units. By default, this function is disabled. For configuration items that do not need to be synchronized, the system lists them in a blacklist. After the runtime synconfig function is enabled, all configurations except those in the blacklist will be synchronized. The following table lists all configuration items in the blacklist.

```

show
ip address
no ip address
ip route

```

```
no ip route
ssh ip
no ssh ip
bond
no bond
clear bond
hostname
no hostname
mnet
no mnet
clear mnet
vlan
no vlan
clear vlan
webui ip
clear webui ip
xmlrpc ip
clear xmlrpc ip
webwall
cluster virtual priority
sdns listener
no sdns listener
clear sdns listener
sdns dps localdetector
sdns dnssec keygen
interface
no interface
clear interface
ha on
ha off
ha unit
no ha unit
clear ha unit
ha synconfig runtime on
ha synconfig runtime off
ha group enable
ha group disable
ha group switch
ha ssf peer
no ha ssf peer
clear ha ssf peer
ha link ffo
ha link network on
ha link network off
clear ha all
write net
write all
write file
log alert
system serialnumber
system fallback
system update
system license
system portmap
no system portmap
ntp on
clear http import error
```



```
clear http error
qos enable
role
clear role name
no role user
clear role user
user
no user
clear users
passwd
system tune route numaaccel on
clear system resource
config
no config
health import request
health import response
no health import request
no health import response
clear health import request
clear health import response
no http error
synconfig
no synconfig
quit
exit
engineering
config terminal
enable
disable
pager
no pager
debug
no debug
ssl export key
ipregion table export
crontab export
ping
no segment name
clear segment name
ip dhcp
accessgroup
no accessgroup
clear accessgroup
accesslist
no accesslist
clear accesslist
slb sip callnum list export
sdns dc
no sdns dc
show sdns dc name
show sdns dc status
clear sdns dc
ssl restore certificate
sdns config
no sdns config
clear sdns config
admin mode separation on
```

admin mode separation off

show ha synconfig

This command is used to display the status of the bootup synconfig function and the runtime synconfig function.

show ha status [group|domain|link|login]

This command is used to display the status of floating IP groups, HA units, and links, the synconfig status, and the login status of the local HA unit.

group domain link login	Optional. This parameter specifies the type of status information to be displayed. • group: displays the status of floating IP groups. • domain: displays the status of all HA units. • link: displays the status of the network and secondary links. • login: displays the login status and synconfig status of the local HA unit.
-------------------------	---

The default value is “group”.

SSF**ha ssf peer <peer_ip>**

This command is used to specify the peer unit for the SSF function.

peer_ip	This parameter specifies the IP address of the peer unit.
---------	---

no ha ssf peer <peer_ip>

This command is used to delete the specified SSF peer unit.

clear ha ssf peer

This command is used to clear all SSF peer units.

ha ssf {on|off} [virtual_service]

This command is used to enable or disable the Session Stateful Failover (SSF) function. By default, the SSF function is enabled for every individual virtual service and disabled globally. To make the SSF function take effect for a virtual service, you need to enable the SSF function globally first. This also applies when you need to enable the SSF function under the segment scope.

virtual_service Optional. This parameter specifies the name of a virtual service. The SSF function takes effect only for TCP, UDP, FTP, and IP virtual services.

If this parameter is not specified, the SSF function is enabled or disabled globally.

This parameter is mandatory when this command is executed under the segment scope.



Note:

- The SSF function supports at most three HA units.
- Make sure SLB and NAT virtual IP addresses are set as floating IP addresses before enabling the HA SSF function. Otherwise, unexpected errors might occur.
- Currently, SSF can be deployed only in the LAN scenario.
- The MTU of the interfaces on the SSF link should be equal to or larger than the sum of that on the service link and 500.
- The bandwidth of the SSF link should be greater than that of the service link.
- To ensure proper SSF functionality, the active and standby appliances should meet the following conditions:
 - SLB configurations on them are the same and are added in the same sequence.
 - Time difference between them should be shorter than 60 seconds.
 - Health check status of real services on them should be the same.
- The virtual services with SSF enabled do not support the TCP Selective Acknowledgment (SACK) function.

Assume that the IP addresses of two HA units are 198.162.1.1 and 198.162.1.2, respectively. If the SSF function needs to be enabled on two units, the following configurations are required:

On unit 1:

```
AN1(config)#ha ssf peer 192.168.1.2
AN1(config)#ha ssf on
```

On unit 2:

```
AN2(config)#ha ssf peer 192.168.1.1
AN2(config)#ha ssf on
```

ha ssf {on|off} nat

This command is used to enable or disable the SSF function for NAT. By default, the SSF function is disabled for NAT.

When this function is enabled, and the “**nat static**” command is configured, the system supports session failover for outbound traffic translated by the “**nat static**” rule, but not for translated inbound traffic.

ha ssf filter {on|off}

Enables or disables HA stateful session failover filter. When enabled, session data is preserved during device failover, allowing sessions to continue seamlessly on the standby device even when the active device fails. By default, it is enabled.

show ha ssf session

This command is used to display all the session information related to the SSF function.

show ha ssf settings

This command is used to display all the settings related to the SSF function.

Example:

```
AN(config)#show ha ssf settings
ha ssf peer 192.168.1.1
ha ssf on
ha ssf off nat
ha ssf on v2
ha ssf on vsudp
```

show statistics ssf

This command is used to display all the statistics related to the SSF function.

clear statistics ssf

This command is used to clear all the statistics related to the SSF function.

ha ssf timeout <timeout>

This command is used to set the timeout value for the SSF session. When this command is not configured, the default SSF session timeout value is 300 seconds.

timeout	This parameter specifies the timeout value in seconds. Its value must be an integer ranging from 60 to 7200.
---------	--

no ha ssf timeout

This command is used to restore the timeout value for the SSF session to the default value.

HA Consistency Check

ha consistency {on|off}

This command is used to enable or disable the HA consistency check function. By default, this function is enabled. After this function is enabled, when the administrator executes the “**ha on**” command, HA units will check whether they are consistent in software version, device model and licensed features. If any of the software version, device model or licensed features is inconsistent, the HA function cannot be enabled. When checking the software version consistency, the system considers the software versions consistent as long as the first two digits of the version numbers are the same. After this function is disabled, when the administrator executes the “**ha on**” command, HA units will not perform the consistency check on the device model and licensed features. HA can work as long as the first two digits of the version numbers are the same.

This function can be disabled only after the SSF and runtime synconfig functions are disabled. By default, this function is enabled. When this function is disabled, SSF and runtime synconfig cannot be enabled. The configuration of this function must be the same on all HA units.

clear ha consistency

This command is used to restore the setting of the HA consistency check function to default. After this command is executed, the HA consistency check function will be enabled.

HA Logging

ha log {on|off}

This command is used to enable or disable the HA logging function. By default, the HA logging function is disabled. If the HA function is enabled, the logging function will be enabled too. If the HA function is disabled, the logging function will be disabled too.

ha log level <level>

This command is used to set the level of the HA logs that the system generates.

level	This parameter specifies the level of HA logs. Its value must be “emerg”, “alert”, “crit”, “err”, “warning”, “notice”, “info”, and “debug”. Once the level of HA logs is specified, the message lower than this level will be ignored.
-------	--

The default value is info.

show ha log [line_number]

This command is used to display HA logs.

line_number Optional. This parameter specifies the number of lines of HA logs to be displayed.

The default value is 100, indicating that the latest 100 lines of HA logs generated by the system will be displayed.

clear ha log

This command is used to clear all the HA logs.

General Configurations

ha on

This command is used to enable the HA function.



Note:

- The HA function cannot be used together with the clustering function.
- IP address pools used by HA must be configured before the HA function is enabled. If IP address pools are configured after the HA function has been enabled, you need restart the HA function, that is to disable it first and then enable it.

ha off [force]

This command is used to disable the HA function. By default, the HA function is disabled.

force This parameter is used to disable the HA function once a hang occurs when a unit is joining the HA domain.

show ha config

This command is used to display all configurations related to the HA function.

For example:

```
AN(config)#show ha config
ha off
ha log off
ha log level info
ha arp interval 30
ha link ffo off
ha link network on
ha floatmac off
ha checkpeer on
ha synconfig bootup on
ha synconfig runtime off
```

<pre>ha heartbeat 1000 3 ha ssf off ha ssf off nat ha ssf on vs1</pre>
--

clear ha all

This command is used to delete all configurations related to the HA function.

show statistics ha

This command is used to display all HA statistics, including HAn event, floating IP group, and failover statistics.

clear statistics ha

This command is used to clear all HA statistics.

Chapter 7 Single System Image (SSI)

Single System Image allows multiple APV appliances providing the same virtual service at the same time to improve performance by forming an SSI domain. Because the SSI feature is based on the HA feature architecture, the SSI feature also includes the HA commands.

ha ssi on/off

This command enables/disables the SSI feature.

ha group port <group_id> <bond_ifname|l2_vip_interface> [system_ifname]

This command is used to add a sub interface of the bond interface or the system interface associated with the layer 2 virtual service to an HA floating IP group. When the status of the HA floating IP group is “Active”, the sub interface of the bond interface or the system interface associated with the layer 2 virtual service is active and can forward data packets; otherwise, when the status of the HA floating IP group is “Init” or “Standby”, the interface is inactive, and cannot forward data packets.

group_id This parameter specifies the ID of the floating IP group. Its value must be an integer ranging from 0 to 13.

bond_ifname|l2_vip_interface This parameter specifies the name of the bond interface or the system interface associated with the layer 2 virtual service. The default bond interface name is bond1, bond2, bond3, bond4 and so on. (Administrators can self-define the bond interface name by using the command “**bond name**”.)

system_ifname Optional. This parameter specifies the name of the system interface in the bond interface, which, by default, is port1, port2, port3, port4 and so on. (Administrators can self-define the name of the system interface by using the command “**interface name**”.)

Note: This parameter must be specified if the parameter “bond_ifname|l2_vip_interface” is set to the name of a bond interface.

no ha group port <group_id> <bond_ifname> <system_ifname>

This command is used to remove the configuration that the status of a system interface in a bond interface the same as an HA floating IP group.

clear ha group port <group_id>

This command is used to dissociate all interfaces from the specified floating IP group.

ha group backup <group_id1> <group_id2>

This command is used to back up two HA floating IP groups on a unit for switch failover.

ha group priority *<unit_name|ssi> <group_id> <priority>*

This command is used to configure or change the priority of a specified floating IP group on all units in the SSI domain. If the priority of a floating IP group in an SSI domain is not specified, this floating IP group does not take effect.

<code>unit_name ssi</code>	This parameter specifies the SSI domain. Its value must be “ssi”, indicating that the priority configured for the floating IP group will be used for the SSI function.
----------------------------	--

The “unit_name” parameter applies only to the HA function. For details, please see descriptions of this command in Chapter 6 High Availability (HA).

<code>group_id</code>	This parameter specifies the ID of the floating IP group.
-----------------------	---

<code>priority</code>	This parameter specifies the priority of a specified floating IP group on the specified unit. Its value must be an integer ranging from 0 to 255. The larger the value, the higher the priority.
-----------------------	--

no ha group priority *<unit_name|ssi> <group_id>*

This command is used to remove the priority of a specified floating IP group on the specified HA unit, or all units in the SSI domain.

Chapter 8 Server Load Balancing (SLB)

The Server Load Balancing (SLB) function distributes client requests to the optimal real services for processing based on specific load balancing policies and methods.

The SLB function consists of the following concepts:

- **Real service:** indicates the backend server actually processing client requests.
- **Real service group:** indicates a group of real services providing the same type of service.
- **Group method:** controls the distribution of traffic among the real services in a real service group.
- **Virtual service:** is the mapping of multiple real services on the APV appliance and is the unified access point of these real services.
- **Load balancing policy:** forwards client requests accessing a virtual service to the specified real service group.

The basic workflow of the SLB function is as follows:

7. The client sends a request to a virtual service on the APV appliance.
8. The appliance forwards the client request to the specified real service group according to the load balancing policy of the virtual service.
9. The real service group forwards the client request to the specified real service according to the group method.
10. The real service returns the response to the appliance. (In triangle transmission mode, the real service returns the response to the upstream router of the appliance.)
11. The appliance forwards the response to the client.

The SLB function supports health check on the real services, ensuring that the client requests are distributed to the available and available real services. In addition, the SLB function also provides other functions, such as fast forwarding.

This chapter covers the commands for configuring the SLB function.

Real Service

The real service is a service running on a physical network device (real server). The real service handles client requests and is represented by an IP address and a service.

Defining Real Service

The following commands are used to define real services based on the protocol type.

In every protocol layer, the supported types of real services are shown in the following table:

Protocol Layer	Service Type
Layer 7	HTTP, HTTPS, DNS, DNSTCP, FTP, RDP, RTSP, SIP-TCP, SIP-UDP, Radauth, Radacct, and Diameter
Layer 4	TCP, TCPS, UDP
Layer 3	IP
Layer 2	L2 IP, L2 MAC



Note: Multiple real services of the same type can share the same IP/port pair.

The total number of real services depends on the system memory:

System Memory	Number of real services
4 GB ≤ Memory < 32 GB	4000
32 GB ≤ Memory < 128 GB	8000
Memory ≥ 128 GB	20000

slb real http <real_service> <ip|domain> [port] [max_connection] [hc_type] [hc_up] [hc_down]

This command is used to define an HTTP real service.

real_service

This parameter specifies the name of the real service. Its value must be a case-sensitive string of 1 to 128 characters and must be enclosed in double quotes when containing special characters, which include the at sign (@), exclamation mark (!), tilde (~), underscore (_), colon (:), hyphen (-), angle brackets (< >), and period (.). The parameter value cannot contain spaces or be the system reserved word “default”, “all”, or “global”, which is case-insensitive.

Note: The name of the real service cannot be the same as that of the virtual service.

ip|domain

This parameter specifies the IP address or domain name of the real service. Its value must be an IPv4 or IPv6 address or a string of 1 to 255 characters.

When a domain name is set, it supports numbers, letters and hyphens (“-”), but cannot end with a hyphen. The domain name can be followed with the “:4/6” suffix to indicate the required resource type, such as “abc:6”. The “:4” suffix or no suffix indicates the IPv4 resource type; the “:6” suffix indicates the IPv6 resource type.

port

Optional. This parameter specifies the port number by which the real service listens to the client requests. Its value must be an integer ranging from 0 to 65,535. If the parameter is set to 0, the real service listens to client

requests on all ports and the value of the “hc_type” parameter must be “icmp” or “none”.

The default value is 80.

max_connection

Optional. This parameter specifies the maximum number of concurrent connections allowed by the real service. Its value must be an integer ranging from 0 to 4,294,967,295. When value 0 is specified, there is no limitation on the maximum number of concurrent connections.

The maximum number of concurrent connections depends on the performance of the server providing the real service. If the set maximum number is larger than what the server can support, the exceeding connections cannot be established.

The default value is 0.

hc_type

Optional. This parameter specifies the type of the basic health check. Its value must be “http”, “http2”, “tcp”, “icmp”, “script-tcp”, “snmp” or “none”.

- If the “port” parameter is set to 0, the value of this parameter must be “icmp” or “none”.
- If this parameter is set to “none”, the system will not perform the basic health check on this real service.

The default value is “tcp”.

hc_up

Optional. This parameter specifies the number of consecutive health check successes for marking the real service as “Up”. Its value must be an integer ranging from 1 to 255.

The default value is 3.

hc_down

Optional. This parameter specifies the number of consecutive health check failures for marking the real service as “Down”. Its value must be an integer ranging from 1 to 255.

The default value is 3.

slb real https <real_service> <ip|domain> [port] [max_connection] [hc_type] [hc_up] [hc_down]

This command is used to define an HTTPS real service.

real_service	This parameter specifies the name of the real service. Its value must be a case-sensitive string of 1 to 128 characters and must be enclosed in double quotes when containing special characters, which include the at sign (@), exclamation mark (!), tilde (~), underscore (_), colon (:), hyphen (-), angle brackets (< >), and period (.). The parameter value cannot contain spaces or be the system reserved word “default”, “all”, or “global”, which is case-insensitive. Note: The name of the real service cannot be the same as that of the virtual service.
ip domain	This parameter specifies the IP address or domain name of the real service. Its value must be an IPv4 or IPv6 address or a string of 1 to 255 characters. When a domain name is set, it supports numbers, letters and hyphens (“-”), but cannot end with a hyphen. The domain name can be followed with the “:4/6” suffix to indicate the required resource type, such as “abc:6”. The “:4” suffix or no suffix indicates the IPv4 resource type; the “:6” suffix indicates the IPv6 resource type.
port	Optional. This parameter specifies the port number by which the real service listens to the client requests. Its value must be an integer ranging from 0 to 65,535. If the parameter is set to 0, the real service listens to client requests on all ports and the value of the “hc_type” parameter must be “icmp” or “none”. The default value is 443.
max_connection	Optional. This parameter specifies the maximum number of concurrent connections allowed by the real service. Its value must be an integer ranging from 0 to 4,294,967,295. When value 0 is specified, there is no limitation on the maximum number of concurrent connections. The maximum number of concurrent connections depends on the performance of the server providing the real service. If the set maximum number is larger than what the server can support, the exceeding connections cannot be established. The default value is 0.
hc_type	Optional. This parameter specifies the type of the basic health check. Its value must be “https”, “https2”, “tcp”, “tcps”, “icmp”, “script-tcp”, “script-tcps”, “snmp” or

“none”.

- If the “port” parameter is set to 0, the value of this parameter must be “icmp” or “none”.
- If this parameter is set to “none”, the system will not perform the basic health check on this real service.

The default value is “tcp”.

hc_up

Optional. This parameter specifies the number of consecutive health check successes for marking the real service as “Up”. Its value must be an integer ranging from 1 to 255.

The default value is 3.

hc_down

Optional. This parameter specifies the number of consecutive health check failures for marking the real service as “Down”. Its value is an integer ranging from 1 to 255.

The default value is 3.

dns domain updateinterval <interval>

This command is used to set the update interval of real service domain name resolution. If this command is not configured, the default update interval of real service domain name resolution is 300s.

interval

This parameter specifies the update interval of real service domain name resolution. Its value must be an integer ranging from 60 to 86,400, in seconds.

no dns domain updateinterval

This command is used to restore the setting of the update interval of real service domain name resolution to default.

show dns domain updateinterval

This command is used to display the setting of the update interval of real service domain name resolution.

dns domain whitelist <ip_group_name>

This command is used to add an IP address group whitelist for the DNS passive discovery function.

If the DNS request hits the configured IP address group whitelist, the system will intercept the IP address from the response returned by the real service, and generate the corresponding route record in the Eroute table. This function takes effect mainly when the parameter “dsthost” of the “**ip eroute**” command is configured as a domain name. If this function is not configured, the system will intercept the IP address from all responses returned by the real service.

The system supports a maximum of one IPv4 address group whitelist and one IPv6 address group whitelist.

<code>ip_group_name</code>	This parameter specifies the existing name of the IP address group.
----------------------------	---

no dns domain whitelist <ip_group_name>

This command is used to delete the specified IP address group whitelist.

show dns domain whitelist

This command is used to display the configurations of all configured IP address group whitelists.

clear dns domain whitelist

This command is used to clear all the configurations of all configured IP address group whitelists.

slb real dns <real_service> <ip> [port] [max_connection] [hc_type] [hc_up] [hc_down] [timeout]

This command is used to define a DNS real service.

<code>real_service</code>	This parameter specifies the name of the real service. Its value must be a case-sensitive string of 1 to 128 characters and must be enclosed in double quotes when containing special characters, which include the at sign (@), exclamation mark (!), tilde (~), underscore (_), colon (:), hyphen (-), angle brackets (< >), and period (.). The parameter value cannot contain spaces or be the system reserved word “default”, “all”, or “global”, which is case-insensitive.
---------------------------	---

Note: The name of the real service cannot be the same as that of the virtual service.

<code>ip</code>	This parameter specifies the IP address of the real service. Its value must be an IPv4 or IPv6 address.
-----------------	---

<code>port</code>	Optional. This parameter specifies the port number by which the real service listens to the client requests. Its value must be an integer ranging from 0 to 65,535. If the
-------------------	--

parameter is set to 0, the real service listens to client requests on all ports and the value of the “hc_type” parameter must be “icmp” or “none”.

The default value is 53.

max_connection

Optional. This parameter specifies the maximum number of concurrent connections allowed by the real service. Its value must be an integer ranging from 0 to 4,294,967,295. When value 0 is specified, there is no limitation on the maximum number of concurrent connections.

The maximum number of concurrent connections depends on the performance of the server providing the real service. If the set maximum number is larger than what the server can support, the exceeding connections cannot be established.

The default value is 0.

hc_type

Optional. This parameter specifies the type of the basic health check. Its value must be “dns”, “icmp”, “udp”, “script-udp”, “snmp” or “none”.

- If the “port” parameter is set to 0, the value of this parameter must be “icmp” or “none”.
- If this parameter is set to “none”, the system will not perform the basic health check on this real service.

The default value is “dns”.

hc_up

Optional. This parameter specifies the number of consecutive health check successes for marking the real service as “Up”. Its value must be an integer ranging from 1 to 255.

The default value is 3.

hc_down

Optional. This parameter specifies the number of consecutive health check failures for marking the real service as “Down”. Its value is an integer ranging from 1 to 255.

The default value is 3.

timeout

Optional. This parameter specifies the timeout period of the real service connection, in seconds. Its value must be an integer ranging from 0 to 1,000,000.

The default value is 60.

slb real dnstcp <real_service> <ip> [port] [max_connection] [hc_type] [hc_up] [hc_down]

This command is used to define a DNS real service that uses the TCP protocol for data transmission.

real_service	This parameter specifies the name of a real service. The value must be a case-sensitive string with a length not greater than 128 bytes. If the value contains special characters, it must be enclosed in double quotes. The value cannot contain spaces or be the system reserved word “default,” “all,” or “global,” which is case-insensitive. Note: The name of a real service cannot be the same as the name of an existing virtual service.
ip	This parameter specifies the IP address of a real service. The value must be an IPv4 or IPv6 address.
port	Optional. This parameter specifies the port number for a real service to listen to the client’s request. The value must be an integer ranging from 0 to 65,535. When the value is zero, it means this real service listens to the client’s request on all ports, and the value of the parameter “hc_hype” can only be “icmp” or “none.” The default value is 53.
max_connection	Optional. This parameter specifies the maximum number of concurrent connections of a real service. The value must be an integer ranging from 0 to 4,294,967,295. When the value is zero, it means there is no maximum number of concurrent connections. The maximum number of concurrent connections depends on the performance of the server that provides real services. If the maximum number exceeds what the server can support, the exceeded connections won’t be established. The default value is zero.
hc_type	Optional. This parameter specifies the type of the basic health check. The value must be “icmp,” “tcp,” “dns-tcp,” “script-tcp,” or “none.” • If the value of the parameter “port” is zero, the value of this parameter can only be “icmp” and “none.” • If the value is “none,” it means the basic health check won’t be done for the real service. The default value is “dns-tcp.”

hc_up Optional. This parameter specifies the number of consecutive successful health checks required by identifying the status of a real service as “Up.” The value must be an integer ranging from 1 to 255.

The default value is three.

hc_down Optional. This parameter specifies the number of consecutive unsuccessful health checks required by identifying the status of a real service as “Down.” The value must be an integer ranging from 1 to 255.

The default value is three.

slb real ftp *<real_service> <ip|domain> [port] [max_connection] [hc_type] [hc_up] [hc_down]*

This command is used to define an FTP real service.

real_service This parameter specifies the name of the real service. Its value must be a case-sensitive string of 1 to 128 characters and must be enclosed in double quotes when containing special characters, which include the at sign (@), exclamation mark (!), tilde (~), underscore (_), colon (:), hyphen (-), angle brackets (< >), and period (.). The parameter value cannot contain spaces or be the system reserved word “default”, “all”, or “global”, which is case-insensitive.

Note: The name of the real service cannot be the same as that of the virtual service.

ip|domain This parameter specifies the IP address or domain name of the real service. Its value must be an IPv4 or IPv6 address or a string of 1 to 255 characters.

When a domain name is set, it supports numbers, letters and hyphens (“-”), but cannot end with a hyphen. The domain name can be followed with the “:4/6” suffix to indicate the required resource type, such as “abc:6”. The “:4” suffix or no suffix indicates the IPv4 resource type; the “:6” suffix indicates the IPv6 resource type.

port Optional. This parameter specifies the port number by which the real service listens to the client requests. Its value must be an integer ranging from 1 to 65,535.

The default value is 21.

max_connection Optional. This parameter specifies the maximum number of concurrent connections allowed by the real service. Its value must be an integer ranging from 0 to 4,294,967,295.

When value 0 is specified, there is no limitation on the maximum number of concurrent connections.

The maximum number of concurrent connections depends on the performance of the server providing the real service. If the set maximum number is larger than what the server can support, the exceeding connections cannot be established.

The default value is 0.

hc_type

Optional. This parameter specifies the type of the basic health check. Its value must be “ftp”, “tcp”, “icmp”, “script-tcp”, “snmp” or “none”.

- If the “port” parameter is set to 0, the value of this parameter must be “icmp” or “none”.
- If this parameter is set to “none”, the system will not perform the basic health check on this real service.

The default value is “tcp”.

hc_up

Optional. This parameter specifies the number of consecutive health check successes for marking the real service as “Up”. Its value must be an integer ranging from 1 to 255.

The default value is 3.

hc_down

Optional. This parameter specifies the number of consecutive health check failures for marking the real service as “Down”. Its value is an integer ranging from 1 to 255.

The default value is 3.



Note: When the FTP basic health check is configured, the commands “**health request**” and “**health server**” should also be configured. A configuration example is as follows:

```
AN(config)#slb real ftp "rf1" 10.8.1.133 21 100 ftp 3 3
AN(config)#health request 1 "ftp://administrator:think@10.8.1.133:/home/test/1.html"
AN(config)#health server "rf1" 1 1
```

slb real tcp <real_service> <ip|domain> <port> [max_connection] [hc_type] [hc_up] [hc_down]

This command is used to define a TCP real service.

real_service

This parameter specifies the name of the real service. Its value must be a case-sensitive string of 1 to 128 characters

and must be enclosed in double quotes when containing special characters, which include the at sign (@), exclamation mark (!), tilde (~), underscore (_), colon (:), hyphen (-), angle brackets (< >), and period (.). The parameter value cannot contain spaces or be the system reserved word “default”, “all”, or “global”, which is case-insensitive.

Note: The name of the real service cannot be the same as that of the virtual service.

ip domain	This parameter specifies the IP address or domain name of the real service. Its value must be an IPv4 or IPv6 address or a string of 1 to 255 characters. When a domain name is set, it supports numbers, letters and hyphens (“-”), but cannot end with a hyphen. The domain name can be followed with the “:4/6” suffix to indicate the required resource type, such as “abc:6”. The “:4” suffix or no suffix indicates the IPv4 resource type; the “:6” suffix indicates the IPv6 resource type.
port	Optional. This parameter specifies the port number by which the real service listens to the client requests. Its value must be an integer ranging from 0 to 65,535. If the parameter is set to 0, the real service listens to client requests on all ports and the value of the “hc_type” parameter must be “icmp” or “none”.
max_connection	Optional. This parameter specifies the maximum number of concurrent connections allowed by the real service. Its value must be an integer ranging from 0 to 4,294,967,295. When value 0 is specified, there is no limitation on the maximum number of concurrent connections. The maximum number of concurrent connections depends on the performance of the server providing the real service. If the set maximum number is larger than what the server can support, the exceeding connections cannot be established. The default value is 0.
hc_type	Optional. This parameter specifies the type of the basic health check. Its value must be “http”, “https”, “http2”, “https2”, “tcp”, “tcps”, “icmp”, “pop3”, “smtp”, “script-tcp”, “script-tcps”, “rtsp-tcp”, “sip-tcp”, “dns-tcp”, “ldap”, “mysql-db”, “mysql-dbs”, “mssql-db”, “mssql-dbs”, “snmp”, “oracle-db” or “none”. • If the “port” parameter is set to 0, the value of this

parameter must be “icmp” or “none”.

- If this parameter is set to “none”, the system will not perform the basic health check on this real service.

The default value is “tcp”.

hc_up

Optional. This parameter specifies the number of consecutive health check successes for marking the real service as “Up”. Its value must be an integer ranging from 1 to 255.

The default value is 3.

hc_down

Optional. This parameter specifies the number of consecutive health check failures for marking the real service as “Down”. Its value is an integer ranging from 1 to 255.

The default value is 3.

slb real tcps <real_service> <ip|domain> <port> [max_connection] [hc_type] [hc_up] [hc_down]

This command is used to define a TCPS real service.

real_service

This parameter specifies the name of the real service. Its value must be a case-sensitive string of 1 to 128 characters and must be enclosed in double quotes when containing special characters, which include the at sign (@), exclamation mark (!), tilde (~), underscore (_), colon (:), hyphen (-), angle brackets (< >), and period (.). The parameter value cannot contain spaces or be the system reserved word “default”, “all”, or “global”, which is case-insensitive.

Note: The name of the real service cannot be the same as that of the virtual service.

ip|domain

This parameter specifies the IP address or domain name of the real service. Its value must be an IPv4 or IPv6 address or a string of 1 to 255 characters.

When a domain name is set, it supports numbers, letters and hyphens (“-”), but cannot end with a hyphen. The domain name can be followed with the “:4/6” suffix to indicate the required resource type, such as “abc:6”. The “:4” suffix or no suffix indicates the IPv4 resource type; the “:6” suffix indicates the IPv6 resource type.

port	Optional. This parameter specifies the port number by which the real service listens to the client requests. Its value must be an integer ranging from 0 to 65,535. If the parameter is set to 0, the real service listens to client requests on all ports and the value of the “hc_type” parameter must be “icmp” or “none”.
max_connection	Optional. This parameter specifies the maximum number of concurrent connections allowed by the real service. Its value must be an integer ranging from 0 to 4,294,967,295. When value 0 is specified, there is no limitation on the maximum number of concurrent connections. The maximum number of concurrent connections depends on the performance of the server providing the real service. If the set maximum number is larger than what the server can support, the exceeding connections cannot be established. The default value is 0.
hc_type	Optional. This parameter specifies the type of the basic health check. Its value must be “tcp”, “tcps”, “https2”, “icmp”, “script-tcp”, “script-tcps”, “https”, “mysql-dbs”, “mssql-dbs”, “snmp” or “none”. • If the “port” parameter is set to 0, the value of this parameter must be “icmp” or “none”. • If this parameter is set to “none”, the system will not perform the basic health check on this real service. The default value is “tcp”.
hc_up	Optional. This parameter specifies the number of consecutive health check successes for marking the real service as “Up”. Its value must be an integer ranging from 1 to 255. The default value is 3.
hc_down	Optional. This parameter specifies the number of consecutive health check failures for marking the real service as “Down”. Its value is an integer ranging from 1 to 255. The default value is 3.

slb real udp <real_service> <ip> <port> [max_connection] [hc_up] [hc_down] [timeout] [hc_type]

This command is used to define a UDP real service.

For existing "**slb real udp**" configurations, this command can also be used to change their parameter configurations.

real_service This parameter specifies the name of the real service. Its value must be a case-sensitive string of 1 to 128 characters and must be enclosed in double quotes when containing special characters, which include the at sign (@), exclamation mark (!), tilde (~), underscore (_), colon (:), hyphen (-), angle brackets (< >), and period (.). The parameter value cannot contain spaces or be the system reserved word "default", "all", or "global", which is case-insensitive.

Note: The name of the real service cannot be the same as that of the virtual service.

ip This parameter specifies the IP address of the real service. Its value must be an IPv4 or IPv6 address.

port Optional. This parameter specifies the port number by which the real service listens to the client requests. Its value must be an integer ranging from 0 to 65,535. If the parameter is set to 0, the real service listens to client requests on all ports and the value of the "hc_type" parameter must be "icmp" or "none".

max_connection Optional. This parameter specifies the maximum number of concurrent connections allowed by the real service. Its value must be an integer ranging from 0 to 4,294,967,295. When value 0 is specified, there is no limitation on the maximum number of concurrent connections.

The maximum number of concurrent connections depends on the performance of the server providing the real service. If the set maximum number is larger than what the server can support, the exceeding connections cannot be established.

The default value is 0.

hc_up Optional. This parameter specifies the number of consecutive health check successes for marking the real service as "Up". Its value must be an integer ranging from 1 to 255.

The default value is 3.

hc_down Optional. This parameter specifies the number of consecutive health check failures for marking the real

service as “Down”. Its value is an integer ranging from 1 to 255.

The default value is 3.

timeout Optional. This parameter specifies the timeout period of the real service connection, in seconds. Its value must be an integer ranging from 0 to 1,000,000. The default value is 60.

hc_type Optional. This parameter specifies the type of the basic health check. Its value must be “icmp”, “udp”, “script-udp”, “radius-auth”, “radius-acct”, “dns”, “snmp”, “ntp”, or “none”.

- If the “port” parameter is set to 0, the value of this parameter must be “icmp” or “none”.
- If this parameter is set to “none”, the system will not perform the basic health check on this real service.

The default value is “icmp”.

slb real ip <real_service> <ip|domain> [max_connection] [hc_type] [hc_up] [hc_down] [udp_timeout]

This command is used to define an IP real service used for Layer 3 load balancing. This command supports Layer 3 load balancing for only TCP and UDP traffic. The TCP connection will be terminated if the TCP connection has been idle out of the maximum idle time set by the command “**system tune tcpidle** <idle_time>”.

real_service This parameter specifies the name of the real service. Its value must be a case-sensitive string of 1 to 128 characters and must be enclosed in double quotes when containing special characters, which include the at sign (@), exclamation mark (!), tilde (~), underscore (_), colon (:), hyphen (-), angle brackets (< >), and period (.). The parameter value cannot contain spaces or be the system reserved word “default”, “all”, or “global”, which is case-insensitive.

Note: The name of the real service cannot be the same as that of the virtual service.

ip|domain This parameter specifies the IP address or domain name of the real service. Its value must be an IPv4 or IPv6 address or a string of 1 to 255 characters.

When a domain name is set, it supports numbers, letters and hyphens (“-”), but cannot end with a hyphen. The domain name can be followed with the “:4/6” suffix to

indicate the required resource type, such as “abc:6”. The “:4” suffix or no suffix indicates the IPv4 resource type; the “:6” suffix indicates the IPv6 resource type.

max_connection

Optional. This parameter specifies the maximum number of concurrent connections allowed by the real service. Its value must be an integer ranging from 0 to 4,294,967,295. When value 0 is specified, there is no limitation on the maximum number of concurrent connections.

The maximum number of concurrent connections depends on the performance of the server providing the real service. If the set maximum number is larger than what the server can support, the exceeding connections cannot be established.

The default value is 0.

hc_type

Optional. This parameter specifies the type of the basic health check. Its value must be “icmp”, “snmp” or “none”. If this parameter is set to “none”, the system will not perform the basic health check on this real service.

The default value is “icmp”.

hc_up

Optional. This parameter specifies the number of consecutive health check successes for marking the real service as “Up”. Its value must be an integer ranging from 1 to 255.

The default value is 3.

hc_down

Optional. This parameter specifies the number of consecutive health check failures for marking the real service as “Down”. Its value is an integer ranging from 1 to 255.

The default value is 3.

udp_timeout

Optional. This parameter specifies the idle timeout value of the real service’s UDP connection, in seconds. Its value must be an integer ranging from 0 to 4,294,967,295.

The default value is 60.

slb real siptcp <real_service> <ip> [port] [max_connection] [hc_type] [hc_up] [hc_down]

This command is used to define a TCP-based Session Initiation Protocol (SIP) real service.

real_service	This parameter specifies the name of the real service. Its value must be a case-sensitive string of 1 to 128 characters and must be enclosed in double quotes when containing special characters, which include the at sign (@), exclamation mark (!), tilde (~), underscore (_), colon (:), hyphen (-), angle brackets (< >), and period (.). The parameter value cannot contain spaces or be the system reserved word “default”, “all”, or “global”, which is case-insensitive. Note: The name of the real service cannot be the same as that of the virtual service.
ip	This parameter specifies the IP address of the real service. Its value must be an IPv4 or IPv6 address.
port	Optional. This parameter specifies the port number by which the real service listens to the client requests. Its value must be an integer ranging from 0 to 65,535. If the parameter is set to 0, the real service listens to client requests on all ports and the value of the “hc_type” parameter must be “icmp” or “none”. The default value is 5060.
max_connection	Optional. This parameter specifies the maximum number of concurrent connections allowed by the real service. Its value must be an integer ranging from 0 to 4,294,967,295. When value 0 is specified, there is no limitation on the maximum number of concurrent connections. The maximum number of concurrent connections depends on the performance of the server providing the real service. If the set maximum number is larger than what the server can support, the exceeding connections cannot be established. The default value is 0.
hc_type	Optional. This parameter specifies the type of the basic health check. Its value must be “tcp”, “icmp”, “script-tcp”, “sip-tcp”, “snmp” or “none”. • If the “port” parameter is set to 0, the value of this parameter must be “icmp” or “none”. • If this parameter is set to “none”, the system will not perform the basic health check on this real service. The default value is “sip-tcp”.

hc_up	Optional. This parameter specifies the number of consecutive health check successes for marking the real service as “Up”. Its value must be an integer ranging from 1 to 255. The default value is 3.
hc_down	Optional. This parameter specifies the number of consecutive health check failures for marking the real service as “Down”. Its value is an integer ranging from 1 to 255. The default value is 3.

slb real sipudp <real_service> <ip> [port] [max_connection] [hc_type] [hc_up] [hc_down] [timeout]

This command is used to define a UDP-based SIP real service.

real_service	This parameter specifies the name of the real service. Its value must be a case-sensitive string of 1 to 128 characters and must be enclosed in double quotes when containing special characters, which include the at sign (@), exclamation mark (!), tilde (~), underscore (_), colon (:), hyphen (-), angle brackets (< >), and period (.). The parameter value cannot contain spaces or be the system reserved word “default”, “all”, or “global”, which is case-insensitive.
--------------	--

Note: The name of the real service cannot be the same as that of the virtual service.

ip	This parameter specifies the IP address of the real service. Its value must be an IPv4 or IPv6 address.
----	--

port	Optional. This parameter specifies the port number by which the real service listens to the client requests. Its value must be an integer ranging from 0 to 65,535. If the parameter is set to 0, the real service listens to client requests on all ports and the value of the “hc_type” parameter must be “icmp” or “none”.
------	--

The default value is 5060.

max_connection	Optional. This parameter specifies the maximum number of concurrent connections allowed by the real service. Its value must be an integer ranging from 0 to 4,294,967,295. When value 0 is specified, there is no limitation on the maximum number of concurrent connections.
----------------	--

The maximum number of concurrent connections depends on the performance of the server providing the real service. If the set maximum number is larger than what the server can support, the exceeding connections cannot be established.

The default value is 0.

hc_type

Optional. This parameter specifies the type of the basic health check. Its value must be “icmp”, “udp”, “script-udp”, “sip-udp”, “snmp” or “none”.

- If the “port” parameter is set to 0, the value of this parameter must be “icmp” or “none”.
- If this parameter is set to “none”, the system will not perform the basic health check on this real service.

The default value is “sip-udp”.

hc_up

Optional. This parameter specifies the number of consecutive health check successes for marking the real service as “Up”. Its value must be an integer ranging from 1 to 255.

The default value is 3.

hc_down

Optional. This parameter specifies the number of consecutive health check failures for marking the real service as “Down”. Its value is an integer ranging from 1 to 255.

The default value is 3.

timeout

Optional. This parameter specifies the timeout period of the real service connection, in seconds. Its value must be an integer ranging from 0 to 4,294,967,295.

The default value is 60.

slb real radauth <real_service> <ip> [port] [max_connection] [hc_type] [hc_up] [hc_down] [timeout]

This command is used to define a Remote Authentication Dial-In User Service (RADIUS) authentication real service.

real_service

This parameter specifies the name of the real service. Its value must be a case-sensitive string of 1 to 128 characters and must be enclosed in double quotes when containing special characters, which include the at sign (@), exclamation mark (!), tilde (~), underscore (_), colon (:),

hyphen (-), angle brackets (< >), and period (.). The parameter value cannot contain spaces or be the system reserved word “default”, “all”, or “global”, which is case-insensitive.

Note: The name of the real service cannot be the same as that of the virtual service.

ip	This parameter specifies the IP address of the real service. Its value must be an IPv4 or IPv6 address.
port	Optional. This parameter specifies the port number by which the real service listens to the client requests. Its value must be an integer ranging from 0 to 65,535. If the parameter is set to 0, the real service listens to client requests on all ports and the value of the “hc_type” parameter must be “icmp” or “none”. The default value is 1812.
max_connection	Optional. This parameter specifies the maximum number of concurrent connections allowed by the real service. Its value must be an integer ranging from 0 to 4,294,967,295. When value 0 is specified, there is no limitation on the maximum number of concurrent connections. The maximum number of concurrent connections depends on the performance of the server providing the real service. If the set maximum number is larger than what the server can support, the exceeding connections cannot be established. The default value is 0.
hc_type	Optional. This parameter specifies the type of the basic health check. Its value must be “icmp”, “udp”, “script-udp”, “radius-auth”, “snmp” or “none”. • If the “port” parameter is set to 0, the value of this parameter must be “icmp” or “none”. • If this parameter is set to “none”, the system will not perform the basic health check on this real service. The default value is “radius-auth”.
hc_up	Optional. This parameter specifies the number of consecutive health check successes for marking the real service as “Up”. Its value must be an integer ranging from 1 to 255.

The default value is 3.

hc_down Optional. This parameter specifies the number of consecutive health check failures for marking the real service as “Down”. Its value is an integer ranging from 1 to 255.

The default value is 3.

timeout Optional. This parameter specifies the timeout period of the real service connection, in seconds. Its value must be an integer ranging from 0 to 4,294,967,295.

The default value is 60.

slb real radacct *<real_service> <ip> [port] [max_connection] [hc_type] [hc_up] [hc_down] [timeout]*

This command is used to define a RADIUS accounting real service.

real_service This parameter specifies the name of the real service. Its value must be a case-sensitive string of 1 to 128 characters and must be enclosed in double quotes when containing special characters, which include the at sign (@), exclamation mark (!), tilde (~), underscore (_), colon (:), hyphen (-), angle brackets (< >), and period (.). The parameter value cannot contain spaces or be the system reserved word “default”, “all”, or “global”, which is case-insensitive.

Note: The name of the real service cannot be the same as that of the virtual service.

ip This parameter specifies the IP address of the real service. Its value must be an IPv4 or IPv6 address.

port Optional. This parameter specifies the port number by which the real service listens to the client requests. Its value must be an integer ranging from 0 to 65,535. If the parameter is set to 0, the real service listens to client requests on all ports and the value of the “hc_type” parameter must be “icmp” or “none”.

The default value is 1813.

max_connection Optional. This parameter specifies the maximum number of concurrent connections allowed by the real service. Its value must be an integer ranging from 0 to 4,294,967,295. When value 0 is specified, there is no limitation on the

maximum number of concurrent connections.

The maximum number of concurrent connections depends on the performance of the server providing the real service. If the set maximum number is larger than what the server can support, the exceeding connections cannot be established.

The default value is 0.

hc_type

Optional. This parameter specifies the type of the basic health check. Its value must be “icmp”, “udp”, “script-udp”, “radius-acct”, “snmp” or “none”.

- If the “port” parameter is set to 0, the value of this parameter must be “icmp” or “none”.
- If this parameter is set to “none”, the system will not perform the basic health check on this real service.

The default value is “radius-acct”.

hc_up

Optional. This parameter specifies the number of consecutive health check successes for marking the real service as “Up”. Its value must be an integer ranging from 1 to 255.

The default value is 3.

hc_down

Optional. This parameter specifies the number of consecutive health check failures for marking the real service as “Down”. Its value is an integer ranging from 1 to 255.

The default value is 3.

timeout

Optional. This parameter specifies the timeout period of the real service connection, in seconds. Its value must be an integer ranging from 0 to 4,294,967,295.

The default value is 60.

slb real rdp <real_service> <ip> [port] [max_connection] [hc_type] [hc_up] [hc_down]

This command is used to define a Remote Desktop Protocol (RDP) real service.

real_service

This parameter specifies the name of the real service. Its value must be a case-sensitive string of 1 to 128 characters and must be enclosed in double quotes when containing special characters, which include the at sign (@),

exclamation mark (!), tilde (~), underscore (_), colon (:), hyphen (-), angle brackets (< >), and period (.). The parameter value cannot contain spaces or be the system reserved word “default”, “all”, or “global”, which is case-insensitive.

Note: The name of the real service cannot be the same as that of the virtual service.

ip	This parameter specifies the IP address of the real service. Its value must be an IPv4 address.
port	Optional. This parameter specifies the port number by which the real service listens to the client requests. Its value must be an integer ranging from 0 to 65,535. If the parameter is set to 0, the real service listens to client requests on all ports and the value of the “hc_type” parameter must be “icmp” or “none”. The default value is 3389.
max_connection	Optional. This parameter specifies the maximum number of concurrent connections allowed by the real service. Its value must be an integer ranging from 0 to 4,294,967,295. When value 0 is specified, there is no limitation on the maximum number of concurrent connections. The maximum number of concurrent connections depends on the performance of the server providing the real service. If the set maximum number is larger than what the server can support, the exceeding connections cannot be established. The default value is 0.
hc_type	Optional. This parameter specifies the type of the basic health check. Its value must be “tcp”, “icmp”, “script-tcp”, “snmp” or “none”. • If the “port” parameter is set to 0, the value of this parameter must be “icmp” or “none”. • If this parameter is set to “none”, the system will not perform the basic health check on this real service. The default value is “icmp”.
hc_up	Optional. This parameter specifies the number of consecutive health check successes for marking the real service as “Up”. Its value must be an integer ranging from 1 to 255.

The default value is 3.

hc_down

Optional. This parameter specifies the number of consecutive health check failures for marking the real service as “Down”. Its value is an integer ranging from 1 to 255.

The default value is 3.

slb real rtsp <real_service> <ip> [port] [max_connection] [hc_type] [hc_up] [hc_down] [timeout]

This command is used to define a Real Time Streaming Protocol (RTSP) real service.

real_service

This parameter specifies the name of the real service. Its value must be a case-sensitive string of 1 to 128 characters and must be enclosed in double quotes when containing special characters, which include the at sign (@), exclamation mark (!), tilde (~), underscore (_), colon (:), hyphen (-), angle brackets (< >), and period (.). The parameter value cannot contain spaces or be the system reserved word “default”, “all”, or “global”, which is case-insensitive.

Note: The name of the real service cannot be the same as that of the virtual service.

ip

This parameter specifies the IP address of the real service. Its value must be an IPv4 or IPv6 address.

port

Optional. This parameter specifies the port number by which the real service listens to the client requests. Its value must be an integer ranging from 1 to 65,535.

The default value is 554.

max_connection

Optional. This parameter specifies the maximum number of concurrent connections allowed by the real service. Its value must be an integer ranging from 0 to 4,294,967,295. When value 0 is specified, there is no limitation on the maximum number of concurrent connections.

The maximum number of concurrent connections depends on the performance of the server providing the real service. If the set maximum number is larger than what the server can support, the exceeding connections cannot be established.

The default value is 0.

hc_type	Optional. This parameter specifies the type of the basic health check. Its value must be “rtsp-tcp”, “tcp”, “icmp”, “script-tcp”, “snmp” or “none”. • If the “port” parameter is set to 0, the value of this parameter must be “icmp” or “none”. • If this parameter is set to “none”, the system will not perform the basic health check on this real service. The default value is “rtsp-tcp”.
hc_up	Optional. This parameter specifies the number of consecutive health check successes for marking the real service as “Up”. Its value must be an integer ranging from 1 to 255. The default value is 3.
hc_down	Optional. This parameter specifies the number of consecutive health check failures for marking the real service as “Down”. Its value is an integer ranging from 1 to 255. The default value is 3.
timeout	Optional. This parameter specifies the timeout period of the real service connection, in seconds. Its value must be an integer ranging from 0 to 4,294,967,295. The default value is 60.

slb real diameter <real_service> <ip> [port] [max_connection] [hc_type] [hc_up] [hc_down]

This command is used to define a Diameter real service.

real_service	This parameter specifies the name of the real service. Its value must be a case-sensitive string of 1 to 128 characters and must be enclosed in double quotes when containing special characters, which include the at sign (@), exclamation mark (!), tilde (~), underscore (_), colon (:), hyphen (-), angle brackets (<>), and period (.). The parameter value cannot contain spaces or be the system reserved word “default”, “all”, or “global”, which is case-insensitive. Note: The name of the real service cannot be the same as that of the virtual service.
--------------	---

ip	This parameter specifies the IP address of the real service. Its value must be an IPv4 or IPv6 address.
port	Optional. This parameter specifies the port number by which the real service listens to the client requests. Its value must be an integer ranging from 0 to 65,535. If the parameter is set to 0, the real service listens to client requests on all ports and the value of the “hc_type” parameter must be “icmp” or “none”. The default value is 3868.
max_connection	Optional. This parameter specifies the maximum number of concurrent connections allowed by the real service. Its value must be an integer ranging from 0 to 4,294,967,295. When value 0 is specified, there is no limitation on the maximum number of concurrent connections. The maximum number of concurrent connections depends on the performance of the server providing the real service. If the set maximum number is larger than what the server can support, the exceeding connections cannot be established. The default value is 0.
hc_type	Optional. This parameter specifies the type of the basic health check. Its value must be “tcp”, “icmp”, “script-tcp”, “diameter-tcp”, “snmp” or “none”. • If the “port” parameter is set to 0, the value of this parameter must be “icmp” or “none”. • If this parameter is set to “none”, the system will not perform the basic health check on this real service. The default value is “tcp”.
hc_up	Optional. This parameter specifies the number of consecutive health check successes for marking the real service as “Up”. Its value must be an integer ranging from 1 to 255. The default value is 3.
hc_down	Optional. This parameter specifies the number of consecutive health check failures for marking the real service as “Down”. Its value must be an integer ranging from 1 to 255. The default value is 3.

diameter server timeout <timeout> [times]

This command is used to set the timeout value of Diameter server responses. When the number of consecutive timeout times exceeds the threshold, the Diameter server will be marked as “Down”. By default, the system does not check whether the Diameter server times out.

timeout This parameter specifies the timeout value of the Diameter server, in milliseconds. Its value must be an integer ranging from 0 to 60,000. The value 0 indicates no timeout.

times This parameter specifies the number of consecutive timeout times required for marking the Diameter server as “Down”. Its value must be an integer ranging from 0 to 10.

The default value is 0, indicating no timeout.

no diameter server timeout

This command is used to delete the timeout setting of Diameter server responses. That means the system will not check whether the Diameter server times out.

show diameter server timeout

This command is used to display the timeout setting of Diameter server responses.

clear diameter server timeout

This command is used to clear the timeout setting of Diameter server responses. That means the system will not check whether the Diameter server times out.

slb real tuxedo <real_service> <ip> <max_connection> <hc_type> <hc_up> <hc_down>

This command is used to configure a Tuxedo real service.

real_service This parameter specifies the name of the real service. Its value must be a case-sensitive string of 1 to 128 characters and must be enclosed in double quotes when containing special characters, which include the at sign (@), exclamation mark (!), tilde (~), underscore (_), colon (:), hyphen (-), angle brackets (< >), and period (.). The parameter value cannot contain spaces or be the system reserved word “default”, “all”, or “global”, which is case-insensitive.

Note: The name of the real service cannot be the same as that of the virtual service.

ip This parameter specifies the IP address of the real service.

	Its value must be an IPv4 or IPv6 address.
max_connection	Optional. This parameter specifies the maximum number of concurrent connections allowed by the real service. Its value must be an integer ranging from 0 to 4,294,967,295. When value 0 is specified, there is no limitation on the maximum number of concurrent connections. The maximum number of concurrent connections depends on the performance of the server providing the real service. If the set maximum number is larger than what the server can support, the exceeding connections cannot be established. The default value is 0.
hc_type	Optional. This parameter specifies the type of the basic health check. Its value must be “icmp”, “snmp” or “none”. If this parameter is set to “none”, the system will not perform the basic health check on this real service. The default value is “icmp”.
hc_up	Optional. This parameter specifies the number of consecutive health check successes for marking the real service as “Up”. Its value must be an integer ranging from 1 to 255. The default value is 3.
hc_down	Optional. This parameter specifies the number of consecutive health check failures for marking the real service as “Down”. Its value must be an integer ranging from 1 to 255. The default value is 3.

slb real l2ip <real_service> <real_ip>

This command is used to define an “l2ip” real service used for Layer 2 load balancing.

real_service	This parameter specifies the name of the real service. Its value must be a case-sensitive string of 1 to 128 characters and must be enclosed in double quotes when containing special characters, which include the at sign (@), exclamation mark (!), tilde (~), underscore (_), colon (:), hyphen (-), angle brackets (< >), and period (.). The parameter value cannot contain spaces or be the system reserved word “default”, “all”, or “global”, which is
--------------	---

case-insensitive.

Note: The name of the real service cannot be the same as that of the virtual service.

real_ip This parameter specifies the IP address of the real service. Its value must be an IPv4 or IPv6 address.

slb real l2mac <real_service> <real_mac> <output_interface>

This command is used to define an “l2mac” real service used for Layer 2 load balancing.

real_service This parameter specifies the name of the real service. Its value must be a case-sensitive string of 1 to 40 characters and must be enclosed in double quotes when containing special characters, which include the at sign (@), exclamation mark (!), tilde (~), underscore (_), colon (:), hyphen (-), angle brackets (< >), and period (.). The parameter value cannot contain spaces or be the system reserved word “default”, “all”, or “global”, which is case-insensitive.

Note: The name of the real service cannot be the same as that of the virtual service.

real_mac This parameter specifies the MAC address of the real service in the format of AB:CD:EF:GH:IJ:KL.

output_interface This parameter specifies the name of the interface bound to the MAC address. Its value must be the system interface, VLAN interface, bond interface or MNET interface.

slb real fwdip <real_service> <ip> <port> [max_connection]

This command is used to configure a FWDIP real service. The FWDIP real service is used to provide load balancing for security devices set up in L3 mode. It supports only ICMP, SNMP and TCP additional health checks. This type of real services can only be added to real service groups that use the hi, chi or rr method, and can be associated with TCPS virtual service only via the default policy. FWDIP real services send the packets to the IP address specified by this command. The destination IP addresses in the packets do not change. The TCP port number will be changed to the one defined in this command.

real_service This parameter specifies the name of the real service. Its value must be a case-sensitive string of 1 to 128 characters and must be enclosed in double quotes when containing special characters, which include the at sign (@), exclamation mark (!), tilde (~), underscore (_), colon (:),

hyphen (-), angle brackets (< >), and period (.). The parameter value cannot contain spaces or be the system reserved word “default”, “all”, or “global”, which is case-insensitive.

Note: The name of the real service cannot be the same as that of the virtual service.

ip	This parameter specifies the security device’s IP address that receives the packets. Its value must be an IPv4 or IPv6 address. If the real service works in L2 bridge mode or supports ARP forwarding, the value must be the interface IP address on the egress node.
port	This parameter specifies the port number of the real service. It must be the same as the port number listened by the virtual service on the egress node.
max_connection	Optional. This parameter specifies the maximum number of concurrent connections allowed by the real service. Its value must be an integer ranging from 0 to 4,294,967,295. When value 0 is specified, there is no limitation on the maximum number of concurrent connections. The maximum number of concurrent connections depends on the performance of the server providing the real service. If the set maximum number is larger than what the server can support, the exceeding connections cannot be established. The default value is 0.

slb real fwdmac <real_service> <output_interface> <real_mac> <port>
[max_connection]

This command is used to configure a FWDMAC real service. The FWDMAC real service is used to provide load balancing for security devices set up in L2 mode. It supports only ICMP, SNMP and TCP additional health checks. This type of real services can only be added to real service groups that use the hi, chi or rr method, and can be associated with TCPS virtual service only via the default policy. FWDMAC real services send the packets to the MAC address specified by this command through the defined output interface. The destination IP addresses in the packets do not change. The TCP port number will be changed to the one defined in this command.

real_service	This parameter specifies the name of the real service. Its value must be a case-sensitive string of 1 to 128 characters and must be enclosed in double quotes when containing special characters, which include the at sign (@), exclamation mark (!), tilde (~), underscore (_), colon (:), hyphen (-), angle brackets (< >), and period (.). The
--------------	--

parameter value cannot contain spaces or be the system reserved word “default”, “all”, or “global”, which is case-insensitive.

Note: The name of the real service cannot be the same as that of the virtual service.

output_interface	This parameter specifies the name of the interface that the real service uses to forward packets. Its value must be the system interface, VLAN interface or bond interface name.
real_mac	This parameter specifies the destination MAC address to which the security device sends packets. Its value must be in the format of “01:23:45:67:89:AB”.
port	This parameter specifies the port number of the real service. It must be the same as the port number listened by the virtual service on the egress node.
max_connection	Optional. This parameter specifies the maximum number of concurrent connections allowed by the real service. Its value must be an integer ranging from 0 to 4,294,967,295. When value 0 is specified, there is no limitation on the maximum number of concurrent connections. The maximum number of concurrent connections depends on the performance of the server providing the real service. If the set maximum number is larger than what the server can support, the exceeding connections cannot be established. The default value is 0.

no slb real

{http|tcp|ftp|udp|tcps|https|dns|dnstcp|siptcp|sipudp|rtsp|rdp|radauth|radacct|ip|l2ip|l2mac|diameter|fwdip|fwdmac|tuxedo} <real_service>

This command is used to delete the specified real service of the specific protocol type.

show slb real

{http|tcp|ftp|udp|tcps|https|dns|dnstcp|siptcp|sipudp|rtsp|rdp|radauth|radacct|ip|l2ip|l2mac|diameter|fwdip|fwdmac|tuxedo} [real_service]

This command is used to display the specified real service of the specific protocol type. If the “real_service” parameter is not specified, all real services of the specific protocol type will be displayed.



Note: If the “ip|domain” parameter in the commands “slb real http”, “slb real https”, “slb real ftp”, “slb real tcp”, “slb real tcps” and “slb real ip” set to

“domain”, the displayed information contains the IP address corresponding to the domain name of the real service when the administrator executes the command of a specific protocol type.

clear slb real

{http|tcp|ftp|udp|tcps|https|dns|dnstcp|siptcp|sipudp|rtsp|rdp|radauth|radacct|ip|l2ip|l2mac|diameter|fwdip|fwdmac|tuxedo}

This command is used to delete all real services of the specific protocol type.



Note: If the parameter "ip|domain" in the commands “**slb real http**,” “**slb real https**,” “**slb real ftp**,” “**slb real tcp**,” “**slb real tcps**,” and “**slb real ip**” is set to "domain," when the command deletes these real services, it also deletes the IP addresses corresponding to the domain name of these real services.

slb real {enable|disable} <real_service> [force_offline]

This command is used to enable or (gracefully/forcibly) disable a real service or reset all connections on a real service. By default, all real services are enabled.

After a real service is gracefully disabled, requests coming from clients it used to serve will still be forwarded to it, but requests coming from new clients will be forwarded to other available real services.

After a real service is forcibly disabled, the system will not forward any requests to it.

After all connections on this real service are reset, the real service will be disabled and no request will be forwarded to it.

force_offline Optional. This parameter specifies whether to forcibly disable the real service, and it is available for only the “**slb real disable**” command. Its value must be:

- 0: gracefully disables the real service.
- 1: forcibly disables the real service.
- 2: resets all connections to the real service.

The default value is 0.

The following gives an example of the Graceful Shutdown function:

```
AN(config)#slb real disable rs1
```

After disabling the real service named “rs1” using the command “**slb real disable**”, the administrator can check the status of the real service by using the command “**show statistics slb real**”.

```
AN(config)#show statistics slb real http rs1
```

```

Real service rs1 10.8.6.42 80 DOWN INACTIVE(waiting)
  Main health check: 10.8.6.42 80 tcp DOWN
  Max Conn Count:      1000
  Current Connection Count: 4572
  Outstanding Request Count: 4215
  Request Per Second:    245
  Total Hits:            311
  Total Bytes In:        39431
  Total Bytes Out:       53466
  Total Packets In:      7541
  Total Packets Out:     3252
  Average Bandwidth In:   543 Kbps
  Average Bandwidth Out:  748 bps
  Average Packets Out Rate: 667 pps
  Average Response time: 32.000 ms

```

As displayed in the above output information, the status of “service” is displayed as “INACTIVE(waiting)”, which means the real service is still processing connection requests or in other words in the process of “Gracefully Shutdown”. During this process, the requests from old clients will still be forwarded to this real service, while the requests from new clients will be forwarded to other available real services.

After a while, the administrator can check the status of the real service again by using the “**show statistics slb real**” command.

```

AN(config)#show statistics slb real http rs1
Real service rs1 10.8.6.42 80 DOWN INACTIVE(suspend)
  Main health check: 10.8.6.42 80 tcp DOWN
  Max Conn Count:      1000
  Current Connection Count: 0
  Outstanding Request Count: 0
  Request Per Second:    0
  Total Hits:            0
  Total Bytes In:        0
  Total Bytes Out:       0
  Total Packets In:      0
  Total Packets Out:     0
  Average Bandwidth In:   0 Kbps
  Average Bandwidth Out:  0 bps
  Average Packets Out Rate: 0 pps
  Average Response time: 0.000 ms

```

As displayed in the above output information, the status of “r1” now is displayed as “INACTIVE (suspend)”, which means that it has been disabled completely.

show slb real all

This command is used to display all real services in the system (including the disabled real services).



Note: If the “ip|domain” parameter in the commands “**slb real http**”, “**slb real https**”, “**slb real ftp**”, “**slb real tcp**”, “**slb real tcps**” and “**slb real ip**” set to “domain”, the IP address corresponding to the domain name of the real service

will be included in the displayed real services.

General Configurations of Real Service

slb real activation *<real_service>* *<recovery_time>* [*warm_up_time*]

This command is used to set the recovery and warm-up time for a real service. The time in which a real service changes from “Down” to “Up” is called “recovery time”. This real service will not receive any client requests during this period. After the recovery time, the real service will be in the warm-up time during which the real service receives client requests slowly. After the warm-up time, the real service can reach its maximum connections. If this command is not configured, the default recovery time and the default warm-up time are both 0 seconds.

<i>real_service</i>	This parameter specifies the name of the real service.
<i>recovery_time</i>	This parameter specifies the recovery time of the real service, in seconds. Its value must be an integer ranging from 0 to 3600.
<i>warm_up_time</i>	Optional. This parameter specifies the warm-up time of the real service, in seconds. Its value must be an integer ranging from 0 to 3600. If the parameter value is set to 0, the real service can reach its maximum connections immediately after the recovery time.

The administrator can check the status of a real service which has been just enabled by using the “**show statistics slb real**” command.

no slb real activation *<real_service>*

This command is used to reset the recovery and warm-up time setting of the specified real service to the default.

show slb real activation *<real_service>*

This command is used to display the recovery and warm-up time of the specified real service.

slb real application cps *<real_service>* *<max_cps>*

This command is used to set the maximum Connections per Second (CPS) for the specified real service to prevent application overload.

<i>real_service</i>	This parameter specifies the name of the real service.
<i>max_cps</i>	This parameter specifies the maximum CPS. Its value must be an integer ranging from 1 to 4,294,967,295.



Note: Currently, the maximum CPS can be set only for the TCP/TCPs, HTTP/HTTPS, FTP/FTPS, UDP or RDP real service. If the maximum CPS is set for a real service in the group using a persistence-type method, the persistence may be disrupted when the CPS on the real service reaches the maximum CPS.

no slb real application cps <real_service>

This command is used to delete the maximum CPS setting of the specified real service.

show slb real application cps [real_service]

This command is used to display the maximum CPS setting of the specified real service. If the “real_service” parameter is not specified, the maximum CPS settings of all real services will be displayed.

clear slb real application cps

This command is used to clear the maximum CPS settings of all real services.

slb real application bandwidth <real_service> <soft_bandwidth_limit> [hard_bandwidth_limit]

This command is used to set the bandwidth limit for the specified real service.

real_service	This parameter specifies the name of the real service.
soft_bandwidth_limit	This parameter specifies the soft bandwidth limit in Kbps. Its value must be an integer ranging from 1 to 4,294,967,295. If the used bandwidth of the real service exceeds the soft bandwidth limit, the system will generate a “Warning” log and continue to forward the requests that should hit this real service to this real service.
hard_bandwidth_limit	Optional. This parameter specifies the hard bandwidth limit in Kbps. Its value must be an integer ranging from 1 to 4,294,967,295. If the used bandwidth of the real service exceeds the hard bandwidth limit, the system will generate a “Warning” log and will not forward the requests that should hit this real service to this real service. The system will reselect a real service based on the group method instead. If this parameter is not specified, the value of the “soft_bandwidth_limit” parameter will be used by default.

no slb real application bandwidth <real_service>

This command is used to delete the bandwidth limit setting of the specified real service.

show slb real application bandwidth [*real_service*]

This command is used to display the bandwidth limit setting of the specified real service. If the “*real_service*” parameter is not specified, the bandwidth limit settings of all real services will be displayed.

clear slb real application bandwidth

This command is used to delete the bandwidth limit settings of all real services.

slb real settings keepdip <*real_service*>

This command is used to configure the specified real service to forward packets without changing the destination IP address.

<i>real_service</i>	This parameter specifies the real service name. Its value must be the name of a TCP, TCPS, HTTP or HTTPS real service.
---------------------	--

no slb real settings keepdip <*real_service*>

This command is used to delete the “**slb real setting keepdip**” command configuration for the specified real service.

show slb real settings keepdip <*real_service*>

This command is used to display the “**slb real setting keepdip**” command configuration for the specified real service.

clear slb real settings keepdip

This command is used to clear the “**slb real setting keepdip**” command configuration for all real services.

Real Service Group

Real service group is a set of real services that provide the same type of service. Client requests hitting this real service group will be forwarded to the real services selected based on the group method.

This section covers the commands for creating the real service group, setting the group method, adding group members, and configuring other group options.

Defining the Real Service Group and Group Method

slb group method <*group_name*> [*method*]

This command is used to create a real service group and set its group method.

group_name	This parameter specifies the name of the real service group. Its value must be a string of 1 to 128 characters. Its value must be enclosed in double quotes when containing any special character or containing only numbers.
method	Optional. This parameter specifies the group method used to load balance the traffic among the real services in the real service group. Its value must be: • rr: Round Robin • grr: Global Round Robin • sr: Shortest Response • lc: Least Connections* • lb: Least Bandwidth* • snmp: Simple Network Management Protocol (SNMP)* • pi: Persistent IP* • pto: Persistent TCP Option* • hi: Hash IP* • hip: Hash (IP+Port)* • chi: Consistent Hash IP* • hh: Hash Header* • chh: Consistent Hash Header* • ph: Persistent Hostname* • pu: Persistent URL* • hq: Hash Query* • ic: Insert Cookie* • hc: Hash Cookie* • pc: Persistent Cookie* • rc: Rewrite Cookie* • ec: Embed Cookie* • sslsid: SSL Session ID* • sipcid: SIP Call ID* • sipuid: SIP User ID* • sipcidps: SSL CallID Persistence Session* • sipuidps: SIP UserID Persistence Session* • rdprt: RDP Routing Token* • radchu: Consistent Hash RADIUS Username

- radchs: Consistent Hash RADIUS Session ID
- radpsun : RADIUS Persistence Session by Username*
- persistence: Individual Session Persistence*
- diametersid: Diameter Session ID*
- tuxedo: Tuxedo*
- orchestrator: Orchestrator

The default value is “rr”. For the preceding group methods marked with “*”, the administrator needs to configure extended parameters. For detailed description of these group methods, please refer to the corresponding commands in the following part.

The total number of real service groups (except for the group using the persistence method) depends on the system memory:

System Memory	Number of Real Service Groups
<= 32 GB	4000
>= 64 GB	8000

The total number of real service groups using the session ID-based general persistence (persistence) method depends on the system memory:

System Memory	Number of Real Service Groups
1 GB	40
2 GB, 4 GB, 8 GB	128
16 GB, 24 GB	1000
32 GB	4000
>= 64 GB	8000

The following part will describe the meaning of every group method and its configuration method.

slb group method <group_name> rr

This command is used to create a real service group using the Round Robin (rr) method. The rr method forwards the client request hitting this real service group to the real services in the round robin manner. When weights (not the default value 1) are assigned to the real services using the “**slb group member**” command, the client requests hitting this real service group will be forwarded to the real services in the weighted round robin manner.

group_name This parameter specifies the name of the real service group. Its value must be a string of 1 to 128 characters. Its value must be enclosed in double quotes when containing any special character or containing only numbers.

slb group method <group_name> grr

This command is used to create a real service group using the Global Round Robin (grr) method. Like the rr method, the grr method also forwards the client request hitting this real service group to the real services in the round robin manner. When weights (not the default value 1) are assigned to the real services using the “**slb group member**” command, the client requests hitting this real service group will be forwarded to the real services in the weighted round robin manner.

From the perspective of request distribution accuracy, the rr method applies to the single CPU environment, while the grr method applies to the multi-core CPU environment. In multi-core CPU environments, the rr method cannot guarantee that requests are distributed in proportion to the weights, and the ratio of requests sent to servers might not be correct. Using the grr method can solve this problem.

group_name	This parameter specifies the name of the real service group. Its value must be a string of 1 to 128 characters. Its value must be enclosed in double quotes when containing any special character or containing only numbers.
------------	---

slb group method <group_name> sr

This command is used to create a real service group using the Shortest Response (sr) method. The sr method forwards the client request hitting this real service group to the real service with the shortest response time.

group_name	This parameter specifies the name of the real service group. Its value must be a string of 1 to 128 characters. Its value must be enclosed in double quotes when containing any special character or containing only numbers.
------------	---

slb group method <group_name> lc [threshold_granularity] [yes|no]

This command is used to create a real service group using the Least Connections (lc) method. The lc method forwards the client request hitting this real service group to the real service with the least connections currently.

group_name	This parameter specifies the name of the real service group. Its value must be a string of 1 to 128 characters. Its value must be enclosed in double quotes when containing any special character or containing only numbers.
------------	---

threshold_granularity	Optional. This parameter specifies the granularity used to classify the number of current connections on the real services into different levels. If the number of current connections on only one real service is at the lowest level, the lc method forwards the client request hitting this real service group to this real service. If the number of current connections on more than one real service is at the lowest
-----------------------	---

level, the lc method forwards the client request hitting this real service group to the real service selected based on the setting of the “yes|no” parameter. Its value must be an integer ranging from 1 to 4,294,967,295. The default value is 10.

If this parameter is set to 10, the connection numbers 0 to 9 are classified into one level, the connection numbers 10 to 19 are classified into the next level, and so forth. Assume that the number of current connections on three real services in the real service group is 21, 11 and 9 respectively. The lc method will forward the client request to the real service whose number of current connections is 9.

yes|no

Optional. This parameter specifies whether to use the rr method to select one real service when the number of current connections on more than one real service is at the lowest level. Its value must be:

- yes: Uses. The system will use the rr method to forward the client requests to multiple real services until the number of current connections on only one real service is at the lowest level.
- no: Not use. The system will randomly select one real service from the real services whose number of current connections is at the lowest level and forward subsequent client requests to this real service until its number of current connections is not at the lowest level.

The default value is “no”.

Assume that the number of current connections on three real services (rs1, rs2 and rs3) in the real service group is 21, 15 and 11 respectively. If this parameter is set to “yes”, the system will forward the client requests hitting this real service group to rs2 and rs3 in the round robin manner until the number of current connections on any real service exceeds 19. If this parameter is set to “no”, the system will randomly select one real service (for example rs2) and forward the client requests hitting this real service group to rs2 until its number of current connections exceeds 19.

Note: If this parameter is set to “yes” and different weights are assigned to the real services using the “**slb group member**” command, the client requests hitting this real service group will be forwarded to the real services whose number of current connections is at the lowest level in the weighted round robin manner.

slb group method <group_name> lb [threshold_granularity]

This command is used to create a real service group using the Least Bandwidth (lb) method. The lb method forwards the client request hitting this real service group to the real service with the least bandwidth usage. The bandwidth usage of the real service can be calculated using the following formula:

Real service's bandwidth usage = Larger one of Inbound bandwidth and Outbound bandwidth ÷ (Bandwidth level granularity × Real service's weight)

group_name	This parameter specifies the name of the real service group. Its value must be a string of 1 to 128 characters. Its value must be enclosed in double quotes when containing any special character or containing only numbers.
threshold_granularity	Optional. This parameter specifies the granularity used to classify the bandwidths of the real services into different levels, in Kbps. Its value must be an integer ranging from 1 to 4,294,967,295. The default value is 10.

To achieve optimal load balancing effects, the administrator should make sure that the product of the bandwidth level granularity multiplying the real service's weight equals to the maximum bandwidth between the real service and the appliance. The real service's weight is configured using the “**slb group member**” command.



Note: The system calculates the inbound bandwidth and outbound bandwidth of the real service every second by using the last five seconds' data. The administrator can view the current inbound bandwidth and outbound bandwidth of the real service by executing the “**show statistics slb real**” command.

slb group method <group_name> snmp [snmp_mode] [community] [oid_count] [oid1] [oid1_weight] [oid2] [oid2_weight] [check_interval]

This command is used to create a real service group using the Simple Network Management Protocol (snmp) method. The snmp method supports two modes:

- CPU mode: The APV appliance obtains the CPU usage of every real service through SNMP every 60 seconds. The client request hitting this real service group will be forwarded to the real service with the lowest CPU usage.
- Weight mode: The APV appliance obtains the value of one or two SNMP OIDs of every real service through SNMP at the configured interval and then calculates the metric weight of every real service. The client request hitting this real service group will be forwarded to the real service with the lowest metric weight. The metric weight can be calculated in the following formula:

Metric weight = Value of OID1 × Weight of OID1 + Value of OID2 × Weight of OID 2

group_name	This parameter specifies the name of the real service group. Its value must be a string of 1 to 128 characters. Its value must be enclosed in double quotes when containing any special character or containing only numbers.
snmp_mode	Optional. This parameter specifies the SNMP mode. Its value must be: • cpu: indicates the CPU mode. When this parameter is set to “cpu”, only the “community” parameter needs to be specified. • weight: indicates the weight mode. When this parameter is set to “weight”, the “community” and other parameters need to be specified. The default value is “cpu”.
community	Optional. This parameter specifies the password for the SNMP network administrator to access the system. Its value must be a string of 1 to 32 characters. The default value is “public”.
oid_count	Optional. This specifies the number of OIDs used in the weight mode. Its value must be 1 or 2. The default value is 1. • If this parameter is set to 1, the parameters “oid1” and “oid1_weight” must be specified. • If this parameter is set to 2, the parameter “oid1”, “oid1_weight”, “oid2”, and “oid2_weight” must be specified. Note: This parameter needs to be specified only when the “snmp_mode” parameter is set to “weight”.
oid1	Optional. This parameter specifies the first SNMP OID used in the weight mode.
oid1_weight	Optional. This parameter specifies the weight of the first SNMP OID used in the weight mode. Its value must be an integer ranging from 1 to 4,294,967,295.
oid2	Optional. This parameter specifies the second SNMP OID used in the weight mode.
oid2_weight	Optional. This parameter specifies the weight of the second SNMP OID used in the weight mode. Its value must be an integer ranging from 1 to 4,294,967,295.
check_interval	Optional. This parameter specifies the interval of the SNMP check in the weight mode, in seconds. Its value

must be an integer ranging from 1 to 4,294,967,295. The default value is 60.

For example:

```
AN(config)#slb group method group1 snmp cpu public
AN(config)#slb group method group1 snmp weight public 2 .1.3.6.1.4.1.7564.30.1
1 .1.3.6.1.4.1.7564.4.2 1 60
```

slb group method <group_name> pi [hash_bits] [first_choice_method] [threshold_granularity]

This command is used to create a real service group using the Persistent IP (pi) method. The pi method forwards the client request hitting this real service group for the first time to one real service selected based on the “first choice method” and records the mapping between the real service and the hash value of the source IP address in the client request. When the subsequent client requests whose source IP addresses have the same hash value hit this real service group, the pi method forwards the client requests to the same real service persistently.

group_name	This parameter specifies the name of the real service group. Its value must be a string of 1 to 128 characters. Its value must be enclosed in double quotes when containing any special character or containing only numbers.
hash_bits	Optional. This parameter specifies the number of bits in the source IP address to be hashed. Its value must be an integer ranging from 0 to 32. The default value is 32.
first_choice_method	Optional. This parameter specifies the “first choice method”. If a client request hits the real service group for the first time, the system will select an available real service based on the “first choice method” for the client request. Its value must be “rr”, “grr”, “sr”, “lc”, or “lb”. The default value is “rr”.
threshold_granularity	Optional. This parameter is available only when the “first_choice_method” parameter is set to “lc” or “lb”. For the description of this parameter, please refer to the commands “ slb group method <group_name> lc ” and “ slb group method <group_name> lb ”.



Note:

- If a real service in the real service group becomes unavailable (disabled or faulty), the client requests that should be persisted to this real service previously will be treated as the client requests hitting this real service group for the first time. The system will select one real service from other available real services based on the pi method. Even if the unavailable real service restores, the system will still not forward the client requests to this

real service to which the client requests should be persisted previously.

The change of a real service's availability will not affect the persistence of other real services. That is, the client requests that should be persisted to other real services previously will still be forwarded to those real services.

- If a real service is deleted from the real service group, the client requests that should be persisted to this real service previously will be treated as the client requests hitting this real service group for the first time. The system will reselect one real service from other available real services based on the pi method. The client requests that should be persisted to other real services previously will still be forwarded to those real services.
- If a new real service is added to the real service group, when the hash table of this real service group is not full, the pi method may forward the new client requests hitting this real service group for the first time to the new real service. If the hash table is full, the pi method will forward the new client requests hitting this real service group for the first time only to the original real services. The administrator can execute the "**slb group flush** <group_name>" command to clear the hash table of this real service group so that client requests can be forwarded to the new real service. Please note that if the hash table of this real service group is cleared, the persistence of original real services will also be cleared.

slb group method <group name> **pto** <tcp_option_kind>
[first_choice_method]

This command is used to create a real service group using the Persistent TCP Option (pto) method. The pto method forwards the TCP request hitting this real service group for the first time to one real service selected based on the "first choice method", and hashes the content of the TCP Option of the specified "kind" value, and then records the mapping between the real service and the hash value. When receiving the subsequent TCP requests whose TCP option of the specified kind has the same hash value, the pto method forwards the client requests to the same real service persistently. The administrator can use "**slb persistence timeout**" command to configure the timeout value of the session persistence.

group_name	This parameter specifies the name of the real service group. Its value must be a string of 1 to 128 characters. Its value must be enclosed in double quotes when containing any special character or containing only numbers.
tcp_option_kind	This parameter specifies the kind value related to the specified TCP Option. Its value must be an integer ranging from 5 to 255.
first_choice_method	Optional. This parameter specifies the "first choice method". If a client request hits the real service group for the first time, the system will select an available real service based on the "first choice method" for the client request. Its value must be "rr", "grr", "sr", "lc", or "lb". The default value is "rr".

slb group method <group_name> hi [hash_bits]

This command is used to create a real service group using the Hash IP (hi) method. The hi method forwards the client request hitting this real service group for the first time to one real service selected based on the hash value of the source IP address and forwards the subsequent client requests whose source IP addresses have the same hash value to the same real service persistently. The persistence of the real service group using the hi method is affected by the number of available real services in the group. When the number of available real services in the real service group changes, the original persistence of the real services will become ineffective. The hi method will reselect real services for client requests based on the hash values of the source IP addresses.

group_name	This parameter specifies the name of the real service group. Its value must be a string of 1 to 128 characters. Its value must be enclosed in double quotes when containing any special character or containing only numbers.
hash_bits	Optional. This parameter specifies the number of bits in the source IP address to be hashed. Its value must be an integer ranging from 0 to 32. The default value is 32.

slb group method <group_name> hip [hash_bits]

This command is used to create a real service group using the Hash IP+port (hip) method. The hip method is similar to the hi method. The only difference between them is that the hip method calculates the hash value of the source IP address and port while the hi method calculates the hash value of the source IP address.

group_name	This parameter specifies the name of the real service group. Its value must be a string of 1 to 128 characters. Its value must be enclosed in double quotes when containing any special character or containing only numbers.
hash_bits	Optional. This parameter specifies the number of bits in the source IP address to be hashed. Its value must be an integer ranging from 0 to 32. The default value is 32.

slb group method <group_name> chi [hash_bits]

This command is used to create a real service group using the Consistent Hash IP (chi) method. The chi method forwards the client request hitting this real service group for the first time to one real service selected based on the hash value of the source IP address and forwards the subsequent client requests whose source IP addresses have the same hash value to the same real service persistently.

group_name	This parameter specifies the name of the real service group. Its value must be a string of 1 to 128 characters. Its value must be enclosed in double quotes when containing any
------------	---

special character or containing only numbers.

hash_bits Optional. This parameter specifies the number of bits in the source IP address to be hashed. Its value must be an integer ranging from 0 to 32. The default value is 32.



Note:

- If a real service in the real service group becomes unavailable (disabled or faulty), for the client requests that should be persisted to this real service previously, the chi method will hash the hash values of the source IP addresses to select real services. If the selected real service is still unavailable, the chi method will continue to hash the hash values to select real services. The chi method supports three times of hashing at most. If the real services selected based on three hash values are all unavailable, the chi method will select an available real service in the round robin manner. The client requests that should be persisted to other real services previously will still be forwarded to those real services.
- If a real service is deleted from or added to the real service group, the client requests that should be persisted to real services in this real service group previously will be treated as the client requests hitting this real service group for the first time. The system will reselect real services for those client requests based on the chi method.

slb group method *<group_name> hh <header_name> [first_choice_method] [threshold_granularity] [prefix] [delimiter]*

This command is used to create a real service group using the Hash Header (hh) method. The hh method forwards the client request hitting this real service group for the first time to one real service selected based on the “first choice method” and records the mapping between the real service and the hash value of the specified request header’s value. When the subsequent client requests whose specified request header’s values have the same hash value hit this real service group, the hh method forwards the client requests to the same real service persistently. The hh method can hash a part or the whole of the specified request header’s value.

group_name This parameter specifies the name of the real service group. Its value must be a string of 1 to 128 characters. Its value must be enclosed in double quotes when containing any special character or containing only numbers.

header_name This parameter specifies the name of the request header whose value will be hashed. Its value must be a string of 1 to 64 case-insensitive characters, and supports all printable ASCII characters (ASCII codes ranging from 33 to 126) except the space, double quotes and colon. Its value must be:

- Standard header or pseudo-header name: such as Accept, Accept-Charset, Accept-Language, Host,

Referer, User-Agent, or X-Forwarded-For, “:authority”

- Non-standard header name: any non-standard header name
- URL: URL “path+query” string in the HTTP request

For example, to hash the HTTP request URL’s “path+query” string, execute the command “slb group method ‘group1’ hh URL”.

first_choice_method	Optional. This parameter specifies the “first choice method”. If a client request hits the real service group for the first time, the system will select an available real service based on the “first choice method” for the client request. Its value must be “rr”, “grr”, “sr”, “lc”, or “lb”. The default value is “rr”.
threshold_granularity	Optional. This parameter is available only when the “first_choice_method” parameter is set to “lc” or “lb”. For the description of this parameter, please refer to the commands “ slb group method <group_name> lc ” and “ slb group method <group_name> lb ”.
prefix	Optional. This parameter specifies the start location of the string to be hashed in the request header’s value. Its value must be a string of 1 to 128 case-sensitive characters and must be enclosed in double quotes when starting with a non-letter character. The parameters “prefix” and “delimiter” together determine the string to be hashed in the request header’s value.
delimiter	Optional. This parameter specifies the delimiter of the string to be hashed in the request header’s value. Its value must be a character, which is case sensitive. Note: The administrator must specify both or neither of the parameters “prefix” and “delimiter”.



Note:

- If neither of the parameters “prefix” and “delimiter” is specified, the whole of the request header’s value will be hashed.
- The request header’s value can match “prefix” only when meeting either of the following conditions:
 - “prefix” exists at the beginning of the request header’s value.
 - “prefix” does not exist at the beginning of the request header’s value, but “delimiter” exists right before “prefix”. The spaces and tabs

between “prefix” and “delimiter” will be ignored.

- If the request header’s value matches both “prefix” and “delimiter”, the string between “prefix” and “delimiter” in the request header’s value will be hashed.
- If the request header’s value does not match “prefix”, the whole of the request header’s value will be hashed.
- If the request header’s value matches “prefix” but not “delimiter”, the string following “prefix” in the request header’s value will be hashed.
- If the configured “prefix” exists in multiple locations in the request header’s value, only the one existing in the first location will be matched.

For example:

To hash the string “13866668888” in the request header’s value “callid=13866668888; ber=12”, the administrator needs to set the “prefix” parameter to “callid=” and set the “delimiter” parameter to “;”.

```
AN(config)#slb group method g1 hh "test" rr 1 "callid=" ";"
```

To hash the string “13866668888” in the request header’s value “username=abc; callid=13866668888; ber=12”, the administrator needs to set the “prefix” parameter to “callid=” and set the “delimiter” parameter to “;”.

```
AN(config)#slb group method g1 hh "test" rr 1 "callid=" ";"
```

slb group method <group_name> chh <header_name>

This command is used to create a real service group using the Consistent Hash Header (chh) method. The chh method forwards the client request hitting this real service group for the first time based on the hash value of the specified request header’s value and forwards the subsequent client requests whose specified request header’s values have the same hash value to the same real service persistently. Different from the hh method, the chh method can hash only the whole of the specified request header’s value.

group_name This parameter specifies the name of the real service group. Its value must be a string of 1 to 128 characters. Its value must be enclosed in double quotes when containing any special character or containing only numbers.

header_name This parameter specifies the name of the request header whose value will be hashed. Its value must be a string of 1 to 64 case-insensitive characters, and supports all printable ASCII characters (ASCII codes ranging from 33 to 126) except the space, double quotes and colon. Its value must be:

- Standard header or pseudo-header name: Accept, Accept-Charset, Accept-Language, Host, Referer, User-Agent and X-Forwarded-For, “:authority”

- Extension header name: all extension header names

**Note:**

- If a real service in the real service group becomes unavailable (disabled or faulty), for the client requests that should be persisted to this real service previously, the chh method will hash the hash values of the request header's value to select real services. If the selected real service is still unavailable, the chh method will continue to hash the hash values to select real services. The chh method supports three times of hashing at most. If the real services selected based on three hash values are all unavailable, the chh method will select an available real service in the round robin manner. The client requests that should be persisted to other real services previously will still be forwarded to those real services.
- If a real service is deleted from or added to the real service group, the client requests that should be persisted to the real services in this real service group previously will be treated as the client requests hitting this real service group for the first time. The system will reselect real services for the client requests based on the chh method.

**slb group method <group_name> ph [first_choice_method]
[threshold_granularity]**

This command is used to create a real service group using the Persistent Hostname (ph) method. The ph method forwards the client request hitting this real service group for the first time to one real service selected based on the “first choice method” and records the mapping between the real service and the hash value of the host name (the Host header's value). When the subsequent client requests whose host names have the same hash value hit this real service group, the ph method forwards the client requests to the same real service persistently.

group_name	This parameter specifies the name of the real service group. Its value must be a string of 1 to 128 characters. Its value must be enclosed in double quotes when containing any special character or containing only numbers.
first_choice_method	Optional. This parameter specifies the “first choice method”. If a client request hits the real service group for the first time, the system will select an available real service based on the “first choice method” for the client request. Its value must be “rr”, “grr”, “sr”, “lc”, or “lb”. The default value is “rr”.
threshold_granularity	Optional. This parameter is available only when the “first_choice_method” parameter is set to “lc” or “lb”. For the description of this parameter, please refer to the commands “ slb group method <group_name> lc ” and “ slb group method <group_name> lb ”.

slb group method <group_name> pu

This command is used to create a real service group using the Persistent URL (pu) method. The pu method forwards the client request hitting this real service group to the real service whose URL tag value is the same as that carried in the request.

The pu method needs to be used together with the Persistent URL policy (configured using the “**slb policy persistent url**” command). The name of the URL tag whose value will be used for matching is specified by the Persistent URL policy and the URL tag value of the real service is specified by the “url_tag_value” parameter in the “**slb group member**” command.

group_name	This parameter specifies the name of the real service group. Its value must be a string of 1 to 128 characters. Its value must be enclosed in double quotes when containing any special character or containing only numbers.
------------	---

**slb group method <group_name> hq [first_choice_method]
[threshold_granularity]**

This command is used to create a real service group using the Hash Query (hq) method. The hq method forwards the client request hitting this real service group for the first time to one real service selected based on the “first choice method” and records the mapping between the real service and the hash value of the specified URL tag’s value. When the subsequent client requests whose specified URL tag’s values have the same hash value hit this real service group, the hq method forwards the client requests to the same real service persistently.

The hq method needs to be used together with the Persistent URL policy (configured using the “**slb policy persistent url**” command). The name of the URL tag whose value will be hashed is specified by the Persistent URL policy

group_name	This parameter specifies the name of the real service group. Its value must be a string of 1 to 128 characters. Its value must be enclosed in double quotes when containing any special character or containing only numbers.
------------	---

first_choice_method	Optional. This parameter specifies the “first choice method”. If a client request hits the real service group for the first time, the system will select an available real service based on the “first choice method” for the client request. Its value must be “rr”, “grr”, “sr”, “lc”, or “lb”. The default value is “rr”.
---------------------	--

threshold_granularity	Optional. This parameter is available only when the “first_choice_method” parameter is set to “lc” or “lb”. For the description of this parameter, please refer to the commands “ slb group method <group_name> lc ” and “ slb group method <group_name> lb ”.
-----------------------	--

slb group method <group_name> ic [cookie_name] [path_attribute]
[first_choice_method] [threshold_granularity]

This command is used to create a real service group using the Insert Cookie (ic) method. The ic method forwards the client request hitting this real service group for the first time to one real service selected based on the “first choice method” and lets the APV appliance insert a Set-Cookie header into the response returned by the real service to mark the real service. When the subsequent client requests that carry the cookie inserted by the APV appliance hit this real service group, the ic method forwards the client requests to the same real service persistently.

The Set-Cookie header contains the header name (Set-Cookie), cookie name/value pair, and several cookie attributes. The header name and the cookie name/value pair are separated by colons, and the cookie name/value pair and cookie attributes are separated by semi colons.

group_name	This parameter specifies the name of the real service group. Its value must be a string of 1 to 128 characters. Its value must be enclosed in double quotes when containing any special character or containing only numbers.
cookie_name	Optional. This parameter specifies the cookie name in the Set-Cookie header. Its value must be a string of 1 to 128 characters. If this parameter is not specified, the system will generate a cookie name randomly. The value of the inserted cookie marks the real service selected based on the “first choice method”. The cookie value is determined by the “slb mode ircookie <ircookie_mode>” command. By default, the ASCII value of the real service name will be used.
path_attribute	Optional. This parameter specifies the path attribute of the cookie. Its value must be: • 0: indicates that the path attribute of the cookie is not inserted. • 1: indicates that “path=/" will be inserted as the path attribute of the cookie. The default value is 0.
first_choice_method	Optional. This parameter specifies the “first choice method”. If a client request hits the real service group for the first time, the system will select an available real service based on the “first choice method” for the client request. Its value must be “rr”, “grr”, “sr”, “lc”, or “lb”. The default value is “rr”.
threshold_granularity	Optional. This parameter is available only when the “first_choice_method” parameter is set to “lc” or “lb”. For

the description of this parameter, please refer to the commands “**slb group method** <group_name> **lc**” and “**slb group method** <group_name> **lb**”.

**Note:**

- When the name of the cookie carried by the client request is the same as that of the inserted cookie but the cookie value does not match any real service name, the ic method will select a real service based on the “first choice method”.
- The ic method is applicable to the scenario where real services cannot insert cookies.

slb group option ic <group_name> <cookie_attribute>

This command is used to configure attributes of the inserted cookie for the specified real service group using the ic method. The attributes of the cookie determine the application scope of the cookie.

group_name	This parameter specifies the name of an existing real service group.
cookie_attribute	This parameter specifies the attribute(s) of the inserted cookie. The following cookie attributes are supported: • “expires”: indicates the expiration time of the cookie. The maximum value of this attribute is 5,256,000 minutes, that is, 10 years. This attribute must be expressed in the format of “expires=days:hours:minutes” or “expires=minutes”. For example, “expires=3” indicates that the expiration time of the cookie is 3 minutes; “expires=2:3” indicates that the expiration time of the cookie is 2 hours and 3 minutes, that is 123 minutes; “expires=1:2:3” indicates that the expiration time of the cookie is 1 day, 2 hours and 3 minutes, that is 1563 minutes. The client request will carry the returned cookie only when the cookie does not expire. • “path”: indicates the path with which the cookie is associated. This attribute supports a maximum of 128 characters without any space. It must be expressed in the format of “path=string”, in which the “string” is case sensitive. Only the client request that matches the configured path string will carry the returned cookie. • “domain”: indicates the domain with which the cookie is associated. This attribute supports a maximum of 128 characters without any space. It must be expressed in the format of “domain=string”, in which the “string” is case sensitive. Only the client request that matches the configured domain string will carry the returned

cookie.

- “secure”: indicates the allowed transmission mode of the cookie. This attribute must be expressed in the format of “secure=yes|no”. When “secure=yes” is specified, the cookie can be transmitted only when the browser and the real service use the HTTPS or other secure protocols. When “secure=no” is specified, the cookie can still be transmitted when the HTTP or other insecure protocols are used.
- “httponly”: indicates whether the cookie can be applied only to the HTTP requests. This attribute must be expressed in the format of “httponly=yes|no”.
- “samesite”: indicates whether to carry cookie in cross-site access scenarios. This attribute must be expressed in the format of “samesite=strict|lax|none”. “strict” only attaches cookies for “same-site” requests. This mode applies to high security requirement applications. “lax” sends cookies for “same-site” requests and “cross-site” top-level navigations using a safe HTTP method. “none” sends cookies for all “same-site” and “cross-site” requests. When the “samesite=none” attribute is inserted, the Secure attribute will also be appended.

This parameter value must be enclosed in double quotes. Its value can contain one or multiple cookie attributes, which are separated by commas.



Note:

- If “secure=yes” is specified, no matter whether the “**http rewrite response cookie secure icookie on**” command is configured, the Secure attribute will always be inserted into the Set-Cookie header. In addition, if you want the cookie inserted for the real service group to be transferred only through the secure protocols, “secure=yes” must be specified.
- Both the commands “**slb group method <group_name> ic**” and “**slb group option ic**” can be used to configure the path attribute for the cookie inserted for the specified real service group. Once the “**slb group option ic**” command has been configured for the real service group, no matter whether the “path” attribute is configured, the system will ignore the setting of the “path_attribute” parameter in the “**slb group method <group_name> ic**” command.

no slb group option ic <group_name>

This command is used to delete the cookie attribute settings of the specified real service group using the ic method.

show slb group option ic [group_name]

This command is used to display the cookie attribute settings of the specified real service group using the ic method. If the “group_name” parameter is not specified, the cookie attribute settings of all real service groups using the ic method will be displayed.

clear slb group option ic [group_name]

This command is used to clear the cookie attribute settings of the specified real service group using the ic method. If the “group_name” parameter is not specified, the cookie attribute settings of all real service groups using the ic method will be cleared.

**slb group method <group_name> rc <cookie_name> [offset]
[first_choice_method] [threshold_granularity]**

This command is used to create a real service group using the Rewrite Cookie (rc) method. The rc method forwards the client request hitting this real service group for the first time to one real service selected based on the “first choice method” and lets the APV appliance to rewrite a part of the specified cookie’s value in the response returned from the real service to mark the real service (“!!” will be inserted to mark the end of the rewriting). When the subsequent client requests hit this real service group, the rc method finds the real service information from the specified cookie and forwards the client requests to the same real service persistently.

group_name	This parameter specifies the name of the real service group. Its value must be a string of 1 to 128 characters. Its value must be enclosed in double quotes when containing any special character or containing only numbers.
cookie_name	This parameter specifies the name of the cookie whose value will be rewritten. A part of the cookie’s value will be replaced by the information marking the real service and “!!”. The information marking the real service is determined by the “ slb mode ircookie <ircookie_mode> ” command. By default, the ASCII value of the real service name will be used.
offset	Optional. This parameter specifies the start location from which the cookie’s value is rewritten, in bytes. Its value must be an integer ranging from 0 to 4,294,967,295. The default value is 0. Assume that the real service “r1” is selected based on the “first choice method”, and the value of the cookie carried in the response is “123abcd”. If this parameter is set to 3, the cookie’s value will be rewritten as “123r1!!”.
first_choice_method	Optional. This parameter specifies the “first choice method”. If a client request hits the real service group for the first time, the system will select an available real service based on the “first choice method” for the client request. Its value must be “rr”, “grr”, “sr”, “lc”, or “lb”.

The default value is “rr”.

threshold_granularity Optional. This parameter is available only when the “first_choice_method” parameter is set to “lc” or “lb”. For the description of this parameter, please refer to the commands “**slb group method** <group_name> **lc**” and “**slb group method** <group_name> **lb**”.



Note:

- If the length of the cookie’s value in the response returned by the real service is less than the sum of the real service information’s length, the offset and 2, the rc method will not rewrite the cookie’s value.
- The rc method is applicable to the scenario where real services in the real service group return cookies with the same name but different values.

slb group method <group_name> **ec** <cookie_name> [*first_choice_method*] [*threshold_granularity*]

This command is used to create a real service group using the Embed Cookie (ec) method. The ec method forwards the client request hitting this real service group for the first time to one real service selected based on the “first choice method” and lets the APV appliance to embed the real service information before the specified cookie’s value (“!!” will be inserted to mark the end of the embedding) in the response returned from the real service. When the subsequent client requests hit this real service group, the ec method finds the real service information from the specified cookie and forwards the client requests to the same real service persistently.

group_name This parameter specifies the name of the real service group. Its value must be a string of 1 to 128 characters. Its value must be enclosed in double quotes when containing any special character or containing only numbers.

cookie_name This parameter specifies the name of the cookie. The information marking the real service and “!!” will be embedded before the cookie value. The information marking the real service is determined by the “**slb mode ircookie** <ircookie_mode>” command. By default, the ASCII value of the real service name will be used.

first_choice_method Optional. This parameter specifies the “first choice method”. If a client request hits the real service group for the first time, the system will select an available real service based on the “first choice method” for the client request. Its value must be “rr”, “grr”, “sr”, “lc”, or “lb”. The default value is “rr”.

threshold_granularity Optional. This parameter is available only when the “first_choice_method” parameter is set to “lc” or “lb”. For the description of this parameter, please refer to the commands “**slb group method** <group_name> **lc**” and

“slb group method <group_name> lb”.



Note:

- The ec method is applicable to the scenario where real services in the real service group return cookies with the same name but different values.
- Different from the rc method, before forwarding the client request to the real service, the ec method will restore the value of the cookie returned by the real service, that is, remove the embedded information marking the real service and “!!” from the cookie’s value.

**slb group method <group_name> hc [first_choice_method]
[threshold_granularity]**

This command is used to create a real service group using the Hash Cookie (hc) method. The hc method needs to be used together with the Persistent Cookie policy (configured using the “**slb policy persistent cookie**” command). The hc method forwards the client request hitting this real service group for the first time to one real service selected based on the “first choice method”, obtains the value of the cookie specified by the Persistent Cookie policy from the request and record the hash value of the cookie’s value. When the subsequent client requests whose cookies’ values have the same hash value hit this real service group, the hc method forwards the client requests to the same real service persistently.

group_name	This parameter specifies the name of the real service group. Its value must be a string of 1 to 128 characters. Its value must be enclosed in double quotes when containing any special character or containing only numbers.
first_choice_method	Optional. This parameter specifies the “first choice method”. If a client request hits the real service group for the first time, the system will select an available real service based on the “first choice method” for the client request. Its value must be “rr”, “grr”, “sr”, “lc”, or “lb”. The default value is “rr”.
threshold_granularity	Optional. This parameter is available only when the “first_choice_method” parameter is set to “lc” or “lb”. For the description of this parameter, please refer to the commands “ slb group method <group_name> lc ” and “ slb group method <group_name> lb ”.

slb group method <group_name> pc [offset]

This command is used to create a real service group using the Persistent Cookie (pc) method. The pc method needs to be used together with the Persistent Cookie policy (configured using the “**slb policy persistent cookie**” command). When the client request hits this real service group, the pc method obtains the value of the cookie specified by the Persistent Cookie policy and forwards the request to the specified real service with the same cookie value.

The cookie value of the real service is configured using the “cookie_value” parameter in the “**slb group member**” command.

group_name This parameter specifies the name of the real service group. Its value must be a string of 1 to 128 characters. Its value must be enclosed in double quotes when containing any special character or containing only numbers.

offset Optional. This parameter specifies the start location from which the cookie value will be obtained, in bytes. Its value must be an integer ranging from 0 to 4,294,967,295. The default value is 0.

Assume that the value of the cookie in the client request is “123abc”. If this parameter is set to 3, the string “abc” will be used as the cookie value in the request to match with the cookie values of the real services.



Note: The pc method is applicable to the scenario where real services in the real service group return cookies with the same name and with fixed but different values.

slb group method <group_name> sslsid [timeout]

This command is used to create a real service group using the SSL Session ID (sslsid) method. The sslsid method forwards the client request hitting this real service group for the first time based on the rr method and records the mapping between the real service and the SSL session ID that the real service assigns to the client. When the subsequent client requests carrying the same SSL session ID hit this real service group, the sslsid method forwards the client requests to the same real service persistently.

group_name This parameter specifies the name of the real service group. Its value must be a string of 1 to 128 characters. Its value must be enclosed in double quotes when containing any special character or containing only numbers.

timeout Optional. This parameter specifies the timeout value of the session entry recording the mapping between the real service and the SSL session ID, in minutes. Its value must be an integer ranging from 1 to 4,294,967,295. The default value is 5.

The session entry will be deleted when it times out.



Note:

- The real service group using the sslsid method can contain only real services of the TCP type and can be associated only with the TCP-type virtual service using the Default policy.
- To make multiple requests to have the same session ID, users can enable

the reuse function of the client browser by:

- Clicking the New Tab Page on Google Chrome.
- Clicking the New Tab Page in 60 seconds after one request is sent on Internet Explorer.
- The TLSv1.3 protocol does not support the resumption mode based on session ID. Therefore, the sslsid method does not apply to TLSv1.3 applications.

slb group method <group_name> {sipcid|sipuid} [first_choice_method] [threshold_granularity]

This command is used to create a real service group using the SIP Call ID (sipcid) or SIP User ID (sipuid) method.

The sipcid method forwards the client INVITE request hitting this real service group for the first time to one real service selected based on the “first choice method” and records the mapping between the real service and the hash value of the Call-ID header’s value in the INVITE request. When the subsequent client INVITE requests whose Call-ID header’s values have the same hash value hit this real service group, the sipcid method forwards the client requests to the same real service persistently.

The sipuid method forwards the client INVITE request hitting this real service group for the first time to one real service selected based on the “first choice method” and records the mapping between the real service and the hash value of the From header’s value in the INVITE request. When the subsequent client INVITE requests whose From header’s values have the same hash value hit this real service group, the sipuid method forwards the client requests to the same real service persistently.

group_name	This parameter specifies the name of the real service group. Its value must be a string of 1 to 128 characters. Its value must be enclosed in double quotes when containing any special character or containing only numbers.
first_choice_method	Optional. This parameter specifies the “first choice method”. If a client request hits the real service group for the first time, the system will select an available real service based on the “first choice method” for the client request. Its value must be “rr”, “grr”, “sr”, “lc”, “lb”, “hi” or “chi”. The default value is “rr”.
threshold_granularity	Optional. This parameter is available only when the “first_choice_method” parameter is set to “lc” or “lb”. For the description of this parameter, please refer to the commands “ slb group method <group_name> lc” and “ slb group method <group_name> lb”.



Note: The real service group using the sipcid or sipuid method can contain only real services of the SIP TCP type or the SIP TCP type.

slb group method <group_name> **sipcidps** [first_choice_method]
[threshold_granularity]

This command is used to create a real service group using the SIP CallID Persistence Session (sipcidps) method. The sipcidps method forwards the client INVITE request hitting this real service group for the first time to the real service selected based on the “first choice method”, and records the mapping between the real service and the INVITE request’s Call-ID header value. When subsequent client INVITE requests with the same Call-ID header value hit this real service group, the sipcidps method forwards the client requests to the same real service persistently. The administrator can use “**slb persistence timeout**” command to configure the timeout value of the session persistence.

group_name	This parameter specifies the name of the real service group. Its value must be a string of 1 to 128 characters. Its value must be enclosed in double quotes when containing any special character or containing only numbers.
first_choice_method	Optional. This parameter specifies the “first choice method”. If a client request hits the real service group for the first time, the system will select an available real service based on the “first choice method” for the client request. Its value must be “rr”, “grr”, “sr”, “lc”, “lb”, “hi” or “chi”. The default value is “rr”.
threshold_granularity	Optional. This parameter is available only when the “first_choice_method” parameter is set to “lc” or “lb”. For the description of this parameter, please refer to the commands “ slb group method <group_name> lc ” and “ slb group method <group_name> lb ”.



Note: The real service group using the sipcidps method can contain only SIP TCP or SIP UDP real services, and can be associated only with SIP TCP or SIP UDP virtual services.

slb group method <group_name> **sipuidps** [first_choice_method]
[threshold_granularity]

This command is used to create a real service group using the SIP UserID Persistence Session (sipuidps) method. The sipuidps method forwards the client INVITE request hitting this real service group for the first time to the real service selected based on the “first choice method”, and records the mapping between the real service and the INVITE request’s From header value. When the subsequent client INVITE requests with the same From header value hit this real service group, the sipuidps method forwards the client requests to the same real service persistently. The administrator can use “**slb persistence timeout**” command to configure the timeout value of the session persistence.

group_name	This parameter specifies the name of the real service group. Its value must be a string of 1 to 128 characters. Its value must be enclosed in double quotes when containing any special character or containing only numbers.
first_choice_method	Optional. This parameter specifies the “first choice method”. If a client request hits the real service group for the first time, the system will select an available real service based on the “first choice method” for the client request. Its value must be “rr”, “grr”, “sr”, “lc”, “lb”, “hi” or “chi”. The default value is “rr”.
threshold_granularity	Optional. This parameter is available only when the “first_choice_method” parameter is set to “lc” or “lb”. For the description of this parameter, please refer to the commands “ slb group method <group_name> lc ” and “ slb group method <group_name> lb ”.



Note: The real service group using the sipuidps method can only contain SIP TCP or SIP UDP real services, and can only be associated with SIP TCP or SIP UDP virtual services.

slb group method <group_name> radchu

This command is used to create a real service group using the Consistent Hash RADIUS Username (radchu) method. The radchu method forwards the client request hitting this real service group for the first time based on the hash value of the RADIUS username and forwards the subsequent client requests whose RADIUS usernames have the same hash value to the same real service persistently.

group_name	This parameter specifies the name of the real service group. Its value must be a string of 1 to 128 characters. Its value must be enclosed in double quotes when containing any special character or containing only numbers.
------------	---



Note: The real service group using the radchu method can contain only real services of the RADIUS authentication type or the RADIUS accounting type

slb group method <group_name> radchs

This command is used to create a real service group using the Consistent Hash RADIUS Session ID (radchs) method. The radchs method forwards the client request hitting this real service group for the first time based on the hash value of the RADIUS session ID and forwards the subsequent client requests whose RADIUS session IDs have the same hash value to the same real service persistently.

group_name This parameter specifies the name of the real service group. Its value must be a string of 1 to 128 characters. Its value must be enclosed in double quotes when containing any special character or containing only numbers.



Note: The real service group using the radchs method can contain only real services of the RADIUS authentication type or the RADIUS accounting type.

slb group method <group_name> radpsun [first_choice_method]

This command is used to create a real service group using the RADIUS Persistence Session by Username (radpsun) method. The radpsun method forwards the RADIUS request hitting this real service group for the first time to the real service selected based on the “first choice method”, and records the mapping between the real service and the hash value of the RADIUS username. When subsequent RADIUS requests with the same username hash value hit this real service group, the radpsun method forwards the client requests to the same real service persistently, and refreshes the timeout value of this RADIUS session every time forwards the request. The administrator can use “slb persistence timeout” command to configure the timeout value of the session persistence.

To tackle the session synchronization issue in HA scenario, a new first choice method “ss”(session synchronization) is specifically designed for the radpsun method. “ss” first choice method can only be used together with the radpsun method, and can help keep the RADIUS sessions in the active HA unit aligned with those in the standby HA unit.

group_name This parameter specifies the name of the real service group. Its value must be a string of 1 to 128 characters. Its value must be enclosed in double quotes when containing any special character or containing only numbers.

first_choice_method Optional. This parameter specifies the “first choice method”. If a client request hits the real service group for the first time, the system will select an available real service based on the “first choice method” for the client request. Its value must be “rr”, “grr”, “sr”, “lc”, “lb” or “ss”.

The default value is “rr”.



Note:

1. For a long RADIUS user name, the radpsun method takes only the preceding 64 characters of it to calculate session persistence.
2. When using the “ss” first choice method, the RADIUS session information can be aligned between the active and standby HA units only if the real services added in the radpsun service group are configured the same on the active and standby.

slb group method <group_name> radchi [hash_bits]

This command is used to create a real service group using the RADIUS Consistent Hash IP (radchi) method. When a RADIUS authentication request hits the radchi group for the first time, the radchi group will select a real service based on the hash value of the IP address of the Framed-IP-Address field and then forward subsequent RADIUS authentication requests whose Framed-IP-Address field have the same hash value to the same real service.

group_name	This parameter specifies the name of the real service group. Its value must be a string of 1 to 128 characters. Its value must be enclosed in double quotes when containing any special character or containing only numbers.
hash_bits	Optional. This parameter specifies the number of bits in the source IP address to be hashed. Its value must be an integer ranging from 0 to 32. The default value is 32.

slb group method <group_name> rdprt [first_choice_method] [threshold_granularity]

This command is used to create a real service group using the RDP Routing Token (rdprt) method. The rdprt method forwards the client request hitting this real service group for the first time to one real service selected based on the “first choice method”. When the subsequent client requests hit this real service group, the rdprt method forwards the client requests to the corresponding real service according to the real service IP address and port information contained in the Routing Tokens of these clients.

group_name	This parameter specifies the name of the real service group. Its value must be a string of 1 to 128 characters. Its value must be enclosed in double quotes when containing any special character or containing only numbers.
first_choice_method	Optional. This parameter specifies the “first choice method”. If a client request hits the real service group for the first time, the system will select an available real service based on the “first choice method” for the client request. Its value must be “rr”, “grr”, “sr”, “lc”, or “lb”. The default value is “rr”.
threshold_granularity	Optional. This parameter is available only when the “first_choice_method” parameter is set to “lc” or “lb”. For the description of this parameter, please refer to the commands “ slb group method <group_name> lc ” and “ slb group method <group_name> lb ”.



Note: The real service group using the rdprt method can contain only real services of the RDP type.

slb group method <group_name> **persistence** <session_id_type>
[first_choice_method] [threshold_granularity]

This command is used to create a real service group using the session ID-based general persistence (persistence) method.

When the client request containing a certain session ID hits this real service group for the first time, the persistence method forwards the client request to one real service selected based on the “first choice method” and creates a dynamic session entry to record the mapping between the real service and the session ID. When the subsequent client requests carrying the same session ID hit this real service group, the persistence method forwards the client requests to the same real service persistently.

The persistence method supports the following types of session IDs:

- Client IP address
- Client IP address and port
- SSL session ID
- String

group_name	This parameter specifies the name of the real service group. Its value must be a string of 1 to 128 characters. Its value must be enclosed in double quotes when containing any special character or containing only numbers.
session_id_type	This parameter specifies the session ID type. Its value must be “ip”, “ipport”, “sslsid” or “string”.
first_choice_method	Optional. This parameter specifies the “first choice method”. If a client request hits the real service group for the first time, the system will select an available real service based on the “first choice method” for the client request. Its value must be “rr”, “grr”, “sr”, “lc”, or “lb”. The default value is “rr”.
threshold_granularity	Optional. This parameter is available only when the “first_choice_method” parameter is set to “lc” or “lb”. For the description of this parameter, please refer to the commands “ slb group method <group_name> lc ” and “ slb group method <group_name> lb ”.

The real service group can obtain the string-type session ID from any of the following locations:

- Specified tag’s value in the URL query part of the HTTP request
- Specified cookie’s value in the HTTP request
- A part or the whole of the specified header’s value in the HTTP request

- Body in the HTTP request (Note: HTTP/2 real service group does not support persistence based on a string obtained from the request body.)
- Specified cookie's value in the HTTP response
- A part or the whole of the specified header's value in the HTTP response
- Body in the HTTP response

The real service group will obtain a string from the preceding locations and obtain the session ID from the string based on the settings of the parameters “offset” and “session_id_length” in the “**slb group persistence value**” command.

**Note:**

- The session ID obtained from the real service group has a higher priority than the session ID obtained from the policy. For the string-type session ID, when the system fails to obtain a string from the real service group as the session ID, the system supports obtaining a string from the load balancing policy as the session ID if load balancing policy is configured,. The real service group supports the following policies:
 - 1、pc (Persistent Cookie)
 - 2、pu (Persistent URL)
 - 3、qc (Qos Cookie)
 - 4、qu (Qos URL)
 - 5、qh (Qos Hostname)
 - 6、qb (Qos Body)
 - 7、hh (Hash Header)
 - 8、hu (Hash URL)
- When the session persistence timeout of the real service group is not configured by executing the command “**slb persistence timeout**”, the default timeout value is 5 minutes.

The followings commands describe how to extract a string from every preceding location for obtaining the session ID.

slb group persistence request urlquery <group_name> <tag_name>

This command is used to configure the specified real service group to extract the string for obtaining the session ID from the specified tag's value in the URL query part of the HTTP request. The real service group will extract the string from the specified tag value to obtain the session ID based on the settings in the “**slb group persistence value**” command.

group_name	This parameter specifies the name of an existing real service group.
------------	--

tag_name	This parameter specifies the name of the tag in the URL query part. Its value must be a string of 1 to 64 case-insensitive characters.
----------	--

no slb group persistence request urlquery <group_name>

This command is used to delete the configuration of extracting the string for obtaining the session ID from the specified tag's value in the URL query part of the HTTP request for the specified real service group.

show slb group persistence request urlquery [group_name]

This command is used to display the configuration of extracting the string for obtaining the session ID from the specified tag's value in the URL query part of the HTTP request for the specified real service group. If the "group_name" parameter is not specified, the configurations of extracting the string for obtaining the session ID from the specified tag's value in the URL query part of the HTTP request for all real service groups will be displayed.

clear slb group persistence request urlquery

This command is used to clear the configurations of extracting the string for obtaining the session ID from the specified tag's value in the URL query part of the HTTP request for all real service groups.

slb group persistence request cookie <group_name> <cookie_name>

This command is used to configure the specified real service group to extract the string for obtaining the session ID from the specified cookie's value in the HTTP request. The real service group will extract the string from the specified cookie value to obtain the session ID based on the settings in the "**slb group persistence value**" command.

group_name	This parameter specifies the name of an existing real service group.
cookie_name	This parameter specifies the name of the cookie. Its value must be a string of 1 to 64 case-insensitive characters.

no slb group persistence request cookie <group_name>

This command is used to delete the configuration of extracting the string for obtaining the session ID from the specified cookie's value in the HTTP request for the specified real service group.

show slb group persistence request cookie [group_name]

This command is used to display the configuration of extracting the string for obtaining the session ID from the specified cookie's value in the HTTP request for the specified real service group. If the "group_name" parameter is not specified, the

configurations of extracting the string for obtaining the session ID from the specified cookie's value in the HTTP request for all real service groups will be displayed.

clear slb group persistence request cookie

This command is used to clear the configurations of extracting the string for obtaining the session ID from the specified cookie's value in the HTTP request for all real service groups.

slb group persistence request header <group_name> <header_name>
[prefix] [delimiter] [flag]

This command is used to configure the specified real service group to extract the string for obtaining the session ID from a part or the whole of the specified header's value in the HTTP request. The real service group will extract the string from a part or the whole of the specified header value to obtain the session ID based on the settings in the “**slb group persistence value**” command.

group_name	This parameter specifies the name of an existing real service group.
header_name	This parameter specifies the name of the HTTP header. Its value must be a string of 1 to 64 case-insensitive characters and supports all printable ASCII characters (ASCII codes ranging from 33 to 126) except the space, double quotes and colon. Its value must be: • Standard header or pseudo-header name: Accept, Accept-Charset, Accept-Language, Host, Referer, User-Agent and X-Forwarded-For, “:authority” • Extension header name: all extension header names
prefix	Optional. This parameter specifies the start location of the string from which the session ID will be obtained in the HTTP header's value. Its value must be a case-sensitive string of 1 to 32 characters and must be enclosed in double quotes when starting with a non-letter character. If its value includes a double quote, use “%q” instead.
delimiter	This parameter specifies the delimiter of the string in the HTTP header's value from which the session ID will be obtained. Its value must be one case-sensitive character enclosed in double quotes. If the “prefix” parameter is specified and the “flag” parameter is set to 0, this parameter must be specified.
flag	Optional. This parameter specifies whether the value of the “delimiter” parameter needs to exist right before the value of the “prefix” parameter in the HTTP header's value when the system matches the HTTP header's value with the

“prefix” parameter. Its value must be:

- 0: Needs. If the value of “prefix” does not exist at the beginning of the HTTP header’s value, the HTTP header’s value can match the “prefix” parameter only when the value of the “delimiter” parameter exists right before the value of the “prefix” parameter in it. The spaces and tabs between the values of the parameters “prefix” and “delimiter” will be ignored.
- 1: Not need.

The default value is 0.



Note:

- If neither of the parameters “prefix” and “delimiter” is specified, the session ID will be obtained from the whole of the HTTP header’s value.
- If the HTTP header’s value in the request matches both “prefix” and “delimiter”, the session ID will be obtained from the string between “prefix” and “delimiter” in the HTTP header’s value.
- If the HTTP header’s value in the request does not match “prefix”, the session ID will be obtained from the whole of the HTTP header’s value.
- If the HTTP header’s value in the request matches “prefix” but does not match “delimiter”, the session ID will be obtained from the string after “prefix” in the HTTP header’s value.
- If the configured “prefix” exists in multiple locations in the HTTP header’s value, only the one existing in the first location will be matched.

For example:

```
AN(config)#slb group persistence request header group1 Myheader username "y" 0
```

After the preceding command is configured, when the HTTP request contain the following information, the request can successfully match the “prefix” parameter and the system will obtain the session ID from the string “123”.

```
GET / HTTP/1.1
Host:aaa
Myheader:username123y
```

no slb group persistence request header <group_name>

This command is used to delete the configuration of obtaining the session ID from a part or the whole of the specified header’s value in the HTTP request for the specified real service group.

show slb group persistence request header [group_name]

This command is used to display the configuration of obtaining the session ID from a part or the whole of the specified header’s value in the HTTP request for the specified real service group. If the “group_name” parameter is not specified, the configurations

of obtaining the session ID from a part or the whole of the specified header's value in the HTTP request for all real service groups will be displayed.

clear slb group persistence request header

This command is used to clear the configurations of obtaining the session ID from a part or the whole of the specified header's value in the HTTP request for all real service groups.

slb group persistence request body *<group_name> <prefix> <delimiter> [flag]*

This command is used to configure the specified real service group to extract the string for obtaining the session ID from the body in the HTTP request. The real service group will extract the string from the HTTP request body to obtain the session ID based on the settings in the “**slb group persistence value**” command.

For the description of the parameters, please refer to the command “**slb group persistence request header** *<group_name> <header_name> [prefix] [delimiter] [flag]*”.

no slb group persistence request body *<group_name>*

This command is used to delete the configuration of extracting the string for obtaining the session ID from the body in the HTTP request for the specified real service group.

show slb group persistence request body *[group_name]*

This command is used to display the configuration of extracting the string for obtaining the session ID from the body in the HTTP request for the specified real service group. If the “group_name” parameter is not specified, the configurations of extracting the string for obtaining the session ID from the body in the HTTP request for all real service groups will be displayed.

clear slb group persistence request body

This command is used to clear the configurations of extracting the string for obtaining the session ID from the body in the HTTP request for all real service groups.

slb group persistence response cookie *<group_name> <cookie_name>*

This command is used to configure the specified real service group to extract the string for obtaining the session ID from the specified cookie's value in the HTTP response. The real service group will extract the string from the specified cookie value to obtain the session ID based on the settings in the “**slb group persistence value**” command.

group_name	This parameter specifies the name of an existing real service group.
------------	--

cookie_name	This parameter specifies the name of the cookie. Its value
-------------	--

must be a string of 1 to 64 case-insensitive characters.

no slb group persistence response cookie <group_name>

This command is used to delete the configuration of extracting the string for obtaining the session ID from the specified cookie's value in the HTTP response for the specified real service group.

show slb group persistence response cookie [group_name]

This command is used to display the configuration of extracting the string for obtaining the session ID from the specified cookie's value in the HTTP response for the specified real service group. If the "group_name" parameter is not specified, the configurations of extracting the string for obtaining the session ID from the specified cookie's value in the HTTP response for all real service groups will be displayed.

clear slb group persistence response cookie

This command is used to clear the configurations of extracting the string for obtaining the session ID from the specified cookie's value in the HTTP response for all real service groups.

slb group persistence response header <group_name> <header_name> [prefix] [delimiter] [flag]

This command is used to configure the specified real service group to extract the string for obtaining the session ID from the specified header's value in the HTTP response. The real service group will extract the string from the specified header value to obtain the session ID based on the settings in the "**slb group persistence value**" command.

For the description of the parameters, please refer to the command "**slb group persistence request header <group_name> <header_name> [prefix] [delimiter] [flag]**".

no slb group persistence response header <group_name>

This command is used to delete the configuration of extracting the string for obtaining the session ID from the specified header's value in the HTTP response for the specified real service group.

show slb group persistence response header [group_name]

This command is used to display the configuration of extracting the string for obtaining the session ID from the specified header's value in the HTTP response for the specified real service group. If the "group_name" parameter is not specified, the configurations of extracting the string for obtaining the session ID from the specified header's value in the HTTP response for all real service groups will be displayed.

clear slb group persistence response header

This command is used to clear the configurations of extracting the string for obtaining the session ID from the specified header's value in the HTTP response for all real service groups.

slb group persistence response body <group_name> <prefix> <delimiter> [flag]

This command is used to configure the specified real service group to extract the string for obtaining the session ID from the body in the HTTP response. The real service group will extract the string from the HTTP response the body to obtain the session ID based on the settings in the “**slb group persistence value**” command.

For the description of the parameters, please refer to the command “**slb group persistence request header** <group_name> <header_name> [prefix] [delimiter] [flag]”.

no slb group persistence response body <group_name>

This command is used to delete the configuration of extracting the string for obtaining the session ID from the body in the HTTP response for the specified real service group.

show slb group persistence response body [group_name]

This command is used to display the configuration of extracting the string for obtaining the session ID from the body in the HTTP response for the specified real service group. If the “group_name” parameter is not specified, the configurations of extracting the string for obtaining the session ID from the body in the HTTP response for all real service groups will be displayed.

clear slb group persistence response body

This command is used to clear the configurations of extracting the string for obtaining the session ID from the body in the HTTP response for all real service groups.

slb group persistence value <group_name> <offset> [session_id_length]

This command is used to configure how to obtain the session ID from the extracted string for the specified real service group. This command applies only to the real service group using the persistence method with the session ID type configured as “string”. If this command is not configured for a real service group, this real service group will use the whole of the extracted string as the session ID.

group_name	This parameter specifies the name of an existing real service group.
offset	This parameter specifies the offset of obtaining the session ID from the extracted string, that is, the start location from which the session ID is obtained, in bytes. Its value must be an integer ranging from 0 to 32.

session_id_length	This parameter specifies the length of the session ID, in bytes. Its value must be an integer ranging from 0 to 64. The default value is 0, indicating that all characters after offsetting will be used to the session ID.
-------------------	---

no slb group persistence value <group_name>

This command is used to delete the configuration about how to obtain the session ID from the extracted string for the specified real service group.

show slb group persistence value [group_name]

This command is used to display the configuration about how to obtain the session ID from the extracted string for the specified real service group. If the “group_name” parameter is not specified, the configurations about how to obtain the session ID from the extracted string for all real service groups will be displayed.

clear slb group persistence value

This command is used to clear the configurations about how to obtain the session ID from the extracted string for all real service groups.

show slb persistence session [group_name]

This command is used to display all the dynamic session entries of the specified real service group using the persistence or diametersid method. If the “group_name” parameter is not specified, the dynamic session entries of all real service groups using the persistence or diametersid method will be displayed.

For example:

AN(config)#show slb persistence session g1	
HTTP/HTTPS dynamic persistent session table for group g1:	
Session ID	Real Server Name
10.8.1.241 0 (3679)	r1

slb group persistence session static <group_name> <static_session_id> <real_service> [port]

This command is used to create a static session entry for the specified real service group using the persistence method. The client requests matching a static session entry will be directly forwarded to the real service specified by the static session entry. A maximum of 256 static session entries can be configured for every real service group.

The static session entry of the real service group has a higher priority than the dynamic session entry. When a client request hits the real service group, the system will first match the session ID with the static session entries. If the session ID does not match any static session entry, the system will then match the session ID with the dynamic session entries.

group_name	This parameter specifies the name of an existing real service group.
static_session_id	This parameter specifies the session ID of the static session entry. Its value must be an IP address or a string of 1 to 64 characters enclosed in double quotes.
real_service	This parameter specifies the name of an existing real service.
port	Optional. This parameter specifies the source port of the client request. This parameter needs to be specified only when the session ID type of this real service group is “ipport”. Its value must be an integer ranging from 0 to 65,535.



Note: When using the static session entry to persist a session, administrators need to execute commands to configure the rule for obtaining the session ID, for example, through the “**slb group persistence request header**” command. For details, refer to the “**slb group method <group_name> persistence**” command.

no slb group persistence session static <group_name> <static_session_id> [port]

This command is used to delete a static session entry of the specified real service group.

show slb group persistence session static [group_name]

This command is used to display all static session entries of the specified real service group. If the “group_name” parameter is not specified, the static session entries of all real service groups will be displayed.

clear slb group persistence session static [group_name]

This command is used to clear all static session entries of the specified real service group. If the “group_name” parameter is not specified, the static session entries of all real service groups will be cleared.

slb group method <group_name> diametersid [first_choice_method] [threshold_granularity]

This command is used to create a real service group using the Diameter Session ID (diametersid) method. The diametersid method forwards the client requests hitting this real service group for the first time to one real service selected based on the “first choice method” and records the mapping between the Diameter session ID and the real service. When the subsequent client requests with the same Diameter session ID

hit this group, the `diametersid` method forwards them to the same real service persistently.

<code>group_name</code>	This parameter specifies the name of the real service group. Its value must be a string of 1 to 128 characters. Its value must be enclosed in double quotes when containing any special character or containing only numbers.
<code>first_choice_method</code>	Optional. This parameter specifies the “first choice method”. If a client request hits the real service group for the first time, the system will select an available real service based on the “first choice method” for the client request. Its value must be “rr”, “grr”, “sr”, “lc”, “lb”, “hi” or “chi”. The default value is “rr”.
<code>threshold_granularity</code>	Optional. This parameter is available only when the “first_choice_method” parameter is set to “lc” or “lb”. For the description of this parameter, please refer to the commands “ slb group method <group_name> lc ” and “ slb group method <group_name> lb ”.

slb group method <group name> **tuxedo** [*first_choice_method*]
[*threshold_granularity*]

This command is used to create a real service group using the Tuxedo method. The Tuxedo method forwards the client request hitting this real service group for the first time to one real service selected based on the “first choice method”. The real service will return a WSH’s IP address and port number, and the APV appliance will record the mapping between the real service and the WSH port number. When the subsequent client requests accessing this WSH IP address and port number hit this real service group, the Tuxedo method forwards the client requests to the same real service persistently.

<code>group_name</code>	This parameter specifies the name of the real service group. Its value must be a string of 1 to 128 characters. Its value must be enclosed in double quotes when containing any special character or containing only numbers.
<code>first_choice_method</code>	Optional. This parameter specifies the “first choice method”. If a client request hits the real service group for the first time, the system will select an available real service based on the “first choice method” for the client request. Its value must be “rr”, “grr”, “sr”, “lc”, or “lb”. The default value is “rr”.
<code>threshold_granularity</code>	Optional. This parameter is available only when the “first_choice_method” parameter is set to “lc” or “lb”. For the description of this parameter, please refer to the commands “ slb group method <group_name> lc ” and

“slb group method <group_name> lb”.

slb group method <group_name> orchestrator

This command is used to create a real service group using the orchestrator method. For a real service group using the orchestrator method, only one real service whose IP address is “0.0.0.0” or “::” can be added. The destination IP address should be unchanged when forwarding packets.

group_name	This parameter specifies the name of the real service group. Its value must be a string of 1 to 128 characters and must be enclosed in double quotes when containing any special character or only numbers.
------------	---

slb group method <group_name> {rr|hi|chi} [route_mode] [hash_mode]

This command is used to create a Layer 2 real service group. A Layer 2 real service can be associated only with a Layer 2 virtual service using the Default policy. When the client request reaches the appliance’s interface corresponding to the destination MAC address (the destination IP address of the request not equaling to the virtual service’s IP address), the client request will hit the Layer 2 virtual service configured on this interface, and then hit the Layer 2 real service group associated using the Default policy. The Layer 2 real service group supports the following group methods:

- rr
- hi
- chi

For the meaning of these three methods, please refer to the corresponding configuration commands.

group_name	This parameter specifies the name of the real service group. Its value must be a string of 1 to 128 characters. Its value must be enclosed in double quotes when containing any special character or containing only numbers.
------------	---

route_mode	Optional. This parameter specifies the routing mode, that is, how the traffic originating from the real service is routed out. Its value must be:
------------	---

- route: indicates that the traffic is routed based on the normal routing rules.
- direct: indicates that the traffic is routed out from the interface corresponding to the Layer 2 virtual service associated with the real service group.

The default value is “direct”.

hash_mode Optional. This parameter specifies the hash mode. Its value must be:

- src: indicates that the source IP address of the client request will be hashed.
- dst: indicates that the destination IP address of the client request will be hashed.
- default: indicates that both the source and destination IP addresses of the client request will be hashed.

The default value is “default”.

This parameter setting takes effect only when the method of this real service group is hi or chi.

no slb group method <group_name>

This command is used to delete the specified real service groups. This command will also delete the policies associated with this real service group, the real services added to this real service group, and other configurations of this real service group.

show slb group method [group_name]

This command is used to display the specified real service group and its method. If the “group_name” parameter is not specified, all real service groups and their methods will be displayed.

clear slb group method

This command is used to clear all real service groups and their methods. This command will also clear the policies associated with every real service group, the real services added to every real service group, and other configurations of every real service group.

slb group settings proxyprotocol <group_name> [proxy_protocol_version]

This command is used to set the specified real service group as the sender of Proxy Protocol packets.

After this command is configured, the device can send Proxy Protocol packets to the real service.

group_name This parameter specifies an existing real service group name. Its value must be a real service group which including real services of TCP, TCPS, HTTP or HTTPS protocol type.

proxy_protocol_version Optional. This parameter specifies the version of the Proxy Protocol. Its value must be:

- v1: indicates that the appliance will send v1 Proxy Protocol packets to the real service.
- v2: indicates that the appliance will send v2 Proxy Protocol packets to the real service.

The default value is “v1”.

no slb group settings proxyprotocol <group_name>

This command is used to delete the configuration of the specified real service group as the sender of Proxy Protocol packets.

show slb group settings proxyprotocol [group_name]

This command is used to display the configuration of the specified real service group as the sender of Proxy Protocol packets. If the “group_name” parameter is not specified, the configuration of all real service groups as the sender of Proxy Protocol packets will be displayed.

clear slb group settings proxyprotocol

This command is used to clear the configuration of all real service groups as the sender of Proxy Protocol packets.

Adding Real Services to the Group

slb group member <group_name> <real_service>
[weight|cookie_value|url_tag_value] [priority]

This command is used to add a real service to the specified real service group.

group_name This parameter specifies the name of an existing real service group.

real_service This parameter specifies the name of an existing real service.

weight|cookie_value|url_tag_value

- **weight:** This parameter specifies the weight of the real service. Its value must be an integer ranging from 1 to 4,294,967,295. The default value is 1. This parameter needs to be specified when the group method is rr, grr, lc or lb, or when the group method is pi, ph, hh, hq, hc, ic, rc, ec, sslsid, radchs and so on and the “first choice method” is rr, grr, lc or lb.
- **cookie_value:** This parameter specifies the cookie value corresponding to the real service. This parameter needs to be specified when the group method is pc. For details, please refer to the command “**slb group method <group_name> pc [offset]**”.

- **url_tag_value**: This parameter specifies the URL tag value corresponding to the real service. This parameter needs to be specified when the group method is pu. For details, please refer to the command “**slb group method <group_name> pu**”.

priority

Optional. This parameter specifies the priority of the real service. Its value must be an integer ranging from 0 to 4,294,967,295. The default value is 0. The larger the value, the higher the priority.

This parameter needs to be used together with the command “**slb group activation**”.



Note:

- The real services added to the same real service group must be of the same protocol type.
- For a group whose members are configured with the “[weight|cookie_value|url_tag_value]” parameter, if the method used by this group is modified, the value of this parameter will be automatically reset to default (“weight” will be changed to 1; “cookie_value” and “url_tag_value” will be changed to the real service name).

The total number of real service group members depends on the system memory:

System Memory	Number of real service group members
4 GB ≤ Memory < 32 GB	4000
32 GB ≤ Memory < 128 GB	8000
Memory ≥ 128 GB	20000

no slb group member <group_name> <real_service>

This command is used to delete a real service from the specified real service group.

show slb group member [group_name]

This command is used to display the real services in the specified real service group. If the “group_name” parameter is not specified, the real services in all real service groups will be displayed.

clear slb group member <group_name>

This command is used to clear all the real services in the specified real service group.

show slb group protocol <group_name>

This command is used to display the protocol type of the specified real service group. If no real service has been added to the specified real service group, the protocol type of this group is “none”. When a real service is added to the specified real service group, the protocol type of this group will become the protocol type of this real

service. After this, only the real services of this protocol type can be added to this group.

General Real Service Group Configuration

slb group {enable|disable} <group_name>

This command is used to enable or disable the specified real service group. By default, every real service group is enabled.

group_name	This parameter specifies the name of an existing real service group.
------------	--



Note: Disabling a real service group will not affect the existing service connections on the real services in this group.

slb group activation <group_name> <num_of_usable_real_service> [num_of_active_real_service] [priority_subgroup_mode]

This command is used to set the member activation policy for the specified real service group.

The real service group supports two member activation modes:

- **Priority-based activation mode:** The system activates a specified number (specified by the “num_of_usable_real_service” parameter) of real services in a group in the descending order of priority. If the number of available real service is less than the setting of the “num_of_active_real_service” parameter, the group is regarded as unavailable and the traffic destined for it will be switched to the group associated with the backup policy.
- **Priority subgroup-based activation mode:** The system divides real services into subgroups according to their priorities and only real services in the highest-priority subgroup process the traffic. When the number of available real services in this subgroup is less than the setting of the “num_of_active_real_service” parameter, this subgroup is regarded as unavailable and the subgroup with the second highest priority will process the traffic. When all priority subgroups are unavailable, the traffic destined for the group will be switched to the group associated with the backup policy.

If this command is not configured for a specified real service group, the system will use the priority-based activation mode; the number of available real services allowed to be used is 0, indicating that all available real services can be used; the minimum number of active real services is also 0, indicating that the group will not perform traffic switchover based on the number of active real services.

group_name	This parameter specifies the name of an existing real service group.
------------	--

num_of_usable_real_ser	This parameter specifies the number of available real services allowed to be used. Its value must be an integer
------------------------	---

vice	ranging from 0 to 65,535. The value 0 indicates that all available real services are usable. If this parameter is set to 2, the two available real services with the highest priorities in this real service group can be used. That is, requests hitting this real service group can be forwarded to these two real services. For the description of the real service's priority, please refer to the “slb group member” command. When the priority subgroup-based activation mode is used (priority_subgroup_mode=1), this parameter must be set to 0.
num_of_active_real_service	Optional. The function of this parameter varies by the selected group member activation mode: • When priority-based activation mode is used (priority_subgroup_mode=0), this parameter specifies the minimum number of active real services for determining group availability. • When priority subgroup-based activation mode is used (priority_subgroup_mode=1), this parameter specifies the minimum number of active real services for determining subgroup availability. Its value must be an integer ranging from 0 to 65,535. The value 0 indicates that the number of active real services is not limited. That is, the group is always regarded as available.
priority_subgroup_mode	Optional. This parameter specifies the group member activation mode. Its value must be: • 0: enables the priority-based activation mode. • 1: enables the priority subgroup-based activation mode. The default value is 0.

no slb group activation <group_name>

This command is used to reset the member activation policy for the specified real service group.

show slb group activation <group_name>

This command is used to display the configuration of the member activation policy for the specified real service group, and the status of all real services in this real service group.

For example:

AN(config)#show slb group activation group1			
Minimum number of active real services for determining group/subgroup availability: 0			
Priority subgroup-based activation mode: 0			
real service	priority	active	reason

slb persistence timeout *<timeout_minutes> [group_name] [idle|duration]*

This command is used to set the timeout value of the session persistence for the specified real service group. If the “group_name” parameter is not specified, this command is used to set the timeout value of the session persistence for the global. If this command is not configured, the timeout value of the session persistence for the global is 0 minutes, indicating that the session persistence never times out.

This command can be configured only for the real service groups using any of the following group methods:

- pi
- ph
- hc
- hh
- hq
- persistence
- sipcidps
- sipuidps
- pto
- radpsun
- diametersid

timeout_minutes	This parameter specifies the timeout value of the session persistence, in minutes. Its value must be an integer ranging from 0 to 50,000. If the “group_name” parameter needs to be specified, the time value cannot be 0.
group_name	Optional. This parameter specifies the name of an existing real service group.
idle duration	Optional. This parameter specifies the timeout mode of the session persistence. This parameter is available only when the “group_name” parameter is specified. Its value must be: • idle: indicates that the appliance will delete the session

persistence when the time that the session persistence is idle exceeds the configured timeout value.

- **duration:** indicates that the appliance will delete the session persistence when the time that the session persistence has been established exceeds the configured timeout value.

The default value is “idle”. For the real service group using the pi, ph, hh, hc, hq or pto group method, only the “idle” mode is supported.



Note: The global timeout value has a lower priority than that set for the individual real service group. The global timeout value can take effect for a real service group only when the timeout value is not set for this real service group.

no slb persistence timeout *[group_name]*

This command is used to delete the timeout value setting of the session persistence for the specified real service group. If the “group_name” parameter is not specified, this command is used to reset the timeout value of the session persistence for the global to the default value.

show slb persistence timeout *[group_name]*

This command is used to display the timeout value setting of the session persistence for the specified real service group. If the “group_name” parameter is not specified, the timeout value setting of the session persistence for the global will be displayed.

slb group flush *<group_name>*

This command is used to clear the hash table used to maintain the session persistence for the specified real service group. When the hash table is cleared, the original session persistence of the real service group will be cleared.

group_name	This parameter specifies the name of an existing real service group. This real service group must use any of the following group methods:
-------------------	---

- pi
- ph
- hc
- hh
- hq
- pto

slb group option portrange *<group_name> <start_port> <end_port> [protocol] [port_type]*

This command is used to define a port range for the specified L2 real service group (L2IP or L2MAC group). Only traffic hitting the specified port range will be load balanced.

The configuration of this command can be displayed by the “**show slb all**” command. All configurations of this command can be cleared by the “**clear slb group method**” command.

group_name	This parameter specifies the name of the L2 SLB real service group, which can be configured by the “ slb group method <group_name> {rr hi chi} [route_mode] [hash_mode] ” command. It can also be a real service group configured by the “ slb group method <group_name> [method] ” command and contains L2IP or L2MAC real services.
start_port	This parameter specifies the start port number of the port range. Its value must be an integer ranging from 0 to 65,535.
end_port	This parameter specifies the end port number of the port range. Its value must be an integer ranging from 0 to 65,535.
protocol	Optional. This parameter specifies the protocol type. Its value must be “tcp”, “udp”, or “all” (TCP and UDP). The default value is “all”.
port_type	Optional. This parameter specifies the port type. Its value must be: • src: indicates the source port. • dst: indicates the destination port. The default value is “dst”.

no slb group option portrange <group_name> <start_port> <end_port> [protocol] [port_type]

This command is used to delete the port range configurations for the specified L2 SLB real service group.

slb group option itcpopt <group_name> <tcp_option_kind> [insert_source_port]

This command is used to configure the specified group to forward clients’ addresses to real services via the specified TCP option. Real services can obtain clients’ real addresses for logging or auditing. The TCP option will be inserted into the first SYN packet or ACK packet for each TCP connection that APV sends to the real service. If

the client sends multiple TCP requests, the client IP address will be inserted in the first SYN and ACK packets of each request.

When the “**slb forwardip**” command is also configured, the client IP addresses obtained from the TCP option in the client’s handshake ACK message will also be forwarded to the real services. If there is no client IP address in the TCP option or IP extraction fails, only the source IP address in the IP header will be forwarded.

<code>group_name</code>	This parameter specifies the name of an existing group. It must be a TCP, TCPS, FTP, or FTPS group.
<code>tcp_option_kind</code>	This parameter specifies the kind value of the TCP option. Its value must be an integer ranging from 31 to 254.
<code>insert_source_port</code>	Optional, this parameter specifies whether to insert the client source port, the value must be: • 1: Insert client source port. • 0: Do not insert the client source port. The default value is 0.

no slb group option itcpopt <group_name>

This command is used to delete the “**slb group option itcpopt**” configuration for the specified group.

show slb group option itcpopt [group_name]

This command is used to display the “**slb group option itcpopt**” configuration for the specified group. If the “group_name” parameter is not specified, the “**slb group option itcpopt**” configurations for all groups will be displayed.

clear slb group option itcpopt [group_name]

This command is used to clear the “slb group option itcpopt” configuration for the specified group. If the “group_name” parameter is not specified, the “slb group option itcpopt” configurations for all groups will be cleared.

slb group application bandwidth <group_name> <soft_bandwidth_limit> [hard_bandwidth_limit]

This command is used to set the bandwidth limit for the specified real service group.

<code>group_name</code>	This parameter specifies the name of an existing real service group.
<code>soft_bandwidth_limit</code>	This parameter specifies the soft bandwidth limit for the group in Kbps. Its value must be an integer ranging from 1

to 4,294,967,295. If the used bandwidth of the group exceeds the soft bandwidth limit, the system will generate a “Warning” log, but requests hitting this group will still be forwarded through it.

hard_bandwidth_limit Optional. This parameter specifies the hard bandwidth limit for the group in Kbps. Its value must be an integer ranging from 1 to 4,294,967,295.

- If the used bandwidth of the group exceeds the hard bandwidth limit, the system will generate a “Warning” log but will not forward requests through it.
- If this parameter is not specified, the value of the “soft_bandwidth_limit” parameter will be used by default.

no slb group application bandwidth <group_name>

This command is used to delete the bandwidth limit configuration for the specified real service group.

show slb group application bandwidth [group_name]

This command is used to display the bandwidth limit configuration for the specified real service group. If the “group_name” parameter is not specified, the bandwidth limit configurations for all real service groups will be displayed.

clear slb group application bandwidth

This command is used to clear the bandwidth limit configurations for all real service groups.

Proxy IP Pool

This section introduces the commands for configuring proxy IP pools. The APV appliance supports both global and per-group proxy IP pools.

If both global and per-group proxy IP pools are configured, the system will preferentially use the per-group proxy IP pool when it tries to set up a connection with a real service in the real service group. When a real service group is configured with multiple proxy IP pools, the principles for selecting one are as follows:

- Select from those that are on the same network segment as the gateway. Among these pools, the one that is in the current NUMA domain has higher priority.
- If none of the IP pools are on the same network segment as the gateway, nor are they in the current NUMA domain, the system will randomly select one.

If only global proxy IP pools are configured, the principles for selecting one are as follows:

- Select from those that are on the same network segment as the gateway. Among these pools, the one that is in the current NUMA domain has higher priority.
- If none of the IP pools are on the same network segment as the gateway, the system will use the interface IP address.

If neither the global nor the per-group proxy IP pool is configured, the system will use the interface IP address.

slb proxyip global <pool_name>

This command is used to configure a proxy IP pool for the global. The number of the proxy IP pools allowed for the global cannot exceed the maximum number of IP pools that can be configured for the system (see the “**ip pool**” command).

pool_name This parameter specifies the name of an existing IP pool.

no slb proxyip global <pool_name>

This command is used to delete a proxy IP pool configured for the global.

slb proxyip group <group_name> <pool_name>

This command is used to configure a proxy IP pool for the specified real service group. The number of the proxy IP pools allowed for a real service group cannot exceed the maximum number of IP pools that can be configured for the system (see the “**ip pool**” command).

group_name This parameter specifies the name of an existing real service group.

pool_name This parameter specifies the name of an existing IP pool.



Note: The IP pool that is bound to a bond interface and the IP pool bound to a VLAN interface whose parent interface is a bond interface will all be allocated to NUMA domain 0. Therefore, during IP pool matching, two or more proxy IP pools might match or not match at the same time. In this circumstance, the appliance will select the first-configured proxy IP pool for packet forwarding.

no slb proxyip group <group_name> <pool_name>

This command is used to delete a proxy IP pool configured for the specified real service group.



Note: For an existing connection established using an IP address in the IP pool, even if the association between the group and the IP pool is cancelled, subsequent requests will still be transmitted over this connection until it times out.

show slb proxyip [group_name]

This command is used to display all proxy IP pools configured for the specified real service group. If the “group_name” parameter is not specified, both the global and per-group proxy IP pools will be displayed.

clear slb proxyip [group_name]

This command is used to clear all proxy IP pools configured for the specified real service group. If the “group_name” parameter is not specified, both the global and per-group proxy IP pools will be cleared.

HTTP/2 Group Support

If real service supports HTTP/2, administrators can add it to a real service group and then enable HTTP/2 for the group using the following command.

http2 group {on|off} <group_name>

This command is used to enable or disable HTTP/2 for the specified HTTP/HTTPS real service group. By default, only HTTP/1 is enabled for HTTP/HTTPS real service groups, and HTTP/2 is disabled. Enabling HTTP/2 for a real service group that does not support HTTP/2 is not recommended.

group_name	This parameter specifies an existing real service group name. Its value must be an HTTP or HTTPS real service group.
------------	--

show http2 group [group_name]

This command is used to display the enabling status of HTTP/2 for the specified HTTP/HTTPS real service groups. If the “group_name” parameter is not specified, the enabling status of HTTP/2 for all HTTP/HTTPS real service groups is displayed.

clear http2 group

This command is used to disable HTTP/2 for all HTTP/HTTPS real service groups.

Virtual Service

A virtual service is the mapping of multiple real services on the appliance. It is used as the unified access point for real services.

Defining the Virtual Service

The following table shows the types of virtual services supported by Layer 7, Layer 4, Layer 3, and Layer 2 protocols:

Protocol Layer	Virtual Service Type
Layer 7	HTTP, HTTPS, DNS, DNSTCP, FTP, FTPS, RDP, SIP TCP, SIP UDP, Radauth, Radacct,

Protocol Layer	Virtual Service Type
	and Diameter
Layer 4	TCP, TCPS, and UDP
Layer 3	IP
Layer 2	L2 IP

**Note:**

- The configured virtual IP address cannot be the same as the IP address of any system interface.
- When the configured virtual IP address is “0.0.0.0”, packets destined to any interface IP address will be load-balanced to the specified real service. In this scenario, the “vport” parameter cannot be set to 0.
- If the configured virtual IP address does not belong to the subnet of any system interface IP address, the system will perform the following operations:
 - (1) If port1 is configured with an IP address, the system will associate the virtual IP address with port1.
 - (2) If port1 is neither configured with an IP address nor is added to any bond interface, the system will prompt an error message.
 - (3) If port1 is added to a bond interface and the bond interface is configured with an IP address, the system will associate the virtual IP address with the bond interface. Otherwise, an error message is displayed.
- When creating a virtual service on a VLAN interface, make sure the configured virtual IP address belongs to the same subnet of the VLAN interface IP.

The total number of virtual services depends on the system memory:

System Memory	Number of virtual services
≤ 32 GB	4000
≥ 64 GB	8000

The total number of the IP addresses for virtual services allowed to be configured varies between system memory.

System Memory	Maximum number of IP addresses for virtual service
$1 \text{ GB} \leq \text{System Memory} \leq 8 \text{ GB}$	2000
$16 \text{ GB} \leq \text{System Memory} \leq 24 \text{ GB}$	4000
$32 \text{ GB} \leq \text{System Memory} \leq 384 \text{ GB}$	8000

slb virtual http <virtual_service> <vip> [vport] [arp_support] [max_connection]

This command is used to configure an HTTP virtual service.

virtual_service This parameter specifies the virtual service name. Its value must be a case-sensitive string of 1 to 128 characters. Numbers, letters, and special characters are supported. The special characters include the at sign (@), exclamation mark (!), tilde (~), underscore (_), colon (:), hyphen (-),

angle brackets (< >), and period (.).

When the parameter value contains special characters, it must be enclosed in double quotes. The specified virtual service name cannot be the system reserved word “default”, “all” or “global”, regardless of its case sensitivity.

<code>vip</code>	This parameter specifies the virtual service’s IP address. Its value must be an IPv4 or IPv6 address.
<code>vport</code>	Optional. This parameter specifies the virtual service’s port number. Its value must be an integer ranging from 0 to 65,535. When the parameter value is 0, the virtual service listens for client requests on all ports. The default value is 80.
<code>arp_support</code>	Optional. This parameter specifies whether the virtual service supports ARP. Its value must be: • <code>arp</code>: supports ARP. • <code>noarp</code>: does not support ARP. The IP address of the virtual service cannot be pinged through or detected by ARP. Clients can directly send packets to the IP address of the real service. In this scenario, the appliance’s interface IP address must be configured as the clients’ gateway. The default value is “arp”.
<code>max_connection</code>	Optional. This parameter specifies the maximum number of concurrent connections allowed for each virtual service. The default value is 0, indicating no limitation on the maximum number of concurrent connections.

slb virtual https <virtual_service> <vip> [vport] [arp_support] [max_connection]

This command is used to configure an HTTPS virtual service.

<code>virtual_service</code>	This parameter specifies the virtual service name. Its value must be a case-sensitive string of 1 to 128 characters. Numbers, letters, and special characters are supported. The special characters include the at sign (@), exclamation mark (!), tilde (~), underscore (_), colon (:), hyphen (-), angle brackets (< >), and period (.).
------------------------------	--

When the parameter value contains special characters, it

must be enclosed in double quotes. The specified virtual service name cannot be the system reserved word “default”, “all” or “global”, regardless of its case sensitivity.

vip This parameter specifies the virtual service’s IP address. Its value must be an IPv4 or IPv6 address.

vport Optional. This parameter specifies the virtual service’s port number. Its value must be an integer ranging from 0 to 65,535. When the parameter value is 0, the virtual service listens for client requests on all ports.

The default value is 443.

arp_support Optional. This parameter specifies whether the virtual service supports ARP. Its value must be:

- **arp**: supports ARP.
- **noarp**: does not support ARP. The IP address of the virtual service cannot be pinged through or detected by ARP. Clients can directly send packets to the IP address of the real service. In this scenario, the appliance’s interface IP address must be configured as the clients’ gateway.

The default value is “arp”.

max_connection Optional. This parameter specifies the maximum number of concurrent connections allowed for each virtual service.

The default value is 0, indicating no limitation on the maximum number of concurrent connections.

slb virtual dns *<virtual_service> <vip> [vport] [arp_support] [max_connection]*

This command is used to configure a DNS virtual service.

virtual_service This parameter specifies the virtual service name. Its value must be a case-sensitive string of 1 to 128 characters. Numbers, letters, and special characters are supported. The special characters include the at sign (@), exclamation mark (!), tilde (~), underscore (_), colon (:), hyphen (-), angle brackets (< >), and period (.).

When the parameter value contains special characters, it must be enclosed in double quotes. The specified virtual service name cannot be the system reserved word “default”, “all” or “global”, regardless of its case

	sensitivity.
vip	This parameter specifies the virtual service's IP address. Its value must be an IPv4 or IPv6 address.
vport	Optional. This parameter specifies the virtual service's port number. Its value must be an integer ranging from 0 to 65,535. When the parameter value is 0, the virtual service listens for client requests on all ports. The default value is 53.
	Note: slb virtual dnstcp and slb virtual dns can configure the same IP address and port.
arp_support	Optional. This parameter specifies whether the virtual service supports ARP. Its value must be: • arp: supports ARP. • noarp: does not support ARP. The IP address of the virtual service cannot be pinged through or detected by ARP. Clients can directly send packets to the IP address of the real service. In this scenario, the appliance's interface IP address must be configured as the clients' gateway. The default value is "arp".
max_connection	Optional. This parameter specifies the maximum number of concurrent connections allowed for each virtual service. The default value is 0, indicating no limitation on the maximum number of concurrent connections.

slb virtual dnstcp <virtual_service> <vip> [vport] [arp_support] [max_connection]

This command is used to define a DNS virtual service that uses the TCP protocol for data transmission.

virtual_service	This parameter specifies the name of a virtual service. The value must be a case-sensitive string with a length not greater than 128 bytes. Numbers, letters, and special characters are supported. If the value contains only numbers or special characters, it must be enclosed in double quotes. The value cannot set to the system reserved word "default," "all," or "global," which is case-insensitive.
vip	This parameter specifies the IP address of a virtual service. The value must be an IPv4 or IPv6 address.

vport	Optional. This parameter specifies the port of a virtual service. The value must be an integer ranging from 0 to 65,535. When the value is zero, it means this virtual service listens to the client's request on all ports. The default value is 53. Note: slb virtual dnstcp and slb virtual dns can configure the same IP address and port.
arp_support	Optional. This parameter specifies whether a virtual service supports the ARP protocol. The value must be: - arp: It means the virtual service supports the ARP protocol. - noarp: It means the virtual service does not support the ARP protocol. The IP address of a virtual service is unreachable and cannot be detected by the ARP protocol. The client can directly send data to the IP address of a real service. In this case, the IP address of the device interface needs to be configured as the client's gateway. The default value is "arp."
max_connection	Optional. This parameter specifies the maximum number of concurrent connections of a virtual service. The default value is zero, which means there is no maximum number of concurrent connections.

slb virtual ftp <virtual_service> <vip> [vport] [max_connection]

This command is used to configure an FTP virtual service.

virtual_service	This parameter specifies the virtual service name. Its value must be a case-sensitive string of 1 to 128 characters. Numbers, letters, and special characters are supported. The special characters include the at sign (@), exclamation mark (!), tilde (~), underscore (_), colon (:), hyphen (-), angle brackets (< >), and period (.). When the parameter value contains special characters, it must be enclosed in double quotes. The specified virtual service name cannot be the system reserved word "default", "all" or "global", regardless of its case sensitivity.
vip	This parameter specifies the virtual service's IP address. Its value must be an IPv4 or IPv6 address.
vport	Optional. This parameter specifies the virtual service's port number. Its value must be an integer ranging from 1 to

65,535.

The default value is 21.

max_connection Optional. This parameter specifies the maximum number of concurrent connections allowed for each virtual service.

The default value is 0, indicating no limitation on the maximum number of concurrent connections.

slb virtual ftps <virtual_service> <vip> [vport] [max_connection]

This command is used to configure an FTPS virtual service. FTPS virtual services support only passive FTP in implicit mode.

virtual_service This parameter specifies the virtual service name. Its value must be a case-sensitive string of 1 to 128 characters. Numbers, letters, and special characters are supported. The special characters include the at sign (@), exclamation mark (!), tilde (~), underscore (_), colon (:), hyphen (-), angle brackets (< >), and period (.).

When the parameter value contains special characters, it must be enclosed in double quotes. The specified virtual service name cannot be the system reserved word “default”, “all” or “global”, regardless of its case sensitivity.

vip This parameter specifies the virtual service’s IP address. Its value must be an IPv4 or IPv6 address.

vport Optional. This parameter specifies the virtual service’s port number. Its value must be an integer ranging from 1 to 65,535.

The default value is 990.

max_connection Optional. This parameter specifies the maximum number of concurrent connections allowed for each virtual service.

The default value is 0, indicating no limitation on the maximum number of concurrent connections.

slb virtual settings proxyprotocol <virtual_service>

This command is used to set the specified virtual service as the receiver of Proxy Protocol packets.

The virtual service can receive v1 and v2 Proxy Protocol packets. After this command is configured, the virtual service can obtain the client IP address from the Proxy

Protocol packets sent by the client. For a HTTP or HTTPS virtual service, after obtaining the IP address of the client, the device will insert it into the X-Forwarded-For field and send it to the real service.

virtual_service	This parameter specifies an existing virtual service name. Its value must be a virtual service of TCP, TCPS, HTTP or HTTPS protocol type.
-----------------	---

no slb virtual settings proxyprotocol <virtual_service>

This command is used to delete the configuration of the specified virtual service as the receiver of Proxy Protocol packets.

show slb virtual settings proxyprotocol [virtual_service]

This command is used to display the configuration of the specified virtual service as the receiver of Proxy Protocol packets. If the “virtual_service” parameter is not specified, the configuration of all virtual services as the receiver of Proxy Protocol packets will be displayed.

clear slb virtual settings proxyprotocol

This command is used to delete the configuration of all virtual services as the receiver of Proxy Protocol packets.

slb ftps datachannel [mode]

This command is used to set the transfer mode of the FTPS data channel. For WinSCP clients, the plaintext transfer mode must be used. If this command is not configured, the FTPS data channel will use the ciphertext transfer mode by default.

mode	Optional. This parameter specifies the transfer mode. Its value must be:
------	--

- 0: transfers data in ciphertext mode.
- 1: transfers data in plaintext mode.

The default value is 0.

no slb ftps datachannel

This command is used to restore the transfer mode of the FTPS data channel to default.

show slb ftps datachannel

This command is used to display the transfer mode setting of the FTPS data channel.

slb ftp datachannel <port_value>

This command is used to modify the transmission port number of the data channel for FTP services in active mode. If this command is not configured, port 20 is used by default.

<code>port_value</code>	This parameter specifies the transmission port number of the data channel for FTP services in active mode. Its value must be an integer ranging from 1 to 65,535.
-------------------------	---

no slb ftp datachannel

This command is used to reset the transmission port number of the data channel for FTP services in active mode to the default value.

show slb ftp datachannel

This command is used to display the transmission port number of the data channel for FTP services in active mode.

ftp passive externalip <virtual_service> <ip>

This command is used to configure an external IP address for the specified FTP or FTPS virtual service to replace the IP address in the PASV reply. In passive FTP, firewalls or NAT gateways deployed between the clients and the appliance may fail to correctly translate the IP address in the PASV reply. After this command is configured, the appliance will replace the IP address in the PASV reply with the specified external IP address for the clients to establish FTP data connection.

<code>virtual_service</code>	This parameter specifies the virtual service name. Its value must be an FTP or FTPS virtual service.
------------------------------	--

<code>ip</code>	This parameter specifies the external IP address used to replace the IP address in the PASV reply.
-----------------	--

no ftp passive externalip [virtual_service]

This command is used to delete the external IP address configuration for the specified FTP or FTPS virtual service. If the “virtual_service” parameter is not specified, the external IP address configurations for all FTP and FTPS virtual services will be cleared.

show ftp passive externalip [virtual_service]

This command is used to display the external IP address configuration for the specified FTP or FTPS virtual service. If the “virtual_service” parameter is not specified, the external IP address configurations for all FTP and FTPS virtual services will be displayed.

slb virtual tcp <virtual_service> <vip> <vport> [arp_support] [max_connection]

This command is used to configure a TCP virtual service.

virtual_service	This parameter specifies the virtual service name. Its value must be a case-sensitive string of 1 to 128 characters. Numbers, letters, and special characters are supported. The special characters include the at sign (@), exclamation mark (!), tilde (~), underscore (_), colon (:), hyphen (-), angle brackets (< >), and period (.). When the parameter value contains special characters, it must be enclosed in double quotes. The specified virtual service name cannot be the system reserved word “default”, “all” or “global”, regardless of its case sensitivity.
vip	This parameter specifies the virtual service’s IP address. Its value must be an IPv4 or IPv6 address.
vport	This parameter specifies the virtual service’s port number. Its value must be an integer ranging from 0 to 65,535. When the parameter value is 0, the virtual service listens for client requests on all ports.
arp_support	Optional. This parameter specifies whether the virtual service supports ARP. Its value must be: • arp: supports ARP. • noarp: does not support ARP. The IP address of the virtual service cannot be pinged through or detected by ARP. Clients can directly send packets to the IP address of the real service. In this scenario, the appliance’s interface IP address must be configured as the clients’ gateway. The default value is “arp”.
max_connection	Optional. This parameter specifies the maximum number of concurrent connections allowed for each virtual service. The default value is 0, indicating no limitation on the maximum number of concurrent connections.

slb virtual tcps <virtual_service> <vip> <vport> [arp_support]
[max_connection]

This command is used to configure a TCPS virtual service.

virtual_service	This parameter specifies the virtual service name. Its value must be a case-sensitive string of 1 to 128 characters. Numbers, letters, and special characters are supported. The
-----------------	---

special characters include the at sign (@), exclamation mark (!), tilde (~), underscore (_), colon (:), hyphen (-), angle brackets (< >), and period (.).

When the parameter value contains special characters, it must be enclosed in double quotes. The specified virtual service name cannot be the system reserved word “default”, “all” or “global”, regardless of its case sensitivity.

vip This parameter specifies the virtual service’s IP address. Its value must be an IPv4 or IPv6 address.

vport This parameter specifies the virtual service’s port number. Its value must be an integer ranging from 0 to 65,535. When the parameter value is 0, the virtual service listens for client requests on all ports.

arp_support Optional. This parameter specifies whether the virtual service supports ARP. Its value must be:

- **arp**: supports ARP.
- **noarp**: does not support ARP. The IP address of the virtual service cannot be pinged through or detected by ARP. Clients can directly send packets to the IP address of the real service. In this scenario, the appliance’s interface IP address must be configured as the clients’ gateway.

The default value is “arp”.

max_connection Optional. This parameter specifies the maximum number of concurrent connections allowed for each virtual service.

The default value is 0, indicating no limitation on the maximum number of concurrent connections.

slb virtual udp <virtual_service> <vip> <vport> [arp_support] [max_connection]

This command is used to configure a UDP virtual service.

virtual_service This parameter specifies the virtual service name. Its value must be a case-sensitive string of 1 to 128 characters. Numbers, letters, and special characters are supported. The special characters include the at sign (@), exclamation mark (!), tilde (~), underscore (_), colon (:), hyphen (-), angle brackets (< >), and period (.).

When the parameter value contains special characters, it

	must be enclosed in double quotes. The specified virtual service name cannot be the system reserved word “default”, “all” or “global”, regardless of its case sensitivity.
vip	This parameter specifies the virtual service’s IP address. Its value must be an IPv4 or IPv6 address.
vport	This parameter specifies the virtual service’s port number. Its value must be an integer ranging from 0 to 65,535. When the parameter value is 0, the virtual service listens for client requests on all ports.
arp_support	Optional. This parameter specifies whether the virtual service supports ARP. Its value must be: • arp: supports ARP. • noarp: does not support ARP. The IP address of the virtual service cannot be pinged through or detected by ARP. Clients can directly send packets to the IP address of the real service. In this scenario, the appliance’s interface IP address must be configured as the clients’ gateway. The default value is “arp”.
max_connection	Optional. This parameter specifies the maximum number of concurrent connections allowed for each virtual service. The default value is 0, indicating no limitation on the maximum number of concurrent connections.

slb mode packetbased <virtual_service>

This command is used to configure packet-based load balancing for the specified UDP virtual service. With this configuration, the packets of one client connection will be distributed to different real services according to the specified group method.

virtual_service This parameter specifies the virtual service name.



Note: The UDP service connections for which packet-based load balancing is configured do not support the HA SSF function.

no slb mode packetbased <virtual_service>

This command is used to delete the packet-based load balancing configuration for the specified UDP virtual service.

show slb mode packetbased

This command is used to display all packet-based load balancing configurations.

clear slb mode packetbased

This command is used to clear all packet-based load balancing configurations.

slb virtual ip <virtual_service> <vip> [max_connection]

This command is used to configure an IP virtual service. IP virtual services support both TCP and UDP.

virtual_service	This parameter specifies the virtual service name. Its value must be a case-sensitive string of 1 to 128 characters. Numbers, letters, and special characters are supported. The special characters include the at sign (@), exclamation mark (!), tilde (~), underscore (_), colon (:), hyphen (-), angle brackets (< >), and period (.).
-----------------	--

When the parameter value contains special characters, it must be enclosed in double quotes. The specified virtual service name cannot be the system reserved word “default”, “all” or “global”, regardless of its case sensitivity.

vip	This parameter specifies the virtual service’s IP address. Its value must be an IPv4 or IPv6 address.
-----	---

max_connection	Optional. This parameter specifies the maximum number of concurrent connections allowed for each virtual service. The default value is 0, indicating no limitation on the maximum number of concurrent connections.
----------------	--

slb virtual siptcp <virtual_service> <vip> [vport] [arp_support] [max_connection]

This command is used to configure a SIP TCP virtual service.

virtual_service	This parameter specifies the virtual service name. Its value must be a case-sensitive string of 1 to 128 characters. Numbers, letters, and special characters are supported. The special characters include the at sign (@), exclamation mark (!), tilde (~), underscore (_), colon (:), hyphen (-), angle brackets (< >), and period (.).
-----------------	--

When the parameter value contains special characters, it must be enclosed in double quotes. The specified virtual service name cannot be the system reserved word “default”, “all” or “global”, regardless of its case sensitivity.

vip	This parameter specifies the virtual service's IP address. Its value must be an IPv4 or IPv6 address.
vport	Optional. This parameter specifies the virtual service's port number. Its value must be an integer ranging from 0 to 65,535. When the parameter value is 0, the virtual service listens for client requests on all ports. The default value is 5060.
arp_support	Optional. This parameter specifies whether the virtual service supports ARP. Its value must be: • arp: supports ARP. • noarp: does not support ARP. The IP address of the virtual service cannot be pinged through or detected by ARP. Clients can directly send packets to the IP address of the real service. In this scenario, the appliance's interface IP address must be configured as the clients' gateway. The default value is "arp".
max_connection	Optional. This parameter specifies the maximum number of concurrent connections allowed for each virtual service. The default value is 0, indicating no limitation on the maximum number of concurrent connections.

slb virtual sipudp <virtual_service> <vip> [vport] [arp_support] [max_connection]

This command is used to configure a SIP UDP virtual service.

virtual_service	This parameter specifies the virtual service name. Its value must be a case-sensitive string of 1 to 128 characters. Numbers, letters, and special characters are supported. The special characters include the at sign (@), exclamation mark (!), tilde (~), underscore (_), colon (:), hyphen (-), angle brackets (< >), and period (.). When the parameter value contains special characters, it must be enclosed in double quotes. The specified virtual service name cannot be the system reserved word "default", "all" or "global", regardless of its case sensitivity.
vip	This parameter specifies the virtual service's IP address. Its value must be an IPv4 or IPv6 address.

vport	Optional. This parameter specifies the virtual service's port number. Its value must be an integer ranging from 0 to 65,535. When the parameter value is 0, the virtual service listens for client requests on all ports. The default value is 5060.
arp_support	Optional. This parameter specifies whether the virtual service supports ARP. Its value must be: - arp: supports ARP. - noarp: does not support ARP. The IP address of the virtual service cannot be pinged through or detected by ARP. Clients can directly send packets to the IP address of the real service. In this scenario, the appliance's interface IP address must be configured as the clients' gateway. The default value is "arp".
max_connection	Optional. This parameter specifies the maximum number of concurrent connections allowed for each virtual service. The default value is 0, indicating no limitation on the maximum number of concurrent connections.

slb virtual radauth <virtual_service> <vip> [vport] [arp_support] [max_connection]

This command is used to configure a RADIUS authentication virtual service.

virtual_service	This parameter specifies the virtual service name. Its value must be a case-sensitive string of 1 to 128 characters. Numbers, letters, and special characters are supported. The special characters include the at sign (@), exclamation mark (!), tilde (~), underscore (_), colon (:), hyphen (-), angle brackets (<>), and period (.). When the parameter value contains special characters, it must be enclosed in double quotes. The specified virtual service name cannot be the system reserved word "default", "all" or "global", regardless of its case sensitivity.
vip	This parameter specifies the virtual service's IP address. Its value must be an IPv4 or IPv6 address.
vport	Optional. This parameter specifies the virtual service's port number. Its value must be an integer ranging from 0 to 65,535. When the parameter value is 0, the virtual service

listens for client requests on all ports.

The default value is 1812.

arp_support

Optional. This parameter specifies whether the virtual service supports ARP. Its value must be:

- **arp**: supports ARP.
- **noarp**: does not support ARP. The IP address of the virtual service cannot be pinged through or detected by ARP. Clients can directly send packets to the IP address of the real service. In this scenario, the appliance's interface IP address must be configured as the clients' gateway.

The default value is "arp".

max_connection

Optional. This parameter specifies the maximum number of concurrent connections allowed for each virtual service.

The default value is 0, indicating no limitation on the maximum number of concurrent connections.

slb virtual radacct <virtual_service> <vip> [vport] [arp_support]
[max_connection]

This command is used to configure a RADIUS accounting virtual service.

virtual_service

This parameter specifies the virtual service name. Its value must be a case-sensitive string of 1 to 128 characters. Numbers, letters, and special characters are supported. The special characters include the at sign (@), exclamation mark (!), tilde (~), underscore (_), colon (:), hyphen (-), angle brackets (< >), and period (.).

When the parameter value contains special characters, it must be enclosed in double quotes. The specified virtual service name cannot be the system reserved word "default", "all" or "global", regardless of its case sensitivity.

vip

This parameter specifies the virtual service's IP address. Its value must be an IPv4 or IPv6 address.

vport

Optional. This parameter specifies the virtual service's port number. Its value must be an integer ranging from 0 to 65,535. When the parameter value is 0, the virtual service listens for client requests on all ports.

The default value is 1813.

arp_support	Optional. This parameter specifies whether the virtual service supports ARP. Its value must be: • arp: supports ARP. • noarp: does not support ARP. The IP address of the virtual service cannot be pinged through or detected by ARP. Clients can directly send packets to the IP address of the real service. In this scenario, the appliance's interface IP address must be configured as the clients' gateway.
-------------	---

The default value is "arp".

max_connection	Optional. This parameter specifies the maximum number of concurrent connections allowed for each virtual service. The default value is 0, indicating no limitation on the maximum number of concurrent connections.
----------------	---

slb virtual rdp *<virtual_service> <vip> [vport] [arp_support] [max_connection]*

This command is used to configure an RDP virtual service.

virtual_service	This parameter specifies the virtual service name. It must be a case-sensitive string of 1 to 128 characters. Numbers, letters, and special characters are supported. The special characters include the at sign (@), exclamation mark (!), tilde (~), underscore (_), colon (:), hyphen (-), angle brackets (<>), and period (.). When the parameter value contains special characters, it must be enclosed in double quotes. The specified virtual service name cannot be the system reserved word "default", "all" or "global", regardless of its case sensitivity.
vip	This parameter specifies the virtual service's IP address. Its value must be an IPv4 address.
vport	Optional. This parameter specifies the virtual service's port number. Its value must be an integer ranging from 0 to 65,535. When the parameter value is 0, the virtual service listens for client requests on all ports. The default value is 3389.
arp_support	Optional. This parameter specifies whether the virtual service supports ARP. Its value must be: • arp: supports ARP.

- **noarp**: does not support ARP. The IP address of the virtual service cannot be pinged through or detected by ARP. Clients can directly send packets to the IP address of the real service. In this scenario, the appliance's interface IP address must be configured as the clients' gateway.

The default value is "arp".

max_connection Optional. This parameter specifies the maximum number of concurrent connections allowed for each virtual service.

The default value is 0, indicating no limitation on the maximum number of concurrent connections.

slb virtual rtsp *<virtual_service> <vip> [vport] [mode] [arp_support] [max_connection]*

This command is used to configure an RTSP virtual service.

virtual_service This parameter specifies the virtual service name. Its value must be a case-sensitive string of 1 to 128 characters. Numbers, letters, and special characters are supported. The special characters include the at sign (@), exclamation mark (!), tilde (~), underscore (_), colon (:), hyphen (-), angle brackets (< >), and period (.).

When the parameter value contains special characters, it must be enclosed in double quotes. The specified virtual service name cannot be the system reserved word "default", "all" or "global", regardless of its case sensitivity.

vip This parameter specifies the virtual service's IP address. Its value must be an IPv4 or IPv6 address.

vport Optional. This parameter specifies the virtual service's port number. Its value must be an integer ranging from 0 to 65,535. When the parameter value is 0, the virtual service listens for client requests on all ports.

The default value is 554.

mode Optional. This parameter specifies the working mode of the RTSP virtual service. Its value must be:

- **redirect**: indicates the redirect mode. In this mode, client requests will be redirected to real services. So, real services must be configured with public IP addresses for clients to access via Internet.

- **nat**: indicates the dynamic NAT mode. In this mode, real services do not need to be configured with public IP addresses. Control data and media streams must pass through the appliance.

The default value is “redirect”.

arp_support

Optional. This parameter specifies whether the virtual service supports ARP. Its value must be:

- **arp**: supports ARP.
- **noarp**: does not support ARP. The IP address of the virtual service cannot be pinged through or detected by ARP. Clients can directly send packets to the IP address of the real service. In this scenario, the appliance’s interface IP address must be configured as the clients’ gateway.

The default value is “arp”.

max_connection

Optional. This parameter specifies the maximum number of concurrent connections allowed for each virtual service.

The default value is 0, indicating no limitation on the maximum number of concurrent connections.

slb virtual diameter *<virtual_service> <vip> [vport] [arp_support] [max_connection]*

This command is used to configure a Diameter virtual service.

virtual_service

This parameter specifies the virtual service name. Its value must be a case-sensitive string of 1 to 128 characters. Numbers, letters, and special characters are supported. The special characters include the at sign (@), exclamation mark (!), tilde (~), underscore (_), colon (:), hyphen (-), angle brackets (< >), and period (.).

When the parameter value contains special characters, it must be enclosed in double quotes. The specified virtual service name cannot be the system reserved word “default”, “all” or “global”, regardless of its case sensitivity.

vip

This parameter specifies the virtual service’s IP address. Its value must be an IPv4 or IPv6 address.

vport

Optional. This parameter specifies the virtual service’s port number. Its value must be an integer ranging from 0 to 65,535. When the parameter value is 0, the virtual service

listens for client requests on all ports.

The default value is 3868.

arp_support

Optional. This parameter specifies whether the virtual service supports ARP. Its value must be:

- **arp**: supports ARP.
- **noarp**: does not support ARP. The IP address of the virtual service cannot be pinged through or detected by ARP. Clients can directly send packets to the IP address of the real service. In this scenario, the appliance's interface IP address must be configured as the clients' gateway.

The default value is "arp".

max_connection

Optional. This parameter specifies the maximum number of concurrent connections allowed for each virtual service.

The default value is 0, indicating no limitation on the maximum number of concurrent connections.

slb virtual tuxedo <virtual_service> <vip> <max_connection>

This command is used to configure a Tuxedo virtual service.

virtual_service

This parameter specifies the virtual service name. Its value must be a case-sensitive string of 1 to 128 characters. Numbers, letters, and special characters are supported. The special characters include the at sign (@), exclamation mark (!), tilde (~), underscore (_), colon (:), hyphen (-), angle brackets (< >), and period (.).

When the parameter value contains special characters, it must be enclosed in double quotes. The specified virtual service name cannot be the system reserved word "default", "all" or "global", regardless of its case sensitivity.

vip

This parameter specifies the virtual service's IP address. Its value must be an IPv4 or IPv6 address.

max_connection

Optional. This parameter specifies the maximum number of concurrent connections allowed for each virtual service.

The default value is 0, indicating no limitation on the maximum number of concurrent connections.



Note: It is recommended to configure a designated IP address for the Tuxedo virtual service. If using the same IP address with other virtual services, other virtual services may not work properly.

slb tuxedo portvalue <virtual_service> <wsl_port> <wsh_port_range>

This command is used to set the WSL port number and WSH port range for the specified Tuxedo virtual service.

virtual_service	This parameter specifies the name of an existing Tuxedo virtual service.
wsl_port	This parameter specifies the WSL listening port. Its value must be an integer ranging from 1 to 65,535.
wsh_port_range	This parameter specifies the WSH port range, in the format of “minport:maxport”. The value of “minport” and “maxport” must be an integer ranging from 1 to 65,535, and the value of “maxport” must be equal to or larger than that of “minport”.

no slb tuxedo portvalue <virtual_service>

This command is used to delete the setting of the WSL port number and WSH port range for the specified Tuxedo virtual service.

show slb tuxedo portvalue [virtual_service]

This command is used to display the setting of the WSL port number and WSH port range for the specified Tuxedo virtual service. If the “virtual_service” parameter is not specified, the settings of the WSL port number and WSH port range for all Tuxedo virtual services will be displayed.

slb tuxedo portposition [position]

This command is used to set the offset of the position at which to place the WSH port numbers. If this command is not configured, WSH port numbers will be placed at the 79 and 80 bytes of the WSL response by default.

position	Optional. This parameter specifies the offset of the position at which to place the WSH port number.
----------	--

The default value is 78.

no slb tuxedo portposition

This command is used to delete the position setting for WSH port numbers.

show slb tuxedo portposition

This command is used to display the position setting for WSH port numbers.

slb virtual l2ip <virtual_service> <vip> [gateway_ip]

This command is used to configure a Layer 2 IP virtual service. The number of Layer 2 IP virtual services is not limited.

virtual_service This parameter specifies the virtual service name. Its value must be a case-sensitive string of 1 to 128 characters. Numbers, letters, and special characters are supported. The special characters include the at sign (@), exclamation mark (!), tilde (~), underscore (_), colon (:), hyphen (-), angle brackets (< >), and period (.).

When the parameter value contains special characters, it must be enclosed in double quotes. The specified virtual service name cannot be the system reserved word “default”, “all” or “global”, regardless of its case sensitivity.

vip This parameter specifies the virtual service’s IP address. Its value must be an IPv4 or IPv6 address.

gateway_ip Optional. This parameter specifies the IP address of the gateway for the specified virtual service.

The default IPv4 gateway address is “0.0.0.0”, and the default IPv6 gateway address is “::”.

no slb virtual

{http|https|dns|dnstcp|ftp|ftps|tcp|tcps|udp|ip|siptcp|sipudp|radauth|radacct|rdp|rtsp|l2ip|diameter|tuxedo} <virtual_service>

This command is used to delete a virtual service of the specified protocol type and all policies associated with this virtual service.

show slb virtual

{http|https|dns|dnstcp|ftp|ftps|tcp|tcps|udp|ip|siptcp|sipudp|radauth|radacct|rdp|rtsp|l2ip|diameter|tuxedo} [virtual_service]

This command is used to display the virtual service of the specified protocol type. If the “virtual_service” parameter is not specified, all virtual services of the specified protocol type will be displayed.

clear slb virtual

{http|https|dns|dnstcp|ftp|ftps|tcp|tcps|udp|ip|siptcp|sipudp|radauth|radacct|rdp|rtsp|l2ip|diameter|tuxedo}

This command is used to clear all virtual services of the specified protocol type.

slb virtual {enable|disable} <virtual_service>

This command is used to enable or disable the specified virtual service. A virtual service that is disabled will not provide the SLB service.

show slb virtual all

This command is used to display virtual services of all protocol types and those that are disabled.

Basic Virtual Service Configurations**slb mode regexcase {on|off} [virtual_service|vlink_name]**

This function is used to enable or disable the function of ignoring the case sensitivity of strings to be matched for the specified virtual service or vlink. This command applies only to the “**slb policy regex**”, “**slb policy header**”, “**http rewrite request url**”, “**http rewrite response url**”, “**http redirect url**”, and “**http rewrite response cachecontrol**” commands. After this function is enabled, the system does not distinguish between upper-case and lower-case letters when trying to match the strings set in these commands. By default, this function is disabled globally. When this function is globally disabled, it does not take effect for any specific virtual service or vlink. When this function is globally enabled, the configuration for specific virtual service or vlink will override the global configuration. If we do not configure any specific virtual service or vlink, the global configuration will take effect.

virtual_service|vlink_name Optional. This parameter specifies the virtual service or vlink name.

If this parameter is not specified, the function of ignoring the case sensitivity of strings to be matched is enabled or disabled for all virtual services and vlinks.

show slb mode regexcase

This command is used to check whether the function of ignoring the case sensitivity of strings to be matched is enabled.

slb virtual application cps <virtual_service> <max_cps>

This command is used to set the maximum number of connections per second (CPS) allowed on the specified virtual service.

virtual_service This parameter specifies the virtual service name. Its value must be a TCP virtual service, such as FTP, TCP, HTTP, and RDP virtual services.

max_cps This parameter specifies the maximum number of CPS. Its value must be an integer ranging from 1 to 4,294,967,295.

no slb virtual application cps <virtual_service>

This command is used to delete the setting of the maximum number of CPS for the specified virtual service.

show slb virtual application cps [virtual_service]

This command is used to display the setting of the maximum number of CPS for the specified virtual service. If the “virtual_service” parameter is not specified, the settings of the maximum number of CPS for all virtual services will be displayed.

clear slb virtual application cps

This command is used to clear the settings of the maximum number of CPS for all virtual services.

slb virtual application bandwidth <virtual_service> <soft_bandwidth_limit> [hard_bandwidth_limit]

This command is used to set the bandwidth limit for the specified virtual service.

virtual_service	This parameter specifies the name of the virtual service.
soft_bandwidth_limit	This parameter specifies the soft bandwidth limit in Kbps. Its value must be an integer ranging from 1 to 4,294,967,295. If the used bandwidth of the virtual service exceeds the soft bandwidth limit, the system will generate a “Warning” log and continue to serve requests.
hard_bandwidth_limit	Optional. This parameter specifies the hard bandwidth limit in Kbps. Its value must be an integer ranging from 1 to 4,294,967,295. • If the used bandwidth of the virtual service exceeds the hard bandwidth limit, the system will generate a “Warning” log but will not serve requests. • If this parameter is not specified, the value of the “soft_bandwidth_limit” parameter will be used by default.

no slb virtual application bandwidth <virtual_service>

This command is used to delete the bandwidth limit setting for the specified virtual service.

show slb virtual application bandwidth [virtual_service]

This command is used to display the bandwidth limit setting of the specified virtual service. If the “virtual_service” parameter is not specified, the bandwidth limit settings of all virtual services will be displayed.

clear slb virtual application bandwidth

This command is used to delete the bandwidth limit settings of all virtual services.

slb virtual health {on|off}

This command is used to enable or disable the virtual service health check function. After this function is enabled, if all real services associated with a virtual service are unavailable, the status of the virtual service will be marked as “Down”. Clients will not send requests to this virtual service. By default, this function is disabled.

The total number of virtual service health checks depends on the system memory:

System Memory	Number of virtual service health checks
<= 32 GB	4000
>= 64 GB	8000

slb virtual settings description <virtual_service> <description>

This command is used to configure the description information of the specified virtual service.

virtual_service	This parameter specifies the name of an existing virtual service.
description	This parameter specifies the virtual service description information. Its value must be a string of 1 to 255 characters.

no slb virtual settings description <virtual_service>

This command is used to delete the description information of the specified virtual service.

show slb virtual settings description [virtual_service]

This command is used to display the description information of the specified virtual service. If the “virtual_service” parameter is not specified, the description information of all virtual services will be displayed.

clear slb virtual settings description

This command is used to delete the description information of all virtual services.

slb virtual settings ecs <virtual_service>

This command is used to enable the ECS (EDNS client subnet) function for the specified virtual service, that is, to insert the source IP address of the client into the DNS request when forwarding the DNS request.

virtual_service This parameter specifies the name of an existing DNS type virtual service.

no slb virtual settings ecs <virtual_service>

This command is used to disable the ECS (EDNS client subnet) function for the specified virtual service.

show slb virtual settings ecs [virtual_service]

This command is used to display the ECS configuration of the specified virtual service. If the “virtual_service” parameter is not specified, the configurations of all virtual services will be displayed.

clear slb virtual settings ecs

This command is used to delete the ECS configurations of all virtual services.

slb virtual settings rts <virtual_service>

This command is used to enable the Return to Source (RTS) function on the specified virtual service. The RTS table supports a maximum of 64 security devices.

virtual_service This parameter specifies the virtual service name.

no slb virtual settings rts <virtual_service>

This command is used to disable the RTS function on the specified virtual service.

clear slb virtual settings rts

This command is used to disable RTS on all virtual services.

show slb virtual settings rts [virtual_service]

This command is used to display the enabling status of RTS on the specified virtual service. If the “virtual_service” parameter is not specified, the enabling status of RTS on all virtual services is displayed.

slb virtual settings interface <virtual_service><interface_name>

This command is used to define the serving interface for the specified virtual service. By default, the virtual service provides services on all interfaces.

virtual_service This parameter specifies the virtual service name.

interface_name This parameter specifies the interface name. Its value must be the system interface, VLAN interface or bond interface name.

no slb virtual settings interface <virtual_service> <interface_name>

This command is used to disable a serving interface for the specified virtual service. If the disabled interface is the only serving interface of the virtual service, the serving interface settings of this virtual service will be restored to defaults, that is, the virtual service will provide services on all interfaces.

clear slb virtual settings interface

This command is used to restore the serving interface settings of all virtual services to defaults, that is, the virtual services will provide services on all interfaces.

show slb virtual settings interface [virtual_service]

This command is used to display the serving interface settings for the specified virtual service. If the “virtual_service” parameter is not specified, the serving interfaces setting for all virtual services are displayed.

show slb summary [virtual_service|vlink_name]

This command is used to display configurations related to the specified virtual service or vlink. If the “virtual_service|vlink_name” parameter is not specified, the configurations related to all virtual services and vlinks will be displayed.



Note: If the “ip|domain” parameter in the commands “**slb real http**”, “**slb real https**”, “**slb real ftp**”, “**slb real tcp**”, “**slb real tcps**” and “**slb real ip**” set to “domain”, the IP address corresponding to the domain name of the real service will be included in the displayed real services.

The following is an example output of this command:

```
AN(config)#show slb summary

#virtual service or vlink
slb virtual http "v1" 10.8.6.235 80 arp 0

#slb policy setting topology(only show 3 layers if policy nesting exists)
slb policy default "v1" "g1"
    slb group method "g1" rr
        slb group member "g1" "r1" 1 0
            slb real http "r1" 10.3.0.20 80 1000 tcp 3 3
            slb real health "hc_1" "r1" 10.8.6.45 80 tcp 3 3
            slb real health "HC_1" "r1" 10.8.6.45 80 tcp 3 3
        slb group member "g1" "r3" 3 0
            slb real http "r3" (www.baidu.com)10.3.0.57 80 10 tcp 3 3
```

Port Ranges of Virtual Services

slb virtual portrange <virtual_service> <start_port> <end_port> [protocol]
[port_type]

This command is used to define a port range for the specified virtual service, so that only traffic hitting the specified port range will be load balanced. A port range can be defined for a virtual service only when this virtual service's port number is set to 0. A maximum of three port ranges can be configured for each virtual service. Duplicate port numbers among the port ranges for one virtual service are not allowed. This also applies to the port ranges for multiple virtual services that share the same IP address.

virtual_service	This parameter specifies the virtual service name. FTP and FTPS virtual services are not supported.
start_port	This parameter specifies the start port number of the port range. Its value must be an integer ranging from 0 to 65,535.
end_port	This parameter specifies the end port number of the port range. Its value must be an integer ranging from 0 to 65,535.
protocol	Optional. This parameter specifies the protocol type. It is available only when the "virtual_service" parameter is set to a Layer 2 virtual service. Its value must be "tcp", "udp", or "all". The default value is "all".
port_type	Optional. This parameter specifies the port type. It is available only when the "virtual_service" parameter is set to a Layer 2 virtual service. Its value must be: • src: indicates the source port. • dst: indicates the destination port. The default value is "dst".

no slb virtual portrange <virtual_service> <start_port> <end_port> [protocol]

This command is used to delete the port range configurations for the specified virtual service.

ftp passive portrange <start_port> <end_port> [virtual_service]

This command is used to configure a port range for data connections in passive FTP or FTPS for the specified virtual service. The system supports 20 to 1000 port ranges, and each port range can contain a maximum of 1000 port numbers.

start_port	This parameter specifies the start port number of the port range. Its value must be an integer ranging from 1024 to 65,535.
end_port	This parameter specifies the end port number of the port range. Its value must be an integer ranging from 1024 to

65,535.

virtual_service	Optional. This parameter specifies the virtual service name. Its value must be an FTP or FTPS virtual service. If this parameter is not specified, a global port range for data connections in passive FTP or FTPS will be configured.
-----------------	--

no ftp passive portrange *[virtual_service]*

This command is used to delete the port range configuration for the specified FTP or FTPS virtual service. If the “virtual_service” parameter is not specified, the global port range configuration will be deleted.

show ftp passive portrange *[virtual_service]*

This command is used to display the port range configuration for the specified FTP or FTPS virtual service. If the “virtual_service” parameter is not specified, the global port range configuration will be displayed.

clear ftp passive portrange

This command is used to clear the port range configurations for the global and individual FTP and FTPS virtual services.

Transfer Mode for Virtual Services

system mode {reverse|transparent|triangle} *[virtual_service]*
[triangle_work_mode]

This command is used to configure the system mode for the specified virtual service. The system supports the following system modes: proxy reverse mode, transparent mode, and triangle mode.

By default, the global system mode is proxy reverse mode.

virtual_service	Optional. This parameter specifies the virtual service name. If this parameter is not specified, the global system mode is configured.
-----------------	--

triangle_work_mode	Optional. This parameter specifies the working mode of the triangle mode. Its value must be either “direct” or “route”:
--------------------	---

- direct: the IP address of the virtual service and that of the real service must be on the same network segment.
- route: the IP address of the virtual service and that of the real service must be on different network segments.

The default value is “direct”.

Note: When the triangle mode is both globally and individually configured, the working mode of the specific virtual service will override the global configuration.

**Note:**

- If no system mode is set for a virtual service, it will use the global system mode.
- Only TCP, UDP and IP virtual services support the triangle mode.
- SIP UDP and SIP TCP virtual services do not support the reverse proxy mode. In reverse proxy mode, these two types of virtual services still work in transparent mode.
- L2 IP and RTSP virtual services do not support the transparent mode.

no system mode <virtual_service>

This command is used to delete the system mode configuration for the specified virtual service.

show system mode [virtual_service]

This command is used to display the system mode configuration for the specified virtual service. If the “virtual_service” parameter is not specified, the global system mode configuration is displayed.

clear system mode

This command is used to clear the system mode configurations for all virtual services.

slb snat virtual <virtual_service> <network_ip> <netmask|prefix>

This command is used to define a Source Network Address Translation (SNAT) rule.

The SNAT rule provides two functions:

1. For a virtual service working in transparent mode and configured with an SNAT rule, if the source IP address in the client request is within the IP network segment defined by the SNAT rule, the client request will be forwarded through the reverse proxy mode; otherwise, the client request will be forwarded through the transparent mode.
2. For a virtual service working in reverse proxy mode and configured with an SNAT rule, if the source IP address in the client request is within the IP network segment defined by the SNAT rule, the client request will be forwarded through the transparent mode; otherwise, the client request will be forwarded through the reverse proxy mode.

A maximum of 10 SNAT rules can be configured for each virtual service.

virtual_service This parameter specifies the name of the virtual service.

network_ip	This parameter specifies an IP network segment to filter the source IP address. Its value must be an IPv4 or IPv6 address.
netmask prefix	This parameter specifies the netmask or prefix length of the IP network segment. • The parameter “netmask” is used for an IPv4 address. It can be a dotted IP address or an integer. If it is an integer, its value must be an integer ranging from 0 to 32. • The parameter “prefix” is used for an IPv6 address. its value must be an integer ranging from 0 to 128.



Note:

- The SNAT rule can be configured for IP, TCP, TCPS, UDP, FTP, FTPS, HTTP and HTTPS virtual services only.
- The SNAT rule might not take effect when the IP addresses of the virtual services and real services are not IPv4 or IPv6 at the same time.

no slb snat virtual <virtual_service> <network_ip> <netmask|prefix>

This command is used to delete an SNAT rule of the specified virtual service.

show slb snat virtual [virtual_service]

This command is used to display all the SNAT rules of the specified virtual service. If the parameter “virtual_service” is not specified, all the SNAT rules in the system will be displayed.

clear slb snat virtual [virtual_service]

This command is used to remove all the SNAT rules of the specified virtual service. If the parameter “virtual_service” is not specified, all the SNAT rules in the system will be removed.

TCP Options for Virtual Services

slb tcpoption wscale <virtual_service> {on|off} [shift_count]

This command is used to enable or disable the TCP Window Scale option for the specified virtual service. By default, this option is disabled.

virtual_service	This parameter specifies the virtual service name.
shift_count	Optional. This parameter specifies the number of bits by which the TCP window is right shifted. Its value must be an integer ranging from 0 to 14. This parameter is available only when the TCP Window Scale option is enabled.

The default value is 3.



Note: Actually used window size = Window size in the TCP packet x 2shift_count.

slb tcption timestamp <virtual_service> {on|off}

This command is used to enable or disable the TCP Timestamp option for the specified virtual service. By default, this function is disabled.

virtual_service This parameter specifies the virtual service name.

slb tcption sack <virtual_service> {on|off}

This command is used to enable or disable the TCP Selective Acknowledgment (SACK) option for the specified virtual service. By default, this function is disabled.

virtual_service This parameter specifies the virtual service name.

slb tcption mss <virtual_service> [size]

This command is used to set the TCP maximum segment size (MSS) for the specified virtual service. When this command is not configured or the MSS for a virtual service is set to 0, the setting will not be displayed by the “**show slb tcption**” and “**show run tcption**” commands.

virtual_service This parameter specifies the virtual service name.

size Optional. This parameter specifies the TCP MSS, in bytes. Its value must be an integer ranging from 536 to 1460.

The default value is 0, indicating that the system MSS will be used.



Note: To make the MSS option take effect for the system or a specified virtual service, the TCP Syncache function must be enabled first.

show slb tcption [virtual_service]

This command is used to display the TCP MSS setting and the status of the following functions for the specified virtual service:

- TCP Window Scale option
- TCP Timestamp option
- TCP SACK

If the “virtual_service” parameter is not specified, the TCP MSS settings and the status of the functions above for all virtual services will be displayed.

HTTP/2 Virtual Service Support

HTTP and HTTPS virtual services support both HTTP/1 and HTTP/2. By default, only HTTP/1 is enabled for HTTP and HTTPS virtual services, and HTTP/2 is disabled. After HTTP/2 is enabled for an HTTP/HTTPS virtual service, it will support both HTTP/1 and HTTP/2 requests and responses.

- HTTP/1 requests are directly forwarded to real service based on the load balance decision, regardless of whether real service supports HTTP/2. No protocol switch is involved.
- Processing of HTTP/2 requests depends on whether real service supports HTTP/2:
 - If real service supports and is enabled with HTTP/2, HTTP/2 requests will be directly forwarded to the real service based on the load balance decision.
 - If real service does not support HTTP/2 or is not enabled with HTTP/2, HTTP/2 requests will be first transformed into HTTP/1 requests and then forwarded to real service. (HTTP/1 responses returned from real service will be first transformed into HTTP/2 responses and then forwarded to client.)

http2 virtual {on|off} <virtual_service>

This command is used to enable or disable HTTP/2 for the specified virtual service. By default, HTTP/2 is disabled.

virtual_service	This parameter specifies an existing virtual service name. Its value must be an HTTP or HTTPS virtual service.
-----------------	--



Note:

- After the administrator enables HTTP/2 for an HTTPS-type virtual service, the virtual service will support the ALPN extension.
- When HTTP/2 is enabled for a virtual service, its associated real service group cannot support the “**slb group option itcpopt**” command.

show http2 virtual [virtual_service]

This command is used to display the enabling status of HTTP/2 for an HTTP/HTTPS virtual service. If the “virtual_service” parameter is not specified, the enabling status of HTTP/2 for all HTTP/HTTPS virtual services will be displayed.

clear http2 virtual

This command is used to disable HTTP/2 for all HTTP/HTTPS virtual services.

Layer 2 SLB Local Exception IP

slb l2vs ipexception <except_ip> [virtual_service]

This command is used to add a local exception IP for the specified L2IP virtual service. After configuring the local exception IP, packets whose the destination IP matches the local exception IP (such as VIP, interface IP, etc.) will still be able to hit the L2IP virtual service and be forwarded to other devices (such as security device), and then be returned to the APV appliance for L4/L7 load balancing. If the “virtual_service” parameter is not specified, this exception IP will take effect for all L2IP virtual services. The system supports configuring a maximum of 100 exception IPs for each L2IP virtual service.

except_ip	This parameter specifies the local exception IP address. Its value must be an IPv4 or IPv6 address.
virtual_service	Optional. This parameter specifies an existing L2IP virtual service. If this parameter is not specified, the system will add the local exception IP for all L2IP virtual services.

no slb l2vs ipexception <except_ip> [virtual_service]

This command is used to delete the specified local exception IP for the L2IP virtual service. If the “virtual_service” parameter is not specified, this command will delete the specified local exception IP for all L2IP virtual services.

show slb l2vs ipexception

This command is used to display the local exception IP configuration of all L2IP virtual services.

clear slb l2vs ipexception [virtual_service]

This command is used to delete all local exception IP configurations of the specified L2IP virtual service. If the “virtual_service” parameter is not specified, this command will clear the local exception IP configuration of all L2IP virtual services.

slb l2group ipexception <except_ip> [group_name]

This command is used to add a local exception IP for the specified L2 real service group (whose members are L2 real services). After configuring the local exception IP, packets whose the destination IP matches the local exception IP (such as VIP, interface IP, etc.) will still be able to hit the L2 real service group and be forwarded (such as forwarding to a security device), and then be returned to the APV appliance for L4/L7 load balancing. If the “group_name” parameter is not specified, this exception IP will take effect for all L2 real service groups. The system supports configuring a maximum of 100 exception IPs for each L2 real service group.

except_ip	This parameter specifies the local exception IP address. Its value must be an IPv4 or IPv6 address.
group_name	Optional. This parameter specifies an existing L2 real

service group. If this parameter is not specified, the system will add the local exception IP for all L2 real service groups.

no slb l2group ipexception <except_ip> [group_name]

This command is used to delete the specified local exception IP for the L2 real service group. If the “group_name” parameter is not specified, this command will delete the specified local exception IP for all L2 real service groups.

show slb l2group ipexception

This command is used to display the local exception IP configuration of all L2 real service groups.

clear slb l2group ipexception [group_name]

This command is used to delete all local exception IP configurations of the specified L2 real service group. If the “group_name” parameter is not specified, this command will clear the local exception IP configuration of all L2 real service groups.

Other Configurations**slb forwardip <virtual_service> <tcp_option_kind> [offset]**

This command is used to configure the specified virtual service to extract client IP address from the specified TCP option when receiving a client’s TCP handshake ACK message. Both IPv4 and IPv6 addresses are supported.

This function can be used together with the “**http xforwardedfor**” or “**slb group option itcport**” command to forward the obtained client IP address to real services.

virtual_service	This parameter specifies the virtual service name. Its value must be a TCP, TCPS, HTTP or HTTPS virtual service.
tcp_option_kind	This parameter specifies the kind value of the TCP option. It must be an integer ranging from 20 to 254.
offset	Optional. This parameter specifies the offset of the TCP option kind’s Value field, in bytes. Its value must be an integer ranging from 0 to 4. The default value is 0.

no slb forwardip <virtual_service>

This command is used to delete the “**slb forwardip**” configuration for the specified virtual service.

show slb forwardip [virtual_service]

This command is used to display the “**slb forwardip**” configuration for the specified virtual service. If the “virtual_service” parameter is not specified, the “**slb forwardip**” configurations for all virtual services will be displayed.

clear slb forwardip [*virtual_service*]

This command is used to clear the “**slb forwardip**” configuration for the specified virtual service. If the “virtual_service” parameter is not specified, the “**slb forwardip**” configurations for all virtual services will be cleared.

Load Balancing Policy

In SLB, the policy associates the virtual service with the real service group.

SLB supports numerous types of policies to meet the deployment requirements. Different types of policies have different priorities. The default priority orders of the policies are as follows (the priority orders of the policies in italic fonts can be adjusted using the customized policy order template.):

1. DoH
2. Redirect
3. Static
4. DNSSEC
5. *QoS Clientport (1)*
6. *QoS Network (2)*
7. *Persistent URL (3)*
8. *Rewrite Cookie (4)*
9. *Insert Cookie (5)*
10. *Persistent Cookie (6)*
11. *QoS Cookie (7)*
12. *QoS Hostname (8)*
13. *QoS URL (9)*
14. *QoS Body (10)*
15. *QoS Dnsdomain (11)*
16. *QoS Dnsqtype (12)*
17. *Regex (13)*

- 18. Header (14)
- 19. Hash URL (15)
- 20. Raduname (RADIUS Username)
- 21. Radsid (RADIUS Session ID)
- 22. Filetype
- 23. QoS Diameter
- 24. QoS Diameterappid
- 25. SIP Domain
- 26. SIP Call
- 27. Default
- 28. Backup



Note: Different from other types of policies, the static policy associates the virtual service with the real service.

When receiving a client request for accessing the virtual service, the system will try to match the client request with the all types of policies for the virtual service in the descending order of the policy priority. If the client request matches a policy and the associated real service group has available real services, the client request hits this policy and the system will stop matching the client request with other policies; if the associated real service group has no available real service (all real services are down, disabled or overloaded), the client request does not hit this policy and the system will continue matching the client request with other policies in the priority order. If the client request does not match any policy with a higher priority than the Default policy, it will match the Default policy. If the associated real service group of the Default policy has available real services, the client request hits the Default policy; otherwise, the client request matches the Backup policy. If the associated real service group of the Backup policy has available real services, the client request hits the Backup policy; otherwise, the system rejects the client request.



Note: The client request can match the Backup policy only when the client request has matched at least one type of policy but the associated real service group of the policy has no available real services. If the client request does not match any policy, it cannot match the Backup policy.

In addition, the system allows configuring multiple policies of the specific type for the virtual service, such as the QoS Clientport policy. These policies of the same type are configured with priorities comparing to the other policies of the same type for the virtual service. When trying to match the client request with this type of policy, the system will try to match the client request with these policies of the same type in

descending order of the comparative priority. The system will continue matching the client request with the policy of the type in the next order only when the client request does not hit any policy of this type.

Different types of virtual service support different types of policies. For example, the TCP/UDP virtual service supports only the Static, QoS Clientport, QoS Network, Default and Backup policies. For the virtual service of the specific type, you can create a custom policy order template using the “**slb policy order**” command and associate the custom policy order template with the specific virtual service using the “**slb virtual order**” command. If no custom policy order template is associated with a virtual service, the virtual service will use the Default policy priority order. For more information, please refer to the section Policy Order Template.

The SLB function supports policy nesting. Policy nesting is implemented using the vlink. When using the policy nesting, you can configure a high-priority policy to associate a virtual service with a vlink and then configure a low-priority policy to associate the vlink with a real service group or another vlink. SLB supports three-layer policy nesting at most. For more information, please refer to the section Policy Nesting.

Defining the Load Balancing Policy



Note: In the following CLIs used for configuring policies, the “virtual_service|vlink_name” parameter indicates that the virtual service can be replaced by the vlink, and the “group_name|vlink_name” parameter indicates that the real service group can be replaced by the vlink.

slb policy doh <virtual_service> <group_name>

This command is used to configure a DNS over HTTPS (DoH) policy to associate an HTTPS-type virtual service with a DNS-type group.

virtual_service	This parameter specifies the name of the HTTPS-type virtual service.
group_name	This parameter specifies the name of the DNS-type group. The group must use any of the following methods: rr, grr, lc, sr, lb, ph, pi, persistence, hi, sslsid, hi, hip, snmp, and chh and chi.



Note:

- The DoH policy does not support vlink.
- The DoH policy does not support source authentication originated from DNS real services.
- The DoH policy does not support DNS64 or DNS46 transition.
- For the HTTPS virtual service with the DoH policy configured, the SSF function does not work.

- The HTTPS virtual service with the DoH policy configured does not support chunked requests.

no slb policy doh *<virtual_service>*

This command is used to delete the DoH policy configured for the specified HTTPS-type virtual service.

show slb policy doh *[virtual_service]*

This command is used to display the DoH policy configured for the specified HTTPS-type virtual service. If the “virtual_service” parameter is not specified, the DoH policies configured for all HTTPS-type virtual services are displayed.

clear slb policy doh

This command is used to clear the DoH policies configured for all HTTPS-type virtual services.

slb policy redirect *<policy_name> <virtual_service> <group_name> <redirected_from_host>*

This command is used to configure a Redirect policy for the specified HTTP or HTTPS virtual service. When the host name in the client request for accessing this virtual service matches the value of the “redirected_from_host” parameter, the client request hits this policy. The appliance will return the client an HTTP redirection response, in which the host part in the redirect URL is the name of the specific real service in the associated real service group.

Multiple Redirect policies can be configured for every HTTP or HTTPS virtual service. However, the values of the “redirected_from_host” parameter in these policies must be different from each other.

policy_name	This parameter specifies the policy name. Its value must be a string of 1 to 128 characters. Its value must be enclosed in double quotes when containing any special character or containing only numbers.
virtual_service	This parameter specifies the name of an existing HTTP or HTTPS virtual service.
group_name	This parameter specifies the name of an existing real service group.
redirected_from_host	This parameter specifies the string used to match the host name in the client HTTP request. Its value must be a case-sensitive string of 1 to 512 characters. The parameter value supports full regular expressions. When the parameter value is a full regular expression, it must begin with the “<regex>” string and be enclosed in double quotes, for example,

“<regex>/(graphics|photos|resource)/(.)\.(gif|jpg|png)” means matching all GIF, JPG, and PNG files in the paths “/graphics”, “/photos”, and “/resource”. For more information, please refer to introductions about regular expressions in Chapter 1.

To ensure that a configured regular expression string is correct, administrators can use the “**slb regextest** <regex> <target_string>” command to test whether a target string will match the configured regular expression.



Note: For the redirect policy to work, please make sure that the names of the real services in the associated real service group are domain names or IP addresses.

no slb policy redirect <policy_name>

This command is used to delete the specified Redirect policy.

show slb policy redirect [policy_name]

This command is used to display the specified Redirect policy. If the “policy_name” parameter is not specified, all Redirect policies will be displayed.

clear slb policy redirect

This command is used to clear all Redirect policies.

slb policy static <virtual_service> <real_service>

This command is used to configure a Static policy for the specified virtual service to associate it with the specified real service. When the client request for accessing this virtual service hits the Static policy, it will be forwarded to the associated real service. Only one Static policy can be configured for every virtual service. Note: HTTP/2-enabled virtual services do not support the Static policy.

virtual_service	This parameter specifies the name of an existing virtual service.
-----------------	---

real_service	This parameter specifies the name of an existing real service.
--------------	--

no slb policy static <virtual_service>

This command is used to delete the Static policy of the specified virtual service.

show slb policy static [virtual_service]

This command is used to display the Static policy of the specified virtual service. If the “virtual_service” parameter is not specified, the Static policies of all virtual services will be displayed.

clear slb policy static

This command is used to clear all the Static policies of all virtual services.

slb policy dnssec <policy_name> <virtual_service> <group_name>

This command is used to configure a Domain Name System Security Extensions (DNSSEC) policy. By using the DNSSEC policy, DNSSEC queries destined for this DNS virtual service will be forwarded to one DNS real service supporting DNSSEC in the associated real service group. Only one DNSSEC policy can be configured for a DNS virtual service. For the DNS virtual service, the priority of the DNSSEC policy is lower than the static policy and higher than other policies.

policy_name This parameter specifies the name of the DNSSEC policy. Its value must be a string of 1 to 128 characters, which must be enclosed in double quotes when beginning with a non-alphabetical character.

virtual_service This parameter specifies the name of the DNS virtual service name.

group_name This parameter specifies the name of the real service group. The DNS real services in this group must support DNSSEC; otherwise, the DNSSEC queries might fail to be processed.



Note:

- DNSSEC responses cannot be saved by DNS Cache.
- The DNSSEC policy cannot be used for Layer 2 IP/MAC-based SLB.
- The DNSSEC policy cannot be used in triangle mode.

no slb policy dnssec <policy_name>

This command is used to delete the specified DNSSEC policy.

show slb policy dnssec [policy_name]

This command is used to display the specified DNSSEC policy. If the “policy_name” parameter is not specified, all DNSSEC policies will be displayed.

clear slb policy dnssec

This command is used to clear all DNSSEC policies.

**slb policy qos clientport <policy_name> {virtual_service|vlink_name}
{group_name|vlink_name} <network_ip> {netmask|prefix} <start_port>
<end_port> <precedence>**

This command is used to configure a QoS Clientport policy for the specified virtual service or vlink. When the source IP address and port in the client request for accessing this virtual service belong to the network segment and port range specified by this policy respectively, the client request hits this policy. Multiple QoS Clientport policies can be configured for every virtual service or vlink.

policy_name	This parameter specifies the policy name. Its value must be a string of 1 to 128 characters. Its value must be enclosed in double quotes when containing any special character or containing only numbers.
virtual_service vlink_name	This parameter specifies the name of an existing virtual service or vlink.
group_name vlink_name	This parameter specifies the name of an existing real service group or vlink.
network_ip	This parameter specifies the network IP address. Its value must be an IPv4 or IPv6 address.
netmask prefix	This parameter specifies the netmask or prefix of the network IP address. • “netmask” indicates the netmask of the IPv4 address. Its value must be in dotted decimal notation or an integer ranging from 0 to 32. • “prefix” indicates the prefix length of the IPv6 address. Its value must be an integer ranging from 0 to 128.
start_port	This parameter specifies the start port of the port range. Its value must be an integer ranging from 0 to 65,535.
end_port	This parameter specifies the end port of the port range. Its value must be an integer ranging from 0 to 65,535. Its value must be equal to or greater than the value of the “start_port” parameter.
precedence	This parameter specifies the priority of the policy comparing to the policies of the same type for the virtual service. Its value must be an integer ranging from 0 to 65,535. The smaller the value, the higher the priority.

no slb policy qos clientport <policy_name>

This command is used to delete the specified QoS Clientport policy.

show slb policy qos clientport [policy_name]

This command is used to display the specified QoS Clientport policy. If the “policy_name” parameter is not specified, all QoS Clientport policies will be displayed.

clear slb policy qos clientport

This command is used to clear all QoS Clientport policies.

slb policy qos network <policy_name> {virtual_service|vlink_name}
{group_name|vlink_name} <network_ip> {netmask|prefix} <precedence>

This command is used to configure a QoS Network policy for the specified virtual service or vlink. When the source IP address in the client request for accessing this virtual service belongs to the network segment specified by this policy, the client request hits this policy. Multiple QoS Network policies can be configured for every virtual service or vlink.

policy_name	This parameter specifies the policy name. Its value must be a string of 1 to 128 characters. Its value must be enclosed in double quotes when containing any special character or containing only numbers.
virtual_service vlink_name	This parameter specifies the name of an existing virtual service or vlink.
group_name vlink_name	This parameter specifies the name of an existing real service group or vlink.
network_ip	This parameter specifies the network IP address. Its value must be an IPv4 or IPv6 address.
netmask prefix	This parameter specifies the netmask or prefix of the network IP address. • “netmask” indicates the netmask of the IPv4 address. Its value must be in dotted decimal notation or an integer ranging from 0 to 32. • “prefix” indicates the prefix length of the IPv6 address. Its value must be an integer ranging from 0 to 128.
precedence	This parameter specifies the priority of the policy comparing to the policies of the same type for the virtual service. Its value must be an integer ranging from 0 to 65,535. The smaller the value, the higher the priority.

no slb policy qos network <policy_name>

This command is used to delete the specified QoS Network policy.

show slb policy qos network [policy_name]

This command is used to display the specified QoS Network policy. If the “policy_name” parameter is not specified, all QoS Network policies will be displayed.

clear slb policy qos network

This command is used to clear all QoS Network policies.

slb policy qos dnsdomain <policy_name> <virtual_service> <group_name>
<host_name> <precedence>

This command is used to configure a QoS Dnsdomain policy for the specified DNS virtual service. When the DNS request destined to this virtual service matches the domain name specified by the “host_name” parameter, the client request hits this policy. The appliance will send it to the specified real service group.

policy_name This parameter specifies the policy name. Its value must be a string of 1 to 128 characters. Its value must be enclosed in double quotes when containing any special character or containing only numbers.

virtual_service This parameter specifies the name of an existing DNS virtual service.

group_name This parameter specifies the name of an existing real service group.

host_name This parameter specifies the domain name string used to match the DNS query. Its value must be a case-sensitive string of 1 to 512 characters. The parameter value supports quick regular expressions. When the parameter value is set to a regular expression, it must be enclosed in double quotes. For information about the quick regular expression please refer to the “Regular Expressions” section in Chapter 1.

To ensure that a configured regular expression string is correct, administrators can use the “**slb regextest** <regex> <target_string>” command to test whether a target string will match the configured regular expression.

precedence This parameter specifies the priority of the policy comparing to the policies of the same type for the virtual service. Its value must be an integer ranging from 0 to 65,535. The smaller the value, the higher the priority.

no slb policy qos dnsdomain <policy_name>

This command is used to delete the specified QoS Dnsdomain policy.

show slb policy qos dnsdomain [policy_name]

This command is used to display the specified QoS Dnsdomain policy. If the “policy_name” parameter is not specified, all QoS Dnsdomain policies will be displayed.

clear slb policy qos dnsdomain

This command is used to clear all QoS Dnsdomain policies.

slb policy qos dnsqtype <policy_name> {virtual_service|vlink_name}
{group_name|vlink_name} <query_type> <precedence>

This command is used to configure a QoS Dnsqtype policy for the specified DNS virtual service or vlink. When the DNS request destined to this virtual service matches the record type specified by the “query_type” parameter, the client request hits this policy. The appliance will send it to the specified real service group.

policy_name	This parameter specifies the policy name. Its value must be a string of 1 to 128 characters. Its value must be enclosed in double quotes when containing any special character or containing only numbers.
virtual_service vlink_name	This parameter specifies the name of an existing DNS-type virtual service or vlink.
group_name vlink_name	This parameter specifies the name of an existing real service group or vlink.
query_type	This parameter specifies the type of the DNS query. Its value must be “A” or “AAAA”.
precedence	This parameter specifies the priority of the policy comparing to the policies of the same type for the virtual service. Its value must be an integer ranging from 0 to 65,535. The smaller the value, the higher the priority.

no slb policy qos dnsqtype <policy_name>

This command is used to delete the specified QoS Dnsqtype policy.

show slb policy qos dnsqtype [policy_name]

This command is used to display the specified QoS Dnsqtype policy. If the “policy_name” parameter is not specified, all QoS Dnsqtype policies will be displayed.

clear slb policy qos dnsqtype

This command is used to clear all QoS Dnsqtype policies.

slb policy persistent url <policy_name> {virtual_service|vlink_name}
<group_name> <url_tag> <precedence>

This command is used to configure a Persistent URL policy for the specified virtual service or vlink. When the URL query part (the part after the “?” character in the URL) of the client request for accessing this virtual service includes the “url_tag” value specified by this policy, the client request hits this policy. The URL query part consists of one or more “tag=value” pairs.

The Persistent URL policy can associate the virtual service only with the real service group using the pu, hq or persistence method. Multiple Persistent URL policies can be configured for every virtual service or vlink.

policy_name	This parameter specifies the policy name. Its value must be a string of 1 to 128 characters. Its value must be enclosed in double quotes when containing any special character or containing only numbers.
virtual_service vlink_name	This parameter specifies the name of an existing virtual service or vlink.
group_name	This parameter specifies the name of an existing real service group.
url_tag	This parameter specifies the string used to match with the URL tag name in the URL query part of the client request. Its value must be a string of 1 to 512 characters.
precedence	This parameter specifies the priority of the policy comparing to the policies of the same type for the virtual service. Its value must be an integer ranging from 0 to 65,535. The smaller the value, the higher the priority.

no slb policy persistent url <policy_name>

This command is used to delete the specified Persistent URL policy.

show slb policy persistent url [policy_name]

This command is used to display the specified Persistent URL policy. If the “policy_name” parameter is not specified, all Persistent URL policies will be displayed.

clear slb policy persistent url

This command is used to clear all Persistent URL policies.

slb policy rcookie <policy_name> {virtual_service|vlink_name} <group_name> <precedence>

This command is used to configure a Rewrite Cookie policy for the specified virtual service or vlink. When the client request for accessing this virtual service contains the cookie rewritten by the appliance for the associated real service group, the client request hits this policy.

The Rewrite Cookie policy can associate the virtual service only with the real service group using the rc, ec or persistence method. Multiple Rewrite Cookie policies can be configured for every virtual service or vlink.

policy_name	This parameter specifies the policy name. Its value must be a string of 1 to 128 characters. Its value must be enclosed in double quotes when containing any special character or containing only numbers.
virtual_service vlink_name	This parameter specifies the name of an existing virtual service or vlink.
group_name	This parameter specifies the name of an existing real service group.
precedence	This parameter specifies the priority of the policy comparing to the policies of the same type for the virtual service. Its value must be an integer ranging from 0 to 65,535. The smaller the value, the higher the priority.

no slb policy rcookie <policy_name>

This command is used to delete the specified Rewrite Cookie policy.

show slb policy rcookie [policy_name]

This command is used to display the specified Rewrite Cookie policy. If the “policy_name” parameter is not specified, all Rewrite Cookie policies will be displayed.

clear slb policy rcookie

This command is used to clear all Rewrite Cookie policies.

slb policy icookie <policy_name> {virtual_service|vlink_name} <group_name> <precedence>

This command is used to configure an Insert Cookie policy for the specified virtual service or vlink. When the client request for accessing this virtual service contains the cookie inserted by the appliance for the associated real service group, the client request hits this policy.

The Insert Cookie policy can associate the virtual service only with the real service group using the ic or persistence method. Multiple Insert Cookie policies can be configured for every virtual service or vlink.

policy_name	This parameter specifies the policy name. Its value must be a string of 1 to 128 characters. Its value must be enclosed in double quotes when containing any special character or containing only numbers.
-------------	--

virtual_service vlink_name	This parameter specifies the name of an existing virtual service or vlink.
group_name	This parameter specifies the name of an existing real service group.
precedence	This parameter specifies the priority of the policy comparing to the policies of the same type for the virtual service. Its value must be an integer ranging from 0 to 65,535. The smaller the value, the higher the priority.

no slb policy icookie <policy_name>

This command is used to delete the specified Insert Cookie policy.

show slb policy icookie [policy_name]

This command is used to display the specified Insert Cookie policy. If the “policy_name” parameter is not specified, all Insert Cookie policies will be displayed.

clear slb policy icookie

This command is used to clear all Insert Cookie policies.

slb policy persistent cookie <policy_name> {virtual_service|vlink_name} <group_name> <cookie_name> <precedence>

This command is used to configure a Persistent Cookie policy for the specified virtual service or vlink. When the client request for accessing this virtual service contains the cookie name specified by this policy, the client request hits this policy.

The Persistent Cookie policy can associate the virtual service only with the real service group using the hc, pc or persistence method. Multiple Persistent Cookie policies can be configured for every virtual service or vlink.

policy_name	This parameter specifies the policy name. Its value must be a string of 1 to 128 characters. Its value must be enclosed in double quotes when containing any special character or containing only numbers.
virtual_service vlink_name	This parameter specifies the name of an existing virtual service or vlink.
group_name	This parameter specifies the name of an existing real service group.
cookie_name	This parameter specifies the string used to match with the cookie name in the client request. Its value must be a string of 1 to 512 characters.

precedence	This parameter specifies the priority of the policy comparing to the policies of the same type for the virtual service. Its value must be an integer ranging from 0 to 65,535. The smaller the value, the higher the priority.
------------	--

no slb policy persistent cookie <policy_name>

This command is used to delete the specified Persistent Cookie policy.

show slb policy persistent cookie [policy_name]

This command is used to display the specified Persistent Cookie policy. If the “policy_name” parameter is not specified, all Persistent Cookie policies will be displayed.

clear slb policy persistent cookie

This command is used to clear all Persistent Cookie policies.

slb policy qos cookie <policy_name> {virtual_service|vlink_name} {group_name|vlink_name} <policy_string> <precedence>

This command is used to configure a QoS Cookie policy for the specified virtual service or vlink. When the client request for accessing this virtual service contains the cookie name and value that matches the value of the “policy_string” parameter, the client request hits this policy. Multiple QoS Cookie policies can be configured for every virtual service or vlink.

policy_name	This parameter specifies the policy name. Its value must be a string of 1 to 128 characters. Its value must be enclosed in double quotes when containing any special character or containing only numbers.
-------------	--

virtual_service vlink_name	This parameter specifies the name of an existing virtual service or vlink.
----------------------------	--

group_name vlink_name	This parameter specifies the name of an existing real service group or vlink.
-----------------------	---

policy_string	This parameter specifies the policy string used to match the cookie value and name in the request. Its value must be a case-sensitive string of 1 to 1025 characters enclosed in double quotes. The parameter value supports both exact match and full regular expression match.
---------------	--

When use exact cookie match, users need to set the value in “name=value” format, for example, “server_A=AuthOK_123”.

When the value is a full regular expression, it must begin

with the “<regex>” string. For example,

“<regex>(server_A)=(AuthOK|AuthNG)(_+)” means matching all the cookies with its cookie name being “server_A” and cookie value containing the string “AuthOK_” or “AuthNG_”. For more information, please refer to introductions about regular expressions in Chapter 1.

To ensure that a configured regular expression string is correct, administrators can use the “**slb regextest** <regex> <target_string>” command to test whether a target string will match the configured regular expression.

precedence	This parameter specifies the priority of the policy comparing to the policies of the same type for the virtual service. Its value must be an integer ranging from 0 to 65,535. The smaller the value, the higher the priority.
------------	--

no slb policy qos cookie <policy_name>

This command is used to delete the specified QoS Cookie policy.

show slb policy qos cookie [policy_name]

This command is used to display the specified QoS Cookie policy. If the “policy_name” parameter is not specified, all QoS Cookie policies will be displayed.

clear slb policy qos cookie

This command is used to clear all QoS Cookie policies.

slb policy qos hostname <policy_name> {virtual_service|vlink_name}
{group_name|vlink_name} <host_name> <precedence>

This command is used to configure a QoS Hostname policy for the specified virtual service or vlink. When the client request for accessing this virtual service contains the host name specified by this policy, the client request hits this policy. Multiple QoS Hostname policies can be configured for every virtual service or vlink.

policy_name	This parameter specifies the policy name. Its value must be a string of 1 to 128 characters. Its value must be enclosed in double quotes when containing any special character or containing only numbers.
virtual_service vlink_name	This parameter specifies the name of an existing virtual service or vlink.
group_name vlink_name	This parameter specifies the name of an existing real

service group or vlink.

host_name	This parameter specifies the string used to match the host name in the client request. Its value must be a case-sensitive string of 1 to 512 characters. The parameter value supports full regular expressions. When the parameter value is a full regular expression, it must begin with the “<regex>” string and be enclosed in double quotes, for example, “<regex>/(graphics photos resource)/(.)\.(gif jpg png)” means matching all GIF, JPG, and PNG files in the paths “/graphics”, “/photos”, and “/resource”. For more information, please refer to introductions about regular expressions in Chapter 1. To ensure that a configured regular expression string is correct, administrators can use the “slb regextest <regex> <target_string>” command to test whether a target string will match the configured regular expression.
precedence	This parameter specifies the priority of the policy comparing to the policies of the same type for the virtual service. Its value must be an integer ranging from 0 to 65,535. The smaller the value, the higher the priority.

no slb policy qos hostname <policy_name>

This command is used to delete the specified QoS Hostname policy.

show slb policy qos hostname [policy_name]

This command is used to display the specified QoS Hostname policy. If the “policy_name” parameter is not specified, all QoS Hostname policies will be displayed.

clear slb policy qos hostname

This command is used to clear all QoS Hostname policies.

**slb policy qos url <policy_name> {virtual_service|vlink_name}
{group_name|vlink_name} <url_string> <precedence>**

This command is used to configure a QoS URL policy for the specified virtual service or vlink. When the URL path and query parts in the client request for accessing this virtual service match the URL string specified by this policy, the client request hits this policy. Multiple QoS URL policies can be configured for every virtual service or vlink.

policy_name	This parameter specifies the policy name. Its value must be a string of 1 to 128 characters. Its value must be enclosed
-------------	---

	in double quotes when containing any special character or containing only numbers.
virtual_service vlink_name	This parameter specifies the name of an existing virtual service or vlink.
group_name vlink_name	This parameter specifies the name of an existing real service group or vlink.
url_string	This parameter specifies the URL string used to match the URL path and query parts in the client request. Its value must be a case-sensitive string of 1 to 512 characters. The parameter value supports prefix match and full regular expressions. When the parameter value is a full regular expression, it must begin with the “<regex>” string and be enclosed in double quotes, for example, “<regex>/(graphics photos resource)/(.)\.(gif jpg png)” means matching all GIF, JPG, and PNG files in the paths “/graphics”, “/photos”, and “/resource”. For more information, please refer to introductions about regular expressions in Chapter 1. To ensure that a configured regular expression string is correct, administrators can use the “slb regextest <regex> <target_string>” command to test whether a target string will match the configured regular expression.
precedence	This parameter specifies the priority of the policy comparing to the policies of the same type for the virtual service. Its value must be an integer ranging from 0 to 65,535. The smaller the value, the higher the priority.

For example,

```
AN(config)#slb policy qos url "policy1" "vs1" "g1" "/b.txt" 1
```

After the above configuration is finished, the URLs “/b.txt? abc=123” and “/b.txt1” can both match this policy.

no slb policy qos url <policy_name>

This command is used to delete the specified QoS URL policy.

show slb policy qos url [policy_name]

This command is used to display the specified QoS URL policy. If the “policy_name” parameter is not specified, all QoS URL policies will be displayed.

clear slb policy qos url

This command is used to clear all QoS URL policies.

slb policy qos body <policy_name> {virtual_service|vlink_name}
{group_name|vlink_name} <prefix> <delimiter> <flag> <precedence>

This command is used to configure a QoS Body policy for the specified virtual service or vlink. When the body of the client request for accessing this virtual service matches the value of the “prefix” parameter, the client request hits this policy and the string between “prefix” and “delimiter” will be used as the session ID for the session persistence of the associated real service group. Multiple QoS Body policies can be configured for every virtual service or vlink.

policy_name	This parameter specifies the policy name. Its value must be a string of 1 to 128 characters. Its value must be enclosed in double quotes when containing any special character or containing only numbers.
virtual_service vlink_name	This parameter specifies the name of an existing virtual service or vlink.
group_name vlink_name	This parameter specifies the name of an existing real service group or vlink.
prefix	This parameter specifies the start location of the session ID string in the client request body. Its value must be a case-sensitive string of 1 to 32 characters and must be enclosed in double quotes when starting with a non-letter character. If its value includes a double quote, use “%q” instead.
delimiter	This parameter specifies the delimiter of the session ID string in the client request body. Its value must be a case-sensitive character enclosed in double quotes. If the “prefix” parameter is specified and the “flag” parameter is set to 0, this parameter must be specified.
flag	This parameter specifies whether the value of the “delimiter” parameter needs to exist right before the value of the “prefix” parameter when the client request is matched with the policy. Its value must be: • 0: Needs. • 1: Not need. The default value is 0.
precedence	This parameter specifies the priority of the policy comparing to the policies of the same type for the virtual service. Its value must be an integer ranging from 0 to

65,535. The smaller the value, the higher the priority.

no slb policy qos body <policy_name>

This command is used to delete the specified QoS Body policy.

show slb policy qos body [policy_name]

This command is used to display the specified QoS Body policy. If the “policy_name” parameter is not specified, all QoS Body policies will be displayed.

clear slb policy qos body

This command is used to clear all QoS Body policies.

slb policy regex <policy_name> {virtual_service|vlink_name} {group_name|vlink_name} <url_string> <precedence>

This command is used to configure a Regex (Regular Expression) policy for the specified virtual service or vlink. When the URL path and query parts in the client request for accessing this virtual service match the value of the “url_regex” parameter, the client request hits this policy. Multiple Regex policies can be configured for every virtual service or vlink.

policy_name	This parameter specifies the policy name. Its value must be a string of 1 to 128 characters. Its value must be enclosed in double quotes when containing any special character or containing only numbers.
virtual_service vlink_name	This parameter specifies the name of an existing virtual service or vlink.
group_name vlink_name	This parameter specifies the name of an existing real service group or vlink.
url_string	This parameter specifies the string used to match the URL path and query parts in the client request. Its value must be a case-sensitive string of 1 to 512 characters. The parameter value supports both quick and full regular expressions. When the parameter value is set to a regular expression, it must be enclosed in double quotes. When the parameter value is a full regular expression, it must begin with the “<regex>” string and be enclosed in double quotes, for example, “<regex>/ (graphics photos resource)/(.+)\.(gif jpg png)” means matching all GIF, JPG, and PNG files in the paths “/graphics”, “/photos”, and “/resource”. For information about the differences between the quick regular expression and the full regular expression, please refer to introductions

about regular expressions in Chapter 1.

To ensure that a configured regular expression string is correct, administrators can use the “**slb regextest** *<regex>* *<target_string>*” command to test whether a target string will match the configured regular expression.

precedence	This parameter specifies the priority of the policy comparing to the policies of the same type for the virtual service. Its value must be an integer ranging from 0 to 65,535. The smaller the value, the higher the priority.
------------	--

no slb policy regex *<policy_name>*

This command is used to delete the specified Regex policy.

show slb policy regex [*policy_name*]

This command is used to display the specified Regex policy. If the “policy_name” parameter is not specified, all Regex policies will be displayed.

clear slb policy regex

This command is used to clear all Regex policies.

slb policy header *<policy_name>* {*virtual_service|vlink_name*}
{group_name|vlink_name} *<header_name>* *<header_string>* *<precedence>*

This command is used to configure a Header policy for the specified virtual service or vlink. When the client request for accessing this virtual service contains the specified header and the header value matches the regular expression specified by this policy, the client request hits this policy. Multiple Header policies can be configured for every virtual service or vlink.

policy_name	This parameter specifies the policy name. Its value must be a string of 1 to 128 characters. Its value must be enclosed in double quotes when containing any special character or containing only numbers.
-------------	--

virtual_service vlink_name	This parameter specifies the name of an existing virtual service or vlink.
----------------------------	--

group_name vlink_name	This parameter specifies the name of an existing real service group or vlink.
-----------------------	---

header_name	This parameter specifies the name of the header. Its value must be a case-sensitive string of 1 to 512 characters. ASCII printable characters except the space, double quotes and colon (ASCII code between 33 and 126) are supported. Its value must be:
-------------	---

	- Standard header or pseudo-header name: Accept, Accept-Charset, Accept-Language, Host, Referer, User-Agent and X-Forwarded-For, “:authority” - Extension header name: all extension header names
header_string	This parameter specifies the string used to match the header value. Its value must be a case-sensitive string of 1 to 512 characters. The parameter value supports both quick and full regular expressions. When the parameter value is set to a regular expression, it must be enclosed in double quotes. When the parameter value is a full regular expression, it must begin with the “<regex>” string and be enclosed in double quotes, for example, “<regex>/(graphics photos resource)/(.)\.(gif jpg png)” means matching all GIF, JPG, and PNG files in the paths “/graphics”, “/photos”, and “/resource”. For information about the differences between the quick regular expression and the full regular expression, please refer to introductions about regular expressions in Chapter 1. To ensure that a configured regular expression string is correct, administrators can use the “slb regextest <regex> <target_string>” command to test whether a target string will match the configured regular expression.
precedence	This parameter specifies the priority of the policy comparing to the policies of the same type for the virtual service. Its value must be an integer ranging from 0 to 65,535. The smaller the value, the higher the priority.

no slb policy header <policy_name>

This command is used to delete the specified Header policy.

show slb policy header [policy_name]

This command is used to display the specified Header policy. If the “policy_name” parameter is not specified, all Header policies will be displayed.

clear slb policy header

This command is used to clear all Header policies.

slb policy hashurl <policy_name> {virtual_service|vlink_name} {group_name|vlink_name}

This command is used to configure a Hash URL policy for the specified virtual service or vlink. The Hash URL policy selects the associated real service group based on the hash value of the URL path and query parts in the client request for accessing this virtual service. The client requests with the same hash value will be forwarded to

the same real service group persistently. Multiple Hash URL policies can be configured for every virtual service or vlink.

policy_name	This parameter specifies the policy name. Its value must be a string of 1 to 128 characters. Its value must be enclosed in double quotes when containing any special character or containing only numbers.
virtual_service vlink_name	This parameter specifies the name of an existing virtual service or vlink.
group_name vlink_name	This parameter specifies the name of an existing real service group or vlink.

no slb policy hashurl <policy_name>

This command is used to delete the specified Hash URL policy.

show slb policy hashurl [policy_name]

This command is used to display the specified Hash URL policy. If the “policy_name” parameter is not specified, all Hash URL policies will be displayed.

clear slb policy hashurl

This command is used to clear all Hash URL policies.

slb policy raduname <policy_name> <virtual_service> <group_name>

This command is used to configure a Raduname (RADIUS Username) policy for the specified virtual service.

The Raduname policy can associate the RADIUS authentication or accounting virtual service only with the real service group using the radchu method. Only one Raduname policy can be configured for every RADIUS authentication or accounting virtual service.

policy_name	This parameter specifies the policy name. Its value must be a string of 1 to 128 characters. Its value must be enclosed in double quotes when containing any special character or containing only numbers.
virtual_service	This parameter specifies the name of an existing RADIUS authentication or accounting virtual service.
group_name	This parameter specifies the name of an existing real service group.

no slb policy raduname <policy_name>

This command is used to delete the specified Raduname policy.

show slb policy raduname [*policy_name*]

This command is used to display the specified Raduname policy. If the “policy_name” parameter is not specified, all Raduname policies will be displayed.

clear slb policy raduname

This command is used to clear all Raduname policies.

slb policy radsid <*policy_name*> <*virtual_service*> <*group_name*>

This command is used to configure a Radsid (RADIUS Session ID) policy for the specified virtual service.

The Radsid policy can associate the RADIUS authentication or accounting virtual service only with the real service group using the radchs method. Only one Radsid policy can be configured for every RADIUS authentication or accounting virtual service.

policy_name	This parameter specifies the policy name. Its value must be a string of 1 to 128 characters. Its value must be enclosed in double quotes when containing any special character or containing only numbers.
virtual_service	This parameter specifies the name of an existing RADIUS authentication or accounting virtual service.
group_name	This parameter specifies the name of an existing real service group.

no slb policy radsid <*policy_name*>

This command is used to delete the specified Radsid policy.

show slb policy radsid [*policy_name*]

This command is used to display the specified Radsid policy. If the “policy_name” parameter is not specified, all Radsid policies will be displayed.

clear slb policy radsid

This command is used to clear all Radsid policies.

slb policy filetype <*policy_name*> <*virtual_service*> <*group_name*> <*filetype*>

This command is used to configure a Filetype policy for the specified virtual service. When the extension name of the file requested by the client RTSP request for

accessing this virtual service matches the extension name specified by this policy, the client request hits this policy.

The Filetype policy can associate only the RTSP virtual service with the real service group. Only one Filetype policy can be configured for every RTSP virtual service.

policy_name	This parameter specifies the policy name. Its value must be a string of 1 to 128 characters. Its value must be enclosed in double quotes when containing any special character or containing only numbers.
virtual_service	This parameter specifies the name of an existing RTSP virtual service.
group_name	This parameter specifies the name of an existing real service group.
filetype	This parameter specifies the file extension name. Its value must be a string of 1 to 512 characters.

no slb policy filetype <policy_name>

This command is used to delete the specified Filetype policy.

show slb policy filetype [policy_name]

This command is used to display the specified Filetype policy. If the “policy_name” parameter is not specified, all Filetype policies will be displayed.

clear slb policy filetype

This command is used to clear all Filetype policies.

slb policy qos diameter <policy_name> <virtual_service> <group_name> <service_context_id> <precedence>

This command is used to configure a QoS Diameter policy for the specified Diameter virtual service. In the SLB scenario for Diameter Credit Control (DCC) application, the client Credit-Control-Request (CCR) destined to this virtual service will hit the policy when the CCR matches the unique identifier of the DCC service specified by “service_context_id” parameter. Multiple QoS Diameter policies can be configured for every virtual service.

policy_name	This parameter specifies the policy name. Its value must be a string of 1 to 128 characters, and must be enclosed in double quotes when containing any special character or only containing numbers.
virtual_service	This parameter specifies the name of an existing Diameter virtual service.

group_name	This parameter specifies the name of an existing real service group. The Diameter policy can associate virtual service and real service group only when the real service group contains Diameter real services.
service_context_id	This parameter specifies the unique identifier of the DCC service type. Its value must be a string of 1 to 512 characters, and must be enclosed in double quotes when containing any special character or only containing numbers.
precedence	This parameter specifies the priority of the policy comparing to the policies of the same type for the virtual service. Its value must be an integer ranging from 0 to 65,535. The smaller the value, the higher the priority.

no slb policy qos diameter <policy_name>

This command is used to delete the specified QoS Diameter policy.

show slb policy qos diameter [policy_name]

This command is used to display the specified QoS Diameter policy. If the “policy_name” parameter is not specified, all QoS Diameter policies will be displayed.

clear slb policy qos diameter

This command is used to clear all QoS Diameter policies.

**slb policy qos diameterappid <policy_name> <virtual_service>
<group_name> <application_id> <precedence>**

This command is used to configure a QoS Diameterappid policy. In a DCC application scenario, when the Application-ID in the header of a Diameter request destined for the specified virtual service matches the “application_id” parameter value, the request hits this policy, and the virtual service will forward the request to the specified group. Multiple QoS Diameterappid policies can be configured for every virtual service.

policy_name	This parameter specifies the policy name. Its value must be a string of 1 to 128 characters, and must be enclosed in double quotes when containing any special character or only containing numbers.
virtual_service	This parameter specifies the name of an existing Diameter virtual service.
group_name	This parameter specifies the name of an existing real service group. The Diameter policy can associate virtual

	service and real service group only when the real service group contains Diameter real services.
<code>application_id</code>	This parameter specifies the application ID to be matched in the Diameter request. Its value must be an integer in the range of 0 to 4,294,967,295.
<code>precedence</code>	This parameter specifies the priority of the policy comparing to the policies of the same type for the virtual service. Its value must be an integer ranging from 0 to 65,535. The smaller the value, the higher the priority.

no slb policy qos diameterappid <policy_name>

This command is used to delete the specified QoS Diameterappid policy.

show slb policy qos diameterappid [policy_name]

This command is used to display the specified QoS Diameterappid policy. If the “policy_name” parameter is not specified, all QoS Diameterappid policies will be displayed.

clear slb policy qos diameterappid

This command is used to clear all QoS Diameterappid policies.

**slb policy sipdomain <policy_name> <virtual_service> <group_name>
<domain_name> <call_type>**

This command is used to configure a SIP Domain policy for the specified SIP TCP or SIP UDP virtual service. When a SIP request accessing this virtual service matches the domain name specified by the “domain_name” parameter and the call flow type specified by the “call_type” parameter, the request hits this policy. The system will forward the SIP request to the specified group.

<code>policy_name</code>	This parameter specifies the policy name. Its value must be a string of 1 to 128 characters. Its value must be enclosed in double quotes when containing any special character or containing only numbers.
<code>virtual_service</code>	This parameter specifies the name of an existing SIP TCP or SIP UDP virtual service.
<code>group_name</code>	This parameter specifies the name of an existing real service group.
<code>domain_name</code>	This parameter specifies the domain name used to match the SIP requests. Fuzzy matching is supported. The policy is hit if the domain name in the SIP requests contains the one specified by this parameter.

<code>call_type</code>	This parameter specifies the type of the SIP call flow to be matched. Its value must be: • <code>caller</code>: indicates the outgoing call flow. • <code>callee</code>: indicates the incoming call flow. • <code>other</code>: indicates all other types of call flows
------------------------	--

no slb policy sipdomain <policy_name>

This command is used to delete the specified SIP Domain policy.

show slb policy sipdomain [policy_name]

This command is used to display the specified SIP Domain policy. If the “policy_name” parameter is not specified, all SIP Domain policies will be displayed.

clear slb policy sipdomain

This command is used to clear all SIP Domain policies.

**slb policy sipcall <policy_name> <virtual_service> <group_name>
<domain_name> <precedence> <call_type> [calling_number_pos]
[called_number_pos]**

This command is used to configure a SIP Call policy for the specified SIP TCP or SIP UDP virtual service. The SIP Call policy must be used together with the call number full matching rule or regex matching rule.

When a SIP request accessing this virtual service matches the domain name specified by the “domain_name” parameter and the call type specified by the “call_type” parameter, the virtual service will obtain the calling number and called number from the SIP request based on the configuration of the “calling_number_pos” and “called_number_pos” parameters. Then, the virtual service will check whether the calling number or called number matches the configured call number full matching rule or regex matching rule.

When both the calling number and called number full matching rules and regex matching rules are configured for the SIP Call policy, the virtual service will determine that the SIP request hit this policy if it matches any of the following rules and forward it to the specified group:

- The calling number matches the calling number full matching rule or regex matching rule.
- The called number matches the called number full matching rule or regex matching rule.

For information about the configuration of the call number full matching rule and regex matching rule, please refer to the “**slb sip callnum list import**” and “**slb sip callnum regex**” commands.

policy_name	This parameter specifies the policy name. Its value must be a string of 1 to 128 characters. Numbers, lower-case and upper-case letters, and special characters including “@”, “-”, “_”, “!” are supported. Its value must be enclosed in double quotes when containing any special character or containing only numbers.
virtual_service	This parameter specifies the name of an existing SIP TCP or SIP UDP virtual service.
group_name	This parameter specifies the name of an existing real service group.
domain_name	This parameter specifies the domain name used to match SIP requests. Fuzzy matching is supported. The policy is hit if the domain name in the SIP requests contains the one specified by this parameter.
precedence	This parameter specifies the priority of the policy comparing to the policies of the same type for the virtual service. Its value must be an integer ranging from 0 to 65,535. The smaller the value, the higher the priority.
call_type	This parameter specifies the SIP call flow type. Its value must be: • caller: indicates the outgoing call flow. • callee: indicates the incoming call flow.
calling_number_pos	This parameter specifies the position from which to obtain the calling number first. Its value must be: • from: indicates that the calling number will be obtained from the “From” header first. • pai: indicates that the calling number will be obtained from the “P-Asserted-Identity” header first. The default value is “from”.
called_number_pos	Optional. This parameter specifies the position from which to obtain the called number first. Its value must be: • to: indicates that the called number will be obtained from the “To” header first. • requestline: indicates that the called number will be obtained from the request line first. The default value is “to”.



Note: The maximum length of a call number supported by this policy is 24 digits. The call numbers that are longer than 24 digits will be treated as vacant numbers.

no slb policy sipcall *<policy_name>*

This command is used to delete the specified SIP Call policy.

show slb policy sipcall [*policy_name*]

This command is used to display the specified SIP Call policy. If the “policy_name” parameter is not specified, all SIP Call policies will be displayed.

clear slb policy sipcall

This command is used to clear all SIP Call policies.

slb sip callnum regex *<policy_name>* *<call_type>* *<regex>*

This command is used to configure a call number regex matching rule for the specified SIP Call policy.

policy_name	This parameter specifies the name of an existing SIP call policy.
call_type	This parameter specifies the SIP call party. Its value must be: • caller: indicates that the regex is used to match calling numbers. • callee: indicates that the regex is used to match called numbers.
regex	This parameter specifies the full regular expression used to match call numbers, which must be enclosed in double quotes. For more information about full regular expression, please refer to Chapter 1.

no slb sip callnum regex *<policy_name>*

This command is used to delete the call number regex matching rule configured for the specified SIP Call policy.

show slb sip callnum regex *<policy_name>* *<call_type>*

This command is used to display the call number regex matching rule configuration for the specified SIP Call policy.

policy_name	This parameter specifies the name of an existing SIP Call policy.
call_type	This parameter specifies the SIP call party. Its value must be: • caller: displays the calling number regex matching rule. • callee: displays the called number regex matching rule.

clear slb sip callnum regex

This command is used to clear all call number regex matching rules configured for SIP Call policies.

slb sip callnum list import <import_type> <policy_name> <call_type> <url>

This command is used to import a call number list for the specified SIP Call policy. XLSX EXCEL and TXT file formats are supported for the call number list.

The number of call numbers allowed to be imported depends on the system memory. Approximately 125000 call numbers are supported on a 1 GB memory appliance. The larger the system memory, the more call numbers are supported.

import_type	This parameter specifies the type of the call number list to be imported. Its value must be: • total: indicates that the existing call number list will be replaced with the one to be imported. • diff: indicates that the existing call number list will be modified by the one to be imported.
policy_name	This parameter specifies the name of an existing SIP Call policy.
call_type	This parameter specifies the SIP call party. Its value must be: • caller: imports the calling number list • callee: imports the called number list.
URL	This parameter specifies the URL address from which to import the call number file. It can be an FTP or HTTP URL. Note: The “path” of the FTP URL (ftp://user:password@host:port/path) should be a relative path.

slb sip callnum list export <policy_name> <call_type> <url>

This command is used to export a call number list of the specified SIP Call policy to the specified URL address.

policy_name	This parameter specifies the name of an existing SIP Call policy.
call_type	This parameter specifies the SIP call party. Its value must be: • caller: exports the calling number list • callee: exports the called number list.
URL	This parameter specifies the URL address to which the call number list will be exported. It must be an FTP URL. Note: The “path” of the FTP URL (ftp://user:password@host:port/path) should be a relative path.

show slb sip callnum list <policy_name> <call_type> [key]

This command is used to display which phone numbers in the caller number list of the specified SIP Call policy match the specified filter string.

policy_name	This parameter specifies the name of an existing SIP Call policy.
call_type	This parameter specifies the SIP call type. Its value must be: • caller: displays the calling number list. • callee: displays the called number list.
key	Optional. This parameter specifies the full regular expression used to filter the call number list, which must be enclosed in double quotes. For more information about full regular expression, please refer to Chapter 1.

clear slb sip callnum list <policy_name> <call_type>

This command is used to delete a call number list of the specified SIP Call policy.

policy_name	This parameter specifies the name of an existing SIP Call policy.
-------------	---

call_type This parameter specifies the SIP call type. Its value must be:

- caller: deletes the calling number list.
- callee: deletes the called number list.

slb group option sipdomain <group_name> <domain_name>

This command is used to set the domain name which will replace the one in the request hitting the specified group.

group_name This parameter specifies the name of an existing real service group.

domain_name This parameter specifies the domain name used to replace the one in the SIP request. Its value must be a string of 1 to 511 characters. Letters, numbers, dots and hyphens are supported, but the value cannot start or end with a dot or hyphen.

no slb group option sipdomain <group_name>

This command is used to delete the configuration of the “**slb group option sipdomain**” command for the specified group.

show slb group option sipdomain [group_name]

This command is used to display the configuration of the “**slb group option sipdomain**” command for the specified group. If the “group_name” parameter is not specified, the configurations of the “**slb group option sipdomain**” command for all groups will be displayed.

slb sip persistence timeout <timeout>

This command is used to set the timeout value of the session persistence based on SIP Call ID. When this command is not configured, the default timeout value is 120s.

timeout This parameter specifies the timeout value for the session persistence based on SIP Call ID. Its value must be an integer in the range of 10 to 3600, in seconds.

show slb sip persistence timeout

This command is used to display the timeout setting for the session persistence based on SIP Call ID.

slb policy default {virtual_service|vlink_name} {group_name|vlink_name}

This command is used to configure a Default policy for the specified virtual service or vlink. When the client request for accessing this virtual service does not match any policy in a higher priority order, the client request hits the Default policy. Only one Default policy can be configured for every virtual service or vlink.

`virtual_service|vlink_name` This parameter specifies the name of an existing virtual service or vlink.

`group_name|vlink_name` This parameter specifies the name of an existing real service group or vlink.



Note: The Default policy cannot associate the virtual service or vlink with the real service group using the “pu” or “pc” method.

no slb policy default {virtual_service|vlink_name}

This command is used to delete the Default policy of the specified virtual service or vlink.

show slb policy default [virtual_service|vlink_name]

This command is used to display the Default policy of the specified virtual service or vlink. If the “virtual_service|vlink_name” parameter is not specified, Default policies of all virtual services or vlinks will be displayed.

clear slb policy default

This command is used to clear all Default policies.

slb policy backup {virtual_service|vlink_name} {group_name|vlink_name}

This command is used to configure a Backup policy for the specified virtual service or vlink. When the client request for accessing this virtual service matches at least one policy in a higher priority order but the associated real service groups of all the matched policies have no available real service, the client request hits the Backup policy. Only one Backup policy can be configured for every virtual service or vlink.

`virtual_service|vlink_name` This parameter specifies the name of an existing virtual service or vlink.

`group_name|vlink_name` This parameter specifies the name of an existing real service group or vlink.

no slb policy backup {virtual_service|vlink_name}

This command is used to delete the Backup policy of the specified virtual service or vlink.

show slb policy backup [virtual_service|vlink_name]

This command is used to display the Backup policy of the specified virtual service or vlink. If the “virtual_service|vlink_name” parameter is not specified, Backup policies of all virtual services or vlinks will be displayed.

clear slb policy backup

This command is used to clear all Backup policies.

show slb policy all

This command is used to display all configured policies.

show slb policy group <group_name>

This command is used to display all policies associated with the specified real service group.

group_name	This parameter specifies the name of an existing real service group.
------------	--

clear slb policy group <group_name>

This command is used to delete all policies associated with the specified real service group.

Policy Nesting

SLB supports three-layer policy nesting at most. In three-layer policy nesting scenario, two vlinks need to be created. In the top-layer policy, vlink1 replaces the real service group; in the second-layer policy, vlink1 replaces the virtual service and vlink2 replaces the real service group; in the bottom-layer policy, vlink2 replaces the virtual service. When receiving the client request for accessing the virtual service of the top-layer policy, the system forwards the client request to vlink1, then forwards the client request to vlink2 according to the second-layer policy and at last forwards the client request to the real service group associated with vlink2 in the bottom-layer policy.

slb vlink <vlink_name>

This command is used to create a vlink.

vlink_name	This parameter specifies the vlink name. Its value must be a case-sensitive string of 1 to 128 characters. Numbers, letters, and special characters are supported. When the parameter value contains special characters, it must be enclosed in double quotes. The specified virtual service name cannot be the system reserved word “default”, “all” or “global”, regardless of its case sensitivity.
------------	--

**Note:**

- In the preceding CLIs used for configuring policies, the “virtual_service|vlink_name” parameter indicates that the virtual service can be replaced by the vlink, and the “group_name|vlink_name” parameter indicates that the real service group can be replaced by the vlink.
- The DoH policy does not support vlink.

no slb vlink <vlink_name>

This command is used to delete the specified vlink.

show slb vlink [vlink_name]

This command is used to display the specified vlink. If the “vlink_name” parameter is not specified, all vlinks will be displayed.

clear slb vlink

This command is used to clear all vlinks.

Policy Order Template

slb policy order <order_template_name> <policy_type> <precedence>

This command is used to create a custom policy order template and set the priority order for the specified type of policy. A maximum of 100 custom policy order templates can be created.

When the “order_template_name” parameter is set to an existing value, this command is also used to modify the priority order for the specified type of policy.

order_template_name	This parameter specifies the name of the custom policy order template. Its value must be a string of 1 to 64 characters.
policy_type	This parameter specifies the policy type. Its value must be the policy types of the default policy order template displayed in the output of the “show slb policy order” command, such as header, ic, and qos-cookie.
precedence	This parameter specifies the priority order of the specified policy type in the template. Its value must be an integer ranging from 1 to 15. If the priority order of a policy type is moved forward in the template, the policy types whose orders are higher than the original order and not lower than the new order of this policy type will be moved backward by one location; if the priority order of a policy type is moved backward in the template, the policy types whose orders are lower than the

original order and not higher than the new order of this policy type will be moved forward by one location.

no slb policy order <order_template_name>

This command is used to delete the specified custom policy order template.

show slb policy order [order_template_name] [policy_type]

This command is used to display the priority order of the specified policy type in the custom policy order template. If the “order_template_name” parameter is specified but the “policy_type” parameter is not specified, the content of the custom policy order template will be displayed. If the “order_template_name” parameter is not specified, the contents of the default policy order template and all custom policy order templates are displayed.

clear slb policy order

This command is used to clear all custom policy order templates.

slb virtual order <virtual_service> <order_template_name>

This command is used to associate the custom policy order template with the specified virtual service. When receiving the client request for accessing this virtual service, the system will try to match the client request with the policies of this virtual service according to the priority orders defined in the custom policy order template. Only one custom policy order template can be associated with every virtual service. If a custom policy order template has been associated with the virtual service, this command is used to modify the policy order template setting of the virtual service. If no custom policy order template is associated with one virtual service, the virtual service will use the default policy order template.

virtual_service	This parameter specifies the virtual service name.
order_template_name	This parameter specifies the name of the custom policy order template. Its value must be the name of the custom policy order template configured using the “ slb policy order ” command.

no slb virtual order <virtual_service> <order_template_name>

This command is used to delete the association between the specified custom policy order template and virtual service.

show slb virtual order [order_template_name]

This command is used to display the virtual services with which the specified custom policy order template is associated. If the “order_template_name” parameter is not specified, the associations between all custom policy order templates and virtual services will be displayed.

clear slb virtual order *[order_template_name]*

This command is used to clear the associations between the specified custom policy order template and all virtual services. If the “order_template_name” parameter is not specified, the associations between all custom policy order templates and virtual services will be cleared.

Other Configurations**slb policy action block** *<policy_name> <response_code>*

This command is used to configure an error code response rule for the specified SLB policy. When a client request hits the specified policy, the system will return the specified HTTP error code.

This command can work together with the “**http import error**” command. When both of the two commands are configured for the same HTTP error code, the customized HTTP error page will be returned.

Each SLB policy can be configured with only one error code response rule or one URL redirect rule.

policy_name	This parameter specifies the name of an existing policy.
response_code	This parameter specifies the HTTP error code. Its value must be an integer ranging from 400 to 599.

slb policy action redirect *<policy_name> <redirect_url>*

This command is used to configure a URL redirect rule for the specified SLB policy. When a client request hits the specified policy, the system will redirect it to the specified URL address.

Each SLB policy can be configured with only one error code response rule or one URL redirect rule.

policy_name	This parameter specifies the name of an existing policy.
redirect_url	This parameter specifies the URL address to which the client requests will be redirected. HTTP and HTTPS URLs are supported.

no slb policy action *<policy_name>*

This command is used to delete the error code response rule or URL redirect rule configuration for the specified policy.

show slb policy action *[policy_name]*

This command is used to display the error code response rule or URL redirect rule configuration for the specified policy. If the “policy_name” parameter is not specified, the error code response rule and/or URL redirect rule configurations for all policies will be displayed.

clear slb policy action

This command is used to clear the error code response rule and/or URL redirect rule configurations for all policies.

Health Check

SLB provides two categories of health checks for the real service:

- **Active health checks:** the system proactively sends health check requests to the real service or its dependent servers to determine the health status of the real service. Active health checks can be further classified into basic health checks, additional health checks, and group member health checks according to their configuration modes.
- **Passive health checks:** the system monitors the traffic anomalies of the real service to determine the health status of the real service.

The health status of a real service will be determined by all health checks associated with the real service, including the basic health checks, additional health checks, the group member health checks, and passive health checks. If the health check relationship of a real service is “AND”, its health status is “UP” only when the results of all kinds of health checks are “UP”. If the health relation of a real service is “OR”, its health status is “UP” as long as the result of any health check is “UP”.

The default health check relationship is “AND” for a real service. Administrators can change the health check relationship for a real service by using the “**health relation**” command. When a real service is added to a real service group with the member’s default health check relationship set, the real service’s health check relationship will be updated to the member’s default health check relationship.

Additional Health Check

slb real health <add_hc_name> <real_service> <ip> <port> [hc_type] [hc_up] [hc_down]

This command is used to configure an additional health check for the specified real service. A maximum of 4000 additional health checks can be configured for all real services.

add_hc_name

This parameter specifies the name of the real service. Its value must be a case-sensitive string of 1 to 128 characters and must be enclosed in double quotes when containing special characters. The parameter value cannot contain spaces or be the system reserved word “default”, “all”, or

“global”, which is case-insensitive.

Note: The parameter value cannot be the same as that of the real service.

real_service	This parameter specifies the name of the real service.
ip	This parameter specifies the destination IP address of the additional health check. Its value must be an IPv4 or IPv6 address of the real service or other services supporting this real service.
port	This parameter specifies the destination port of the additional health check. Its value must be an integer ranging from 0 to 65,535. When the parameter value is 0, the type of this additional health check can only be ICMP or the real service specified by the “real_service” parameter is used for Layer 2 SLB only.
hc_type	Optional. This parameter specifies the type of the additional health check. Its value must be “http”, “https”, “http2”, “https2”, “tcp”, “ftp”, “ntp”, “icmp”, “udp”, “dns”, “dns-tcp”, “pop3”, “smtp”, “script-tcp”, “diameter-tcp”, “script-udp”, “script-tcps”, “radius-auth”, “radius-acct”, “sip-tcp”, “sip-udp”, “rtsp-tcp”, “ldap”, “mysql-db”, “mysql-dbs”, “mssql-db”, “mssql-dbs”, “oracle-db” and “snmp”. The default value is “tcp”. Note: • The parameter value can be “ldap” only when the type of the real service is “TCP”. For real services of other types, “ldap” cannot take effect. • If the “ip” parameter is set to an IPv6 address, the health check type cannot be “tcps”, “radius-auth”, “radius-acct”, “sip-tcp”, “sip-udp” or “rtsp-tcp”.
hc_up	Optional. This parameter specifies the number of consecutive health check successes for marking the real service as “Up”. Its value must be an integer ranging from 1 to 255. The default value is 3.
hc_down	Optional. This parameter specifies the number of consecutive health check successes for marking the real service as “Down”. Its value must be an integer ranging from 1 to 255. The default value is 3.

For health check types supported by the real service of each type, please refer to the following table.

Real Service Type	Supported Health Check Type
HTTP	Icmp, tcp, script-tcp, http, http2
HTTPS	icmp, tcp, tcps, script-tcp, script-tcps, https, https2
FTP	icmp, tcp, ftp, script-tcp
DNS	icmp, udp, script-udp, dns
SIP-TCP	icmp, tcp, script-tcp, sip-tcp
SIP-UDP	icmp, udp, script-udp, sip-udp
RTSP	icmp, tcp, script-tcp, rtsp-tcp
RDP	icmp, tcp, script-tcp
Radauth	icmp, udp, script-udp, radius-auth
Radacct	icmp, udp, script-udp, radius-acct
TCP	icmp, tcp, tcps, script-tcp, script-tcps, http, https, rtsp-tcp, sip-tcp, dns-tcp, ldap, mysql-db, mysql-dbs, mssql-db, mssql-dbs, oracle-db pop3, smtp
TCPS	icmp, tcp, tcps, script-tcp, script-tcps, https, mysql-dbs, mssql-dbs
UDP	icmp, udp, script-udp, radius-auth, radius-acct, dns, ntp
IP	icmp
Diameter	icmp, tcp, script-tcp, diameter-tcp
DNSTCP	icmp, tcp, dns-tcp, script-tcp

no slb real health <add_hc_name>

This command is used to delete the specified additional health check.

show slb real health [real_service]

This command is used to display all additional health checks configured for the specified real service. If the “real_service” parameter is not specified, the additional health checks configured for all real services will be displayed.

clear slb real health [real_service]

This command is used to clear all additional health checks configured for the specified real service. If the “real_service” parameter is not specified, the additional health checks configured for all real services will be cleared.

Group Health Check

slb health <group_hc_name> [group_hc_type] [interval] [timeout] [hc_up] [hc_down] [http_method|reset_handshake] [url_path] [expected_codes]

This command is used to define or update a group health check. A maximum of 4000 group health checks can be configured in the system.

group_hc_name This parameter specifies the name of the group health check. Its value must be a case-sensitive string of 1 to 128 characters and must be enclosed in double quotes when containing special characters.

group_hc_type	Optional. This parameter specifies the type of the group health check. Its value must be “http”, “https”, “http2”, “https2”, “tcp”, “tcps”, “ftp”, “ntp”, “icmp”, “udp”, “dns”, “dns-tcp”, “pop3”, “smtp”, “script-tcp”, “diameter-tcp”, “script-udp”, “script-tcps”, “radius-auth”, “radius-acct”, “sip-tcp”, “sip-udp”, “rtsp-tcp”, “ldap”, “mysql-db”, “mysql-dbs”, “mssql-db”, “mssql-dbs”, “oracle-db” and “snmp”. The default value is “icmp”.
interval	Optional. This parameter specifies the interval between two health checks, in seconds. Its value must be an integer ranging from 1 to 10,000. The default value is 30.
timeout	Optional. This parameter specifies the allowed maximum time interval between the health check request and its corresponding health check response, in seconds. Its value must be an integer ranging from 1 to 10,000. The default value is 3. Note: The timeout value cannot be larger than the interval value.
hc_up	Optional. This parameter specifies the number of consecutive health check successes for marking the real service as “Up”. Its value must be an integer ranging from 1 to 255. The default value is 3.
hc_down	Optional. This parameter specifies the number of consecutive health check successes for marking the real service as “Down”. Its value must be an integer ranging from 1 to 255. The default value is 3.
http_method reset_handshake	Optional. The “http_method” parameter specifies the HTTP method of the HTTP health check request. This parameter is available only when the “type” parameter is set to “http” or “https”. Its value must be “GET” or “HEAD”. The default value is “GET”. The “reset_handshake” parameter specifies whether the TCP handshake reset mode is enabled for the TCP health check. If the TCP handshake reset mode is enabled, the appliance will send the RESET packet directly to close the TCP connection after receiving the SYN ACK packet from the real service during the TCP health check. This parameter is available only when the “type” parameter is set to “tcp”. Its value must be: • enable: The TCP handshake reset mode is enabled.

- **disable:** The TCP handshake reset mode is disabled. The default value is “disable”.

<code>url_path</code>	Optional. This parameter specifies the URL address in the HTTP health check request. This parameter is available only when the “type” parameter is set to “http” or “https”. Its value must be a string of 1 to 128 characters, starting with “/”. The default value is “/”.
<code>expected_codes</code>	Optional. This parameter specifies the HTTP status code(s) expected in the HTTP health check response. This parameter is available only when the “type” parameter is set to “http” or “https”. This parameter supports the HTTP status codes ranging from 100 to 599. Its value can contain one or more HTTP status codes and must be enclosed in double quotes. Multiple HTTP status codes must be separated by comma or “-”. The default value is 200. Note: The system will regard one HTTP health check as successful only when the status code in the HTTP health check response matches the value of the “expected_codes” parameter.

no slb health <group_hc_name>

This command is used to delete the specified group health check.

show slb health [group_hc_name]

This command is used to display the specified group health check. If the “group_hc_name” parameter is not specified, all group health checks will be displayed.

clear slb health

This command is used to clear all group health checks.

slb group health <group_name> <group_hc_name>

This command is used to associate the group health check with the specified real service group. When a group health check is associated with the specified real service group, the health status of every real service in this group will be checked. A real service group can be associated with a maximum of five group health checks.

<code>group_name</code>	This parameter specifies the name of the real service group.
<code>group_hc_name</code>	This parameter specifies the name of the group health check.

no slb group health <group_name> <group_hc_name>

This command is used to delete the association of the group health check with the specified real service group.

show slb group health [group_name]

This command is used to display the association of the group health checks with the specified real service group. If the “group_name” parameter is not specified, all associations of group health checks with real service groups will be displayed.

clear slb group health [group_name]

This command is used to clear all associations of group health checks with the specified real service group. If the parameter “group_name” is not specified, all associations of group health checks with real service groups will be cleared.

slb group threshold <threshold_granularity>

This command is used to configure a rule for determining the group health check result as being Up. When a real service group has multiple health checks, the final result will be set to Up only when the number of “Up” health checks is equal to or larger than the specified threshold.

threshold_granularity	This parameter specifies the threshold for the number of “Up” group health checks. Its value must be an integer ranging from 1 to 5.
-----------------------	--

no slb group threshold

This command is used to delete the configuration of the “slb group threshold” command.

show slb group threshold

This command is used to display the configuration of the “slb group threshold” command.

show health group status <group_name>

This command is used to display the information and the health check status of all real services in the specified real service group. Displayed information includes the name, IP address, port number, and health check result. The reason is displayed if the health check result is Down.

Additional Configurations for Specific Types of Health Checks

➤ LDAP Health Check

health ldap {real_service|add_hc_name|group_hc_name} [bind_dn]
[password] [search_dn] [filter_keyword]

This command is used to set the parameters required by the specified LDAP type real service basic or additional health check or group health check. A maximum of 80 commands can be configured in the system.

For existing “**health ldap**” configurations, this command can also be used to change their parameter configurations.

`real_service|add_hc_name|group_hc_name` This parameter specifies the name of the real service or real service additional health check or group health check.

- `real_service`: indicates that the parameters are set for the LDAP type real service basic health check.
- `add_hc_name`: indicates that the parameters are set for the LDAP type real service additional health check.
- `group_hc_name`: indicates that the parameters are set for the LDAP type group health check.

Note:

- The type of the real service can only be “tcp”. For real services of other types, this command will not work.
- The type of the real service basic or additional health check or group health check can only be “ldap”. For other types of real service basic and additional health checks and group health checks, this command will not work.

`bind_dn` Optional. This parameter specifies the unique Distinguished Name (DN) to perform binding operation. Its value must be a string of 1 to 255 characters. The default value is empty.

`password` Optional. This parameter specifies the password to log into the LDAP server. Its value must be a string of 1 to 255 characters. The default value is empty.

`search_dn` Optional. This parameter specifies the DN to perform the search operation. Its value must be a string of 1 to 255 characters. The default value is empty.

`filter_keyword` Optional. This parameter specifies the filter keyword to search the DN. Its value must be a string of 1 to 255 characters. With the filter keyword configured, the LDAP real service will return the result set matching the filter. If the parameter value is empty, all the results matching the “`search_dn`” parameter will be returned. It is recommended to specify the “`search_dn`” parameter more accurately to reduce related network traffic. The default

value is empty.

no health ldap {real_service|add_hc_name|group_hc_name}

This command is used to delete the parameter settings of the specified LDAP type real service basic or additional health check or group health check.

show health ldap [real_service|add_hc_name|group_hc_name]

This command is used to display the parameter settings of the specified LDAP type real service basic or additional health check or group health check. If the “real_service|add_hc_name|group_hc_name” parameter is not specified, the parameter settings of all LDAP type real service basic and additional health checks and group health checks will be displayed.

clear health ldap

This command is used to clear the parameter settings of all LDAP type real service basic and additional health checks and group health checks.

➤ RADIUS Health Check

health radius auth {real_service|add_hc_name|group_hc_name} <secret_string> <username> <password> [response_code] [attribute_list]

This command is used to set the parameters required by the specified RADIUS authentication type real service basic or additional health check or group health check. With this command configured, a series of handshakes using the RADIUS protocol will be performed in the system. If the RADIUS real service returns the expected authentication response, then the RADIUS authentication of this real service works well; otherwise, it does not work. A maximum of 80 “**health radius auth**” and “**health radius acct**” commands can be configured in the system.

For existing “**health radius auth**” configurations, this command can also be used to change their parameter configurations.

real_service|add_hc_name|group_hc_name This parameter specifies the name of the real service or real service additional health check or group health check.

- real_service: indicates that the parameters are set for the RADIUS authentication type real service basic health check.
- add_hc_name: indicates that the parameters are set for the RADIUS authentication type real service additional health check.
- group_hc_name: indicates that the parameters are set for the RADIUS authentication type group health check.

Note: The type of the real service basic or additional health

	check or the type of the group health check can be only “radius-auth”. For other types of real service basic and additional health checks and group health checks, this command will not work.
secret_string	This parameter specifies the secret key shared between the RADIUS client and server. This secret key is used to encrypt the password to log into the RADIUS server and should be obtained from the RADIUS server beforehand. Its value must be a string of 1 to 127 characters enclosed in double quotes.
username	This parameter specifies the username to log into the RADIUS server. Its value must be a string of 1 to 31 characters enclosed in double quotes.
password	This parameter specifies the password to log into the RADIUS server. Its value must be a string of 1 to 31 characters enclosed in double quotes.
response_code	Optional. This parameter specifies the expected response code returned by the RADIUS server, which can be used to determine the health status of the RADIUS server. Its value must be: • 2: indicates that the RADIUS server accepts the authentication request. When this parameter value is 2, and the username and password provided are both correct, if the response code returned by the RADIUS server is 2, then the RADIUS server is marked as “Up”; otherwise, it is marked as “Down”. • 3: indicates that the RADIUS server rejects the authentication request. When this parameter value is 3 and the password provided is wrong, if the response code returned by RADIUS server is 3, then the RADIUS server is marked as “Up”; otherwise, it is marked as “Down”. The default value is 2.
attribute_list	Optional. This parameter specifies the attribute list of the RADIUS authentication. The supported attributes includes “User-Name”, “User-Password”, “NAS-IP-Address” and “NAS-Port”. Its value can include three attributes at most, in the format of “attribute-name1=attribute-value1,attribute-name2=attribute-value2”. Each attribute is a string of 1 to 31 characters and multiple attributes should be separated by comma (.). Spaces are not allowed between attributes. For example:

“NAS-IP-Address=192.168.1.2,NAS-Port=2012”.

no health radius auth {real_service|add_hc_name|group_hc_name}

This command is used to delete the parameter settings of the specified RADIUS authentication type real service basic or additional health check or group health check.

show health radius auth [real_service|add_hc_name|group_hc_name]

This command is used to display the parameter settings of the specified RADIUS authentication type real service basic or additional health check or group health check. If the “real_service|add_hc_name|group_hc_name” parameter is not specified, the parameter settings of all RADIUS authentication type real service basic and additional health checks and group health checks will be displayed.

clear health radius auth

This command is used to clear the parameter settings of all RADIUS authentication type real service basic and additional health checks and group health checks.

health radius acct {real_service|add_hc_name|group_hc_name} <secret_string> [response_code]

This command is used to set the parameters required by the specified RADIUS accounting type real service basic or additional health check or group health check.

For existing “**health radius acct**” configurations, this command can also be used to change their parameter configurations.

real_service|add_hc_name|group_hc_name This parameter specifies the name of the real service or real service additional health check or group health check.

- real_service: indicates that the parameters are set for the RADIUS accounting type real service basic health check.
- add_hc_name: indicates that the parameters are set for the RADIUS accounting type real service additional health check.
- group_hc_name: indicates that the parameters are set for the RADIUS accounting type group health check.

Note: The type of the real service basic or additional health check or the type of the group health check can be only “radius-acct”. For other types of real service basic and additional health checks and group health checks, this command will not work.

secret_string This parameter specifies the secret key shared between the RADIUS client and server. This secret key is used to encrypt the password to log into the RADIUS server.

response_code Optional. This parameter specifies the expected response code returned by the RADIUS server, which can be used to determine the health status of the RADIUS server. If the parameter value is “5”, it means the RADIUS server starts the response accounting request. If the response code returned by RADIUS server is 5, and then the RADIUS server is marked as “Up”; otherwise, it is marked as “Down”. The default value is 5.

no health radius acct {real_service|add_hc_name|group_hc_name}

This command is used to delete the parameter settings of the specified RADIUS accounting type real service basic or additional health check or group health check.

show health radius acct [real_service|add_hc_name|group_hc_name]

This command is used to display the parameter settings of the specified RADIUS accounting type real service basic or additional health check or group health check. If the “real_service|add_hc_name|group_hc_name” parameter is not specified, the parameter settings of all RADIUS accounting type real service basic and additional health checks and group health checks will be displayed.

clear health radius acct

This command is used to clear the parameter settings of all RADIUS accounting type real service basic and additional health checks and group health checks.

clear health radius all

This command is used to clear the parameter settings of all RADIUS authentication and accounting type real service basic and additional health checks and group health checks.

➤ Diameter Health Check

health diameter required [origin_host] [origin_realm] [host_ip] [vendor_id] [product_name]

Sets the AVP (Attribute-Value Pair) required by the basic or additional health check of a Diameter real service or group health check globally.

For the existing **health diameter required** configuration, this command can also be used to modify its configured parameter.

If this command is not configured, by default, the system will set a predefined AVP ("array.hc.diameter.org" "hc.diameter.org" 11.1.111.11 11 "HC Interface") required by the basic or additional health check of a Diameter real service or group health check for the global.

origin_host	Optional. This parameter specifies the Origin-Host AVP value, which identifies the host that initiated the Diameter message. The value must be a string with a length not greater than 128 bytes. The default value is “array.hc.diameter.org.”
origin_realm	Optional. This parameter specifies the Origin-Realm AVP value, which identifies the domain of the Diameter message sender. The value must be a string with a length not greater than 128 bytes. The default value is “hc.diameter.org.”
host_ip	Optional. This parameter specifies the Host-IP-Address AVP value, which identifies the IP address of the Diameter message sender. The default value is “11.1.111.11.”
vendor_id	Optional. This parameter specifies the Vendor-Id AVP value, which identifies the vendor of the Diameter device. The value must be an integer ranging from 0 to 4,294,967,295. The default value is 11.
product_name	Optional. This parameter specifies the Product-Name AVP value, which identifies the name assigned by the vendor for the Diameter device. The value must be a string with a length not greater than 128 bytes. The default value is “HC Interface.”

no health diameter required

Resets the AVP configuration required by the basic or additional health check of a Diameter real service or group health check globally.

health diameter optional authapp <auth_app_id>

Sets the optional **AVP Auth-Application-Id** configuration for the basic or additional health check of a Diameter real service or group health check globally. If this command is not configured, the Diameter health check packets sent from APV won't contain this parameter.

auth_app_id	This parameter specifies the Auth-Application-Id AVP value, which is used to broadcast APV's support for authentication and authorization options. The value must be an integer ranging from 0 to 4,294,967,295.
-------------	---

no health diameter optional authapp

Deletes the optional **AVP Auth-Application-Id** configuration for the basic or additional health check of a Diameter real service or group health check globally.

health diameter optional acctapp <acct_app_id>

Sets the optional **AVP Acct-Application-Id** configuration for the basic or additional health check of a Diameter real service or group health check globally. If this command is not configured, the Diameter health check packets sent from APV won't contain this parameter.

acct_app_id	This command is used to specify the Acct-Application-Id AVP value, which is used to broadcast APV's support for accounting options. The value must be an integer ranging from 0 to 4,294,967,295.
-------------	--

no health diameter optional acctapp

Deletes the optional **AVP Acct-Application-Id** configuration for the basic or additional health check of a Diameter real service or group health check globally.

health diameter optional vendorspec <vendor_id> <auth_app_id> <acct_app_id>

Sets the optional **AVP Vendor-Specific-Application-Id** configuration for the basic or additional health check of a Diameter real service or group health check globally, which is used to broadcast APV's support for the specific Diameter devices of vendor. If this command is not configured, the Diameter health check packets sent from APV won't contain this parameter.

vendor_id	This parameter specifies the Vendor-Id AVP value, which identifies the vendor of the Diameter device. The value must be an integer ranging from 0 to 4,294,967,295.
-----------	--

auth_app_id	This parameter specifies the Auth-Application-Id AVP value, which is used to broadcast the support of the vendor's specific Diameter device for authentication and authorization options. The value must be an integer ranging from 0 to 4,294,967,295.
-------------	--

acct_app_id	This parameter specifies the Acct-Application-Id AVP value, which is used to broadcast the support of the vendor's specific APV for accounting options. The value must be an integer ranging from 0 to 4,294,967,295.
-------------	--

no health diameter optional vendorspec

Deletes the optional **AVP Vendor-Specific-Application-Id** configuration for the basic or additional health check of a Diameter real service or group health check globally.

health diameter dpr on

Enables the DPR (Disconnect-Peer-Request) extension option of the Diameter health check globally. After enabled, APV will send a DPR packet after it receives a correct

CEA (Capabilities-Exchange-Answer) packet. If the real service returns a correct DPA (Disconnect-Peer-Answer) packet, the result of the health check will be Up, otherwise, Down. By default, this feature is disabled, which means APV determines the result of the health check based on whether it receives a correct CEA packet without sending a DPR packet.

health diameter dpr off

Disables the DPR extension option of the Diameter health check.

show health diameter config

Displays all the additional configurations for the basic or additional health check of a Diameter real service or group health check.

clear health diameter config

Deletes all the additional configurations for the basic or additional health check of a Diameter real service or group health check, and restores the required configurations of the AVP and DPR extension options to their default settings.

➤ Database Health Check

health dbserver {*real_service*|*add_hc_name*|*group_hc_name*}
<database_name> *<username>* *<password>* [*sql_statement*]
[expected_response] [*row*] [*column*]

This command is used to set the parameters required by the specified Database type real service basic or additional health check or group health check.

For existing “**health dbserver**” configurations, this command can also be used to change their parameter configurations.

<i>real_service</i> <i>add_hc_name</i> <i>group_hc_name</i>	This parameter specifies the name of the real service or real service additional health check or group health check.
---	--

- *real_service*: indicates that the parameters are set for the Database type real service basic health check.
- *add_hc_name*: indicates that the parameters are set for the Database type real service additional health check.
- *group_hc_name*: indicates that the parameters are set for the Database type group health check.

Note: The type of the real service basic or additional health check or the type of the group health check can be only “oracle-db”, “mysql-db”, “mysql-dbs”, “mssql-db” or “mssql-dbs”. For other types of real service basic and additional health checks and group health checks, this command will not work.

database_name	This parameter specifies the name of the Oracle, MySQL or MsSQL database server. For the Oracle database server, its value must be a string of 1 to 128 characters; for the MySQL database server, its value must be a string of 1 to 60 characters; for the MsSQL database server, its value must be a string of 1 to 128 characters.
username	This parameter specifies the username to log into the Oracle, MySQL or MsSQL database server. For the Oracle database server, its value must be a string of 1 to 128 characters; for the MySQL database server, its value must be a string of 1 to 16 characters; for the MsSQL database server, its value must be a string of 1 to 256 characters.
password	This parameter specifies the password to log into the Oracle, MySQL or MsSQL database server. For the Oracle database server, its value must be a string of 1 to 128 characters; for the MySQL database server, its value must be a string of 1 to 128 characters; for the MsSQL database server, its value must be a string of 1 to 128 characters.
sql_statement	Optional. This parameter specifies the SQL statement that is used for health check. Its value must be a string of 1 to 128 characters. Only the SELECT statement is supported. The SELECT statement should be enclosed in double quotes. Keywords, “select”, “from”, “where”, “like” and so on, are case insensitive. For example, “select name from student where grade > 8 and age < 16”. When this parameter is not specified, the health check will only try to establish connections with the database server. The default value is empty. This parameter only takes effects for the Oracle database health check.
expected_response	Optional. This parameter specifies the expected response to the specified SQL statement. Its value must be a string of 1 to 128 characters. If the value of “expected_response”, which is case insensitive, is included in the response of the Oracle database, they will be reported as being matched. For example, if this parameter is configured as “zhangsan” and the response of the database server is “ZHANGSAN123”, they will be reported as being matched. When this parameter is not specified, the health check will only try to establish connections with the database server. The default value is empty.

	This parameter only takes effects for the Oracle database health check. This parameter needs to be specified only when the “sql_statement” parameter is specified.
row	Optional. This parameter specifies the row from which the value of the “expected_response” is searched for. Its value must be a string of 1 to 128 characters. The default value is 1.
	This parameter only takes effects for the Oracle database health check.
column	Optional. This parameter specifies the column from where the value of the “expected_response” is searched for. Its value must be a string of 1 to 128 characters. The default value is 1.
	This parameter only takes effects for the Oracle database health check.

**Note:**

- The Database type health check does not support the SSL handshake with two-way authentication.
- Each health check of the oracle database type only supports one SQL statement.

no health dbserver *{real_service|add_hc_name|group_hc_name}*

This command is used to delete the parameter settings of the specified Database type real service basic or additional health check or group health check.

show health dbserver *[real_service|add_hc_name|group_hc_name]*

This command is used to display the parameter settings of the specified Database type real service basic or additional health check or group health check. If the “real_service|add_hc_name|group_hc_name” parameter is not specified, the parameter settings of all Database type real service basic and additional health checks and group health checks will be displayed.

clear health dbserver

This command is used to clear the parameter settings of all Database type real service basic and additional health checks and group health checks.

➤ **POP3 Health Check**

health pop3 *{real_service|add_hc_name|group_hc_name}* *<username>*
<password>

This command is used to set the parameters required by the specified POP3-type basic or additional health check for real service or POP3-type group health check.

For existing “**health pop3**” configurations, this command can also be used to change their parameter values.

<code>real_service add_hc_name group_hc_name</code>	This parameter specifies the name of the basic or additional health check for the real service or the group health check. • <code>real_service</code>: indicates that the parameters are set for the POP3-type real service basic health check. • <code>add_hc_name</code>: indicates that the parameters are set for the POP3-type real service additional health check. • <code>group_hc_name</code>: indicates that the parameters are set for the POP3 type group health check.
<code>username</code>	This parameter specifies the username to log into the POP3 server. Its value must be a string of 1 to 128 characters and must be enclosed in double quotes.
<code>password</code>	This parameter specifies the password to log into the POP3 server. Its value must be a string of 1 to 128 characters and must be enclosed in double quotes. Note: For popular POP3 mail servers, the value of the “password” parameter is often not the password of the email account specified by the “username” parameter, but the authentication code generated by the server to start POP3 service.



Note:

- To ensure the POP3-type group health check work properly, the same usernames and passwords should be configured for all servers in the POP3 server group.
- To ensure the POP3 health check work properly, user must specify a pair of health check request and expected response by executing the commands “**health request**” and “**health server**”.

no health pop3 {real_service|add_hc_name|group_hc_name}

This command is used to delete the parameter values set for the specified POP3-type basic or additional health check for real service or POP3-type group health check.

show health pop3 [real_service|add_hc_name|group_hc_name]

This command is used to display the parameter values set for the specified POP3-type basic or additional health check for real service or POP3-type group health check. If

the “real_service|add_hc_name” parameter is not specified, the system will display the configurations of all POP3-type basic and additional health checks for real service and POP3-type group health checks.

clear health pop3

This command is used to clear the parameter values set for all POP3-type basic and additional health checks for real service and POP3-type group health checks.

➤ **SNMP Health Check**

health snmp {real_service|add_hc_name|group_hc_name} [community] [version]

This command is used to set the parameters required by the specified SNMP-type basic or additional health check for real service or group health check. At most 80 commands can be configured.

For existing “health snmp” configurations, this command can also be used to change their parameter configurations.

real_service add_hc_name group_hc_name	This parameter specifies the name of the basic or additional health check for the real service or the group health check. • real_service: indicates that the parameters are set for the SNMP-type real service basic health check. • add_hc_name: indicates that the parameters are set for the SNMP-type real service additional health check. • group_hc_name: indicates that the parameters are set for the SNMP-type group health check.
community	Optional. This parameter specifies the password for the SNMP network administrator to access the system. Its value must be a string of 1 to 32 characters. The default value is “public”.
version	Optional. This parameter specifies the SNMP version. Its value must be “v1” or “v2c”. The default value is “v1”.

no health snmp {real_service|add_hc_name|group_hc_name}

This command is used to delete the parameter values set for the specified SNMP-type basic or additional health check for real service or the group health check.

show health snmp [real_service|add_hc_name|group_hc_name]

This command is used to display the parameter values set for the specified SNMP-type basic or additional health check for real service or the group health check. If the “real_service|add_hc_name|group_hc_name” parameter is not specified, the system will display the configurations of all SNMP-type basic and additional health checks for real service and the group health checks.

clear health snmp

This command is used to clear the parameter values set for all SNMP-type basic and additional health checks for real service and the group health checks.

➤ NTP Health Check

health ntpauth {real_service|add_hc_name|group_hc_name} <auth_id> <type> <key>

This command is used to set the parameter required by the basic or additional health check of an NTP real service or group health check. After this command is configured, the system will establish a UDP connection with a real service and send a request of NTP clock synchronization. If an expected NTP response is received, the system will determine this real service can work properly.

For the existing **health ntpauth** configuration, this command can also be used to modify its configured parameter.

real_service add_hc_name group_hc_name	This parameter specifies the names of the following items: real services, the additional health check of real services, or group health check. • real_service: set the parameter for the basic health check of an NTP real service. • add_hc_name: set the parameter for the additional health check of an NTP real service. • group_hc_name: set the parameter for NTP group health check. Note: The basic or additional health check of a real service or group health check’s type can only be “ntp.” For other types of the basic or additional health check of a real service and group health check, this command doesn’t work.
auth_id	This parameter specifies a key serial number for authentication. The value must be an integer ranging from 1 to 65535.
type	This parameter specifies the algorithm that generates the authentication data feeds. The value must be MD5, SHA1, SHA224, SHA256, SHA384, SHA512, MDC2,

and RIPEMD160.

key	This parameter specifies the key used to authenticate the client. The value must be a string with a length not greater than 64 bytes. If the string is 1 to 20 bytes long, the value will be an ASCII string; if the string is 21 to 64 bytes long, the value will be a hexadecimal string. When the value contains double quotes, it must be entered in hexadecimal format.
-----	--

no health ntpauth {*real_service*|*add_hc_name*|*group_hc_name*}

This command is used to delete the parameter configuration of the basic or additional health check of an NTP real service or group health check.

show health ntpauth [*real_service*|*add_hc_name*|*group_hc_name*]

This command is used display the parameter configuration of the basic or additional health check of an NTP real service or group health check. If the parameter “*real_service*|*add_hc_name*|*group_hc_name*” is not specified, the parameter configurations of the basic or additional health check of all NTP real services and group health check will be displayed.

➤ Custom Health Check Request and Response

health request <*request_index*> <*request_string*>

This command is used to define a custom health check request and insert it into the specified index location of the health check request table. This command supports defining the custom health check requests for basic, additional and group health checks of HTTP, HTTPS, HTTP/2, HTTPS/2, FTP, DNS, DNS-TCP, POP3, SMTP, UDP and SNMP types. By default, the health check request of each index location in the health check request table is “HEAD /HTTP/1.0\r\n\r\n”.

This command is also used to modify the defined health check request.

request_index	This parameter specifies the index location of the health check request in the health check request table. Its value must be an integer ranging from 0 to 999.
---------------	--

request_string	This parameter specifies the content of the health check request. Its value must be a string with a maximum length of 895 characters. If the parameter value contains numbers or spaces, it must be enclosed in double quotes. The parameter format needs to meet the RFC specifications of specific protocol; the character type of the parameter can be either ASCII-type or HEX-type. The following configuration takes DNS requests as an example:
----------------	--

- For ASCII-type character, the DNS request format is

“DNS:<domain_name>”, such as
“DNS:www.mailserver.com.

- For HEX-type character, the DNS request format is
“<hex_value>”, such as
“4b530000000100000000000003777777016103636f6
d0000010001” °

Meanwhile, the corresponding DNS responses need to be specified through the “**health response**” command.

Note:

- For HTTP/2, the format of this parameter’s value is **HEAD /** or **GET /**, such as **GET /index.html**.
- For SMTP, it is recommended to use EHLO and HELO commands to define SMTP-type health check request, which only supports ASCII-type character and must end with “\r\n”. The format is “HELO <domain_name>\r\n”, for example, “HELO mailserver.com\r\n”.
- It is recommended to use the STAT, NOOP and RSET commands to define the POP3-type health check request, which must be in upper case and ended with “\r\n”, for example, “STAT\r\n”.
- The value should be an SNMP OID if it is configured for an SNMP health check request.

For more about how to configure this parameter, please contact Customer Support.

For example:

```
AN(config)#health request 0 "GET /a.html HTTP/1.0\r\n\r\n"
AN(config)#health request 1 ".1.3.6.1.4.1.7564.3.1.0"
```



Note: For the HTTP health check, the response packet of the real service can be received by segments and will be used to match with the expected response to determine whether they are matched. For the HTTPS health check, the entire SSL response packet of the real service will be received and then used to match with the expected response to determine whether they are match. The SSL response packet allowed on the system is no more than 10 KB. If the requested file is more than 10 KB, it will lead to the health check failure.

no health request <request_index>

This command is used to reset the health check request in the specified index location of the health check request table to the default configuration.

show health request [request_index]

This command is used display the health check request in the specified index location of the health check request table. If the “request_index” parameter is not specified or is set to 65535, the entire health check request table will be displayed.

clear health request

This command is used to reset all health check requests in the health check request table to default configurations.

health import request <request_index> <url>

This command is used to import a health check request file into the specified index location in the health check request table from the HTTP or FTP server. When the imported health request file contains HTTP requests, the imported request file requires two additional newlines.

request_index	This parameter specifies the index location in the health check request table. Its value must be an integer ranging from 0 to 999.
---------------	--

url	This parameter specifies the HTTP or FTP URL of the health check request file.
-----	--

Note: The “path” of the FTP URL (ftp://user:password@host:port/path) should be a relative path.

no health import request <request_index>

This command is used to delete the health check request file imported into the specified index location of the health check request table.

show health import request <request_index> [output_mode]

This command is used to display the health check request file imported into the specified index location of the health check request table.

request_index	This parameter specifies the index location of the health check request file in the health check request table.
---------------	---

output_mode	Optional. This parameter specifies the output mode of the health check request file. Its value must be:
-------------	---

- binary: indicates binary output mode.
- text: indicates text output mode.

The default value is “binary”.

clear health import request

This command is used to delete all imported health check request files.

health load request <request_index>

This command is used to load the health check request file imported in the specified index location of the health check request table.

request_index	This parameter specifies the index location of the health check request file in the health check request table.
---------------	---

health response <response_index> <response_string> [relation]

This command is used to define a health check response and insert it into the specified index location of the health check response table. This command supports defining the custom health check responses for basic, additional and group health checks of HTTP, HTTPS, HTTP/2, HTTPS/2, FTP, DNS, DNS-TCP, POP3, SMTP, UDP and SNMP types. By default, the health check response in each index location of the health check response table is “200 OK”.

This command is also used to modify the defined health check response.

response_index	This parameter specifies the index location of the expected health check response in the health check response table. Its value must be an integer ranging from 0 to 999.
----------------	---

response_string	This parameter specifies the content of the expected health check response. Its value must be a string of 1 to 895 characters. If the parameter value contains numbers or spaces, it must be enclosed in double quotes. The parameter format needs to meet the RFC specifications of specific protocol; the character type of the parameter can be either ASCII-type or HEX-type. The following configuration takes DNS response as an example:
-----------------	---

- For ASCII-type character, both DNS IPv4 response and IPv6 response need to be configured in format as “DNS:<IPv4_address>” and “DNS:<IPv6_address>”.
- For HEX-type character, the DNS request format is “<hex_value>”, such as “01010102”.

When this command is used to configure health check responses, this parameter supports multiple response strings and 5 at most, which are separated by semicolons. The entire value must be enclosed in double quotes. An example is “id:909;status:online;code:123”. In addition, the parameter value supports quick and full regular expressions. When the parameter value is set to a regular expression, it must be enclosed in double quotes. When the parameter value is a full regular expression, it must begin with the “<regex>” string to make a difference with the

quick regular expression. For information about the differences between the quick regular expression and the full regular expression, please refer to introductions about regular expressions in Chapter 1.

Note:

- SMTP-type health check response only supports ASCII-type character. If the value of “response_string” is included in the SMTP server response, they will be reported as being matched.
- It is recommended to define the value of “response_string” as “OK” for POP3 health check response. The value of “response_string” is case sensitive.
- The response of the SNMP-type health check should be the same as the query result of the OID. The OID is specified by the parameter “request_string” of the command “health request”.

For more about how to configure this parameter, please contact Customer Support.

relation

This parameter specifies the relation between multiple response strings. Its value must be:

- and: health check is successful when the response contains all the expected response strings.
- or: health check is successful when the response contains any one of the response strings.

The default value is “and”.

For example:

```
AN(config)#health response 5 "200 OK"
AN(config)#health response 1 "7E550BD4D1C9A00005012647666050"
```

no health response <response_index>

This command is used to reset the health check response in the specified index location of the health check response table to the default configuration.

show health response [response_index]

This command is used to display the health check response in the specified index location of the health check response table. If the “response_index” parameter is not specified or is set to “65535”, the entire health check response table will be displayed.

clear health response

This command is used to reset all health check responses in the health check response table to default configurations.

health import response <response_index> <url>

This command is used to import a health check response file into the specified index location of the health check response table from the HTTP or FTP server.

response_index	This parameter specifies the index location of the health check response table. Its value must be an integer ranging from 0 to 999.
url	This parameter specifies the HTTP or FTP URL of the health check response file. Its value must be a string of 1 to 1023 bytes. Note: The “path” of the FTP URL (ftp://user:password@host:port/path) should be a relative path.

no health import response <response_index>

This command is used to delete the health check response file imported into the specified index location of the health check response table.

show health import response <response_index> [output_mode]

This command is used to display the health check response file imported into the specified index location of the health check response table.

response_index	This parameter specifies the index location of the health check response file in the health check response table.
output_mode	Optional. This parameter specifies the output mode of the health check response file. Its value must be: • binary: indicates binary output mode. • text: indicates text output mode. The default value is “binary”.

clear health import response

This command is used to clear all imported health check response files.

health load response <response_index>

This command is used to load the health check response file imported into the specified index location of the health check response table.

response_index This parameter specifies the index location of the expected health check response file in the health check response table.

health server {*real_service*|*add_hc_name*|*group_hc_name*} <*request_index*> <*response_index*>

This command is used to associate the real service basic or additional health check or group health check with the custom health check request and health check response.

real_service|add_hc_name|group_hc_name This parameter specifies the name of the real service or real service additional health check or group health check.

- **real_service**: indicates the real service basic health check.
- **add_hc_name**: indicates the real service additional health check.
- **group_hc_name**: indicates the group health check.

Note: The type of the real service basic or additional health check or the type of the group health check can be only “http”, “https”, “http2”, “https2”, “ftp”, “dns”, “pop3”, “smtp” or “udp”. For other types of real service basic and additional health checks and group health checks, this command will not work. When the health check type is “http” or “https”, the **url_path** and **expected_codes** parameters configured in the “**slb health**” command do not take effect.

request_index This parameter specifies the index location of the health check request in the health check request table.

response_index This parameter specifies the index location of the expected health check response in the health check response table.

no health server {*real_service*|*add_hc_name*|*group_hc_name*}

This command is used to delete the association of the specified real service basic or additional health check or group health check with the custom health check request and health check response.

show health server [*real_service*]

This command is used to display the status of the specified real service and the related health checks (including the basic health check and the additional health check). If the “**real_service**” parameter is not specified, all real services and the related health checks will be displayed.

APV1(config)#**show health server**

----- Server Status -----				
real server name	status			
r1	UP			
r2	UP			
r3	DOWN			
----- Health Check -----				
real server name	ip	:port	status	hct
r1	172.16.63.201	:80	UP	tcp
r2	172.16.63.200	:80	UP	tcp
r3	172.163.25.1	:80	DOWN	tcp

rqr rpr checklist

clear health server

This command is used to clear the association of all the real service basic and additional health checks and group health checks with the custom health check requests and health check responses.

➤ Script Health Check

health checker <checker_name> <request_index> <response_index>
[timeout] [flag]

This command is used to define a health check used by the script health check. A maximum of 1000 health checks used by the script health check can be configured in the system.

checker_name	This parameter specifies the name of the health check used by the script health check. Its value must be a case-sensitive string of 1 to 128 characters and must be enclosed in double quotes when containing special characters.
request_index	This parameter specifies the index location of the health check request in the health check request table.
response_index	This parameter specifies the index location of the expected health check response in the health check response table.
timeout	Optional. This parameter specifies timeout value of the health check, in seconds. Its value must be an integer ranging from 1 to 3600. The default value is 3.
flag	Optional. This parameter specifies a flag to mark the health check as success or failure. Its value must be: 0: indicates that if the response returned by the real service matches the expected health check response specified by “response_index” parameter, the real service will be marked as “Down”. 1: indicates that if the response returned by the real service matches the expected health check response

specified by “response_index” parameter, the real service will be marked as “Up”.

- 2: indicates that if the response returned by the real service matches the expected health check response specified by “response_index” parameter, the real service will be marked as “Down”.
- 3: indicates that if the response returned by the real service matches the expected health check response specified by “response_index” parameter, the real service will be marked as “Up”.

When the parameter is set to 0 or 1, the characters in the health check requests and health check responses must be ASCII characters; when the parameter is set to 2 or 3, the characters in the health check requests and health check responses must be HEX characters.

The default value is 1.

no health checker <checker_name>

This command is used to delete the specified health check used by the script health check.

show health checker [checker_name]

This command is used to display the specified health check used by the script health check. If the “checker_name” parameter is not specified, all health checks used by the script health check will be displayed.

clear health checker

This command is used to clear all health checks used by the script health check.

health list <list_name>

This command is used to create a health check list used by the script health check. A maximum of 100 health check lists can be configured in the system.

list_name	This parameter specifies the name of the health check list used by the script health check. Its value must be a case-sensitive string of 1 to 128 characters. If the parameter value contains special characters, it must be enclosed in double quotes.
-----------	---

no health list <list_name>

This command is used to delete the specified health check list used by the script health check and the included health checks.

show health list [list_name]

This command is used to display the specified health check list used by the script health check and the included health checks. If the “list_name” parameter is not specified, all health lists used by the script health check and the included health checks will be displayed

clear health list

This command is used to clear health check lists used by the script health check and the included health checks.

health member <list_name> <checker_name> [place_index]

This command is used to add a health check used by the script health check to the specified health check list. For the script health check, a maximum of ten health checks can be included in one health check list and a health check can be added to multiple health check lists.

list_name	This parameter specifies the name of the health check list used by the script health check.
checker_name	This parameter specifies the name of the health check used by the script health check.
place_index	Optional. This parameter specifies the location into which the health check is inserted. Its value must be an integer ranging from 0 to 10. The default value is 0, which indicates that the health check will be inserted as the last entry. If the parameter value is not larger than the number of the health checks currently included in this list, the health check will be inserted into the specified location of the list; otherwise, it will be inserted as the last entry.

**Note:**

- The setting of the “place_index” parameter will not be saved into memory.
- The administrator can view the health checks in the health check list by using the “**show health list**” command.

no health member <list_name> <checker_name>

This command is used to delete the specified health check from the specified health check list. After a health check is deleted, the health checks behind this health check will be automatically moved forward by one location.

clear health member <list_name>

This command is used to clear all health checks from the specified health check list.

health app {real_service|add_hc_name|group_hc_name} <list_name> [frequency] [hc_local_ip] [hc_local_port]

This command is used to associate the specified script type real service basic or additional health check or group health check with the health check list. Every script type health check can be associated with one health check list only. The system will execute the health checks included in the health check list in sequence after they are associated. Once one health check is “Down”, the result of the script type health check is “Down”.

`real_service|add_hc_name` This parameter specifies the name of the real service or real service additional health check or group health check.
`e|group_hc_name`

- `real_service`: indicates the real service basic health check.
- `add_hc_name`: indicates the real service additional health check.
- `group_hc_name`: indicates the group health check.

Note: The type of the real service basic or additional health check or group health check can be only “script-tcp”, “script-udp” or “script-tcps”. For other types of real service basic and additional health checks and group health checks, this command will not work.

`list_name` This parameter specifies the name of the health check list used by the script health check.

`frequency` Optional. This parameter specifies the frequency of the additional health check, in seconds. Its value must be an integer ranging from 1 to 65,535. The default value is 2.

`hc_local_ip` Optional. This parameter specifies the source IP address of the health check. Its value must be an IPv4 or IPv6 address. For the IPv4 address, the default value is “0.0.0.0”; for the IPv6 address, the default value is “::”. “0.0.0.0” or “::” indicates that the system will automatically determine the source IP address.

`hc_local_port` Optional. This parameter specifies the source port number of the health check. Its value must be an integer ranging from 0 to 65,535. The default value is 0, which indicates that the system will automatically determine the source port number.

Note: If multiple script health checks are configured with the same IP address and port, some health checks will not work and the system will prompt the warning message: It is recommended to use the default source port number.



Note: Please add one or multiple health checks to the health check list before configuring this command. Otherwise, this command will not work.

no health app {*real_service*|*add_hc_name*|*group_hc_name*} <*list_name*>

This command is used to delete the association of the specified script type real service basic or additional health check or group health check with the health check list.

show health app [*real_service*|*add_hc_name*|*group_hc_name*]

This command is used to display the association of the specified script type real service basic or additional health check or group health check with the health check list. If the “*real_service*|*add_hc_name*|*group_hc_name*” parameter is not specified, all associations of script type health checks with the health check lists will be displayed.

clear health app

This command is used to clear the associations of all script type health checks with the health check lists.

Layer 2 SLB Health Check

health l2slb route {on|off}

This command is used to enable or disable the route mode for Layer 2 SLB health check. If the route mode is enabled, the health check requests for L2 real services are sent to the destination IP address according to the routing table. If the route mode is disabled, the health check requests for L2 real services are sent to the MAC address corresponding to L2 real services. By default, this function is disabled.

show health l2slb

This command is used to display the status of the route mode of Layer 2 SLB health check.

health ipreflect <*reflector_name*> <*ip_address*> <*port*> [*protocol*]

This command is used to create a health check reflector used for Layer 2 SLB TCP health check. For the two-arm Layer 2 SLB health check, one APV appliance sends the health check request, the other APV appliance with the reflector configured returns the health check response. If the first APV appliance receives the correct response, the corresponding real service will be marked as “Up”; otherwise, it will be marked as “Down”. For the single-arm Layer 2 SLB health check, the APV appliance uses the first port to send the health check request, and uses another port with the reflector configured to return the health check response. If the first port receives the correct response, the corresponding real service will be marked as “Up”; otherwise, it will be marked as “Down”.

reflector_name This parameter specifies the reflector name. Its value must be a case-sensitive string of 1 to 128 characters.

ip_address This parameter specifies the IP address bound to the reflector. Its value must be an IPv4 address or IPv6 address. If the parameter value is specified as “0.0.0.0”, it

	means the system will return the health check response using the IPv4 address of an interface on the appliance. If the parameter value is specified as “::”, it means the system will return the health check response using the IPv6 address of an interface on the appliance.
port	This parameter specifies the port number listened to by the reflector.
protocol	Optional. This parameter specifies the type of the health check protocol. Its value must be “tcp”. The default value is “tcp”.

no health ipreflect <reflector_name>

This command is used to delete the specified health check reflector.

show health ipreflect

This command is used to delete all health check reflectors.

clear health ipreflect

This command is used to clear all health check reflectors.

Passive Health Check

health passive on

This command is used to enable the passive health check function. By default, this function is disabled. This function can be enabled only when the health check function has been enabled.

health passive off

This command is used to disable the passive health check function.

health passive http <template_name> <abnormal_resp_code> [resp_timeout] [statistical_period] [abnormal_resp_threshold] [url_list]

This command is used to create an HTTP-type passive health check template. A maximum of 80 HTTP-type passive health check templates can be created.

template_name	This parameter specifies the name of the passive health check template. Its value must be a string of 1 to 128 characters.
abnormal_resp_code	This parameter specifies the abnormal HTTP response status code. Multiple status codes separated by semicolon (;) are supported. This parameter supports the following status codes: 100, 101, 200, 206, 301, 302, 303, 304, 305,

	307, 400, 401, 402, 403, 404, 405, 406, 407, 416, 500, 501, 502, 503, 504 and 505.
resp_timeout	Optional. This parameter specifies the timeout value of HTTP responses in seconds. Its value must be an integer ranging from 1 to 60. The default value is 10.
statistical_period	Optional. This parameter specifies the statistical period of abnormal HTTP responses in seconds. Its value must be an integer ranging from 1 to 60. The default value is 10. Note: The statistical period of abnormal HTTP responses must be equal to or greater than the timeout value of HTTP responses.
abnormal_resp_threshold	Optional. This parameter specifies the abnormal HTTP response count threshold within a statistical period. Its value must be an integer ranging from 1 to 100,000. The default value is 10.
url_list	Optional. This parameter specifies the name of the URL list monitored by the passive health check. Its value must be the name of the URL list that has been configured using the “ health passive urllist name ” command. If this parameter is not specified, the default value is empty, indicating that the passive health check will monitor all URLs of the associated real services and groups.

no health passive http <template_name>

This command is used to delete the specified HTTP-type passive health check template.

show health passive http [template_name]

This command is used to display the specified HTTP-type passive health check template. If the “template_name” parameter is not specified, all HTTP-type passive health check templates will be displayed.

clear health passive http

This command is used to clear all HTTP-type passive health check templates.



Note: HTTP-type passive health check templates that have been applied will not be cleared.

health passive urllist name <url_list>

This command is used to create a passive health check URL list, which is used as the monitoring object of the HTTP-type passive health check template.

<code>url_list</code>	This parameter specifies the name of the passive health check URL list. Its value must be a string of 1 to 128 characters.
-----------------------	--

no health passive urllist name <url_list>

This command is used to delete the specified passive health check URL list.

show health passive urllist name [url_list]

This command is used to display the specified passive health check URL list. If the “url_list” parameter is not specified, all passive health check URL lists will be displayed.

clear health passive urllist name

This command is used to clear all passive health check URL lists.



Note: URL lists that have been used by HTTP-type passive health check templates will not be cleared.

health passive urllist member <url_list> <url>

This command is used to add a URL member to the specified passive health check URL list. A maximum of 100 URL members can be added to one URL list.

<code>url_list</code>	This parameter specifies the name of an existing passive health check URL list.
-----------------------	---

<code>url</code>	This parameter specifies the URL to be added to the passive health check URL list as a member. Its value must be a string of 1 to 128 characters.
------------------	---

no health passive urllist member <url_list> <url>

This command is used to delete a specific URL member from the specified passive health check URL list.

show health passive urllist member [url_list]

This command is used to display the URL members of the specified passive health check URL list. If the “url_list” parameter is not specified, the URL members of all passive health check URL lists will be displayed.

clear health passive urllist member [url_list]

This command is used to clear the URL members of the specified passive health check URL list. If the “url_list” parameter is not specified, the URL members of all passive health check URL lists will be cleared.

health passive apply real http <real_service> <template_name>

This command is used to apply the HTTP-type passive health check template to the specified real service.

real_service	This parameter specifies the name of an existing real service. The protocol type of the real service must be HTTP or HTTPS.
--------------	---

template_name	This parameter specifies the name of an existing HTTP-type passive health check template.
---------------	---

no health passive apply real http <real_service>

This command is used to cancel the application of the HTTP-type passive health check template to the specified real service.

show health passive apply real http [real_service]

This command is used to display the HTTP-type passive health check template applied to the specified real service. If the “real_service” parameter is not specified, the HTTP-type passive health check template applied to all real services will be displayed.

clear health passive apply real http

This command is used to cancel the application of the HTTP-type passive health check template to all real services.

health passive apply group http <group_name> <template_name>

This command is used to apply the HTTP-type passive health check template to the specified group.

group_name	This parameter specifies the name of an existing group. The protocol type of the group must be HTTP or HTTPS.
------------	---

template_name	This parameter specifies the name of an existing HTTP-type passive health check template.
---------------	---

no health passive apply group http <group_name>

This command is used to cancel the application of the HTTP-type passive health check template to the specified group.

show health passive apply group http [group_name]

This command is used to display the HTTP-type passive health check template applied to the specified group. If the “group_name” parameter is not specified, the HTTP-type passive health check template applied to all groups will be displayed.

clear health passive apply group http

This command is used to cancel the application of the HTTP-type passive health check template to all groups.

health passive tcp <template_name> [statistical_period] [reset_threshold] [zerowindow_threshold]

This command is used to create a TCP-type passive health check template. A maximum of 80 TCP-type passive health check templates can be created.

template_name	This parameter specifies the name of the passive health check template. Its value must be a string of 1 to 128 characters.
statistical_period	Optional. This parameter specifies the statistical period of abnormal TCP responses in seconds. Its value must be an integer ranging from 1 to 60. The default value is 10.
reset_threshold	Optional. This parameter specifies the RESET packet count threshold within a statistical period. Its value must be an integer ranging from 0 to 100,000. 0 indicates not monitoring RESET packets. The default value is 10.
zerowindow_threshold	Optional. This parameter specifies the Zero-Window packet ratio threshold within a statistical period in percentage. Its value must be an integer ranging from 0 to 100. 0 indicates not monitoring Zero-Window packets. The default value is 40.

no health passive tcp <template_name>

This command is used to delete the specified TCP-type passive health check template.

show health passive tcp [template_name]

This command is used to display the specified TCP-type passive health check template. If the “template_name” parameter is not specified, all TCP-type passive health check templates will be displayed.

clear health passive tcp

This command is used to clear all TCP-type passive health check templates.



Note: TCP-type passive health check templates that have been applied will not be cleared.

health passive apply real tcp <real_service> <template_name>

This command is used to apply the TCP-type passive health check template to the specified real service.

real_service	This parameter specifies the name of an existing real service. The protocol type of the real service must be TCP, TCPS, HTTP or HTTPS.
template_name	This parameter specifies the name of an existing TCP-type passive health check template.

no health passive apply real tcp <real_service>

This command is used to cancel the application of the TCP-type passive health check template to the specified real service.

show health passive apply real tcp [real_service]

This command is used to display the TCP-type passive health check template applied to the specified real service. If the “real_service” parameter is not specified, the TCP-type passive health check template applied to all real services will be displayed.

clear health passive apply real tcp

This command is used to cancel the application of the TCP-type passive health check template to all real services.

health passive apply group tcp <group_name> <template_name>

This command is used to apply the TCP-type passive health check template to the specified group.

group_name	This parameter specifies the name of an existing group. The protocol type of the group must be TCP, TCPS, HTTP or HTTPS.
template_name	This parameter specifies the name of an existing TCP-type passive health check template.

no health passive apply group tcp <group_name>

This command is used to cancel the application of the TCP-type passive health check template to the specified group.

show health passive apply group tcp [group_name]

This command is used to display the TCP-type passive health check template applied to the specified group. If the “group_name” parameter is not specified, the TCP-type passive health check template applied to all groups will be displayed.

clear health passive apply group tcp

This command is used to cancel the application of the TCP-type passive health check template to all groups.

General Configurations of Health Check**health {on|off}**

This command is used to enable or disable the health check function. If this function is disabled, all health checks (including the basic health check, additional health check and group health check) will be disabled. By default, this function is disabled.



Note: With the health check function disabled, execution of the “**health on**” command will reset the health check early warning counter.

health reliable {on|off} [real_service]

This command is used to enable or disable the reliable health check function for the specified real service. When the “real_service” parameter is not specified, this function will be enabled or disabled for the global. For a specific real service, this function takes effect only when it is enabled for both the global and this real service.

By default, this function is enabled for both the global and every real service.

real_service	Optional. This parameter specifies the real service name. It must be an HTTP, HTTPS, TCP, TCPS, UDP, SIP or RTSP type real service.
--------------	---



Note: Reliable health check works only on real services without additional health checks.

show health reliable [real_service]

This command is used to display the enabling/disabling status of the reliable health check function for the specified real service. If the “real_service” parameter is not specified, the enabling/disabling status of this function for both the global and every real service is displayed.

clear health reliable

This command is used to restore the configuration of the reliable health check function for both the global and every real service to default.

health relation <real_service> <relationship> [additional_relationship]

This command is used to set the health check relationship for the specified real service (including the basic health check, additional health checks, group health checks, and passive health checks).

<code>real_service</code>	This parameter specifies the name of the real service.
<code>relationship</code>	This parameter specifies the relationship among health checks of the real service. Its value can be: • <code>and</code>: the real service is marked as “UP” only when the results of all health checks are “UP”. This value is case-insensitive. • <code>or</code>: the real service is marked as “UP” as long as the result of any health check is “UP”. This value is case-insensitive. The default value is “and”.
<code>additional_relationship</code>	This parameter specifies the relationship among health checks of the real service. Its value can be: • <code>and</code>: the overall result of additional health checks for the real service is marked as “UP” only when the results of all additional health checks are “UP”. This value is case-insensitive. • <code>or</code>: the overall result of additional health checks for the real service is marked as “UP” as long as the result of any additional health check is “UP”. This value is case-insensitive. • <code>none</code>: the system does not treat all additional health checks as a whole when determining the health status of the real service. This value is case-insensitive. The default value is “none”.

show health relation [*real_service*]

This command is used to display the relationship among all health checks of the specified real service. If the “`real_service`” parameter is not specified, the relationship among all health checks of every real service will be displayed.

health group relation <*group_name*> <*relationship*>

This command is used to set the member’s default health check relationship for the specified group.

After this command is configured, the health check relationship (“**health relation**” settings) of real services added to the group subsequently will be updated to the member’s default health check relationship.

Please note that the health check relationship of real services added to the group earlier will not be updated. To use consistent health check relationship for all real services in the group, you can use the “**health group update relation**” command to update the health check relationship for these real services.

group_name	This parameter specifies the name of the group.
relationship	This parameter specifies the default relationship among health checks of each group member. Its value can be: • and: the group member is marked as “UP” only when the results of all health checks are “UP”. This value is case-insensitive. • or: the group member is marked as “UP” as long as the result of any health check is “UP”. This value is case-insensitive. The default value is “and”.

health group update relation <group_name>

This command is used to update all real services’ health check relationship to the member’s default health check relationship for the specified group.

no health group relation <group_name>

This command is used to delete the setting of the member’s default health check relationship for the specified group.

show health group relation [group_name]

This command is used to display the setting of the member’s default health check relationship for the specified group. If the “group_name” parameter is not specified, the settings of the member’s default health check relationship for all groups are displayed.

clear health group relation

This command is used to clear the settings of the member’s default health check relationship for all groups.

health interval <interval> [server_timeout] [real_service|add_hc_name]

This command is used to set the interval and timeout value of the specified basic health check or additional health check. The basic health checks and additional health checks without the customized setting of health check interval and response timeout value via this command will use the global setting. The default interval of the health check and response timeout value for the global is five seconds.

interval	This parameter specifies the interval between health checks of the real service, in seconds. Its value must be an integer ranging from 1 to 1,000,000.
server_timeout	Optional. This parameter specifies the timeout value of the health check response, in seconds. Its value must be an

integer ranging from 1 to 1,000,000. The default value is 5.

Note: The timeout value of the health check cannot be larger than its interval.

`real_service|add_hc_name` Optional. This parameter specifies the real service name or additional health check name. If this parameter is not specified, the global health check interval and timeout value of health check response will be set.

The default value is empty.



Note: This command takes effect for the basic and additional health checks only excluding the script health checks.

no health interval *<real_service|add_hc_name>*

This command is used to reset the interval and timeout value of the health check to default values for the specified basic health check or additional health check.

show health interval *[real_service|add_hc_name]*

This command is used to display the interval and timeout value of the health check for the specified basic health check or additional health check. If the “`real_service|add_hc_name`” parameter is not specified, the health check interval and timeout settings for the global and all basic health checks and additional health checks will be displayed.

clear health interval

This command is used to reset the health check interval and timeout settings to default values for the global and all basic health checks and additional health checks.

health proxyip {on|off} *[start_port] [end_port]*

This command is used enable the system to use the health check proxy IP address pool or disable the system from using the health check proxy IP address pool. By default, this function is disabled.

The system supports global health check proxy IP address pools (configured by “**health proxyip group**”) and dedicated pools for groups (configured by “**health proxyip global**”). When this function is enabled, the system will select source IP addresses from the configured health check proxy IP address pool and select source port numbers from the port range configured by the “`start_port`” and “`end_port`” parameters.

A group will preferentially use the dedicated pool. If a group has multiple dedicated pools, it will follow these principles to select a pool:

- Select from those that are on the same network segment as the gateway. Among these pools, the one that is in the current NUMA domain has higher priority.

- If none of the pools are on the same network segment as the gateway, nor are they in the current NUMA domain, the system will randomly select one.

If only global pools are configured, the selection principles are as follows:

- Select from those that are on the same network segment as the gateway. Among these pools, the one that is in the current NUMA domain has higher priority.
- If none of the pools are on the same network segment as the gateway, the system will use the interface IP address.

If neither the global nor the dedicated pool is configured, the system will use the interface IP address.

start_port Optional. This parameter specifies the start port number. Its value must be an integer ranging from 3000 to 60,000. The default value is 3000.

end_port Optional. This parameter specifies the end port number. Its value must be an integer ranging from 3000 to 60,000. The default value is 5000.

Note: the end port number must be equal to or larger than the start port number.

health proxyip group <group_name> <pool_name>

This command is used to configure a health check IP address pool for the specified group.

group_name This parameter specifies the name of an existing real service group.

pool_name This parameter specifies the name of an existing IP address pool.

no health proxyip group <group_name> <pool_name>

This command is used to cancel the association between the specified group and the specified IP address pool.

health proxyip global <pool_name>

This command is used to configure a health check IP address pool for the global.

pool_name This parameter specifies the name of an existing IP address pool.

no health proxyip global <pool_name>

This command is used to delete the specified health check IP address pool for the global.

show health proxyip [group_name]

This command is used to display the configuration of the “**health proxyip**” command and the configurations of all health check IP address pools for the specified group. If the “group_name” parameter is not specified, the configuration of the “**health proxyip**” command and the configurations of health check IP address pools for all groups and the global will be displayed.

clear health proxyip [group_name]

This command is used to clear the configurations of all health check IP address pools for the specified group. If the “group_name” parameter is not specified, the configurations of health check IP address pools for all groups and the global will be cleared and the port range configuration will be restored to default.

health failover {enable|disable}

This command is used to enable or disable the automatic failover function. By default, this function is disabled.

In clustering, if this function is enabled and all real services on the master node fail, the backup node will take over and the clustering function on the master node will be disabled. If one or more real services on the original master node become available again, the clustering function will be enabled again and all traffic will be switched back to it with the preemption function enabled.

health failover retries <number_of_retries>

This command is used to set the maximum number of failures allowed the master node to try to provide the service. Automatic failover will be performed if the number of failures that the master node tries to provide the service reaches the value specified by the “number_of_retries” parameter. The maximum number of failures allowed the master node to try to provide the service is three by default.

number_of_retries	This parameter specifies the maximum number of failures allowed the master node to try to provide the service. Its value must be an integer ranging from 0 to 65,535.
-------------------	---

no health failover retries

This command is used to reset the maximum number of failures allowed the master node to try to provide the service to 3.

show health failover

This command is used to display the current configuration of the automatic failover function.

health earlywarning <threshold>

This command is used to enable the health early warning function. This function can be used to check the real service status by setting a global threshold for the response time of all real services. If the response time of a real service exceeds the threshold, it indicates this real service responds slowly, and it might be in abnormal status.

With this function enabled, the system will detect the event that real services' response time exceeds the threshold, and use a counter to record the times that the event occurs. Based on these records, the system will create "Warning" logs to notify the administrators of the real service's abnormal status.

For the real services without health check configured, this function is unavailable. By default, this function is disabled.

threshold This parameter specifies the threshold of the health check response, in milliseconds. If the parameter value is 0, it means this function is disabled.

**Note:**

- The system will record "Warning" logs only when the recorded times that a real services' response time consecutively exceeds the threshold is the power of 2 (1, 2, 4, 8...). Once the response time returns to the normal level, which is below the threshold, related old records will be cleared. The counter will begin to collect new records.
- At most 1024 records are allowed on the counter. If the number of records exceeds 1024, the counter will be reset to 0 and start to recount.

show health earlywarning

This command is used to display the settings of the threshold for the health early warning function



Note: With the health check function disabled, it will reset the early warning counter by executing the command "**health on**".

clear health earlywarning

This command is used to reset the health early warning function to the default configuration.

health connclose {fin|rst} [real_service]

This command is used to configure the health check connection close mode for the specified real service. If the "real_service" parameter is not specified, this command will configure the health check connection close mode for all real services.

When "**health connclose fin**" is configured, the system will close the health check connection after receiving the FIN message from the real service. When "**health**

connclose rst” is configured, the system will proactively send a RST to close the health check connection after the health check is completed.

For the real service without this command configured, the FIN close mode for health check connections will be used by default.



Note: Configuration of this command takes effect on only TCP-based health check types.

show health connclose *[real_service]*

This command is used to display the configuration of the health check connection close mode for the specified real service. If the “real_service” parameter is not specified, the configuration of the health check connection close mode for all real services will be displayed.

health action disable *[force_offline] [real_service]*

This command is used to configure the system to disable the specified real service when its health check result is Down.

After a real service is disabled because its health check fails, it will not be automatically enabled even if its health check succeeds again. It can only be enabled manually.

After a real service is gracefully disabled, requests coming from clients it used to serve will still be forwarded to it, but requests coming from new clients will be forwarded to other available real services.

After a real service is forcibly disabled, the system will not forward any requests to it.

After all connections on this real service are reset, the real service will be disabled and no request will be forwarded to it.

force_offline Optional. This parameter specifies how to disable the real service. Its value must be:

- 0: gracefully disables the real service.
- 1: forcibly disables the real service.
- 2: resets all connections to the real service.

The default value is 0.

real_service Optional. This parameter specifies the real service name. When this parameter is not specified, this command will be configured for all real services.



Note: When a real service is gracefully or forcibly disabled, all connections will be reset. However, when its health check succeeds again, it will still work in the gracefully or forcibly disable mode.

no health action disable <real_service>

This command is used to delete the “**health action disable**” configuration of the specified real service.

show health action disable [real_service]

This command is used to display the “**health action disable**” configuration of the specified real service. If the “real_service” parameter is not specified, the “**health action disable**” configurations of all real services will be displayed.

clear health action disable

This command is used to clear the “**health action disable**” configurations of all real services.

show health template <application_type>

This command is used to display the health check configuration examples for a specified type of application.

application_type	This parameter specifies the type of the application protocol. Its value must be “ftp”, “telnet”, “smtp”, “ldap”, “radius-auth”, “radius-acct” or “all”. If the parameter value is “all”, health check configuration examples for all applications will be displayed.
------------------	---

health real threshold <threshold> [real_service]

This command is used to configure a rule for determining the additional health check result as being Up for the specified real service. When a real service has multiple additional health checks, the final result of its additional health checks will be set to Up only when the number of “Up” additional health checks is equal to or larger than the specified threshold.

threshold	This parameter specifies the threshold for the number of “Up” additional health checks.
real_service	Optional. This parameter specifies the real service name. When this parameter is not specified, this command will be configured for all real services.



Note: For the real service configured with the “**health real threshold**” command, the “**health relation**” configuration does not take effect on its additional health checks.

no health real threshold *<real_service>*

This command is used to delete the configuration of the “**health real threshold**” command for the specified real service.

show health real threshold *[real_service]*

This command is used to display the configuration of the “**health real threshold**” command for the specified real service. When the “*real_service*” parameter is not specified, the configurations of the “**health real threshold**” command for all real services will be displayed.

clear health real threshold

This command is used to clear the configurations of the “**health real threshold**” command for all real services.

show health status

This command is used to display the configurations of health checks, the status of real services and associated health checks.

Other SIP Commands**sip nat** *<virtual_ip> <virtual_port> <real_ip> <real_port> [protocol] [timeout] [persistence_mode]*

This command is used to configure a SIP NAT rule to translate the specified real service’s IP address and port number in responses to the specified virtual service’s IP address and port number.

<i>virtual_ip</i>	This parameter specifies the virtual service’s IP address to which the source IP address will be translated. Its value must be an IPv4 or IPv6 address.
<i>virtual_port</i>	This parameter specifies the virtual service’s port number to which the source port number will be translated. Its value must be an integer ranging from 0 to 65,535. The value 0 indicates that the source port number will be used.
<i>real_ip</i>	This parameter specifies the source IP address in the response, that is, the IP address of the real service. Its value must be an IPv4 or IPv6 address.
<i>real_port</i>	This parameter specifies the source port number in the response, that is, the port number of the real service. Its value must be an integer ranging from 0 to 65,535. The value 0 indicates all port numbers.
<i>protocol</i>	Optional. This parameter specifies the protocol of the

	response to be translated. Its value must be “udp” or “tcp”. The default value is “udp”.
timeout	Optional. This parameter specifies the timeout value of the SIP NAT rule, in seconds. Its value must be an integer ranging from 0 to 65,535. The default value is 60.
persistence_mode	Optional. This parameter specifies the persistence mode of the SIP NAT session. Its value must be: • callid: indicates the session ID. • userid: indicates the user ID. The default value is “callid”.

no sip nat <real_ip> <real_port> [protocol]

This command is used to delete a SIP NAT rule configuration for the specified real service.

show sip nat

This command is used to display all SIP NAT rule configurations.

clear sip nat

This command is used to clear all SIP NAT rule configurations.

sip multireg {on|off}

This command is used to enable or disable the SIP registration information synchronization function. After this function is enabled, clients’ registration requests will be advertised to all real services in a SIP real service group for registration information synchronization, but processed by only one real service in the group. By default, this function is disabled.

show sip multireg

This command is used to display the status of the SIP registration information synchronization function.

Basic SLB Commands**SLB Mode****slb mode ircookie <ircookie_mode>[group_name] [password]**

This command is used to configure the format of real service information in the cookie when the Insert Cookie, Rewrite Cookie, or Embed cookie method is used.

If this command is not configured, the system uses the ASCII value of the real service name in the cookie to indicate the real service.

- ircookie_mode** This parameter specifies the format of real service information in the cookie when the Insert Cookie, Rewrite Cookie, or Embed Cookie method is used. Its value must be:
- **plainname:** uses the ASCII value of the real service name to indicate the real service in the cookie. For example, “aTc8acd” in cookie “name=aTc8acd!!9”.
 - **hexname:** uses the hexadecimal value of the real service name to indicate the real service in the cookie. For example, “456143” in cookie “name=456143!!04”.
 - **ip:** uses the hexadecimal value of the real service’s IP address to indicate the real service in the cookie. For example, “0A010203” in cookie “name=0A010203” or “name=0A010203!!9”.
 - **enc_name:** uses the AES encryption value of the real service name to indicate the real service in the cookie. For example, “C88BA867994F440963F55B727CDD3CB” in cookie “name=C88BA867994F440963F55B727CDD3CB7!!9”.
 - **enc_ip:** uses the AES encryption value of the real service’s IP address to indicate the real service in the cookie. For example, “B9F574EF92836DFD2CC0EE03E7A0E717” in cookie “name=B9F574EF92836DFD2CC0EE03E7A0E717!!03”.

Note: “!!” marks the end of the rewritten part.

group_name Optional, this parameter specifies the name of the existing real service group. If this parameter is not specified, the value will be set as “global” by default, which means that the format configuration for real service information in the cookie will be applied globally.

password This parameter specifies the AES encryption password

no slb mode ircookie <group_name>

This command is used to delete the format configuration for specified real service group in the cookie when the Insert Cookie, Rewrite Cookie, or Embed cookie method is used.

show slb mode ircookie *[group_name]*

This command is used to display the format configuration for specified real service group in the cookie when the Insert Cookie, Rewrite Cookie, or Embed cookie method is used. If the “group_name” is not specified, the format configuration of all real service groups will be displayed.

clear slb mode ircookie

This command is used to restore the format configuration for real service information in the cookie when the Insert Cookie, Rewrite Cookie, or Embed cookie method is used.

slb mode icookie *<insert_mode>*

This command is used to control the insert cookie behavior for responses to the requests hitting the Insert Cookie method to fit different browsers.

When this command is not configured, the system inserts a cookie into every response to the request that hits the Insert Cookie method.

insert_mode	This parameter specifies the insert cookie behavior for responses to the requests that hit the Insert Cookie method. Its value must be:
-------------	---

- always: inserts cookies into responses to all requests.
- onlyone: only inserts cookies into responses to the requests that do not contain cookies.

show slb mode icookie

This command is used to display the configuration of the “slb mode icookie” command.

clear slb mode icookie

This command is used to restore the configuration of the “slb mode icookie” command to default.

slb mode keepport {on|off}

This command is used to enable or disable the SLB port keeping mode. When it is enabled, the system will continue using the client source port number while attempting to establish a connection with the real service. By default, the SLB port keeping mode is disabled.



Note:

- Double NUMA domain devices that have enabled NUMA function (using the “**system tune numa auto**” command) do not support SLB port keeping mode.
- After DirectFWD is enabled (using the command **slb directfwd on**), the SLB port keeping mode will not take effect on layer 4 load balancing.
- After the SLB port keeping mode is enabled, the system might not keep using the client source port while attempting to establish a connection with the real service if the port is occupied (for example, by a long-lived TCP connection).

show slb mode keepport

This command is used to display the configuration of the SLB port keeping mode.

SLB DirectFWD

slb directfwd {on|off} [virtual_service]

This command is used to enable or disable the DirectFWD function for a virtual service or globally. This function supports both the IPv4 and IPv6 environments. By default, this function is globally disabled and enabled per virtual service. For more information about the DirectFWD function, please refer to the ArrayOS APV 10.0 User Guide.

Note that this function only applies to TCP load balancing, but not to scenarios where the real service is of the TCPS type. After this function is enabled, the system will not collect the client connection RTT statistics of the TCP virtual service. The value of “Average client connection RTT” item in the output of the “**show statistics slb virtual tcp**” command will be 0. For the TCP real service without a basic health check, the system will not collect the average response time of real services. The value of the “Average Response time” item in the output of the “**show statistics slb real tcp**” command will be 0.

<code>virtual_service</code>	Optional. This parameter specifies the name of a virtual service. This function only supports TCP virtual services. If this parameter is not specified, the function will be enabled or disabled for the global.
------------------------------	--

The default value is empty.

The maximum segment size (MSS) of packets sent to the server is set to 1460 by default when **slb directfwd** is used with WebAgent. The default MSS value can be changed for each virtual service using the **slb tcpoption mss** command without enabling the **slb directfwd syncache** command.



Note: For segment user, the DirectFWD function takes effect for a virtual service only when it is enabled globally.

slb directfwd statistics {on|off}

This command is used to enable or disable the SLB statistics collection function. After this function is enabled, the system will collect SLB statistics in the DirectFWD mode. By default, this function is enabled.

slb directfwd syncache {on|off}

This command is used to enable or disable the Syncache function in the DirectFWD mode. This function helps defend against synflood attacks. By default, this function is disabled.



Note: **slb directfwd syncache** sends the SYN/ACK packet on behalf of the client. This packet might be different from the server expected to receive from the client. The APV's TCP option might also be different from the server's. For example, traffic is forwarded from WebAgent to SLB.

show slb directfwd [virtual_service]

This command is used to display the status of the DirectFWD function for the specified virtual service. If the parameter “virtual_service” is not specified, the status of following functions will be displayed:

- DirectFWD function
- SLB statistics collection function in the DirectFWD mode
- Syncache function in the DirectFWD mode

clear slb directfwd

This command is used to restore the following functions:

- DirectFWD function for global and TCP virtual service
- SLB statistics collection function in the DirectFWD mode
- Syncache function in the DirectFWD mode

Actively Closing TCP Connections

slb mode activeclose {on|off} [real_service]

This command is used to enable or disable the function of actively closing TCP connections for the specified real service. If the “real_service” parameter is not specified, this command is used to enable or disable the function of actively closing TCP connections globally. After this function is enabled, if an IP, TCP, TCPS, HTTP or HTTPS real service becomes “Down”, the system will close the related TCP

connection. If this function is disabled, the system will not close the TCP connection related to the real service that is Down.

By default, this function is enabled for every real service, but it is globally disabled. To make this function take effect for a real service (including the segment scenario), you need to firstly enable this function globally.

The system also supports the function of passively closing TCP connections. This function checks the health status of the real service by examining each packet sent to it. If the real service becomes “Down”, the system will reset the TCP connection. The function of passively closing TCP connections is always in the enabled status, regardless of whether the “**slb mode activeclose**” command is configured.

<code>real_service</code>	Optional. This parameter specifies the real service name. The default value is empty, indicating that the function is globally enabled or disabled.
---------------------------	--

show slb mode activeclose

This command is used to display the status of the function of actively closing TCP connections, including the function status at the global level and the real service level.

slb mode lineup {on|off} <virtual_service|vlink_name> [length] [timeout]

This command is used to enable or disable the new connection caching function for the specified virtual service or vlink. After this function is enabled, the system caches new connections when the real service is overloaded.

<code>virtual_service vlink_name</code>	Optional. This parameter specifies the virtual service or vlink name. The TCP, TCPS, HTTP, and HTTPS virtual services are supported.
<code>length</code>	Optional. This parameter specifies the upper limit of new connections to be cached. The minimum value is 1. The maximum value varies with system memories: • System memory <= 8 GB: 2000 • System memory = 16 or 24 GB: 4000 • System memory >= 32 GB: 8000 The default value is 1000.
<code>timeout</code>	Optional. This parameter specifies the timeout value for new connection caching. Its value ranges from 1 to 60, in seconds. The default value is 5.

show slb mode lineup [virtual_service|vlink_name]

This command is used to display the configuration of the new connection caching function for the specified virtual service or vlink.

Overload Persistence

slb overload persistence {on|off}

This command is used to enable or disable the overload persistence function. When this function is enabled, the connections that hit cookie-related methods will not be restricted by the maximum number of connections supported by the real service. This function works only for the Persistent Cookie, Insert Cookie, Rewrite Cookie, and Embed Cookie methods. By default, this function is disabled.

show slb overload persistence

This command is used to display the status of the overload persistence function.

Host Node

slb node name <node_name> <ip|domain>

This command is used to manually create an SLB host node.

The SLB host node is a group of real services with the same IP address or domain name. After this command is used, the system creates IP-type host nodes for real services of IP address type, and domain-type host nodes for real services of domain name type. In addition, the system also supports the automatic creation of host nodes based on the IP address or domain name of the real service when a real service is created.

Manually created SLB host nodes can only be deleted manually; automatically generated SLB host nodes are automatically deleted when real services are deleted.

The administrator can execute the command “**write memory**” to save the configuration of the manually created host node; the configuration of the automatically generated host node cannot be saved.

The number of SLB host nodes that the system supports manually configured is the same as that of the real service.

node_name	This parameter specifies the name of the SLB node. Its value must be a case-sensitive string of 1 to 255 characters and must be enclosed in double quotes when containing special characters. The parameter value cannot contain spaces or be the system reserved word “default”, “all”, or “global”, which is case-insensitive.
ip domain	This parameter specifies the IP address or domain name of the SLB node. Its value must be an IPv4 or IPv6 address or a string of 1 to 255 characters.

no slb node name <node_name>

This command is used to delete a manually configured SLB host node. Before deleting a host node, you need to delete the associated real service. This command cannot delete an auto-generated host node. Auto-generated host nodes can only be automatically deleted when real services are deleted.

show slb node [node_name]

This command is used to display the configuration of the specified SLB host node. When the “node_name” parameter is not specified, the command will display the configuration of all SLB nodes.

clear slb node

This command is used to clear all manually configured SLB host nodes.

slb node enable <node_name>

This command is used to enable a specified SLB host node. When a host node is enabled, all real services associated with this host node will be enabled.

By default, all host nodes are enabled. This command takes effect on all auto-generated or manually configured host nodes.

node_name	This parameter specifies the name of an existing SLB host node.
-----------	---

slb node disable <node_name> [force_offline]

This command is used to disable a specified SLB host node. When a host node is disabled, all real services associated with this host node will be disabled. This command takes effect on all auto-generated or manually configured host nodes.

node_name	This parameter specifies the name of an existing SLB host node.
-----------	---

force_offline	Optional. This parameter specifies whether to forcibly disable the real service. Its value must be:
---------------	---

- 0: gracefully disables the real service.
- 1: forcibly disables the real service.
- 2: resets all connections to the real service.

The default value is 0.

show statistic slb node [node_name]

This command is used to display the statistics of the specified SLB host node. If the parameter “node_name” is not specified, statistics for all SLB host nodes will be displayed.

node_name	Optional. This parameter specifies the name of an existing SLB host node.
-----------	---

Other Basic SLB Commands

show slb all

This command is used to display all SLB configurations, including virtual service, real service, group, group method, group member, and policy configurations.



Note: If the “ip|domain” parameter in the commands “**slb real http**”, “**slb real https**”, “**slb real ftp**”, “**slb real tcp**”, “**slb real tcps**” and “**slb real ip**” set to “domain”, the IP address corresponding to the domain name of the real service will be included in the displayed real services.

clear slb all

This command is used to clear all SLB configurations.

slb regextest <regex> <target_string>

This command is used to test whether the specified target string will match the configured regular expression, which helps ensure the correctness of a configured regular expression.

regex	This parameter specifies the regular expression to be configured. Its value must be a standard Perl regular expression enclosed in double quotes and cannot begin with the “<regex>” string even if it is a full regular expression.
-------	--

target_string	This parameter specifies the target string to be matched, such as the host name and URL string.
---------------	---

slb timeout <virtual_service> <timeout> [timeout_type]

This command is used to set the timeout value of TCP connections for the specified virtual service. The timeout value set by this command takes precedence over the timeout value set by the “**system tune tcpidle**” command.

If this command is not configured, the system uses the “**system tune tcpidle**” configuration as the timeout value of TCP connections.

virtual_service	This parameter specifies the virtual service name.
timeout	This parameter specifies the timeout value of TCP connections, in seconds. Its value must be an integer ranging from 1 to 2,592,000.
timeout_type	Optional. This parameter specifies the TCP connection state for which the TCP timeout setting is configured. Its value must be: • idle: indicates the idle state. • fin_wait_1: indicates the fin_wait_1 state. • fin_wait_2: indicates the fin_wait_2 state. • closing: indicates the closing state. • time_wait: indicates the time_wait state. • close_wait: indicates the close_wait state. • last_ack: indicates the last_ack state. The default value is “idle”.



Note: The timeout of the TCP connection in the **TIME_WAIT** status might be inaccurate when the administrator sets the “**timeout_type**” parameter to “**time_wait**” and the timeout parameter to different values for multiple virtual services.”

no slb timeout <virtual_service> [timeout_type]

This command is used to delete the TCP connection timeout setting for a TCP connection state the specified virtual service. If the “timeout_type” parameter is not specified, TCP connection timeout settings for all TCP connection states will be deleted.

show slb timeout [virtual_service]

This command is used to display the TCP connection timeout setting for the specified virtual service. If the “virtual_service” parameter is not specified, the TCP connection timeout settings for all virtual services are displayed.

show slb connection current [connection_number] [filter]

This command is used to display the established connections, which match the specified filter, from the client to the virtual service then to the real service. Note: This command does not support the display of connections when both virtual service and real service use HTTP/2.

connection_number	Optional. This parameter specifies the number of connections to be displayed. Its value must be an integer
-------------------	--

ranging from 1 to 20,000. The default value is 100.

filter

Optional. This parameter specifies the keyword-based filter. The following keywords are supported:

- **client**: indicates the client's IP address, in the format of "client=x.x.x.x/x".
- **vs**: indicates the virtual service. Its value must be the virtual service name, in the format of "vs=value".
- **rs**: indicates the real service. Its value must be the real service name, in the format of "rs=value".

You can configure multiple keyword-based filters, and they are not subject to the sequence. Each keyword-based filter must be enclosed in double quotes. When multiple keyword-based filters are configured, they must be separated with spaces. For example, "client=10.8.6.12/24" "vs=vip" "rs=rs1".

If this parameter is not specified, the connections established from all clients to virtual services and then to real services are displayed.



Note:

- When the function of reusing server connections for multiple transactions is enabled (see the "**http serverconnreuse** {on|off}" command), this command will not display any HTTP connections established from the client to the virtual service then to the real service because the virtual service and the real service will reuse the existing connections between them.
- For FTP connections, this command displays control connections only, not data connections.

show slb connection live *[filter]*

This command is used to display the newly established connections, which match the specified filter, from the client to the virtual service and then to the real service after this command is executed.

filter

Optional. This parameter specifies the keyword-based filter. The following keywords are supported:

- **client**: indicates the client's IP address, in the format of "client=x.x.x.x/x".
- **vs**: indicates the virtual service. Its value must be the virtual service name, in the format of "vs=value".
- **rs**: indicates the real service. Its value must be the real service name, in the format of "rs=value".

You can configure multiple keyword-based filters, and they are not subject to the sequence. Each keyword-based filter must be enclosed in double quotes. When multiple keyword-based filters are configured, they must be separated with spaces. For example, “client=10.8.6.12/24” “vs=vip” “rs=rs1”.

If this parameter is not specified, all connections newly established from clients to virtual services and then to real services after this command is executed are displayed.



Note:

- If this command is executed in multiple windows of the terminal emulator (such as SecureCRT), the command output will be displayed in only the latest execution window.
- When the function of reusing server connections for multiple transactions is enabled (see the “**http serverconnreuse** {on|off}” command), this command will not display any connections established from the client to the virtual service then to the real service because the virtual service and the real service will reuse the existing connections between them.
- For FTP connections, this command displays control connections only, not data connections.

no connection <protocol> [local_ip] [local_port|icmp_id] [remote_ip] [remote_port]

This command is used to delete a connection with the specified protocol, IP address, and port number filters.

protocol	This parameter specifies the protocol type of the connection to be deleted. Its value must be “tcp”, “udp”, “icmp” or “all” (TCP, ICMP and UDP).
local_ip	Optional. This parameter specifies the local IP address. Its value can be an IPv4 or IPv6 address. The default value is “0.0.0.0”, indicating all local IP addresses.
local_port icmp_id	Optional. This parameter specifies the local port number or ICMP ID. The default value is 0, indicating all local port numbers.
remote_ip	Optional. This parameter specifies the peer IP address. Its value can be an IPv4 or IPv6 address. The default value is “0.0.0.0”, indicating all peer IP addresses.

`remote_port` Optional. This parameter specifies the peer port number.
The default value is 0, indicating all peer port numbers.



Note: Connections used by the Fastlog and SNMP trap modules cannot be deleted using the “**no connection**” command.

show connection [*connection_number*] [*filter_string*]

This command is used to display the number of connections and network connections matching the specified keyword-based filter.

`connection_number` Optional. This parameter specifies the number of connections to be displayed. The value is an integer between 1 and 20,000. The default value is 1000.

`filter_string` Optional. This parameter specifies the keyword-based filter. The following keywords are supported:

- `ip`: indicates the local or peer IP address, in the format of “`ip=x.x.x.x`”.
- `port`: indicates the local or peer port number. Its value must be an integer ranging from 0 to 65,535, in the format of “`port=value`”.
- `proto`: indicates the protocol type. Its value must be “`icmp`”, “`tcp`”, “`udp`”, or “`all`”, in the format of “`proto=value`”.
- `state`: indicates the connection status. Its value must be “`closed`”, “`listen`”, “`syn_sent`”, “`syn_rcvd`”, “`established`”, “`close_wait`”, “`fin_wait1`”, “`closing`”, “`last_ack`”, “`fin_wait2`”, or “`time_wait`”, in the format of “`state=value`”.
- `format`: indicates the format in which the information is displayed. Its value must be “`count`” or “`data`”, in the format of “`format=value`”.

You can configure multiple keyword-based filters, and they are not subject to the sequence. When multiple keyword-based filters are configured, they must be separated with commas and enclosed in double quotes. For example,
“`ip=10.8.1.1,port=80,proto=tcp,state=ESTABLISHED,format=data`”.

The keywords above and their values are case-insensitive.

If this parameter is not specified, all network connections will be displayed.

The following is an example output of this command:

AN(config)#show connection					
Proto	Local Address	Foreign Address	state	expire	Interface

TCP	10.8.1.193:50345	10.3.1.195:50345	ESTABLISHED	59	su
ICMP	10.3.0.2:*	*.*	CLOSED	0	unknown
TCP	10.3.0.2:*	*.*	LISTEN	0	unknown
UDP	10.3.0.2:*	*.*	CLOSED	0	unknown
AN(config)#show connection					
"ip=172.16.77.86,port=80,proto=tcp,state=ESTABLISHED,format=count"					
Host: AN					
Current time : Thu Sep 26 17:32:15 GMT (+0000) 2015					
TCP Total: 0					
ESTABLISHED: 0					
UDP Total: 0					



Note:

- After a Process Control Block (PCB) is created using the “**fwd tcp**”, “**fwd udp**” or “**nat static**” command, the PCB will be kept in the appliance and listen for TCP and UDP traffic. The “expire” field in the command output is displayed as 0.
- This command does not display the network connections of loopback IP addresses.
- If the value of the parameter “icmp_id” is 0, the output result will display “*” after executing the “**show connection**” command.

diameter reliable {on|off}

This command is used to enable or disable Diameter reliable transmission. By default, this function is disabled.

show diameter reliable

This command is used to display the status of the Diameter reliable transmission function.

diameter mark server on

This command is used to enable the function that uses the diameter traffic to detect whether the health status of the real service is down. By default, this function is disabled.

When this function is enabled, the system will detect the health status of the diameter real service according to the diameter response. The two scenarios are as follows:

- After sending the diameter packet, if the system does not receive the diameter response from the real service in 2 seconds, the system will mark the diameter service as down.

- While the system sends the CER message and waits for the CEA message from the real service, if the system receives RESET packets as many times as the consecutive health check failures (The default is 3.), the system will mark the diameter service as down.

diameter mark server off

This command is used to disable the function that uses diameter traffic to detect whether the health status of the real service is down. When this function is disabled, the system will mark the health status of the real service according to the health check result of the diameter real service.

show diameter mark server

This command is used to display the status of the function that uses diameter traffic to detect whether the health status of the real service is down.

Compatibility Check

show slb group compatible real <real_service>

This command is used to display all real service groups compatible with the specified real service. If a real service is compatible with a real service group, this real service can be added to the group as a member.

show slb group compatible virtual <virtual_service>

This command is used to display all real service groups compatible with the specified virtual service. If a virtual service is compatible with a real service group, this virtual service can be associated with this group using the load balancing policy.

show slb real compatible groups <group_name>

This command is used to display all real services compatible with the specified real service group. If a real service group is compatible with a real service, this real service can be added to the group as a member.

show slb virtual compatible groups <group_name>

This command is used to display all virtual services compatible with the specified real service group. If a real service group is compatible with a virtual service, this virtual service can be associated with this group using the load balancing policy.

show slb policy compatible <group_name> <virtual_service>

This command is used to display all load balancing policies used to associate the specified real service group with the specified virtual service.

For example:

```
AN(config)#show slb policy compatible g1 v1
qos clientport
```

```

qos network
qos cookie
qos hostname
qos url
regex
header
default
backup
redirect

```

show slb real compatible healthcheck <real_service_type>

This command is used to display the types of the health checks compatible with the specified type of the real service.

real_service_type	This parameter specifies the type of the real service. Its value must be “http”, “https”, “tcp”, “tcps”, “dns”, “ftp”, “udp”, “ip”, “rtsp”, “sip-tcp”, “sip-udp”, “l2ip”, “l2mac”, “rdp”, “radius-auth”, “radius-acct”, “diameter” or “all”. If the parameter value is “all”, the types of health checks compatible with every type of real service will be displayed.
-------------------	--

For example:

```

AN(config)#show slb real compatible healthcheck all
tcp:
icmp/tcp/http/tcps/none/script-tcp/rtsp-tcp/sip-tcp/script-tcps/https/ldap/mysql-db/mysql-dbs/mssql
l-db/mssql-dbs
udp: icmp/none/dns/script-udp/radius-auth/radius-acct/udp
ftp: icmp/tcp/none/script-tcp
http: icmp/tcp/http/none/script-tcp
https: icmp/tcp/tcps/none/script-tcp/script-tcps/https
tcps: icmp/tcp/tcps/none/script-tcp/script-tcps/https/mysql-dbs/mssql-dbs
dns: icmp/none/dns/script-udp/udp
l2ip: none
l2mac: none
ip:
icmp/tcp/http/tcps/script/carp/none/dns/script-tcp/script-udp/radius-auth/radius-acct/arp/rtsp-tcp/rt
sp-udp/sip-tcp/sip-udp/script-tcps/https/ldap/icmp6/udp/mysql-db/mysql-dbs/mssql-db/mssql-dbs
suptcp: icmp/tcp/none/script-tcp/sip-tcp
sipudp: icmp/none/script-udp/sip-udp/udp
radacct: icmp/none/script-udp/radius-acct/udp
radauth: icmp/none/script-udp/radius-auth/udp
rtsp: icmp/tcp/none/script-tcp/rtsp-tcp
rdp: icmp/tcp/none/script-tcp
diameter: icmp/tcp/none/script-tcp

```

SLB Statistics

The following commands are used to view SLB-related statistics.

When the statistics for a virtual service (by the “**show statistics slb virtual**” command) or real service (by the “**show statistics slb real**” command) are displayed on a user terminal, the bandwidth expression follows these principles:

- When the bandwidth is smaller than or equal to 16 Kbps, it is expressed in bps.
- When the bandwidth is greater than 16 Kbps and smaller than or equal to 16 Mbps, it is expressed in Kbps.
- When the bandwidth is greater than 16 Mbps, it is expressed in Mbps.

Real Service Statistics

show statistics slb real

{dns|dnstcp|ftp|http|https|ip|l2ip|l2mac|rdp|rtsp|siptcp|sipudp|tcp|tcps|udp|radauth|radacct|diameter|tuxedo} [real_service]

This command is used to display the statistics of the specified real service of the specific type. If the “real_service” parameter is not specified, the statistics of all real services of the specific type will be displayed.



Note: If the “ip|domain” parameter in the commands “**slb real http**”, “**slb real https**”, “**slb real ftp**”, “**slb real tcp**”, “**slb real tcps**” and “**slb real ip**” set to “domain”, the IP address corresponding to the domain name of the real service will be included in the displayed real services.

show statistics slb real all

This command is used to display the statistics of all real services.



Note: If the “ip|domain” parameter in the commands “**slb real http**”, “**slb real https**”, “**slb real ftp**”, “**slb real tcp**”, “**slb real tcps**” and “**slb real ip**” set to “domain”, the IP address corresponding to the domain name of the real service will be included in the displayed real services.

clear statistics slb real

{dns|dnstcp|ftp|http|https|ip|l2ip|l2mac|rdp|rtsp|siptcp|sipudp|tcp|tcps|udp|radauth|radacct|diameter|tuxedo} [real_service]

This command is used to clear the statistics of the specified real service of the specific type. If the “real_service” parameter is not specified, the statistics of all real services of the specific type will be cleared.

clear statistics slb real all

This command is used to clear the statistics of all real services.

Group Statistics

show statistics slb group [group_name]

This command is used to display the statistics of the specified real service group. If the “group_name” parameter is not specified, the statistics of all real service groups will be displayed.

clear statistics slb group [group_name]

This command is used to clear the statistics of the specified real service group. If the “group_name” parameter is not specified, the statistics of all real service groups will be cleared.

Virtual Service Statistics**show statistics slb virtual**

**{dns|dnstcp|ftp|https|http|https|ip|l2ip|rdp|rtsp|siptcp|sipudp|tcp|tcps|udp|
radauth|radacct|diameter|tuxedo} [virtual_service]**

This command is used to display the statistics of the specified virtual service of the specific type. If the “virtual_service” parameter is not specified, the statistics of all virtual services of the specific type will be displayed.

show statistics slb virtual all

This command is used to display the statistics of all virtual services.

clear statistics slb virtual

**{dns|dnstcp|ftp|https|http|https|ip|l2ip|rdp|rtsp|siptcp|sipudp|tcp|tcps|udp|
radauth|radacct|diameter|tuxedo} [virtual_service]**

This command is used to clear the statistics of the specified virtual service of the specific type. If the “virtual_service” parameter is not specified, the statistics of all virtual services of the specific type will be cleared.

clear statistics slb virtual all

This command is used to clear the statistics of all virtual services.

show statistics slb vlink [vlink_name]

This command is used to display the statistics of the specified vlink. If the “vlink_name” parameter is not specified, the statistics of all vlinks will be displayed.

clear statistics slb vlink [vlink_name]

This command is used to clear the statistics of the specified vlink. If the “vlink_name” parameter is not specified, the statistics of all vlinks will be displayed.

show statistics slb policy virtual [virtual_service|vlink_name]

This command is used to display the statistics of all policies related to the specified virtual service and vlink. If the “virtual_service” or “vlink_name” parameter is not

specified, the statistics of all policies related to all virtual services and vlinks will be displayed.

show statistics slb summary [*virtual_service*|*vlink_name*]

This command is used to display the statistics (including the SSL statistics) related to the specified virtual service or vlink. If the “virtual_service” or “vlink_name” parameter is not specified, the statistics (including the SSL statistics) related to all virtual services or vlinks will be displayed.

For example:

```
AN(config)#show statistics slb summary
http virtual service "v1" (10.8.1.138 80)
    Max Connection Count:      0
    Current Connection Count:  1
    Total Connection Count:    117
    Total Bytes In:            1462859
    Total Bytes Out:           94272
    Total Packets In:          36317
    Total Packets Out:         466
    qos clientport hits : 0
    qos network hits : 0
    persistent url hits : 0
    rcookie hits : 0
    icookie hits : 0
    persistent cookie hits : 0
    qos cookie hits : 0
    qos hostname hits : 0
    qos url hits : 0
    regex hits : 0
    header hits : 0
    hashurl hits : 0
    redirect hits : 0
    raduname hits : 0
    radsid hits : 0
    default hits : 117
        default policy for http virtual service "v1" has been matched 117 times
            Group Name          Method      Hits
            g1                  rr          117

            Per Member Statistics
            Real Service          Hits
            r1                    60
            r2                    57

            Real service r1 10.8.1.131 80 UP ACTIVE
                Main health check: 10.8.1.131 80 tcp UP
                Max Conn Count:      1000
                Current Connection Count:  3
                Outstanding Request Count: 0
                Total Hits:           60
                Total Bytes In:       47850
                Total Bytes Out:      16780
                Total Packets In:     216
```


Total Packets Out:	223
Average Response time:	1.757 ms
Real service r2 10.8.1.131 80 UP ACTIVE	
Main health check:	10.8.1.131 80 tcp UP
Max Conn Count:	1000
Current Connection Count:	1
Outstanding Request Count:	1
Total Hits:	57
Total Bytes In:	44753
Total Bytes Out:	15907
Total Packets In:	204
Total Packets Out:	211
Average Response time:	1.765 ms
static hits :	0
backup hits :	0
cache hits :	0
vlink "vlink1"	
qos clientport hits :	0
qos network hits :	0
persistent url hits :	0
rcookie hits :	0
icookie hits :	0
persistent cookie hits :	0
qos cookie hits :	0
qos hostname hits :	0
qos url hits :	0
regex hits :	0
header hits :	0
hashurl hits :	0
redirect hits :	0
raduname hits :	0
radsid hits :	0
default hits :	0
backup hits :	0



Note: If the “policy hits” (such as “default hits”) is not 0, the related information including virtual services, groups and real services will be displayed.

Load Balancing Policy Statistics

show statistics slb policy static *[virtual_service]*

This command is used to display the statistics of the Static policy for the specified virtual service. If the “virtual_service” parameter is not specified, the statistics of Static policies for all virtual services will be displayed.

show statistics slb policy default *[virtual_service]*

This command is used to display the statistics of the Default policy for the specified virtual service. If the “virtual_service” parameter is not specified, the statistics of Default policies for all virtual services will be displayed.

show statistics slb policy backup [virtual_service]

This command is used to display the statistics of the Backup policy for the specified virtual service. If the “virtual_service” parameter is not specified, the statistics of Backup policies for all virtual services will be displayed.

show statistics slb policy doh [virtual_service]

This command is used to display the statistics of the DoH policy for the specified virtual service. If the “virtual_service” parameter is not specified, the statistics of DoH policies for all virtual services will be displayed.

clear statistics slb policy doh [virtual_service]

This command is used to clear the statistics of the DoH policy for the specified virtual service. If the “virtual_service” parameter is not specified, the statistics of DoH policies for all virtual services will be cleared.

The following commands are used to display the statistics of the specified load balancing policy of the specific type. If the “policy_name” parameter is not specified, the statistics of all load balancing policies of the specific type will be displayed.

show statistics slb policy header [policy_name]**show statistics slb policy redirect [policy_name]****show statistics slb policy persistent url [policy_name]****show statistics slb policy persistent cookie [policy_name]****show statistics slb policy icookie [policy_name]****show statistics slb policy rcookie [policy_name]****show statistics slb policy qos body [policy_name]****show statistics slb policy qos url [policy_name]****show statistics slb policy qos hostname [policy_name]****show statistics slb policy qos cookie [policy_name]****show statistics slb policy qos network [policy_name]****show statistics slb policy qos clientport [policy_name]****show statistics slb policy qos dnsdomain [policy_name]****show statistics slb policy qos dnsqtype [policy_name]****show statistics slb policy qos diameter [policy_name]**

show statistics slb policy qos diameterappid *[policy_name]*

show statistics slb policy regex *[policy_name]*

show statistics slb policy hashurl *[policy_name]*

show statistics slb policy radsid *[policy_name]*

show statistics slb policy raduname *[policy_name]*

show statistics slb policy filetype *[policy_name]*

show statistics slb policy dnssec *[policy_name]*

show statistics slb policy sipdomain *[policy_name]*

show statistics slb policy sipcall *[policy_name]*

clear statistics slb policy static *[virtual_service]*

This command is used to clear the statistics of the Static policy for the specified virtual service. If the “virtual_service” parameter is not specified, the statistics of Static policies for all virtual services will be cleared.

clear statistics slb policy default *[virtual_service]*

This command is used to clear the statistics of the Default policy for the specified virtual service. If the “virtual_service” parameter is not specified, the statistics of Default policies for all virtual services will be cleared.

clear statistics slb policy backup *[virtual_service]*

This command is used to clear the statistics of the Backup policy for the specified virtual service. If the “virtual_service” parameter is not specified, the statistics of Backup policies for all virtual services will be cleared.

The following commands are used to clear the statistics of the specified load balancing policy of the specified type. If the “policy_name” parameter is not specified, the statistics of all load balancing policies of the specified type will be cleared.

clear statistics slb policy header *[policy_name]*

clear statistics slb policy redirect *[policy_name]*

clear statistics slb policy persistent url *[policy_name]*

clear statistics slb policy persistent cookie *[policy_name]*

clear statistics slb policy icookie *[policy_name]*

```

clear statistics slb policy rcookie [policy_name]
clear statistics slb policy qos body [policy_name]
clear statistics slb policy qos url [policy_name]
clear statistics slb policy qos hostname [policy_name]
clear statistics slb policy qos cookie [policy_name]
clear statistics slb policy qos network [policy_name]
clear statistics slb policy qos clientport [policy_name]
clear statistics slb policy qos dnsdomain [policy_name]
clear statistics slb policy qos dnsqtype [policy_name]
clear statistics slb policy qos diameter [policy_name]
clear statistics slb policy qos diameterappid [policy_name]
clear statistics slb policy regex [policy_name]
clear statistics slb policy hashurl [policy_name]
clear statistics slb policy radsid [policy_name]
clear statistics slb policy raduname [policy_name]
clear statistics slb policy filetype [policy_name]
clear statistics slb policy dnssec [policy_name]
clear statistics slb policy sipdomain [policy_name]
clear statistics slb policy sipcall [policy_name]

```

Health Check Statistics

```
show statistics health [real_service]
```

This command is used to display the statistics of health checks (including the basic health check, additional health check and group health check) of the specified real service. If the “real_service” parameter is not specified, the statistics of health checks of all real services will be displayed.

For example:

```

AN(config)#show statistics health
Health check active
update time: Wed May  9 11:37:38 2019
server name: r1

```

```

server ip: 10.3.0.20
server port: 80
type of health check: tcp
health: UP
latest health status: UP
latest health status cause: Tcp connection successful
latest UP: Wed May 9 11:30:22 2019
latest DOWN: Wed May 9 11:27:52 2019
number of times server is up: 40
number of times server is down: 10
connections attempted: 50
connections succeeded: 40
connections failed not due to time out: 0
connections failed due to time out: 10

```

```

=====RS Additional Health Check(s):=====
=====Slb Group Health Check(s):=====

```



Note: The “latest UP” entry displays the last time the real server status turned from Down to Up, while the “latest DOWN” entry displays the last time the real server status turned from Up to Down. If the real server status has never turned Down, the “latest UP” entry will not display any data.

clear statistics health

This command is used to clear the statistics of health checks (including the basic health check, additional health check and group health check) of all real services.

show statistics passivehealth real http *[real_service]*

This command is used to display the statistics of the HTTP passive health check applied to the specified real service. If the “real_service” parameter is not specified, the statistics of the HTTP passive health check applied to all real services will be displayed.

show statistics passivehealth real tcp *[real_service]*

This command is used to display the statistics of the TCP passive health check applied to the specified real service. If the “real_service” parameter is not specified, the statistics of the TCP passive health check applied to all real services will be displayed.

show statistics passivehealth group http *[group_name]*

This command is used to display the statistics of the HTTP passive health check applied to the specified group. If the “group_name” parameter is not specified, the statistics of the HTTP passive health check applied to all groups will be displayed.

show statistics passivehealth group tcp *[group_name]*

This command is used to display the statistics of the TCP passive health check applied to the specified group. If the “group_name” parameter is not specified, the statistics of the TCP passive health check applied to all groups will be displayed.

clear statistics passivehealth

This command is used to display the statistics related to the passive health check function.

Other Statistics**show statistics slb proxyip [group_name]**

This command is used to display the statistics of the proxy IP pool of the specified real service group. If the “group_name” parameter is not specified, the statistics of proxy IP pools of all real service groups will be displayed.

clear statistics slb proxyip [group_name]

This command is used to clear the statistics of the proxy IP pool of the specified real service group. If the “group_name” parameter is not specified, the statistics of proxy IP pools of all real service groups will be cleared.

show statistics sip nat

This command is used to display the statistics of all SIP NAT rules.

clear statistics sip nat

This command is used to clear the statistics of all SIP NAT rules.

show statistics slb all

This command is used to display the statistics of all real services, groups, policies and virtual services.

clear statistics slb all

This command is used to clear the statistics of all real services, groups, policies and virtual services.

show statistics slb global

This command is used to display SLB global statistics.

SLB Summary

SLB Type	Priority (1 is highest)	Virtual Service	Real Service	Health check	Scenarios
Layer 7 HTTP/HTTPS	2	IP + Port + proto (HTTP, HTTPS)	IP + Port + proto (HTTP, HTTPS)	None HTTP HTTPS TCP TCPS ICMP Additional Script	1. Balance traffic according to application protocol headers. For example, HTTP headers. 2. Cache feature is needed.

Chapter 8 Server Load Balancing (SLB)

SLB Type	Priority (1 is highest)	Virtual Service	Real Service	Health check	Scenarios
Layer 7 DNS/ DNSTCP	2	IP + Port + proto (DNS)	IP + Port + proto (DNS)	None DNS DNSTCP ICMP Additional Script	DNS requests DNS cache feature can be applied for better performance.
Layer 7 FTP	2	IP + Port + proto (FTP)	IP + Port + proto (FTP)	None TCP ICMP Additional Script	FTP traffic.
Layer 7 SIP	2	IP + Port + proto (SIP-TCP, SIP-UDP)	IP + Port + proto (SIP-TCP, SIP-UDP)	None TCP TCPS ICMP Additional Script SIP-TCP SIP-UDP	Balance VOIP traffic.
Layer 7 RTSP	2	IP + Port + proto (RTSP)	IP + Port + proto (RTSP)	None TCP ICMP Additional Script RTSP-TCP	Balance real time media traffic.
Layer 4	2	IP + port	IP + Port	None TCP TCPS ICMP Additional Script	1. Balance traffic according to TCP/UDP headers. 2. TCP port or UDP port is specified to determine a particular service.
Port range (for Layer 7)	3	Layer 7 VS + Port range	Layer 7 RS Layer 7 RS (0 port)	Non-zero port RS: Layer 7 health check Zero port RS: ICMP Additional	In addition to Layer 7 SLB, cross-port and dynamic port ore balance is supported.
Port range (for Layer 4)	3	Layer 4 VS + Port range	Layer 4 RS Layer 4 RS (0 port)	Non-zero port RS: Layer 4 health check Zero port RS: ICMP Additional	In addition to Layer 4 SLB, cross-port and dynamic port application traffic balance is supported.
Layer 3	4	IP	IP	None ICMP Additional	In addition to port range SLB, cross-protocol

Chapter 8 Server Load Balancing (SLB)

SLB Type	Priority (1 is highest)	Virtual Service	Real Service	Health check	Scenarios
					application traffic balance is supported Currently, only TCP and UDP protocol are supported.
Layer 2	1	IP + port ranges	IP, MAC	ARP Additional (only ICMP)	1. The backend real services don't have usable IP addresses so that the traffic can't be balanced according to IP addresses; 2. The backend real services are not the destination of the input traffic (for example, virus scanners check every packet before forwarding it to the real destination).

		Basic Methods				IP-based Methods				Header/Request-based Methods												Cookie-based Methods				General	Others									
		gr	rr	lc	amp	sr	fb	pi	hi	hip	chi	hh	ph	chh	pto	pu	hq	slid	spcid	sluid	slpcid	sluidps	diametrsd	re	ec	ic	pc	hc	persistence	tuxado	rdprt	radchu	radcha	radpsn	radchi	
Basic Policies	Static	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	
	Default	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	
	Backup	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	
	Persistent URL	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	
	Hash URL	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	
Persistent Policies	Persistent Cookie	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×
	Reverse Cookie	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	
	Insert Cookie	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	
	Radname	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	
	Raduid	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	
	SIP Domain	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	
	SIP Call	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	
	FileType	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	
	DNSSEC	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×
	QoS Policies	QoS Cookie	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×
QoS Hostname		×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×
QoS URL		×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×
QoS Network		×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×
QoS Clientport		×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×
Rapex		×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×
Header		×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	
QoS Deadomain		×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×
QoS Body		×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×
QoS Diameter		×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×
QoS Diametertrappid	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	
Others	Redirect	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	
		gr-Global Round Robin rr-Round Robin lc-Least Connections amp-SNIP sr-Shortest Response fb-Least Bandwidth				pi-Persistent IP hi-Hash IP hip-Hash IP and Port chh-Consistent Hash IP				hh-Hash Header ph-Persistent Hostname chh-Consistent Hash Header pto-Persistent TCP Option pu-Persistent URL hq-Hash Query				slid-Persistent SIP SID spcid-SIP Call ID sluid-SIP User ID slpcid-SIP CallID Persistence Session sluidps-SIP UserID Persistence Session diametrsd-Persistent Diameter SID				re-Reverse Cookie ec-Embed Cookie ic-Insert Cookie pc-Persistent Cookie hc-Hash Cookie persistence-Persistence					tuxado-Tuxado rdprt-RDP Routing Token radchu-Consistent Hash RADU/S Username radcha-Consistent Hash RADU/S Session ID rdpsn-RADU/S Persistence Session by Username radchi-RADU/S Consistent Hash IP													

- ★ The policy can collaborate with the group method.

- ★ The policy can collaborate with the group method.
- ✗ The policy cannot collaborate with the group method.

Chapter 9 Application Security Orchestrator

Traffic Listener

orchestrator listener name <listener_name> <deployment_mode>

This command is used to create a traffic listener.

listener_name This parameter specifies the name of the traffic listener. Its value must be a case-sensitive string of 1 to 128 characters. Numbers, letters and special characters are supported. When the parameter value contains special characters or only numbers, it must be enclosed in double quotes.

Applicable special characters are: “@”, “!”, “~”, “_”, “:”, “-”, “<”, “>”, “.”.

Each traffic listener’s name must be unique.

deployment_mode This parameter specifies the deployment mode of the traffic listener. Its value must be:

- inb_proxy: inbound proxy mode
- inb_transparent: inbound transparent mode
- outb_transparent: outbound transparent mode

no orchestrator listener name <listener_name>

This command is used to delete the specified traffic listener and its configurations.

show orchestrator listener name [listener_name]

This command is used to display the specified traffic listener. If the parameter “listener_name” is not specified, the system will display all traffic listeners.

clear orchestrator listener name

This command is used to clear all traffic listeners and their configurations.

orchestrator listener interface uplink <listener_name>
{system_ifname|bond_ifname|vlan_ifname}

This command is used to set certain system interfaces, bond interfaces or VLAN interfaces as the uplink interfaces for the specified traffic listener.

listener_name This parameter specifies the name of an existing traffic listener.

system_ifname	This parameter specifies the name of a system interface, which can be customized with the “interface name” command. By default, the system interface name is port1, port2, port3, port4....
bond_ifname	This parameter specifies the bond interface name. By default, the bond interface name is bond1, bond2, bond3, bond4....
vlan_ifname	This parameter specifies the VLAN interface name.

no orchestrator listener interface uplink <listener_name>
{system_ifname|bond_ifname|vlan_ifname}

This command is used to stop using certain system interfaces, bond interfaces or VLAN interfaces as the uplink interfaces for the specified traffic listener.

show orchestrator listener interface uplink [listener_name]

This command is used to display the uplink interfaces of the specified traffic listener. If the parameter “listener_name” is not specified, the system will display the uplink interfaces of all traffic listeners.

clear orchestrator listener interface uplink [listener_name]

This command is used to clear the uplink interface configuration of the specified traffic listener. If the parameter “listener_name” is not specified, the uplink interface configuration of all traffic listeners will be cleared.

orchestrator listener interface downlink <listener_name>
{system_ifname|bond_ifname|vlan_ifname}

This command is used to set certain system interfaces, bond interfaces or VLAN interfaces as the downlink interfaces for the specified traffic listener.

listener_name	This parameter specifies the name of an existing traffic listener.
system_ifname	This parameter specifies the name of a system interface, which can be customized with the “interface name” command. By default, the system interface name is port1, port2, port3, port4...

bond_ifname	This parameter specifies the bond interface name. By default, the bond interface name is bond1, bond2, bond3, bond4...
vlan_ifname	This parameter specifies the VLAN interface name.

no orchestrator listener interface downlink <listener_name>
{system_ifname|bond_ifname|vlan_ifname}

This command is used to stop using certain system interfaces, bond interfaces or VLAN interfaces as the downlink interfaces for the specified traffic listener.

show orchestrator listener interface downlink [/listener_name]

This command is used to display the downlink interfaces of the specified traffic listener. If the parameter “listener_name” is not specified, the system will display the downlink interfaces of all traffic listeners.

clear orchestrator listener interface downlink [/listener_name]

This command is used to clear the downlink interface configuration of the specified traffic listener. If the parameter “listener_name” is not specified, the downlink interface configuration of all traffic listeners will be cleared.

Security Device

orchestrator device l2 <security_device> <to_interface> <from_interface>

This command is used to create a L2 security device. For the same physical L2 device, connecting it to an orchestrator via different physical or logical interfaces creates multiple L2 security devices.

This command is also used to update the value of the parameters “to_interface” and “from_interface” for an existing L2 security device.

security_device	This parameter specifies the name of the L2 security device. Its value must be a case-sensitive string of 1 to 128 characters. Numbers, letters and special characters are supported. When the parameter value contains special characters or only numbers, it must be enclosed in double quotes. Applicable special characters are: “@”, “!”, “~”, “_”, “:”, “-”, “<”, “>”, “.”. Each L2 security device’s name must be unique and different from that of the security devices of other types.
-----------------	--

to_interface	This parameter specifies the name of the interface that sends requests to the L2 security device. Its value must be the name of a system interface, bond interface or VLAN interface.
from_interface	This parameter specifies the name of the interface that receives requests from the L2 security appliance. Its value can be a system interface, an aggregate interface, or a VLAN interface.

Note: The parameters “from_interface” and “to_interface” cannot be set to the same physical interface or two logical interfaces that bound to the same physical interface because two VLAN interfaces binding on the same physical interface have the same MAC address.

no orchestrator device l2 <security_device>

This command is used to delete the specified L2 security device.

show orchestrator device l2 [security_device]

This command is used to display the specified L2 security device. If the parameter “security_device” is not specified, the system will display all L2 security devices.

clear orchestrator device l2

This command is used to clear all L2 security devices.

orchestrator device l3 <security_device> <to_ip> <from_ip>

This command is used to create a L3 security device. For the same physical L3 device, connecting it to an orchestrator via different physical or logical interfaces creates multiple L3 security devices.

This command is also used to update the value of the parameters “to_ip” and “from_ip” for an existing L3 security device.

security_device	This parameter specifies the name of the L3 security device. Its value must be a case-sensitive string of 1 to 128 characters. Numbers, letters and special characters are supported. When the parameter value contains special characters or only numbers, it must be enclosed in double quotes.
-----------------	---

Applicable special characters are: “@”, “!”, “~”, “_”, “:”, “-”, “<”, “>”, “.”.

Each L3 security device’s name must be unique and

different from that of the security devices of other types.

to_ip This parameter specifies the IP address of the L3 security device interface that receives requests. Its value must be an IPv4 or IPv6 address.

from_ip This parameter specifies the IP address of the L3 security device interface that sends requests. Its value must be an IPv4 or IPv6 address.

no orchestrator device l3 <security_device>

This command is used to delete the specified L3 security device.

show orchestrator device l3 [security_device]

This command is used to display the specified L3 security device. If the parameter “security_device” is not specified, the system will display all L3 security devices.

clear orchestrator device l3

This command is used to clear all L3 security devices.

orchestrator device l4 <security_device> <from_ip>

This command is used to create a L4 security device.

This command is also used to update the value of the “from_ip” parameter for an existing L4 security device.

security_device This parameter specifies the name of the L4 security device. Its value must be the name of an existing TCP-type SLB real service.

from_ip This parameter specifies the IP address of the L4 security device interface that sends requests. Its value must be an IPv4 or IPv6 address.

no orchestrator device l4 <security_device>

This command is used to delete the specified L4 security device.

show orchestrator device l4 [security_device]

This command is used to display the specified L4 security device. If the parameter “security_device” is not specified, the system will display all L4 security devices.

clear orchestrator device l4

This command is used to clear all L4 security devices.

Security Service

orchestrator service l2 <security_service> [down_action] [method]

This command is used to create a L2 security service.

This command is also used to update the value of the parameters “down_action” and “method” for an L2 security service.

security_service

This parameter specifies the name of the L2 security service. Its value must be a case-sensitive string of 1 to 128 characters. Numbers, letters and special characters are supported. When the parameter value contains special characters or only numbers, it must be enclosed in double quotes.

Applicable special characters are: “@”, “!”, “~”, “_”, “:”, “-”, “<”, “>”, “.”.

Each L2 security service’s name must be unique and different from that of the security services of other types.

down_action

Optional. This parameter specifies what action the system takes when health check results of all security devices for this service are down. Its value must be:

- bypass
- deny

The default value is “bypass”.

method

Optional. This parameter specifies the for load balancing algorithm applied to the security devices of the service.

- pi : Persistent IP. The first method is rr.
- chi : Consistent Hash IP
- hip : Hash (IP+Port)

The default value is pi.

no orchestrator service l2 <security_service>

This command is used to delete the specified L2 security service.

show orchestrator service l2 [security_service]

This command is used to display the specified L2 security service. If the parameter “security_service” is not specified, the system will display all L2 security services.

clear orchestrator service l2

This command is used to clear all L2 security services.

orchestrator service l3 <security_service> [down_action] [method]

This command is used to create a L3 security service.

This command is also used to update the value of the parameters “down_action” and “method” for an L3 security service.

security_service

This parameter specifies the name of the L3 security service. Its value must be a case-sensitive string of 1 to 128 characters. Numbers, letters and special characters are supported. When the parameter value contains special characters or only numbers, it must be enclosed in double quotes.

Applicable special characters are: “@”, “!”, “~”, “_”, “:”, “-”, “<”, “>”, “.”.

Each L3 security service’s name must be unique and different from that of the security services of other types.

down_action

Optional. This parameter specifies what action the system takes when health check results of all security devices for this service are down. Its value must be:

- bypass
- deny

The default value is “bypass”.

method

Optional. This parameter specifies the load balancing algorithm applied to the security devices of the service.

- pi : Persistent IP. The first choice method is rr.
- chi : Consistent Hash IP
- hip : Hash (IP+Port)

The default value is pi.

no orchestrator service l3 <security_service>

This command is used to delete the specified L3 security service.

show orchestrator service l3 [security_service]

This command is used to display the specified L3 security service. If the parameter “security_service” is not specified, the system will display all L3 security services.

clear orchestrator service l3

This command is used to clear all L3 security services.

orchestrator service member <security_service> <security_device> [weight]

This command is used to add the specified security device to the specified security service. At most 64 security devices can be allocated to each security service; and every security device can only be added to one security service. An L2 security device can only be added to the L2 security service, and an L3 security device to the L3 security service.

This command is also used to update the value of the “weight” parameter for the specified security device.

security_service	This parameter specifies the name of an existing L2 or L3 security service.
security_device	This parameter specifies the name of an existing L2 or L3 security device.
weight	Optional. This parameter specifies the weight of the security device. Its value must be an integer ranging from 1 to 4,294,967,295. The value needs to be set when the security service uses the pi algorithm. The default value is 1.

no orchestrator service member <security_service> <security_device>

This command is used to delete the specified L2 security device from the specified service.

show orchestrator service member [security_service]

This command is used to display all security devices of the specified security service. If the parameter “security_service” is not specified, the system will display security devices for all security services.

clear orchestrator service member [security_service]

This command is used to delete all security devices of the specified security service. If the parameter “security_service” is not specified, the system will delete security devices for all security services.

orchestrator service flush <security_service>

This command is used to clear the hash table used by the specified security service to maintain the session persistence. After the hash table is cleared, all the established session persistence of the security service will be cleared.

security_service	This parameter specifies the name of an existing L2 or L3 security service with the pi method.
------------------	--

orchestrator service l4 <security_service>

This command is used to create a L4 security service. The traffic sent from the listener to the L4 security service will be processed according to the related TCP-type SLB virtual service.

security_service	This parameter specifies the name of the L4 security service. Its value must be an existing TCP-type SLB virtual service.
------------------	---

no orchestrator service l4 <security_service>

This command is used to delete the specified L4 security service.

show orchestrator service l4 [security_service]

This command is used to display the specified L4 security service. If the parameter “security_service” is not specified, the system will display all L4 security services.

clear orchestrator service l4

This command is used to clear all L4 security services.

orchestrator service tap <security_service> <output_interface> <dst_mac>

This command is used to create a TAP security service.

This command is also used to update the value of the parameters “output_interface” and “real_mac” for a TAP security service.

security_service	This parameter specifies the name of the TAP security service. Its value must be a case-sensitive string of 1 to 128 characters. Numbers, letters and special characters are supported. When the parameter value contains special characters or only numbers, it must be enclosed in double quotes.
------------------	---

Applicable special characters are: “@”, “!”, “~”, “_”, “:”, “-”, “<”, “>”, “.”.

Each TAP security service’s name must be unique and

different from that of the security services of other types.

output_interface	This parameter specifies the name of the interface that forwards packets. Its value must be the name of a system interface, VLAN interface or bond interface.
dst_mac	This parameter specifies the destination MAC address that receives the packets forwarded by the TAP security device. The value should be in the format of “xx:xx:xx:xx:xx:xx”.

no orchestrator service tap <security_service>

This command is used to delete the specified TAP security service.

show orchestrator service tap [security_service]

This command is used to display the specified TAP security service. When the parameter “security_service” is not specified, all TAP security services will be displayed.

clear orchestrator service tap

This command is used to clear all TAP security services.

orchestrator service enable <security_service>

This command is used to enable the specified L2, L3 or TAP security service. By default, L2, L3 or TAP security services are enabled.

security_service	This parameter specifies the name of an existing L2, L3 or TAP security service.
------------------	--

orchestrator service disable <security_service>

This command is used to disable the specified L2, L3 or TAP security service.

security_service	This parameter specifies the name of an existing L2, L3 or TAP security service.
------------------	--



Note: When L2/L3 security services are disabled, the system will process L2/L3 traffic according to the “down_action” parameter of security services.

show orchestrator service enable

This command is used to display the enabled L2, L3 and TAP security services.

show orchestrator service disable

This command is used to display the disabled L2, L3 and TAP security services.

Security Service Chain

orchestrator chain name <security_service_chain>

This command is used to create a security service chain.

security_service_chain

This parameter specifies the name of the security service chain. Its value must be a case-sensitive string of 1 to 128 characters. Numbers, letters and special characters are supported. When the parameter value contains special characters or only numbers, it must be enclosed in double quotes.

Applicable special characters are: “@”, “!”, “~”, “_”, “.”, “_”, “<”, “>”, “.”.

The name of each security service chain must be unique.

no orchestrator chain name <security_service_chain>

This command is used to delete the specified security service chain.

show orchestrator chain name

This command is used to display all security service chains.

clear orchestrator chain name

This command is used to clear all security service chains.

orchestrator chain member <security_service_chain> <security_service>
<order>

This command is used to add the specified security service to the security service chain. At most, each security chain can accommodate 8 security services.



Note:

- If the security service chain has no security service, the traffic sent to the security service chain will be directly forwarded by the router.
- The same L4 security service cannot be added to different security service chains.

This command is also used to update the “order” parameter of the specified security service of the security service chain.

<code>security_service_chain</code>	This parameter specifies the name of an existing security service chain.
<code>security_service</code>	This parameter specifies the name of an existing security service.
<code>order</code>	This parameter specifies the checking priority of the specific security service in the security service chain. Its value must be an integer ranging from 0 to 65,535. The smaller the value is, the higher the priority is. The priority of the security services of the same security service chain must be different.

no orchestrator chain member *<security_service_chain> <security_service>*

This command is used to delete a security service from the specified security chain.

show orchestrator chain member *[security_service_chain]*

This command is used to display all security services of the specified security service chain. When the parameter “security_service_chain” is not specified, security services of all security service chains will be displayed.

clear orchestrator chain member *[security_service_chain]*

This command is used to clear all security services of the specified security service chain. When the parameter “security_service_chain” is not specified, security services of all security service chains will be cleared.

Health Check

orchestrator health on

This command is used to enable the health check function for security orchestration. By default, the health check function is enabled. The health check function is applicable for L2/L3 security devices. Health check for L4 security devices is conducted with the associated SLB virtual service for TCP traffic.

orchestrator health off

This command is used to disable the health check function for security orchestration.

orchestrator health service check *<hc_name> <security_service>
<hc_type> <port> [hc_mode] [interval] [timeout] [hc_up] [hc_down]*

This command is used to configure a health check for the specified L2/L3 security service. A maximum of 6 health checks can be provided for each security service.

This command is used to update the value of the health check parameters for security services.

hc_name	This parameter specifies the health check name. Its value must be a case-sensitive string of 1 to 128 characters. Numbers, letters and special characters are supported. When the parameter value contains special characters or only numbers, it must be enclosed in double quotes. Applicable special characters are: “@”, “!”, “~”, “_”, “:”, “-”, “<”, “>”, “.”. The name of each health check must be unique.
security_service	This parameter specifies the name of an existing L2/L3 security service.
hc_type	This parameter specifies the health check type. Its value must be “icmp”, “tcp”, “half-tcp”, “http” or “https”.
port	This parameter specifies the destination port for health check. Its value must be an integer ranging from 0 to 65,535. When the value of “hc_type” is “icmp”, the value of “port” must be “0”.
hc_mode	Optional. This parameter specifies the health check mode. Its value must be: • “bidirect”: Both L2 and L3 security services support this mode. In this mode, the value of “hc_type” must be “tcp”. • “unidirect”: Only L3 security service supports this mode. In this mode, the value of “hc_type” must be “icmp”, “tcp”, “half-tcp”, “http” or “https”. Note: In the unidirect health check mode, the system conducts health check for both uplink and downlink when the value of “hc_type” is “icmp”; and the system conducts health check only for the uplink when the value of “hc_type” is “tcp”, “half-tcp”, “http” or “https”. • empty: When the “hc_type” is “tcp”, the default value is “bidirect”. When the “hc_type” is “icmp”, “half-tcp”, “http” or “https”, the default value is “unidirect”. By default, the value is empty.
interval	Optional. This parameter, measured in seconds, specifies the health check interval for the security device. Its value must be an integer ranging from 1 to 10,000. The default value is 10.

timeout	Optional. This parameter, measured in seconds, specifies the maximum interval allowed from sending a health check request to receiving the response. Its value must be an integer ranging from 1 to 10,000. The default value is 5. Note: The value of “timeout” is no larger than that of “interval”.
hc_up	Optional. This parameter specifies for how many consecutive times the security device should pass the health check before it is determined as in the “Up” status. Its value must be an integer ranging from 1 to 255. The default value is 3.
hc_down	Optional. This parameter specifies for how many consecutive times the security device should fail the health check before it is determined as in the “Down” status. Its value must be an integer ranging from 1 to 255. The default value is 3.

no orchestrator health service check <hc_name>

This command is used to delete the specified health check.

show orchestrator health service check [security_service]

This command is used to display all health checks configured for the specified L2/L3 security service. If the parameter “security_service” is not specified, the system will display health checks configured for all L2/L3 security services.

clear orchestrator health service check [security_service]

This command is used to clear all health checks configured for the specified L2/L3 security service. If the parameter “security_service” is not specified, the system will clear health checks configured for all L2/L3 security services.

orchestrator health device check <hc_name> <security_device> <hc_type> <ip> <port> [hc_mode] [interval] [timeout] [hc_up] [hc_down]

This command is used to configure an additional health check for the specified L3 security device. The maximum number of additional health checks supported by each security device depends on the system memory size. The maximum number of additional health checks supported by all security devices is the same as that of each security device, as shown in the following table.

This command is also used to change the values of the additional health check parameters for the specified security device.

Specification	Memory			
	<4G	≥4G	≥32G	≥128G

Specification	Memory			
The maximum number of additional health checks supported by each security device	0	64	128	256

hc_name	This parameter specifies the health check name. Its value must be a case-sensitive string of 1 to 128 characters. Numbers, letters and special characters are supported. When the parameter value contains special characters or only numbers, it must be enclosed in double quotes. Applicable special characters are: “@”, “!”, “~”, “_”, “:”, “_”, “<”, “>”, “.”. The name of each health check must be unique.
security_service	This parameter specifies the name of an existing L3 security device.
hc_type	This parameter specifies the health check type. Its value must be “icmp”, “tcp”, “half-tcp”, “http” or “https”.
ip	This parameter specifies the destination IP for health check. Its value must be an IPv4 or IPv6 address on the L3 security device.
port	This parameter specifies the destination port for health check. Its value must be an integer ranging from 0 to 65,535. When the value of “hc_type” is “icmp”, the value of “port” must be “0”.
interval	Optional. This parameter, measured in seconds, specifies the health check interval for the security device. Its value must be an integer ranging from 1 to 10,000. The default value is 10.
timeout	Optional. This parameter, measured in seconds, specifies the maximum interval allowed from sending a health check request to receiving the response. Its value must be an integer ranging from 1 to 10,000. The default value is 5. Note: The value of “timeout” is no larger than that of “interval”.
hc_up	Optional. This parameter specifies for how many consecutive times the security device should pass the health check before it is determined as in the “Up” status. Its value must be an integer ranging from 1 to 255. The default value is 3.
hc_down	Optional. This parameter specifies for how many consecutive times the security device should fail the health check before it is determined as in the “Down” status. Its

value must be an integer ranging from 1 to 255. The default value is 3.

no orchestrator health device check <hc_name>

This command is used to delete the specified L3 additional health check.

show orchestrator health device check [security_device]

This command is used to display all additional health checks configured for the specified L3 security device. If the parameter “security_device” is not specified, the system will display additional health checks configured for all L3 security devices.

clear orchestrator health device check [security_device]

This command is used to clear all additional health checks configured for the specified L3 security device. If the parameter “security_device” is not specified, the system will clear additional health checks configured for all L3 security devices.

orchestrator health bind <hc_name> <request_index> <response_index>

This command is used to bind the specified HTTP health check to the custom health check request and response.

hc_name	This parameter specifies the HTTP health check name configured for L3 security services.
request_index	This parameter specifies the index location of the health check request in the health check request table.
response_index	This parameter specifies the index location of the expected health check response in the health check response table.

no orchestrator health bind <hc_name>

This command is used to delete the bond between the specified HTTP health check and the custom health check request and response.

show orchestrator health bind [hc_name]

This command is used to display the bond between the specified HTTP health check and the custom health check request and response. If the parameter “hc_name” is not specified, the system will display all bonds between HTTP health checks and custom health check requests and responses.

clear orchestrator health bind

This command is used to clear all bonds between HTTP health checks and custom health check requests and responses.

show orchestrator health device status [security_device]

This command is used to show the health status of the specified security device. If the parameter “security_device” is not specified, the system will display the health status of all security devices.

show orchestrator health service status [security_service]

This command is used to display the health status of the specified security service and its security devices. If the parameter “security_service” is not specified, the system will display the health status of all security services and their security devices.

show orchestrator health chain status [security_service_chain]

This command is used to display the health status of the specified security service chain and its security services. If the parameter “security_service_chain” is not specified, the system will display the health status of all security service chains and their security services.

Orchestration Policy

orchestrator policy name <orchestration_policy> <precedence>
 <scope_type> {listener_name|security_service_chain|slb_group_name}
 <policy_type> [security_service_chain|slb_virtual_service]

This command is used to create an orchestration policy.

This command is also used to modify the settings of the “precedence” parameter of orchestration policy.

orchestration_policy This parameter specifies the name of the orchestration policy. Its value must be a case-sensitive string of 1 to 128 characters. Numbers, letters, and special characters are supported. When the parameter value contains special characters, it must be enclosed in double quotes.

Supported special characters: “@”, “!”, “~”, “_”, “:”, “-”, “<”, “>”, “.”.

The name of each orchestration policy must be unique.

precedence This parameter specifies the priority of orchestration policy. Its value must be an integer ranging from 1 to 4,294,967,295. The larger the value, the higher the priority.

scope_type This parameter specifies the application scope of the orchestration policy. Its value must be:

- listener
- chain
- group

listener_name|security_s This parameter specifies the name of the traffic listener, the

ervice_chain slb_group_ name	name of the orchestrator chain or the name of the SLB real service group.
policy_type	This parameter specifies the type of the orchestration policy. Its value must be: - chain: Associate the orchestration policy to an orchestration chain. - virtual: Associate the orchestration policy to an SLB virtual service. - deny: Block the traffic When the “scope_type” parameter is set to “chain”, this parameter cannot be set to deny; when “scope_type” parameter is set to “group”, this parameter should be set to “chain”.
security_service_chain sl b_virtual_service	Optional. This parameter specifies the name of a security service chain or the name of an SLB virtual service. The default value is empty.

no orchestrator policy name <orchestration_policy>

This command is used to delete the specified orchestration policy.

show orchestrator policy name [orchestration_policy]

This command is used to display the specified orchestration policy. If the “orchestration_policy” parameter is not specified, all orchestration policies will be displayed.

clear orchestrator policy name

This command is used to clear all orchestration policies.

Orchestration Rules

Network Orchestration Rules

orchestrator rule network <rule_name> <src_ip> <src_port> <dst_ip> <dst_port> <protocol>

This command is used to create an orchestration rule. When the traffic hits the orchestration rule, the system forwards traffic to the orchestration policy (configured by the “**orchestrator policy rule**” command) associated with the rule for processing.

This command is also used to update the parameter settings of the orchestration rule.

rule_name	This parameter specifies the name of the orchestration rule. Its value must be a case-sensitive string of 1 to 128 characters. Numbers, letters, and special characters are supported. When the parameter value contains special characters, it must be enclosed in double quotes. Supported special characters: “@”, “!”, “~”, “_”, “:”, “-”, “<”, “>”, “.”. The name of each network security rule must be unique.
src_ip	This parameter specifies the source IP address, subnet or IP range. Its value must be an IPv4 or IPv6 address. Its format is: • Single IP • IP subnet/netmask or prefix length • IP range: IP1-IP2
src_port	This parameter specifies the source port number or port range. Its value must be an integer ranging from 1 to 65,535. The port range must be in the format of “X-Y”. For example, “1000-2000” indicates the source port ranging from 1000 to 2000. Port 0 is a wildcard. When the “protocol” parameter is set to “icmp” or “other”, its value must be 0.
dst_ip	This parameter specifies destination IP address, subnet or IP rang. For details, see the description for the “src_ip” parameter.
dst_port	This parameter specifies the destination port number or port range. For details, see the description for the “src_port” parameter.
protocol	This parameter specifies the matched protocol type. Its value must be “TCP”, “UDP”, “ICMP”, “other” or “any”. If the parameter value is “other”, the rule matches packets of all protocols except TCP, UDP, and ICMP; if the parameter value is “any”, the rule matches packets of all protocols.

no orchestrator rule network <rule_name>

This command is used to delete the specified orchestration rule.

show orchestrator rule network [rule_name]

This command is used to display the specified orchestration rule. If the “rule_name” parameter is not specified, all orchestration rules will be displayed.

clear orchestrator rule network

This command is used to clear all orchestration rules.

Application Orchestration Rules

orchestrator rule app <rule_name> <src_ip> <src_port> <dst_ip> <protocol>

Creates an application orchestration rule. When the traffic matches the application orchestration rule, the system forwards the traffic to the orchestration policy associated to the rule (configured using the command **orchestrator policy rule**) for processing. Note that DPI needs to be enabled to match the traffic of the application layer.

This command can also update the parameter settings of an application orchestration rule.

rule_name	This parameter specifies the name of an application orchestration rule. The value must be a case-sensitive string with a length not greater than 128 characters. Letters, numbers, and special characters are supported. When the value contains only numbers or special characters, it must be enclosed by double quotes. Supported special characters: the at sign (@), exclamation mark (!), tilde (~), underscore (_), colon (:), hyphen (-), angle brackets (< >), and period (.). The name of each application rule must be unique.
src_ip	This parameter specifies the source IP address, IP subnet, or IP address range. The value must be an IPv4 or IPv6 address. The format must be: • A single IP address • IP subnet/mask or prefix length • IP address range: IP1-IP2
src_port	The parameter specifies a source port or port range. The value must be an integer ranging from 1 to 65,535. The format of the port range should be “X-Y.” For example, “1000-2000” means the port range is from 1000 to 2000. This parameter’s value can also be set to zero, which means all ports can be used. When protocol is set to icmp or other, this parameter’s value must be zero.
dst_ip	This parameter specifies the destination IP address, IP subnet, or IP address range. This parameter’s meaning and its value are the same as src_ip.

protocol	This parameter specifies the type of the application layer protocol. The value must be “http,” “dns,” “ftp,” “ssh,” “telnet,” “smtp,” “pop3,” “imap,” “snmp,” “ntp,” or “tcp.”
----------	--

no orchestrator rule app <rule_name>

Deletes an application orchestration rule.

show orchestrator rule app <rule_name>

Displays a specified application orchestration rule. If no value is assigned to the **rule_name** parameter, the system will display all application orchestration rules.

clear orchestrator rule app

Deletes all application orchestration rules.

orchestrator policy rule <orchestration_policy> <rule_name>

This command is used to associate the specified orchestration rule with the specified orchestration policy. An orchestration policy can be associated with 32 orchestration rules at most.

security_policy	This parameter specifies the name of an existing orchestration policy.
-----------------	--

rule_name	This parameter specifies the name of an existing orchestration rule.
-----------	--

no orchestrator policy rule <orchestration_policy> <rule_name>

This command is used to delete the association between the specified orchestration rule and the specified orchestration policy.

show orchestrator policy rule [orchestration_policy]

This command is used to display the association between the orchestration rule and the specified orchestration policy. If the “orchestration_policy” parameter is not specified, all the association between orchestration rules and the specified orchestration policy will be displayed.

clear orchestrator policy rule [orchestration_policy]

This command is used to clear the association between the orchestration rule and the specified orchestration policy. If the “orchestration_policy” parameter is not specified, all the association between orchestration rules and the specified orchestration policy will be cleared.

EPolicy Integration

By integrating with ePolicy, the orchestration function supports recording the HTTP log for security service chain. Administrators can use the command “**show log buff**” to display the HTTP log related to the security service chain.

orchestrator epolicy attach script <security_service_chain> <script_name>

This command is used to associate the specified ePolicy event handler script with the specified security service chain. A security service chain can be associated with a maximum of 10 event handler scripts, while a event handler script can be associated with multiple security service chain.

security_service_chain This parameter specifies the name of an existing security service chain.

script_name This parameter specifies the name of the imported event handler script. The event handler script must be imported by the “**epolicy import script**” demand.



Note: For ePolicy event handler script for integration with orchestrator, please contact Customer Support.

no orchestrator epolicy attach script <security_service_chain>
<script_name>

This command is used to delete the association between the specified ePolicy event handler script and the specified security service chain.

show orchestrator epolicy attach [security_service_chain]

This command is used to display the association between the ePolicy event handler script and the specified security service chain. If the “security_service_chain” parameter is not specified, all the associations between the ePolicy event handler script and the specified security service chain will be displayed.

clear orchestrator epolicy attach [security_service_chain]

This command is used to clear the association between the ePolicy event handler script and the specified security service chain. If the “security_service_chain” parameter is not specified, all the association between the ePolicy event handler script and the specified security service chain will be cleared.

Orchestrator Statistics

show statistics orchestrator device [security_device]

This command is used to display the statistics of the specified security device. If the “security_device” parameter is not specified, statistics for all security devices will be displayed.

clear statistics orchestrator device [security_device]

This command is used to clear the statistics of specified security device. If the “security_device” parameter is not specified, the statistics of all security devices will be cleared.

show statistics orchestrator health device [security_device]

This command is used to display the health statistics of the specified security device. If the “security_device” parameter is not specified, the health statistics of all security devices will be displayed.

clear statistics orchestrator health device [security_device]

This command is used to clear the health statistics of the specified security device. If the “security_device” parameter is not specified, the health statistics of all security devices will be cleared.

show statistics orchestrator service [security_service]

This command is used to display the statistics of the specified security service. If the “security_device” parameter is not specified, the statistics of all security services will be displayed.

clear statistics orchestrator service [security_service]

This command is used to clear the statistics of the specified security service. If the “security_device” parameter is not specified, the statistics of all security services will be cleared.

show statistics orchestrator chain [security_service_chain]

This command is used to display the statistics of the specified security service chain. If the “security_service_chain” parameter is not specified, the statistics of all security service chains will be displayed.

clear statistics orchestrator chain [security_service_chain]

This command is used to clear the statistics of the specified security service chain. If the “security_service_chain” parameter is not specified, the statistics of all security service chains will be cleared.

show statistics orchestrator policy [orchestration_policy]

This command is used to display the statistics of the specified orchestration policy. If the “orchestration_policy” parameter is not specified, the statistics of all orchestration policies will be displayed.

clear statistics orchestrator policy [*orchestration_policy*]

This command is used to clear the statistics of the specified orchestration policy. If the “orchestration_policy” parameter is not specified, the statistics of all orchestration policies will be cleared.

Functions and Specifications

Specification	Memory			
	<4G	≥4G	≥32G	≥128G
Number of orchestrator listener	0	4	4	4
Number of security device	0	64	128	256
Number of security device	0	16	32	64
Number of security service chain	0	32	64	128
Total number of the security service chain member	0	128	256	512
Number of orchestration rules	0	128	256	512
Total number of orchestration policy	0	64	128	256
Total number of orchestration rules associated with orchestration policies	0	512	1024	2048

Other Orchestration Commands**show orchestrator all**

This command is used to display all orchestration configuration.

clear orchestrator all

This command is used to clear all orchestration configuration.

Chapter 10 Segment Management

System Mode

segment name <segment_name>

This command is used to create a segment and specify its name.

segment_name This parameter specifies the segment name. Its value must be a string of 1 to 128 characters.

Letters (A-Z and a-z), numbers (0-9), and special characters “@”, “-”, “_” and “!” are supported. It does not support “@@” and cannot begin or end with a “@” symbol.

If the parameter value consists of pure numbers or contains special characters, it must be enclosed in double quotes.



Note: When the SSL function is configured in the segment mode, the total length of the segment name, virtual host name or real host name, and the associated SNI domain name cannot exceed 240 characters.

The maximum number of “**segment name**” items allowed on the APV varies between system memory. See the following table for details.

System Memory	Maximum “segment name” Items
1 GB ≤ System Memory ≤ 24 GB	8
System Memory = 32 GB	64
64 GB ≤ System Memory ≤ 384 GB	1024

no segment name <segment_name>

This command is used to delete the specified segment and all configurations related to it.

clear segment name

This command is used to clear all segments and related configurations.

show segment name [segment_name]

This command is used to display the specified segment. If the “segment_name” parameter is not specified, all segments will be displayed.

segment user <segment_user> <segment_name> <password> [level]

This command is used to create a segment administrator account, set its user name, password and privilege level, and associate it with the specified segment.

Each segment administrator account can be associated with only one segment. Each segment can have multiple segment administrator accounts.

For an existing administrator account, to modify its privilege level, you need to delete it and then recreate one.

segment_user	This parameter specifies the name of the segment administrator account. Its value must be a string of 1 to 32 characters. Numbers and letters (case-sensitive) are supported. Special characters are not supported, but you can use “\$” to end the parameter value. If the parameter value ends with “\$” or consists of pure numbers, it must be enclosed in double quotes.
segment_name	This parameter specifies the name of an existing segment.
password	This parameter specifies the password. When the password force mode is disabled, its value must be a case-sensitive string of 1 to 116 characters. If the parameter value contains numbers or special characters, it must be enclosed in double quotes. When the password force mode is enabled, its value must meet the password complexity requirements in the password force mode (see the “passwd forcemode” command).
level	Optional. This parameter specifies the privilege level for the segment administrator account. Its value must be: • enable: segment administrator accounts with this privilege level are only allowed to execute enable-level CLI commands supported in the segment. • config: segment administrator accounts with this privilege level are allowed to execute all config-level CLI commands supported in the segment. • api: segment administrator accounts with this privilege level are allowed to access API-based Web service supported in the segment, but are not allowed to execute any CLI commands. The default value is “config”.

segment passwd <segment_user> <password>

This command is used to change the password of the specified segment administrator account.

<code>segment_user</code>	This parameter specifies the name of an existing segment administrator account.
<code>password</code>	This parameter specifies the new password. When the password force mode is disabled, its value must be a case-sensitive string of 1 to 116 characters. If the parameter value contains numbers or special characters, it must be enclosed in double quotes. When the password force mode is enabled, its value must meet the password complexity requirements in the password force mode (see the “passwd forcemode” command).

no segment user <segment_user>

This command is used to delete the specified segment administrator account.

show segment user [segment_name]

This command is used to display all administrator accounts configured for the specified segment. If the “segment_name” parameter is not specified, the administrator account configurations for all segments will be displayed.

clear segment users [segment_name]

This command is used to clear all administrator accounts configured for the specified segment. If the “segment_name” parameter is not specified, all segment administrator accounts will be cleared.

segment interface <segment_name> {system_ifname|bond_ifname|vlan_ifname}

This command is used to add the specified system interface, bond interface or VLAN interface to the specified segment.

<code>segment_name</code>	This parameter specifies the name of an existing segment.
<code>system_ifname</code>	This parameter specifies the name of the system interface, which can be customized by the “interface name” command. The default system interface name is port1, port2, port3, port4, etc.
<code>bond_ifname</code>	This parameter specifies the bond interface name.

The default bond interface name is bond1, bond2, bond3, bond4, etc.

`vlan_ifname` This parameter specifies the name of a VLAN interface, which can be customized by the “**segment vlan**” command.



Note: A segment can only use interfaces that have been added to it. To use an interface that does not belong to it, for example, in specific circumstances, please contact Customer Support.

no segment interface *<segment_name>*
{system_ifname|bond_ifname|vlan_ifname}

This command is used to delete an interface configuration for the specified segment.

show segment interface *[segment_name]*

This command is used to display all interface configurations for the specified segment. If the “segment_name” parameter is not specified, the interface configurations for all segments will be displayed.

clear segment interface *[segment_name]*

This command is used to clear all interface configurations for the specified segment. If the “segment_name” parameter is not specified, the interface configurations for all segments will be cleared.

segment ip address *{system_ifname|bond_ifname|vlan_ifname|mnet_ifname}*
<segment_ip> {netmask|prefix} [nat_mode] [unit_name]

This command is used to set the IP address and netmask or prefix length for the system interfaces, bond interfaces, VLAN interfaces or MNET interfaces in segments and configure address translation rules.

<code>system_ifname</code>	This parameter specifies the system interface name.
<code>bond_ifname</code>	This parameter specifies the bond interface name.
<code>vlan_ifname</code>	This parameter specifies the VLAN interface name.
<code>mnet_ifname</code>	This parameter specifies the MNET interface name.
<code>segment_ip</code>	This parameter specifies the IP address to be translated, which can be an IPv4 or IPv6 address.
<code>netmask prefix</code>	This parameter specifies the netmask or prefix length of the IP address to be translated.

- “netmask” is used for an IPv4 address. Its value can be

	a dotted IP address or an integer ranging from 0 to 32.
	“prefix” is used for an IPv6 address. Its value should be an integer ranging from 0 to 128.
nat_mode	Optional, this parameter specifies the mode of NAT translation of the segment IP address. The value must be as follows: - internal_ip: In this mode, the system statically translates the segment IP address to this internal IP address for NAT translation. When the parameter segment_ip and the IP address of a real service are not in the same network segment, NAT needs to be configured for the real service (using the command segment nat). Note: At this time this parameter’s value cannot be the IP address in the following network segment: 7.0.0.0/8, 8.0.0.0/8, 9.0.0.0/8, 1237::/64, 1238::/64, 1239::/64. - auto: In this mode, the system automatically translates the segment IP address to an IP address selected from the IP subnet configured by the “system tune segment iprange” command, and the system automatically generates a NAT rule for the real service. - empty: The system does not perform NAT. When this parameter is not specified, the system will not perform address translation. The default value is empty.
unit_name	Optional. This parameter specifies the HA unit name. If this parameter is not specified, this command will be configured for the local unit.

**Note:**

- In HA scenario, before performing synchronization operations, make sure the “**segment ip address**” command is configured with the “unit_name” option. Otherwise, the configurations of this command will not be synchronized.
- In HA scenario, the internal IP addresses of different HA nodes on the same MNET interface must be the same. In other cases, the internal IP cannot be the same.
- In the segment, when the value of the parameter “internal_ip|auto” is auto, the real service must be configured in the segment first and then the static route. When deleting the configuration of the real service, the corresponding static route configuration must be deleted first.
- If the parameter **internal_ip|auto** is specified, the VIP in the global SLB configuration cannot be of the same network segment as that of the NAT

converted IP address. If parameter “internal_ip|auto” is not specified, the VIP in the global SLB configuration cannot be of the same network segment as that of segment IP address.

- Before SLB and SSL settings are configured in the segment, you need to configure networking using the command such as **segment ip address** in the system mode.
- In actual use, the parameter **internal_ip** is not allowed to be configured as IP network segment automatically assigned by the system.
- In a segment, if the IP address of a configured real service is not in the same network segment as the IP address of the segment, a static route needs to be added to this real service, and the destination network of the static route is the IP address of this real service. The subnet mask is 32 bits (IPv4) or 128 bits (IPv6).
- If an additional health check is configured for a real service, and the destination IP address of this health check is not in the same network segment as the IP address of the real service, a NAT rule needs to be configured for the destination IP address for this health check (using the command **segment nat**).

no segment ip address <segment_interface_name> [ip_type] [auto]
[unit_name]

This command is used to delete the IP address setting for the specified segment interface.

segment_interface_name This parameter specifies the name of the segment interface.

ip_type This parameter specifies the IP address type of the segment interface. Its value must be:

- 4: indicates the IPv4 address.
- 6: indicates the IPv6 address.

The default value is 4.

auto When this parameter is set to “auto”, the system will delete the IP network segment automatically allocated during NAT translation.

unit_name Optional. This parameter specifies the HA unit name. If this parameter is not specified, this command will be configured for the local unit.

show segment ip address

This command is used to display the IP address settings of all segment interfaces.

clear segment ip address

This command is used to clear all segment interface IP address settings.

segment mnet {system_ifname|bond_ifname|vlan_ifname}
<mnet_interface_name>

This command is used to create a Multi-Netting (MNET) interface for the specified system interface, bond interface or VLAN interface within a segment. The system supports creating at most 4126 segment MNET interfaces.

system_ifname	This parameter specifies the system interface name, which, by default, is port1, port2, port3, port4, etc. (Administrators can self-define the system interface name by using the command “ interface name ”).
bond_ifname	This parameter specifies the bond interface name, which, by default, is bond1, bond2, bond3, bond4, etc. (Administrators can self-define the bond interface name by using the command “ bond name ”).
vlan_ifname	This parameter specifies the VLAN interface name. (Administrators can create VLAN interfaces by using the command “ vlan ”).
mnet_interface_name	This parameter specifies the MNET interface name, which should be an alphanumeric string of up to 31 characters.

no segment mnet <mnet_interface_name>

This command is used to delete a specified segment MNET interface.

mnet_interface_name	This parameter specifies the name of the segment MNET interface which will be deleted.
---------------------	--

show segment mnet

This command is used to display the configurations of all segment MNET interfaces.

clear segment mnet

This command is used to clear the configurations of all segment MNET interfaces.

segment vlan {system_ifname|bond_ifname} <vlan_ifname> <vlan_tag>

This command is used to create a segment VLAN interface on the specified system interface or bond interface. The system supports creating at most 4094 VLAN interfaces, including system VLAN interfaces and segment VLAN interfaces.

System VLAN interfaces and segment VLAN interfaces cannot share the same interface name or ID.

system_ifname	This parameter specifies the system interface name, which, by default, is port1, port2, port3, port4, etc. (Administrators
---------------	--

can self-define the system interface name by using the command “**interface name**”).

bond_ifname	This parameter specifies the bond interface name, which, by default, is bond1, bond2, bond3, bond4, etc. (Administrators can self-define the bond interface name by using the command “ bond name ”).
vlan_ifname	This parameter specifies the VLAN interface name, which should be an alphanumeric string of up to 31 characters.
vlan_tag	This parameter specifies the ID for the VLAN interface being created. It can be any integer ranging from 1 to 4094.



Note: Please do not configure VLAN on interface that has been configured as bond interface.

no segment vlan <vlan_ifname>

This command is used to delete the specified segment VLAN interface.

show segment vlan

This command is used to display all segment VLAN interfaces.

clear segment vlan

This command is used to clear all segment VLAN interfaces.

segment {enable|disable}

This command is used to enable or disable the segment NAT. By default, it is disabled.

segment nat <segment_name> <segment_ip> <internal_ip> {netmask|prefix}

This command is used to configure an address translation rule for the specified segment. Each segment can have at most 4096 address translation rules.

segment_name	This parameter specifies the name of an existing segment.
segment_ip	This parameter specifies the IP address to be translated, which can be an IPv4 or IPv6 address.
internal_ip	This parameter specifies the IP address after translation.
netmask prefix	This parameter specifies the netmask or prefix length of the IP address to be translated. “netmask” is used for an IPv4 address. Its value can be a dotted IP address or an integer ranging from 0 to 32.

- “prefix” is used for an IPv6 address. Its value should be an integer ranging from 0 to 128.

**Note:**

- In SLB reverse mode, if the address translation rule is not configured for the IP address of the real service (by the command “**segment nat**”), the real service will use the system route.
- If you need to configure the static route to the networks with the same destination IP addresses for different segments, you need to use the command **segment nat** to configure NAT rules for each segment.
- In actual use, the parameter **internal_ip** is not allowed to be configured as IP network segment automatically assigned by the system.

no segment nat <segment_name> <segment_ip> <internal_ip>
{netmask|prefix}

This command is used to delete the specified address translation rule setting for the specified segment.

show segment nat [segment_name]

This command is used to display all address translation rule settings for the specified segment. If the “segment_name” parameter is not specified, the address translation rule settings for all segments will be displayed.

clear segment nat [segment_name]

This command is used to clear all address translation rule settings for the specified segment. If the “segment_name” parameter is not specified, the address translation rule settings for all segments will be cleared.

segment ha group id <group_id>

This command is used to define a segment floating IP group for the local unit. A maximum of 1024 groups can be defined for each unit, including both system floating IP groups and segment floating IP groups.

group_id	This parameter specifies the ID of the floating IP group. Its value must be an integer in the range of 0 to 1023.
----------	---

no segment ha group id <group_id>

This command is used to delete the specified segment floating IP group from the local unit.

segment ha group {enable|disable} <group_id>

This command is used to enable or disable the specified segment floating IP group on the local unit. By default, all segment floating IP groups are disabled. When a segment

floating IP group is enabled on multiple HA units, only the HA unit whose group status is “Active” will provide services in that group. The floating IP groups can be associated with the application services by executing the commands “**segment ha group fip**” and “**segment ha group fiprange**”.

group_id	This parameter specifies the ID of the floating IP group. Its value must be an integer ranging from 0 to 1024. “1024” means enabling or disabling all segment floating IP groups on the local unit.
----------	---

segment ha group preempt {on|off} <group_id>

This command is used to enable or disable the preempt mode for the specified segment floating IP group. With the preempt mode enabled, the status of a floating IP group on the available unit with the highest group priority will be always kept as “Active”. By default, the preempt mode is disabled for the floating IP group.

group_id	This parameter specifies the ID of the floating IP group. Its value must be an integer ranging from 0 to 1024. “1024” means enabling or disabling the preempt mode for all segment floating IP groups.
----------	--

segment ha group priority <unit_name> <group_id> <priority>

This command is used to configure a priority for the specified segment floating IP group residing on the specified HA unit.

unit_name	This parameter specifies the HA unit name. It can be the local unit or a peer unit.
group_id	This parameter specifies the ID of an existing floating IP group.
priority	This parameter specifies the priority value. Its value must be an integer ranging from 0 to 255. A larger value indicates a higher priority.

no segment ha group priority <unit_name> <group_id>

This command is used to delete the priority setting for the specified segment floating IP group on the specified HA unit.

segment ha group fip <group_id> <floating_ip> <segment_name> [interface_name] [floating_mac]

This command is used to associate the specified segment floating IP group with the specified segment and configure a floating IP address for the segment floating IP group.

group_id	This parameter specifies the ID of an existing segment floating IP group.
floating_ip	This parameter specifies the floating IP address, which can be an IPv4 or IPv6 address.
segment_name	This parameter specifies the name of an existing segment.
interface_name	Optional. This parameter specifies the interface to be bound to the floating IP address. Its value must be the name of a system interface, bond interface, MNET interface, VLAN interface or “auto”. The value “auto” means that an interface will be automatically selected. Note: It is not recommended to set this parameter to the interface of the primary link. When a floating MAC address needs to be set, this parameter cannot be set to the member interface of a bond interface. The default value is “auto”. When you want to configure the “floating_mac” parameter and meanwhile use the default value for this parameter, “auto” must be specified. If the “floating_mac” parameter will not be configured, it is allowed to not set this parameter to indicate that the default value will be used.
floating_mac	Optional. This parameter specifies the floating MAC address, in the format “xx:xx:xx:xx:xx:xx”. Its value cannot be the same as any of the following MAC addresses: system interface MAC address; MAC address bound to other system or bond interface in the same floating IP group; MAC address bound to other floating IP groups; MAC address of system interface added to a bridge. The default value is empty, indicating that the floating MAC function will not be used.

no segment ha group fip <group_id> <floating_ip> <segment_name>

This command is used to delete a floating IP address configuration for the specified segment.

segment ha group fiprange <group_id> <start_floating_ip>
<end_floating_ip> <segment_name> [interface_name] [floating_mac]

This command is used to associate the specified segment floating IP range with the specified segment and configure a floating IP range for the segment floating IP group.

Each floating IP range contains at most 256 IP addresses.

The total number of floating IP addresses and floating IP ranges configured for a floating IP group cannot exceed 16.

<code>group_id</code>	This parameter specifies the ID of an existing segment floating IP group.
<code>start_floating_ip</code>	This parameter specifies the start IP address of the floating IP range, which can be an IPv4 or IPv6 address.
<code>end_floating_ip</code>	This parameter specifies the end IP address of the floating IP range, which can be an IPv4 or IPv6 address.
<code>segment_name</code>	This parameter specifies the name of an existing segment.
<code>interface_name</code>	Optional. This parameter specifies the interface to be bound to the floating IP address. Its value must be the name of a system interface, bond interface, MNET interface, VLAN interface or “auto”. The value “auto” means that an interface will be automatically selected. Note: It is not recommended to set this parameter to the interface of the primary link. When a floating MAC address needs to be set, this parameter cannot be set to the member interface of a bond interface.

The default value is “auto”.

When you want to configure the “floating_mac” parameter and meanwhile use the default value for this parameter, “auto” must be specified. If the “floating_mac” parameter will not be configured, it is allowed to not set this parameter to indicate that the default value will be used.

<code>floating_mac</code>	Optional. This parameter specifies the floating MAC address, in the format “xx:xx:xx:xx:xx:xx”. Its value cannot be the same as any of the following MAC addresses: system interface MAC address; MAC address bound to other system or bond interface in the same floating IP group; MAC address bound to other floating IP groups; MAC address of system interface added to a bridge.
---------------------------	--

The default value is empty, indicating that the floating MAC function will not be used.

no segment ha group fiprange <group_id> <start_floating_ip>
<end_floating_ip> <segment_name>

This command is used to delete a floating IP range configuration for the specified segment.

clear segment ha group all

This command is used to clear all segment floating IP groups and related configurations.

segment ha decision rule <condition_name> <action_name> [group_id]

This command is used to configure a failover rule for the specified segment floating IP group. The failover rule indicates the failover operation to be performed when the result of a specified health check changes from “Up” to “Down”. A health check condition can be used for configuring a maximum of 1024 failover rules.

condition_name	This parameter specifies the name of the health check condition. Its value must be the name of a real health check condition or a vcondition. The system supports the following values: • (Predefined) PORT_1~PORT_32: port health check conditions • GATEWAY_1~GATEWAY_32: gateway health check conditions • CPU_UTIL: CPU utilization health check condition • CPU_TEMP: CPU temperature health check condition • SSL_CARD: SSL card health check condition • SSL_AE_UTIL: SSL card AE core utilization health check condition • SSL_SE_UTIL: SSL card SE core utilization health check condition • SYSTEM_MEM_UTIL: system memory utilization health check • NETWORK_MEM_UTIL: network memory utilization health check • SWAP_MEM_UTIL: SWAP memory utilization health check • CONN_MEM_UTIL: connection memory utilization health check • SYSTEM_DISK_UTIL: system disk space utilization health check condition • DATA_DISK_UTIL: data disk space utilization health check condition • Names of customized health check condition groups (vconditions), which can be configured by the “monitor vcondition name” command.
----------------	---

<code>action_name</code>	This parameter specifies the failover operation to be performed when the result of a specified health check is “Down”. Its value must be: • <code>Group_Failover</code>: Switch over the status of the segment floating IP group. For this action, the system will select a new unit based on the health condition and group priority, and change the status of the segment floating IP group enabled on that unit to be “Active” to take over the services. • <code>Unit_Failover</code>: Switch over the status of all segment floating IP groups enabled on a unit. • <code>Reboot</code>: Switch over the status of all segment floating IP groups enabled on a unit, and then restart the unit.
<code>group_id</code>	This parameter specifies the ID of the floating IP group for which the failover rule takes effect. This parameter is available only when the parameter “<code>action_name</code>” is set to “<code>Group_Failover</code>”.

no segment ha decision rule <condition_name> <action_name> [group_id]

This command is used to delete a failover rule of the specified segment floating IP group.

clear segment ha decision rule

This command is used to clear all segments' failover rules.

show segment ha config

This command is used to display the HA related configurations of all segments.

show segment ip eroute <segment_name>

This command is used to display the Eroutes that the system generates for the specified segment.

show segment config [segment_name]

This command is used to display all configurations related to the specified segment. If the “segment_name” parameter is not specified, the configurations related to all segments will be displayed.

show segment summary [segment_name]

This command is used to display the configurations for the specified segment in summary. If the “segment_name” parameter is not specified, the configurations for all segments will be displayed in summary.

write segment file

This command is used to save all running configurations within segments into a backup file.

config segment file

This command is used to load the configurations saved by the “**write segment file**” command.

write segment memory [segment_name]

This command is used to save all configurations within the specified segment into the hard disk as the next-time startup configuration. If the “segment_name” parameter is not specified, all configurations within segments will be saved. After this command is executed, it is recommended to run the “**write memory**” command.

Segment Mode

A segment has its own CLI commands for configuring SLB and SSL, and all segments support the same set of CLI commands. For Config-level segment administrator accounts, all CLI commands supported in the segment will be displayed with the input of the question mark in Config mode. Please refer to command descriptions in corresponding chapters. The following commands are available only in the segment mode.

password <password>

This command is used to change the password of the segment administrator account.

password	This parameter specifies the new password. When the password force mode is disabled, its value must be a case-sensitive string of 1 to 116 characters. If the parameter value contains numbers or special characters, it must be enclosed in double quotes. When the password force mode is enabled, its value must meet the password complexity requirements in the password force mode (see the “ passwd forcemode ” command).
----------	---

Chapter 11 HTTP Proxy

While providing HTTP and HTTPS SLB services, the APV appliance can function as a proxy to implement the functions of rewrite, security control, error processing, HTTPS forwarding control, data compression, and cache for HTTP services.

By applying configurations of proxy functions to a specific HTTP or HTTPS virtual service, the appliance will perform reverse proxy operations on traffic arriving at the virtual service. For example, after applying the cache function to virtual service “httpsvs” using the “**cache on httpsvs**” command, the appliance will save responses returned from real service in its buffer memory.

HTTP/2 Support

HTTP proxy functions support both HTTP/1 and HTTP/2. By default, HTTP/1 is enabled. The enabling status of HTTP/2 depends on whether the associated virtual service is enabled with HTTP/2 (using the “**http2 virtual on**” command).

If a virtual service is enabled with HTTP/2, related HTTP proxy configurations applied to the virtual service will also support HTTP/2. For example, the following commands enable HTTP/2 cache for virtual service “httpsvs”.

```
AN(config)#http2 virtual on httpsvs
AN(config)#cache on httpsvs
```



Note: The following HTTP proxy functions do not support HTTP/2:

- Function of splicing the TCP connections at the client and real service sides (configured by the “**http connsplice on**” command)
- Function of accelerating HTTP request processing (configured by the “**http turbo**” command)

HTTP Rewrite

The HTTP rewrite function supports the following rewriting actions on HTTP requests and responses:

- Rewrites the Host header in requests and the Location header in responses to implement redirection.
- Adds HTTP headers to carry specific information to clients or real services.
- Deletes HTTP headers to hide information about the proxy process and real service from clients.
- Rewrites contents in responses, so that clients can access internal network resources.

Header Rewrite

➤ URL Redirect

The following commands help implement HTTP redirect by header rewrite. Administrators can configure the appliance to directly return an HTTP redirect response when receiving the specified type of HTTP request, so that the browser will request for the new URL address. The appliance can also be configured to rewrite the URL address in requests before forwarding the requests to real services. For redirect responses returned from real services, the appliance supports the rewriting of the protocol, host name, and port number in the URL address.

http redirect url <virtual_service> <policy_name> <priority> <orig_host>
<orig_path> <new_protocol> <new_host> <new_path> <status_code>

This command is used to configure a URL redirect rule. When the Path value in the request line and the Host value in the Host header match the values of the “orig_path” and “orig_host” parameters respectively, the appliance will use the values of the “new_protocol”, “new_host”, and “new_path” to compose an HTTP 301, 302, 303, or 307 response to the client browser. Then the browser will request for the new URL address.

For appliances with 1 GB or 2 GB memory, a maximum of 200 URL redirect rules can be configured. For appliances with 4 GB memory or above, a maximum of 400 URL redirect rules can be configured.

virtual_service	This parameter specifies the virtual service name. Its value must be an HTTP or HTTPS virtual service.
policy_name	This parameter specifies the name of the URL redirect rule. Its value must be a string of 1 to 40 characters, and enclosed in double quotes when containing any special characters or containing only numbers.
priority	This parameter specifies the priority of the URL redirect rule. For appliances with 1 GB or 2 GB memory, its value must be an integer ranging from 1 to 200. For appliances with 4 GB memory or above, its value must be an integer ranging from 1 to 400. A higher value indicates a higher priority. When multiple URL redirect rules are matched, the appliance preferentially executes the one with the highest priority.
orig_host	This parameter specifies the string used to match the Host value in the Host header, which is case-sensitive. The parameter value supports both quick and full regular expressions. When the parameter value is set to a regular expression, it must be enclosed in double quotes. When the parameter value is a full regular expression, it must begin with the “<regex>” string and be enclosed in double quotes, for example, “<regex>/(graphics photos resource)/(.)\.(gif jpg png)”

means matching all GIF, JPG, and PNG files in the paths “/graphics”, “/photos”, and “/resource”. For information about the differences between the quick regular expression and the full regular expression, please refer to introductions about regular expressions in Chapter 1.

To ensure that a configured regular expression string is correct, administrators can use the “**slb regextest** *<regex>* *<target_string>*” command to test whether a target string will match the configured regular expression.

orig_path

This parameter specifies the string used to match the Path value in the HTTP request line, which is case-sensitive. The parameter value supports both quick and full regular expressions. When the parameter value is set to a regular expression, it must be enclosed in double quotes. When the parameter value is a full regular expression, it must begin with the “<regex>” string and be enclosed in double quotes, for example, “<regex>/(graphics|photos|resource)/(.)\.(gif|jpg|png)” means matching all GIF, JPG, and PNG files in the paths “/graphics”, “/photos”, and “/resource”. For information about the differences between the quick regular expression and the full regular expression, please refer to introductions about regular expressions in Chapter 1.

To ensure that a configured regular expression string is correct, administrators can use the “**slb regextest** *<regex>* *<target_string>*” command to test whether a target string will match the configured regular expression.

new_protocol

This parameter specifies the Protocol value in the Location header of the response to the browser. Its value must be “http” or “https”.

new_host

This parameter specifies the Host value in the Location header of the response to the browser.

new_path

This parameter specifies a new string, which is used to replace the string that matches the “orig_path” parameter value.

status_code

This parameter specifies the HTTP status code in the response to the browser. Its value must be “301”, “302”, “303”, or “307”.

For example:

```
AN(config)#http redirect url vhost redirectpolicy 10 www.mailserver.com /market https
mailserver.com /support 301
```

With this command, the URL string to be matched is “/market”, and “/support” will be used as the new Path value. Therefore, after this command is executed, the request for “http://www.mailserver.com/market/faq/index.html” will be redirected to “https://mailserver.com/support/faq/index.html”, and an HTTP 301 response will be sent to the browser.

no http redirect url <virtual_service> <policy_name>

This command is used to delete a URL redirect rule configuration for the specified virtual service.

show http redirect url [virtual_service]

This command is used to display the URL redirect rule configurations for the specified virtual service. If the “virtual_service” parameter is not specified, the URL redirect rule configurations for all virtual services will be displayed.

clear http redirect url <virtual_service>

This command is used to clear the URL redirect configurations for the specified virtual service.

virtual_service	This parameter specifies the virtual service name. Its value must be: • Virtual service name: clears the URL redirect rule configurations for this virtual service. • “all”: clears the URL redirect rule configurations for all virtual services.
-----------------	--

http redirect https <virtual_service>

This command is used to enable the function of redirecting HTTP requests to HTTPS for the specified virtual service. Upon receiving an HTTP request, the appliance will reply with an HTTP redirect response in which the Protocol value of the Location header is changed to “https”.

virtual_service	This parameter specifies the virtual service name. Its value must be an HTTP virtual service.
-----------------	---

no http redirect https <virtual_service>

This command is used to disable the function of redirecting HTTP requests to HTTPS for the specified virtual service.

show http redirect https

This command is used to display the virtual services for which the function of redirecting HTTP requests to HTTPS is enabled.

clear http redirect https

This command is used to disable the function of redirecting HTTP requests to HTTPS for all virtual services.

http rewrite request url <virtual_service> <policy_name> <priority>
<orig_host> <orig_path> <new_host> <new_path>

This command is used to configure an HTTP request rewrite rule to modify the Path value in the request line and the Host value in the Host header of the request sent from the client.

For appliances with 1 GB or 2 GB memory, a maximum of 200 HTTP request rewrite rules can be configured. For appliances with 4 GB memory or above, a maximum of 400 HTTP request rewrite rules can be configured.

virtual_service	This parameter specifies the virtual service name. Its value must be an HTTP or HTTPS virtual service.
policy_name	This parameter specifies the name of the HTTP request rewrite rule.
priority	This parameter specifies the priority of the HTTP request rewrite rule. For appliances with 1 GB or 2 GB memory, its value must be an integer ranging from 1 to 200. For appliances with 4 GB memory or above, its value must be an integer ranging from 1 to 400. When multiple HTTP request rewrite rules are matched, the appliance preferentially executes the HTTP request rewrite rule with the highest priority.
orig_host	This parameter specifies the string used to match the Host value, which is case-sensitive. The parameter value supports both quick and full regular expressions. When the parameter value is set to a regular expression, it must be enclosed in double quotes. When the parameter value is a full regular expression, it must begin with the “<regex>” string and be enclosed in double quotes, for example, “<regex>^www.xyz.” means matching all host names beginning with “www.xyz.”. For information about the differences between the quick regular expression and the full regular expression, please refer to introductions about regular expressions in Chapter 1.

Note: If the “orig_path” parameter needs to be a full regular expression, the “orig_host” parameter value must also be a full regular expression.

To ensure that a configured regular expression string is correct, administrators can use the “**slb regextest** <regex> <target_string>” command to test whether a target string

will match the configured regular expression.

`orig_path`

This parameter specifies the string used to match the Path value in the HTTP request line, which is case-sensitive. The parameter value supports both quick and full regular expressions. When the parameter value is set to a regular expression, it must be enclosed in double quotes. When the parameter value is a full regular expression, it must begin with the “<regex>” string and be enclosed in double quotes, for example,

“<regex>/(graphics|photos|resource)/(.)\.(gif|jpg|png)”

means matching all GIF, JPG, and PNG files in the paths “/graphics”, “/photos”, and “/resource”. For information about the differences between the quick regular expression and the full regular expression, please refer to introductions about regular expressions in Chapter 1.

To ensure that a configured regular expression string is correct, administrators can use the “**slb regextest** <regex> <target_string>” command to test whether a target string will match the configured regular expression.

`new_host`

This parameter specifies the Host value in the Host header of the rewritten HTTP request. The value “%r” indicates that the “ip:port” of the selected real service will be used as the value of the new Host, and this value must be enclosed in double quotes. If the port number of the selected real service is 0, it will be replaced by the port number used to connect this real service.

`new_path`

This parameter specifies the Path value in the request line of the rewritten HTTP request.



Note: If the Path does not need to be rewritten, administrators can set the value of “orig_path” and “new_path” as “/”.

no http rewrite request url <virtual_service> <policy_name>

This command is used to delete an HTTP request rewrite rule configuration for the specified virtual service.

show http rewrite request url [virtual_service]

This command is used to display the HTTP request rewrite rule configurations for the specified virtual service. If the “virtual_service” parameter is not specified, the HTTP request rewrite rule configurations for all virtual services will be displayed.

clear http rewrite request url <virtual_service>

This command is used to clear the HTTP request rewrite rule configurations for the specified virtual service.

virtual_service This parameter specifies the virtual service name. Its value must be:

Virtual service name: clears the HTTP request rewrite rule configurations for this virtual service.

“all”: clears the HTTP request rewrite rule configurations for all virtual services.

http rewrite response url *<virtual_service> <policy_name> <priority> <orig_protocol> <orig_host> <orig_path> <new_protocol> <new_host> <new_path>*

This command is used to configure an HTTP response rewrite rule for the specified virtual service to modify the Location header in the HTTP response sent from the real service.

For appliances with 1 GB or 2 GB memory, a maximum of 200 HTTP response rewrite rules can be configured. For appliances with 4 GB memory or above, a maximum of 400 HTTP response rewrite rules can be configured.

virtual_service This parameter specifies the virtual service name. Its value must be an HTTP or HTTPS virtual service.

policy_name This parameter specifies the name of the HTTP response rewrite rule.

priority This parameter specifies the priority of the HTTP response rewrite rule. For appliances with 1 GB or 2 GB memory, its value must be an integer ranging from 1 to 200. For appliances with 4 GB memory or above, its value must be an integer ranging from 1 to 400. A higher value indicates a higher priority. When multiple HTTP response rewrite rules are matched, the appliance preferentially executes the HTTP response rewrite rule with the highest priority.

orig_protocol This parameter specifies the Protocol value in the Location header of the HTTP response. Its value must be “http”, “https”, or “both”.

orig_host This parameter specifies the string used to match the Host value in the Location header of the HTTP response, which is case-sensitive. The parameter value supports both quick and full regular expressions. When the parameter value is set to a regular expression, it must be enclosed in double quotes. When the parameter value is a full regular expression, it must begin with the “<regex>” string and be

enclosed in double quotes, for example, “<regex>^www.xyz.” means matching all host names beginning with “www.xyz.”. For information about the differences between the quick regular expression and the full regular expression, please refer to introductions about regular expressions in Chapter 1.

Note: If the “orig_path” parameter needs to be a full regular expression, the “orig_host” parameter value must also be a full regular expression.

To ensure that a configured regular expression string is correct, administrators can use the “**slb regextest** <regex> <target_string>” command to test whether a target string will match the configured regular expression.

orig_path

This parameter specifies the string used to match the Path value in the HTTP response, which is case-sensitive. The parameter value supports both quick and full regular expressions. When the parameter value is set to a regular expression, it must be enclosed in double quotes. When the parameter value is a full regular expression, it must begin with the “<regex>” string and be enclosed in double quotes, for example, “<regex>/(graphics|photos|resource)/(.)\.(gif|jpg|png)” means matching all GIF, JPG, and PNG files in the paths “/graphics”, “/photos”, and “/resource”. For information about the differences between the quick regular expression and the full regular expression, please refer to introductions about regular expressions in Chapter 1.

To ensure that a configured regular expression string is correct, administrators can use the “**slb regextest** <regex> <target_string>” command to test whether a target string will match the configured regular expression.

new_protocol

This parameter specifies the Protocol value in the Location header of the rewritten HTTP response. Its value must be “http”, “https” or “both”.

new_host

This parameter specifies the Host field in the Location header of the redirected response. The value “%h” means that the Host field in the Location header will be replaced with the host name and port number in the client request; the value “%hn” means that the host name and port number in the Location header will be replaced with the host name in the client request, and a self-defined port number or domain name suffix such as “.org” can be added after “%hn”. For example, when the Host field in the client request is “abc.com:8080”:

- To keep the source host name and port number unchanged, set this parameter to “%h”. Then the Host field in the redirected response will still be “abc.com:8080”.
- To keep only the source host name unchanged and rewrite the port number to 443, set this parameter to “%hn”. Then the Host field in the redirected response will be “abc.com:443”. To rewrite the source Host field to “abc.com.net”, set this parameter to “%hn.net”.

new_path

This parameter specifies the Path value in the Location header of the rewritten HTTP response.



Note: If the Path does not need to be rewritten, administrators can set the value of “orig_path” and “new_path” as “/”.

no http rewrite response url <virtual_service> <policy_name>

This command is used to delete an HTTP response rewrite rule configuration for the specified virtual service.

show http rewrite response url [virtual_service]

This command is used to display the HTTP response rewrite rule configurations for the specified virtual service. If the “virtual_service” parameter is not specified, the HTTP response rewrite rule configurations for all virtual services will be displayed.

clear http rewrite response url <virtual_service>

This command is used to clear the HTTP response rewrite rule configurations for the specified virtual service.

virtual_service

This parameter specifies the virtual service name. Its value must be:

- Virtual service name: clears the HTTP response rewrite rule configurations for this virtual service.
- “all”: clears the HTTP response rewrite rule configurations for all virtual services.

http rewrite response https <virtual_service>

This command is used to enable the function of redirecting HTTP responses to HTTPS for the specified virtual service. Upon receiving an HTTP response, the appliance will change the Protocol value in the Location header of the HTTP response to “https”.

virtual_service

This parameter specifies the virtual service name. Its value must be an HTTP or HTTPS virtual service.

no http rewrite response https <virtual_service>

This command is used to disable the function of redirecting HTTP responses to HTTPS for the specified virtual service.

show http rewrite response https

This command is used to display the virtual services for which the function of redirecting HTTP responses to HTTPS is enabled.

clear http rewrite response https

This command is used to disable the function of redirecting HTTP responses to HTTPS for all virtual services.

http rewrite response port <virtual_service> <rewrite_action>

This command is used to enable the function of rewriting the port number in the Location header of the HTTP response.

virtual_service This parameter specifies the virtual service name. It must be an HTTP or HTTPS virtual service.

rewrite_action This parameter specifies the rewrite action. Its value must be “remove”, indicating that the port number will be deleted.

no http rewrite response port <virtual_service>

This command is used to disable the function of rewriting the port number in the Location header of the HTTP response for the specified virtual service.

show http rewrite response port [virtual_service]

This command is used to display the status of the function of rewriting the port number in the Location header of the HTTP response for the specified virtual service. If the “virtual_service” parameter is not specified, the status of the function of rewriting the port number for all virtual services will be displayed.

clear http rewrite response port <virtual_service>

This command is used to disable the function of rewriting the port number in the Location header of the HTTP response for the specified virtual service.

virtual_service This parameter specifies the virtual service name. Its value must be:

- Virtual service name: disables the function of rewriting the port number in the Location header of the HTTP response for this virtual service.

- “all”: disables the function of rewriting the port number in the Location header of the HTTP response for all virtual services.

➤ Adding and Deleting Specified Header Information

The appliance can be configured to add or delete some specific header information before forwarding HTTP requests or responses, so as to provide clients or real services with the specified information or hide the specified information from clients.

http xforwardedfor on *[virtual_service] [mode] [customized_name] [chain] [port_support]*

This command is used to enable the function of inserting the client IP address into HTTP requests for the specified virtual service. After this function is enabled, the appliance will transfer client IP addresses to real services. By default, this function is disabled globally and enabled per virtual service. This function takes effect for the specified virtual service only when it is enabled both globally and for the virtual service.

When the “**slb forwardip**” command is also configured, the client IP addresses obtained from the TCP option in the client’s handshake ACK message will also be forwarded to the real services. If there is no client IP address in the TCP option or IP extraction fails, only the source IP address in the IP header will be forwarded.

virtual_service Optional. This parameter specifies the virtual service name. Its value must be an HTTP or HTTPS virtual service. If this parameter is not specified, the function of inserting the client IP address into HTTP requests is enabled globally.

mode Optional. This parameter specifies the mode of transferring client IP addresses to real services. Its value must be:

- **header**: inserts an HTTP header to transfer the client IP address.
- **url**: inserts a URL query string to transfer the client IP address.
- **cookie**: inserts an HTTP cookie to transfer the client IP address.
- **all**: indicates all of the methods above.

The default value is “header”.

customized_name Optional. This parameter specifies the customized name for the client IP address in the inserted HTTP header, URL query string, or HTTP cookie.

The default value is “X-Forwarded-For”.

If the “mode” parameter is set to “header” or “all”, the

“customized_name” parameter can only be the customized name or the standard header name “X-Forwarded-For”, but not other standard header name or the header name specified in the “**http rewrite request removeheader** <virtual_service> <header_name>” command.

chain Optional. This parameter is used to enable the function of inserting the client’s IP address into an existing field. Its value must be “chain”, which indicates that the client’s IP address will be added to an existing field. This parameter is available only when the “mode” parameter is set to “header” or “all”.

The default value is the double quote symbol (“”), indicating that this function is disabled.

port_support Optional. This parameter specifies whether to transfer the client’s port number together with the client’s IP address to the real service. Its value must be “ipport”, which indicates that the client’s port number will be transferred to the real service together with the client’s IP address. If this parameter is not specified, only the client’s IP address will be transferred to the real service.

The default value is empty.

http xforwardedfor off [virtual_service]

This command is used to disable the function of inserting the client IP address into HTTP requests for the specified virtual service. If the “virtual_service” parameter is not specified, this function is disabled globally.

show http xforwardedfor

This command is used to display the global status of the function of inserting the client IP address into HTTP requests and its status for individual virtual services.

http owa {on|off}

This command is used to enable or disable the Outlook Web Access (OWA) subsystem function. After this function is enabled, the appliance will insert “FRONT-END-HTTPS: on” into the HTTP requests bound for the virtual service specified by the “http owa virtual” command. By default, this function is disabled.

show http owa status

This command is used to display the status of the OWA subsystem function.

http owa virtual <virtual_service>

This command is used to enable the function of inserting “FRONT-END-HTTPS: on” into HTTP requests for the specified virtual service.

virtual_service This parameter specifies the virtual service name. Its value must be an HTTP or HTTPS virtual service.

no http owa virtual <virtual_service>

This command is used to disable the function of inserting “FRONT-END-HTTPS: on” into HTTP requests for the specified virtual service.

show http owa virtual

This command is used to display the virtual services for which the function of inserting “FRONT-END-HTTPS: on” into HTTP requests is enabled.

clear http owa virtual

This command is used to disable the function of inserting “FRONT-END-HTTPS: on” into HTTP requests for all virtual services.

http serverpersist {on|off}

This command is used to globally enable or disable the function of keeping persistent connections to real services. After this function is enabled, the appliance will insert the Keep-Alive header into HTTP requests before forwarding them to real services. Then the connections to real services will persist. By default, this function is enabled.



Note: The function of this command is invalid for the HTTP/2 protocol. After enabling the “**http serverpersist**” function, sending HTTP/2 requests will not insert the Keep-Alive header.

http serverpersist real <real_service> off

This command is used to disable the function of keeping persistent connections to the specified real service.

real_service This parameter specifies the real service name. Its value must be an HTTP or HTTPS virtual service.

no http serverpersist real <real_service> off

This command is used to enable the function of keeping persistent connections to the specified real service.

show http serverpersist

This command is used to display the status of the function of keeping persistent connections to real services.

clear http serverpersist

This command is used to restore the function of keeping persistent connections to real services.

http modifyheader keepalive {on|off}

This command is used to enable or disable the function of inserting “Keep-Alive: timeout=15, max=100” into HTTP responses returned from real services. After this function is enabled, the appliance will insert “Keep-Alive: timeout=15, max=100” into HTTP responses returned from real services when either of the following conditions is met:

- The client uses HTTP/1.1 and the client request does not contain “Connection: Close”.
- The client uses HTTP/1.0 and the client request contains “Connection: Keep-Alive”.



Note: The HTTP Connection field passthrough function does not support “http modifyheader keepalive on”.

show http modifyheader keepalive

This command is used to display the status of the function of inserting “Keep-Alive: timeout=15, max=100” into HTTP responses returned from real services.

http rewrite response cookie secure {on|off} [virtual_service]

This command is used to enable or disable the function of securely applying cookies returned from the real service. After this function is enabled, the appliance will add a “secure” tag to the Set-Cookie header of the received HTTP response to prevent the client from transferring the cookie to unsecure connections. By default, this function is disabled globally and enabled per virtual service. This function takes effect for the specified virtual service only when it is enabled both globally and for the virtual service.

When this function is globally disabled, it does not take effect for any specific virtual services.

<code>virtual_service</code>	Optional. This parameter specifies the virtual service name. Its value must be an HTTPS virtual service. If this parameter is not specified, the function is globally enabled or disabled.
------------------------------	--

http rewrite response cookie secure icoookie {on|off} [virtual_service]

This command is used to enable or disable the function of securely applying the cookies inserted by the appliance for the specified virtual service. After this function is enabled, the Set-Cookie header inserted by the appliance will carry the Secure attribute to prevent the client from transferring the cookie to unsecure connections. For this function to take effect, the “**http rewrite response cookie secure on**” command must be executed. By default, this function is enabled both globally and per virtual service. When this function is globally disabled, it does not take effect for any specific virtual services.

virtual_service	Optional. This parameter specifies the virtual service name. Its value must be an HTTPS virtual service. If this parameter is not specified, the function is globally enabled or disabled.
-----------------	--

show http rewrite response cookie secure

This command is used to display the global status of the functions of securely applying cookies sent from the real service and cookies inserted by the appliance and their status for individual virtual services.

clear http rewrite response cookie secure

This command is used to restore the global status of the functions of securely applying cookies returned from the real service and cookies inserted by the appliance and disable these functions for individual virtual services.

http rewrite response cookie samesite on [virtual_service] [cookie_value]

This command is used to enable the SameSite function for the specified virtual service and configure the virtual service to insert the specified SameSite value into cookies returned from real services.

To enable the SameSite function for the global, execute the “**http rewrite response cookie samesite on**” command without any parameter values.

To configure a SameSite cookie value for all virtual services, execute the “**http rewrite response cookie samesite on all**” command with the required “cookie_value” parameter.

By default, the SameSite function is enabled for the global, and disabled for single virtual services. The “cookie_value” setting is empty for both the global and single virtual services. If the SameSite function is disabled for the global, SameSite configurations for single virtual services do not take effect.

virtual_service	Optional. This parameter specifies the virtual service name. Its value must be an HTTP or HTTPS virtual service.
-----------------	--

To configure the same SameSite value for all virtual services, set this parameter to “all”.

cookie_value	Optional. This parameter specifies the value of the SameSite attribute. Its value must be: • strict: only attaches cookies for “same-site” requests. • lax: sends cookies for “same-site” requests and “cross-site” top-level navigations using a safe HTTP method. • none: sends cookies for all “same-site” and “cross-site” requests. When the “samesite=none” attribute is inserted, the Secure attribute will also be appended. When “samesite=none” is specified, the virtual service must be of the HTTPS type.
--------------	---

The default value is “none”.

http rewrite response cookie samesite off [virtual_service]

This command is used to delete the “**http rewrite response cookie samesite**” configuration of the specified virtual service. If the “virtual_service” parameter is not specified, the command will disable the SameSite function for the global.

show http rewrite response cookie samesite

This command is used to display the configuration of the “**http rewrite response cookie samesite**” command.

clear http rewrite response cookie samesite

This command is used to restore the configuration of the “**http rewrite response cookie samesite**” command to default.

http rewrite response cachecontrol {on|off} [virtual_service]

This command is used to enable or disable the function of rewriting the Cache-Control header of HTTP responses for the specified virtual service. By default, this function is disabled globally and enabled per virtual service. This function takes effect for the specified virtual service only when it is enabled both globally and for this virtual service. The appliance will rewrite the Cache-Control header of HTTP responses for this virtual service based on the rule configured using the “http rewrite response cachecontrol rule” command.

virtual_service	Optional. This parameter specifies the virtual service name. Its value must be an HTTP or HTTPS virtual service. If this parameter is not specified, the function is globally enabled or disabled.
-----------------	---

show http rewrite response cachecontrol status

This command is used to display the status of the function of rewriting the Cache-Control header of HTTP responses.

http rewrite response cachecontrol rule *<virtual_service>* *<url_regex>*
<cache_directives> *<priority>*

This command is used to configure a rule for rewriting the Cache-Control header of HTTP responses for the specified virtual service. When an HTTP response matches the specified URL, the appliance will add a Cache-Control header with a value specified by the “cache-directives” parameter into the HTTP response.

For appliances with 1 GB or 2 GB memory, a maximum of 200 rules for rewriting the Cache-Control header of the HTTP response can be configured. For appliances with 4 GB memory or above, a maximum of 400 rules as such can be configured.

virtual_service This parameter specifies the virtual service name. Its value must be an HTTP or HTTPS virtual service.

url_regex This parameter specifies the URL regular expression to be matched. Its value must be a quick regular expression string of 1 to 128 characters, enclosed in double quotes.

For information about the quick regular expression, please refer to the “Regular Expressions” section in Chapter 1.

cache_directives This parameter specifies the directive of the Cache-Control header to be added. Its value must be a string of 1 to 64 characters. The supported value types are as follows:

Standard directive name: “public”, “private”, “no-cache”, “no-store”, “no-transform”, “must-revalidate”, “proxy-revalidate”, “max-age”, “s-maxage”, and “cache-extension”. For details, see RFC 2616.

Non-standard directive not defined in RFC.

This parameter value can contain one or more directives. When multiple directives are specified, they should be separated using “,” and enclosed in double quotes.

Example:

“no-cache,must-revalidate” indicates that the web page will not be cached. To reopen the web page, the client must re-access the real service.

priority This parameter specifies the priority of the rule. Its value must be an integer ranging from 0 to 65,535. A higher value indicates a higher priority.

no http rewrite response cachecontrol rule *<virtual_service>* *<url_regex>*

This command is used to delete a rule configuration for rewriting the Cache-Control header of HTTP responses for the specified virtual service.

show http rewrite response cachecontrol rule [virtual_service]

This command is used to display the rule configurations for rewriting the Cache-Control header of HTTP responses for the specified virtual service. If the “virtual_service” parameter is not specified, the rule configurations for rewriting the Cache-Control header of HTTP responses for all virtual services will be displayed.

clear http rewrite response cachecontrol [virtual_service]

This command is used to clear the rule configurations for rewriting the Cache-Control header of HTTP responses for the specified virtual service. If the “virtual_service” parameter is not specified, the rule configurations for rewriting the Cache-Control header of HTTP responses for all virtual services will be cleared.

http mask server {on|off}

This command is used to enable or disable the function of hiding real service information from clients. After this function is enabled, the appliance will remove the Server header from the responses to clients. By default, this function is disabled.

http mask via {on|off}

This command is used to enable or disable the function of hiding proxy information from clients. After this function is enabled, the appliance will remove the Via header from the responses to clients, so that the clients are unaware of the proxy process on the appliance. By default, this function is disabled.

show http mask

This command is used to display the status of the function of hiding real service information and proxy information from clients.

➤ **Adding and Deleting Arbitrary Header Information**

Administrators can use the following commands to define any header information to be added to or removed from HTTP requests and any header information to be removed from HTTP responses.

http rewrite request insertheader <virtual_service> <header_string>

This command is used to insert a header string into the HTTP requests for the specified virtual service.

When a header string to be added is already configured for the specified virtual service, this command will modify the header string configuration for the virtual service.

virtual_service This parameter specifies the virtual service name. Its value

must be an HTTP or HTTPS virtual service.

header_string This parameter specifies the header string to be inserted into HTTP requests. Its value must be a string of 2 to 500 characters enclosed in double quotes. For this parameter, the “%” symbol can be used for escaping. “%n” represents a line separator; “%q” represents a double quote; “%%” represents the percent symbol itself. **Note:** The specified header string must contain the colon symbol, for example, “FRONT-END-HTTPS: on”.

no http rewrite request insertheader <virtual_service>

This command is used to delete the configuration about inserting a header string into HTTP requests for the specified virtual service.

show http rewrite request insertheader [virtual_service]

This command is used to display the configuration about inserting a header string into HTTP requests for the specified virtual service. If the “virtual_service” parameter is not specified, the configurations about inserting a header string into HTTP requests for all virtual services will be displayed.

clear http rewrite request insertheader [virtual_service]

This command is used to clear the configuration about inserting a header string into HTTP requests for the specified virtual service. If the “virtual_service” parameter is not specified, the configurations about inserting a header string into HTTP requests for all virtual services will be cleared.

http rewrite response insertheader <virtual_service> <header_string>

This command is used to insert a header string into the HTTP responses for the specified virtual service.

When a header string to be added is already configured for the specified virtual service, this command will modify the header string configuration for the virtual service.

virtual_service This parameter specifies the virtual service name. Its value must be an HTTP or HTTPS virtual service.

header_string This parameter specifies the header string to be inserted into HTTP responses. Its value must be a string of 2 to 500 characters enclosed in double quotes. For this parameter, the “%” symbol can be used for escaping. “%n” is used for separating header strings; “%q” represents a double quote; “%%” represents the percent symbol itself. **Note:** The specified header string must contain the colon symbol, for

example, “FRONT-END-HEADER: on”.

no http rewrite response insertheader <virtual_service>

This command is used to delete the configuration about inserting a header string into HTTP responses for the specified virtual service.

show http rewrite response insertheader [virtual_service]

This command is used to display the configuration about inserting a header string into HTTP responses for the specified virtual service. If the “virtual_service” parameter is not specified, the configurations about inserting a header string into HTTP responses for all virtual services will be displayed.

clear http rewrite response insertheader <virtual_service>

This command is used to clear the configuration about inserting a header string into HTTP responses for the specified virtual service.

virtual_service	This parameter specifies the virtual service name. Its value must be: • Virtual service name: clears the configuration about inserting a header string into HTTP responses for this virtual service. • “all”: clears the configurations about inserting a header string into HTTP responses for all virtual services.
-----------------	---

http rewrite request removeheader <virtual_service> <header_name>

This command is used to remove the specified header from HTTP requests for the specified virtual service. The system supports a maximum of 42 configurations about removing a specified header from HTTP requests.

virtual_service	This parameter specifies the virtual service name. Its value must be an HTTP or HTTPS virtual service.
header_name	This parameter specifies the HTTP header to be removed. Its value must be a string of 1 to 100 characters.



Note: If the “mode” parameter in the “**http xforwardedfor on** [virtual_service] [mode] [customized_name] [chain] [port_support]” command is set to “header” or “all”, the “header_name” in this command cannot be the same as the value of the “customized_name” parameter in the “**http xforwardedfor on** [virtual_service] [mode] [customized_name] [chain] [port_support]” command.

no http rewrite request removeheader <virtual_service>

This command is used to delete the configurations about removing a header from HTTP requests for the specified virtual service.

show http rewrite request removeheader [virtual_service]

This command is used to display the configurations about removing a header from HTTP requests for the specified virtual service. If the “virtual_service” parameter is not specified, the configurations about removing a header from HTTP requests for all virtual services will be displayed.

clear http rewrite request removeheader [virtual_service]

This command is used to clear the configurations about removing a header from HTTP requests for the specified virtual service. If the “virtual_service” parameter is not specified, the configurations about removing a header from HTTP requests for all virtual services will be cleared.

http rewrite response removeheader <virtual_service> <header_name>

This command is used to remove the specified header from HTTP responses for the specified virtual service. The system supports a maximum of 42 configurations about removing a header from HTTP responses.

virtual_service	This parameter specifies the virtual service name. Its value must be an HTTP or HTTPS virtual service.
header_name	This parameter specifies the HTTP header to be removed. Its value must be a string of 1 to 100 characters.



Note: This command does not support removing the “Connection” header from the HTTP response.

no http rewrite response removeheader <virtual_service> <header_name>

This command is used to delete a configuration about removing the specified header from HTTP responses for the specified virtual service.

show http rewrite response removeheader [virtual_service]

This command is used to display the configurations about removing a header from HTTP responses for the specified virtual service. If the “virtual_service” parameter is not specified, the configurations about removing a header from HTTP responses for all virtual services will be displayed.

clear http rewrite response removeheader [virtual_service]

This command is used to clear the configurations about removing a header from HTTP responses for the specified virtual service. If the “virtual_service” parameter is not specified, the configurations about removing a header from HTTP responses for all virtual services will be cleared.

Content Rewrite

This section covers commands for configuring the HTTP content rewrite function. This function rewrites the response body returned from real services based on HTTP content rewrite rules. Administrators can define the type of files, name of files, and type of responses that need to be rewritten.

http rewrite body {on|off} [virtual_service]

This command is used to enable or disable the HTTP content rewrite function for the specified virtual service. By default, this function is disabled globally and enabled per virtual service. This function takes effect for a virtual service only when it is enabled both globally and for this virtual service.

virtual_service Optional. This parameter specifies the virtual service name.

If this parameter is not specified, the function is globally enabled or disabled.



Note:

During HTTP content rewrite, the appliance also performs the following header rewrite actions:

- Change the Content-Length header in HTTP responses to “Transfer-Encoding: chunked”. This is because the content rewrite module will divide the data into several blocks and send the blocks in batches after rewriting one or more blocks. With the “Transfer-Encoding: chunked” header, the appliance can send data to the client without knowing the total length of all data blocks or waiting until all blocks are rewritten.
- Remove the Content-Type header from the HTTP response (by changing it to Xontent-Type), and then add the original Content-Type header and its value with an extra value “charset=utf-8”. This is because the appliance encodes the file using UTF-8 while it rewrites the file. Adding this value ensures that the browser correctly displays the page that has been rewritten.
- Remove the Accept-Encoding header from the HTTP request.

show http rewrite body status

This command is used to display the global status of the HTTP content rewrite function and its status for individual virtual services.

http rewrite body rule <rule> [flags] [encode] [virtual_service]

This command is used to configure an HTTP content rewrite rule. Upon receiving an HTTP response from the real service, the appliance will rewrite the matched string as defined in the rule.

rule This parameter specifies the content rewrite rule. Two kinds of rules are supported:

- ProxyHTMLURLMap from-pattern to-pattern

The URLs inside HTML tags will be rewritten. The “from-pattern” variable specifies the source string to be rewritten. The “to-pattern” variable specifies the target string. For example, “ProxyHTMLURLMap 10.3.139.172 172.16.85.77” indicates that 10.3.139.172 in the URL address will be rewritten to 172.16.85.77. This kind of rewrite rules applies only to the rewriting of HTML and XHTML files.

- “Substitute s/from-pattern/to-pattern/”

All strings that match the rewrite rule will be rewritten. The “from-pattern” variable specifies the source string. The “to-pattern” variable specifies the target string. This rule supports rewriting only HTML files. This rule also supports Chinese and Japanese strings encoded into the Unicode format. For a Unicode string, it must be prefixed with “%u” to make a difference with normal English characters. In addition, the “encode” parameter must be defined. If the rule string contains double quotes, they must be converted to “%uQUAT”, such as “”.

The rule contains the “/” character, and therefore when the source string or target string also contains the “/” character, “Substitute s\|from-pattern\|to-pattern\|” should be used to avoid conflicts between the rule format and content.

The case of the “ProxyHTMLURLMap” and “Substitute s” rules cannot be modified, and the whole rule string must be less than or equal to 900 characters and enclosed in double quotes.

flags

Optional. This parameter specifies the match mode of the strings. Its value must be:

- “-R”: A string will be rewritten when any part of it matches the value of “from-pattern”.
- “-I”: String matching is case-insensitive.
- “-iR”: Partial match is supported, and string match is case-insensitive.
- Empty: A string will be rewritten only when it exactly matches the value of “from-pattern”. The matching is case-insensitive.

The default value is empty.

encode	Optional. This parameter specifies the encoding format. Its value must be: - gbk: rewrites GBK characters. - gb2312: rewrites GB2312 characters. - utf-8: rewrites Japanese or non-GBK/GB2312 Chinese characters. - empty: rewrites non-Chinese/Japanese characters. This parameter must be set if this command is configured for Chinese or Japanese character rewrite. The default value is null.
virtual_service	Optional. This parameter specifies the virtual service name. If it is not specified, the HTTP content rewrite rule will be applied to all virtual services. The default value is empty.

**Note:**

- When both the “ProxyHTMLURLMap” and “Substitute” rules are configured and they specify the same value of “from-pattern” and of “to-pattern”, the “ProxyHTMLURLMap” rule preferentially takes effect.
- Adding or deleting a content rewrite rule will cause the ongoing rewrite operations fail, and the appliance will reset related connections. Therefore, adding or deleting content rewrite rules during the appliance’s running time is not recommended.
- If the HTTP content rewrite function is enabled and content rewrite rules are configured, the length of each line in HTTP responses cannot be greater than 1 MB; otherwise, the appliance will send an RST packet to terminate the TCP connection.

no http rewrite body rule <rule> [flags] [encode] [virtual_service]

This command is used to delete an HTTP content rewrite rule configuration.

show http rewrite body rule

This command is used to display all configurations of HTTP content rewrite rules.

clear http rewrite body rule

This command is used to clear all configurations of HTTP content rewrite rules.

http rewrite body mimetype <mime_type>

This command is used to define the type of files to be rewritten, so the appliance will rewrite only the specified type of files. To define multiple file types, the administrator needs to repeatedly execute this command with the desired value of the “mime_type”

parameter. When this command is not configured, the appliance rewrites only text/html files.

The HTTP content rewrite function supports the rewriting of the following file types:

- text/html
- text/plain
- text/richtext
- text/xml
- application/xml
- application/xhtml+xml
- text/css
- text/javascript
- application/javascript

mime_type This parameter specifies the type of files to be rewritten. Its value must be:

- html: indicates the text/html file type.
- plain: indicates the text/plain file type.
- richtext: indicates the text/richtext file type.
- xml: indicates the text/xml and application/xml file types.
- xhtml: indicates the application/xml and application/xhtml+xml file types.
- css: indicates the text/css file type.
- js: indicates the text/javascript and application/javascript file types.

no http rewrite body mimetype <mime_type>

This command is used to delete the configurations about the specified type of files that need to be rewritten.

show http rewrite body mimetype

This command is used to display the configurations about all types of files that need to be rewritten.

clear http rewrite body mimetype

This command is used to clear the configurations about all types of files that need to be rewritten.

http rewrite body url list <url_list> <url>

This command is used to create a URL list and add the specified URL string to the list. Multiple URL strings can be added to the same URL list by repeated execution of this command. This command must be used together with the HTTP content rewrite-permitted rule (configured by the “**http rewrite body url permit**” command) and HTTP content rewrite-denied rule (configured by the “**http rewrite body url deny**” command).

For an existing URL list, this command will change the configuration of the URL list.

url_list This parameter specifies the URL list name. Its value must be a string of 1 to 40 characters. The parameter value is case-sensitive, and special characters are not supported.

url This parameter specifies the URL string to be added to the URL list. Its value must be a string of 1 to 512 characters. Generally, it is set to an extension name or file name. The parameter supports regular expressions. The following wildcards are supported:

- “^” means matching the starting characters of the URL.
- “*” means matching zero or more characters ahead of it.
- “\$” means matching the ending characters of the URL.

When the parameter value contains wildcards, it must be enclosed in double quotes.

no http rewrite body url list <url_list> <url>

This command is used to delete a URL string from the specified URL list.

show http rewrite body url list [url_list]

This command is used to display the specified URL list and the URL strings added to it. If the “url_list” parameter is not specified, all URL lists and the URL strings in these lists will be displayed.

clear http rewrite body url list [url_list]

This command is used to delete the specified URL list and its URL strings. If the “url_list” parameter is not specified, all URL lists and their URL strings will be cleared.

http rewrite body url permit <virtual_service> <url_list>

This command is used to configure an HTTP content rewrite-permitted rule based on the specified URL list for the specified virtual service. When an HTTP request matches any of the URL strings in the URL list, the appliance will rewrite the corresponding HTTP response.

virtual_service	This parameter specifies the virtual service name. Its value must be an HTTP or HTTPS virtual service.
url_list	This parameter specifies the URL list name.

no http rewrite body url permit <virtual_service> <url_list>

This command is used to delete the URL list-based HTTP content rewrite-permitted rule for the specified virtual service.

show http rewrite body url permit [virtual_service]

This command is used to display the URL list-based HTTP content rewrite-permitted rule configured for the specified virtual service. If the “virtual_service” parameter is not specified, the URL list-based HTTP content rewrite-permitted rules configured for all virtual services will be displayed.

clear http rewrite body url permit <virtual_service>

This command is used to delete the URL list-based HTTP content rewrite-permitted rule configured for the specified virtual service.

virtual_service	This parameter specifies the virtual service name. Its value must be: • Virtual service name: deletes the URL list-based HTTP content rewrite-permitted rule configured for this virtual service. • “all”: clears the URL list-based HTTP content rewrite-permitted rules configured for all virtual services.
-----------------	--

http rewrite body url deny <virtual_service> <url_list>

This command is used to configure an HTTP content rewrite-denied rule based on the specified URL list for the specified virtual service. When an HTTP request matches any of the URL strings in the URL list, the appliance will not rewrite the corresponding response. For requests that do not match the rule, the appliance will rewrite the corresponding responses.

virtual_service	This parameter specifies the virtual service name.
url_list	This parameter specifies the URL list name.



Note: Each virtual service can be associated with only one URL list-based HTTP content rewrite-permitted rule or HTTP content rewrite-denied rule.

no http rewrite body url deny <virtual_service> <url_list>

This command is used to delete the URL list-based HTTP content rewrite-denied rule configured for the specified virtual service

show http rewrite body url deny [virtual_service]

This command is used to display the URL list-based HTTP content rewrite-denied rule configured for the specified virtual service. If the “virtual_service” parameter is not specified, the URL list-based HTTP content rewrite-denied rules configured for all virtual services will be displayed.

clear http rewrite body url deny <virtual_service>

This command is used to delete the URL list-based HTTP content rewrite-denied rule for the specified virtual service.

virtual_service This parameter specifies the virtual service name. Its value must be:

- Virtual service name: deletes the URL list-based HTTP content rewrite-denied rule configured for this virtual service.
- “all”: clears the URL list-based HTTP content rewrite-denied rules configured for all virtual services.

http rewrite body statuscode <status_code>

This command is used to configure an HTTP content rewrite rule based on the specified status code. Only HTTP responses that contain the specified status code will be rewritten.

When this command is not configured, the appliance rewrites only HTTP responses that contain the 200 status code.

status_code This parameter specifies the HTTP status code. Its value must be “101”, “200”, “206”, “301”, “302”, “303”, “305”, “307”, “400”, “401”, “402”, “403”, “404”, “405”, “406”, “407”, “416”, “500”, “501”, “502”, “503”, “504” or “505”.

no http rewrite body statuscode <status_code>

This command is used to delete a status code-based HTTP content rewrite rule configuration.

show http rewrite body statuscode

This command is used to display all status code-based HTTP content rewrite rule configurations.

clear http rewrite body statuscode

This command is used to clear all status code-based HTTP content rewrite rule configurations, that is, the appliance will rewrite only HTTP responses that contain the 200 status code.

http rewrite body limit <size>

This command is used to set the upper length limit for rewritable HTTP body. When this command is not configured, the default upper length limit is 5120 KB.

size	This parameter specifies the upper length limit for rewritable HTTP body, in Kbytes. Its value must be an integer ranging from 0 to 1048576. When the value 0 is set, the system will not rewrite HTTP content.
------	---

no http rewrite body limit

This command is used to restore the upper length limit setting for HTTP content rewrite.

show http rewrite body limit

This command is used to display the upper length limit setting for HTTP content rewrite.

HTTP Security Control

Administrators can improve the security of HTTP services and protect real services against buffer overflow attacks, directory traversal attacks, and unauthorized access by:

Configuring URL filter

Configuring domain names allowed to be accessed

Configuring HTTP methods allowed to be used by clients

URL Filtering

Administrators can configure URL filter rules based on ASCII codes, the threshold for each part of HTTP requests, the value type of URL variable, control characters, and URL keyword to filter HTTP requests.

filter vip [virtual_service]

This command is used to enable the URL filtering function for the specified virtual service. By default, this function is disabled both globally and per virtual service.

virtual_service	Optional. This parameter specifies the virtual service name. Its value must be an HTTP or HTTPS virtual service. If this
-----------------	--

parameter is not specified, the function is enabled globally.

no filter vip [*virtual_service*]

This command is used to disable the URL filtering function for the specified virtual service. If the “virtual_service” parameter is not specified, this function is disabled globally.

show filter vip [*virtual_service*]

This command is used to display the status of the URL filtering function for the specified virtual service.

virtual_service Optional. This parameter specifies the virtual service name. Its value must be:

- Virtual service name: displays the status of the URL filtering function for this virtual service.
- “global”: displays the global status of the URL filtering function.
- “all”: displays both the global status of the URL filtering function and its status for individual virtual services.

The default value is “all”.

clear filter vip [*virtual_service*]

This command is used to disable the URL filtering function for the specified virtual service.

virtual_service Optional. This parameter specifies the virtual service name. Its value must be:

- Virtual service name: disables the URL filtering function for this virtual service.
- “global”: disables the global URL filtering function.
- “all”: disables the URL filtering function both globally and for individual virtual services.

The default value is “all”.

filter mode {passive|active} [*virtual_service*]

This command is used to configure the mode of processing bad HTTP requests for the specified virtual service.

Bad HTTP requests refer to the following types of requests:

- A part of an HTTP request surpasses the specified length threshold.
- The URL variable contains the specified type of value.
- Requests with a URL address that contains control characters after the filtering function based on control characters is enabled.
- The HTTP request matches the specified ASCII code range.

The appliance supports the following modes of processing bad requests:

- **passive:** Bad requests are permitted to pass through. The appliance keeps a record of the bad requests.
- **active:** Bad requests are denied. The appliance keeps a record of each denial action.

When this command is not configured, the appliance denies all bad requests and keeps a record of each denial action.

virtual_service Optional. This parameter specifies the virtual service name. Its value must be an HTTP or HTTPS virtual service. If this parameter is not specified, the global mode of processing bad requests is configured.

show filter mode *[virtual_service]*

This command is used to display the configuration about the mode of processing bad HTTP requests for the specified virtual service.

virtual_service Optional. This parameter specifies the virtual service name. Its value must be:

- **Virtual service name:** displays the configuration about the mode of processing bad HTTP request for this virtual service.
- **“global”:** displays the configuration about the global mode of processing bad HTTP requests.
- **“all”:** displays the configuration about both the global mode of processing bad HTTP requests and those for individual virtual services.

The default value is “all”.

clear filter mode *[virtual_service]*

This command is used to restore the configuration about the mode of processing bad HTTP requests for the specified virtual service.

virtual_service Optional. This parameter specifies the virtual service name.

Its value must be:

- Virtual service name: restores the configuration about the mode of processing bad HTTP requests for this virtual service.
- “global”: restores the configuration about the global mode of processing bad HTTP requests.
- “all”: restores the configuration about both the global mode of processing bad HTTP requests and those for individual virtual services.

The default value is “all”.

filter length {url|query|queryvariable|querydata|header|request} <length> [virtual_service]

This command is used to set a length threshold for each part of the HTTP request for the specified virtual service. The appliance supports the setting of a length threshold for the following parts of HTTP requests:

- url: indicates the path and query part in URL. For example, “/123.txt?test=123” in URL “www.abc.com/123.txt?test=123”.
- query: indicates the URL query part.
- queryvariable: indicates the variable name of the URL query.
- querydata: indicates the variable value of the URL query.
- header: indicates the HTTP header.
- request: indicates data in the HTTP request.

When any part of an HTTP request crosses the specified length threshold, the appliance will process the request based on the configuration of the “filter mode” command. When this command is not configured, the default length thresholds, as explained for the “length” parameter, will be used

length	This parameter specifies the length threshold for each part of the HTTP request:
--------	--

- URL: Its value must be an integer ranging from 0 to 20,480 in bytes. The default value is 1024.
- query: Its value must be an integer ranging from 0 to 20,480 in bytes. The default value is 1024.
- queryvariable: Its value must be an integer ranging from 0 to 20,480 in bytes. The default value is 128.
- querydata: Its value must be an integer ranging from 0 to 20,480 in bytes. The default value is 512.
- header: Its value must be an integer ranging from 0 to

204,800 in bytes. The default value is 1024.

- request: Its value must be an integer ranging from 0 to 4,294,967,295 in bytes. The default value is 10,000.

virtual_service

Optional. This parameter specifies the virtual service name. Its value must be an HTTP or HTTPS virtual service. If this parameter is not specified, the global length threshold for each part of the HTTP request is set.



Note: For packets transmitted using HTTP/2, the existence of the PRIORITY field in the packet header will increase the header length by 5 bytes, PADDED field by 1 byte, but the increased header length will not be counted into the total header length judged by the “**filter length header**” command.

show filter length [*virtual_service*]

This command is used to display the length threshold setting for each part of the HTTP request for the specified virtual service.

virtual_service

Optional. This parameter specifies the virtual service name. Its value must be:

- Virtual service name: displays the length threshold setting for each part of the HTTP request for this virtual service.
- “global”: displays the global length threshold setting for each part of the HTTP request.
- “all”: displays both the global length threshold setting for each part of the HTTP request and those for individual virtual services.

The default value is “all”.

clear filter length [*virtual_service*]

This command is used to restore the length threshold setting for each part of the HTTP request for the specified virtual service.

virtual_service

Optional. This parameter specifies the virtual service name. Its value must be:

- Virtual service name: restores the length threshold setting for each part of the HTTP request for this virtual service.
- “global”: restores the global length threshold setting for each part of the HTTP request.
- “all”: restores both the global length threshold setting for each part of the HTTP request and those for

individual virtual services.

The default value is “all”.

filter type {integer|string} <variable_name> [virtual_service]

This command is used to configure a variable-based URL filter rule. This rule will check whether the URL variable’s value matches the specified value type. The following value types are supported:

- integer
- string

If the value of URL variable matches the specified value type, the appliance will process the request based on the configuration of the “**filter mode**” command.

variable_name	This parameter specifies the variable name. Its value must be a case-sensitive string of 1 to 127 characters.
virtual_service	Optional. This parameter specifies the virtual service name. Its value must be an HTTP or HTTPS virtual service. If this parameter is not specified, a global variable-based URL filter rule is configured.

no filter type {integer|string} <variable_name> [virtual_service]

This command is used to delete a variable-based URL filter rule configuration for the specified virtual service. If the “virtual_service” parameter is not specified, a global variable-based URL filter rule configuration will be deleted.

show filter type {integer|string} [virtual_service]

This command is used to display the variable-based URL filter rule configuration for the specified virtual service.

virtual_service	Optional. This parameter specifies the virtual service name. Its value must be: • Virtual service name: displays the variable-based URL filter rule configuration for this virtual service. • “global”: displays the global variable-based URL filter rule configuration. • “all”: displays both the global variable-based URL filter rule configuration and those for individual virtual services.
-----------------	---

The default value is “all”.

clear filter type {integer|string} [virtual_service]

This command is used to clear the variable-based URL filter rule configuration for the specified virtual service.

virtual_service Optional. This parameter specifies the virtual service name. Its value must be:

- Virtual service name: clears the variable-based URL filter rule configuration for this virtual service.
- “global”: clears the global variable-based URL filter rule configuration.
- “all”: clears both the global variable-based URL filter rule configuration and those for individual virtual services.

The default value is “all”.

filter request controlchar {on|off}

This command is used to enable or disable the URL filtering function based on control characters (non-escapable characters). By default, this function is enabled.

After this function is enabled, the appliance will check whether the URL contains control characters. If yes, the request is considered bad, and the appliance will process the request based on the configuration of the “**filter mode**” command:

- If “**filter mode active**” is configured, the request is denied.
- If “**filter mode passive**” is configured, the appliance will keep the control characters unchanged and directly forward the request.

After this function is disabled:

- For URLs containing escapable characters, the appliance will directly translate these characters based on the translation rules.
- For URLs containing control characters, the appliance will keep them unchanged.

Permitted escaping patterns are as follows:

- %XX: The range of “XX” is 00 to FF, excluding the range of 00 to 1F and 7F.
- %uXXXX: The range of “XXXX” is 0000 to FFFF.

The following table lists some translation examples when this function is enabled:

URL\Mode	filter mode active	filter mode passive
http://abc.com (The URL does not contain control characters.)	The appliance permits the request to pass through.	The appliance permits the request to pass through.
http://abc.com/%00	The appliance will deny	The appliance keeps the

URL\Mode	filter mode active	filter mode passive
http://abc.com/%1F (Both “00” and “1F” are control characters.)	both requests.	control characters unchanged and directly forwards the requests.
http://abc.com/%u1234 (The URL does not contain control characters.)	The appliance permits the request to pass through.	The appliance permits the request to pass through.

show filter request controlchar

This command is used to display the status of the control character-based URL filtering function.

filter url character <start_ascii_value> <end_ascii_value> [virtual_service]

This command is used to configure an ASCII-based URL filter rule for the specified virtual service.

start_ascii_value	This parameter specifies the starting code of the ASCII code range to be matched. Its value must be an integer ranging from 0 to 255.
end_ascii_value	This parameter specifies the ending code of the ASCII code range to be matched. Its value must be an integer ranging from 0 to 255.
virtual_service	Optional. This parameter specifies the virtual service name. Its value must be an HTTP or HTTPS virtual service. If this parameter is not specified, a global ASCII-based URL filter rule is configured.

no filter url character <start_ascii_value> <end_ascii_value> [virtual_service]

This command is used to delete an ASCII-based URL filter rule configuration for the specified virtual service. If the “virtual_service” parameter is not specified, a global ASCII-based URL filter rule configuration will be deleted.

show filter url character [virtual_service]

This command is used to display the ASCII-based URL filter rule configurations for the specified virtual service.

virtual_service	Optional. This parameter specifies the virtual service name. Its value must be: - Virtual service name: displays the ASCII-based URL filter rule configuration for this virtual service. “global”: displays the global ASCII-based URL filter rule configuration. “all”: displays both the global ASCII-based URL filter
-----------------	--

rule configurations and those for individual virtual services.

The default value is “all”.

clear filter url character [*virtual_service*]

This command is used to clear the ASCII-based URL filter rule configurations for the specified virtual service.

virtual_service Optional. This parameter specifies the virtual service name. Its value must be:

- Virtual service name: clears the ASCII-based URL filter rule configurations for this virtual service.
- “global”: clears the global ASCII-based URL filter rule configurations.
- “all”: clears both the global ASCII-based URL filter rule configurations and those for individual virtual services.

The default value is “all”.

filter url keyword default {permit|deny} [*virtual_service*]

This command is used to configure a default URL keyword-based filter rule for the specified virtual service.

This command is used in together with the “filter url keyword” command. When the default URL keyword-based filter rule needs to be modified, the administrator must make sure no URL keyword-based filter rule has been configured by the “**filter url keyword**” command.

When the default URL keyword-based filter rule is not configured, the appliance permits HTTP requests that do not match the URL keyword-based filter rule configured using the “filter url keyword” command to pass through.

virtual_service Optional. This parameter specifies the virtual service name. Its value must be an HTTP or HTTPS virtual service. If this parameter is not specified, a global default URL keyword-based filter rule is configured.

show statistics filter url keyword default [*virtual_service*]

This command is used to display the total hits of the default URL keyword-based filter rule for the specified virtual service.

virtual_service Optional. This parameter specifies the virtual service name.

Its value must be:

- Virtual service name: displays the total hits of the default URL keyword-based filter rule for this virtual service.
- “global”: displays the total hits of the global default URL keyword-based filter rule.
- “all”: displays both the total hits of the global default URL keyword-based filter rule and those for individual virtual services.

The default value is “all”.

clear statistics filter url keyword default [*virtual_service*]

This command is used to clear the total hits of the default URL keyword-based filter rule for the specified virtual service.

virtual_service Optional. This parameter specifies the virtual service name. Its value must be:

- Virtual service name: clears the total hits of the default URL keyword-based filter rule for this virtual service.
- “global”: clears the total hits of the global default URL keyword-based filter rule.
- “all”: clears the total hits of both the global default URL keyword-based filter rule and those for individual virtual services.

The default value is “all”.

filter url keyword {permit|deny} <string> [*virtual_service*]

This command is used to configure a URL keyword-based filter rule for the specified virtual service. This command is used in together with the “**filter url keyword default**” command:

- If the default URL keyword-based filter rule is configured as “filter url keyword default permit”, the “filter url keyword” command must be configured with the “deny” option.
- If the default URL keyword-based filter rule is configured as “filter url keyword default deny”, the “filter url keyword” command must be configured with the “permit” option.

string This parameter specifies the URL keyword to be matched. Its value must be a string of 1 to 127 characters. This parameter value supports Perl-based regular expression. For example, “*” means matching zero or more characters

ahead of it, which is different from its meaning in wildcard expression. If the “*” symbol needs to be matched, “*” must be used for translation. A single “*” should be avoided in Perl-based regular expression. “.*” in regular expression is used to match all URLs, which is the same as “*” in wildcard expression.

virtual_service Optional. This parameter specifies the virtual service name. Its value must be an HTTP or HTTPS virtual service. If this parameter is not specified, a global URL keyword-based filter rule is configured.

no filter url keyword {permit|deny} <string> [virtual_service]

This command is used to delete a URL keyword-based filter rule configuration for the specified virtual service. If the “virtual_service” parameter is not specified, the global URL keyword-based filter rule configuration will be deleted.

show filter url keyword [virtual_service]

This command is used to display the URL keyword-based filter rule configuration for the specified virtual service.

virtual_service Optional. This parameter specifies the virtual service name. Its value must be:

- Virtual service name: displays the URL keyword-based filter rule configuration for this virtual service.
- “global”: displays the global URL keyword-based filter rule configuration.
- “all”: displays both the global URL keyword-based filter rule configuration and those for individual virtual services.

The default value is “all”.

clear filter url keyword [virtual_service]

This command is used to clear the URL keyword-based filter rule configuration for the specified virtual service.

virtual_service Optional. This parameter specifies the virtual service name. Its value must be:

- Virtual service name: clears the URL keyword-based filter rule configuration for this virtual service.
- “global”: clears the global URL keyword-based filter rule configuration.

- “all”: clears both the global URL keyword-based filter rule configuration and those for individual virtual services.

The default value is “all”.

show statistics filter url keyword {deny|permit} [string] [virtual_service]

This command is used to display the total hits of the URL keyword-based filter rule for the specified virtual service.

virtual_service Optional. This parameter specifies the virtual service name. Its value must be:

- Virtual service name: displays the total hits of the URL keyword-based filter rule for this virtual service.
- “global”: displays the total hits of the global URL keyword-based filter rule.
- “all”: displays the total hits of both the global URL keyword-based filter rule and those for individual virtual services.

The default value is “all”.

clear statistics filter url keyword {deny|permit} [string] [virtual_service]

This command is used to clear the total hits of the URL keyword-based filter rule for the specified virtual service.

virtual_service Optional. This parameter specifies the virtual service name. Its value must be:

- Virtual service name: clears the total hits of the URL keyword-based filter rule for this virtual service.
- “global”: clears the total hits of the global URL keyword-based filter rule.
- “all”: clears the total hits of both the global URL keyword-based filter rule and those for individual virtual services.

The default value is “all”.

filter url keyword match <string> [virtual_service]

This command is used to check whether the specified keyword matches the URL keyword-based filter rule for the specified virtual service, which helps confirm the correctness of the configured URL keyword.

string	This parameter specifies the URL keyword to be checked.
virtual_service	Optional. This parameter specifies the virtual service name. If this parameter is not specified, this command will check whether the specified keyword matches the global URL keyword-based filter rule.

filter alert <email_address> <threshold> [virtual_service]

This command is used to configure an alert rule. When the number of dropped HTTP requests reaches the specified threshold, the appliance will send an alert mail to the specified Email address. Before this command is configured, the “ip nameserver <ip>” command must be executed to configure a DNS server; otherwise, the appliance may fail to send alert mails.

email_address	This parameter specifies the Email address to which the alert mails will be sent. Its value must be a string of 1 to 63 characters enclosed in double quotes.
threshold	This parameter specifies the threshold for the number of dropped HTTP requests. Its value must be an integer ranging from 0 to 2,147,483,647. When the value 0 is specified, the appliance will send an alert mail whenever an HTTP request is processed, regardless of whether the request is permitted or dropped.
virtual_service	Optional. This parameter specifies the virtual service name. Its value must be an HTTP or HTTPS virtual service. If this parameter is not specified, a global alert rule is configured.

show filter alert [virtual_service]

This command is used to display the alert rule configuration for the specified virtual service.

virtual_service	Optional. This parameter specifies the virtual service name. Its value must be: - Virtual service name: displays the alert rule configuration for this virtual service. “global”: displays the global alert rule configuration. “all”: displays both the global alert rule configuration and those for individual virtual services. The default value is “all”.
-----------------	---

clear filter alert [virtual_service]

This command is used to clear the alert rule configuration for the specified virtual service.

virtual_service Optional. This parameter specifies the virtual service name. Its value must be:

- Virtual service name: clears the alert rule configuration for this virtual service.
- “global”: clears the global alert rule configuration.
- “all”: clears both the global alert rule configuration and those for individual virtual services.

The default value is “all”.

show filter all

This command is used to display all configurations related to the URL filtering function.

Access Control

http permit host <host_name>

This command is used to configure a domain name allowed to be accessed. After at least one domain name is configured by using this command, other domain names that are not configured using this command cannot be accessed. When this command is not configured, all domain names can be accessed by default.

host_name This parameter specifies the domain name or IP addresses to be added. When its value is a domain name composed of pure numbers or an IP address, it must be enclosed in double quotes.

no http permit host <host_name>

This command is used to remove the specified domain name from the list of domain names allowed to be accessed. After the last domain name allowed to be accessed is removed from the list, clients can access all domain names.

show http permit host

This command is used to display the domain names allowed to be accessed.

clear http permit host

This command is used to clear all domain names allowed to be accessed. After this command is executed, clients are allowed to access all domain names.

http permit method <method> [vip]

This command is used to configure the permitted HTTP method for the specified virtual IP address. After at least one permitted HTTP method is configured by using this command, HTTP requests that attempt to access by using other HTTP methods will be denied. For HTTP requests that access other virtual IP addresses, the globally permitted HTTP methods apply.

method	This parameter specifies the permitted HTTP method. Its value must be “get”, “post”, “patch”, “put”, “delete”, “trace”, “connect”, “options”, “head”, “propfind”, “proppatch”, “mkcol”, “copy”, “move”, “lock”, “unlock”, “purge”, “rpc_in_data”, “rpc_out_data”, “rdg_in_data”, or “rdg_out_data”.
vip	Optional. This parameter specifies the virtual IP address. The default value is “0.0.0.0”, indicating that the globally permitted HTTP method is configured.



Note: When assign “rdg_in_data” or “rdg_out_data” HTTP method to a virtual IP, please make sure that the virtual host with which the virtual IP is associated is using the same certificate as that of the related real server. Meanwhile, administrators must use command “**http mask via on**” to enable the function of hiding real service information from clients.

no http permit method <method> [vip]

This command is used to delete the permitted HTTP method configuration for the specified virtual IP address. If the “vip” parameter is not specified, the configuration of the globally permitted HTTP methods will be deleted.

show http permit method [vip]

This command is used to display the configuration of the permitted and forbidden HTTP methods for the specified virtual IP address.

vip	Optional. This parameter specifies the virtual IP address. Its value must be: • Virtual IP address: displays the configuration of the permitted and forbidden HTTP methods for this virtual IP address. • “0.0.0.0”: displays the configuration of the globally permitted and forbidden HTTP methods. • Empty: displays both the configuration of the globally permitted and forbidden HTTP methods and those for individual virtual IP addresses. The default value is empty.
-----	--

clear http permit method [vip]

This command is used to restore the configuration of the permitted and forbidden HTTP methods for the specified virtual IP address.

vip	Optional. This parameter specifies the virtual IP address. Its value must be: • Virtual IP address: restores the configuration of the permitted and forbidden HTTP methods for this virtual IP address. • “0.0.0.0”: restores the configuration of the globally permitted and forbidden HTTP methods. • Empty: restores both the configuration of the globally permitted and forbidden HTTP methods and those for individual virtual IP addresses. The default value is empty.
-----	--

HTTP Error Processing

http import error <error_code> <virtual_service> <url>

This command is used to import a customized HTTP error page for the specified virtual service. Its size should not exceed 10 MB. After a customized HTTP error page is imported for a virtual service, the “http error” command must be executed to enable the error page for this virtual service. When a response returned from the real service or generated by the appliance matches the specified error code, the appliance will send the customized error page instead of the received or generated response to the client.

error_code	This parameter specifies the HTTP error code. Its value must be an integer ranging from 400 to 599.
virtual_service	This parameter specifies the virtual service name. Its value must be: • Virtual service name: imports a customized HTTP error page for this virtual service. Its value must be an TCP, TCPS, HTTP or HTTPS virtual service. • “default”: imports a global customized HTTP error page. Note: In the segment mode, this parameter cannot be set to default.
url	This parameter specifies the path of the customized error page. Its value must be an FTP-type or HTTP-type URL.

Note: The “path” of the FTP URL (ftp://user:password@host:port/path) should be a relative path.

**Note:**

- To ensure a proper display of the HTTP error page, please change its encoding to “UTF-8 without BOM” before importing it.
- On WebUI, in addition to URL, the system also supports importing error pages through local files.
- When the **virtual_service** value is a TCP or TCPS virtual service, the **error_code** value must be 503.

no http import error <error_code> <virtual_service>

This command is used to delete a customized error page that has been imported for the specified virtual service.

virtual_service This parameter specifies the virtual service name. Its value must be:

- Virtual service name: deletes a customized HTTP error page that has been imported for this virtual service.
- “default”: deletes a global customized HTTP error page that has been imported.

Note: In the segment mode, this parameter cannot be set to **default**.

show http import error [error_code] [virtual_service]

This command is used to display the content of a customized HTTP error page for the specified virtual service. The “virtual_service” parameter is available only when the “error_code” parameter is set. If the “error_code” and “virtual_service” parameters are not specified, all customized HTTP error pages that have been imported will be displayed.

- When the “error_code” parameter is set to an error code:
- If the “virtual_service” parameter is set to a virtual service name, this command displays the content of the specified customized HTTP error page imported for this virtual service.
- If the “virtual_service” parameter is set to “default”, this command displays the content of the specified global customized HTTP error page that has been imported.
- If the “virtual_service” parameter is not specified, this command displays both the global configuration about the specified error code and those for individual virtual services.

When the “error_code” parameter is set to 0:

- If the “virtual_service” parameter is set to a virtual service name, this command displays all customized HTTP error pages imported for this virtual service.
- If the “virtual_service” parameter is set to “default”, this command displays all global customized HTTP error pages that have been imported.
- If the “virtual_service” parameter is not specified, this command displays all customized HTTP error pages that have been imported.

clear http import error [error_code] [virtual_service]

This command is used to delete a customized HTTP error page that has been imported for the specified virtual service. The “virtual_service” parameter is available only when the “error_code” parameter is set. If the “error_code” and “virtual_service” parameters are not specified, all HTTP error pages that have been imported will be cleared.

When the “error_code” parameter is set to an error code:

- If the “virtual_service” parameter is set to a virtual service name, this command deletes the specified customized HTTP error page that has been imported for this virtual service.
- If the “virtual_service” parameter is set to “default”, this command deletes the specified global customized HTTP error page that has been imported.
- If the “virtual_service” parameter is not specified, this command clears all customized HTTP error pages that have been imported with the specified error code.

When the “error_code” parameter is set to 0:

- If the “virtual_service” parameter is set to a virtual service name, this command clears all customized HTTP error pages that have been imported for this virtual service.
- If the “virtual_service” parameter is set to “default”, this command clears all global customized HTTP error pages that have been imported.
- If the “virtual_service” parameter is not specified, this command clears all customized HTTP error pages that have been imported.

http error <error_code> <virtual_service>

This command is used to enable a customized HTTP error page for the specified virtual service. A maximum of 30 customized HTTP error pages can be enabled for each virtual service.

error_code	This parameter specifies the HTTP error code. Its value must be the same as that specified in the corresponding “http import error” command.
------------	--

virtual_service	This parameter specifies the virtual service name. Its value must be: - Virtual service name: enables a customized HTTP error page for this virtual service. “default”: enables a global customized HTTP error page. Note: In the segment mode, this parameter cannot be set to default.
-----------------	---

**Note:**

- A virtual service will use the global customized HTTP error page only when no customized HTTP error page is enabled for it.
- A virtual service will use the default HTTP error page when no customized HTTP error page is enabled globally and for it.
- To replace a customized HTTP error page with a new one that is imported using the “**http import error**” command, you must first execute the “**no http error**” or “**clear http error**” command to disable the existing one and then the “**http error**” command to enable the new one.

no http error <error_code> <virtual_service>

This command is used to disable a customized HTTP error page for the specified virtual service.

virtual_service	This parameter specifies the virtual service name. Its value must be: - Virtual service name: disables a customized HTTP error page for this virtual service. “default”: disables a global customized HTTP error page. Note: In the segment mode, this parameter cannot be set to default.
-----------------	---

show http error [error_code] [virtual_service]

This command is used to display the content of a customized HTTP error page that has been enabled for the specified virtual service. The “virtual_service” parameter is available only when the “error_code” parameter is set. If the “error_code” and “virtual_service” parameters are not specified, all customized HTTP error pages that have been enabled will be displayed.

When the “error_code” parameter is set to an error code:

- If the “virtual_service” parameter is set to a virtual service name, this command displays the content of the specified customized HTTP error page for this virtual service.
- If the “virtual_service” parameter is set to “default”, this command displays the content of the specified global customized HTTP error page.
- If the “virtual_service” parameter is not specified, this command displays all virtual services for which the specified customized HTTP error page has been enabled and the global status of this customized HTTP error page.

When the “error_code” parameter is set to 0:

- If the “virtual_service” parameter is set to a virtual service name, this command displays all customized HTTP error pages that have been enabled for this virtual service.
- If the “virtual_service” parameter is set to “default”, this command displays all global customized HTTP error pages that have been enabled.
- If the “virtual_service” parameter is not specified, this command displays all customized HTTP error pages that have been enabled.

clear http error [error_code] [virtual_service]

This command is used to disable a customized HTTP error page for the specified virtual service. The “virtual_service” parameter is available only when the “error_code” parameter is set. If the “error_code” and “virtual_service” parameters are not specified, all customized HTTP error pages will be disabled.

When the “error_code” parameter is set to an error code:

- If the “virtual_service” parameter is set to a virtual service name, this command disables the specified customized HTTP error page for this virtual service.
- If the “virtual_service” parameter is set to “default”, this command disables the specified global customized HTTP error page.
- If the “virtual_service” parameter is not specified, this command disables the specified customized HTTP error page both globally and for individual virtual services.

When the “error_code” parameter is set to 0:

- If the “virtual_service” parameter is set to a virtual service name, this command disables all customized HTTP error pages for this virtual service.
- If the “virtual_service” parameter is set to “default”, this command disables all global customized HTTP error pages.
- If the “virtual_service” parameter is not specified, this command disables all customized HTTP error pages.

**http redirect error <error_code> <virtual_service> <redirect_url> [prefix]
[encoding_mode]**

This command is used to configure an error code-based URL redirect rule. When the specified error code is generated for the specified virtual service, the appliance will return an HTTP redirect response and uses the specified URL in its Location header. The matching error code can be one returned from the real service or one generated by the appliance. A maximum of 30 error code-based URL redirect rules can be configured for each virtual service.

error_code	This parameter specifies an error code. Its value must be an integer ranging from 400 to 599 or from 901 to 906. For details about error codes in the range of 400 to 599, see related RFC specifications. The meanings of error codes in the range of 901 to 906 are as follows: • 901: The SSL virtual host requires the client certificate. • 902: The SSL client certificate's key size is too small. • 903: The SSL client certificate is not trusted. • 904: The SSL client certificate is expired. • 905: The SSL client certificate is invalid. • 906: The SSL client certificate has been revoked.
virtual_service	This parameter specifies the virtual service name. Its value must be an HTTP or HTTPS virtual service. When an error code in the range of 901 to 906 is specified, the virtual service must be of the HTTPS type.
redirect_url	This parameter specifies the URL to which the requests will be redirected. Its value must be a string of 1 to 767 characters beginning with “http” or “https”.
prefix	Optional. This parameter specifies the name of the URL query. Its value must be a string of 1 to 40 characters. Only letters in the range of a to z and A to Z are supported. If this parameter is specified, the original URL will be used as the value of the new URL query. For example, if this parameter value is “abc”, the format of query in the Location header of the redirect response is “redirect_url?abc=original_url”. If this parameter is not specified, the new URL does not contain the original URL.
encoding_mode	Optional. This parameter specifies the encoding mode of the original URL to be contained in the new URL. This parameter is available only when the “prefix” parameter is specified. Its value must be “base64”, indicating that the original URL will be encoded using the base64 algorithm. If this parameter is not specified, the original URL is displayed using plaintext.

**Note:**

- This function takes effect only when HTTP requests contain the Host header.
- If both the “**http redirect error**” and “**http import error**” or “**ssl import error**” commands are configured, the “**http redirect error**” command takes effect.

no http redirect error *<error_code>* *<virtual_service>*

This command is used to delete an error code-based URL redirect rule configuration for the specified virtual service.

show http redirect error *[virtual_service]*

This command is used to display the error code-based URL redirect rule configurations for the specified virtual service. If the “*virtual_service*” parameter is not specified, the error code-based URL redirect rule configurations for all virtual services will be displayed.

clear http redirect error *[virtual_service]*

This command is used to clear the error code-based URL redirect rule configurations for the specified virtual service. If the “*virtual_service*” parameter is not specified, the error code-based URL redirect rule configurations for all virtual services will be cleared.

HTTPS Forwarding Control

When SSL client authentication is enabled, administrators can define client certificate fields to be transferred to the real services as well as the Distinguished Name (DN) field separator, certificate encoding mode, and OID name of the client certificate fields as required. Administrators can also configure ACL rules to check the authentication information provided by clients to restrict the access level of clients, hence protecting network resources provided by real services.

http xclientcert virtual *<virtual_service>* *[mode]* *[certificate_type]*

This command is used to enable the function of inserting the client certificate into HTTP requests for the specified virtual service. This function takes effect only after client authentication is enabled (by the “**ssl settings clientauth**” command) for the SSL virtual host bound to the specified virtual service.

<i>virtual_service</i>	This parameter specifies the virtual service name. Its value must be an HTTPS virtual service.
<i>mode</i>	Optional. This parameter specifies the mode of transferring the client certificate to real services. Its value must be: • header: inserts an HTTP header to transfer the client certificate.

- **cookie:** inserts an HTTP cookie to transfer the client certificate.

The default value is “header”.

certificate_type

Optional. This parameter specifies the certificate type. Its value must be:

- **PEM:** indicates the PEM format certificate, which has the “-----BEGIN CERTIFICATE-----” and “-----END CERTIFICATE-----” line. Every 64 bits of the encoded certificate content is separated using “;”.
- **body:** indicates the certificate content body, without the begin and end lines and delimiters.

The default value is “body”.

Note: When the PEM format is used, both the encoded certificate content and cookie will use “;” as a separator. Administrators must make sure the appliance can normally forward the client certificate in the PEM format to the real service based on the actual applications.

no http xclientcert virtual <virtual_service>

This command is used to disable the function of inserting the client certificate into HTTP requests for the specified virtual service.

show http xclientcert virtual

This command is used to display all virtual services for which the function of inserting the client certificate into HTTP requests is enabled.

clear http xclientcert virtual

This command is used to disable the function of inserting the client certificate into HTTP requests for all virtual services.

http xclientcert header [header_name] [virtual_service]

This command is used to configure a customized name for the HTTP header used to transfer the client certificate for the specified HTTPS-type virtual service or the global. The configuration for an individual virtual service has a higher priority than the global configuration. If this command is not configured, the system will use the default value “X-Client-Cert” as the name of the HTTP header used to transfer the client certificate.

header_name

Optional. This parameter specifies the customized name for the HTTP header used to transfer the client certificate to the real service. Its value must be a string of 1 to 127 characters. When the string length is 127 characters, the

last character must be “:”.

The default value is “X-Client-Cert”.

virtual_service	Optional. This parameter specifies the name of an existed virtual service. Its value must be the name of a HTTPS-type virtual service. If this parameter is not specified, the configuration will take effect globally, that is, for all HTTPS-type virtual services.
-----------------	---

no http xclientcert header *[virtual_service]*

This command is used to delete the customized name of client certificate header configured for the specified virtual service. When the “virtual_service” parameter is not specified, the system will only delete the global configuration of this function and restore to the default value.

show http xclientcert header *[virtual_service]*

This command is used to display the customized name of the client certificate header configured for the specified virtual service. When the “virtual_service” parameter is not specified, the system will display all individual and global configurations of this function.

clear http xclientcert header

This command is used to restore the global customized name of client certificate header to the default value, and clear all configurations of this function for individual virtual services.

http xclientcert plaintext *<mode> <field_name> <virtual_service> [customized_name] [format_opt]*

This command is used to insert the specified certificate field into HTTP requests for the specified virtual service

mode	This parameter specifies the mode of transferring the certificate field to the real service. Its value must be:
------	---

- header: inserts an HTTP header to transfer the certificate field.
- url: inserts a URL query string to transfer the certificate field.
- cookie: inserts an HTTP cookie to transfer the certificate field.
- all: indicates all of the three modes above.

field_name	This parameter specifies the standard name of the certificate field. Its value must be:
------------	---

- **Subject:** transfers the subject DN of a client certificate to the real service.
- **Issuer:** transfers the issuer DN of a client certificate to the real service.
- **Validity:** transfers the certificate's period of validity to the real service. Its format is "From <NotBefore> To <NotAfter>". For example, "From Dec 19 5:54:42 2007 GMT To Dec 19 5:54:42 2008 GMT".
- **Serial:** transfers the certificate's serial number to the real service.
- **NotBefore:** transfers the certificate's start date to the real service.
- **NotAfter:** transfers the certificate's expiry date to the real service.
- **CommonName:** transfers the certificate's subject common name to the real service.
- **PublicKey:** transfers the public key of the certificate to the real service. The public key is transferred in HEX mode. For example, the public key "0x00 0x43 0x78 0xed" is transferred to the real service in the form of "00 43 78 ed". **When the filed name is specified as "PublicKey", only the public key modulus is sent to the real service.**
- **RDN:** transfers the content specified by RDN to the real service. RDN must be defined in the format of "<scope>.<symbol or OID>" or "<OID expression>". For information about the value of "scope" and "symbol", see the following table.

The parameter value is case-insensitive.

For "scope":

Scope	Description
Subject	The value of the symbol or specific OID will be searched in the client certificate's subject DN.
Issuer	The value of the symbol or specific OID will be searched in the client certificate's issuer DN.
Ext	The value of the symbol or specific OID will be searched in the client certificate's external field. The client certificate must be in the SSL v2.0 or SSL v3.0 version.
OID or <null>	The value of the specific OID will be searched in the client certificate's TBS (To Be Signed).

For "symbol":

Symbol	OID	Standard Name
C	2.5.4.6	Country Name

Symbol	OID	Standard Name
ST	2.5.4.8	State or Province Name
L	2.5.4.7	Locality Name
O	2.5.4.10	Organization Name
OU	2.5.4.11	Organizational Unit
CN	2.5.4.3	Common Name
serialNumber	2.5.4.5	Serial Number
dnQualifier	2.5.4.46	DN Qualifier
pseudonym	2.5.4.65	Pseudonym
title	2.5.4.12	Title
generationQualifier	2.5.4.44	Generation Qualifier
initials	2.5.4.43	Initials
name	2.5.4.41	Name
givenName	2.5.4.42	Given Name
surname	2.5.4.4	Surname
UID	0.9.2342.19200300.100.1.1	User ID
DC	0.9.2342.19200300.100.1.25	Domain Component
emailAddress	1.2.840.113549.1.9.1	Email Address
{OID expression}		OID information, for example: 1.2.3.4



Note: When there is more than one value to the same symbol in a specific scope, the appliance will transfer all of them to the real service, and one digital number will be appended to the customized name from the second symbol. The digital number is increased from 1.

The following command is an example:

```
AN(config)#http xclientcert plaintext cookie Subject.OU vs1 OU positive
```

If the client certificate has the following subject DN (“OU” in the scope of “subject” has two values: “Dev” and “TM”):

```
U=US, ST=Milpitas, L=Milpitas, O= Organization, OU=Dev, OU=TM, CN=abc,
emailAddress=abc@mailserver.com
```

Then the real service will receive the following cookie (the integer “1” is added after the second customized name “OU”):

```
Cookie: OU=Dev, OU1=TM
```

virtual_service This parameter specifies the virtual service name. Its value must be an HTTPS virtual service.

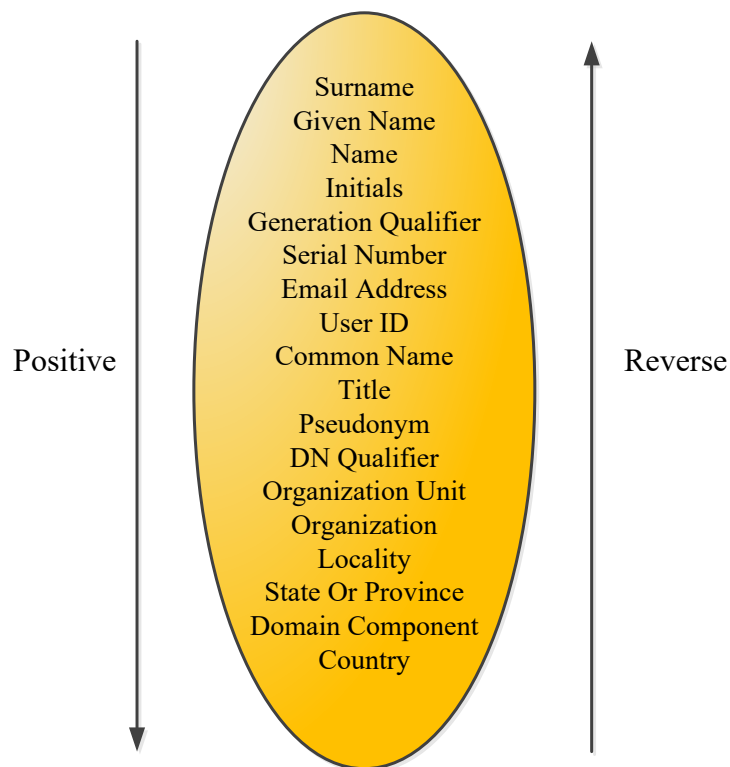
customized_name Optional. This parameter specifies a customized name for the certificate field to be inserted into the HTTP header, URL query string, or HTTP cookie. If this parameter is not specified, the value of the “field_name” parameter will be used as the customized name.

format_opt

Optional. This parameter specifies the format of the certificate field forwarded to the real service. Its value is case-insensitive.

When the “field_name” parameter is set to “Subject” or “Issuer”, the “format_opt” parameter defines the order for transferring the certificate field. Its value must be:

- positive: The transfer starts from the smallest to the largest scope. (See the following example.)
- reverse: The transfer starts from the largest to the smallest scope.
- original: The transfer follows the sequence as parsed from the client certificate.



Assuming that the Subject DN field of a client certificate is “C=CN,O=Array,OU=TM,ST=BJ,CN=abc,emailAddress=abc@mailserver.com”. When the “field_name” parameter is set to “subject”:

- If the “format_opt” parameter is set to “positive”, the Subject DN field will be transferred in the following order:
emailAddress=abc@mailserver.com,CN=abc,OU=TM,O=Array,ST=BJ,C=CN
- If the “format_opt” parameter is set to “reverse”, the Subject DN field will be transferred in the following

order:

C=CN,ST=BJ,O=Array,OU=TM,CN=abc,emailAddress=abc@mailserver.com

- If the “format_opt” parameter is set to “original”, the Subject DN field will be transferred in the following order:

C=CN,O=Array,OU=TM,ST=BJ,CN=abc,emailAddress=abc@mailserver.com

When the “field_name” parameter is set to “Validity”, “NotBefore”, or “NotAfter”, the “format_opt” parameter defines the date/time format. Its value must be:

- digital: All date and time information is expressed using the digital number, except the GMT expression.
- latin: Month will be expressed in English word. Other date and time information is expressed using the digital number.
- W3C: Standard time format. The local time zone information from the client certificate will be used.

The default value is “digital”.

The following are examples of the date and time when the “field_name” parameter is set to “Validity”:

- When the “format_opt” parameter is set to “digital”, the date and time format is “Valid from 2008-01-01 20:01:01 GMT to 2010-01-01 20:01:00 GMT”.
- When the “format_opt” parameter is set to “latin”, the date and time format is “From Jan 31 15:35:5 2008 GMT To Jan 30 15:35:5 2009 GMT”.
- When the “format_opt” parameter is set to “w3c”, the date and time format is “From 2008-01-31T15:35:05Z To 2009-01-30T15:35:05Z”.

When the “field_name” parameter is set to “ext.<OID>”, the value of the “format_opt” parameter must be “unparsed” or “parsed”.

Take the extension part of the X509 certificate as an example:

Extension ::= SEQUENCE {

extnID OBJECT IDENTIFIER,

critical BOOLEAN DEFAULT FALSE,

extnValue OCTET STRING }

Among which, “extnID” indicates the extended OID;
“critical” indicates whether the extension is important;
“extnValue” indicates the extension value.

- unparsed: “extnValue” is encoded in DER, which is expressed by three parts: type, length and value. In the “unparsed” mode, the entire “extnValue” will be forwarded to the real service.
- parsed: “extnValue” is also encoded in DER, which is expressed by three parts: type, length and value. In the “parsed” mode, only the value part of “extnValue” will be forwarded to the real service.

The default value is “unparsed”.

The following is an example of the transferred content when the “field_name” parameter is set to “ext.<OID>”:

In this example, the extension OID is 0.1.2.3, and the value of “extnValue” is “0x0c 0x06 0x36 0x35 0x34 0x33 0x32 0x31”. “0c” represents the value type and “06” represents the value length.

- If “format_opt” is set to “unparsed”, “0x0c 0x06 0x36 0x35 0x34 0x33 0x32 0x31” will be forwarded.
- If “format_opt” is set to “parsed”, “0x36 0x35 0x34 0x33 0x32 0x31” will be forwarded.

The entire “extnValue” will be forwarded to the real service when the value of “extnValue” is one of the following types:

- SEQUENCE
- SET
- Untagged data

For example, the following is an extension of which the value type is SEQUENCE:

404 30 31: SEQUENCE {

406 06 3: OBJECT IDENTIFIER issuerAltName (2 5 29 18)

411 04 24: OCTET STRING, encapsulates {

413 30 22: SEQUENCE {

415 86 20: [6] 'http://www.nist.gov/'


```
:      }  
  
:      }  
  
:      }
```

After the “**http xclientcert plaintext header** “**ext.2.5.29.18**” vs1 “**url1**” “**parsed**”” or “**http xclientcert plaintext header** “**ext.2.5.29.18**” vs1 “**url1**” “**unparsed**”” command is executed, the same result “0x30 0x22 0x86 0x20...” will be sent to the real service.

When the value type of “extnValue” is a time string, the appliance will transfer it using either of the following formats:

- Generalized Time
- UTC time



Note: Multiple transfer modes can be set for the same certificate field. However, only one customized name and one format are allowed for the same certificate field. That is, the newest customized name and format of the certificate field will overwrite the customized name and format of the field in earlier “**http xclientcert plaintext**” configurations.

no http xclientcert plaintext <mode> <field_name> <virtual_service>

This command is used to delete the configuration about inserting the certificate field into HTTP requests for the specified virtual service.

show http xclientcert plaintext

This command is used to display all configurations about inserting the certificate field in HTTP requests.

clear http xclientcert plaintext

This command is used to clear all configurations about inserting the certificate field into HTTP requests for all HTTPS virtual services.

http xclientcert rdnsep <virtual_service> [separator] [pre|post]

This command is used to modify the configuration of the DN field separator for the specified virtual service.

When this command is not configured, all virtual services use comma “,” as the default DN field separator, and the separator is placed after every DN field.

virtual_service	This parameter specifies the virtual service name. Its value must be an HTTPS virtual service.
-----------------	--

separator	Optional. This parameter specifies the DN field separator. Its value must be a string of 1 to 3 characters enclosed in double quotes. Letters (A to Z and a to z), numbers (1 to 9), and the “%” symbol are not supported. The default value is “,”.
pre post	Optional. This parameter specifies where to place the DN field separator. Its value must be: pre: places the separator before the DN field. post: places the separator after the DN field. The default value is “post”.

no http xclientcert rdns sep *[virtual_service]*

This command is used to restore the DN field separator configuration for the specified virtual service. If the “virtual_service” parameter is not specified, the DN field separator configurations for all virtual services will be restored.

show http xclientcert rdns sep *[virtual_service]*

This command is used to display the DN field separator configuration for the specified virtual service. If the “virtual_service” parameter is not specified, the DN field separator configurations for all virtual services will be displayed.

http xclientcert dnencoding <*virtual_service*> *[encoding]*

This command is used to specify the encoding format for transferring the client certificate DN for the specified virtual service.

When this command is not configured, all virtual services use the UTF-8 encoding format.

virtual_service	This parameter specifies the virtual service name. Its value must be an HTTPS virtual service.
encoding	Optional. This parameter specifies the encoding format. Its value must be “UTF-8”, “GB2312”, “GBK” or “GB18030”. The default value is “UTF-8”.

no http xclientcert dnencoding <*virtual_service*>

This command is used to restore the encoding format configuration of the client certificate DN for the specified virtual service.

show http xclientcert dnencoding [virtual_service]

This command is used to display the encoding format configuration of the client certificate DN for the specified virtual service. If the “virtual_service” parameter is not specified, the encoding format configurations of the client certificate DN for all virtual services will be displayed.

clear http xclientcert dnencoding

This command is used to restore the encoding format configurations of the client certificate DN for all virtual services.

http xclientcert oidname <virtual_service> <oid> <customized_name>

This command is used to configure a customized name for the OID of the client certificate field for the specified virtual service.

virtual_service	This parameter specifies the virtual service name. It must be an HTTPS virtual service.
oid	This parameter specifies the OID of the client certificate field. Its value must be enclosed in double quotes.
customized_name	This parameter specifies the customized name for the OID of the client certificate field. Its value must be a string of 1 to 31 characters.

no http xclientcert oidname <virtual_service> [oid]

This command is used to delete the customized OID name configuration of the client certificate field for the specified virtual service. If the “oid” parameter is not specified, the customized OID name configurations of all client certificate fields for the specified virtual service will be cleared.

show http xclientcert oidname [virtual_service]

This command is used to display the customized OID name configuration of the client certificate field for the specified virtual service. If the “virtual service” parameter is not specified, the customized OID name configurations of the client certificate field for all virtual services will be displayed.

clear http xclientcert oidname

This command is used to clear the customized OID name configurations of the client certificate field for all virtual services.

http xclientcert pemsep <virtual_service> [separator]

This command is used to set the separator used by the specified virtual service when inserting a PEM certificate in the HTTP request.

If this command is not configured, the system uses semicolons (“;”) as separators by default.

<code>virtual_service</code>	This parameter specifies the virtual service. Its value must be an HTTPS virtual service.
<code>separator</code>	Optional. This parameter specifies the separator used in the PEM certificates. Its value must be a string of 1 to 6 characters, enclosed in double quotes. The default value is “;”.

no http xclientcert pemsep [*virtual_service*]

This command is used to restore the PEM certificate separator setting for the specified virtual service to default, that is, semicolons will be used as the separator. If the “*virtual_service*” parameter is not specified, the PEM certificate separator settings for all virtual services will be restored to default.

show http xclientcert pemsep [*virtual_service*]

This command is used to display the PEM certificate separator setting for the specified virtual service. If the “*virtual_service*” parameter is not specified, the PEM certificate separator settings for all virtual services are displayed.

http xciphersuite virtual <*virtual_service*>

This command is used to enable cipher suite forwarding for the specified virtual service. After this function is enabled, the virtual service will forward the cipher suite it negotiated with the client to the real service.

By default, this function is disabled for each virtual service.

<code>virtual_service</code>	This parameter specifies the virtual service name. Its value must be an HTTPS virtual service.
------------------------------	--

no http xciphersuite virtual <*virtual_service*>

This command is used to disable cipher suite forwarding for the specified virtual service.

show http xciphersuite virtual

This command is used to display the status of cipher suite forwarding for each virtual service.

clear http xciphersuite virtual

This command is used to disable cipher suite forwarding for all virtual services.

http xciphersuite header [*header_name*] [*virtual_service*]

This command is use to set the HTTP header name used to forward cipher suites for the specified virtual service.

The virtual services without this command configured will use the global header configuration. By default, the global header configuration is “X-Client-Ciphersuite”.

header_name Optional. This parameter specifies the customized name for the HTTP header used to transfer the cipher suite to the real service. Its value must be a string of 1 to 127 characters. When the string length is 127 characters, the last character must be “: ” (a white space following a colon).

The default value is “X-Client-Ciphersuite”.

virtual_service Optional. This parameter specifies the virtual service name. Its value must be:

- HTTPS virtual service name: customizes HTTP header name for this virtual service.
- Empty: customizes the global HTTP header name.

no http xciphersuite header [*virtual_service*]

This command is used to delete the configuration of the “**http xciphersuite header**” command for the specified virtual service.

virtual_service Optional. This parameter specifies the virtual service name. Its value must be:

- HTTPS virtual service name: resets the header configuration for this virtual service to default. After the reset, this virtual service will use the global header configuration.
- Empty: resets the global header configuration to default (“X-Client-Ciphersuite”).

show http xciphersuite header

This command is used to display all configurations of the “**http xciphersuite header**” command.

clear http xciphersuite header

This command is used to clear all configurations of the “**http xciphersuite header**” command. After this command is executed, the global header configuration will be reset to “X-Client-Ciphersuite” and all virtual services will use the global header configuration.

http acl url <virtual_service> <url> <access_level> [priority]

This command is used to define an ACL rule for the specified virtual service. Upon receiving HTTP requests for the specified network resource, the appliance will process the requests based on the access level defined in the ACL rule.

The maximum number of ACL rules supported by the system depends on the system memory, as shown in the following table:

System Memory	Max Number of ACL Rules
512 MB	50
1 GB	100
2 GB	200
>= 4 GB	1000

virtual_service	This parameter specifies the virtual service name. Its value must be an HTTPS virtual service.
url	This parameter specifies the URL string of network resources to be matched. Its value must be a case-sensitive string of 1 to 512 characters. The parameter value supports both quick and full regular expressions. When the parameter value is set to a regular expression, it must be enclosed in double quotes. When the parameter value is a full regular expression, it must begin with the “<regex>” string and be enclosed in double quotes, for example, “<regex>/((graphics photos resource)/(.+)).(gif jpg png)” means matching all GIF, JPG, and PNG files in the paths “/graphics”, “/photos”, and “/resource”. For information about the differences between the quick regular expression and the full regular expression, please refer to introductions about regular expressions in Chapter 1. To ensure that a configured regular expression string is correct, administrators can use the “slb regextest <regex> <target_string>” command to test whether a target string will match the configured regular expression.
access_level	This parameter specifies the access level. Its value must be 1, 2, or 3. For details about the access levels, see the following table.

priority Optional. This parameter specifies the priority of the ACL rule. Its value must be an integer ranging from 1 to 1000. A smaller value indicates a higher priority.

The default value is 100.

Access Level	Description
1	Network resources can be accessed only through HTTPS with or without client authentication. If the mandatory mode of SSL client authentication is enabled for the SSL virtual host associated with the virtual service, clients must provide certificates for authentication.
2	Network resources can be accessed only through HTTPS and clients must provide certificates for authentication.
3	This level is used in the following scenarios: - Scenario 1 (recommended): Network resources can be accessed only through HTTPS. Clients must provide certificates and client authentication must be disabled for the SSL virtual host associated with the virtual service. - Scenario 2: Network resources can be accessed only through HTTPS. Clients must provide certificates. Client authentication must be enabled but the mandatory mode of client authentication must be disabled for the SSL virtual host associated with the virtual service. In this scenario, some browsers might not send certificates for authentication even the certificates have been loaded, which results in authentication failures. Therefore, level 3 is recommended for scenario 1. The following configurations must be available before level 3 is set: - Execute the “ssl import rootca” command to import the root CA certificate for the SSL virtual host. - Execute the “ssl globals renegotiation on” command to enable global SSL renegotiation. - Execute the “ssl settings renegot” command to enable SSL renegotiation for the SSL virtual host.



Note:

- When the “access_level” parameter is set to 1 or 2, if a received SSL request does not meet the ACL rule, the appliance will return a 403 response. When the “access_level” parameter is set to 3, if a received SSL request does not meet the ACL rule, the appliance will return a response with a status code in the range of 901 to 906 as required.
- In HTTP/2 over TLS deployment scenario, this command does not support level 3 configuration.

no http acl url <virtual_service> <url>

This command is used to delete an ACL rule configuration for the specified virtual service.

show http acl url *[virtual_service]*

This command is used to display the ACL rule configuration for the specified virtual service. If the “virtual_service” parameter is not specified, the ACL rule configurations for all virtual services will be displayed.

clear http acl url *[virtual_service]*

This command is used to clear all ACL rule configurations for the specified virtual service. If the “virtual_service” parameter is not specified, the ACL rule configurations for all virtual services will be cleared.

HTTP Compression

The HTTP compression function compresses the data returned from real services, which helps accelerate the download speed at the client side. Administrators can define the data to be compressed based on the browser type and file type. Related commands for configuring HTTP compression are as follows.

http compression {on|off} *[virtual_service]*

This command is used to enable or disable the HTTP compression function. After this function is enabled, TEXT, XML, and HTML files will be compressed by default. Administrators can use the “**http compression policy useragent**” command to set other types of files to be compressed. By default, this function is disabled globally and enabled per virtual service. The data for the specified virtual service will be compressed only when the HTTP compression function is enabled both globally and for the virtual service.

virtual_service	Optional. This parameter specifies the virtual service name. Its value must be a virtual service of the HTTP or HTTPS type. If this parameter is not specified, the HTTP compression function is globally enabled or disabled.
-----------------	--



Note: After the HTTP compression function is enabled, if the HTTP/1.0 response from a real service includes attributes only supported by HTTP/1.1, such as “Transfer-Encoding: chunked”, the response cannot be displayed on the browser. This is due to the limitation of RFC. For details, please access the following links:

- <http://tools.ietf.org/html/rfc2616>
- <http://tools.ietf.org/html/rfc2145#section-2.1>

show http compression settings

This command is used to display the global status of the HTTP compression function and its status for individual virtual services.

http compression policy useragent <user_agent_string> <mime_type>

This command is used to configure an HTTP compression policy based on the file type and browser type. For one user agent, multiple HTTP compression policies with different file types can be configured.

user_agent_string This parameter specifies the user agent string, such as “MSIE 8.0”, “MSIE 9.0”, “Mozilla/5.0”, “Chrome”, and “Safari”. The value is case-insensitive, and must be enclosed in double quotes.

mime_type This parameter specifies the type of files that are allowed to be compressed. Its value must be “doc”, “xls”, “ppt”, “js”, “css”, or “pdf”.

http compression advanced useragent on

This command is used to configure commonly-used HTTP compression policies at one time. After this command is executed, the following configurations are generated:

- **http compression policy useragent "MSIE 6.0" css**
- **http compression policy useragent "MSIE 6.0" js**
- **http compression policy useragent "MSIE 7.0" css**
- **http compression policy useragent "MSIE 7.0" js**
- **http compression policy useragent "MSIE 8.0" css**
- **http compression policy useragent "MSIE 8.0" js**
- **http compression policy useragent "Opera" css**
- **http compression policy useragent "Opera" js**
- **http compression policy useragent "Mozilla/5.0" css**
- **http compression policy useragent "Mozilla/5.0" js**

Based on these configurations, the appliance will compress the files of the JS and CSS types when the browser is MSIE 6.0, MSIE 7.0, MSIE 8.0, Opera, or the browser's user agent string contains “Mozilla/5.0”, like Firefox, Chrome, and mobile browsers such as Safari, Opera, and Blackberry.

no http compression policy useragent <user_agent_string> <mime_type>

This command is used to delete the specified HTTP compression policy configuration.

show http compression policy useragent

This command is used to display all HTTP compression policy configurations.

clear http compression policy useragent

This command is used to clear all HTTP compression policy configurations.

http compression policy urlexclude <virtual_service> <url>

This command is used to configure an HTTP compression exclusion policy. When the URL in an HTTP request sent to the specified virtual service matches the policy, the appliance will not compress the file in the corresponding response. This command takes precedence over the “**http compression policy useragent**” command.

virtual_service	This parameter specifies the virtual service name.
url	This parameter specifies the URL to be matched. Its value must be a string of 1 to 255 characters. The following wildcards are supported for setting the parameter value: • “^” means matching the starting characters of the URL. • “*” means matching zero or more characters ahead of it. • “\$” means matching the ending characters of the URL. When the parameter value contains wildcards, it must be enclosed in double quotes.

no http compression policy urlexclude <virtual_service> <url>

This command is used to delete an HTTP compression exclusion policy configuration for the specified virtual service.

show http compression policy urlexclude [virtual_service]

This command is used to display all HTTP compression exclusion policy configurations of the specified virtual service. If the “virtual_service” parameter is not specified, the HTTP compression exclusion policy configurations of all virtual services will be cleared.

clear http compression policy urlexclude [virtual_service]

This command is used to clear all HTTP compression exclusion policy configurations. If the “virtual_service” parameter is not specified, the HTTP compression exclusion policy configurations of all virtual services will be cleared.

show statistics compression [virtual_service]

This command is used to display the compression statistics of the specified virtual service. If the “virtual_service” parameter is not specified, the compression statistics of all virtual services will be cleared.

Example:

```

AN(config)#show statistics compression
Global Compression Statistics:
Throughput Statistics:
    29003769 Total bytes sent out to client
    16423821 Total bytes sent to compression
    23412681 Total bytes rcvd from compression
        0 Sent bytes/second
        0 Rcvd bytes/second
    33746 Peak Sent bytes/second
    48049 Peak Rcvd bytes/second
        0 Currently active transactions

Content Statistics:
    349443 HTML's compressed
        0 TEXT's compressed
        0 XML's compressed
        0 DOC's compressed
        0 PPT's compressed
        0 XLS's compressed
        0 CSS's compressed
        0 JS's compressed
        0 PDF's compressed
    349443 requests attempted
    349443 content length transactions
        0 chunk encoding transactions
        0 fin terminated transactions
        0 Http 1.0 response
    349443 Http 1.1 response

Compression Ratio Statistics:
    0% compression ratio of compressible data

```

The following table shows the meaning of each item in the command output:

- **Throughput Statistics**

Statistics	Description
Total bytes sent to compression	Total bytes of data sent to the compression module.
Total bytes rcvd from compression	Total bytes of compressed data.
Sent bytes/second	Total bytes of compressed data in the last second. This is calculated by subtracting the bytes of compressed data 1 second ago from the bytes of compressed data till now.
Rcvd bytes/second	Total bytes of data sent to the compression module in the last second. This is calculated by subtracting the bytes of data to be compressed 1 second ago from the bytes of data to be compressed till now.
Peak Sent bytes/second	Maximum bytes of sent data per second from the beginning to now.
Peak Rcvd bytes/second	Maximum bytes of compressed data per second from the beginning to now.
Currently active transactions	Total number of HTTP connections in which response data is being compressed.

- **Content Statistics**

Statistics	Description
HTML's compressed	Total number of compressed HTTP responses whose types are

Statistics	Description
	HTML.
TEXT's compressed	Total number of compressed HTTP responses whose types are TEXT.
XML's compressed	Total number of compressed HTTP responses whose types are XML.
DOC's compressed	Total number of compressed HTTP responses whose types are DOC.
PPT's compressed	Total number of compressed HTTP responses whose types are PPT.
XLS's compressed	Total number of compressed HTTP responses whose types are XLS.
CSS's compressed	Total number of compressed HTTP responses whose types are CSS.
JS's compressed	Total number of compressed HTTP responses whose types are JS.
PDF's compressed	Total number of compressed HTTP responses whose types are PDF.
requests attempted	Total number of compressed HTTP responses. It equals to the sum of all individual types of compressed responses.
content length transactions	Total number of compressed HTTP responses containing Content-Length headers.
chunk encoding transactions	Total number of compressed HTTP responses containing "Transfer-Encoding: chunked".
fin terminated transactions	Total number of compressed HTTP responses which are FIN-terminated, which means the entire responding process is completed in a short-living connection.
Http 1.0 response	Total number of compressed HTTP/1.0 responses.
Http 1.1 response	Total number of compressed HTTP/1.1 responses.

clear statistics compression [virtual_service]

This command is used to clear all compression statistics of the specified virtual service. If the "virtual_service" parameter is not specified, the compression statistics of all virtual services will be cleared.

HTTP Cache

The HTTP cache function supports the caching of both compressed and uncompressed data. It reduces communications between the client and the real service, accelerating the speed of acquiring resources and lightening the burden of real services.

cache {on|off} [virtual_service]

This command is used to enable or disable the HTTP cache function for the specified virtual service.

Disabling the HTTP cache function does not change the current cache configuration or contents in the system.

By default, this function is disabled globally and enabled per virtual service. The data of a specified virtual service will be cached only when the HTTP cache function is enabled both globally and for the specified virtual service.

virtual_service Optional. This parameter specifies the virtual service name. It must be an HTTP or HTTPS virtual service. If this parameter is not specified, the HTTP cache function is enabled or disabled globally.

show cache status

This command is used to display the status of the HTTP cache function.

cache settings objectsize <size>

This command is used to change the maximum size of a cache object.

If this command is not configured, the default maximum size of a cache object is 5120 KB.

size This parameter specifies the maximum size of a cache object, in Kbytes. The minimum value is 1. The maximum value varies by system memories, as shown in the following table.

System Memory	Max Size of Cache Object
2 GB	1024 KB (1 MB)
4 GB	10,240 KB (10 MB)
8 GB	20,480 KB (20 MB)
16 GB/24 GB/32 GB/64 GB	40,960 KB (40 MB)

cache settings expire <expire_time>

This command is used to change the expiration time of cache objects.

If this command is not configured, the default expiration time of cache objects is 82,800 seconds (23 hours).

expire_time This parameter specifies the expiration time of cache objects. Its value must be in the format of “hh:mm:ss” or “ss”. When the value is set in the format of “ss”, it must be an integer ranging from 0 to 2,147,483,646, and enclosed in double quotes. The value 0 indicates that the cache object will expire immediately after it is cached.

Three types of cache expiration time are involved during the cache process:

- Cache expiration time defined by the cache-related field (the Expires and Cache-Control headers) in the HTTP responses
- Cache expiration time set by the “**cache settings expire**” command

- Cache expiration time specified by the “ttl” parameter in the “**cache filter rule**” command

The three types of cache expiration time are prioritized as follows:

1. The cache expiration time set by the “ttl” parameter in the “**cache filter rule**” command will be preferentially used. If the “ttl” parameter is not specified, the cache expiration time specified by the “**cache settings expire**” command applies.
2. For the cache content that does not match any cache filter rule, the expiration time defined in the cache-related headers of the HTTP response applies.
3. If the cache expiration time is unavailable in the cache-related headers of the HTTP response, the cache expiration time set by the “**cache settings expire**” command applies.



Note: When the duration of cache objects reaches the expiration time, the appliance will not delete the expired cache objects until new HTTP traffic enters the cache module. If no new HTTP traffic enters the cache module, the expired cache objects will remain in the module.

show cache settings

This command is used to display settings about the expiration time of cache objects and the maximum size of a cache object.

cache evict <host_name> <url_regex>

This command is used to delete the cache objects that match the specified host name and URL regular expression.

host_name	This parameter specifies the host name of the cache objects. Its value must be a string of 1 to 80 characters in the format of “host name” or “host name:port”.
url_regex	This parameter specifies the regular expression for the URL string of the cache objects. Its value must be a string of 1 to 256 characters in the format of “[^string1[*string2[*stringN]][\$]”, and enclosed in double quotes. The following wildcards are supported: • “^” means matching the starting characters of the URL. • “*” means matching zero or more characters ahead of it. • “\$” means matching the ending characters of the URL.

The maximum number of cache objects varies by system memories, as shown in the following table. The system will generate a log of the “notice” level when the number of cache objects exceeds the limit.

System Memory	Max Number of Cache Objects
2 GB/4 GB lite	32 K
4 GB	64 K
8 GB	128 K
16 GB	256 K
24 GB	384 K

show cache content <host_name> <url_regex>

This command is used to display information about the cache objects that match the specified host name and URL regular expression. The maximum length of the output content is 120 KB, with a maximum of about 1700 items displayed at each execution.

clear cache content

This command is used to clear all cache objects.

cache filter {on|off}

This command is used to enable or disable the cache filter function. By default, this function is disabled.

show cache filter status

This command is used to display the status of the cache filter function.

cache filter rule <host_name> <url> <cache_behavior>

This command is used to create a cache filter rule to define the cache behavior of the appliance for the objects that match the “host_name” and “url” parameters.

host_name This parameter specifies the host name to be matched. Its value must be a string of 1 to 79 characters in the format of “host_name” or “host_name:port”.

url This parameter specifies the URL string to be matched. Its value must be a string of 1 to 255 characters and is case-sensitive. The parameter value supports full regular expressions. When the parameter value is a full regular expression, it must begin with the “<regex>” string and be enclosed in double quotes, for example, “<regex>/(graphics|photos|resource)/(.)\.(gif|jpg|png)” means matching all GIF, JPG, and PNG files in the paths “/graphics”, “/photos”, and “/resource”. For more information, please refer to introductions about regular expressions in Chapter 1.

To ensure that a configured regular expression string is correct, administrators can use the “**slb regextest <regex> <target_string>**” command to test whether a target string

will match the configured regular expression.

If a URL matches two or more cache filter rules, the rule with the longest match is selected.

cache_behavior

This parameter specifies the cache actions to be performed for the matching object. The system supports the following cache attributes:

- **cache:** defines whether to cache the matching objects. Its value is “cache=yes”, “cache=no”, or empty. The default value is empty. For information about each value, see the following table.
- **override:** defines cache-related information to be ignored. This parameter is available only when “cache=yes” is set. Its value is “override=nsreq”, “override=nsresp”, “override=ncreq”, “override=ncresp”, “override=au”, “override=co”, “override=pr”, “override=sc”, “override=va”, “override=ttl”, “override=all” or “override=none”. This parameter supports the setting of multiple values separated by commas, such as “override=nsreq,nsresp”. The default value is “override=all”. For information about each value, see the following table.
- **urlquery:** defines whether to ignore the URL query string in the request. Its value is “urlquery=yes”, “urlquery=no”, or empty. The default value is empty. For information about each value, see the following table.
- **ttl:** defines the expiration time of the cache object, in seconds. Its format is “ttl=n”. The value range of “n” is 1 to 2,147,483,646. This attribute takes effect only when the following conditions are all met:
 - The request’s version is not HTTP1.0.
 - The Cache-Control header in the request contains no-cache and “override=ncreq” is set, or the Cache-Control header in the response contains no-cache and “override=ncresp” is set, or the Cache-Control headers in both the request and response do not contain no-cache.
 - “override=all” or “override=ttl” is set, or “override=all” or “override=ttl” is not set but the Cache-Control header in the request does not

contain max-age or the Cache-Control header in the response does not contain s-maxage, max-age, and expires.

The default value is empty. For information about each value, see the following table.

At least one of the “cache”, “urlquery” and “ttl” attributes must be set. When two or all of them are set, they are not subject to the setting sequence, and each attribute setting must be enclosed in double quotes and separated by spaces. For example: “cache=yes” “override=nsreq,nsresp” “urlquery=yes”.

Cache filter rules define the cache behavior based on the attribute setting. The following table lists the cache behavior corresponding to each attribute setting.

Attribute	Value	Behavior
cache	yes	Requests that carry any of the following cache-related information will be cached: - Request with the “Cache-Control no-store” directive - Request with the “Cache-Control no-cache” directive - Request with the Authorization header - Request with the Cookie header Responses that carry any of the following cache-related information will be cached: - Response with the “Cache-Control no-store” directive - Response with the “Cache-Control no-cache” directive - Response with the “Cache-Control private” directive - Response with the Set-Cookie header - Response with the Vary header
	no	No cache even instructions in the cache-related headers allow the objects to be cached.
	not set	Follow the instructions in cache-related headers.
override	nsreq	Ignore “Cache-Control: no-store” in the request.
	nsresp	Ignore “Cache-Control: no-store” in the response.
	ncreq	Ignore “Cache-Control: no-cache” in the request.
	ncresp	Ignore “Cache-Control: no-cache” in the response.
	au	Ignore the Authorization header in the request.
	co	Ignore the Cookie header in the request.
	pr	Ignore “Cache-Control: private” in the response.

Attribute	Value	Behavior
	sc	Ignore the Set-Cookie header in the response.
	va	Ignore the Vary header in the response.
	ttl	Ignore “maxage”, “S-maxage”, and other directives related to the cache expiration time in the Cache-Control header of the request and response.
	all	Ignore all directives in the Cache-Control header as well as the Set-Cookie, Cookie, Authorization, and Vary headers of the request and response.
	none	Follow the instructions in cache-related headers.
urlquery	yes	Ignore the query string in the URL.
	no	Do not ignore the query string in the URL.
	not set	Do not ignore the query string in the URL.
ttl	new_ttl_value	Use the new TTL value as the expiration time for the matching cache object.
	not set	Use the expiration time set by the “ cache settings expire ” command or use the expiration time specified in the cache-related headers.

Examples:

1. Cache all JPG files, and process others files based on the instructions in cache-related headers.

```
AN(config)#cache filter rule www.xyz.com ".*\.jpg" "cache=yes"
```

2. Cache all JPG files and ignore “Cache-Control: no-store” in the client request. Other files are processed based on the instructions in cache-related headers.

```
AN(config)#cache filter rule www.xyz.com ".*\.jpg" "cache=yes" "override=nsreq"
```

3. Cache JPG, GIF, and HTML files, and other files are not cached. The cache expiration time for JPG files is determined by the directives in the Cache-Control header, and the cache expiration time for GIF and HTML files is 200,000s.

```
AN(config)#cache filter rule www.xyz.com ".*\.jpg" "cache=yes"
AN(config)#cache filter rule www.xyz.com ".*\.gif" "cache=yes" "override=all"
"ttl=200000"
AN(config)#cache filter rule www.xyz.com ".*\.html" "cache=yes" "override=all"
"ttl=200000"
AN(config)#cache filter rule www.xyz.com / "cache=no"
```

4. Do not cache JPG and GIF files, and other files are processed based on the instructions in cache-related headers.

```
AN(config)#cache filter rule www.xyz.com ".*\.jpg" "cache=no"
AN(config)#cache filter rule www.xyz.com ".*\.gif" "cache=no"
```

5. Do not cache JPG and GIF files, and cache other files.

```
AN(config)#cache filter rule www.xyz.com ".*\.jpg" "cache=no"
AN(config)#cache filter rule www.xyz.com ".*\.gif" "cache=no"
AN(config)#cache filter rule www.xyz.com / "cache=yes"
```

6. Cache JPG and GIF files for 200,000s, and other files use the expiration time defined in cache-related headers.

```
AN(config)#cache filter rule www.xyz.com ".*\.jpg" "override=all" "ttl=200000"
```

```
AN(config)#cache filter rule www.xyz.com ".*\.gif" "override=all" "ttl=200000"
AN(config)#cache filter rule www.xyz.com / "cache=yes"
```

7. Configure the appliance not to match the query string in the URL for HTML files and match the whole URL for other files.

```
AN(config)#cache filter rule www.xyz.com ".*\.html*" "urlquery=yes"
```

no cache filter rule *<host_name>* *<url>*

This command is used to delete the cache filter rule that matches the specified host name and URL.

show cache filter hostname *<host_name>*

This command is used to display all cache filter rules matching the specified host name.

show cache filter all

This command is used to display all cache filter rules.

clear cache filter hostname *<host_name>*

This command is used to clear all cache filter rules matching the specified host name.

clear cache filter all

This command is used to clear all cache filter rules.

cache filter match *<host_name>* *<url_regex>*

This command is used to check whether the specified host name and URL regular expression match an existing cache filter rule.

host_name This parameter specifies the host name to be tested.

url_regex This parameter specifies the URL regular expression to be tested.

show statistics cachefilter *<host_name>* *<url_regex>*

This command is used to display the cache filter statistics related to the specified host name and URL regular expression.

clear statistics cachefilter [*host_name*]

This command is used to clear all cache filter statistics related to the specified host name.

host_name This parameter specifies the host name. Its value must be:

- *host name*: clears all cache filter statistics related to

this host name.

- “all”: clears all cache filter statistics.
- “global”: clears the global cache filter statistics.

The default value is “global”.

cache imgopt {on|off} [virtual_service]

This command is used to enable or disable the cache image optimization function for the specified virtual service. When this function is enabled, the system will convert BMP/JPEG/PNG images to WebP/JPEG images.

By default, this function is disabled globally and enabled for every virtual service. The cache images for the specified virtual service will be optimized only when this function is enabled both globally and for the virtual service.

Source images in PNG format can only be converted to WebP format, not JPEG format.

virtual_service	Optional. This parameter specifies the virtual service name. Its value must be a virtual service of the HTTP or HTTPS type. If this parameter is not specified, the cache image optimization function is globally enabled or disabled.
-----------------	--

cache imgopt sizelimit <min_size>

This command is used to set the minimum size of the source image allowed to be optimized for the cache image optimization function. Source images smaller than this size will no longer be optimized.

min_size	This parameter specifies the minimum size of the source image to be optimized, in Kbytes. Its value must be an integer no less than 1.
----------	--

The default value is 1.

cache imgopt format [target_format]

This command is used to set the target format for the cache image optimization function. The target format must be WebP or JPEG.

When the target format is set to WebP, the system will still determine the target format according to the browser's own support for the WebP format. When the client browser is of Chrome9.0 and Opera11.10 or later, the source image will be successfully optimized to WebP format. For other browsers, the image will be optimized to JPEG format.

target_format Optional, this parameter specifies the target format for image optimization function. Its value must be webp or jpeg. The default value is jpeg.

show cache imgopt settings

This command is used to display the status of the HTTP image optimization function globally and for individual virtual services.

cache imgopt urlexclude <virtual_service> <url>

This command is used to configure an image optimization exclusion policy for the specified virtual service. When the request URL in an HTTP request sent to the specified virtual service matches the policy, the appliance will not optimize the image in the corresponding response.

virtual_service This parameter specifies the virtual service name. Its value must be a virtual service of the HTTP or HTTPS type.

url This parameter specifies the URL to be matched. Its value must be a string of 1 to 255 characters. Its value supports the following wildcards:

- “^” means matching the starting characters of the URL.
- “*” means matching zero or more characters ahead of it.
- “\$” means matching the ending characters of the URL.

When the parameter value contains wildcards, it must be enclosed in double quotes.

no cache imgopt urlexclude <virtual_service> <url>

This command is used to delete an image optimization exclusion policy for the specified virtual service.

show cache imgopt urlexclude [virtual_service]

This command is used to display all image optimization exclusion policies of the specified virtual service. If the “virtual_service” parameter is not specified, the image optimization exclusion policies of all virtual services will be displayed.

clear cache imgopt urlexclude [virtual_service]

This command is used to clear all image optimization exclusion policy configurations. If the “virtual_service” parameter is not specified, the HTTP image optimization exclusion policy configurations of all virtual services will be cleared.

show statistics cache [virtual_service]

This command is used to display all HTTP cache statistics for the specified virtual service. If the “virtual_service” parameter is not specified, the HTTP cache statistics for all virtual services will be displayed.

Example:

```
AN(config)#show statis cache
```

Reverse Proxy Cache Global Statistics:

Basic Statistics:

```
Requests received: 3601254
Requests with GET method: 3601254
Requests with HEAD method: 0
Requests with PURGE method: 0
Requests with POST method: 0
Number of open client connections: 115
Number of open server connections: 115
Requests redirected to HTTPS: 0
Requests redirected based on regex match: 0
Requests forwarded with rewritten url: 0
Locations rewritten to HTTPS: 0
Locations rewritten based on regex match: 0
Cache skip, cache off: 3601254
Cache hit, reply using cache: 0
Cache hit, reply with "Not Modified": 0
Cache hit, reply with "Precondition Failed": 0
Cache hit, revalidate: 0
Cache miss, noncacheable requests: 3601254
Cache miss, create new entry: 0
Cache miss, create new entry, resp noncacheable: 0
Hit ratio: 0.00%
```

(Notice: the real server's time should be in sync with this machine.

Otherwise, the time difference could expire the cachable objects resulting in low cache hit ratio.)

Advanced Statistics:

```
Number of cache objects: 0
Number of cache frames: 0
Successful cache probes: 0
Max cache objects: 65536
Max cache frames: 438044
Times of cache drain: 0
```

Why were certain requests sent to the server?

a) We had to revalidate the cached object due to:

```
Request with "no-cache": 0
Request with "max-age=0": 0
Cached object had "no-cache": 0
Cache object expired: 0
```

b) We had to bypass cache for some requests because:

```
Cache was filling when request was made: 0
Revalidation failed due to IMS mismatched: 0
Client has newer copy, cannot send from cache: 0
```

Object in cache is chunked, cannot give to 1.0 client: 0
 Network memory utilization was too high: 0

c) Request cannot be served from cache because:

Cache filter denied caching: 0
 Requests with "no-store": 0
 Requests with "authorization": 0
 Requests with cookies: 0
 Requests with range: 0
 Requests non GET, non HEAD: 0
 Requests URL too long: 0
 Requests host too long: 0

d) Error occurred while doing cache lookup

Network memory shortage when cache hit (200, 304): 0
 Cache was not accessible: 0
 Fail to send cache lookup to cache: 0
 Fail to find url and host: 0
 Fail to parse cache specific http request headers: 0
 Fail to create a new cache object: 0
 Noncacheable requests due to other errors: 3601254

Why were certain responses not stored in cache?

a) HTTP directive in response told us not to cache

HTTP response code not 200, 300 or 301: 0
 Response had a "no-store": 0
 Response had a "private": 0
 Response had a "set-cookie": 0
 Response had a "vary": 0

b) The response did not meet our guidelines for cacheability

Response noncacheable too big: 0

c) Error occurred when trying to cache response

Cache storage limit exceeded based on header data: 0
 Cache storage limit exceeded based on payload: 0
 Network memory shortage when storing response body: 0
 Cache object was deleted before response arrived: 0
 Fail to parse cache specific http response headers: 0
 Fail to store response headers in cache: 0
 Fail to store response body in cache: 0
 Cache object was aborted due to connection reset: 0
 Noncacheable responses due to other errors: 0

The following tables list the meanings of items in the command output:

- **Basic Statistics**

Output Item	Description
Requests received	Total number of received requests.
Requests with GET method	Total number of received GET requests.
Requests with HEAD method	Total number of received HEAD requests.
Requests with PURGE method	Total number of received PURGE requests.

Output Item	Description
Requests with POST method	Total number of received POST requests.
Requests redirected based on regex match	Number of requests redirected based on the configured URL regular expression rules.
Requests forwarded with rewritten url	Number of requests forwarded after URLs are rewritten.
Number of open client connections	Total number of open connections to clients.
Number of open server connections	Total number of open connections to servers.
Cache miss, new entry created	Number of times that a new entry is created because no cache object is matched.
Cache miss, noncacheable requests	Number of requests that cannot be cached (The appliance considers a request non-cacheable if it contains some specific contents, such as a long URL or a "Cache-Control: no-cache" directive).
Cache revalidate	Number of requests that have matching cache objects but need to be revalidated for some reasons (such as revalidation requests generated by the client or the proxy server, or no-cache directives generated by the proxy server).
Cache hit, reply using cache	Number of requests that have matching cache objects and do not need to be revalidated, and therefore the appliance replies using the cache objects.
Cache hit, reply with "Not Modified"	Number of requests that carry "If-Modified-Since" to which the appliance replies with a 304 response after deciding that the requested copy at the client side does not need to be revalidated.
Cache hit, reply with "Precondition Failed"	Number of requests that carry the "If-Match" and "If-Unmodified-Since" headers to which the appliance replies with a 412 response after deciding that the requested content has no matching cache object.
Cache miss, create new entry, resp noncacheable	Number of times that a request has no matching cache object, and therefore the appliance creates a cache to be filled but the response received from the real service cannot be cached.
cache skip, cache off	Total number of requests that are not processed using the cache function.
Hit ratio	Percentage of requests that are replied using matching cache objects or 304 responses.
Locations rewritten to HTTPS	Number of responses rewritten to HTTPS responses.
Locations rewritten based on regex match	Number of responses in which the Location headers are rewritten based on the configured URL regular expression.

- **Advanced Statistics**

Output Item	Description
Number of cache frames	Number of network memory blocks used by the cache.

Output Item	Description
Successful cache probes	Number of times that the appliance searches the cache table and finds matching cache objects. This does not mean that the appliance replies using the cache object. If the found cache object has expired or the request contains cookie information, the appliance will not reply using the cache object.
Cache revalidate, request with "no-cache"	Number of requests that have matching cache objects but contain "Cache-Control: no-cache", and therefore the appliance sends them to the real service and updates the cache objects using the received responses.
Cache revalidate, client IMS forward	Number of requests that carry "If-Modified-Since" and for which the matching cache objects have expired, and therefore the appliance sends the request to the real service and updates the cache objects using the received responses. Note: If the real service returns a 304 response, the appliance updates only the timestamp of the cache object.
Cache revalidate, proxy IMS forward	Number of requests that do not carry "If-Modified-Since" and for which the matching cache objects have expired, and therefore the appliance inserts "If-Modified-Since" in the requests, sends them to the real service, and updates the cache objects using the received responses.
Cache revalidate, not modified	Number of received 304 responses.
Cache miss, requests with cookies	Number of requests sent to the real service instead of being replied using the cache objects because the requests contain cookie information.
Cache miss, requests with range	Number of requests sent to the real service instead of being replied using the cache objects because the requests contain the Range header.
Cache miss, HTTP version mismatch	Number of requests sent to the real service instead of being replied using the cache objects due to HTTP version mismatch. This counter should always be zero.
Cache miss, IMS mismatch	Number of requests that carry "If-Modified-Since" and for which the matching cache objects have expired, and therefore the appliance sends the requests to the real service.
Cache miss, server driven negotiation	Number of requests sent to the real service because the requests contain the Vary header, which requires the comparison of some headers, and the appliance considers this is a cache miss according to the comparison result.
Cache miss, negative entry hit	Number of requests that trigger negative cache hits. Negative cache hit refers to the matching of a cache object that has an error code in the response header, such as 404, 302, and 503.
Requests redirected to HTTPS	Number of requests that are redirected to HTTPS.
Request with "maxage=0"	Number of requests that contain the "maxage=0" field.

Output Item	Description
Cached object had "no-cache"	Number of requests that contain the "no-cache" field.
Cache object expired	Number of times that the cached file expires.
Cache was filling when request was made	Number of requests initiated when the cache entry is being filled.
Revalidation failed due to IMS mismatched	Number of revalidation failures caused by mismatches between the "If-Modified-Since" field in the request and the "Last-Modified" field on the appliance.
Client has newer copy, cannot send from cache	Number of times that the appliance does not send a copy of the cached content because the client's copy is considered newer after a comparison between the "If-Modified-Since" field and the "Last-Modified" field.
Object in cache is chunked, cannot give to 1.0 client	Number of times that HTTP/1.0 clients fail to access the appliance because the cache objects are saved in chunked mode, which is compatible with HTTP/1.1.
Network memory utilization was too high	Number of times that mbuf insufficiency occurs.
Cache filter denied caching	Number of times that the cache filter refuses to cache.
Requests with "no-store"	Number of requests that contain the "Cache-Control: no-store" field.
Requests with "authorization"	Number of requests that contain the Authorization header.
Requests with cookies	Number of requests that contain the Cookie header.
Requests with range	Number of requests that contain the Range header.
Requests non GET, non HEAD	Number of requests that do not use the GET or HEAD method.
Requests URL too long	Number of requests that contain an excessively long URL.
Requests host too long	Number of requests that contain an excessively long host name.
Network memory shortage when cache hit (200, 304)	Number of times that memory shortage occurs when a cache object is hit, and the appliance will reply using a 200 or 304 response.
Cache was not accessible	Number of times that the cache module is not ready for caching.
Fail to send cache lookup to cache	Number of communication failures between the client and the cache module.
Fail to find url and host	Number of times that the host and URL cannot be extracted from buffer due to logic errors.
Fail to parse cache specific http request headers	Number of times that error occurs when the cache module tries to parse a request.
Fail to create a new cache object	Number of times that the appliance fails to add a cache object because of internal logic errors.

Output Item	Description
Noncacheable requests due to other errors	Number of requests that are non-cacheable due to errors unlisted.
HTTP response code not 200, 300 or 301	Number of responses whose status code is not 200, 300 or 301.
Response had a "no-store"	Number of responses that contain the "Cache-Control: no-store" field.
Response had a "private"	Number of responses that contain the "Cache-Control: private" field.
Response had a "set-cookie"	Number of responses that contain the Set-Cookie header.
Response had a "vary"	Number of responses that contain the Vary header.
We got cache miss for HEAD or PURGE method	Number of HEAD or PURGE requests that are not cached.
Could not revalidate with HEAD or PURGE method	Number of times that revalidation for HEAD or PURGE requests fails.
Response noncacheable too big	Number of responses that fail to be cached because of too large size.
Cache storage limit exceeded based on header data	Number of times that mbuf insufficiency occurs when the appliance is processing header information.
Cache storage limit exceeded based on payload	Number of times that mbuf insufficiency occurs when the appliance is processing the response body.
Network memory shortage when storing response body	Number of times that network memory shortage occurs when the appliance is caching the response body.
Cache object was deleted before response arrived	Number of times that the cache entry is deleted before the response from the real service arrives. (The proper procedure demands that the cache entry be established first and then the response is received.)
Fail to parse cache specific http response headers	Number of times that error occurs when the appliance parses the response header.
Fail to store response headers in cache	Number of times that the appliance fails to send response headers to cache.
Cache object was aborted due to connection reset	Number of times that cache suspension occurs due to connection reset.
Noncacheable responses due to other error	Number of responses that cannot be cached due to errors unlisted.
Fail to store response body in cache	Number of times that the appliance fails to send the response body to cache.
Max cache objects	Maximum number of cache objects.
Max cache frames	Maximum number of network buffers used by cache.
Times of cache drain	Counts of cache exhaustion.

clear statistics cache *[virtual_service]*

This command is used to clear all cache statistics for the specified virtual service.

virtual_service This parameter specifies the virtual service name. Its value must be:

- Virtual service name: clears the cache statistics for this virtual service.
- “all”: clears the cache statistics for all virtual services.
- “global”: clears the global cache statistics.

The default value is “global”.

Other HTTP Commands

http serverconnreuse {on|off}

This command is used to globally enable or disable the function of reusing server connections for multiple transactions. This function takes effect only after the long-living connection function is enabled (by the “http serverpersist on” command). After the reuse of server connections is enabled, each server connection can deal with multiple transactions. If this function is disabled, each server connection can only deal with a single transaction. By default, this function is enabled.



Note: The transparent mode does not support the reuse of server connections. In this mode, the function of reusing server connections is always in the enabled status (cannot be disabled) but does not take effect. For details about the transparent mode, please refer to Chapter 7 Server Load Balancing in the ArrayOS APV User Guide.

http serverconnreuse real <real_service> off

This command is used to force every server connection to be used for a single transaction for the specified real service.

real_service This parameter specifies the real service name. Its value must be an HTTP or HTTPS real service.

no http serverconnreuse real <real_service> off

This command is used to enable the function of reusing server connections for multiple transactions for the specified real service.

http serverconnreuse maxreq [num_of_conn_reuse] [real_service]

This command is used to configure the maximum number of HTTP requests allowed on each reused TCP connection for the specified HTTP or HTTPS real service. If this command is not configured, the global default value is 0, indicating no limitation on the number of HTTP requests allowed on each reused TCP connection.

num_of_conn_reuse	Optional. This parameter specifies the maximum number of HTTP requests allowed on each reused TCP connection. Its value must be an integer ranging from 0 to 1,000,000. The default value is 0, indicating no limitation on the number of HTTP requests allowed on each reused TCP connection.
real_service	Optional. This parameter specifies the name of an existing HTTP or HTTPS real service. If this parameter is not specified, the maximum number of HTTP requests allowed on each reused TCP connection will be configured for the global.



Note: If real service is not specified, the maximum number of HTTP requests allowed on each reused TCP connection is configured globally. This configuration does not take effect on the existing HTTP or HTTPS backend service, but only on the newly added HTTP or HTTPS backend service.

no http serverconnreuse maxreq [*real_service*]

This command is used to reset the maximum number of HTTP requests allowed on each reused TCP connection to the current global configuration for the specified real service. If the “real_service” parameter is not configured, the system will only delete the global configuration of this function and restore to the default value.

http serverconnreuse timeout [*timeout*] [*real_service*]

This command is used to configure the timeout value of the reused TCP connection for the specified HTTP-type or HTTPS-type real service. If this command is not configured, the timeout value of the reused TCP connection is 180 seconds.

timeout	Optional. This parameter specifies the timeout value of the reused TCP connection, in seconds. Its value must be an integer ranging from 1 to 86,400. The default value is 180.
real_service	Optional. This parameter specifies the name of an existing HTTP-type or HTTPS-type real service. If this parameter is not specified, the timeout value of the reused TCP connection will be configured for the global.

no http serverconnreuse timeout [*real_service*]

This command is used to reset the timeout value of the reused TCP connection to the current global configuration for the specified real service. If the “real_service” parameter is not configured, the system will only delete the global configuration of this function and restore to the default value.

show http serverconnreuse

This command is used to display the status (on or off) of the function of reusing server connections for multiple transactions and the configurations of the “**http serverconnreuse maxreq**” and “**http serverconnreuse timeout**” commands.

clear http serverconnreuse

This command is used to restore the configuration about the function of reusing server connections for multiple transactions and the configurations of the “**http serverconnreuse maxreq**” and “**http serverconnreuse timeout**” commands.

http connkeep {on|off} [virtual_service]

This command is used to enable or disable the Connection field (in the HTTP header) passthrough function for specified virtual service. When this function is enabled, the appliance will directly forward the Connection field in the HTTP request/response header without modification. When this function is disabled, for HTTP requests, the appliance will modify the Connection field in the HTTP header according to the configuration of the “**http serverconnreuse {on|off}**” command; for HTTP responses, the appliance will modify the HTTP header according to the HTTP request’s settings.

By default, this function is disabled for the global and enabled for individual virtual services. For a specific virtual service, this function takes effect only when it is enabled for both the global and this virtual service.

virtual_service	Optional. This parameter specifies an existing virtual service name. Its value must be an HTTP or HTTPS virtual service. When this parameter is not specified, this function will be enabled for the global.
-----------------	--

show http connkeep [virtual_service]

This command is used to display the configuration of the Connection field (in the HTTP header) passthrough function for specified virtual service.

virtual_service	Optional. This parameter specifies the virtual service name. Its value must be: • Virtual service name: displays the configuration of this function for the specified virtual service. • empty: displays the global configuration setting of this function. • all: displays both the global configuration setting of this function and the configuration of this function for all virtual services.
-----------------	---

The default value is empty.

http connsplice {on|off} [virtual_service]

This command is used to enable or disable the function of splicing the TCP connection from the client to the APV appliance and the TCP connection from the APV appliance to the real service for the specified virtual service. When the “virtual_service” parameter is not specified, this command will enable or disable this function for the global. With this function enabled, all HTTP requests on the TCP connection of the same client will be forwarded only to the same real service through the spliced TCP connection to this real service. By default, this function is disabled for the global and enabled for individual virtual services. For a specific virtual service, this function takes effect only when it is enabled for both the global and this virtual service.

virtual_service	Optional. This parameter specifies the virtual service name. Its value must be an HTTP or HTTPS virtual service. When this parameter is not specified, this function will be enabled for the global.
-----------------	--

Please note that if connections are spliced, the packets sent by the real service and containing connection states, such as FIN, will be forwarded to the client directly.



Note: It is highly recommended that the “**http serverpersist on**” command be used together with this function.

The function takes effect only when the following conditions are both met:

- The function of reusing server connections for multiple transactions has been disabled (by executing the “**http serverconnreuse off**” command);
- The HTTP cache function is disabled or the specified object cannot be cached (by executing the “**cache {on|off}**” command).

show http connsplice [*virtual_service*]

This command is used to display the configuration of the “**http connsplice**” command for the specified virtual service. If the “virtual_service” parameter is not specified, the configuration of this command for the global will be displayed.

http authsplice {on|off} <*virtual_service*>

This command is used to enable or disable the function of splicing the 401 response connections for the specified virtual service. When this function is enabled, if the real service returns a 401 response, the APV appliance will splice the TCP connection from the client to the APV appliance and the TCP connection from the APV appliance to the real service, regardless of whether the “**http connsplice off**” command is configured. By default, this function is enabled.

show http authsplice [*virtual_service*]

This command is used to display the configuration about splicing the 401 response connections for the specified virtual service. If the “virtual_service” parameter is not specified, the configurations about splicing the 401 response connections for all virtual services will be displayed.

http shuntreset {on|off}

This command is used to enable or disable the function of terminating non-reusable server connections by sending RST packets. After this function is enabled, the appliance will send RST packets to terminate non-reusable server connections. When this function is disabled, the appliance closes server connections only after receiving FIN packets. By default, this function is disabled.

show http shuntreset

This command is used to display the status of handling non-reusable server connections.

http serverconnip <virtual_service> [header_name]

This command is used to configure a server connection IP rule for the specified virtual service. Upon receiving HTTP requests sent to the virtual service, the appliance will obtain an IP address (both IPv4 and IPv6 addresses are supported) from the specified HTTP header and use it as the source IP to connect the real service. This command takes effect only in transparent mode.

If the specified virtual service already has a server connection IP rule, this command will modify the rule configuration.

virtual_service This parameter specifies the virtual service name. Its value must be an HTTP or HTTPS virtual service.

header_name Optional. This parameter specifies the name of the HTTP request header. Its value must be a case-insensitive string of 1 to 100 characters. This parameter is generally set to the name of an HTTP header inserted by the appliance, and cannot be a standard HTTP header.

The default value is “X-Forwarded-For”.

no http serverconnip <virtual_service>

This command is used to delete the server connection IP rule for the specified virtual service.

show http serverconnip [virtual_service]

This command is used to display the server connection IP rule for the specified virtual service. If the “virtual_service” parameter is not specified, the server connection IP rules for all virtual services will be displayed.

clear http serverconnip

This command is used to clear all server connection IP rules.

http turbo <virtual_service>

This command is used to enable the function of accelerating HTTP request processing for the specified virtual service. By default, this function is disabled.

virtual_service This parameter specifies the virtual service name. Its value must be an HTTP virtual service.



Note: After this function is enabled for the specified virtual service, the HTTP compression and cache functions will not be available for this virtual service.

no http turbo <virtual_service>

This command is used to disable the function of accelerating HTTP request processing for the specified virtual service.

show http turbo [virtual_service]

This command is used to display the configuration about accelerating HTTP request processing for the specified virtual service. If the “virtual_service” parameter is not specified, the configurations about accelerating HTTP request processing for all virtual services will be displayed.

clear http turbo

This command is used to restore the configurations about the function of accelerating HTTP request processing for all virtual services.

http maxheadersize [header_size]

This command is used to set the total length of HTTP and HTTPS request header and request line for the global. After this command is executed, when the total length of the received HTTP or HTTPS request header and request line is greater than the configured threshold, the system sends a Reset message to reset the connection. If this command is not configured. By default, the total length of the HTTP and HTTPS request header and request line for the global is 200 KB.

header_size Optional. This parameter specifies the total length of HTTP and HTTPS request header and request line. Its value must be an integer ranging from 1 to 10,485,760 (10M). The default value is 204,800 (200KB).

Note: When the total length of the HTTP request header and request line is 10M, each core’s utilization of the CPU of the APV device may be higher.

no http maxheadersize

This command is used to restore the total length of the HTTP and HTTPS request header and request line for the global.

show http maxheadersize

This command is used to display the total length of the HTTP and HTTPS request header and request line for the global.

http xurlencode common *[virtual_service]*

This command is used to define the common method of encoding the information inserted in an HTTP request URL for a specified virtual service. That is, the system will encode all characters except 0-9, a-z, A-Z and “%”. By default, the system adopts the common method. For this command, the configuration of every virtual service takes precedence over the global configuration.

virtual_service Optional. This parameter specifies the name of an existing virtual service. Its value must be an HTTP or HTTPS virtual service. The default value is empty, indicating the system configures the encoding method for all newly created HTTP and HTTPS virtual services.

http xurlencode space *[virtual_service]*

This command is used to define the space method of encoding the information inserted in an HTTP request URL for a specified virtual service. That is, the system will only encode the space character and nonprintable ASCII characters. By default, the system encodes all characters except 0-9, a-z, A-Z and “%”. For this command, the configuration of every virtual service takes precedence over the global configuration.

virtual_service Optional. This parameter specifies the name of an existing virtual service. Its value must be an HTTP or HTTPS virtual service.

The default value is empty, indicating the system configures the encoding method for all newly created HTTP and HTTPS virtual services.

show http xurlencode *[virtual_service]*

This command is used to display the method of encoding the information inserted in an HTTP request URL sent to a virtual service. If the “virtual_service” parameter is not specified, the system will display the global configuration and the configuration of all virtual services.

Chapter 12 DNS Cache

This chapter covers the commands for configuring DNS Cache.

dns cache {on|off}

This command is used to enable or disable the DNS cache function. When this function is enabled, the system will cache the received DNS responses and use the DNS cache to resolve domain names that have been previously queried. This function is disabled by default.

dns cache expire <min_seconds> <max_seconds>

This command is used to set the minimum and maximum DNS cache expiration time.

- If the Time To Live (TTL) of the DNS response is smaller than the minimum DNS cache expiration time, the DNS cache entry will expire after the minimum DNS cache expiration time ends.
- If the TTL of the DNS response is larger than the maximum DNS cache expiration time, the DNS cache entry will expire after the maximum DNS cache expiration time ends.
- If the TTL of the DNS response is larger than the minimum DNS cache expiration time and smaller than the maximum DNS cache expiration time, the DNS cache entry will expire after the TTL ends.

When this command is not configured, the default minimum and maximum DNS cache expiration time is 60s and 3600s respectively.

min_seconds	This parameter specifies the minimum DNS cache expiration time in seconds. Its value must be an integer ranging from 0 to 999,999. If the value is set to 0, the minimum DNS cache expiration time is 999,999s.
max_seconds	This parameter specifies the maximum DNS cache expiration time, in seconds. Its value must be an integer ranging from 0 to 999,999. If the value is set to 0, the minimum DNS cache expiration time is 999,999s. When the “min_seconds” parameter is set to a value other than “0”, the value of “max_seconds” must be larger than the value of “min_seconds”

show dns cache setting

This command is used to display the status of the DNS cache function and the settings about the minimum and maximum DNS cache expiration time.

clear dns all

This command is used to restore configurations about the DNS cache function and the minimum and maximum DNS cache expiration time.

clear dns cache content

This command is used to clear all dynamic DNS cache entries.

dns cache host <host_name> <ip>

This command is used to add a static DNS cache entry.

host_name This parameter specifies the host name. Its value must be a string of 1 to 128 characters.

ip This parameter specifies the IP address of the specified host name. Its value must be an IPv4 or IPv6 address.

no dns host <host_name>

This command is used to delete a static DNS cache entry.

show dns cache host

This command is used to display all static DNS cache entries.

clear dns host

This command is used to clear all static DNS cache entries.

show statistics dns cache

This command is used to display DNS cache statistics.

clear statistics dns cache

This command is used to clear all DNS cache statistics.

dns rewrite response rcode <virtual_service> <original_rcode> <rcode>

This command is used to configure the rewriting of the DNS response code (rcode) for the specified virtual service.

virtual_service This parameter specifies the name of an existing virtual service.

original_rcode This parameter specifies the original response code. Only when the DNS response code matches this parameter's value does the rewrite take effect. The value must be an integer ranging from 0 to 15 and enclosed by double quotes. The default value is **all**, which means it matches all

response codes.

rcode This parameter specifies the rewritten DNS response code. The value must be an integer ranging from 0 to 15.

no dns rewrite response rcode <virtual_service>

This command is used to delete the rewriting configuration of the DNS response code (rcode) for the specified virtual service.

show dns rewrite response rcode [virtual_service]

This command is used to display the rewriting configuration of the DNS response code for the specified virtual service. If the “virtual_service” parameter is not specified, the rewriting configuration for all virtual services will be displayed.

clear dns rewrite response rcode

This command is used to clear the rewriting configuration of the DNS response code for all virtual service.

dns server source <server_ip> <source_ip>

This command is used to associate the source IP address that the appliance uses to access the DNS server with the specified DNS server. The IP addresses of both parameters must be both IPv4 or IPv6 addresses. The system supports adding accessible source IP addresses for a maximum of 128 DNS servers.

server_ip This parameter specifies the IP address of the DNS server.

source_ip This parameter specifies the source IP address to be used when accessing the DNS server

no dns server source <server_ip>

This command is used to disassociate the relationship between the specified DNS server and the source IP address used by the appliance to access the DNS server.

show dns server source [server_ip]

This command is used to display the relationship between the specified DNS server and the source IP address used by the appliance to access the DNS server. If the parameter “server_ip” is not specified, the relationship between the source IP address used by the appliance to access the DNS server and all DNS servers will be displayed.

clear dns server source

This command is used to disassociate the relationship between the all DNS servers and the source IP addresses used by the appliance to access the DNS server.

Chapter 13 SSL Acceleration

This chapter covers the CLI commands for configuring the Secure Sockets Layer (SSL) function.

The SSL protocol establishes a data security layer between the application protocols (such as HTTP and FTP) and TCP/IP protocols, thus providing privacy, authentication and integrity for the services on the TCP/IP connections.

In normal situations, when communicating with the client using SSL, the SLB real service consumes many system resources to perform SSL encryption and decryption. By employing the SSL proxy technology, the APV appliance offloads the SSL encryption and decryption from the real service to the APV appliance with high SSL processing performance. This not only ensures the encrypted communication between the SLB virtual service, the client and the SLB real service, but also enhances the server performance and improves user experience.

SSL Host Configuration

ssl host {virtual|real} <host_name> [slb_service]

This command is used to create an SSL host and associate it with a specified SLB service. Before associating the SSL host with an SLB service, you need to create the SLB service first. Multiple different SLB services can be associated with the same SSL host to save SSL resources. For now, an SSL host can be associated with a maximum of 2000 SLB services.

virtual real	This parameter specifies the type of the SLB service with which the SSL host is associated. - virtual: indicates that the SSL host is associated with SLB virtual service. In this case, the SSL host acts as an SSL server and is called SSL virtual host. - real: indicates that the SSL host is associated with SLB real service. In this case, the SSL host acts as an SSL client and is called SSL real host. The SSL virtual host and SSL real host are different entities and require different configuration parameters.
host_name	This parameter specifies the SSL virtual or real host name. Its value must be a string of 1 to 240 characters. Numbers, letters and underscores are supported but its value cannot start with the underscore. Special characters and white space characters are not supported. When this command is used under a segment scope, the upper length limit of this parameter value is 100 characters.

slb_service	Optional. This parameter specifies the name of the SLB service with which the SSL host is associated. The SSL virtual host must be associated with the SLB virtual service of the HTTPS, TCPS or FTPS type. The SSL real host must be associated with the SLB real service of the HTTPS or TCPS type. If this parameter is not specified, the system will create the SSL host only.
-------------	---



Note: An SSL virtual host cannot be associated with an SLB virtual service that uses the “0.0.0.0” IP address, unless the virtual host has SSL interception enabled.

no ssl host {virtual|real} <host_name> <slb_service>

This command is used to disassociate the SSL host from the specified SLB service.

show ssl host

This command is used to display the currently configured SSL hosts and the associated SLB services.

clear ssl host <host_name>

This command is used to delete the specified SSL virtual or real host and clear all its configurations, including the private key and certificate. After this command is executed, to reconfigure this SSL host, you need to create the private key and purchase a new certificate for it again.

show ssl session <host_name> [num_of_conn]

This command is used to display the SSL connection information of the specified SSL virtual host or real host, including client IP address, server IP address, session ID and cipher suite.

host_name	This parameter specifies the SSL virtual or real host name.
-----------	---

num_of_conn	Optional. This parameter specifies the number of connections to be displayed. Its value must be an integer ranging from 1 to 1000. The default value is 100.
-------------	--

ssl sni <host_name> <domain_name>

This command is used to associate a domain name with an SSL virtual host or real host. The Server Name Indication (SNI) function provides the SSL authentication for each website that runs on an SSL server when the SSL client accesses the SSL server by a domain name instead of an IP address of the SSL server. A maximum of 64 domain names can be associated with an SSL virtual host, and a real host can have only one domain name associated with it.

This command can be configured for an enabled SSL host.

host_name	This parameter specifies the name of the SSL virtual host or real host.
domain_name	This parameter specifies the domain name. Its value must be a string of 1 to 255 characters. If the parameter value does not begin with a character, the parameter value must be enclosed in double quotes. When the “host_name” parameter is set to a virtual host name, this parameter supports wildcard “*”, which is used to match zero or more characters ahead of it. The parameter value must be in the format of “*.domain.com”. Other formats, such as “a*.domain.com” and “*a.domain.com”, are not supported.

Example:

When this parameter is set to “*.domain.com”, all domain names ending with “.domain.com” will be associated with the specified virtual host. If both “www.domain.com” and “*.domain.com” are configured, the former will be preferentially matched.

show ssl sni <host_name>

This command is used to display the domain name associated with the specified SSL host.

clear ssl sni <host_name> <domain_name>

This command is used to clear the association of the domain name with the specified SSL host.



Note: After this command is executed, all the SSL configurations (including all the private keys) associated with the specified SSL host of the specified domain name will be removed. Once the private key is removed, the administrator might have to pay for a new certificate.

ssl settings autosni <real_host_name>

This command is used to enable automatic acquisition of SNI for the specified SSL real host. When this function is enabled, the SSL real host will get the SNI from the SSL virtual host and then send it to the real service when trying to connect the real service. By default, this function is enabled.

real_host_name	This parameter specifies the SSL real host name.
----------------	--

ssl snmp oid vhost <virtual_host_name> <index>

This command is used to configure a static SNMP OID for the specified virtual host. The static SNMP OID of a virtual host is composed of a predefined SNMP OID prefix and the index specified by the “index” parameter in the command. For example, if the SNMP OID prefix of the virtual host is “.1.3.6.1.4.1.7564.20.2.4.1.4” and the SNMP OID index is 4000, the entire SNMP OID will be “.1.3.6.1.4.1.7564.20.2.4.1.4.4000”. The static SNMP OID will be never changed after being configured, and it has a higher priority than the dynamic generated SNMP OID. If no static SNMP OID is configured for a virtual host, the virtual host will use the dynamic generated one, which might change.

virtual_host	This parameter specifies the name of an existing virtual host.
index	This parameter specifies the SNMP OID index, that is, the last number in the SNMP OID. Its value must be an integer ranging from 4000 to 7999.

no ssl snmp oid vhost <virtual_host_name>

This command is used to delete the static SNMP OID of the specified virtual host. After the static SNMP OID is deleted, the virtual host will use the dynamic generated SNMP OID.

show ssl snmp oid vhost [virtual_host_name]

This command is used to display the static SNMP OID of the specified virtual host. If the “virtual_host_name” parameter is not specified, the static SNMP OIDs of all virtual hosts will be displayed.

clear ssl snmp oid vhost

This command is used to clear the static SNMP OIDs of all virtual hosts.

SSL Certificate Management

**ssl csr <host_name> [key_length] [certificate_index]
[signature_algorithm_index] [domain_name] [csr_type_index]**

This command is used to generate a Certificate Signing Request (CSR) for the specified SSL virtual or real host as well as a key pair. The system will request you to enter the information required for generating the CSR. You can choose to set the private key as exportable and set the passphrase for the private key to protect it. In addition, this command will also generate a test certificate for the SSL virtual host. This certificate is only used for testing purposes, not for production systems.

host_name	This parameter specifies the SSL virtual or real host name.
-----------	---

key_length	Optional. This parameter specifies the length of the generated key pair, in bits. Its value must be 1024, 2048 or 4096. The default value is 2048.
certificate_index	Optional. This parameter specifies the index of the generated test certificate. Its value must be 1, 2 or 3. The default value is 1.
signature_algorithm_index	Optional. This parameter specifies the index of the CSR signature algorithm. - For the appliance with the FIPS HSM installed, the value must be 1, which indicates sha1RSA. - For the appliance without the FIPS HSM installed, to generate a PKCSv1.5 RSA CSR, the value must be 1, 2, 3 or 4, indicating sha256RSA, sha384RSA, sha512RSA, and sha1RSA respectively; to generate an RSASSA-PSS CSR, the value must be 1, 2 or 3. The default value is 1 whether the FIPS HSM is installed on the APV appliance or not.
domain_name	Optional. This parameter specifies the domain name associated with the SSL virtual host. - If the “domain_name” parameter is specified, a CSR will be generated for the specified domain name associated with the specified SSL virtual host. - If the “domain_name” parameter is not specified, a CSR will be generated for the specified SSL virtual host or SSL real host.
csr_type_index	Optional. This parameter specifies the index of the CSR type. Its value must be: 0: generates a PKCSv1.5 RSA CSR. 1: generates an RSASSA-PSS CSR. The default value is 0.

**Note:**

- The signature algorithm of the test certificate uses the setting of the “signature_algorithm_index” parameter.
- The passphrase for the private key supports all types of characters and has no length limit. However, the length of the passphrase cannot be 0.
- Generating a CSR for an index with an activated certificate is supported.

```
ssl ecc csr <host_name> [curve_name] [certificate_index]  
[signature_algorithm_index] [domain_name]
```

This command is used to generate a CSR based on the Elliptic Curve Cryptography (ECC) for the specified SSL virtual or real host as well as a key pair. The system will request you to enter the information required for generating the CSR. You can choose to set the private key as exportable and set the passphrase for the private key to protect it. In addition, this command will also generate a test certificate for the SSL virtual or real host. This certificate is only used for testing purposes, not for production systems.

host_name	This parameter specifies the SSL virtual or real host name.
curve_name	Optional. This parameter specifies the elliptic curve name. Its value must be “prime256v1”, “secp384r1”, or “secp521r1”. The default value is “prime256v1”.
certificate_index	Optional. This parameter specifies the index of the generated test certificate. Its value must be 1, 2 or 3. The default value is 1.
signature_algorithm_index	Optional. This parameter specifies the index of the CSR signature algorithm. Its value must be 1, 2, 3 or 4, indicating sha256ECDSA, sha384ECDSA, sha512ECDSA, and sha1ECDSA respectively. The default value is 1.
domain_name	Optional. This parameter specifies the domain name. • If the “domain_name” parameter is specified, a CSR will be generated for the specified domain name associated with the specified SSL virtual host. • If the “domain_name” parameter is not specified, a CSR will be generated for the specified SSL virtual host.

In the following example, an SSL CSR and a key pair are generated. If the domain name is not specified, the system will request you to enter the following information:

```
AN(config)#ssl csr "ssl_vhost"
```

```
We will now gather some required information about your ssl virtual host,
```

```
This information is encoded into your certificate
```

```
Two character country code for your organization (eg. US):
```

```
State or province []:
```

```
Location or local city []:
```

```
Organization Name:
```

```
Organizational Unit:
```

```

Organizational Unit []:
Organizational Unit []:
Do you want to use the virtual host name "ssl_vhost" as the Common Name
(recommended)?(Y/N):
Email address of administrator []:
Do you want the private key to be exportable [Yes/(No)]:
Enter passphrase for the private key:
Confirm passphrase for the private key:

```

If the domain name is specified, the system will request you to enter the following information:

```

AN(config)#ssl csr "ssl_vhost" 2048 1 1 www.foo.com
We will now gather some required information about your ssl virtual host,
This information is encoded into your certificate
Two character country code for your organization (eg. US):
State or province []:
Location or local city []:
Organization Name:
Organizational Unit:
Organizational Unit []:
Organizational Unit []:
Do you want to use the SNI domain-name "www.foo.com" (of virtual host " ssl_vhost ")
as the Common Name? (recommended) [Y/N]:
Email address of administrator []:
Do you want the private key to be exportable [Yes/(No)]:
Enter passphrase for the private key:
Confirm passphrase for the private key:

```

In the above information, the Subject field “State or province”, “Location or local city” and “Email address of administrator” are optional. You can set a maximum of three values for the “Organizational Unit” field. The lengths of these Subject fields in the CSR should conform to the following limits:

- Two Character Country Code: 2 bytes
- Common Name: 64 bytes
- State or Province: 64 bytes
- Location or Local City: 64 bytes
- Organization Name: 64 bytes
- Organizational Unit: 64 bytes
- Email Address for Administrator: 80 bytes

If the domain name is specified, the field “Subject Alternate Name” will be prompted. It is optional and supports wildcard “*”. The length of this field is as follows:

- Subject Alternate Name: 256 bytes

After the above information is entered, the system will generate a CSR. You can copy it to the Email and send it to the Certificate Authority (CA) for certificate signing.

**Note:**

- The signature algorithm of the test certificate uses the setting of the “signature_algorithm_index” parameter.
- The passphrase for the private key supports all types of characters and has no length limit. However, the length of the passphrase cannot be 0.
- When the certificate related to a certificate index is in the active status, it is not allowed to generate a CSR for this index.

no ssl csr <host_name> [domain_name] [csr_type]

This command is used to delete the CSR of the specified SSL virtual or real host.

host_name This parameter specifies the SSL virtual or real host name.

domain_name Optional. This parameter specifies the domain name associated with the SSL virtual host. Its value must be:

- Domain name: deletes the CSR for this domain name.
- Empty: deletes the CSR for the specified virtual host or SSL real host (use “” to represent empty).

The default value is empty.

csr_type Optional. This parameter specifies the type of CSR to be deleted. Its value must be:

- rsa: deletes only the RSA CSR.
- ecc: deletes only the ECC CSR.
- all: deletes all types of CSRs.

The default value is “all”.



Note: The test certificate and key pair generated by the “ssl csr”, “ssl ecc csr” command will not be deleted.

show ssl csr <host_name> [domain_name] [csr_type]

This command is used to display the CSR of the specified SSL virtual or real host.

host_name This parameter specifies the SSL virtual or real host name.

domain_name Optional. This parameter specifies the domain name associated with the SSL virtual host. Its value must be:

- Domain name: displays the CSR for this domain name.
- Empty: displays the CSR for the specified virtual host or SSL real host (use “” to represent empty).

The default value is empty.

csr_type Optional. This parameter specifies the type of CSR to be displayed. Its value must be:

- **rsa**: displays only the RSA CSR.
- **ecc**: displays only the ECC CSR.
- **all**: displays all types of CSRs.

The default value is “all”.



Note: When the “**ssl import ...**” commands in the following part are executed, and the copy-n-paste way is used for importing, make sure that the “...” string is added below the content manually pasted to the CLI to end the import operation.

ssl import key <host_name> [key_index] [domain_name] [tftp_ip|url]
[file_name|password]

Imports a private key for the specified SSL virtual or real host and associate this private key with the specified key index. A maximum of three private keys can be imported for one SSL host.

When using the copy-n-paste way to import the private key, you only need to execute the command “**ssl import key**” and copy and paste the content of the private key to the CLI. This command can also be used to import the private key through TFTP, FTP, SFTP, HTTP, HTTPS, and SCP.

For the appliance with the FIPS HSM installed, only the RSA public-private key pair exported earlier can be imported. The exported RSA public-private key pair is protected by the masking secret of the FIPS HSM. If masking secret of the FIPS HSM is changed, the exported RSA public-private key pair cannot be imported back to the appliance.

host_name This parameter specifies the SSL virtual or real host name.

key_index Optional. This parameter specifies the key index with which the imported private key is associated. Its value must be 1, 2 or 3, corresponding to the three keys imported for the SSL host. The default value is 1.

domain_name Optional. This parameter specifies the domain name associated with the SSL virtual host.

- If the “domain_name” parameter is specified, the private key will be imported for the specified domain name associated with the specified SSL virtual host.
- If the “domain_name” parameter is not specified, the private key will be imported for the specified SSL host.

The default value is empty.

Note: If you need to import the private key for an SSL virtual or real host via URL (the value of the parameter “tftp_ip|url” is URL), this parameter’s value should be an empty string.

tftp_ip|url

Optional. This parameter specifies the IP address of the private key on the TFTP server, or URL of the private key.

- “tftp_ip” is used to configure the IP address of a TFTP server. Its value must be an IPv4 address. This parameter needs to be set only when you use TFTP to import the private key.
- “url” is used to configure the URL of the private key on the FTP, SFTP, TFTP, HTTP, HTTPS, or SCP server. Its value needs to be enclosed in double quotes. This parameter needs to be set only when you use URL to import the private key.

The default value is empty.

file_name|password

Optional. This parameter specifies the name of the private key’s file on the TFTP server or the password of the private key’s file obtained through a URL. The maximum length of the value is limited by the maximum length of the command line supported by the CLI. Currently, the CLI supports a maximum length of 1024 bytes for a command line.

- When the parameter “tftp_ip|url” is set to the IP address of the TFTP server, its value is the file name of the private key on the TFTP server. If the parameter “domain_name” is specified, its default value will be <hostname>_<domainname>.key.
- If the parameter “domain_name” is not specified, its default value will be <virtual_host_name>.key.
- When the parameter “tftp_ip|url” is set to the URL of the private key’s file, its value is the password of the private key. For the certificates’ files in the PFX format, the password must be set; for the certificates’ files in the PEM format, it’s optional to set the password. The value must be enclosed in double quotes.

The default value is empty.

**Note:**

- When manually copying and pasting the content of the private key to the CLI, for the appliance without the FIPS HSM installed, make sure that the content starts with the string “-----BEGIN RSA PRIVATE KEY-----” and ends with the string “-----END RSA PRIVATE KEY-----”; for the appliance with the FIPS HSM installed, make sure that the contents starts with the string “-----FIPS PUBLIC KEY-----” and ends with the string “-----FIPS END KEYS-----”.
- This command can be used to import unencrypted private key from the TFTP server. However, this is insecure and therefore is not recommended.

ssl export key <host_name> [key_index] [domain_name] [key_type]

This command is used to display the exportable private key of the specified SSL virtual or real host. You can copy the private key and save it locally for future use.

For the appliance with FIPS HSM installed, the exported file is RSA public-private key pair.

host_name	This parameter specifies the SSL virtual or real host name.
key_index	Optional. This parameter specifies the index number associated with a private key. Its value must be 0, 1, 2, or 3. The default value is 0, indicating the private key that is currently activated.
domain_name	Optional. This parameter specifies the domain name associated with the SSL virtual host. Its value must be: • Domain name: displays the private key for this domain name. • Empty: displays the private key for the specified virtual host (use “” to represent empty). The default value is empty.
key_type	Optional. This parameter specifies the type of private key to be displayed. Its value must be: • rsa: displays only the RSA private key. • ecc: displays only the ECC private key. • all: displays all types of private keys. The default value is “all”.

show ssl status key <host_name> [domain_name]

This command is used to display the key status of the specified SSL virtual host or real host.

host_name	This parameter specifies the SSL virtual or real host name.
domain_name	Optional. This parameter specifies the domain name associated with the SSL virtual host. It is available only when the “host_name” parameter is set to a virtual host name. • If this parameter is specified, the key status of this domain name associated with the SSL virtual host will be displayed. • If this parameter is not specified, the key status of all domain names associated with the SSL virtual host will be displayed.

ssl sm2 import enckey <host_name> [key_index] [tftp_ip|url] [file_name]

This command is used to import the private key of an SM2 encryption key pair (encrypted private key) for a specified SSL virtual or real host, and associate this private key with a key index. This command supports two ways to import. The administrator can manually paste the content of the encrypted private key on the command line, or import the encrypted private key via TFTP.

host_name	This parameter specifies the name of an SSL virtual or real host.
key_index	Optional. This parameter specifies an index for the imported encrypted private key. The value must be 1, 2, or 3, which corresponds to the three SM2 encrypted private keys imported for the SSL host. The default value is one (1).
tftp_ip url	Optional. This parameter specifies the IP address of the encrypted private key on the TFTP server, or the URL of the encrypted private key. • “tftp_ip” is used to configure the IP address of the TFTP server. The value must be an IPv4 address. This parameter needs to be set only when you use TFTP to import the encrypted private key’s file. • “url” is used to configure the URL of the encrypted private key on the FTP, SFTP, TFTP, HTTP, HTTPS, or SCP server. Its value needs to be enclosed in double quotes. This parameter needs to be set only when you use URL to import the encrypted private key’s file. The default value is empty.
file_name	Optional. This parameter specifies the name of the encrypted private key’s file. The default value is “<Host

name>.key.”

This parameter needs to be set only when you use TFTP to import the encrypted private key, that is, the value of the parameter “tftp_ip|url” is the IP address of the TFTP server.



Note: If the encrypted certificate and signed certificate are imported for the indices of three certificates of each SSL virtual host, the system supports importing encrypted private keys for a maximum of 167 SSL virtual host. This limitation is applicable to only Passcards.

ssl sm2 import encertificate <host_name> [certificate_index] [tftp_ip|url] [file_name]

This command is used to import an SM2 encrypted certificate for a specified SSL virtual or real host, and associate this certificate with a certificate index. This command supports two ways to import. The administrator can manually paste the content of the certificate on the command line, or import the certificate via TFTP. The imported certificate needs to be activated using the command **ssl activate certificate**.

host_name This parameter specifies the name of an SSL virtual or real host.

certificate_index Optional. This parameter specifies an index for the imported SM2 encrypted certificate. The value must be 1, 2, or 3, which corresponds to the three SM2 encrypted certificate imported for the SSL host.

The default value is one (1).

tftp_ip|url Optional. This parameter specifies the IP address of the encrypted certificate on the TFTP server, or URL of the encrypted certificate.

- “tftp_ip” is used to configure the IP address of the TFTP server. The value must be an IPv4 address. This parameter needs to be set only when you use TFTP to import the SM2 encrypted certificate.
- “url” is used to configure the URL of the SM2 encrypted certificate on the FTP, SFTP, TFTP, HTTP, HTTPS, or SCP server. Its value needs to be enclosed in double quotes. This parameter needs to be set only when you use URL to import the SM2 encrypted certificate.

The default value is empty.

file_name Optional. This parameter specifies the name of the SM2 encrypted certificate’s file on the TFTP server. The default value is “<Host name>.cert.”

This parameter needs to be set only when you use TFTP to import the SM2 encrypted certificate, that is, the value of the parameter “tftp_ip|url” is the IP address of the TFTP

server.

ssl sm2 import signkey <host_name> <key_index> [tftp_ip|url] [file_name]

This command is used to import the private key of an SM2 signing key pair (signed private key) for a specified SSL virtual or real host, and associate this private key with a key index. This command supports two ways to import. The administrator can manually paste the content of the signed private key on the command line, or import the signed private key via TFTP.

host_name	This parameter specifies the name of an SSL virtual or real host.
key_index	Optional. This parameter specifies an index for the imported signed private key. The value must be 1, 2, or 3, which corresponds to the three signed private keys imported for the SSL host. The default value is one (1).
tftp_ip url	Optional. This parameter specifies the IP address of the signed private key on the TFTP server, or the URL of the signed private key. “tftp_ip” is used to configure the IP address of the TFTP server. The value must be an IPv4 address. This parameter needs to be set only when you use TFTP to import the signed private key. “url” is used to configure the URL of the signed private key on the FTP, SFTP, TFTP, HTTP, HTTPS, or SCP server. Its value needs to be enclosed in double quotes. This parameter needs to be set only when you use URL to import the signed private key. The default value is empty.
file_name	Optional. This parameter specifies the name of the signed private key’s file on the TFTP server. The default value is “<Host name>.key.” This parameter needs to be set only when you use TFTP to import the signed private key, that is, the value of the parameter “tftp_ip url” is the IP address of the TFTP server.



Note: If the encrypted certificate and signed certificate are imported for the indices of three certificates of each SSL virtual host, the system supports importing signed private keys for a maximum of 167 SSL virtual host. This limitation is applicable to only Passcards.

ssl sm2 import signcertificate <host_name> [certificate_index] [tftp_ip|url] [file_name]

This command is used to import an SM2 signed certificate for a specified SSL virtual or real host, and associate this certificate with a certificate index. This command

supports two ways to import. The administrator can manually paste the content of the certificate on the command line, or import the certificate via TFTP. The imported certificate needs to be activated using the command **ssl activate certificate**.

host_name	This parameter specifies the name of an SSL virtual or real host.
certificate_index	Optional. This parameter specifies an index for the imported SM2 signed certificate. The value must be 1, 2, or 3, which corresponds to the three SM2 signed certificate imported for the SSL host. The default value is one (1).
tftp_ip url	Optional. This parameter specifies the IP address of the SM2 signed certificate on the TFTP server or the URL of the SM2 signed certificate. • “tftp_ip” is used to configure the IP address of the TFTP server. The value must be an IPv4 address. This parameter needs to be set only when you use TFTP to import the SM2 signed certificate. • “url” is used to configure the URL of the SM2 signed certificate on the FTP, SFTP, TFTP, HTTP, HTTPS, or SCP server. Its value needs to be enclosed in double quotes. This parameter needs to be set only when you use URL to import the SM2 signed certificate. The default value is empty.
file_name	Optional. This parameter specifies the name of the SM2 signed certificate’s file on the TFTP server. The default value is “<Host name>.crt.” This parameter needs to be set only when you use TFTP to import the SM2 signed certificate, that is, the value of the parameter “tftp_ip url” is the IP address of the TFTP server.

ssl sm2 import challengekey <host_name> [key_index] [tftp_ip|url] [file_name]

This command is used to import a challenge key for a specified SSL virtual or real host, decrypt the imported digital envelopes in the CFCA format, and associate the challenge key with an index. The system supports importing a maximum of three private keys for a single SSL host.

host_name	This parameter specifies the name of an SSL virtual or real host.
key_index	Optional. This parameter specifies an index for the imported challenge key. The value must be 1, 2, or 3, which corresponds to the three challenge keys imported for

	the SSL host.
	The default value is one (1).
tftp_ip url	Optional. This parameter specifies the IP address of the challenge key on the TFTP server or URL of the challenge key.
	• “tftp_ip” is used to configure the IP address of the TFTP server. The value must be an IPv4 address. This parameter needs to be set only when you use TFTP to import the challenge key. • “url” is used to configure the URL of the challenge key on the FTP, SFTP, TFTP, HTTP, HTTPS, or SCP server. Its value needs to be enclosed in double quotes. This parameter needs to be set only when you use URL to import the challenge key.
	The default value is empty.
file_name	Optional. This parameter specifies the name of the challenge key’s file on the TFTP server. The default value is “<Host name>.key.”
	This parameter needs to be set only when you use TFTP to import the challenge key, that is, the value of the parameter “tftp_ip url” is the IP address of the TFTP server.

ssl sm2 export challengekey <host_name> [key_index]

This command is used to export a challenge key for a specified SSL virtual or real host.

host_name	This parameter specifies the name of an SSL virtual or real host.
key_index	Optional. This parameter specifies an index for the exported challenge key. The value must be 1, 2, or 3. The default value is one (1).

ssl sm2 import encevp <host_name> [certificate_index] [format_index] [url]

This command is used to import a digital envelope in a specified format for a specified SSL virtual or real host, and associate the digital envelope with a certificate index. The digital envelope returned from CA contains an encrypted certificate and encrypted private key.

host_name	This parameter specifies the name of an SSL virtual or real host.
-----------	---

certificate_index	Optional. This parameter specifies an index for the certificate. The value must be 1, 2, or 3. The default value is one.
format_index	Optional. This parameter specifies an index of the digital envelope format. The value must be 1 or 2. 1: SCCA format. 2: CFCA format. The default value is one (1).
url	Optional. This parameter specifies the URL of the digital envelope on the FTP, SFTP, TFTP, HTTP, HTTPS, or SCP server. Its value needs to be enclosed in double quotes. The default value is empty.

ssl import certificate <host_name> [certificate_index] [domain_name] [tftp_ip|url] [filename|password]

This command is used to import a certificate for the specified SSL virtual or real host and associate the certificate with the specified certificate index. You need to activate the desired certificate using the command “**ssl activate certificate**”.

When using the copy-n-paste way to import the certificate, you only need to execute the command “**ssl import certificate**” and copy and paste the content of the certificate to the CLI. Only certificates in the PEM format can be imported in the copy-n-paste way. This command can also be used to import certificates in the PEM, DER and PFX formats through TFTP, FTP, SFTP, HTTP, HTTPS, and SCP.

host_name	This parameter specifies the SSL virtual or real host name.
certificate_index	Optional. This parameter specifies the certificate index with which the imported certificate is associated. Its value must be 1, 2 or 3, corresponding to the three certificates imported for the SSL host. The default value is 1. For the SSL host, the private key and certificate associated with the same certificate index must match each other; otherwise, the system will prompt an error message.
domain_name	Optional. This parameter specifies the domain name associated with the SSL virtual host. • If the “domain_name” parameter is specified, the certificate will be imported for the specified domain name associated with the specified SSL virtual host. • If the “domain_name” parameter is not specified, the

certificate will be imported for the specified SSL host.

The default value is empty.

Note: If you need to import the certificate for an SSL virtual or real host via URL (the value of the parameter “tftp_ip|url” is URL), this parameter’s value should be an empty string.

tftp_ip|url

Optional. This parameter specifies the IP address of the certificate on the TFTP server, or the URL of the certificate.

- “tftp_ip” is used to configure the IP address of a TFTP server. Its value must be an IPv4 address. This parameter needs to be set only when you use TFTP to import the certificate.
- “url” is used to configure the URL of the certificate on the FTP, SFTP, TFTP, HTTP, HTTPS, or SCP server. Its value needs to be enclosed in double quotes. This parameter needs to be set only when you use URL to import the certificate.

The default value is empty.

file_name|password

Optional. This parameter specifies the name of the certificate’s file on the TFTP server or the password of the certificate’s file obtained through a URL.

- When the value of the parameter “tftp_ip|url” is set to the IP address of the TFTP server, the value of this parameter is the name of the certificate’s file on the TFTP server. If the parameter "domain name" is specified, its default value will be <hostname>_<domainname>.crt. If the parameter “domain_name” is not specified, its default value will be <virtual_host_name>.crt.
- When the parameter “tftp_ip|url” is set to the URL of the certificate’s file, its value is the password of the certificate’s file. The value must be enclosed in double quotes.

The default value is empty.

show ssl import certificate match <host_name> [certificate_index]
[domain_name] [certificate_type] [url] [password]

This command is used to check whether the specified certificate has been imported into the system.

To specify the certificate to be checked, you can manually paste the certificate content in the command's interactive mode or specify the HTTP/FTP/TFTP URL address from which the certificate file is imported.

host_name	This parameter specifies the SSL virtual host or SSL real host name.
certificate_index	Optional. This parameter specifies the certificate index to be matched. Its value must be 1, 2, 3 or 0, corresponding to the three certificates imported for the SSL host and all certificates. The default value is 0.
domain_name	Optional. This parameter specifies the associated domain name of the SSL host to be matched. To skip the setting of this parameter, use double quotes ("") to indicate that the value is empty. The default value is empty.
certificate_type	Optional. This parameter specifies the certificate type to be matched. Its value must be: • rsa: matches the RSA certificate type. • ecc: matches the ECC certificate type. • all: matches both the RSA and ECC certificate types. The default value is "all".
url	Optional. This parameter specifies the URL where the certificate file locates. To skip the setting of this parameter, use double quotes ("") to indicate that the value is empty. The default value is empty.
password	Optional. This parameter specifies the password of the certificate file. The default value is empty.

no ssl certificate <host_name> <certificate_index> [domain_name]
[certificate_type]

This command is used to delete a certificate of the specified SSL host.

The system supports deleting activated certificates.

host_name	This parameter specifies the SSL virtual host or real host name.
certificate_index	This parameter specifies the certificate index with which the imported certificate is associated. Its value must be 1, 2

or 3.

domain_name	Optional. This parameter specifies the domain name associated with the SSL virtual host. Its value must be: - Domain name: deletes the certificate for this domain name. - Empty: deletes the certificate for the specified virtual host (use “” to represent empty). The default value is empty.
certificate_type	Optional. This parameter specifies the type of certificate to be deleted. Its value must be: - rsa: deletes only the RSA certificate. - ecc: deletes only the ECC certificate. - all: deletes all certificates. The default value is “all”.

ssl activate certificate *<host_name> [certificate_index] [domain_name] [certificate_type]*

This command is used to activate a certificate of the specified SSL virtual or real host. You can activate an RSA or RSAPSS certificate and an ECC certificate for each domain name associated with an SSL virtual host. An SSL virtual host that is not associated with a domain name can have one active RSA or RSAPSS certificate and one active ECC certificate. Each real host can have only one active RSA or RSAPSS certificate and one active ECC certificate.

host_name	This parameter specifies the SSL virtual or real host name.
certificate_index	Optional. This parameter specifies the certificate index. Its value must be 1, 2 or 3, corresponding to the three certificates imported for the SSL host. The default value is 1.
domain_name	Optional. This parameter specifies the domain name associated with the SSL virtual host. Its value must be: - Domain name: activates the certificate for this domain name. - Empty: activates the certificate for the specified virtual host (use “” to represent empty). The default value is empty.
certificate_type	Optional. This parameter specifies the type of certificate to

be activated. Its value must be:

- `rsa`: activates the RSA or RSAPSS certificate.
- `ecc`: activates the ECC certificate.
- `all`: activates all types of certificates.

The default value is “all”.

ssl deactivate certificate <host_name> <domain_name> <certificate_type>

This command is used to deactivate the certificate of the specified SSL virtual host or real host. You can also deactivate the current active certificate by activating another one using the “**ssl activate certificate** <host_name> [*certificate_index*] [*domain_name*] [*certificate_type*]” command. **Note:** The SSL host needs to be disabled by using the “**ssl stop**” command before you deactivate its currently active certificate by using the “**ssl deactivate certificate**” command, or replace an active certificate of it with another one by using the “**ssl activate certificate**” command.

host_name	This parameter specifies the SSL virtual host or real host name.
domain_name	This parameter specifies the domain name associated with the SSL virtual host. Its value must be • Domain name: deactivates the certificate for this domain name. • “”: deactivates the certificate for the specified virtual host.
certificate_type	This parameter specifies the type of certificate to be deactivated. Its value must be: • <code>rsa</code>: deactivates the RSA or RSAPSS certificate. • <code>ecc</code>: deactivates the ECC certificate. • <code>all</code>: deactivates all types of certificates.

show ssl certificate <host_name> [*display_mode*] [*certificate_index*] [*domain_name*] [*certificate_type*]

This command is used to display the certificate information about the specified SSL virtual or real host.

host_name	This parameter specifies the SSL virtual or real host name. Its value must be: • SSL virtual or real host name: indicates that information about certificate on the specified SSL
-----------	---

virtual or real host will be displayed.

- all: indicates that information about the server certificate, rootca certificate, interca certificate, real host certificate and global CA certificate of the virtual host will be displayed.

When the parameter value is all, the parameter after “host_name” will be invalid. The system will output all the display modes, certificate indexes, associated domain names, certificate types and other information.

display_mode

Optional. This parameter specifies the certificate display mode for the specified SSL virtual or real host. Its value must be:

- complete: indicates that the system will display the complete content of the certificate.
- simple: indicates that the system will display only the “Issuer”, “Validity” and “Subject” fields of the certificate.

The default value is “complete”.

certificate_index

Optional. This parameter specifies the certificate index. Its value must be:

- 1, 2 or 3: indicates that the system will display the information about the certificate with the specified index of the SSL host.
- 0: indicates that the system will display the information about the certificate activated for the SSL host.

The default value is 0.

domain_name

Optional. This parameter specifies the domain name associated with the SSL virtual host. Its value must be:

- Domain name: displays the certificate information for this domain name.
- Empty: displays the certificate information for the specified virtual host (use “” to represent empty).

The default value is empty.

certificate_type

Optional. This parameter specifies the type of certificate to be displayed. Its value must be:

- rsa: displays only the RSA or RSAPSS certificate information.

- ecc: displays only the ECC certificate information.
- all: displays the information of all types of certificates.

The default value is “all”.

show ssl status certificate <host_name> [domain_name]

This command is used to display the certificate status of the specified SSL virtual or real host.

host_name	This parameter specifies the SSL virtual host or real host name.
domain_name	Optional. This parameter specifies the domain name associated with the SSL virtual host. It is available only when the “host_name” parameter is set to a virtual host name. • If this parameter is specified, the certificate status of this domain name associated with the SSL virtual host will be displayed. • If this parameter is not specified, the certificate status of all domain names associated with the SSL virtual host will be displayed.

ssl import interca [host_name] [domain_name] [tftp_ip|url] [file_name]

This command is used to import a trusted CA certificate for the specified SSL virtual host, SSL real host or the global. The trusted CA certificate imported for the global will be added to the preinstalled trusted CA certificate list and will be used by SSL real hosts that do not have trusted CA certificates to validate the SSL server certificates.

When using the copy-n-paste way to import the trusted CA certificate, you only need to execute the command “**ssl import rootca**” and copy and paste the content of the trusted CA certificate to the CLI. Only trusted CA certificates in the PEM format can be imported in the copy-n-paste way. This command can also be used to import trusted CA certificates in the PEM and DER formats through TFTP, FTP, SFTP, HTTP, HTTPS, and SCP.

host_name	Optional. This parameter specifies the SSL host name. Its value must be: • SSL virtual host name: indicates that the trusted CA certificate is imported for the SSL virtual host with the client authentication function enabled to validate client certificates. • SSL real host name: indicates that the trusted CA certificate is imported for the SSL real hosts. The
-----------	--

imported certificate will be used to validate SSL server certificates.

- ALL: indicates that the trusted CA certificate is imported into the preinstalled trusted CA certificate list (as the root CA of the certificate chain) to validate SSL server certificates by all SSL real hosts.

The default value is “ALL”.

domain_name

Optional. This parameter specifies the domain name.

- If the “domain_name” parameter is specified, the root CA certificate will be imported for the specified domain name associated with the specified SSL virtual host.
- If the “domain_name” parameter is not specified, the root CA certificate will be imported for the specified SSL virtual host.

The default value is empty.

Note: If you need to import the trusted CA certificate for an SSL virtual or real host via URL (the value of the parameter “tftp_ip|url” is URL), this parameter’s value should be an empty string.

tftp_ip|url

Optional. This parameter specifies the IP address or the URL of the TFTP server of the trusted CA certificate.

- “tftp_ip” is used to configure the IP address of a TFTP server. Its value must be an IPv4 address. This parameter needs to be set only when you use TFTP to import the trusted CA certificate.
- “url” is used to configure the URL of the trusted CA certificate on the FTP, SFTP, TFTP, HTTP, HTTPS, or SCP server. Its value needs to be enclosed in double quotes. This parameter needs to be set only when you use URL to import the trusted CA certificate.

The default value is empty.

file_name

Optional. This parameter specifies the file name of the trusted CA certificate on the TFTP server. The maximum length of its value is restricted by the maximum length of the command line supported by CLI. Currently, CLI supports the command line with the maximum length of 1024 characters.

- For the trusted CA certificate imported for the SSL

virtual host, the default value is “<host_name>.cert”.

- For the trusted CA certificate imported into the preinstalled trusted CA certificate list, the default value is “grootca.crt”.

This parameter needs to be set only when you use TFTP to import the trusted CA certificate, that is, the value of the parameter “tftp_ip|url” is the IP address of the TFTP server.



Note: Before executing this command to import the trusted CA certificate for the SSL virtual host, make sure that this SSL virtual host is in the inactive status; otherwise, the import will fail. If this SSL virtual host is activated, please use the command “**ssl stop**” to disable this SSL virtual host.

ssl import rootca [*host_name*] [*domain_name*] [*tftp_ip|url*] [*file_name*]

This command is used to import a trusted CA certificate for the specified SSL virtual host, SSL real host or the global. The trusted CA certificate imported for the global will be added to the preinstalled trusted CA certificate list and will be used by SSL real hosts that do not have trusted CA certificates to validate the SSL server certificates.

When using the copy-n-paste way to import the trusted CA certificate, you only need to execute the command “**ssl import rootca**” and copy and paste the content of the trusted CA certificate to the CLI. Only trusted CA certificates in the PEM format can be imported in the copy-n-paste way. This command can also be used to import trusted CA certificates in the PEM and DER formats through TFTP, FTP, SFTP, HTTP, HTTPS, and SCP.

- | | |
|------------------|--|
| host_name | Optional. This parameter specifies the SSL host name. Its value must be: <ul style="list-style-type: none"> • SSL virtual host name: indicates that the trusted CA certificate is imported for the SSL virtual host with the client authentication function enabled to validate client certificates. • SSL real host name: indicates that the trusted CA certificate is imported for the SSL real hosts. The imported certificate will be used to validate SSL server certificates. • ALL: indicates that the trusted CA certificate is imported into the preinstalled trusted CA certificate list (as the root CA of the certificate chain) to validate SSL server certificates by all SSL real hosts. |
|------------------|--|

The default value is “ALL”.

- | | |
|--------------------|---|
| domain_name | Optional. This parameter specifies the domain name. <ul style="list-style-type: none"> • If the “domain_name” parameter is specified, the root |
|--------------------|---|

CA certificate will be imported for the specified domain name associated with the specified SSL virtual host.

- If the “domain_name” parameter is not specified, the root CA certificate will be imported for the specified SSL virtual host.

The default value is empty.

Note: If you need to import the trusted CA certificate for an SSL virtual or real host via URL (the value of the parameter “tftp_ip|url” is URL), this parameter’s value should be an empty string.

tftp_ip|url

Optional. This parameter specifies the IP address of the trusted CA certificate on the TFTP server, or URL of the trusted CA certificate.

- “tftp_ip” is used to configure the IP address of a TFTP server. Its value must be an IPv4 address. This parameter needs to be set only when you use TFTP to import the trusted CA certificate.
- “url” is used to configure the URL of the trusted CA certificate on the FTP, SFTP, TFTP, HTTP, HTTPS, or SCP server. Its value needs to be enclosed in double quotes. This parameter needs to be set only when you use URL to import the private key.

The default value is empty.

file_name

Optional. This parameter specifies the file name of the trusted CA certificate on the TFTP server. The maximum length of its value is restricted by the maximum length of the command line supported by CLI. Currently, CLI supports the command line with the maximum length of 1024 characters.

- For the trusted CA certificate imported for the SSL virtual host, the default value is “<host_name>.cert”.
- For the trusted CA certificate imported into the preinstalled trusted CA certificate list, the default value is “grootca.cert”.

This parameter needs to be set only when you use TFTP to import the trusted CA certificate, that is, the value of the parameter “tftp_ip|url” is the IP address of the TFTP server.



Note: Before executing this command to import the trusted CA certificate for the SSL virtual host, make sure that this SSL virtual host is in the inactive

status; otherwise, the import will fail. If this SSL virtual host is activated, please use the command “**ssl stop**” to disable this SSL virtual host.

no ssl rootca <host_name> [certificate_number]

This command is used to delete a trusted CA certificate from the specified SSL virtual host, SSL real host or the global.

host_name This parameter specifies the SSL host name. Its value must be:

- SSL virtual host name: indicates that the trusted CA certificate will be deleted from the SSL virtual host.
- SSL real host name: indicates that the trusted CA certificate will be deleted from the SSL real hosts.
- ALL: indicates that the trusted CA certificate will be deleted from the preinstalled trusted CA certificate list.

The default value is “ALL”.

certificate_number Optional. This parameter specifies the number of the trusted CA certificate to be deleted. You can view the numbers of the trusted CA certificates by executing the “**show ssl rootca**” command. If this parameter is not specified, the system will delete all trusted CA certificates.



Note: Before executing this command to delete the trusted CA certificate from an SSL virtual host, make sure that this SSL virtual host is in the inactive status; otherwise, the deletion will fail. If this SSL virtual host is activated, please use the command “**ssl stop** <host_name>” to disable this SSL virtual host.

show ssl rootca [host_name] [display_mode] [domain_name]

This command is used to display all trusted CA certificates of the specified SSL virtual host, SSL real host or the global.

host_name Optional. This parameter specifies the SSL host name. Its value must be:

- SSL virtual host name: indicates that all trusted CA certificates of the SSL virtual host will be displayed.
- SSL real host name: indicates that all trusted CA certificates of the SSL real hosts will be displayed.
- ALL: indicates that all trusted CA certificates in the preinstalled trusted CA certificate list will be displayed.

The default value is “ALL”.

display_mode

Optional. This parameter specifies the display mode of the trusted CA certificate. Its value must be:

- complete: indicates that the system will display the complete content of the trusted CA certificate.
- simple: indicates that the system will display only the “Issuer”, “Validity” and “Subject” fields of the trusted CA certificate.

The default value is “complete”.

domain_name

Optional. This parameter specifies the domain name.

- If the “domain_name” parameter is specified, the root CA certificate of the specified domain name associated with the specified SSL virtual host will be displayed.
- If the “domain_name” parameter is not specified, the root CA certificate of the specified SSL virtual host will be displayed.

no ssl sni rootca <virtual_host_name> [domain_name] [certificate_index]

This command is used to delete a root CA certificate of the specified domain name for the specified SSL virtual host.

virtual_host_name

This parameter specifies the name of the SSL virtual host.

domain_name

Optional. This parameter specifies the domain name. If the “domain_name” parameter is not specified, all the root CA certificates of each domain name for the specified SSL virtual host will be deleted.

certificate_index

Optional. This parameter specifies the index of the root CA certificate to be deleted. You can view the index of the root CA certificate by executing the “**show ssl rootca**” command.



Note: The root CA certificate can be deleted only when the root CA certificate is inactive.

ssl import interca <virtual_host_name> [domain_name] [tftp_ip|url] [file_name]

This command is used to import an intermediate CA certificate for the specified SSL virtual host to compose a complete certificate chain with the activated certificate and trusted CA certificate of the SSL virtual host. The system supports importing a

maximum of 8 intermediate CA certificates for the constructed certificate chain. If the imported CA certificate is more than eight, all intercas cannot be sent.

When using the copy-n-paste way to import the intermediate CA certificate, you only need to execute the command “**ssl import interca**” and copy and paste the content of the intermediate CA certificate to the CLI. Only intermediate CA certificates in the PEM format can be imported in the copy-n-paste way. This command can also be used to import intermediate CA certificates in the PEM and DER formats through TFTP, FTP, SFTP, HTTP, HTTPS, and SCP.

virtual_host_name	This parameter specifies the SSL virtual host name.
domain_name	Optional. This parameter specifies the domain name. • If the “domain_name” parameter is specified, the intermediate CA certificate will be imported for the specified domain name associated with the specified SSL virtual host. • If the “domain_name” parameter is not specified, the intermediate CA certificate will be imported for the specified SSL virtual host. The default value is empty. Note: If you need to import the intermediate CA certificate for an SSL virtual host via URL (the value of the parameter “tftp_ip url” is URL), this parameter’s value should be an empty string.
tftp_ip url	Optional. This parameter specifies the IP address of the intermediate CA certificate on the TFTP server, or the URL of the intermediate CA certificate. • “tftp_ip” is used to configure the IP address of a TFTP server. Its value must be an IPv4 address. This parameter needs to be set only when you use TFTP to import the intermediate CA certificate. • “url” is used to configure the URL of the intermediate CA certificate on the FTP, SFTP, TFTP, HTTP, HTTPS, or SCP server. Its value needs to be enclosed in double quotes. This parameter needs to be set only when you use URL to import the intermediate CA certificate. The default value is empty.
file_name	Optional. This parameter specifies the file name of the intermediate CA certificate on the TFTP server. The maximum length of its value is restricted by the maximum length of the command line supported by CLI. Currently,

CLI supports the command line with the maximum length of 1024 characters. The default value is “<virtual_host_name>.crt”.

This parameter needs to be set only when you use TFTP to import the intermediate CA certificate, that is, the value of the parameter "tftp_ip|url" is the IP address of the TFTP server.

no ssl interca <virtual_host_name> [certificate_number]

This command is used to delete an intermediate CA certificate from the specified SSL virtual host

virtual_host_name	This parameter specifies the SSL virtual host name.
certificate_number	Optional. This parameter specifies the number of the intermediate CA certificate to be deleted. You can view the numbers of the intermediate CA certificates by executing the “ show ssl interca ” command. If this parameter is not specified, the system will delete all intermediate CA certificates.

show ssl interca <virtual_host_name> [display_mode] [domain_name]

This command is used to display all intermediate CA certificates of the specified SSL virtual host.

virtual_host_name	This parameter specifies the SSL virtual host name.
display_mode	Optional. This parameter specifies the display mode of the intermediate CA certificate. Its value must be: • complete: indicates that the system will display the complete content of the intermediate CA certificate. • simple: indicates that the system will display only the “Issuer”, “Validity” and “Subject” fields of the intermediate CA certificate. The default value is “complete”.
domain_name	Optional. This parameter specifies the domain name. • If the “domain_name” parameter is specified, the intermediate CA certificate of the specified domain name associated with the specified SSL virtual host will be displayed. • If the “domain_name” parameter is not specified, the intermediate CA certificate of the specified SSL virtual

host will be displayed.

no ssl sni interca <virtual_host_name> [domain_name] [certificate_index]

This command is used to delete an intermediate CA certificate of the specified domain name for the specified SSL virtual host.

virtual_host_name	This parameter specifies the SSL virtual host name.
domain_name	Optional. This parameter specifies the domain name. If the “domain_name” parameter is not specified, all the intermediate CA certificates of each domain name for the specified SSL virtual host will be deleted.
certificate_index	Optional. This parameter specifies the index of the intermediate CA certificate to be deleted. You can view the index of the intermediate CA certificate by executing the “ show ssl interca ” command.



Note: The intermediate CA certificate can be deleted only when the intermediate CA certificate is inactive.

ssl backup certificate <host_name> <file_name> <password>
[domain_name]

This command is used to back up the certificate and private key currently used by the specified SSL virtual or real host to a PFX file.

- For RSA and ECC, the system backs up the currently used RSA certificate and its private key, and ECC certificate and its private key into a PFX file respectively.
- For SM2, the system generates two .tgz files for the SM2 certificate. One .tgz file is encrypted and contains activated and non-activated certificates, private key, trusted CA certificate (see the command **ssl import rootca**), and intermediate CA certificate (see the command **ssl import interca**). The other .tgz file is not encrypted and contains activated and non-activated certificates, trusted CA certificate, intermediate CA certificate, and an imported challenge key.

The system packs this PFX file, the CRL CA certificates (refer to the “**ssl import crlca**” command), the trusted CA certificates (refer to the “**ssl import rootca**” command) and intermediate CA certificates (refer to the “**ssl import interca**” command) of the SSL host into a .tgz file. This .tgz file can be saved in the system or on a specified TFTP server. When anyone accesses this file, the correct password is required.

host_name	This parameter specifies the SSL virtual or real host name. The value must be:
	• The name of an SSL virtual or real host: Back up the certificate and private key for the SSL virtual or real

host.

- all: Back up the certificates, private keys, CA certificates, intermediate CA certificates, CRL CA certificates, challenge keys, and the certificates related to the SNI domain name for all SSL virtual and real hosts. Note that the generated backup package is an encrypted file that cannot be opened or unzipped.

file_name

This parameter specifies the name of the backup file. Numbers, letters and underscores are supported. It is recommended to enclose its value by double quotes.

- When the .tgz file is saved in the system: Its value must be a valid system file name (excluding the path and extension name).
- When the .tgz file is saved on the TFTP server: Its value must be in the format of “tftp://server_ip/filename”.

The maximum length of its value is restricted by the maximum length of the command line supported by CLI. Currently, CLI supports the command line with the maximum length of 1024 characters.

password

This parameter specifies the password for accessing the backup file. Numbers, letters and underscores are supported. It is recommended to enclose its value by double quotes. The maximum length of its value is restricted by the maximum length of the command line supported by CLI. Currently, CLI supports the command line with the maximum length of 1024 characters.

domain_name

Optional. This parameter specifies the domain name associated with the SSL host.

- If the parameter “domain_name” parameter is specified, the certificate and private key of the specified domain name associated with the specified SSL virtual host will be backed up.
- If the parameter “domain_name” **parameter** is not specified, the certificate and private key of the specified SSL host will be backed up.



Note:

- Before executing this command, make sure that the certificate to be backed up has been activated using the “**ssl activate certificate**” command.
- Even if you run this command several times to back up the certificate and

private key file to the system, the system retains only the two most recent backup files.

no ssl backup certificate *<host_name> <file_name> [domain_name]*

This command is used to delete the backup file of the certificate and private key saved in the system for the specified SSL host. The value of the “file_name” parameter must be a valid system file name. If “domain_name” the parameter is not specified, the backup file name that contains the certificate and private key of the specified SSL host will be removed; otherwise, the backup file name that contains the certificate and private key of the specified domain name associated with the specified SSL virtual host will be removed.

show ssl backup certificate *<host_name> [domain_name]*

This command is used to display the backup file of the certificate and private key saved in the system for the specified SSL host. If the “domain_name” parameter is not specified, the backup file name that contains the certificate and private key of the specified SSL host will be displayed; otherwise, the backup file name that contains the certificate and private key of the specified domain name associated with the specified SSL virtual host will be displayed.

ssl restore certificate *<host_name> <file_name> <password> [domain_name]*

This command is used to restore the .tgz backup file saved in the system or on the TFTP server for the specified SSL virtual or real host to the system.

host_name	This parameter specifies the SSL virtual or real host name.
file_name	This parameter specifies the name of the backup file to be restored. The file name does not support special characters except underscores. • When the .tgz file is saved in the system: Its value must be a valid system file name (excluding the path). • When the .tgz file is saved on the TFTP server: Its value must be in the format of “tftp://server_ip/filename”.
password	This parameter specifies the password for accessing the backup file to be restored.
domain_name	Optional. This parameter specifies the domain name associated with the SSL virtual host. • If the “domain_name” parameter is specified, the certificate and private key of the specified domain name associated with the specified SSL virtual host will be restored.

- If the “domain_name” parameter is not specified, the certificate and private key of the specified SSL host will be restored.



Note: The command cannot be synchronized between devices via the synchronization function.

SSL Protocol Version and Cipher Suite

ssl globals protocol virtual <version>

This command is used to set the global default SSL protocol(s) for all SSL virtual hosts. SSL protocol versions allowed to be set include SSLv3, TLSv1.0, TLSv1.1, TLSv1.2 and TLSv1.3. When this command is not configured, the global default SSL protocol setting for all SSL virtual hosts is TLSv1.2. For the appliance with FIPS HSM installed, the global default SSL protocol setting for all SSL virtual hosts is TLSv1.0.

When an SSL virtual host is newly created, the system will automatically configure the SSL protocol setting for this SSL virtual host, which is the same as the global default SSL protocol(s) for all SSL virtual hosts. You can modify the SSL protocol setting of the SSL virtual host by using the “**ssl settings protocol**” command.

version This parameter specifies the default SSL protocol versions supported by all SSL virtual hosts.

Its value must be:

- SSLv3: indicates that the SSLv3 protocol is supported.
- TLSv1: indicates that the TLSv1.0 protocol is supported.
- TLSv11: indicates that the TLSv1.1 protocol is supported.
- TLSv12: indicates that the TLSv1.2 protocol is supported.
- TLSv13: indicates that the TLSv1.3 protocol is supported.
- ALL: indicates that all the protocols above are supported.

To support multiple protocols, separate them with colons (:).

no ssl globals protocol virtual

This command is used to reset the global default SSL protocol setting for all SSL virtual hosts to the default value.

ssl globals protocol real <version>

This command is used to set the global default SSL protocol(s) for all SSL real hosts. SSL protocol versions allowed to be set include SSLv3, TLSv1.0, TLSv1.1, TLSv1.2 and TLSv1.3. When this command is not configured, the global default SSL protocol setting for all SSL real hosts is TLSv1.0, TLSv1.1 and TLSv1.2. For the appliance with FIPS HSM installed, the global default SSL protocol setting for all SSL real hosts is TLSv1.0.

When an SSL real host is newly created, the system will automatically configure the SSL protocol setting for this SSL real host, which is the same as the global default SSL protocol(s) for all SSL real hosts. You can modify the SSL protocol setting of the SSL real host by using the “**ssl settings protocol**” command.

version	This parameter specifies default SSL protocol versions supported by all SSL real hosts. Its value must be: • SSLv3: indicates that the SSLv3 protocol is supported. • TLSv1: indicates that the TLSv1.0 protocol is supported. • TLSv11: indicates that the TLSv1.1 protocol is supported. • TLSv12: indicates that the TLSv1.2 protocol is supported. • TLSv13: indicates that the TLSv1.3 protocol is supported. • ALL: indicates that all the protocols above are supported. To support multiple protocols, separate them with colons (:).
---------	---

no ssl globals protocol real

This command is used to reset the global default SSL protocol setting for all SSL real hosts to the default value.

ssl settings protocol <host_name> <version>

This command is used to set the SSL protocol(s) for the specified SSL virtual host or real host. SSL protocols allowed to be set include SSLv3, TLSv1.0, TLSv1.1, TLSv1.2 and TLSv1.3. When this command is not configured, all SSL virtual hosts and real hosts use their global default SSL protocol version respectively.

For the appliance with FIPS HSM installed, SSL virtual and real hosts support SSLv3 and TLSv1.0 protocols.

host_name	This parameter specifies the SSL virtual host or real host name.
version	This parameter specifies the SSL protocol versions supported by the SSL host. Its value must be: • SSLv3: indicates that the SSLv3 protocol is supported. • TLSv1: indicates that the TLSv1.0 protocol is supported. • TLSv11: indicates that the TLSv1.1 protocol is supported. • TLSv12: indicates that the TLSv1.2 protocol is supported. • TLSv13: indicates that the TLSv1.3 protocol is supported. • ALL: indicates that all the protocols above are supported. To support multiple protocols, separate them with colons (:).

For example:

```
AN(config)#ssl settings protocol rhost1 SSLv3
AN(config)#ssl settings protocol vhost1 SSLv3:TLSv12
AN(config)#ssl settings protocol vhost2 ALL
```

ssl settings fallbackscsv <virtual_host_name>

This command is used to enable the TLS Fallback Signaling Cipher Suite Value (SCSV) function to prevent protocol degrade attacks.

When the function is enabled, if the ciphersuite used by a client carries the TLS_FALLBACK_SCSV option, the appliance will check whether the SSL protocol version used by the client is lower than the highest SSL protocol version supported by the virtual host. If yes, the appliance will return an 86 error and close the connection.

When the function is disabled, as long as the SSL protocol version used by the client is supported by the virtual host, the appliance will continue the SSL handshake.

virtual_host_name	This parameter specifies the SSL virtual host name.
-------------------	---

no ssl settings fallbackscsv <virtual_host_name>

This command is used to disable the TLS Fallback SCSV function for the specified SSL virtual host.

ssl settings ciphersuite <host_name> <cipher_string>

This command is used to set the cipher suite(s) for the specified SSL virtual host or real host.

host_name This parameter specifies the SSL virtual host or real host name.

cipher_string This parameter specifies cipher suites supported by the SSL host. Its value is case-insensitive. To support two or more cipher suites, separate them with colons (:).

For the appliance with FIPS HSM installed, SSL virtual hosts and real hosts support the cipher suites “DES-CBC3-SHA:AES128-SHA:AES256-SHA:!SSLv2”.

For the appliance without FIPS HSM installed, supported cipher suites are listed in the following tables. In the table, “Y” indicates that the cipher suite is supported; “N” indicates that the cipher suite is not supported.

- Cipher suites supported by virtual hosts with different protocol versions

Cipher Suites	Bits	SSL Protocol – Virtual Host				
		SSLv3.0	TLSv1	TLSv1.1	TLSv1.2	TLSv1.3
RC4-MD5	128	Y	Y	Y	Y	N
RC4-SHA	128	Y	Y	Y	Y	N
DES-CBC-SHA	64	Y	Y	Y	N	N
DES-CBC3-SHA	192	Y	Y	Y	Y	N
AES128-SHA	128	Y	Y	Y	Y	N
AES256-SHA	256	Y	Y	Y	Y	N
AES128-SHA256	128	N	N	N	Y	N
AES256-SHA256	256	N	N	N	Y	N
AES128-GCM-SHA256	128	N	N	N	Y	N
AES256-GCM-SHA384	256	N	N	N	Y	N
ECDHE-RSA-AES128-SHA	128	Y	Y	Y	Y	N
ECDHE-RSA-AES256-SHA	256	Y	Y	Y	Y	N

Cipher Suites	Bits	SSL Protocol – Virtual Host				
		SSLv3.0	TLSv1.0	TLSv1.1	TLSv1.2	TLSv1.3
ECDHE-ECDSA-AES128-SHA	128	Y	Y	Y	Y	N
ECDHE-ECDSA-AES256-SHA	256	Y	Y	Y	Y	N
ECDHE-RSA-AES128-SHA256	128	N	N	N	Y	N
ECDHE-RSA-AES256-SHA384	256	N	N	N	Y	N
ECDHE-RSA-AES128-GCM-SHA256	128	N	N	N	Y	N
ECDHE-RSA-AES256-GCM-SHA384	256	N	N	N	Y	N
ECDHE-ECDSA-AES128-SHA256	128	N	N	N	Y	N
ECDHE-ECDSA-AES256-SHA384	256	N	N	N	Y	N
ECDHE-ECDSA-AES128-GCM-SHA256	128	N	N	N	Y	N
ECDHE-ECDSA-AES256-GCM-SHA384	256	N	N	N	Y	N
TLS-AES128-GCM-SHA256	128	N	N	N	N	Y
TLS-AES256-GCM-SHA384	256	N	N	N	N	Y

- Cipher suites supported by real hosts with different protocol versions

Cipher Suite	Bits	SSL Protocol – Real Host				
		SSLv3.0	TLSv1.0	TLSv1.1	TLSv1.2	TLSv1.3
RC4-MD5	128	Y	Y	Y	Y	N
RC4-SHA	128	Y	Y	Y	Y	N
DES-CBC3-SHA	192	Y	Y	Y	Y	N
AES128-SHA	128	Y	Y	Y	Y	N
AES256-SHA	256	Y	Y	Y	Y	N
AES128-SHA256	128	N	N	N	Y	N
AES256-SHA256	256	N	N	N	Y	N
AES128-GCM-SHA256	128	N	N	N	Y	N

Cipher Suite	Bits	SSL Protocol – Real Host				
AES256-GCM-SHA384	256	N	N	N	Y	N
ECDHE-RSA-AES128-SHA	128	Y	Y	Y	Y	N
ECDHE-RSA-AES256-SHA	256	Y	Y	Y	Y	N
ECDHE-ECDSA-AES128-SHA	128	Y	Y	Y	Y	N
ECDHE-ECDSA-AES256-SHA	256	Y	Y	Y	Y	N
ECDHE-RSA-AES128-SHA256	128	N	N	N	Y	N
ECDHE-RSA-AES256-SHA384	256	N	N	N	Y	N
ECDHE-RSA-AES128-GCM-SHA256	128	N	N	N	Y	N
ECDHE-RSA-AES256-GCM-SHA384	256	N	N	N	Y	N
ECDHE-ECDSA-AES128-SHA256	128	N	N	N	Y	N
ECDHE-ECDSA-AES256-SHA384	256s	N	N	N	Y	N
ECDHE-ECDSA-AES128-GCM-SHA256	128	N	N	N	Y	N
ECDHE-ECDSA-AES256-GCM-SHA384	256	N	N	N	Y	N
TLS-AES128-GCM-SHA256	128	N	N	N	N	Y
TLS-AES256-GCM-SHA384	256	N	N	N	N	Y

For the table information above, administrators should notice the following:

- “ECDHE-ECDSA...” cipher suites apply only to ECC certificates, and “ECDHE-RSA...” cipher suites apply only to RSA certificates.
- When the “cipher_string” parameter is set to “EXP-RC4-MD5” or “EXP-DES-CBC-SHA”, the system supports only keys with length less than or equal to 512 bits.
- When the HTTP protocol version is HTTP/2, the set cipher suites must contain one or more of the following ones. Otherwise, the real service will not send the ALPN extension to negotiate the use of HTTP/2 protocol during the TLS handshake with the client. However, the virtual service will still receive HTTP/2 requests.

- ECDHE-RSA-AES128-GCM-SHA256
- ECDHE-RSA-AES256-GCM-SHA384
- ECDHE-ECDSA-AES128-GCM-SHA256
- ECDHE-ECDSA-AES256-GCM-SHA384.



Note: It is recommended that only experienced administrators should use this command. For any questions regarding these settings, please contact Customer Support.

Signature Algorithm and Elliptic Curve

ssl settings signalgo <host_name> <signature_algorithm>

This command is used to set the signature algorithm supported by the Server Key Exchange message for the specified SSL host. This configuration applies to only SSL protocol versions earlier than TLSv1.3.

If this command is not configured, all SSL hosts will use the default setting “sha256ECDSA:sha256RSA:sha384ECDSA:sha384RSA:sha512ECDSA:sha512RSA:sha224ECDSA:sha224RSA:sha1ECDSA:sha1RSA”, which is ranked in the descending order of priority.

host_name	This parameter specifies the SSL virtual host or real host name.
signature_algorithm	This parameter specifies the signature algorithm supported by the Server Key Exchange message. Its value must be sha256ECDSA, sha256RSA, sha384ECDSA, sha384RSA, sha512ECDSA, sha512RSA, sha224ECDSA, sha224RSA, sha1ECDSA or sha1RSA. To set two or more signature algorithms, separate them with colons (:).

ssl settings clientcert signalgo <virtual_host_name> <signature_algorithm>

This command is used to set the signature algorithm supported for client certificate requests for the specified SSL virtual host. This configuration applies to only SSL protocol versions earlier than TLSv1.3. If this command is not configured, all SSL hosts will use the default setting “sha256ECDSA:sha256RSA:sha384ECDSA:sha384RSA:sha512ECDSA:sha512RSA:sha224ECDSA:sha224RSA:sha1ECDSA:sha1RSA”, which is ranked in the descending order of priority.

host_name	This parameter specifies the SSL virtual host name.
signature_algorithm	This parameter specifies the signature algorithm supported by the SSL virtual host. Its value must be sha256ECDSA, sha256RSA, sha384ECDSA, sha384RSA, sha512ECDSA, sha512RSA, sha224ECDSA, sha224RSA, sha1ECDSA or

sha1RSA. To set two or more signature algorithms, separate them with colons (:).



Note :

- When a virtual host is running SSLv3, TLSv1 or TLSv1.1, the signature algorithm setting for the virtual host must contain sha1RSA or sha1ECDSA; otherwise, the SSL handshake will fail.
- If “sha512RSA” is set as the first choice of the signature algorithm, the client that uses a 512-bit RSA key will reset the connection as a result of the failure to complete the client authentication.

ssl settings curves <host_name> <elliptic_curve>

This command is used to set the elliptic curve supported by the Server Key Exchange message for the specified SSL host.

If this command is not configured, all SSL hosts will use the default setting “secp256r1:secp384r1:secp521r1”, which is ranked in the descending order of priority.

host_name	This parameter specifies the SSL virtual host or real host name.
elliptic_curve	This parameter specifies the name of the elliptic curve supported by the Server Key Exchange message. Its value must be secp256r1, secp384r1 or secp521r1. To set two or more elliptic curves, separate them with colons (:).

TLSv1.3 Feature Options

ssl settings tlsv13 signalgo certrequest <virtual_host_name> <signature_algorithm>

This command is used to set TLSv1.3 signature algorithms supported for certificate requests for the specified SSL virtual host. When this command is not configured, the default supported signature algorithms are ecdsa:rsae:rsapss, which correspond to ECDSA, RSAE and RSAPSS respectively.

- ECDSA: includes ecdsa_secp256r1_sha256, ecdsa_secp384r1_sha384 and ecdsa_secp521r1_sha512.
- RSAE: include rsa_pss_rsae_sha256, rsa_pss_rsae_sha384 and rsa_pss_rsae_sha512.
- RSAPSS: includes rsa_pss_pss_sha256, rsa_pss_pss_sha384 and rsa_pss_pss_sha512.

virtual_host_name	This parameter specifies the SSL virtual host name.
signature_algorithm	This parameter specifies the signature algorithms. Its value must be “ecdsa”, “rsae” or “rsapss”. To set two or more

signature algorithms, separate them with colons (:).

ssl settings tlsv13 signalgo certverify <host_name> <signature_algorithm>

This command is used to set TLSv1.3 signature algorithms supported for certificate verification for the specified SSL host. When this command is not configured, the default supported signature algorithms are `ecdsa:rsae:rsapss`, which correspond to ECDSA, RSAE and RSAPSS respectively.

- ECDSA: includes `ecdsa_secp256r1_sha256`, `ecdsa_secp384r1_sha384` and `ecdsa_secp521r1_sha512`.
- RSAE: include `rsa_pss_rsae_sha256`, `rsa_pss_rsae_sha384` and `rsa_pss_rsae_sha512`.
- RSAPSS: includes `rsa_pss_pss_sha256`, `rsa_pss_pss_sha384` and `rsa_pss_pss_sha512`.

host_name This parameter specifies the SSL virtual host or real host name.

signature_algorithm This parameter specifies the signature algorithms. Its value must be “ecdsa”, “rsae” or “rsapss”. To set two or more signature algorithms, separate them with colons (:).



Note: Real hosts do not support the negotiation of RSAPSS signature algorithms with TLSv1.2 SSL servers.

ssl settings tlsv13 earlydata <host_name>

This command is used to enable the TLSv1.3 zero round-trip time (0-RTT) mode for the specified SSL host. By default, the 0-RTT mode is enabled.

host_name This parameter specifies the SSL virtual host or real host name.

no ssl settings tlsv13 earlydata <host_name>

This command is used to disable the TLSv1.3 0-RTT mode for the specified SSL host.

ssl settings tlsv13 earlydatasize <virtual_host_name> <size>

This command is used to set the maximum early data size supported by the specified SSL virtual host. When this command is not configured, the maximum early data size is 1024 bytes by default.

virtual_host_name This parameter specifies the SSL virtual host name.

size This parameter specifies maximum early data size. Its value must be an integer ranging from 1 to 16384.

ssl settings tlsv13 ticketlifetime <virtual_host_name> <lifetime>

This command is used to set the lifetime of tickets in the NewSessionTicket messages for the specified virtual host. When this command is not configured, the default ticket lifetime is 7200s.

virtual_host_name	This parameter specifies the SSL virtual host name.
lifetime	This parameter specifies the ticket lifetime. Its value must be an integer ranging from 60 to 604800, in seconds.

ssl settings tlsv13 psk mode <real_host_name> <ke_mode>

This command is used to set the TLSv1.3 PSK exchange mode for the specified SSL real host. The supported PSK exchange modes include “psk_dhe_ke” and “psk_ke:psk_dhe_ke”. The default PSK exchange mode is “psk_ke:psk_dhe_ke”.

real_host_name	This parameter specifies the SSL real host name.
ke_mode	This parameter specifies the TLSv1.3 PSK exchange mode. Its value must be “psk_dhe_ke” or “psk_ke:psk_dhe_ke”.

Client Authentication

ssl settings clientauth <host_name>

This command is used to enable the client authentication function for the specified SSL real or virtual host. This function should be enabled for every SSL real or virtual host individually. By default, this function is disabled for every SSL real or virtual host.

- After this function is enabled for an SSL virtual host, the SSL virtual host will send the certificate request to the client during the SSL handshake and validate the client certificate. To communicate with this SSL virtual host, all SSL clients must provide the client certificate.
- After this function is enabled for an SSL real host, the SSL real host will send its certificate to the SSL backend server when receiving the certificate request from the SSL backend server.

host_name	This parameter specifies the SSL virtual or real host name.
-----------	---

**Note:**

- When the client authentication function is enabled for the SSL virtual host, the APV appliance can forward the fields of the received client certificate to the backend server through the HTTP header or HTTP cookie. For configuration methods, please refer to the section HTTPS Forwarding Control in Chapter 10.
- For a virtual host that uses the TLSv1.2 protocol and RSA cipher suite, if

the client authentication function is enabled, SSL connections will fail when the Certificate Verify message from a client contains SSH224 as the hash algorithm.

- When the cipher suite ECDHE-SM4-SM3 and ECDHE-SM4-GCM-SM3 is configured for a virtual host, even if client authentication is disabled for the SSL virtual host, two-way SSL will still be performed by the SSL virtual host during an SSL handshake.

no ssl settings clientauth <host_name>

This command is used to disable the client authentication function for the specified SSL real or virtual host.

ssl settings certfilter <virtual_host_name> <condition_1> [condition_2]

This command is used to configure a certificate filter for the client authentication function of the specified SSL virtual host.

Certificate filter configurations are optional for the SSL virtual host. One SSL virtual host supports a maximum of 1024 certificate filters, among which the relationship is “OR”. If any certificate filters have been configured and the client authentication is enabled for an SSL virtual host, the client certificate must match at least one certificate filter; otherwise, the SSL virtual host will reject the client access.

A certificate filter consists of at least one matching condition and at most two matching conditions. The matching condition is a filter string configured based on the “Subject” or “Issuer” field of the client certificate. Even if only a part of the field matches the filter string, the filter is considered being hit. If a certificate filter consists of two matching conditions, the client certificate can match this certificate filter only when it meets both conditions.

virtual_host_name	This parameter specifies the SSL virtual host name.
condition_1	This parameter defines a filter string based on the “Subject” or “Issuer” field of the client certificate. Its value must be a case-sensitive string of 1 to 63 characters enclosed in double quotes. Its value is in the format of “subject:/C=US” or “issuer:/C=US”. The filter string can contain multiple filter keywords which are separated by “/”, such as “subject:/C=US/OU=example”. The relationship among filter keywords is “AND”. The filter string can also contain Chinese characters encoded into the Unicode format. For a Unicode string, it must be in lower case and prefixed with “%u” to make a difference with normal English characters, such as “subject:/CN=%u4f60%u597d”.
condition_2	Optional. This parameter defines the second filter string based on the “Subject” or “Issuer” field of the client certificate. The configuration method of this parameter is

the same as that of the “condition_1” parameter.

The APV appliance supports configuring the filter keywords based on all the Relative Distinguished Names (RDNs) and corresponding OIDs of the “Subject” or “Issuer” field, as shown in the following table:

RDN	Standard Name	OID
C	Country Name	2.5.4.6
ST	State or Province Name	2.5.4.8
L	Locality Name	2.5.4.7
O	Organization Name	2.5.4.10
OU	Organizational Unit Name	2.5.4.11
CN	Common Name	2.5.4.3
serialNumber	Serial Number	2.5.4.5
dnQualifier	DN Qualifier	2.5.4.46
pseudonym	Pseudonym	2.5.4.65
title	Title	2.5.4.12
generationQualifier	Generation Qualifier	2.5.4.44
initials	Initials	2.5.4.43
name	Name	2.5.4.41
givenName	Given Name	2.5.4.42
surname	Surname	2.5.4.4
UID	User ID	0.9.2342.19200300.100.1.1
DC	Domain Component	0.9.2342.19200300.100.1.25
emailAddress	Email Address	1.2.840.113549.1.9.1
{OID expression}	OID information, for example: 1.2.3.4	

For example:

```
AN(config)#ssl settings certfilter vhost
"subject:/C=US/O=Array/OU=QA/emailAddress=admin@mailserver.com" "issuer:/C=US"
```

In this example, client certificates can pass the certificate validation only when the following conditions are both met

- In the “Subject” field, “C” is “US”, “O” is “Array”, “OU” is “QA” and “emailAddress” is “admin@mailserver.com”.
- In the “Issuer” field, “C” is “US”.

```
AN(config)#ssl settings certfilter vhost "subject:/2.5.4.6=JP"
```

In this example, “subject:/2.5.4.6” stands for “C (country)” in the “Subject” field. After this command is executed, only the client certificates where the value of the OID “2.5.4.6” is “JP” in the “Subject” field can pass the certificate validation.



Note: Before executing the “ssl settings clientauth” and “ssl settings certfilter” commands, make sure that the status of the SSL host involved is inactive. Otherwise, the commands will fail to be executed. If the status of the SSL host is active, you can execute the “ssl stop <host_name>” command to disable the SSL host.

no ssl settings certfilter <virtual_host_name> <condition_1> [condition_2]

This command is used to delete a certificate filter configured for the client authentication function of the specified SSL virtual host.

ssl settings ocsf <virtual_host_name> <ocsp_server_url>

This command is used to configure an Online Certificate Status Protocol (OCSP) server for the specified SSL virtual host. When any OCSP server is configured for an SSL virtual host, the OCSP certificate revocation check function is enabled for the SSL virtual host. The OCSP configurations are optional for the SSL virtual host. When this command is configured and the client authentication is enabled for an SSL virtual host, the SSL virtual host will act as the OCSP client and check whether the client certificate is revoked by communicating with the OCSP server.

When receiving the client certificate from the client, the SSL virtual host will first communicate with the OCSP server specified in the client certificate to check whether the client certificate has been revoked. If this OCSP server does not return the “Good” or “Revoked” response, the SSL virtual host will then communicate with the OCSP server configured using this command to check whether the client certificate has been revoked.

virtual_host_name	This parameter specifies the SSL virtual host name.
ocsp_server_url	This parameter specifies the URL of the OCSP server. Its value must start with “http://” or “https://”. The maximum length of its value is restricted by the maximum length of the command line supported by CLI. Currently, CLI supports the command line with the maximum length of 1024 characters.



Note:

1. When both the OCSP certificate revocation check and Certificate Revocation list (CRL) online/offline certificate revocation check functions mentioned below are enabled for the SSL virtual host, only the OCSP certificate revocation check function will take effect.
2. To enable the OCSP function to take effect, administrators must import the trusted CA certificate of clients’ certificates for the virtual host.

no ssl settings ocsf <virtual_host_name>

This command is used to disable the OCSP certificate revocation check function for the specified SSL virtual host.

ssl settings ocsfstapling <virtual_host_name>

This command is used to enable the OCSP Stapling function for the specified SSL virtual host. By default, this function is disabled for all SSL virtual hosts.

`virtual_host_name` This parameter specifies the name of an existing SSL virtual host.

**Note:**

1. The OCSP Stapling function of SSL virtual hosts does not support the Multiple Certificate Status Request extension defined by RFC 6961.
2. If an SSL virtual host uses a certificate chain, it must have the intermediate CA certificates imported to support OCSP Stapling.
3. If the certificate of an SSL virtual host does not include the OCSP extension, it cannot support OCSP Stapling.
4. The OCSP Stapling function of SSL virtual hosts does not support IPv6.
5. The system supports OCSP based on the HTTP protocol only.
6. For OCSP responses carrying “nextupdate”, the system will send OCSP requests to the OCSP responder based on the time specified by “nextupdate”. For OCSP responses that do not carry “nextupdate”, the system will not send OCSP requests to the OCSP responder.

no ssl settings ocspestapling <virtual_host_name>

This command is used to disable the OCSP Stapling function for the specified SSL virtual host.

show ssl status ocspestapling <virtual_host_name> [domain_name]

This command is used to display the enabling/disabling status of the OCSP Stapling function and related status information for the specified domain name associated with the specified SSL virtual host.

`virtual_host_name` This parameter specifies the name of an existing SSL virtual host.

`domain_name` Optional. This parameter specifies the domain name associated with the virtual host.

- If the “domain_name” parameter is specified, the enabling/disabling status of the OCSP Stapling function and related status information for the specified domain name associated with the specified SSL virtual host will be displayed.
- If the “domain_name” parameter is not specified, the enabling/disabling status of the OCSP Stapling function and related status information for the specified SSL virtual host will be displayed.

ssl import clientkey [virtual_host_name] [clientkey_url]

This command is used to import the SSL client private key for the specified SSL virtual host to communicate with other servers, such as the OCSP or Lightweight Directory Access Protocol (LDAP) server securely using SSL.

virtual_host_name	Optional. This parameter specifies the SSL virtual host name. Its value must be: • SSL virtual host name: indicates that the SSL client private key is imported for the specified SSL virtual host. • ALL: indicates that the SSL client private key is imported for all the SSL virtual hosts. The default value is “ALL”.
clientkey_url	Optional. This parameter specifies the URL of the SSL client private key on the FTP, SFTP, TFTP, HTTP, HTTPS, or SCP server. The maximum length of its value is restricted by the maximum length of the command line supported by CLI. Currently, CLI supports the command line with the maximum length of 1024 characters. If this parameter is not specified, the administrator needs to paste the SSL client private key to the CLI manually. The default value is empty.

no ssl clientkey <virtual_host_name>

This command is used to delete the SSL client private key of the specified SSL virtual host.

virtual_host_name	This parameter specifies the SSL virtual host name. Its value must be: • SSL virtual host name: indicates that the SSL client private key imported for the specified SSL virtual host will be deleted. • ALL: indicates that the SSL client private key imported for all the SSL virtual hosts will be deleted.
-------------------	--

ssl import clientcert [virtual_host_name] [clientcert_url]

This command is used to import the SSL client certificate for the specified SSL virtual host to communicate with other servers, such as the OCSP or LDAP server.

virtual_host_name	Optional. This parameter specifies the SSL virtual host name. Its value must be: • SSL virtual host name: indicates that the SSL client certificate is imported for the specified SSL virtual host. • ALL: indicates that the SSL client certificate is imported for all the SSL virtual hosts.
-------------------	--

The default value is “ALL”.

clientcert_url Optional. This parameter specifies the URL of the SSL client certificate on the FTP, SFTP, TFTP, HTTP, HTTPS, or SCP server. The maximum length of its value is restricted by the maximum length of the command line supported by CLI. Currently, CLI supports the command line with the maximum length of 1024 characters. If this parameter is not specified, the administrator needs to paste the SSL client certificate to the CLI.

The default value is empty.

no ssl clientcert <virtual_host_name>

This command is used to delete the SSL client certificate of the specified SSL virtual host.

virtual_host_name This parameter specifies the SSL virtual host name. Its value must be:

- SSL virtual host name: indicates that the SSL client certificate imported for the specified SSL virtual host will be deleted.
- ALL: indicates that the SSL client certificate imported for all the SSL virtual hosts will be deleted.

show ssl clientcert <virtual_host_name> [display_mode]

This command is used to display the SSL client certificate of the specified SSL virtual host.

virtual_host_name This parameter specifies the SSL virtual host name.

display_mode Optional. This parameter specifies the display mode of the SSL client certificate. Its value must be:

- complete: indicates that the system will display the complete content of the SSL client certificate.
- simple: indicates that the system will display only the “Issuer”, “Validity” and “Subject” fields of the SSL client certificate.

The default value is “complete”.

ssl settings crl online <virtual_host_name>

This command is used to enable the CRL online certificate revocation check function for the specified SSL virtual host. When this function is enabled, the SSL virtual host

will online download the CRL file from the CRL Distribution Point (CDP) specified in the received client certificate to check whether the client certificate is revoked. This command can be configured only after the client authentication function is enabled for the SSL virtual host.

`virtual_host_name` This parameter specifies the SSL virtual host name.

no ssl settings crl online *<virtual_host_name>*

This command is used to disable the CRL online certificate revocation check function for the specified SSL virtual host.

ssl import crlca *[host_name] [domain_name] [tftp_ip|url] [file_name]*

This command is used to import a CRL CA certificate for the specified SSL virtual host or real host or into the global CRL CA certificate list.

When receiving the client certificate from the client, the SSL virtual host can use the downloaded CRL file regularly issued by the CA to check whether the client certificate is revoked. When receiving the server certificate from the SSL server, the SSL real host can use the downloaded CRL file regularly issued by the CA to check whether the server certificate is revoked. The CRL CA certificate is used to validate the downloaded CRL files for the CRL offline certificate revocation check function.

When using the copy-n-paste way to import the CRL CA certificate, you only need to execute the “**ssl import crlca**” command and copy and paste the content of the CRL CA certificate to the CLI. Only the CRL CA certificate in PEM can be imported in the copy-n-paste way. This command can also be used to import the CRL CA certificate in the PEM and DER formats through TFTP, FTP, SFTP, HTTP, HTTPS, and SCP.

`host_name` Optional. This parameter specifies the SSL virtual host name or real host name. Its value must be:

- SSL virtual host name or real host name: indicates that the CRL CA certificate is imported for the specified SSL virtual host or real host to validate the CRL files downloaded for this SSL host.
- ALL: indicates that the CRL CA certificate is imported into the global CRL CA certificate list to validate the global CRL files.

The default value is “ALL”.

`domain_name` Optional. This parameter specifies the domain name.

- If the “`domain_name`” parameter is specified, the CRL CA certificate will be imported for the specified domain name associated with the specified SSL virtual host.
- If the “`domain_name`” parameter is not specified, the

CRL CA certificate will be imported for the specified SSL virtual host.

The default value is empty.

Note: If you need to import the CRL CA certificate for an SSL virtual or real host via URL (the value of the parameter “tftp_ip|url” is URL), this parameter’s value should be an empty string.

tftp_ip|url

Optional. This parameter specifies the IP address of the CRL CA certificate on the TFTP server, or the URL the CRL CA certificate.

- “tftp_ip” is used to configure the IP address of a TFTP server. Its value must be an IPv4 address. This parameter needs to be set only when you use TFTP to import the CRL CA certificate.
- “url” is used to configure the URL of the CRL CA certificate on the FTP, SFTP, TFTP, HTTP, HTTPS, or SCP server. Its value needs to be enclosed in double quotes. This parameter needs to be set only when you use URL to import the CRL CA certificate.

The default value is empty.

file_name

Optional. This parameter specifies the file name of the CRL CA certificate on the TFTP server. The maximum length of its value is restricted by the maximum length of the command line supported by CLI. Currently, CLI supports the command line with the maximum length of 1024 characters.

- When the CRL CA certificate is imported for the specified SSL host, the default value is “<host_name>.cert”.
- When the CRL CA certificate is imported into the global CRL CA certificate list, the default value is “gcrca.cert”.

This parameter needs to be set only when you use TFTP to import the CRL CA certificate, that is, the value of the parameter “tftp_ip|url” is the IP address of the TFTP server.



Note: The downloaded CRL file can be used to check whether the client or server certificate is revoked only when it is issued by the CA same as that of the CRL CA certificate and the client or server certificate; otherwise, this CRL file will be ignored during client or server authentication.

no ssl crlca <host_name> [certificate_number]

This command is used to delete a CRL CA certificate from the specified SSL virtual host or real host or the global CRL CA certificate list.

host_name	This parameter specifies the SSL virtual host name or real host name. Its value must be: • SSL virtual host name or real host name: indicates that the CRL CA certificate of the specified SSL virtual host or real host will be deleted. • ALL: indicates that the CRL CA certificate in the global CRL CA certificate list will be deleted.
certificate_number	Optional. This parameter specifies the number of the CRL CA certificate to be deleted. You can view the numbers of the CRL CA certificates by executing the “show ssl crlca” command. If this parameter is not specified, the system will delete all CRL CA certificates.

show ssl crlca [host_name] [display_mode] [domain_name]

This command is used to display all CRL CA certificates of the specified SSL virtual host or real host or the global CRL CA certificate list.

host_name	Optional. This parameter specifies the SSL virtual host name or real host name. Its value must be: • SSL virtual host name or real host name: indicates that all CRL CA certificates of the specified SSL virtual host or real host will be displayed. • ALL: indicates that all CRL CA certificates in the global CRL CA certificate list will be displayed.
-----------	---

The default value is “ALL”.

display_mode	Optional. This parameter specifies the display mode of the CRL CA certificate. Its value must be: • complete: indicates that the system will display the complete content of the CRL CA certificate. • simple: indicates that the system will display only the “Issuer”, “Validity” and “Subject” fields of the CRL CA certificate.
--------------	---

The default value is “complete”.

domain_name	Optional. This parameter specifies the domain name. • If the “domain_name” parameter is specified,, the CRL CA certificate of the specified domain name associated
-------------	---

with the specified SSL virtual host will be displayed.

- If the “domain_name” parameter is not specified, the CRL CA certificate of the specified SSL virtual host will be displayed.

no ssl sni crlca <host_name>[domain_name][certificate_index]

This command is used to delete a CRL CA certificate of the specified domain name for the specified SSL virtual host or real host.

host_name	This parameter specifies the SSL virtual host name or real host name.
domain_name	Optional. This parameter specifies the domain name. If the “domain_name” parameter is not specified, all the CRL CA certificates of each domain name for the specified SSL virtual host will be deleted.
certificate_index	Optional. This parameter specifies the index of the CRL CA certificate to be deleted. You can view the index of the CRL CA certificate by executing the “ show ssl crlca ” command.

ssl settings crl offline <host_name> <cdp_name> <cdp_url> [domain_name] [time_interval] [delay_time] [tls_flag]

This command is used to configure a CDP and the CRL downloading rule for the specified SSL virtual host or real host. When any CDP is configured for an SSL host, the CRL offline certificate revocation check function is enabled for the SSL host.

The SSL virtual host will download the CRL file from the CDP at the interval specified by the CRL downloading rule, and use the latest downloaded CRL file to check whether the client certificate is revoked. The SSL real host will use the downloaded CRL file to check the received server certificate.

The system supports downloading the CRL file using the HTTP, FTP and LDAP protocols.

For every SSL host, a maximum of 10 CDPs can be configured.

For an SSL virtual host, this command can be configured only after the client authentication function is enabled for it.

host_name	This parameter specifies the SSL virtual host name or real host name.
cdp_name	This parameter specifies the name of the CDP. Its value must be a string of 1 to 31 characters. Numbers, letters and underscores are supported. Its value must not be “all” or

	“ALL”.
<code>cdp_url</code>	This parameter specifies the HTTP, FTP or LDAP URL from which the CRL file is downloaded. Its value must be a string of 1 to 1022 characters.
<code>domain_name</code>	Optional. This parameter specifies the domain name. - If the “domain_name” parameter is specified, the CRL of the specified domain name associated with the specified SSL virtual host will be downloaded. - If the “domain_name” parameter is not specified, the CRL of the specified SSL virtual host will be downloaded.
<code>time_interval</code>	Optional. This parameter specifies the interval at which the CRL file is downloaded, in minutes. Its value must be an integer ranging from 1 to 2,147,483,647. The default value is 1440.
<code>delay_time</code>	Optional. This parameter specifies the time that the downloaded CRL file will be delayed to expire, in minutes. Its value must be an integer ranging from 0 to 2,147,483,647. The default value is 0. - When its value is greater than 0, the system will check whether the downloaded CRL file expires. If the current time is later than the next update time (that is the expiration time) contained in the CRL file plus the delay time, the CRL file expires. The system will reject the SSL connections using this CRL file to validate the client certificate; otherwise, the CRL file is valid. - When its value is 0, the system will not check whether the downloaded CRL file expires.
<code>tls_flag</code>	Optional. This parameter specifies whether to access the LDAP server over the TLS protocol. Its value must be: 0: indicates that the TLS protocol will not be used to access the LDAP server. 1: indicates that the TLS protocol will be used to access the LDAP server. The default value is 0.

For example:

```
AN(config)#ssl settings crl offline "ssl_vhost" "cdp1" "http://10.8.6.50/xxx.crl" 1440 0
```

**Note:**

- Before configuring this command, please import the CRL CA certificate for the SSL virtual host using the “**ssl import crlca**” command to validate the downloaded CRL file.
- It is recommended not to configure too many CRL downloading rules with too short time interval (1 minute for example), which can cause failure to establish an SSH connection for the appliance.
- A maximum of 4010 CDP can be configured by executing the commands “**ssl settings crl offline**” and “**ssl globals crl cdp**”.

no ssl settings crl offline <host_name> [cdp_name]

This command is used to delete the CDP and the CRL downloading rule configured for the specified SSL virtual host or real host. When all CDPs of an SSL host are deleted, the CRL offline certificate revocation check function is disabled for this SSL host.

host_name This parameter specifies the SSL virtual host name or real host name.

cdp_name Optional. This parameter specifies the name of the CDP. Its value must be:

- **CDP name:** indicates that the system will delete the specified CDP and the CRL downloading rule configured for the SSL virtual host or real host.
- **ALL:** indicates that the system will delete all CDPs and CRL downloading rules configured for the SSL virtual host or real host.

The default value is “ALL”.

ssl globals crl cdp <cdp_name> <cdp_url> [time_interval] [delay_time] [tls_flag]

This command is used to configure a CDP and the CRL downloading rule for the global. The system will download the CRL file from the CDP at the interval specified by the CRL downloading rule. The system supports downloading the CRL file using the HTTP, FTP and LDAP protocols.

After the global CDP is configured, you can associate the global CDP with the specified SSL virtual host or real host using the “**ssl globals crl host**” command to implement the function same as the “**ssl settings crl offline**” command. A maximum of 10 global CDPs can be configured. One global CDP can be associated with multiple SSL hosts and multiple global CDPs can be associated with only one SSL host.

<code>cdp_name</code>	This parameter specifies the name of the CDP. Its value must be a string of 1 to 31 characters. Numbers, letters and underscores are supported. Its value must not be “all” or “ALL”.
<code>cdp_url</code>	This parameter specifies the HTTP, FTP or LDAP URL from which the CRL file is downloaded. Its value must be a string of 1 to 1023 characters.
<code>time_interval</code>	Optional. This parameter specifies the interval at which the CRL file is downloaded, in minutes. Its value must be an integer ranging from 1 to 2,147,483,647. The default value is 1440.
<code>delay_time</code>	Optional. This parameter specifies the time that the downloaded CRL file will be delayed to expire, in minutes. Its value must be an integer ranging from 0 to 2,147,483,647. The default value is 0. - When its value is greater than 0, the system will check whether the downloaded CRL file expires. If the current time is later than the next update time (that is the expiration time) contained in the CRL file plus the delay time, the CRL file expires. The system will reject the SSL connections using this CRL file to validate the client certificate; otherwise, the CRL file is valid. - When its value is 0, the system will not check whether the downloaded CRL file expires.
<code>tls_flag</code>	Optional. This parameter specifies whether to access the LDAP server over the TLS protocol. Its value must be: 0: indicates that the TLS protocol will not be used to access the LDAP server. 1: indicates that the TLS protocol will be used to access the LDAP server. The default value is 0.

For example:

```
AN(config)#ssl globals crl cdp "cdp1" "http://10.8.6.50/xxx.crl" 1440 0
```



Note:

- Before configuring this command, please import the CRL CA certificate into the global CRL CA certificate list using the “**ssl import crlca**” command to validate the CRL file downloaded using this command.

- A maximum of 4010 CDP can be configured by executing the commands “**ssl settings crl offline**” and “**ssl globals crl cdp**”.

no ssl globals crl cdp [cdp_name]

This command is used to delete the CDP and the CRL downloading rule configured for the global.

cdp_name	Optional. This parameter specifies the name of the global CDP. Its value must be: • CDP name: indicates that the system will delete the specified CDP and the CRL downloading rule configured for the global. • ALL: indicates that the system will delete all CDPs and CRL downloading rules configured for the global. The default value is “ALL”.
----------	---

ssl globals crl host <cdp_name> <host_name>

This command is used to associate a global CDP with the specified SSL virtual host or real host. When a global CDP is associated with an SSL host, the CRL offline certificate revocation check function of this SSL host can use the CRL file downloaded from this global CDP. One global CDP can be associated with multiple SSL hosts and multiple global CDPs can be associated with one SSL host.

cdp_name	This parameter specifies the name of the global CDP.
host_name	This parameter specifies the SSL virtual host name or real host name.

no ssl globals crl host <cdp_name> <host_name>

This command is used to disassociate the specified global CDP from the specified SSL virtual host or real host.

ssl load crl

This command is used to download the CRL file from the CDPs configured using the “**ssl settings crl offline**” and “**ssl globals crl cdp**” commands again immediately.

ssl import crlfilter <cdp_name> <tftp_ip> [file_name]

This command is used to import the CRL filter file for the specified global CDP to filter the CRL files to be downloaded. The filter file must contain simple regular expressions, which support “\d”, “\w” and the formats allowed by the OpenLDAP server.

To download all the CRL files from a global CDP, please make sure that the desired CRL filter file has been imported for the global CDP.

<code>cdp_name</code>	This parameter specifies the name of the global CDP.
<code>tftp_ip</code>	This parameter specifies the IP address of the TFTP server from which the CRL filter file is imported. Its value must be an IPv4 address.
<code>file_name</code>	Optional. This parameter specifies the name of the CRL filter file on the TFTP server. Its value must be a string of 1 to 1023 characters. The default value is “crlfilter.conf”.



Note: This command works only for the global CDPs configured using the “`ssl globals crl cdp`” command.

`ssl globals crl studyfilter <cdp_name> [time_interval]`

This command is used to enable the CRL filter study function for the specified global CDP to download the CRL files more efficiently.

<code>cdp_name</code>	This parameter specifies the name of the global CDP.
<code>time_interval</code>	Optional. This parameter specifies the interval between CRL filter studies, in minutes. Its value must be an integer ranging from 1 to 2,147,483,647. The default value is 10,080.



Note: This command works only for the global CDPs configured using the “`ssl globals crl cdp`” command

`no ssl crlfilter <cdp_name>`

This command is used to delete all imported, parsed and studied CRL filter files of the specified global CDP.

`ssl globals fastcrl {on|off}`

This command is used to enable or disable the CRL memory function. When this function is enabled, the system will immediately load the CRL files downloaded for the CRL offline certificate revocation check function into the memory, which speeds up the check of the certificate revocation status. By default, this function is disabled.

The maximum size of the CRL memory depends on the system memory, as shown in the following table:

System Memory	Maximum Size of CRL Memory
1GB or less than 1GB	10 MB
2 GB	25 MB
4 GB	150 MB

System Memory	Maximum Size of CRL Memory
8 GB	200 MB
16 GB	600 MB
Larger than 16 GB	1200 MB



Note: When the CRL files cache reaches the limit of the allocated memory, the system will automatically disable the CRL memory function.

show ssl crlstatus [*host_name*] [*cdp_name*]

This command is used to display the CRL files downloaded from the specified CDP or all CDPs for the specified SSL virtual host or real host or for the global. Every displayed CRL file entry includes the CDP name, the update time of the CRL file and the status of the CRL file. The CRL file has three statuses: success, failed and downloading.

- host_name** Optional. This parameter specifies the SSL virtual host name or real host name.
- SSL virtual host or real host name: indicates that the system will display the CRL files downloaded for the specified SSL virtual host or real host.
 - ALL: indicates that the system will display the CRL files downloaded for the global.
- The default value is “ALL”.
- cdp_name** Optional. This parameter specifies the name of the CDP.
- CDP: indicates that the system will display the CRL files downloaded from the specified CDP.
 - ALL: indicates that the system will display the CRL files downloaded from all CDPs.
- The default value is “ALL”.

The following is an output example:

```
AN(config)#show ssl crlstatus all cdp1
Host "vhost":
  CRL Distribution Point "cdp1":
    Parse status      : N/A
    Study time        : N/A
    Study status      : N/A
    Download time     : Tue Apr 25 15:40:41 2017
    Download status   : Downloading
    Nextupdate        : N/A
    Delay             : N/A
    Validity          : Valid
```

ssl settings crlmode certcdpfirst <*virtual_host_name*>

This command is used to configure the Client Certificate CDP First mode for the specified SSL virtual host. In this mode:

- If the client certificate carries a CDP and the CDP is the LDAP or LDAPS Full Name or DA (Directory Address) format, the system will first use the address in the CDP specified in the client certificate to look up the CRL memory. If the system fails to find an applicable CRL in this way, the system will check each CRL downloaded from the CDPs associated with the SSL virtual host to verify the certificate status.
- If the client certificate carries a CDP and the CDP is the HTTP or FTP Full Name, the system will check each CRL downloaded from the HTTP and FTP Full Name addresses specified in the CDPs associated with the SSL virtual host to verify the client certificate status.
- If the client certificate does not carry a CDP, the system will check each CRL downloaded from the CDPs associated with the SSL virtual host to verify the certificate status.

Before enabling this mode, administrators should enable the CRL memory function through the “**ssl globals fasterl on**” command, and configure the CDP and the CRL downloading rule for the virtual host using one of the two following methods:

- Execute the “**ssl settings crl offline**” command for the specified virtual host.
- Execute the “**ssl globals crl cdp**” command to configure a global CDP and CRL downloading rule, and then associate the global CDP to the specified virtual host through the “**ssl globals crl host**” command.

`virtual_host_name` This parameter specifies the SSL virtual host name.



Note: Only one of the commands “**ssl settings crlmode certcdpfirst**”, “**ssl settings crlmode certcdponly**” and “**ssl settings crlmode hostedponly**” can be configured for a specified SSL virtual host.

ssl settings crlmode certcdponly *<virtual_host_name> [certificate_status]*

This command is used to configure the Client Certificate CDP Only mode for the specified SSL virtual host. After this mode is enabled, the SSL virtual host will only use the address in the CDP specified in the client certificate to look up the CRL memory. If the virtual host fails to find an applicable CRL in this way or the client certificate does not carry CDP, the virtual host will decide the status of the certificate according to the value of the “`certificate_status`” parameter.

Before enabling this mode, administrators should enable the CRL memory function through the “**ssl globals fasterl on**” command, and configure the CDP and the CRL downloading rule for the virtual host using one of the two following methods:

- Execute the “**ssl settings crl offline**” command for the specified virtual host.

- Execute the “**ssl globals crl cdp**” command to configure a global CDP and CRL downloading rule, and then associate the global CDP to the specified virtual host through the “**ssl globals crl host**” command.

virtual_host_name	This parameter specifies the SSL virtual host name.
certificate_status	Optional. This parameter specifies the validity of the client certificate when the virtual host fails to find an applicable CRL according to the address in the CDP carried in the client certificate. Its value must be: • valid: means to report the client certificate status as valid. • revoked: means to report the client certificate status as revoked. The default value is valid.



Note: Only one of the commands “**ssl settings crlmode certcdpfirst**”, “**ssl settings crlmode certcdponly**” and “**ssl settings crlmode hostcdponly**” can be configured for a specified SSL virtual host.

ssl settings crlmode hostcdponly <virtual_host_name>

This command is used to configure the SSL Host CDP Only mode for the specified SSL virtual host. After this mode is enabled, the SSL virtual host will check each CRL downloaded from its associated CDPs to verify the certificate status.

Before enabling this mode, administrators should enable the CRL memory function through the “**ssl globals fasterl on**” command, and configure the CDP and the CRL downloading rule for the virtual host using one of the two following methods:

- Execute the “**ssl settings crl offline**” command for the specified virtual host.
- Execute the “**ssl globals crl cdp**” command to configure a global CDP and CRL downloading rule, and then associate the global CDP to the specified virtual host through the “**ssl globals crl host**” command.

virtual_host_name	This parameter specifies the SSL virtual host name.
-------------------	---



Note: Only one of the commands “**ssl settings crlmode certcdpfirst**”, “**ssl settings crlmode certcdponly**” and “**ssl settings crlmode hostcdponly**” can be configured for a specified SSL virtual host.

Advanced Options for Client Authentication

ssl settings authmandatory <virtual_host_name>

This command is used to enable the mandatory mode of client authentication for the specified SSL virtual host.

The system supports two client authentication modes: mandatory and non-mandatory. When the SSL server sends a certificate request to the client, if the client clicks “Cancel” to reject sending the client certificate to the server or the client certificate fails the authentication:

- In mandatory mode, the SSL server will discard this SSL connection and reject all requests of the client.
- In non-mandatory mode, the SSL server will permit the client to access limited network resources (which are configured using the “**http acl url**” command. Note that the access level should be “level 1”).

By default, the mandatory mode of client authentication mode is enabled.

`virtual_host_name` This parameter specifies the SSL virtual host name.

no ssl settings authmandatory <virtual_host_name>

This command is used to disable the mandatory mode of client authentication for the specified SSL virtual host.

ssl settings acceptchain <host_name>

This command is used to enable the certificate chain support for the specified SSL virtual or real host. When the certificate chain support is enabled, the authentication will succeed when the SSL host finds that a CA certificate in the certificate chain sent from the peer exists in the trusted CA certificate list. For the SSL virtual host, this command takes effect only when the client authentication function is enabled.

`host_name` This parameter specifies the SSL virtual or real host name.



Note: After the certificate chain support was enabled through the “ssl settings acceptchain” command, the appliance returns 903 if the encoding method of the Issuer field in the user certificate was inconsistent with that of the Subject field in the Sub-CA certificate.

no ssl settings acceptchain <host_name>

This command is used to disable the certificate chain support for the specified SSL virtual or real host.

ssl settings clientcert minkey <virtual_host_name> <key_length>

This command is used to set the minimum RSA key length in the client certificate required for accessing the specified SSL virtual host. This configuration is optional for the SSL virtual host. If this command is not configured, the SSL virtual host will not check the RSA key length in the client certificate.

If the RSA key length in the client certificate is smaller than the configured minimum RSA key length:

- For the SSL virtual host associated with the HTTPS virtual services, the system will reject the client access and generate the error code “902: The SSL client certificate's key size is too small”.
- For the SSL virtual host associated with the TCPS and FTPS virtual services, the system will reject the client access and generate logs.

virtual_host_name	This parameter specifies the SSL virtual host name.
key_length	This parameter specifies the minimum RSA key length in the client certificate required for accessing the specified SSL virtual host, in bits. • For the appliance with the FIPS HSM installed, the value must be 1024, 2048 or 4096. • For the appliance without the FIPS HSM installed, the value must be 512, 1024, 2048 or 4096.



Note: Before executing this command, make sure that the client authentication function has been enabled for the SSL virtual host using the “**ssl settings clientauth**” command.

no ssl settings clientcert minkey <virtual_host_name>

This command is used to delete the setting of the minimum RSA key length in the client certificate required for accessing the specified SSL virtual host.

ssl settings minimum <virtual_host_name> <cipher_strength> <redirect_url>

This command is used to set the minimum cipher length required for the client to access the specified SSL virtual host. This configuration is optional for the SSL virtual host.

If the client connecting to the SSL virtual host does not support the minimum cipher length specified by the “cipher_strength” parameter, the client request will be redirected to the URL specified by the “redirect_url” parameter.

virtual_host_name	This parameter specifies the SSL virtual host name.
cipher_strength	This parameter specifies the minimum cipher length required for the client to access the specified SSL virtual host. Its value must be 40, 56, 128, 168, 256 or 512.
redirect_url	This parameter specifies the URL to which the client request will be redirected. The maximum length of its value is restricted by the maximum length of the command line supported by CLI. Currently, CLI supports the command line with the maximum length of 1024

characters.



Note: This command is available only for the SSL virtual hosts associated with the HTTPS virtual services.

no ssl settings minimum <virtual_host_name>

This command is used to delete the setting of the minimum cipher length required for the client to access the specified SSL virtual host.

Server Authentication

ssl globals verifycert {on|off}

This command is used to enable or disable the server authentication function for all SSL real hosts. This function allows the SSL real host to validate the certificate of the SSL server when the APV appliance acts as the SSL proxy client. If this function is enabled, the SSL real host will initiate the SSL handshake with one-way authentication. When this function is disabled, the SSL real host will not validate the certificate of the SSL server during the SSL handshake. By default, this function is enabled.

ssl settings servername <real_host_name> <common_name>

This command is used to enable the check of the SSL server's common name and specify the expected SSL server common name for the specified SSL real host. When this check is enabled, after the SSL server certificate is successfully validated, the SSL real host will also check whether the common name of the SSL server matches the expected SSL server common name. If not, the SSL real host will reject the server certificate.

<code>real_host_name</code>	This parameter specifies the SSL real host name.
<code>common_name</code>	This parameter specifies the expected SSL server common name. Its value must be a string of 1 to 64 characters. Numbers, letters and underscores are supported.

no ssl settings servername <real_host_name>

This command is used to disable the check of the SSL server's common name for the specified SSL real host.

SSL Renegotiation



Note: SSL renegotiation refers to the SSL secure renegotiation. Unsecure renegotiations are not supported.

The following commands are used to control the enabling status of SSL renegotiation globally and for individual SSL virtual hosts.



Note: In HTTP/2 over TLS deployment scenario, SSL renegotiation must be disabled.

ssl globals renegotiation {on|off}

This command is used to enable or disable the SSL renegotiation function globally (all SSL hosts). By default, this function is disabled globally.

ssl settings renegot <host_name>

This command is used to enable the SSL renegotiation function for the specified SSL host. By default, this function is disabled for every SSL host. The function takes effect only when it is enabled both on the global and on the specified SSL host.

host_name This parameter specifies the SSL virtual host name.

no ssl settings renegot <host_name>

This command is used to disable the SSL renegotiation function for the specified SSL host.

ssl settings clientreneg <real_host_name> [reneg_mode]

This command is used to specify the extension force mode of the secure renegotiation extension for the SSL real host. By default, this function is disabled.

real_host_name This parameter specifies the SSL real host name.

reneg_mode Optional. This parameter specifies the extension force mode of the secure renegotiation. Its value must be:

- 1: indicates that the serverhello sent by the server does not carry secure renegotiation extensions.
- 2: indicates that when the serverhello sent by the server does not carry the secure renegotiation extension item, the real host sends RESET to disconnect the connection.

The default value is 1.

SSL Session Function Options

ssl settings reuse <host_name>

This command is used to enable the SSL session reuse function for the specified SSL virtual or real host. By default, this function is enabled for every SSL real and virtual hosts.

host_name This parameter specifies the SSL virtual or real host name.



Note: A reused session still adopts the cipher suite negotiated in the previous session. When cipher suite settings are changed, SSL real host will not initiate session reuse, and SSL virtual host will reset the connection.

no ssl settings reuse <host_name>

This command is used to disable the SSL session reuse function for the specified SSL virtual or real host.

ssl settings insertclientip <virtual_host_name> <ip_type> <extension_type>

This command is used to enable the function of inserting the client IP address into the ServerHello packet for the specified virtual host. This command can also be used to modify the client IP address inserted into the ServerHello packet for the specified virtual host.

virtual_host_name This parameter specifies the SSL virtual host name.

ip_type This parameter specifies the type of the client IP address to be inserted.

- ipv4: indicates the IPv4 address.
- ipv6: indicates the IPv6 address.

extension_type This parameter specifies the extension type of ServerHello packet. Its value must be an integer ranging from 57 to 65,535.



Note:

- The enabling of this function will cause inaccessible website errors. The administrator should execute this command with caution.
- This command only takes effect when the protocol version set for the SSL virtual host is one or more of TLSv1.0, TLSv1.1 or TLSv1.2.

no ssl settings insertclientip <virtual_host_name> [ip_type]

This command is used to disable the insertion of the client IP address into the ServerHello packet for the specified virtual host.

<code>virtual_host_name</code>	This parameter specifies the name of an existing SSL virtual host.
<code>ip_type</code>	Optional. This parameter specifies the type of the client IP address to be deleted. • <code>ipv4</code>: indicates the IPv4 address. • <code>ipv6</code>: indicates the IPv6 address. The default value is “IPv4”.

ssl globals sessiontimeout <timeout>

This command is used to set the timeout value for the SSL session. When this command is not configured, the default SSL session timeout value is 1800 seconds.

<code>timeout</code>	This parameter specifies the timeout value in seconds. Its value must be an integer ranging from 60 to 86,400.
----------------------	--

ssl globals sendclosenotify {on|off}

This command is used to enable or disable the function of sending the SSL Close Notify message for all SSL virtual and real hosts when the SSL host closes the SSL connection. When this function is enabled, to close the SSL connection during the SSL session, the SSL virtual or real host will send the SSL Close Notify message to the peer. By default, this function is enabled.

ssl globals ignoreclosenotify {on|off}

This command is used to enable or disable the function of ignoring the SSL Close Notify message sent from the peer for all SSL virtual and real hosts. By default, this function is enabled.

During the SSL session, if any party wants to close the SSL connection, it needs to send the SSL Close Notify message to the peer.

- If this function is enabled, when the SSL connection is closed, the system will ignore the improper closing of the SSL connection and keep on reusing the SSL session pertaining to this SSL connection even if the SSL virtual or real host does not receive the SSL Close Notify message from the peer.
- If this function is disabled, when the SSL connection is closed, the SSL virtual or real host needs to receive the SSL Close Notify message sent from the peer; otherwise, the SSL session pertaining to this SSL connection will be marked as invalid and cleared.

SSL Error Page Customization

ssl import error <error_code> <url> [host_name]

This command is used to import a customized SSL error page that will be returned to the client for the specified SSL host or for the global (all SSL hosts). Its size should not exceed 10 MB. You can import a static error page for the specified SSL error code. The customized static error page must be a static HTML without pictures and flashes.

error_code	This parameter specifies the error code for which the customized error page is imported. Currently, only the following error codes are supported: • 901: SSL host requires the client certificate; • 902: The certificate's key size is too small; • 903: The certificate is not trusted; • 904: The certificate is expired; • 905: The certificate isn't validate yet; • 906: The certificate has been revoked.
url	This parameter specifies the HTTP or FTP URLs of the customized error page to be imported. The maximum length of its value is restricted by the maximum length of the command line supported by CLI. Currently, CLI supports the command line with the maximum length of 1024 characters. Note: The “path” of the FTP URL (ftp://user:password@host:port/path) should be a relative path.
host_name	Optional. This parameter specifies the SSL host name. Its value must be: • SSL host name: indicates that the customized error page is imported for the specified SSL host. • ALL: indicates that the customized error page is imported for the global. The default value is “ALL”.



Note:

- To ensure a proper display of the SSL error page, please change its encoding to “UTF-8 without BOM” before importing it.
- On WebUI, in addition to URL, the system also supports importing error pages through local files.

no ssl import error <error_code> [host_name]

This command is used to delete the customized SSL error page imported for the specified SSL host or the global.

show ssl import error page <error_code> [host_name]

This command is used to display the customized SSL error page imported for the specified SSL host or the global.

show ssl import error list

This command is used to display the list of the customized SSL error pages imported for individual SSL hosts and for the global.

clear ssl import error

This command is used to clear all customized SSL error pages imported for individual SSL hosts and for the global.

ssl load error <error_code> [host_name]

This command is used to enable the customized SSL error page imported for the specified SSL host or for the global. When the SSL client fails the client authentication, the enabled customized error page will be displayed on the SSL client.

error_code	This parameter specifies the error code for which the customized SSL error page has been imported using the “ssl import error” command. Currently, only the following error codes are supported: • 901: SSL host requires the client certificate; • 902: The certificate's key size is too small; • 903: The certificate is not trusted; • 904: The certificate is expired; • 905: The certificate isn't validate yet; • 906: The certificate has been revoked.
host_name	Optional. This parameter specifies the SSL host name. Its value must be: • SSL host name: indicates that the customized error page imported for the specified SSL host will be enabled. • ALL: indicates that the customized error page imported for the global will be enabled. The default value is “ALL”.



Note: When an error code is generated by an SSL host, if the customized error page of this error code has been enabled for this SSL host, this customized

error page will be returned; if no customized error page of this error code has been enabled for this SSL host but the customized error page of this error code has been enabled for the global, the customized error page enabled for the global will be returned; if no customized error page of this error code has been enabled for the global, the default error page will be returned.

no ssl load error <error_code> [host_name]

This command is used to disable the customized error page that has been enabled for the specified SSL host or for the global.

show ssl load error page <error_code> [host_name]

This command is used to display the customized error page that has been enabled for the specified SSL host or for the global.

show ssl load error list

This command is used to display the list of the customized error pages that have been enabled for individual SSL hosts and for the global.

clear ssl load error

This command is used to disable all customized SSL error pages that have been enabled for individual SSL hosts and for the global.

SSL Tunneling

**ssl channel name <channel_name> <ip> <port> <encryption_port>
<direction> <host_name>**

Creates an SSL channel. After this command is run, traffic will go through the SSL channel only if one of the following conditions is met:

- The destination IP address and the destination port (the parameter **direction** is **out**) match the IP address and port of the outgoing SSL channel, respectively.
- The source IP address and source port (the parameter **direction** is **in**) match the IP address and port of the incoming SSL channel, respectively.

Furthermore, after running the command, the system will automatically generate related SLB and SSL configurations based on the traffic's direction. The details are listed as follows:

- The **direction** parameter is **in**: The system automatically generates a TCP real service and TCPS virtual service and associates them using a static policy. It also automatically generates an SSL virtual host and associates it with a virtual service.

- The **direction** parameter is **out**: The system automatically generates a TCPS real service and TCP virtual service and associates them using a static policy. It also automatically generates an SSL real host and associates it with a real service.

For auto-generate virtual and real hosts, you need to manually add certificates to them. A maximum of 200 SSL channels can be created.

channel_name	The name of an SSL channel. The value must be a string with a length not greater than 120 bytes. Numbers, case-sensitive letters, and special characters are supported. When the value contains special characters, it must be enclosed in double quotes. The value cannot be set to a system reserved keyword such as “default,” “all,” or “global,” which are case-insensitive.
ip	An internal IP address for the channel. Typically it is an IP address that provides encryption or decryption for the APV, that is, an IP address automatically generated for SLB virtual service and SLB real service.
port	An internal port for an SSL channel. The value must be an integer ranging from 1 to 65,535. • When the direction value is in, port specifies the port of a TCP real service. • When the direction value is out, port specifies the port of a TCP virtual service.
encrypt_port	An encrypted port of an SSL channel. The value must be an integer ranging from 1 to 65,535. • When the direction value is in, encrypt_port specifies the port of a TCPS virtual service. • When the direction value is out, encrypt_port specifies the port of a TCPS real service.
direction	The direction of an SSL channel. The value must be in or out .
host_name	The name of an SSL host. The value can be the name of an existing SSL virtual or real host, or the name of a newly generated SSL virtual or real host. The value must be a string with a length not greater than 240 bytes. Numbers, letters, and underscore are supported, but the string cannot start with the underscore. Special characters are not supported. • When the direction value is in, host_name needs to

be set to an SSL virtual host.

- When the **direction** value is **out**, **host_name** needs to be set to an SSL real host.

Examples:

➤ The “**in**” direction:

```
AN(config) ssl channel name channel1 1.1.1.1 6000 443 in ssl_vhost1
```

After the above command is run, the TCPS virtual service “channel1” and TCP real service “channel1_chauto” will be automatically generated. The details are listed as follows:

```
slb virtual tcps channel1 1.1.1.1 443
slb real tcp channel1_chauto 1.1.1.1 6000 tcp
slb policy static channel1 channel1_chauto
ssl host virtual ssl_vhost1 channel1
system mode transparent channel1
```

➤ The “**out**” direction:

```
AN(config) ssl channel name channel1 1.1.1.1 6000 443 out ssl_rhost1
```

After the above command is run, the TCP virtual service “channel1_chauto” and TCPS real service “channel1” are automatically generated. The details are listed as follows:

```
slb virtual tcp channel1_chauto 1.1.1.1 6000 noarp
slb real tcps channel1 1.1.1.1 443 tcp
slb policy static channel1_chauto channel1
ssl host real ssl_rhost1 channel1
system mode transparent channel1_chauto
```

Note that when the auto-generate commands above are displayed using the command **show**, they are preceded by a number sign (#).



Note:

- The APV’s backend traffic does not allow the access to the virtual services of an SSL channel in the **in** direction. The APV’s incoming traffic does not allow the access to the virtual services of an SSL channel in the **out** direction.
- By default, “noarp” is enabled for the virtual services of the SSL channel in

the **out** direction.

- By default, the health checks are enabled for virtual services of an SSL channel (using the command **slb virtual health on**).
- The auto-generate configurations during the creation of an SSL channel can only be deleted using the command **no ssl channel name** and **clear ssl channel name**.

no ssl channel name *<channel_name>*

Deletes the configuration of an SSL channel.

show ssl channel name

Displays the configurations of all SSL channels.

clear ssl channel name

Deletes the configurations of all SSL channels. Note that after running this command, the SSL hosts generated using the **ssl channel name** command will not be deleted.

show ssl channel relations *[channel_name]*

Displays the configuration of an SSL channel and the auto-generate related configuration. If no value is assigned to the **channel_name** parameter, the configurations of all SSL channels and auto-generate related configurations will be displayed.

show ssl channel status *[channel_name]*

Displays the status and the statistics of an SSL channel. If no value is assigned to the **channel_name** parameter, the status and statistics of all SSL channels will be displayed.

SSL General Commands

ssl globals statistics handshake on *[duration]*

This command is used to enable the SSL handshake success rate statistics function. When this function is enabled, the system will count the handshake success rate of all SSL hosts and their associated SLB services within the specified time period. By default, this function is disabled.

duration	Optional, this parameter specifies the statistical time of the SSL handshake success rate. Its value must be an integer ranging from 1 to 86,400, in seconds. The default value is 300. When the value is less than or equal to 1000, the statistical sampling period of handshake success rate is 1 second; When the value is greater than 1000, the statistical sampling period of handshake success rate are duration/1000 seconds.
----------	--

Note: If the value is greater than 1000, the sampling period is duration/1000, you need to wait for a period of time before the system displays the statistics of the SSL handshake success rate.

ssl globals statistics handshake off

This command is used to disable the SSL handshake success rate statistics function. When this function is disabled, all existing statistics will be cleared.

ssl globals ems {on|off}

This command is used to enable or disable TLS Extended Master Secret (EMS) extension support. When TLS EMS support is enabled, the system will support negotiation of the TLS EMS extension. By default, TLS EMS support is disabled.

ssl settings alpn <virtual_host_name>

This command is used to enable ALPN support for the specified TCPS type virtual host. By default, ALPN support is disabled for all TCPS type virtual hosts.

virtual_host_name	This parameter specifies the name of a TCPS type virtual host.
-------------------	--

no ssl settings alpn <virtual_host_name>

This command is used to disable ALPN support for the specified TCPS type virtual host.

ssl globals serialnumber {upper|lower}

This command is used to set the letter case of client certificate's serial number when the certificate or serial number is forwarded to the real service. With “**ssl globals serialnumber upper**” configured, the serial number will be printed in uppercase letters. With “**ssl globals serialnumber lower**” configured, the serial number will be printed in lowercase letters.

The default configuration for the system is “**ssl globals serialnumber upper**”.

ssl globals certexpdays <days>

This command is used to set the certificate expiration notification days. After executing this command, for the activated SSL host, if the certificate expiration days are less than or equal to the set notification days, the system will record related system logs. The notification rules are as follows:

- If the certificate expiration time is longer than or equal to 30 days, the system will record a system log in “notice” level.

- If the certificate expiration time is longer than or equal to 7 days and less than 30 days, the system will record a system log in “warning” level.
- If the certificate expiration time is less than 7 days, the system will record a system log in “critical” level.
- If the certificate has expired, the system will record a system log in “critical” level.

days This parameter specifies the certificate expiration notification days. Its value must be an integer ranging from 1 to 180. The default value is 60.

**Note:**

- The system will not notify the expiration time of CRL CA certificates and global root CA certificates.
- The system will not notify the expiration time of SSL client certificates.

ssl globals certchkinterval [interval]

This command is used to set the detection interval of the certificate expiration time.

interval This parameter specifies the interval, in minute, at which the certificate expiration time is checked. Its value must be an integer ranging from 1 to 1440.

The default value is 1440.

show ssl globals

This command is used to display all SSL global configurations.

show ssl settings <host_name>

This command is used to display the configurations of the specified SSL virtual or real host, including the host name, cipher suite, protocol and so on.

ssl start <host_name>

This command is used to enable the specified SSL virtual or real host. When an SSL host is enabled, the SSL settings of this SSL host will take effect for all SLB services associated with it. By default, the SSL host is disabled.

host_name This parameter specifies the SSL virtual or real host name. Its value must be:

- SSL host name: indicates that the specified SSL virtual or real host is enabled.
- ALL: indicates that all SSL virtual and real hosts are

enabled.

ssl stop <host_name>

This command is used to disable the specified SSL virtual or real host. This command will not delete the configurations of this SSL host such as the private key and certificate. When an SSL host is disabled, the SSL settings of this SSL host will stop taking effect for all SLB services associated with it.

host_name This parameter specifies the SSL virtual or real host name. Its value must be:

- SSL host name: indicates that the specified SSL virtual or real host is disabled.
- ALL: indicates that all SSL virtual and real hosts are disabled.

show ssl status host

This command is used to display the status of the currently configured SSL virtual and real hosts.

show statistics ssl [host_name]

This command is used to display the SSL connection statistics and session statistics of the specified SSL virtual or real host. If the “host_name” parameter is not specified, the statistics of all SSL virtual and real hosts will be displayed.

For example:

```
AN#show statistics ssl
SSL Connection Statistics for "ssl_test"
    Open SSL connections: 106
    Accepted SSL connections: 38287224
    Requested SSL connections: 38387157
    5 minutes requested rate: 1203 connections/sec

SSL Session Statistics for "ssl_test"
    Resumed SSL sessions: 38368158
    Resumable SSL sessions: 43
    Session Misses: 0

SSL handshake statistics (300s):
    Global: success: 1   all: 1   success rate: 100.00%
```



Note: “Resumed SSL sessions”, “Resumable SSL sessions” and “Session Misses” in SSL statistics are cumulative values. Among them, the value of “Resumable SSL sessions” will not decrease even when SSL session times out.

clear statistics ssl [host_name]

This command is used to clear the statistics of the specified SSL virtual or real host. If the “host_name” parameter is not specified, the statistics of all SSL virtual and real hosts will be cleared.

FIPS

ssl fips so hsminit

This command is used to initialize the Federal Information Processing Standard (FIPS) Hardware Security Module (HSM). FIPS HSM uses the Security Officer (SO) account to complete the initialization. The default username and password of SO are “cavium” and “default” respectively. After the administrator logs into the FIP HSM using the default username and password of SO, new SO and Crypto User (CU) will be configured. With the initialization completed, the APV system will automatically reboot.

Please note that this operation also clears public-private key pairs and certificates, CSRs and root CA certificates, intermediate CA certificates and CRLs configured for all SSL hosts from the FIPS HSM. Before performing this operation, it is recommended to back up these items resident on the FIPS HSM via FIPS cloning. For details of FIPS cloning, please refer to FIPS User Guide.

ssl fips user login

This command is used to allow the CU to log into the FIPS HSM. Upon successful login, the CU can perform FIPS cryptographic operation.

ssl fips user logout

This command is used to allow the CU to log out of the FIPS HSM. After logout, no FIPS cryptographic operation can be performed. It is recommend that administrators log into FIPS HSM upon system bootup, and stay in login state during runtime to avoid any interruptions on SSL traffic.

ssl fips user autologin

This command is used to enable autologin function. With this function enabled, the administrator needs to run the “write memory” command to save the setting permanently so that the CU can automatically log into the FIPS HSM on each subsequent bootup.

no ssl fips user autologin

This command is used to disable autologin function. After this command is executed, the administrator needs to run the “write memory” command to save the setting permanently so that CU will be required to manually log in after each reboot.

ssl fips so hsmzeroize

This command is used to reset the FIPS HSM to factory default. After this command is executed, the system will automatically reboot.

Please note that this operation also clears public-private key pairs and certificates, CSRs and root CA certificates, intermediate CA certificates and CRLs configured for all SSL hosts from the FIPS HSM. Before performing this operation, it is recommended to back up these items resident on the FIPS HSM via FIPS cloning. For details of FIPS cloning, please refer to FIPS User Guide.



Note: If SO fails to log into FIPS HSM 20 consecutive times, the entire FIPS HSM will also be reset to factory default.

Cryptographic Algorithm

The following commands are applicable to certain countries.

- **ssl gmsdf random**
- **ssl gmsdf reset**
- **show ssl gmsdf device**
- **show ssl gmsdf permission**
- **ssl gmsdf failover**
- **ssl gmsdf guoxin admin**
- **no ssl gmsdf guoxin admin**
- **ssl gmsdf guoxin operator**
- **no ssl gmsdf guoxin operator**
- **ssl gmsdf guoxin login admin**
- **ssl gmsdf guoxin login operator**
- **ssl gmsdf guoxin logout**
- **ssl gmsdf guoxin changepin**
- **ssl gmsdf sanwei admin**
- **no ssl gmsdf sanwei admin**
- **ssl gmsdf sanwei operator**
- **no ssl gmsdf sanwei operator**
- **ssl gmsdf sanwei login**
- **ssl gmsdf sanwei logout**
- **ssl gmsdf sanwei changepin**

Chapter 14 SSL Interception

SSL Interception and Certificate Settings

ssli on *<host_name>* *<distribution_mode>*

This command is used to enable SSL interception for the specified SSL virtual host or real host and set the distribution mode for ingress and egress nodes. Administrators can use the “**show ssl status host**” command to display the enabling status of SSL interception. Note: After this command is executed, client authentication and renegotiation will be disabled and cannot be enabled.

host_name	This parameter specifies the SSL virtual host or real host name.
distribution_mode	This parameter specifies the distribution mode of the ingress and egress nodes: • 0: indicates that the ingress and egress nodes are distributed on two APV appliances. • 1: indicates that the ingress and egress nodes are integrated on one APV appliance.



Note:

After SSL interception is enabled, the client authentication and renegotiation functions will be automatically disabled. Besides, the protocol versions supported by the SSL host are changed to “SSLv3:TLSv1:TLSv1.1:TLSv1.2”, and the cipher suite settings for this SSL host are also changed to all the cipher suites that are supported by these four protocol versions. The protocol versions and cipher suites cannot be changed anymore.

ssli off *<host_name>*

This command is used to disable SSL interception for the specified SSL virtual host or real host.

ssli cacert rsa *<virtual_host_name>* *[key_length]* *[certificate_index]*
[signature_algorithm_index]

This command is used to generate an RSA CA certificate and private key for the specified SSL virtual host. The certificates generated by this command can be activated, viewed and deleted by using the “**ssl activate certificate**”, “**show ssl certificate**” and “**no ssl certificate**” commands.

virtual_host_name	This parameter specifies the SSL virtual host name.
key_length	Optional. This parameter specifies the length of the generated RSA key pair, in bits. Its value must be 512, 1024, 2048 or 4096 (only some specific APV models

support 4096-bit certificates).

The default value is 2048.

certificate_index

Optional. This parameter specifies the index of the generated certificate. Its value must be 1, 2 or 3.

The default value is 1.

signature_algorithm_index

Optional. This parameter specifies the index of the certificate's signature algorithm. Its value must be 1, 2, 3 or 4, indicating sha256RSA, sha384RSA, sha512RSA and sha1RSA respectively.

The default value is 1.

ssli cacert ecc <virtual_host_name> [curve_name] [certificate_index] [signature_algorithm_index]

This command is used to generate an ECC CA certificate and private key for the specified SSL virtual host. The certificates generated by this command can be activated, viewed and deleted by using the “**ssl activate certificate**”, “**show ssl certificate**” and “**no ssl certificate**” commands.

virtual_host_name

This parameter specifies the SSL virtual host name.

curve_name

Optional. This parameter specifies the elliptic curve name. Its value must be “prime256v1”, “secp384r1” or “secp521r1”.

The default value is “prime256v1”.

certificate_index

Optional. This parameter specifies the index of the generated certificate. Its value must be 1, 2 or 3.

The default value is 1.

signature_algorithm_index

Optional. This parameter specifies the index of the certificate's signature algorithm. Its value must be 1, 2, 3 or 4, indicating sha256ECDSA, sha384ECDSA, sha512ECDSA and sha1ECDSA respectively.

The default value is 1.

ssli settings certcache timeout <timeout>

This command is used to set a timeout value for simulated certificates that are added to cache. If this command is not configured, the default timeout value for simulated certificates is 1800s. If a simulated certificate is not hit within the specified period, it will be moved out of the cache.

timeout This parameter specifies the timeout value for simulated certificates that are added to cache. Its value must be integer ranging from 60 to 86,400, in seconds.

The number of simulated certificates that can be cached depends on the system memory:

System Memory	Number of Simulated Certificates that Can be Cached
4 GB/8 GB	40,000
16GB/24 GB	80,000
32 GB	160,000
64 GB/96 GB/128 GB	320,000

show ssli settings

This command is used to display all global configurations related to SSL interception.

Domain List

Using the domain list function, the administrator can apply bypass or interception domain lists or strings to an SSL interception virtual host. Bypass domain lists (strings) and interception domain lists (strings) are mutually exclusive. They cannot be configured on the same virtual host simultaneously. When a client's SNI or server certificate's Common Name or Subject Alternative Name (SAN) matches a bypass domain list or string, the virtual host will allow the SSL traffic to pass through without being decrypted. When a client's SNI or server certificate's Common Name or SAN matches an interception domain list or string, the virtual host will intercept the SSL traffic.

ssli domainlist {on|off}

This command is used to enable or disable the domain list function. By default, this function is disabled.



Note: The ingress node performs domainlist match check only when attempting to obtain server certificates. Therefore, after the system is running for some time, newly added domainlist configurations may not take effect on the certificates that have already been cached. Administrators can set the certificate cache timeout (configured by the “**ssli settings certcache timeout**” command) to the minimum value “60” to avoid the prementioned problem.

show ssli domainlist status

This command is used to display the enabling status of the domain list function.

ssli domainlist list <list_name> <list_type>

This command is used to create a domain list. The system supports a maximum of 256 domain lists.

<code>list_name</code>	This parameter specifies the domain list name. Its value must be a string of 1 to 64 characters, which is case-sensitive.
<code>list_type</code>	This parameter specifies the type of the domain list. Its value must be: • <code>bypass</code>: indicates that a bypass domain list will be created. • <code>intercept</code>: indicates that an interception domain list will be created.

no ssl domainlist list <list_name>

This command is used to delete the specified domain list and all items contained in it.

show ssl domainlist list [list_type]

This command is used to display all configured domain lists of the specified type.

<code>list_type</code>	This parameter specifies the type of domain lists to be displayed. Its value must be: • <code>bypass</code>: displays bypass domain lists. • <code>intercept</code>: displays interception domain lists. • <code>all</code>: displays all types of domain lists.
------------------------	---

The default value is “all”.

clear ssl domainlist list [list_type]

This command is used to clear all domain lists and items contained in them.

<code>list_type</code>	This parameter specifies the type of domain lists to be cleared. Its value must be: • <code>bypass</code>: clears bypass domain lists. • <code>intercept</code>: clears interception domain lists. • <code>all</code>: clears all types of domain lists.
------------------------	---

The default value is “all”.

ssl domainlist item <list_name> <domain_regex>

This command is used to add a domain string as an item to the specified domain list. Each domain list can contain a maximum of 1024 items.

<code>list_name</code>	This parameter specifies the domain list name.
<code>domain_regex</code>	This parameter specifies the domain string to be added to the domain list as an item. Its value must be a string of 1 to 64 characters, which is case-insensitive. The parameter value supports quick regular expression in the format of “[^]string.[*string2[*.stringN]][\$]”, which must be enclosed in double quotes. For information about the quick regular expression, please refer to the “Regular Expressions” section in Chapter 1.

no ssl domainlist item <list_name> <domain_regex>

This command is used to remove the specified item from the specified domain list.

show ssl domainlist item <list_name>

This command is used to display all items contained in the specified domain list.

clear ssl domainlist item <list_name>

This command is used to remove all items from the specified domain list.

ssl domainlist apply list <virtual_host_name> <list_name>

This command is used to apply a domain list to the specified virtual host. A maximum of 256 domain lists can be applied to each virtual host.

<code>virtual_host_name</code>	This parameter specifies the SSL virtual host name.
--------------------------------	---

<code>list_name</code>	This parameter specifies the domain list name.
------------------------	--



Note: If a virtual host already has a bypass domain list or domain string applied, it cannot have interception domain lists or domain strings applied any more. It is the same in reverse.

no ssl domainlist apply list <virtual_host_name> <list_name>

This command is used to cancel the application of the specified domain list to the specified virtual host.

ssl domainlist apply item <virtual_host_name> <domain_regex> <item_type>

This command is used to apply a bypass or an interception domain string to the specified virtual host. Each virtual host can have at most 1024 individual domain strings applied.

virtual_host_name	This parameter specifies the SSL virtual host name.
domain_regex	This parameter specifies the domain string to be applied. Its value must be a string of 1 to 64 characters, which is case-insensitive. The parameter value supports quick regular expression in the format of “[^]string.[*string2[*stringN]][\$]”, which must be enclosed in double quotes. For information about the quick regular expression, please refer to the “Regular Expressions” section in Chapter 1.
item_type	This parameter specifies the type of the domain string to be applied. Its value must be: • bypass: indicates that a bypass domain string will be applied. • intercept: indicates that an interception domain string will be applied.



Note: If a virtual host already has a bypass domain string or domain list applied, it cannot have interception domain strings or domain lists applied any more. It is the same in reverse.

no ssl domainlist apply item <virtual_host_name><sni_regex>

This command is used to cancel the application of the domain string to the specified virtual host.

show ssl domainlist apply <virtual_host_name> [type]

This command is used to display all domain strings and (or) domain lists applied to the specified virtual host.

virtual_host_name	This parameter specifies the SSL virtual host name.
type	Optional. This parameters specifies the SSL domain list type to be displayed. Its value must be: • item: displays the domain strings applied to the virtual host by the “ssl domainlist apply item” command. • list: displays the domain lists applied to the virtual host by the “ssl domainlist apply list” command. • all: displays all domain strings and domain lists applied to the virtual host. The default value is “all”.

clear domainlist apply <virtual_host_name> [type]

This command is used to cancel the application of domain strings and (or) domain lists to the specified virtual host.

<code>virtual_host_name</code>	This parameter specifies the SSL virtual host name.
<code>type</code>	Optional. This parameters specifies the SSL domain list type to be cleared. Its value must be: • <code>item</code>: cancels the application of the domain strings to the virtual host by the “ssli domainlist apply item” command. • <code>list</code>: cancels the application of the domain lists to the virtual host by the “ssli domainlist apply list” command. • <code>all</code>: cancels the application of all domain strings and domain lists to the virtual host. The default value is “all”.

show statistics ssli domainlist [*virtual_host_name*]

This command is used to display the domain list matching statistics on the specified virtual host. When the matched domain list name is displayed as “-”, it means this domain string is individually applied to the virtual host. If the “`virtual_host_name`” parameter is not specified, the domain list matching statistics on all virtual hosts are displayed.

clear statistics ssli domainlist [*virtual_host_name*]

This command is used to clear the domain list matching statistics on the specified virtual host. If the “`virtual_host_name`” parameter is not specified, the domain list matching statistics on all virtual hosts are cleared.

show ssli domainlist match <*virtual_host_name*> <*domain_name*>

This command is used to check whether the specified virtual host has the specified domain string applied.

<code>virtual_host_name</code>	This parameter specifies the SSL virtual host name.
<code>domain_name</code>	This parameter specifies the domain string to be checked.

URL Filtering

ssli webclassify {on|off} <*virtual_host_name*>

This command is used to enable the website classification function for the specified virtual host of the SSL interception module to implement intelligent URL filtering. This function can be enabled only after the global website classification function is

enabled (“**webclassify on**”) and the SSL interception function is enabled on the virtual host. For descriptions on the global website classification function commands, see the “Website Classification” section in Chapter 2.

virtual_host_name	This parameter specifies the SSL virtual host name. Its value must be the name of the virtual host associated with the TCPS virtual service on the ingress node.
-------------------	--

ssli webclassify defaction <virtual_host_name> <action>

This command is used to configure the specified virtual host to bypass or intercept traffic that accesses a website whose category cannot be recognized. If this command is not configured, the system will not intercept such traffic by default.

virtual_host_name	This parameter specifies the SSL virtual host name. Its value must be the name of the virtual host associated with the TCPS virtual service on the ingress node.
-------------------	--

action	This parameter specifies the processing mode. Its value must be:
--------	--

- 0: bypasses traffic that accesses an unrecognized website category.
- 1: intercepts traffic that accesses an unrecognized website category.

ssli webclassify url bypass <virtual_host_name> <category_name>

This command is used to define a bypass website category for the specified virtual host, that is, the virtual host will not intercept traffic that accesses a website belonging to this website category. The function of this configuration is the same as that of bypass domain lists (strings). When a virtual host has both domain lists (strings) and website categories configured, the system will preferentially match domain lists (strings).

For the same virtual host, the “**ssli webclassify url bypass**” and “**ssli webclassify url intercept**” commands cannot be configured simultaneously.

virtual_host_name	This parameter specifies the SSL virtual host name. Its value must be the name of the virtual host associated with the TCPS virtual service on the ingress node.
-------------------	--

category_name	This parameter specifies the website category. Its value must be the name (case sensitive) of the website category, enclosed in double quotes, such as “Real Estate”. The names of website categories supported by the system are as follows:
---------------	---

- Real Estate (Category ID: 1)

- Computer and Internet Security (Category ID: 2)
- Financial Services (Category ID: 3)
- Business and Economy (Category ID: 4)
- Computer and Internet Info (Category ID: 5)
- Auctions (Category ID: 6)
- Shopping (Category ID: 7)
- Cult and Occult (Category ID: 8)
- Travel (Category ID: 9)
- Abused Drugs (Category ID: 10)
- Adult and Pornography (Category ID: 11)
- Home (Category ID: 12)
- Military (Category ID: 13)
- Social Network Category ID: 14)
- Dead Sites (Category ID: 15)
- Individual Stock Advice and Tools (Category ID: 16)
- Training and Tools (Category ID: 17)
- Dating (Category ID: 18)
- Sex Education (Category ID: 19)
- Religion (Category ID: 20)
- Entertainment and Arts (Category ID: 21)
- Personal Sites and Blogs (Category ID: 22)
- Legal (Category ID: 23)
- Local Information (Category ID: 24)
- Streaming Media (Category ID: 25)
- Job Search (Category ID: 26)
- Gambling (Category ID: 27)
- Translation (Category ID: 28)
- Reference and Research (Category ID: 29)
- Shareware and Freeware (Category ID: 30)
- P2P (Category ID: 31)
- Marijuana (Category ID: 32)
- Hacking (Category ID: 33)
- Games (Category ID: 34)
- Philosophy and Political Advocacy (Category ID: 35)

- Weapons (Category ID: 36)
- Pay to Surf (Category ID: 37)
- Hunting and Fishing (Category ID: 38)
- Society (Category ID: 39)
- Educational Institutions (Category ID: 40)
- Online Greeting cards (Category ID: 41)
- Sports (Category ID: 42)
- Swimsuits & Intimate Apparel (Category ID: 43)
- Questionable (Category ID: 44)
- Kids (Category ID: 45)
- Hate and Racism (Category ID: 46)
- Online Personal Storage (Category ID: 47)
- Violence (Category ID: 48)
- Keyloggers and Monitoring (Category ID: 49)
- Search Engines (Category ID: 50)
- Internet Portals (Category ID: 51)
- Web Advertisements (Category ID: 52)
- Cheating (Category ID: 53)
- Gross (Category ID: 54)
- Web based Email (Category ID: 55)
- Malware Sites (Category ID: 56)
- Phishing and Other Frauds (Category ID: 57)
- Proxy Avoid and Anonymizers (Category ID: 58)
- Spyware and Adware (Category ID: 59)
- Music (Category ID: 60)
- Government (Category ID: 61)
- Nudity (Category ID: 62)
- News and Media (Category ID: 63)
- Illegal (Category ID: 64)
- CDNs (Category ID: 65)
- Internet Communications (Category ID: 66)
- Bot Nets (Category ID: 67)
- Abortion (Category ID: 68)
- Health & Medicine (Category ID: 69)

- Confirmed SPAM Sources (Category ID: 70)
- SPAM URLs (Category ID: 71)
- Unconfirmed SPAM Sources (Category ID: 72)
- Open HTTP Proxies (Category ID: 73)
- Dynamic Content (Category ID: 74)
- Parked Sites (Category ID: 75)
- Alcohol and Tobacco (Category ID: 76)
- Private IP Addresses (Category ID: 77)
- Image and Video Search (Category ID: 78)
- Fashion and Beauty (Category ID: 79)
- Recreation and Hobbies (Category ID: 80)
- Motor Vehicles (Category ID: 81)
- Web Hosting (Category ID: 82)

no ssl webclassify url bypass <virtual_host_name> <category_name>

This command is used to delete an “ssl webclassify url bypass” configuration of the specified virtual host.

clear ssl webclassify url bypass <virtual_host_name>

This command is used to clear all “ssl webclassify url bypass” configurations of the specified virtual host.

ssl webclassify url intercept <virtual_host_name> <category_name>

This command is used to define an interception website category for the specified virtual host, that is, the virtual host will intercept all traffic that accesses a website belonging to this website category. The function of this configuration is the same as that of interception domain lists (strings). When a virtual host has both domain lists (strings) and website categories configured, the system will preferentially match domain lists (strings).

For the same virtual host, the “ssl webclassify url bypass” and “ssl webclassify url intercept” commands cannot be configured simultaneously.

virtual_host_name	This parameter specifies the SSL virtual host name. Its value must be the name of the virtual host associated with the TCPS virtual service on the ingress node.
category_name	This parameter specifies the website category. Its value must be the name (case sensitive) of the website category, enclosed in double quotes, such as “Real Estate”. The names of website categories supported by the system are as

follows:

- Real Estate (Category ID: 1)
- Computer and Internet Security (Category ID: 2)
- Financial Services (Category ID: 3)
- Business and Economy (Category ID: 4)
- Computer and Internet Info (Category ID: 5)
- Auctions (Category ID: 6)
- Shopping (Category ID: 7)
- Cult and Occult (Category ID: 8)
- Travel (Category ID: 9)
- Abused Drugs (Category ID: 10)
- Adult and Pornography (Category ID: 11)
- Home (Category ID: 12)
- Military (Category ID: 13)
- Social Network Category ID: 14)
- Dead Sites (Category ID: 15)
- Individual Stock Advice and Tools (Category ID: 16)
- Training and Tools (Category ID: 17)
- Dating (Category ID: 18)
- Sex Education (Category ID: 19)
- Religion (Category ID: 20)
- Entertainment and Arts (Category ID: 21)
- Personal Sites and Blogs (Category ID: 22)
- Legal (Category ID: 23)
- Local Information (Category ID: 24)
- Streaming Media (Category ID: 25)
- Job Search (Category ID: 26)
- Gambling (Category ID: 27)
- Translation (Category ID: 28)
- Reference and Research (Category ID: 29)
- Shareware and Freeware (Category ID: 30)
- P2P (Category ID: 31)
- Marijuana (Category ID: 32)
- Hacking (Category ID: 33)

- Games (Category ID: 34)
- Philosophy and Political Advocacy (Category ID: 35)
- Weapons (Category ID: 36)
- Pay to Surf (Category ID: 37)
- Hunting and Fishing (Category ID: 38)
- Society (Category ID: 39)
- Educational Institutions (Category ID: 40)
- Online Greeting cards (Category ID: 41)
- Sports (Category ID: 42)
- Swimsuits & Intimate Apparel (Category ID: 43)
- Questionable (Category ID: 44)
- Kids (Category ID: 45)
- Hate and Racism (Category ID: 46)
- Online Personal Storage (Category ID: 47)
- Violence (Category ID: 48)
- Keyloggers and Monitoring (Category ID: 49))
- Search Engines (Category ID: 50)
- Internet Portals (Category ID: 51)
- Web Advertisements (Category ID: 52)
- Cheating (Category ID: 53)
- Gross (Category ID: 54)
- Web based Email (Category ID: 55)
- Malware Sites (Category ID: 56)
- Phishing and Other Frauds (Category ID: 57)
- Proxy Avoid and Anonymizers (Category ID: 58)
- Spyware and Adware (Category ID: 59)
- Music (Category ID: 60)
- Government (Category ID: 61)
- Nudity (Category ID: 62)
- News and Media (Category ID: 63)
- Illegal (Category ID: 64)
- CDNs (Category ID: 65)
- Internet Communications (Category ID: 66)
- Bot Nets (Category ID: 67)

- Abortion (Category ID: 68)
- Health & Medicine (Category ID: 69)
- Confirmed SPAM Sources (Category ID: 70)
- SPAM URLs (Category ID: 71)
- Unconfirmed SPAM Sources (Category ID: 72)
- Open HTTP Proxies (Category ID: 73)
- Dynamic Content (Category ID: 74)
- Parked Sites (Category ID: 75)
- Alcohol and Tobacco (Category ID: 76)
- Private IP Addresses (Category ID: 77)
- Image and Video Search (Category ID: 78)
- Fashion and Beauty (Category ID: 79)
- Recreation and Hobbies (Category ID: 80)
- Motor Vehicles (Category ID: 81)
- Web Hosting (Category ID: 82)

no ssli webclassify url intercept <virtual_host_name> <category_name>

This command is used to delete an “**ssli webclassify url intercept**” configuration of the specified virtual host.

clear ssli webclassify url intercept <virtual_host_name>

This command is used to clear all “**ssli webclassify url intercept**” configurations of the specified virtual host.

show ssli webclassify settings <virtual_host_name>

This command is used to display the enabling status of the website classification function, the configurations of bypass or interception website categories, and the configuration of the “**ssli webclassify defaction**” command on the specified virtual host of the SSL interception module.

show statistics ssli webclassify <virtual_host_name>

This command is used to display the statistics related to the website classification function on the specified virtual host of the SSL interception module, including the hit counts of website categories and record information about local database, cache and online lookups as well as the matching statistics of the “**ssli webclassify defaction**” command.

clear statistics ssli webclassify

This command is used to clear the statistics related to the website classification function on the specified virtual host of the SSL interception module, including the hit counts of website categories and record information about local database, cache and online lookups as well as the matching statistics of the “**ssli webclassify defaction**” command.

Chapter 15 Secure Application Access (SAA)

AAA

aaa {on|off} <virtual_service>

This command is used to enable or disable the AAA function on the specified virtual service. When this function is enabled, end users will have to log in before gaining access to internal resources. When this function is disabled, end users will obtain authorized resources without being authenticated. By default, the AAA function is disabled on all virtual services.

virtual_service	This parameter specifies the name of an existing virtual service. Its value must be an HTTPS virtual service.
-----------------	---



Note: Virtual services enabled with the HTTP/2 protocol does not support the AAA function.

show aaa status [virtual_service]

This command is used to display the enabling status of the AAA function on the specified virtual service. If the “virtual_service” parameter is not specified, the enabling status of the AAA function on all virtual services will be displayed.

aaa server <type> <server_name> [display_string]

This command is used to define an AAA server of the specified type. The system supports SAML SP, RADIUS, LDAP and OAuth as AAA servers. A maximum of 32 AAA servers of each type can be configured.

type	This parameter specifies the AAA server’s type. Its value must be “samlsp”, “radius”, “ldap” or “oauth”.
server_name	This parameter specifies the AAA server’s name. Its value must be a string of 1 to 32 characters.
display_string	Optional. This parameter specifies the display string on the login page of the AAA server. Its value must be a string of 1 to 127 characters.

no aaa server <server_name>

This command is used to delete the specified AAA server.

server_name	This parameter specifies the name of an existing AAA
-------------	--

server.

show aaa server [type]

This command is used to display AAA servers of the specified type.

type Optional. This parameter specifies the AAA server's type. Its value must be "samlsp", "radius", "ldap", "oauth" or empty. If this parameter is not specified, all types of AAA servers will be displayed.

The default value is empty.

aaa method server <method_name> <authentication_server> [authorization_server] [display_string]

This command is used to create an AAA method with the specified authentication server (and authorization server).

method_name This parameter specifies the name of the AAA method. Its value must be a string of 1 to 32 characters, which is case insensitive. When the parameter value starts with a number, it must be enclosed in double quotes.

authentication_server This parameter specifies the authentication server. Its value must be the name of an existing AAA server. Three authentication servers can be set. They can be of the same type or of different types. When two or three authentication server are set, the server names should be separated with commas (",") and the entire parameter value should be enclosed in double quotes, such as "auth_server1, auth_server2, auth_server3".

authorization_server This parameter specifies the authorization server.

When the "authentication_server" parameter specifies only one authentication server. The value of this parameter can be the name of an existing AAA server, "none" or empty. The value "none" means the authorization step will be skipped. When no value is set, the authentication server will also be used as the authorization server. The default value is empty.

When the "authentication_server" specifies two or three authentication servers, the value of this parameter must be the name of an existing AAA server or "none".

display_string Optional. This parameter specifies the display string of the AAA method on the login page. If this parameter is not set,

its default value is the same as the value of the “method_name” parameter.

**Note:**

1. SAML SP servers cannot be used together with other types of AAA servers, such as LDAP, RADIUS and OAuth AAA servers.
2. After an authentication server is set using this command, if an authentication server that is different from the authentication server is set, they must use the same user name and password.

no aaa method server <method_name>

This command is used to delete the specified AAA method.

show aaa method server [method_name]

This command is used to display the specified AAA method configuration. If the “method_name” is not configured, all AAA method configurations will be displayed.

aaa method bind <method_name> <virtual_service>

This command is used to associate the specified AAA method with the specified virtual service. Each service can be associated with five AAA methods at most.

method_name This parameter specifies the name of an existing AAA method.

virtual_service This parameter specifies the name of an existing virtual service. Its value must be an HTTPS virtual service.

no aaa method bind <method_name> <virtual_service>

This command is used to cancel the specified AAA method’s association with the specified virtual service.

show aaa method bind [method_name]

This command is used to display the virtual services associated with the specified AAA method. If the “method_name” parameter is not specified, the virtual services associated with all AAA methods will be displayed.

clear aaa method bind [method_name]

This command is used to cancel the specified AAA method’s association with virtual services. If the “method_name” parameter is not specified, all AAA methods’ associations with virtual services will be cancelled.

aaa method rank {on|off} <virtual_service>

This command is used to enable or disable the AAA method rank function for the specified virtual service. After this function is enabled, if a virtual service is bound to multiple AAA methods, the system will rank these AAA methods based on the binding sequence, and the login page will display the AAA method that is bound to the virtual service first. If this function is disabled, all AAA methods will be displayed on the login page, and users can select one for authentication. By default, this function is disabled on all virtual services.

virtual_service	This parameter specifies the name of an existing virtual service. Its value must be an HTTPS virtual service.
-----------------	---

aaa role name <role_name> [description]

This command is used to add a user role. When the setting of “role_name” is an existing one, this command can also be used to update configurations of this user role. The system supports a maximum of 256 user roles.

role_name	This parameter specifies the name of the user role to be added. Its value must be a string of 1 to 127 characters.
description	Optional. This parameter specifies descriptions related to the user role. Its value must be a string of 1 to 255 characters.

The default value is empty.

no aaa role name <role_name>

This command is used to delete the specified user role.

show aaa role name [role_name]

This command is used to display the specified user role. If the “role_name” parameter is not specified, all user roles will be displayed.

clear aaa role name

This command is used to clear all user roles.

aaa role qualification <role_name> <qual_name> [description]

This command is used to add a qualification rule for the specified user role. A maximum of 32 qualification rules can be added for each user role.

role_name	This parameter specifies the name of an existing user role.
qual_name	This parameter specifies the qualification rule name. Its value must be a string of 1 to 127 characters.

description Optional. This parameter specifies descriptions of the qualification rule. Its value must be a string of 1 to 255 characters.

The default value is empty.

no aaa role qualification <role_name> <qual_name>

This command is used to delete a qualification rule configuration for the specified user role.

show aaa role qualification [role_name]

This command is used to display the qualification rule configurations for the specified user role. If the “role_name” parameter is not specified, the qualification rule configurations for all user roles will be displayed.

clear aaa role qualification

This command is used to clear the qualification rule configurations for all user roles.

aaa role condition <role_name> <qual_name> <condi_string>

This command is used to add a condition to the specified user role and qualification rule. If multiple conditions are configured for a qualification rule, end users can obtain the user role only when meeting all conditions in the associated qualification rule. Each qualification rule can have at most 32 conditions.

role_name	This parameter specifies the user role name.
qual_name	This parameter specifies the name of an existing qualification rule.
condi_string	This parameter specifies the condition string. Its value must be a string of 1 to 511 uppercase characters enclosed in double quotes. Its format must be “condition string [IS NOT] [value]”. For how to specify the “condition string” and “value”, please refer to the following table.

Example:

If a “stuff” user role is assigned to all users who log in on the first day of every month, and this user role is associated with a qualification rule “work”, you can execute the following command to add a condition to the “work” qualification rule:

```
AN(config)#aaa role condition stuff work “LOGINDAY IS 1”
```

The following table displays the supported condition strings:

Condition String	Meaning	Value
LOGINYEAR	The year when the end user logs in.	1970 to 2999.
LOGINMONTH	The month when the end user logs in.	1 to 12.
LOGINDAY	The day of month when the end user logs in.	1 to 31.
LOGINWEEK	The weekday when the end user logs in.	1 to 7.
LOGINDATE	The date when the end user logs in, including the year, month, day, hour and minute.	yyyyMMddhhmm
LOGINTIME	The time when the end user logs in.	00:00 to 23:59
USERNAME	The user name.	Alphanumeric, special printable ASCII characters and multi-byte characters.
GROUPNAME	The group which the user name belongs to.	Alphanumeric, special printable ASCII characters and multi-byte characters.
AUTHMETHOD	The authentication method.	Alphanumeric, special printable ASCII characters and multi-byte characters.
SRCIP	The source IP address of the end user.	IPv4 or IPv6 address. For example: 10.10.10.0/24 10.10.10.0/255.255.255.0 10.10.10.1 2012:1030::1 2012:1030::1/64

no aaa role condition *<role_name>* *<qual_name>*

This command is used to delete all condition configurations for the specified user role and qualification rule.

show aaa role condition *[role_name]*

This command is used to display all qualification rule and condition configurations for the specified user role. If the “role_name” parameter is not specified, the qualification rule and condition configurations for all user roles will be displayed.

aaa role bind *<role_name>* *<virtual_service>*

This command is used to associate the specified user role with the specified virtual service. One virtual service can be associated with 32 user roles at most.

role_name This parameter specifies the name of an existing user role.

virtual_service This parameter specifies the name of an existing virtual service. Its value must be an HTTPS virtual service.

no aaa role bind *<role_name>* *<virtual_service>*

This command is used to cancel the specified role’s association with the specified virtual service.

show aaa role bind *[role_name]*

This command is used to display the virtual service associated with the specified user role. If the “role_name” parameter is not specified, the virtual services associated with all user roles will be displayed.

clear aaa role bind [role_name]

This command is used to cancel the specified AAA role’s associations with virtual services. If the “role_name” parameter is not specified, all AAA roles’ associations with virtual services will be cancelled.

show aaa all

This command is used to display all configurations related to the AAA function, including SAML SP, LDAP, RADIUS and OAuth configurations.

clear aaa all

This command is used to clear all configurations related to the AAA function, and restore the enabling status of the AAA function and AAA method rank function to default settings.

SAML**aaa samlsp idp metadata <server_name> <url>**

This command is used to import the metadata file of the Identity Provider (IdP) into the specified SAML SP server. Note that if the metadata file of the IdP server is updated, administrators must execute this command again to import the updated metadata file. The new file will overwrite the original one.

server_name	This parameter specifies the name of an existing SAML SP server.
url	This parameter specifies the download address of the IdP server’s metadata file. Its value must be a string of 1 to 900 characters. HTTP, HTTPS and FTP URL addresses are supported.

no aaa samlsp idp metadata <server_name>

This command is used to delete the metadata file of the IdP server from the specified SAML SP server.

show aaa samlsp idp metadata <server_name>

This command is used to display the content of the IdP server’s metadata file on the specified SAML SP server.

show aaa samlsp sp metadata <server_name> [virtual_service]

This command is used to display the address from which to download the metadata file of the SAML SP server that is associated with the specified virtual service. If the “virtual_service” parameter is not specified, the addresses for downloading the metadata files of the SAML SP servers that are associated with all virtual services will be displayed.

aaa samlsp idp attributes <server_name> <username> [password]

This command is used to set the attribute name used to obtain user identities for the specified SAML SP server. The system now supports only the username attribute. When the IdP server returns a SAML response, the SAML SP server will get user identity information from the username attribute of the response.

The attribute setting will be saved in the metadata file of the SAML SP server. If it is updated, the SAML SP server’s metadata file on the IdP server must also be updated. Otherwise, the update will not take effect.

server_name	This parameter specifies the name of an existing SAML SP server.
username	This parameter specifies the username attribute. Its value must be a string of 1 to 900 characters. The special value “subject.nameid” is also supported, indicating the NameID field in the SAML response. When this command is used for the Web SSO function, the parameter value must be “uid”, indicating the username field.
password	This parameter specifies the password attribute. Its value must be a string of 1 to 900 characters. This parameter is used for the Web SSO function only.

aaa samlsp sp acs <server_name> [type]

This command is used to set the protocol binding types supported by the Assertion Consumer Service (ACS) of the specified SAML SP server. The IdP server will send SAML assertion responses to the ACS module of the SAML SP server using the configured protocol.

The binding setting will be saved in the metadata file of the SAML SP server. If it is updated, the SAML SP server’s metadata file on the IdP server must also be updated. Otherwise, the update will not take effect.

server_name	This parameter specifies the name of an existing SAML SP server.
type	Optional. This parameter specifies the protocol binding types supported by the ACS service. Its value must be: • POST: indicates the HTTP POST protocol binding type.

- **Artifact:** indicates the HTTP Artifact protocol binding type.

The default value is “post”.

aaa samlsp sp slo <server_name> [type]

This command is used to set the protocol binding types supported by the Single Logout (SLO) service of the SAML SP server. The IdP server will communicate with the SLO service of the SAML SP server using the configured protocol.

The binding configuration will be saved in the metadata file of the SAML SP server. If it is updated, the metadata file of the SAML SP server on the IdP server must also be updated. Otherwise, the update will not take effect.

server_name This parameter specifies the name of an existing SAML SP server.

type Optional. This parameter specifies the protocol binding types supported by the SLO server. Its value must be:

- **redirect:** indicates the HTTP Redirect protocol binding type.
- **post:** indicates the HTTP POST protocol binding type.
- **both:** indicates both of the protocol binding types above.

The default value is “both”.

show aaa samlsp settings [server_name]

This command is used to display all configurations for the specified SAML SP server, that is, the configuration of the following commands. If the “server_name” parameter is not specified, the configuration of the following commands on all SAML SP servers will be displayed.

- **aaa saml idp attributes**
- **aaa samlsp sp slo**
- **aaa samlsp sp acs**

clear aaa samlsp settings [server_name]

This command is used to clear all configurations for the specified SAML SP server, including the imported metadata file of the IdP server and the configuration of the following commands. If the “server_name” parameter is not specified, the imported metadata files and the configurations of the following commands on all SAML SP servers will be cleared.

- **aaa saml idp attributes**

- `aaa samlsp sp slo`
- `aaa samlsp sp acs`

RADIUS

aaa radius host *<radius_server_name>* *<ip>* *<authentication_port>* *<secret>* *<retries>* *<timeout>* [*index*]

This command is used to configure a RADIUS host for the specified RADIUS server. A maximum of three RADIUS hosts can be configured for one RADIUS server.

radius_server_name	This parameter specifies the name of an existing RADIUS server.
ip	This parameter specifies the IP address of the RADIUS host. The system supports only IPv4 RADIUS hosts.
authentication_port	This parameter specifies the port number used for RADIUS authentication. Its value must be an integer ranging from 1 to 65,535.
secret	This parameter specifies the shared secret used for encrypting passwords and exchanging responses between the APV appliance and the RADIUS server. Its value must be a string of 1 to 80 characters.
retries	This parameter specifies the maximum number of retry times for the APV appliance to connect the RADIUS server. Its value must be an integer ranging from 1 to 65,535.
timeout	This parameter specifies the timeout value, in seconds. Its value must be an integer ranging from 1 to 65535.
index	Optional. This parameter specifies the index number of the RADIUS host. Its value must be 1, 2 or 3. The default value is 1.

no aaa radius host *<radius_server_name>* *<index>*

This command is used to delete a RADIUS host configuration for the specified RADIUS server.

show aaa radius host [*radius_server_name*]

This command is used to display the RADIUS host configurations for the specified RADIUS server. If the “radius_server_name” parameter is not specified, the RADIUS host configurations for all RADIUS servers will be displayed.

aaa radius attribute group <radius_server_name> <attribute_id>

This command is used to specify an attribute used to obtain the external RADIUS group of an end user from the RADIUS entry for the specified RADIUS server. Please note that individual attributes may vary depending on the individual network requirements.

radius_server_name This parameter specifies the name of an existing RADIUS server.

attribute_id This parameter specifies the attribute ID used to obtain the external RADIUS group of an end user. The system supports the following attribute IDs:

- 1: User-Name
- 2: User-Password
- 3: CHAP-Password
- 4: NAS-IP-Address
- 5: NAS-Port
- 6: Service-Type
- 7: Framed-Protocol
- 8: Framed-IP-Address
- 9: Framed-IP-Netmask
- 10: Framed-Routing
- 11: Filter-Id
- 12: Framed-MTU
- 13: Framed-Compression
- 14: Login-IP-Host
- 15: Login-Service
- 16: Login-TCP-Port
- 17: (unassigned)
- 18: Reply-Message
- 19: Callback-Number
- 20: Callback-Id
- 21: (unassigned)
- 22: Framed-Route
- 23: Framed-IPX-Network
- 24: State
- 25: Class

- 26: Vendor Specific
- 27: Session Timeout
- 28: Idle-Timeout
- 29: Termination-Action
- 30: Called-Station-Id
- 31: Calling-Station-Id
- 32: NAS-Identifier
- 33: Proxy-State
- 34: Login-LAT-Service
- 35: Login-LAT-Node
- 36: Login-LAT-Group
- 37: Framed-AppleTalk-Link
- 38: Framed-AppleTalk-Network
- 39: Framed-AppleTalk-Zone
- 40:-59 (rev. for accounting)
- 60: CHAP-Challenge
- 61: NAS-Port-Type
- 62: Port-Limit
- 63: Login-LAT-Port

no aaa radius attribute group <radius_server_name>

This command is used to delete the “**aaa radius attribute group**” configuration of the specified RADIUS server.

show aaa radius attribute group [radius_server_name]

This command is used to display the “**aaa radius attribute group**” configuration for the specified RADIUS server. If the “radius_server_name” parameter is not specified, the “**aaa radius attribute group**” configurations for all RADIUS servers will be displayed.

aaa radius defaultgroup <radius_server_name> <group>

This command is used to configure the default group for the specified RADIUS server. The default group will be assigned to authenticated end users for whom no RADIUS group is obtained.

radius_server_name	This parameter specifies the name of an existing RADIUS server.
--------------------	---

group This parameter specifies the name of the default RADIUS group. Its value must be a string of 1 to 80 characters.

no aaa radius defaultgroup <radius_server_name>

This command is used to delete the “aaa radius defaultgroup” configuration for the specified RADIUS server.

show aaa radius defaultgroup [radius_server_name]

This command is used to display the “aaa radius defaultgroup” configuration for the specified RADIUS server. If the “radius_server_name” parameter is not specified, the “aaa radius defaultgroup” configurations for all RADIUS servers will be displayed.

aaa radius nasip <radius_server_name> <nasip>

This command is used to set or change the IP address of NAS (Network Access Server) for the specified RADIUS server. If this command is not configured, the system will select an available port IP address as the NAS IP address in the sequence of “port1, port2, port3...”.

radius_server_name This parameter specifies the name of an existing RADIUS server.

group This parameter specifies the IP address NAS.



Note:

The NAS IP address must be specified when only the bond or VLAN interface is configured with the IP address but no system interface is configured with the IP address.

no aaa radius nasip <radius_server_name>

This command is used to delete the NAS IP configuration for the specified RADIUS server.

show aaa radius nasip [radius_server_name]

This command is used to display the NAS IP configuration for the specified RADIUS server. If this command is not configured, the NAS IP configurations for all RADIUS servers will be displayed.

LDAP

aaa ldap host <ldap_server_name> <ip> <port> <username> <password> <base_dn> <timeout> [index] [tls_flag]

This command is used to configure an LDAP host for the specified LDAP server. A maximum of three LDAP hosts can be configured for one LDAP server.

ldap_server_name	This parameter specifies the name of an existing LDAP server.
ip	This parameter specifies the IP address of the LDAP host. The system supports only IPv4 LDAP hosts.
port	This parameter specifies the port number of the LDAP host. Its value must be an integer ranging from 1 to 65,535.
username	This parameter specifies the user name of the LDAP server administrator. Its value must be a string of 1 to 127 characters.
password	This parameter specifies the password of the LDAP server administrator.
base_dn	This parameter specifies the Distinguished Name (DN) of the LDAP entry at which to start the search for users. Its value must be a string of 1 to 900 characters.
timeout	This parameter specifies the timeout value of the search, in seconds. Its value must be an integer ranging from 1 to 65,535.
index	Optional. This parameter specifies the host index. Its value must be 1, 2 or 3. The default value is 1.
tls_flag	Optional. This parameter specifies whether to access the LDAP server over the TLS protocol. Its value must be: • “tls”: indicates that the LDAP server is accessed over the TLS protocol. • empty: indicates the LDAP server is not accessed over the TLS protocol. The default value is empty.

no aaa ldap host <ldap_server_name> <index>

This command is used to delete an LDAP host of the specified LDAP server.

show aaa ldap host [ldap_server_name]

This command is used to display the LDAP server host(s) configured for the specified LDAP server.

aaa ldap idletime <ldap_server_name> [idle_time]

This command is used to set the idle timeout value for the specified LDAP server. The connection to the LDAP server will be terminated when the connection is idle for the specified timeout value.

<code>ldap_server_name</code>	This parameter specifies the name of an existing LDAP server.
<code>idle_time</code>	Optional. This parameter specifies the timeout value of idle connections, in seconds. Its value must be an integer ranging from 60 to 3000. The default value is 600.

no aaa ldap idletime *<ldap_server_name>*

This command is used to delete the idle connection timeout setting for the specified LDAP server.

show aaa ldap idletime *[ldap_server_name]*

This command is used to display the idle connection timeout setting for the specified LDAP server. If the “`ldap_server_name`” parameter is not set, the idle connection timeout settings for all LDAP servers will be displayed.

aaa ldap searchfilter *<ldap_server_name> <filter_string>*

This command is used to configure a search filter for the specified LDAP server. The search filter plays an important role in authenticating and authorizing users through LDAP. For the functions of the search filter in static and dynamic binding, please refer to the commands “**aaa ldap bind dynamic**” and “**aaa ldap bind static**”.

<code>ldap_server_name</code>	This parameter specifies the name of an existing LDAP server.
<code>filter_string</code>	This parameter specifies a filter string used to search for the LDAP entries. Its value must be a string of 1 to 80 characters enclosed in double quotes. The filter string consists of: • attribute: Common Name (cn), Distinguished Name (dn), User Id (uid), Organization Unit (ou) and so on. • comparison operator: “>”, “<” or “=”. • logical operator: “& (and)”, “ (or)”, “! (not)”, “= (equal to)”, or “* (any)”. The filter string can contain at most three tokens represented by “<USER>”, which is case-insensitive.

no aaa ldap searchfilter <ldap_server_name>

This command is used to delete the search filter configured for the specified LDAP server.

show aaa ldap searchfilter [ldap_server_name]

This command is used to display the search filter configured for the specified LDAP server. If the “ldap_server_name” parameter is not specified, the search filters configured for all LDAP servers will be displayed.

aaa ldap attribute group <ldap_server_name> <attribute>

This command is used to specify the attribute used to obtain the external LDAP group of the user from the LDAP entry for the specified LDAP server.

ldap_server_name	This parameter specifies the name of an existing LDAP server.
attribute	This parameter specifies the name of the attribute used to obtain the external LDAP group of the user from the LDAP entry. Its value must be a string of 1 to 80 characters.

no aaa ldap attribute group<ldap_server_name>

This command is used to delete the “aaa ldap attribute group” configuration for the specified LDAP server.

show aaa ldap attribute group [ldap_server_name]

This command is used to display the “aaa ldap attribute group” configuration for the specified LDAP server. If the “ldap_server_name” parameter is not specified, the “aaa ldap attribute group” configurations for all LDAP servers will be displayed.

aaa ldap defaultgroup <ldap_server_name> <group>

This command is used to configure a default LDAP group for the specified LDAP server. For an authenticated user whose LDAP group cannot be obtained from the LDAP entries, the user will be assigned the default group.

ldap_server_name	This parameter specifies an existing name of the LDAP server.
group	This parameter specifies the default group name. Its value must be a string of 1 to 80 characters.

no aaa ldap defaultgroup <ldap_server_name>

This command is used to delete the default group configuration for the specified LDAP server.

show aaa ldap defaultgroup [ldap_server_name]

This command is used to display the default group configuration for the specified LDAP server. If the “ldap_server_name” parameter is not specified, the default group configurations for all LDAP servers will be displayed.

aaa ldap bind dynamic <ldap_server_name>

This command is used to enable the “dynamic” LDAP bind mode for the specified LDAP server. In this case, AAA will fetch the DN from the LDAP server first.

After the “dynamic” LDAP bind mode is enabled, AAA sends a bind request containing the end user’s username and password to the LDAP server and then a search request containing the search filter string configured by the command “**aaa ldap searchfilter**” to obtain the LDAP entry of the end user. Then AAA sends the DN obtained from the LDAP entry together with the password of the end user in another bind request to the LDAP server. After the end user passes the authentication, AAA reuses the obtained LDAP entry to authorize the end user.

ldap_server_name	This parameter specifies the name of an existing LDAP server.
------------------	---

no aaa ldap bind static <ldap_server_name>

This command is used to disable the “dynamic” LDAP bind mode for the specified LDAP server.

aaa ldap bind static <ldap_server_name> <dn_prefix> <dn_suffix>

This command is used to enable the “static” LDAP bind mode for the specified LDAP server. In this case, the system will construct the user’s DN by concatenating the strings “<dn_prefix><USER><dn_suffix>”. <USER> is the username used to log into the virtual site. “<dn_prefix>” and “<dn_suffix>” must be the same for all users using the same virtual site.

After the “static” LDAP bind mode is enabled, AAA sends the DN (<dn_prefix><USER><dn_suffix>) together with the password of the end user in a bind request to the LDAP server. After the end user passes the authentication, AAA sends a search request containing the search filter string configured by the command “**aaa ldap searchfilter**” to obtain the LDAP entry of this end user. Then, it authorizes the end user based on the obtained LDAP entry.

Note that the “static” and “dynamic” LDAP bind functions cannot be enabled at the same time.

ldap_server_name	This parameter specifies the name of an existing LDAP server.
------------------	---

dn_prefix	This parameter specifies the DN prefix extracted from the LDAP server. Its value must be a string of 1 to 80
-----------	--

characters.

dn_suffix This parameter specifies the DN suffix extracted from the LDAP server. Its value must be a string of 1 to 80 characters.

no aaa ldap bind static <ldap_server_name>

This command is used to disable the “static” LDAP bind mode for the specified LDAP server.

show aaa ldap bind [/ldap_server_name]

This command is used to display the configuration of the LDAP bind mode for the specified LDAP server. If the “ldap_server_name” parameter is not specified, the configurations of the LDAP bind mode for all LDAP servers will be displayed.

OAuth

aaa oauth vendor <server_name> <vendor_name>

This command is used to set the vendor name of the specified OAuth server. The OAuth vendors supported by the system include:

- Google
- WeChat

When the Google OAuth server (“goo_server” for example) is defined, the system automatically adds the following configurations:

- **aaa oauth tokenurl “goo_server”**
“https://accounts.google.com/o/oauth2/token”
- **aaa oauth authenticatorurl “goo_server”**
“https://accounts.google.com/o/oauth2/auth”
- **aaa oauth jwksurl “goo_server”**
“https://www.googleapis.com/oauth2/v3/certs”

When the WeChat OAuth server (“wech_server” for example) is defined, the system automatically adds the following configurations:

- **aaa oauth tokenurl “wech_server”**
“https://api.weixin.qq.com/sns/oauth2/access_token”
- **aaa oauth authenticatorurl “wech_server”**
“https://open.weixin.qq.com/connect/qrconnect”
- **aaa oauth resourceurl “wech_server”**
“https://api.weixin.qq.com/sns/userinfo”

server_name This parameter specifies the name of an existing OAuth

server.

vendor_name This parameter specifies the vendor name of the third-party OAuth server. Its value must be:

- google: indicates the Google OAuth server.
- wechat: indicates the WeChat OAuth server.

aaa oauth authenticatorurl <server_name> <authenticator_url>

This command is used to change the URL of the specified OAuth server's login page.

server_name This parameter specifies the name of an existing OAuth server.

authenticator_url This parameter specifies the URL of the third-party OAuth server's login page. Its value must be a string of 1 to 900 characters.

aaa oauth tokenurl <server_name> <token_url>

This command is used to set the URL from which to obtain an access token from the specified OAuth server.

server_name This parameter specifies the name of an existing OAuth server.

token_url This parameter specifies the URL where the OAuth client obtains the access token from the OAuth server. Its value must be a string of 1 to 900 characters.

aaa oauth jwksurl <server_name> <jwks_url>

This command is used to set the URL from which to obtain the JSON Web Key (JWK) set of the specified OAuth server. This command needs to be configured only for the Google OAuth server currently.

server_name This parameter specifies the name of an existing OAuth server.

jwks_url This parameter specifies the URL where to obtain the JWK set of the OAuth server. Its value must be a string of 1 to 900 characters.

aaa oauth registerid <server_name> <register_id>

This command is used to set or change the registered client ID for the OAuth client to communicate with the specified OAuth server.

server_name	This parameter specifies the name of an existing OAuth server.
register_id	This parameter specifies the registered client ID for the OAuth client. Its value must be a string of 1 to 128 characters.

aaa oauth registersecret <server_name> <register_secret>

This command is used to set or change the registered client secret for the OAuth client to communicate with the specified OAuth server.

server_name	This parameter specifies the name of an existing OAuth server.
register_secret	This parameter specifies the registered client secret for the OAuth client. Its value must be a string of 1 to 128 characters.

aaa oauth redirecturl <server_name> <redirect_url>

This command is used to set or change the URL to which the specified OAuth server will redirect responses.

server_name	This parameter specifies the name of an existing OAuth server.
redirect_url	This parameter specifies the URL to which the OAuth server will redirect responses. Its value must be a string of 1 to 900 characters. For the Google OAuth server, its value must be the same as the Redirection URL ("https://<virtual_service_domain_name>/prx/000/http/localhost/oauth_code") registered on Google's third-party developer platform. For the WeChat OAuth server, its value must be its value must be in the format of "https://<virtual_service_domain_name>/prx/000/http/localhost/oauth_wechat_code" and the virtual service domain name must have been registered on WeChat's developer platform.

aaa oauth resourceurl <server_name> <resource_url>

This command is used to set or change the URL from which the OAuth client obtains the user information from the specified OAuth server. This command needs to be configured only for the WeChat OAuth server currently. The Google OAuth server

will return the user information in the Access Token responses and therefore this configuration is not required.

server_name	This parameter specifies the name of an existing OAuth server.
resource_url	This parameter specifies the URL where the OAuth client obtains the user information from the resource server. Its value must be a string of 1 to 900 characters.

aaa method register <method_name>

This command is used to set the AAA method used for OAuth application registration.

method_name	This parameter specifies the name of an existing AAA method. Its value must be the name of a RADIUS or LDAP authentication method (the AAA method is used for authentication only). SAML SP and OAuth AAA methods cannot be used for OAuth application registration.
-------------	--

no aaa method register <method_name>

This command is used to delete the “aaa method register” configuration of the specified AAA method.

show aaa method register [method_name]

This command is used to display the “aaa method register” configuration of the specified AAA method. If the “method_name” is not specified, the “aaa method register” configurations of all AAA methods will be displayed.

aaa oauth registration <server_name> [save_password]

This command is used to enable or disable post-OAuth user registration for the specified OAuth server.

When this function is enabled, OAuth users are required to register to the system after passing the OAuth authentication. During the user registration, users need to authenticate themselves to the authentication server in the AAA method specified by the “**aaa method register**” command. After the user passes the authentication, the system will bind the obtained OAuth user IDs (UIDs) with the usernames used for registration. The usernames used for registration instead of the obtained OAuth usernames will be used for further authorization.

By default, post-OAuth user registration is enabled.

server_name	This parameter specifies the name of an existing OAuth server.
-------------	--

save_password Optional. This parameter specifies whether to save the user password. Its value must be:

- **save:** saves the user password.
- **nosave:** does not save the user password.

Note that when the Web SSO function is used, the parameter value must be “save”.

no aaa oauth registration <server_name>

This command is used to disable post-OAuth user registration for the specified OAuth server.

aaa oauth prefixasusername <server_name>

This command is used to enable the option of using an email account prefix as the OAuth username for the specified OAuth server. By default, this option is disabled.

no aaa oauth prefixasusername <server_name>

This command is used to disable the option of using an email account prefix as the OAuth username for the specified OAuth server.

aaa oauth authorizationfilter <server_name> <authorization_filter>

This command is used to configure or change the post-OAuth authorization filter for the specified OAuth server. The system will continue to perform authorization for an OAuth user after OAuth authentication only when the OAuth username (email account for the Google OAuth server or nickname for the WeChat OAuth server) matches the post-OAuth authorization filter. If the post-OAuth authorization filter is not configured, the system will continue to perform authorization for all users passing OAuth authentication.

server_name This parameter specifies the name of an existing OAuth server.

authorization_filter This parameter specifies the string to filter OAuth usernames. Its value must be a string of 1 to 64 characters.

no aaa oauth authorizationfilter <server_name>

This command is used to delete the post-OAuth authorization filter configured for the specified OAuth server.

aaa oauth bind import <url>

This command is used to import the OAuth user bind information from the specified URL address.

url This parameter specifies the URL address of the OAuth user bind information. Its value must be an HTTP or FTP URL address.

Note: The “path” of the FTP URL (ftp://user:password@host:port/path) should be a relative path.

aaa oauth bind export <url>

This command is used to export the OAuth user bind information to the specified URL address.

url This parameter specifies the URL address to store the OAuth user bind information. Its value must be an FTP URL address and contains the name of the file to be exported. For example, if file name is “oauth_bind”, the URL address should be set to “ftp://test:abc@10.4.21.112/oauth_bind”.

no aaa oauth bind <virtual_service> <oauth_uid>

This command is used to delete the bind information of an OAuth user from the specified virtual service.

virtual_service This parameter specifies the name of an existing virtual service.

oauth_uid This parameter specifies the OAuth user ID.

show aaa oauth bind [virtual_service] [oauth_uid]

This command is used to display the bind information of an OAuth user on the specified virtual service.

clear aaa oauth bind <virtual_service>

This command is used to clear all OAuth user bind information on the specified virtual service.

show aaa oauth settings

This command is used to display all configurations related to OAuth.

Session Management

aaa session virtual limit <virtual_service> <max_session_number>

This command is used to set the maximum number of concurrent sessions allowed on the specified virtual service. **Note:** When modifying this setting, make sure the new number is larger than the number of current existing sessions.

virtual_service	This parameter specifies the name of an existing virtual service.
max_session_number	This parameter specifies the maximum number of concurrent sessions. The value 0 indicates no limitation on the number of concurrent sessions on a virtual service.

The number of sessions supported by the system varies by the system memory, as shown in the following table (The actual numbers may be different depending on the memory usage):

System Memory	Maximum Number of Sessions
1 GB	500
2 GB	1000
4 GB	2000
8 GB	4000
16 GB	8000
32 GB	16,000
64 GB	32,000
128 GB	64,000

no aaa session virtual limit <virtual_service>

This command is used to delete the setting of the maximum number of concurrent sessions allowed on the specified virtual service.

show aaa session virtual limit [virtual_service]

This command is used to display the setting of the maximum number of concurrent sessions allowed on the specified virtual service. If the “virtual_service” parameter is not specified, the settings of the maximum number of concurrent sessions allowed on all virtual services will be displayed.

clear aaa session virtual limit

This command is used to clear the settings of the maximum number of concurrent sessions allowed on all virtual services.

aaa session virtual maxperuser <virtual_service> <maximum_session_number>

This command is used set the maximum number of concurrent sessions allowed per user on the specified virtual service. **Note:** The maximum number of sessions allowed per user cannot be larger than the total maximum number of concurrent sessions allowed on the virtual service.

virtual_service	This parameter specifies the name of an existing virtual service.
maximum_session	This parameter specifies the maximum number of concurrent sessions allowed per user. The value 0 indicates no limitation on the number of concurrent sessions per user.

The number of sessions supported by the system varies by the system memory, as shown in the following table (The actual numbers may be different depending the memory usage):

System Memory	Maximum Number of Sessions
1 GB	500
2 GB	1000
4 GB	2000
8 GB	4000
16 GB	8000
32 GB	16,000
64 GB	32,000
128 GB	64,000

no aaa session virtual maxperuser <virtual_service>

This command is used to delete the setting of the maximum number of concurrent sessions allowed per user on the specified virtual service.

show aaa session virtual maxperuser [virtual_service]

This command is used to display the setting of the maximum number of concurrent sessions allowed per user on the specified virtual service. If the “virtual_service” parameter is not specified, the settings of the maximum number of concurrent sessions allowed per user on all virtual services will be displayed.

clear aaa session virtual maxperuser

This command is used to clear the settings of the maximum number of concurrent sessions allowed per user on all virtual services.

aaa session virtual reuse {on|off} <virtual service>

This command is used to enable or disable the session reuse function for the specified virtual service. This function can be enabled only for the virtual service that is associated with a SAML SP server. By default, this function is disabled.

virtual_service	This parameter specifies the name of an existing virtual service.
-----------------	---

show aaa session virtual reuse [virtual_service]

This command is used to display the enabling status of the session reuse function on the specified virtual service. If the “virtual_service” parameter is not specified, the enabling status of the session reuse function on all virtual services will be displayed.

aaa session virtual timeout idle <virtual_service> <idle_time>

This command is used to set the timeout value of idle sessions on the specified virtual service. If this command is not configured, the default time value of idle sessions is 3600s.

virtual_service	This parameter specifies the name of an existing virtual service.
idle_timeout	This parameter specifies the timeout value of idle sessions. Its value must be an integer ranging from 1 to 86,400, in seconds.

no aaa session virtual timeout idle <virtual_service>

This command is used to restore the timeout setting for idle sessions on the specified virtual service to the default value.

show aaa session virtual timeout idle [virtual_service]

This command is used to display the timeout setting for idle sessions on the specified virtual service. If the “virtual_service” parameter is not specified, the timeout setting for idle sessions on all virtual services will be displayed.

clear aaa session virtual timeout idle

This command is used to restore the timeout settings for idle sessions on all virtual services to the default value.

aaa session virtual timeout lifetime <virtual_service> <lifetime>

This command is use to set the expiration time of sessions on the specified virtual service. If this command is not configured, the default expiration time of sessions is 86,400s. Note: The expiration time of sessions must be larger than the timeout value of idle sessions.

virtual_service	This parameter specifies the name of an existing virtual service.
lifetime	This parameter specifies the expiration time of sessions. Its value must be an integer ranging from 1 to 94,608,000, in seconds.

no aaa session virtual timeout lifetime <virtual_service>

This command is used to restore the session expiration setting on the specified virtual service to the default value.

show aaa session virtual timeout lifetime *[virtual_service]*

This command is used to display the session expiration setting on the specified virtual service. If the “virtual_service” parameter is not specified, the session expiration settings on all virtual services will be displayed.

clear aaa session virtual timeout lifetime

This command is used to restore the session expiration setting on the specified virtual service to the default value.

show aaa session virtual settings *[virtual_service]*

This command is used to display all configurations related to session management on the specified virtual service. If the “virtual_service” parameter is not specified, the session management configurations on all virtual services will be displayed.

clear aaa session virtual settings *[virtual_service]*

This command is used to clear all configurations related to session management on the specified virtual service. If the “virtual_service” parameter is not specified, the session management configurations on all virtual services will be cleared.

show aaa session active *[filter_string]*

This command is used to display active sessions that match the specified filter string.

filter_string Optional. This parameter specifies the filter string. Its value must be “all” or a string composed of one or more keywords in the format of “Keyword1=xxx, Keyword2=yyy, ...” enclosed in double quotes. The keywords are not subject to sequence.

When the value is “all” or when this parameter is not specified, all current active sessions will be displayed. When the value is a string, it supports the following keywords:

- id: indicates the session ID.
- user: indicates the username.
- vs: indicates the virtual service name.
- format: its value must be “count” or “data”, which indicates the number of sessions and detailed session information, respectively.

Example:

“id=123456, user = Jack, vs = v-web, format = data”

Example:

AN(config)#show aaa session active all				
User Name	Session ID	Age	Last Active	
_5ad71ec964956f67cf8f82538926c6a01d126786f7	4D5A56F8		00:11:47	00:11:47
_5611a9b3f629663fd5428219b039047f65e054a7c0	4D5A56F7		00:17:03	00:17:03
_ab167aa63b844b83f3ef97abf57165ba1951b7dd4f	4D5A56F6		00:01:44	00:01:40
_e1aab484dd1220b3e0ccf8bcc06031f3eb5381d480	4D5A56F9		415402:13:01	
415402:13:01				
_58041654444a23e8e92438c9f0c57fdffb3c19fa80	4D5A56FA		415402:13:01	
415402:13:01				

aaa session kill [filter_string]

This command is used to terminate the active sessions that match the specified filter string.

filter_string Optional. This parameter specifies the filter string. Its value must be “all” or a string composed of one or multiple keywords in the format of “Keyword1=xxx, Keyword2=yyy, ...” enclosed in double quotes. The keywords are not subject to sequence.

When the value is “all” or when this parameter is not specified, all current active sessions will be terminated. When the value is a string, it supports the following keywords:

- id: indicates the session ID.
- user: indicates the username.
- vs: indicates the virtual service name.

Example:

“id=123456, user = Jack, vs = v-web”

Web SSO

aaa sso {on|off} <virtual_service>

This command is used to enable or disable the Web SSO function for the specified virtual service. The system relies on SAML, LDAP, RADIUS and OAuth authentication to support web SSO. In application scenarios where web SSO is needed, the SAML, LDAP, RADIUS or OAuth authentication function must be deployed first. By default, this function is disabled.

<code>virtual_service</code>	This parameter specifies the name of an existing virtual service. Its value must be an HTTPS virtual service.
------------------------------	---



Note: When Web SSO is deployed and implemented based on SAML authentication, administrators must make sure the APV appliance can obtain user credentials from the IdP server. The AG series can function as an IdP server to cooperate with the APV appliance to support the Web SSO function. For more information, please contact Customer Support.

aaa sso post *<virtual_service> <hostname> <login_url> <username_field> <password_field> [post_host] [post_url] [other_post_field] [other_header_field]*

This command is used to configure an HTTP POST SSO rule for the specified virtual service. Using this command, administrators can specify the URL address of a login page, and the system will submit the credentials of end users to this address. End users can access multiple backend resources without re-entering the credentials. The system supports a maximum of 64 HTTP POST SSO rules.

<code>virtual_service</code>	This parameter specifies the name of an existing virtual service. Its value must be an HTTPS virtual service.
------------------------------	---

<code>hostname</code>	This parameter specifies the host name of the backend server. Its value must be a string of 1 to 128 characters.
-----------------------	--

<code>login_url</code>	This parameter specifies the URL address of the login page. Its value must be a string of 1 to 900 characters.
------------------------	--

<code>username_field</code>	This parameter specifies the field used to post the username for authentication. Its value must be a string of 1 to 64 characters.
-----------------------------	--

<code>password_field</code>	This parameter specifies the field used to post the password for authentication. Its value must be a string of 1 to 32 characters.
-----------------------------	--

<code>post_host</code>	Optional. This parameter specifies the target host name of the POST request, which can include the port number if needed. Its value must be a string of 1 to 128 characters, in the format of “hostname” or “hostname:port”. By default, the value of the “hostname” parameter is used.
------------------------	---

<code>post_url</code>	Optional. This parameter specifies the target URL address to which the POST request is directed. Its value must be a string of 1 to 900 characters. By default, the value of the “login_url” parameter is used.
-----------------------	---

<code>other_post_field</code>	Optional. This parameter specifies a set of fields that are required by the backend service in addition to the username and password. Its value must be a string of 1 to 1024
-------------------------------	---

characters. It can be a string of only characters or a string containing multiple “field=value” pairs. In addition, it supports tokens, which will be dynamically replaced by actual values.

Meanings of supported tokens are as follows:

- `<IP_ADDR_UINT>`: Client IP address in the unsigned integer format, such as 1677920266
- `<IP_ADDR_DOTDEC>`: Client IP address in the dotted decimal format, such as 10.8.3.100

For example:

“domain=abc&deptname=xyz&ipaddress=<IP_ADDR_DOTDEC>”

`other_header_field`

Optional. This parameter specifies a set of HTTP header fields that are required by the backend service for user authentication. Multiple HTTP header fields must be separated by “\r\n”. Its value should be a string of 1 to 1024 characters.

For example: “User-Agent: Mozilla/4.0 (compatible; MSIE 8.0; Windows NT 5.1; Trident/4.0; .NET CLR 2.0.50727; .NET CLR 3.0.4506.2152; .NET CLR 3.5.30729)\r\nCookie: PBack=0;\r\n”

no aaa sso post *<virtual_service>* *<hostname>* *<login_url>*

This command is used to delete an HTTP POST SSO rule for the specified virtual service.

show aaa sso [*virtual_service*]

This command is used to display all HTTP POST SSO rules configured for the specified virtual service. If the “virtual_service” parameter is not specified, the HTTP POST SSO rules configured for all virtual services will be displayed.

clear aaa sso [*virtual_service*]

This command is used to clear all HTTP POST SSO rules configured for the specified virtual service. If the “virtual_service” parameter is not specified, the HTTP POST SSO rules configured for all virtual services will be cleared.

Chapter 16 Webagent

Webagent Service

webagent service <webagent_service> <ip> <port> [max_connection]

This command is used to configure a Webagent service.

webagent_service	This parameter specifies the name of the real service. Its value must be a case-sensitive string of 1 to 40 characters and must be enclosed in double quotes when containing special characters. The parameter value cannot contain spaces or be the system reserved word “default”, “all”, or “global”, which is case-insensitive.
ip	This parameter specifies the IP address of the Webagent service. Its value must be an IPv4 or IPv6 address.
port	This parameter specifies the port number by which the real service listens to the client requests. Its value must be an integer ranging from 1 to 65,535.
max_connection	Optional. This parameter specifies the maximum number of concurrent connections allowed by the real service. Its value must be an integer ranging from 1 to 4,294,967,295. The default value is 0, indicating no limitation on the maximum number of concurrent connections.

The maximum number of Webagent services supported by the system depends on the system memory:

System Memory	Maximum Number of Webagent Services Supported
2G	500
4 GB/8 GB/16 GB/24 GB/32 GB/64 GB/96 GB/128 GB	4000

no webagent service <webagent_service>

This command is used to delete the specified Webagent service.

show webagent service [webagent_service]

This command is used to display the specified Webagent service. If the “webagent_service” parameter is not specified, all Webagent services are displayed.

clear webagent service

This command is used to clear all Webagent services.

show statistics webagent service [*webagent_service*]

This command is used to display the statistics of the specified Webagent service. If the “webagent_service” parameter is not specified, the statistics of all Webagent services are displayed.

webagent link <*webagent_service*> <*virtual_service*>

This command is used to create a link between the specified Webagent service and a virtual service. Multiple links can be created on a Webagent service.

webagent_service This parameter specifies the Webagent service name.

virtual_service This parameter specifies the virtual service name.

no webagent link <*webagent_service*> [*virtual_service*]

This command is used to delete the link between the specified Webagent service and a virtual service. If the “virtual_service” parameter is not specified, the links between the specified Webagent service and all virtual services are deleted.

show webagent link [*webagent_service*]

This command is used to display the links created on the specified Webagent service. If the “webagent_service” parameter is not specified, the links created on all Webagent services are displayed.

clear webagent link

This command is used to clear all Webagent links.

Webagent DNS

webagent dns cache {on|off}

This command is used to enable or disable the Webagent DNS cache function. This function is disabled by default.

webagent dns cache expire <*min_seconds*> <*max_seconds*>

This command is used to set the minimum and maximum expiration time for Webagent DNS caches.

- If the DNS response Time to Live (TTL) value is smaller than the minimum expiration time for Webagent DNS caches, the DNS cache will expire after the minimum DNS cache expiration time ends.
- If the DNS response TTL is larger than the maximum expiration time for Webagent DNS caches, the DNS cache will expire after the maximum DNS cache expiration time ends.

- If the DNS response TTL is larger than the minimum expiration time and smaller than the maximum expiration time for Webagent DNS caches, the DNS cache will expire after the TTL ends.

When this command is not configured, the default minimum and maximum expiration time for Webagent DNS caches are 60s and 3600s respectively.

min_seconds	This parameter specifies the minimum expiration time for Webagent DNS caches, in seconds. Its value must be an integer ranging from 0 to 999,999. If the value is set to 0, the minimum expiration time for Webagent DNS caches is 0.
max_seconds	This parameter specifies the maximum expiration time for Webagent DNS caches, in seconds. Its value must be an integer ranging from 0 to 999,999. If the value is set to 0, the maximum cache expiration time for Webagent DNS caches is 999,999. When the “max_seconds” parameter is set to a value other than 0, the value of “max_seconds” must be larger than the value of “min_seconds”.



Note: In the command **llb dns ttl**, if the value of the parameter **seconds** is zero, the cache record of Webagent DNS won't be displayed.

show webagent dns cache setting

This command is used to display the status of the Webagent DNS cache function and the minimum and maximum expiration time for Webagent DNS caches.

clear webagent dns all

This command is used to restore the default configuration of the Webagent DNS cache function and of the minimum and maximum expiration time for Webagent DNS caches.

clear webagent dns cache content

This command is used to clear all dynamic Webagent DNS resource records.

webagent dns host <host> <ip_address>

This command is used to add a static A or AAAA resource record for the specified DNS domain name. the system supports a maximum of 256 static resource records. Each domain can have at most eight static A resource records and eight static AAAA resource records.

host	This parameter specifies the DNS domain name. Its value must be a string of 1 to 255 characters in the format of “www.xyz.com”, and cannot end with “.”. Every section separated by “.” cannot be longer than 63 characters.
------	--

<code>ip_address</code>	This parameter specifies the IP address for the domain name. Its value must be an IPv4 or IPv6 address.
-------------------------	---

no webagent dns host <host> [*ip_address*]

This command is used to delete a static resource record for the specified domain name. If the “ip_address” parameter is not specified, all static resource records for the specified domain name are deleted.

show webagent dns host [*type*] [*host*]

This command is used to display the specified type of resource records for a domain name on Webagent DNS.

<code>type</code>	Optional. This parameter specifies the type of DNS resource records. Its value must be: • static: displays only static resource records. • dynamic: displays only dynamic resource records. • all: displays all resource records. The default value is “all”.
<code>host</code>	Optional. This parameter specifies the domain name on Webagent DNS.

clear webagent dns host

This command is used to clear all static Webagent DNS resource records.

Webagent Access Control

The following commands are used to control which services that clients can access via the Webagent service.

Webagent access control rules must be first added to the access group, and then can be applied to a Webagent service. The system supports a maximum of 1024 Webagent access control rules. If a request hits both an access-permitted rule and an access-denied rule, the former takes effect.

webagent acl permit http host <item_name> <host>

This command is used to create an access-permitted control rule. HTTP and HTTPS requests accessing the specified host name will be permitted. Each access-permitted control rule can contain only one host name.

<code>item_name</code>	This parameter specifies the name of the access-permitted control rule. Its value must be a case-sensitive string of 1 to 40 characters and must be enclosed in double quotes when containing special characters. The parameter value cannot
------------------------	--

contain spaces or be the system reserved word “default”, “all”, or “global”, which is case-insensitive.

host

This parameter specifies the host name to be added to the access control rule. Its value is case-insensitive. The parameter value supports both quick and full regular expressions. When the parameter value is set to a regular expression, it must be enclosed in double quotes. When the parameter value is a full regular expression, it must begin with the “<regex>” string. For information about the differences between the quick regular expression and the full regular expression, please refer to introductions about regular expressions in Chapter 1.

To ensure that a configured regular expression string is correct, administrators can use the “**slb regextest** <regex> <target_string>” command to test whether a target string will match the configured regular expression.

webagent acl deny http host <item_name> <host>

This command is used to create an access-denied control rule. HTTP and HTTPS requests accessing the specified host name will be denied. Each access-denied control rule can contain only one host name.

item_name

This parameter specifies the name of the access-denied control rule. Its value must be a case-sensitive string of 1 to 40 characters and must be enclosed in double quotes when containing special characters. The parameter value cannot contain spaces or be the system reserved word “default”, “all”, or “global”, which is case-insensitive.

host

This parameter specifies the host name to be added to the access control rule, which is case-insensitive. The parameter value supports both quick and full regular expressions. When the parameter value is set to a regular expression, it must be enclosed in double quotes. When the parameter value is a full regular expression, it must begin with the “<regex>” string. For information about the differences between the quick regular expression and the full regular expression, please refer to introductions about regular expressions in Chapter 1.

To ensure that a configured regular expression string is correct, administrators can use the “**slb regextest** <regex> <target_string>” command to test whether a target string will match the configured regular expression.

no webagent acl item <item_name>

This command is used to delete the specified access control rule.

show webagent acl item *[item_name]*

This command is used to display the host name contained in the specified access control rule. If the “item_name” parameter is not specified, the host names contained in all access control rules are displayed.

clear webagent acl item

This command is used to clear all access control rules.

webagent acl group name *<group_name>*

This command is used create an access group. A maximum of 1024 access groups can be created.

group_name	This parameter specifies the name of the access group. Its value must be a case-sensitive string of 1 to 40 characters and must be enclosed in double quotes when containing special characters. The parameter value cannot contain spaces or be the system reserved word “default”, “all”, or “global”, which is case-insensitive.
------------	---

no webagent acl group name *<group_name>*

This command is used to delete the specified access group.

clear webagent acl group name

This command is used to clear all access groups. It will also remove all access control rules from the access groups.

webagent acl group member *<group_name> <item_name>*

This command is used to add an access control rule to the specified access group.

group_name	This parameter specifies the access group name.
item_name	This parameter specifies the access control rule name.

no webagent acl group member *<group_name> <item_name>*

This command is used to remove an access control rule from the specified access group.

show webagent acl group member *[group_name]*

This command is used to display all access control rules contained in the specified access group. If the “group_name” parameter is not specified, the access control rules contained in all access groups are displayed.

clear webagent acl group member <group_name>

This command is used clear all access control rules from the specified access group.

webagent acl group apply <webagent_service> <group_name>

This command is used to associate a Webagent service with an access group. Each Webagent service can be associated with only one access group. Each access group can be associated with multiple Webagent services.

no webagent acl group apply <webagent_service>

This command is used to disassociate the specified Webagent service with the access group.

show webagent acl group apply [group_name]

This command is used to display the Webagent services associated with the specified access group. If the “group_name” parameter is not specified, the Webagent services associated with all access groups are displayed.

Chapter 17 Quality of Service (QoS)

This chapter covers the commands for configuring the QoS function.

QoS Queue

qos interface <interface_name> [direction] [bandwidth]

This command allows users to configure the QoS feature on the specified interface.

interface_name	Specify the name of the QoS interface. It may be system interface, VLAN interface or bond interface. Note: The QoS interface does not support MNET interface.
direction	IN or OUT, which specifies the input or output direction respectively. The default value is OUT.
bandwidth	The maximum bit rate allowed for all queues on the specified interface. The suffix can be b, KB, MB, or GB. The default value is 1 GB.

no qos interface <interface_name> [direction]

This command allows users to delete the QoS configuration on the specified interface. The “direction” parameter is optional, which specifies to delete the QoS configuration on the IN or OUT direction of the specified interface.

show qos interface [interface_name] [direction]

This command allows users to view configurations of QoS interfaces.

direction	Optional. If specified, the QoS statistics of this interface will be displayed.
direction	Optional. Specify to display the QoS statistics on the IN or OUT direction.

qos enable <interface_name> [direction]

This command allows users to enable QoS feature on the specified interface. The “direction” parameter is optional, which specifies to enable QoS feature on the IN or OUT direction of the specified interface.

qos disable <interface_name> [direction]

This command allows users to disable QoS feature on the specified interface. The “direction” parameter is optional, which specifies to disable QoS feature on the IN or OUT direction of the specified interface.

qos queue root <queue_name> <interface_name> [direction] [bandwidth] [priority] [borrow] [default]

This command allows users to create a QoS root queue on the specified interface.

queue_name	An assigned name, in the form of a character string, to the queue. Note: If the assigned name begins with a numeric character, then the string needs to be framed in double quotes.
interface_name	Specify the name of QoS interface. It may be system interface, VLAN interface or bond interface.
direction	IN or OUT. Optional and the default value is OUT.
bandwidth	Specify the maximum bit rate to be processed by the queue. Optional and the default value is 1 GB.
priority	Optional. The range is from 0 to 7, with 7 being the highest and 0 being the lowest. The default value is 1.
borrow	It can be BORROW or UNBORROW, to configure whether the specified queue can borrow bandwidth from the parent or not. Optional, and the default value is UNBORROW.
default	Specify a queue to be the default queue. The packets not matched by other queues are assigned to this one. Only one default queue is required. Optional, and the default value is NONDEFAULT

no qos queue root <queue_name>

This command allows users to delete a QoS root queue.

show qos queue root [queue_name]

This command allows users to view configurations of all QoS root queues. The “queue_name” parameter is optional, which specifies to display the configuration of the specified QoS root queue.

qos queue sub <queue_name> <parent_queue> [bandwidth] [priority] [borrow] [default]

This command allows users to create a QoS sub queue for a root queue on the specified interface.

queue_name	An assigned name, in the form of a character string, to the sub queue. Note: If the assigned name begins with a numeric character, then the string needs to be framed in
------------	--

	double quotes.
parent_queue	The parent queue of the sub queue being configured. It may be a root queue or a sub queue (sub-queues can also have their sub-queues).
bandwidth	Specify the maximum bit rate to be processed by the sub queue. Optional, and the default value is 1GB.
priority	Optional. The range is from 0 to 7, with 7 being the highest and 0 being the lowest. The default value is 1.
borrow	It can be BORROW or UNBORROW, to configure whether the specified sub queue can borrow bandwidth from the parent or not. Optional, and the default value is UNBORROW.
default	Specify the sub queue to be the default sub queue. The packets not matched by other sub queues are assigned to this one. Only one default sub queue is required. Optional, and the default value is NONDEFAULT.

no qos queue sub <queue_name>

This command allows users to delete a QoS sub queue.

show qos queue sub [queue_name]

This command allows users to view configurations of all QoS sub queues. If “queue_name” is supplied, this command will display configurations of the specified QoS sub queue.

show qos queue all

This command allows users to view all configurations of all QoS queues (including root queues and sub queues).

clear qos interface <interface_name> [direction]

This command allows users to clear configurations of the specified QoS interface. The “direction” parameter is optional, which specifies to clear configuration on the IN or OUT direction of the specified interface.

QoS Filter Rule

qos filter <filter_name> <queue_name> <src_addr> <smask> <sport> <dst_addr> <dmask> <dport> <proto> [priority]

This command is used to configure a QoS filter rule for packet classification. QoS filter rules determine packet classification rules defined statically. The system supports a maximum of 1024 QoS filter rules.

filter_name	This parameter specifies the name for a QoS filter rule. Its value must be a string of 1 to 31 characters. Numbers, letters and special characters are supported. When special characters are included, it must be enclosed in double quotes. It cannot be set to “all”.
queue_name	This parameter specifies the name for the queue to which the matched packets are forwarded. Its value must be a string of 1 to 31 characters. Numbers, letters and special characters are supported. When special characters are included, it must be enclosed in double quotes.
src_addr	This parameter specifies the source IP address of the packets. Its value must be an IPv4 address. When “src_addr” and “smask” are all set to “0.0.0.0”, all the source IP addresses will be matched.
smask	This parameter specifies the mask of the source IP address. Its value must be a dotted IP address.
sport	This parameter specifies the port number of the packets. Its value must be an integer ranging from 0 to 65,535. If the parameter is set to 0, all the port numbers will be matched. This parameter takes effect only when the “proto” parameter is set to TCP or UDP.
dst_addr	This parameter specifies the destination IP address of the packets. Its value must be an IPv4 address. When “dst_addr” and “dmask” are all set to “0.0.0.0”, all the destination IP addresses will be matched.
dmask	This parameter specifies the mask of the destination IP address. Its value must be a dotted IP address.
dport	This parameter specifies the port number of the packets. Its value must be an integer ranging from 0 to 65,535. If the parameter is set to 0, all the port numbers will be matched. This parameter takes effect only when the “proto” parameter is set to TCP or UDP.
proto	This parameter specifies the protocol type of the packets. Its value must be: • TCP: indicates TCP packets. • UDP: indicates UDP packets. • any: indicates packets of all protocols.

priority Optional. This parameter specifies the priority of the QoS filter rule. Its value must be an integer ranging from 1 to 255. The larger the value, the higher the priority.

The default value is 1.

no qos filter *<filter_name>*

This command is used to delete a Layer 4 QoS filter rule.

show qos filter *[filter_name]*

This command is used to view configurations of all QoS filter rules. If “filter_name” is supplied, the configuration of the specified QoS filter rule will be displayed.

Other QoS Commands

show qos all

This command allows users to view all QoS configurations.

clear qos all

This command allows users to clear all QoS configurations.

show statistics qos *[interface_name] [direction]*

This command allows users to view QoS statistics.

interface_name Optional. If specified, the QoS statistics of this interface will be displayed.

direction Optional. Specify to display the QoS statistics on the IN or OUT direction.

clear statistics qos *[interface_name] [direction]*

This command allows users to clear QoS statistics.

interface_name Optional. If specified, the QoS statistics of this interface will be displayed.

direction Optional. Specify to display the QoS statistics on the IN or OUT direction.

Chapter 18 Link Load Balancing (LLB)

For users who access the Internet through multiple links, it is necessary to load balance the traffic passing back and forth. LLB is a function designed to balance the load of inbound and outbound traffic on the APV appliance. This chapter introduces related LLB commands.

LLB Route

The LLB function selects the link to forward the packets mainly based on the Eroute. Multiple routes are supported on the APV appliance. The system matches the route for the packets based on the priority of the route. Please see the following table for the priority of each type of route.

Route Name	Priority
Preferential Eroute (Eroute-P)	2001-2999
Interface Route (Iroute)	2000
Direct Route (Droute)	2000
RTS	1999
IP region route	1001-1999 (Default: 1999)
Normal Eroute (Eroute-N)	1001-1999
IPflow	1000-1999 (Default: 1000)
Static Route	101-132 (IPv4) 101-228 (IPv6)
Dynamic Route	101-132 (IPv4) 101-228 (IPv6)
LLB Link Route	2
Default Route	1

Interface Route (Iroute)

After the administrator configures the interface IP address, the system will automatically generate a route from any IP/port pair to the network segment of this interface. This route is called Interface Route (Iroute).

Direct Route (Droute)

The route generated through ARP study after the interface IP address is configured is called Direct Route (Droute). The Droute will expire 1 hour after it is generated.

Return to Source (RTS)

After the Return to Source (RTS) route function is enabled, when a packet is forwarded using the route selected from two or more routes with the same priority, the system performs the reverse route lookup to find the original link through which the request reaches the device and generates an RTS route for this link. The RTS route makes sure that the response will be returned using the same link.

IP region

The system will automatically generate an IP region route after the IP region table is bound to the gateway. The IP region route will be hit if the destination IP address of a packet belongs to a subnet in the IP region table.

Eroute

Eroute allows the system to select a route for the outbound traffic based on the packet quintuple (source IP, destination IP, source port, destination port and protocol type). The Eroute with a priority in the range of 2001 to 2009 is called Preferential Eroute (Eroute-P) and the Eroute with a priority in the range of 1001 to 1999 is called Normal Eroute (Eroute-N).

IPflow

After the IPflow routing function is enabled, when a route is selected to forward packets, the system will automatically generate an IPflow route based on the source IP address. Subsequent packets from the same source IP address will always be forwarded using this IPflow route.

Static Route

The static route must be configured manually. The priority of the static route is equal to the netmask or the prefix length plus 100. For example, if the netmask of the static route is 24, the priority is 124.

Dynamic Route

The system will automatically generate the dynamic route using the dynamic routing protocol, such as OSPF and BGP.

LLB Link Route

LLB link route is the route generated using the “**llb link route**” command.

Default Route

The default route must be configured manually. The packets will be forwarded using the default route only when no available route is matched.



Note: For the RTS route, IPflow route, IP region route and the Eroute-N with the same priority, the matching order is the RTS route, IPflow route, IP region route and Eroute-N.

llb link route <link_name> <route_ip> [weight] [hc_srcip]
[bandwidth_threshold]

This command is used to define an LLB link, or modify the bandwidth threshold for an existing LLB link. After this command is executed, the system will automatically generate an Eroute with the priority of 2 for the new LLB link. The system supports a maximum of 128 LLB links, including both IPv4 and IPv6 LLB links.

This command is also used to modify the weight value and bandwidth threshold for an existing LLB link.

link_name	This parameter specifies an LLB link name. Its value must be a case-sensitive string of 1 to 31 characters. Numbers, letters and special characters are supported. When special characters are included, it must be enclosed in double quotes.
route_ip	This parameter specifies the gateway IP address of this LLB link. It must be an IPv4 or IPv6 address.
weight	Optional. This parameter specifies the weight value of the LLB link, which is used by the weighted round robin or hash ip method to select a route. Its value must be an integer ranging from 1 to 1,048,576. The larger the weight, the higher the possibility that the LLB link will be selected. The default value is 1.
hc_srcip	Optional. This parameter specifies the source IP address of the health check. Its value must be an IPv4 or IPv6 address. For an IPv4 link, the default value is “0.0.0.0”. For an IPv6 link, the default value is “::”. If the default value is specified, the system will automatically select the interface IP address that is in the same subnet as the gateway as the source IP address.
bandwidth_threshold	Optional. This parameter specifies the maximum bandwidth allowed for this link, in Mbps or Gbps. This value is not a strict limit. When the traffic load exceeds the specified bandwidth threshold, the packets will not be dropped but forwarded using other links; if no available link is found, the packets will still be forwarded using this link. The default value is 0 Mbps, which means no bandwidth limitation.

Example:

```
AN(config)#llb link route "link1" 10.191.2.100 1 10.191.2.105 10Mbps
```

To change the bandwidth threshold for this LLB link to 20 Mbps, execute the following command:

```
AN(config)#llb link route "link1" 10.191.2.100 1 10.191.2.105 20Mbps
```



Note: The configuration synchronization will fail if the “hc_srcip” parameter is set to a non-default value and the parameter value is not equal to any IP address on the peer. To avoid the configuration synchronization failure, it is

recommended to set the “hc_srcip” parameter to the default value.

no llb link route <link_name>

This command is used to delete the configurations of the specified LLB link.

show llb link route [link_name]

This command is used to display the configurations of the specified LLB link. If the “link_name” parameter is not specified, the configurations of all LLB links will be displayed.

clear llb link route

This command is used to clear the configurations of all LLB links.

llb link {enable|disable} <link_name>

This command is used to enable or disable the specified LLB link. When an LLB link is disabled, the packet will be forwarded using other available LLB links. If no available link can be found, the packet on this LLB link will be discarded. By default, it is enabled.

**Note:**

Network traffic cannot flow in through a disabled link except in the following conditions:

- If the incoming traffic is not resolved by the LLB DNS, it might flow in through a disabled link;
- If the RTS route function is enabled and the incoming traffic flows in through a disabled link, the returned traffic will also go through the disabled link since the priority of the RTS route is higher than LLB link route.

llb link bw_priority <priority>

This command is used to set a bandwidth priority for the outbound LLB link. If this command is not configured, the default bandwidth priority of the outbound LLB link is 1800.

priority

This parameter specifies the bandwidth priority for the outbound LLB link. Its value must be an integer ranging from 0 to 1999. A larger value indicates a higher priority.



Note: If the priority of the Eroute matched with the outbound traffic is higher than the bandwidth priority for the outbound LLB link, the LLB link specified by the Eroute is not impacted by the bandwidth threshold; otherwise it will be impacted by the bandwidth threshold.

show llb link bw_priority

This command is used to display the priority of the bandwidth configured for the outbound LLB link.

ip eroute <name> <priority> {srcip|src_ip_group} {srcmask|prefix} <srcport> {dsthost|dst_ip_group} {dstmask|prefix} <dstport> <protocol> <gateway_ip> [weight]

This command is used to define an Eroute. Eroute allows the system to select a route for the outbound traffic based on the packet quintuple (source IP/source IP group, destination IP/destination IP group, source port, destination port and protocol type). The Eroute with a priority in the range of 2001 to 2009 is called Preferential Eroute (Eroute-P) and the Eroute with a priority in the range of 1001 to 1999 is called Normal Eroute (Eroute-N).

name This parameter specifies the Eroute name. Its value must be a case-sensitive string of 1 to 103 characters.

Note: The parameter value cannot be empty.

priority This parameter specifies the priority of the Eroute. Its value must be an integer ranging from 1001 to 2999 with 2000 excluded.

srcip|src_ip_group This parameter specifies an IP address or IP address group of the source subnet. Its value must be an IPv4 or IPv6 address.

srcmask|prefix This parameter specifies the netmask or prefix length of the source IP address.

- “srcmask” is used for an IPv4 address. It must be a dotted IP address or an integer ranging from 0 to 32.
- “prefix” is used for an IPv6 address. It must be an integer ranging from 0 to 128.

When the parameter “srcip|src_ip_group” is specified as the source IP address group, this parameter is invalid.

srcport This parameter specifies the source port number or port range from 1 to 65,535. The port range must be in the format of “X-Y”, for example, “1000-2000” indicates the source port ranging from 1000 to 2000. Port 0 is a wildcard, which is ignored unless the protocol is TCP or UDP.

dsthost|dst_ip_group This parameter specifies a destination domain name, IP address or IP address group. When the parameter value is an IP address or IP address group, both IPv4 and IPv6 are supported.

When the parameter value is a domain name, it supports

quick regular expression, which must be enclosed in double quotes. For more information, please refer to introductions about regular expressions in Chapter 1.

Note: The regular expression string must contain “.”. Neither “.” nor “-” can appear at the beginning or end of the string. The maximum length of the regular expression string is 254 characters, and the maximum length of each section separated by “.” is 63 characters.

When the parameter value is a domain name, the required resource type is indicated by the value of the “gateway_ip” parameter.

- If the IPv4 resource is required, the “gateway_ip” parameter value must be an IPv4 address.
- If the IPv6 resource is required, the “gateway_ip” parameter value must be an IPv6 address.

dstmask|prefix This parameter specifies the netmask or prefix length of the destination host. For details, see the description for the “srcmask|prefix” parameter.

When the parameter “dsthost|dst_ip_group” is specified as the destination IP address group, this parameter is invalid.

dstport This parameter specifies the destination port number or port range. For details, see the description for the “srcport” parameter.

protocol This parameter specifies the matched protocol type. Its value must be “TCP”, “UDP” or “any”. If the parameter value is “any”, it indicates that packets of all protocols can match this Eroute.

gateway_ip This parameter specifies the gateway IP address. Its value must be an IPv4 or IPv6 address.

weight Optional. This parameter specifies the weight value of this Eroute for the weighted round robin and hash ip method. Its value must be an integer ranging from 1 to 1,048,576. The default value is 1.



Note:

- An Eroute cannot be added if it has the same source subnet, destination subnet, source port range, destination port range or protocol type with that of the existing Eroute.
- For information about the match order of the Eroute-N and other routes with the same priority, please see the descriptions in the beginning of this

chapter.

- One IP address group can associate with multiple Eroutes.
- When the value of the “dsthost” parameter is destination domain name, at most 512 Eroutes can be configured.

The maximum number of Eroute entries allowed on the APV appliance varies with system memories. Please see the following table for details.

System Memory	Maximum Eroute Entries
≤4 GB	5,000
8 GB	10,000
16 GB	20,000
24 GB	20,000
32 GB	40,000
64 GB	40,000
96 GB	80,000
128 GB	80,000

no ip eroute <name>

This command is used to delete the specified Eroute.

show ip eroute [ipv4|ipv6] [all|valid|invalid]

This command is used to display the routes in the Eroute table, including Eroutes, LLB link routes, IP region routes, Iroutes, the default route, the static route, dynamic routes and Droutes.

ipv4|ipv6

Optional. This parameter specifies the type of the Eroute to be displayed in the output results. Its value must be “IPv4” or “IPv6”, which is case-insensitive. If this parameter is not specified, all IPv4 and IPv6 Eroute configurations will be displayed.

all|valid|invalid

Optional. The parameter value must be:

- all: displays all configurations of Eroutes generated by the IP region routes.
- valid: displays available Eroutes. For Eroute generated via domain name resolution, TTL will be displayed.
- invalid: displays unavailable Eroutes and the reason for the unavailability.



Note: The RTS and IPflow routes are kept in the RTS route table and IPflow route table respectively instead of the Eroute table.

clear ip eroute

This command is used to clear the Eroute table and the statistics of Eroutes, Droutes, IPflow routes and LLB link routes.

ipregion route <ipregion_name> <gateway_ip> [priority] [weight] [start_port] [end_port]

This command is used to create an IP region route for the specified IP region table. After this command is configured, the system will automatically generate the corresponding Eroute for every IP network segment in the IP region table. The maximum number of IP region routes supported on the APV appliance is related to the maximum number of Eroutes supported on the APV appliance. For details, please see descriptions for the “**ip eroute**” command.

ipregion_name	This parameter specifies the name of the IP region table. Its value must be a string of 1 to 24 characters. The parameter value must be the name defined by the command “ipregion table import <ipregion_name> <url>” or the name of the predefined IP region table. The predefined IP region tables follow the naming convention: predefined_<country code>_<v4 or v6>. For example, the IPv4 region table for the United States is predefined_US_v4. The administrator can execute the “show ipregion name” command to view the existing IP region tables.
gateway_ip	This parameter specifies the gateway IP address. Its value must be an IPv4 or IPv6 address.
priority	Optional. This parameter specifies the priority of this route. Its value must be an integer ranging from 1001 to 1999. The default value is 1999.
weight	Optional. This parameter specifies the weight value of the IP region route for the weighted round robin and hash ip method. Its value must be an integer ranging from 1 to 1,048,576. The default value is 1.
start_port	Optional. This parameter specifies the start port number. Its value must be an integer ranging from 1 to 65,535. The default value is 1.
end_port	Optional. This parameter specifies the end port number. Its value must be an integer ranging from 1 to 65,535. The default value is 65,535.



Note: For information about the matching order of the IPflow route and other routes with the same priority, please see the descriptions in the beginning of this chapter.

no ipregion route <ipregion_name>

This command is used to delete the IP region route created for the specified IP region table. The Eroutes generated based on this IP region route will also be deleted.



Note: The Eroutes generated based on this IP region route exist as a whole in the system. Administrators cannot change or remove a single Eroute.

show ipregion route

This command is used to display all IP region routes.

clear ipregion route

This command is used to clear all IP region routes.

show ipregion match <ip_address>

This command is used to find the IP range containing the specified IP address in the configured IP region table, and display all IP region tables, IP ranges and related descriptions where the matched IP address locates.

ip_address This parameter specifies the IP address to be matched.
Both IPv4 and IPv6 addresses are supported.

ip ipflow {on|off}

This command is used to enable or disable the IPflow routing function. When a route is selected to forward packets, the system will generate an IPflow route based on the source and destination IP addresses and the gateway. Subsequent packets with the same source and destination IP addresses are forwarded using this IPflow route. By default, the IPflow routing function is disabled.

The maximum number of IPflow entries supported on the APV appliance varies with system memories. Please see the following table for details.

System Memory	Maximum IPflow Entries
1 GB	10,000
2 GB	20,000
4 GB	1,000,000
8 GB	2,400,000
16 GB	5,000,000
≥32 GB	10,000,000

ip ipflow priority <priority>

This command is used to set the priority for the IPflow route. The default priority of the IPflow route is 1000.

priority This parameter specifies the priority of the IPflow route.
Its value must be an integer ranging from 1000 to 1999.

**Note:**

- Because the priority of the Eroute is higher than the default priority of the IPflow route, it is recommended to set the priority of the IPflow route greater than 1000 to ensure it can take effect.
- The new IPflow route will overwrite the oldest one if IPflow routes reach the maximum number.
- For information about the matching order of the IPflow route and other routes with the same priority, please see the descriptions in the beginning of this chapter.

ip ipflow expire <timeout>

This command is used to set the timeout value for the IPflow route. The default timeout value for the IPflow route is 60 seconds. If an IPflow route is hit, this IPflow route will be retimed. If an IPflow route is not hit in the specified time, this IPflow route will be deleted. When the system attempts to create an IPflow route, it will first check whether the same IPflow route already exists. If yes, the system will update the existing one instead of creating one.

timeout	This parameter specifies the timeout value for the IPflow route, in seconds. Its value must be an integer ranging from 1 to 86,400.
---------	---

show ip ipflow

This command is used to display the configurations of the IPflow routing function.

clear ip ipflow

This command is used to reset the configurations of the IPflow routing function to defaults.

ip rts on [all|gateway]

This command is used to enable the RTS routing function. When a packet is forwarded using the route selected from two or more routes with the same priority, the system performs the reverse route lookup to find the original link through which the request reaches the device and generates an RTS route for this link. Then the response will be returned using this RTS route.

all	Optional. This parameter specifies the mode of the RTS routing function. Its value must be: • all: indicates that the “all” mode is applied to the system. The system records information about all external senders. All responses will be sent back along the routes used to forward the corresponding requests. • gateway: indicates that the “gateway” mode is applied to the system. The system records information only
-----	---

about the gateways. Only responses to these gateways will be sent back along the routes used to forward the corresponding requests.

The default value is “all”.

Note: If the parameter value is specified as “gateway”, the “gateway” recorded by the system should be the gateway configured by the commands “**ip route default**”, “**ip eroute**” or “**ip route static**”.



Note:

- To ensure that the RTS routing function can work properly, the Eroute configured by the “**ip eroute**” command must be a Normal Eroute (with a priority in the range of 1001 to 1999).
- To assure that the RTS routing function can work properly, please configure the basic route information before the RTS routing function is enabled.
- The new RTS route will overwrite the oldest one if RTS routes reach the maximum number.
- For information about the matching order of the RTS route and other routes with the same priority, please see the descriptions in the beginning of this chapter.

The maximum number of RTS entries allowed on the APV appliance varies with system memories. Please see the following table for details.

System Memory	Maximum RTS Entries
1 GB	10,000
2 GB	20,000
4 GB	40,000
8 GB	60,000
16 GB	150,000
32 GB	250,000
>32 GB	400,000

ip rts off

This command is used to disable the RTS route function. By default, the RTS route function is disabled.

ip rts expire [seconds]

This command is used to specify the timeout value for the RTS route. If an RTS route is hit, this RTS route will be retimed; if an RTS route is not hit in the specified time, this RTS route will be deleted. When the system attempts to create an RTS route, it will first check whether the same RTS route already exists. If yes, the system will update the existing one instead of creating one.

seconds Optional. This parameter specifies the timeout value for the RTS route, in seconds. Its value must be an integer ranging from 1 to 2,147,483. The default value is 60.

show ip rts

This command is used to display the configurations of the RTS routing function.

clear ip rts

This command is used to reset the configurations of the RTS routing function to defaults.

clear droute

This command is used to clear all Droutes.

Outbound Link Load Balancing

llb method outbound {rr|wrr|sr}

This command is used to define the following methods for outbound LLB:

- Round Robin (rr)
- Weighted Round Robin (wrr)
- Shortest Response (sr)



Note:

- The “sr” method cannot be used to load balance IP fragments, non-TCP, non-UDP packets, and reassembled UDP packets.
- The “sr” method cannot be used together with the DirectFWD function.

llb method outbound hi [*hash_type*]

This command is used to define the Hash IP (hi) method for outbound LLB. After the hi method is configured, the system will calculate the hash value of the IP address to achieve outbound LLB.

hash_type Optional, this parameter specifies the IP address type selected by the hi method for hash calculation. Its value must be:

- src: indicates the source IP address.
- dst: indicates the destination IP address.

The default value is “src”.



Note: Before configuring the “hi” method, the weight of each route needs to be specified by using the command “**llb link route**” or “**ip eroute**”.

llb method outbound hip *[hash_type]*

This command is used to define the Hash IP+Port (hip) method for outbound LLB. After the hip method is configured, the system will calculate the hash value of the IP address and port to achieve outbound LLB.

hash_type Optional, this parameter specifies the IP address and port type selected by the hip method for hash calculation. Its value must be:

- **src**: indicates the source IP address and port.
- **dst**: indicates the destination IP address and port.

The default value is “src”.

llb method outbound lb

This command is used to define the Least Bandwidth (lb) method for outbound LLB. After the lb method is configured, the system will select the link with the smallest bandwidth usage to forward outbound traffic.

To use this method, administrators need to specify the maximum bandwidth allowed for each link through the “**llb link route**” command. If a link has no bandwidth limit configured, the lb method will not select this link. If the bandwidth limit is configured in none of the links, the wrr method will be used instead.

llb method outbound lc

This command is used to define the Least Connections (lc) method for outbound LLB. After the lc method is configured, the system will select the outbound link with the least number of connections to forward outbound traffic.

The lc method is only recommended when all LLB links are configured with NAT and all LLB outbound traffic can be NATted. If all LLB links are not configured with NAT, the lc method actually functions as the wrr method. It is not recommended to use this method in other situations.

llb method outbound dd *[netmask] [prefix] [detecting_mode]*

This command is used to define the Dynamic Detecting (dd) method for outbound LLB. After the dd method is configured, the system will send DD packets to the destination of the traffic through different LLB links and selects the link with the shortest response time based on the DD results. The system supports detecting the TCP, UDP and ICMP packets.

netmask	Optional. This parameter specifies the netmask of the destination IPv4 address. Its value must be an integer ranging from 1 to 32. The default value is 24.
prefix	Optional. This parameter specifies the prefix length of the destination IPv6 address. Its value must be an integer ranging from 1 to 128. The default value is 64.
detecting_mode	Optional. This parameter specifies the detecting mode of the dd method. Its value must be: - tcp: indicates that the system will perform dynamic detecting on the TCP traffic only. - all: indicates that the system will perform dynamic detecting on the TCP, UDP and ICMP traffic. The default value is “tcp”.



Note: Please clear the information in the DD table using the “**clear llb dd table**” command first before changing the detecting mode of the dd method.

show llb method

This command is used to display the configurations for inbound and outbound balancing methods.

llb dd static <network_ip> {netmask|prefix}

This command is used to configure static detecting objects for the dd method to find the link with the shortest response time. If the parameters “network_ip” and “netmask|prefix” identify a single IP address, the system will perform DD detection on this IP address when it receives a packet whose destination IP address belongs to the same network segment of the specified IP address. If the two parameters identify a network segment, the system will perform DD detection on the network segment when it receives a packet whose destination IP belongs to this network segment. The system supports a maximum of 1024 static detecting objects.

network_ip	This parameter specifies the IP address or a network segment for the static detecting object. Its value must be an IPv4 or IPv6 address.
netmask prefix	This parameter specifies the netmask or prefix of the IP address. - The value “netmask” is used for an IPv4 address. It must be a dotted IP address or an integer ranging from 0 to 32. - The value “prefix” is used for an IPv6 address. It must be an integer ranging from 0 to 128.

no llb dd static <network_ip> {netmask|prefix}

This command is used to delete the specified static detecting object.

show llb dd static

This command is used to display all static detecting objects.

clear llb dd static

This command is used to clear all detecting static objects.

llb link health remote {enable| disable}

This command is used to enable or disable the DD-based health check function. Before using this function, the administrator must enable the dd method. If the DD-based health check function is disabled and no link is available, the packet will be forwarded based on the wrr or rr method; if the DD-based health check function is enabled and no available LLB link to the destination is detected (response timeout), the packet will be discarded. By default, this function is disabled.

show llb dd table [destination_ip] [netmask|prefix]

This command is used to display the detailed information of the DD table. Administrators can also view the information about the specified destination IP network segment. If no IP network segment is specified, all dynamic detecting information will be displayed.

destination_ip	Optional. This parameter specifies the destination IP address in the DD table. Its value must be an IPv4 or IPv6 address. If it is set to "0.0.0.0", all IPv4 dynamic detecting results will be displayed; if it is set to ":::", all IPv6 dynamic detecting results will be displayed. If no destination IP address is specified, all dynamic detecting results will be displayed.
netmask prefix	Optional. This parameter specifies the netmask or prefix length of the destination IP address. The default value is 255.255.255.255 for the IPv4 address, and 128 for the IPv6 address.

Example:

AN(config)#show llb dd table

Table of LLB DD static route

Destination: 10.3.0.10/24, ICMP, Expire:47h59m59s, type:static, Invalid result:0
 gateway: 10.8.1.1, response time: 3ms, hits: 0, status: UP.
 gateway: 192.168.1.30, response time: Timeout, hits: 0, status:DOWN.

Table of LLB DD route

Destination: 110.3.1.10/24, ICMP, Expire:47h59m22s, type:dynamic, Invalid result:1

gateway:	10.8.1.1, response time: Timeout, hits: 0, status: UP.
gateway:	192.168.1.30, response time: Timeout, hits: 0, status:DOWN.

In the above example, “response time” represents the time taken for the appliance to reach the destination IP address, and “status” indicates the status of the appliance gateway. In the given example, the status of “status” is “UP” but the “response time” status is “Timeout”, which means that the traffic of the appliance can successfully reach the gateway, but not the destination IP address.

clear llb dd table

This command is used to clear the information in the DD table.

➤ Rsite

llb rsite health {on|off}

This command is used to enable or disable the accessibility check for the LLB remote site.

llb rsite net <rsite_name> <network_ip> {netmask|prefix} <checker_name> [check_ip]

This command is used to define an LLB remote site and associate it with an accessibility check rule. A maximum of 1024 LLB remote sites can be configured. Each LLB remote site can be associated with a maximum of 1024 IP addresses, and each IP address can be associated with one accessibility check rule.

rsite_name	This parameter specifies the name of the LLB remote site. Its value must be a string of 1 to 31 characters. If the parameter value begins with a number, it must be enclosed in double quotes.
network_ip	This parameter specifies the network address to which the LLB remote site belongs or the IP address of the LLB remote site. Its value must be an IPv4 or IPv6 address.
netmask prefix	This parameter specifies the netmask or prefix length of the remote site network segment or IP address. • “netmask” is used for an IPv4 address. Its value must be a dotted IP address or an integer, ranging from 1 to 32. • “prefix” is used for an IPv6 address. Its value must be an integer ranging from 1 to 128.
checker_name	This parameter specifies the accessibility check rule to be associated with the remote site. Its value must be the name of the accessibility check rule specified by the “ llb rsite checker ” command.

check_ip Optional. This parameter specifies the IP address in the LLB remote site network to be checked. This parameter is not required to be specified when the parameter “network_ip” is set to an IP address.



Note: The system will not detect the LLB remote site anymore if it is deleted.

no llb rsite net <rsite_name> [network_ip] [netmask|prefix]

This command is used to delete the specified LLB remote site. If the parameters “network_ip” and “netmask|prefix” are specified, only the corresponding IP network segment is deleted from the LLB remote site.

show llb rsite net [rsite_name]

This command is used to display the specified LLB remote site. If the “rsite_name” parameter is not specified, all LLB remote sites will be displayed.

clear llb rsite net

This command is used to clear all LLB remote sites.

llb rsite checker <checker_name> {tcp|icmp} [port] [interval] [rsite_up] [rsite_down]

This command is used to define an accessibility check rule for an LLB remote site. A maximum of 32 accessibility check rules can be defined.

checker_name	This parameter specifies the name of the accessibility check rule. Its value must be a string of 1 to 31 characters. If the name begins with a number, it must be enclosed in double quotes.
tcp icmp	This parameter specifies the type of the accessibility check rule.
port	Optional. This parameter specifies the destination port for the TCP accessibility check rule. Its value must be an integer ranging from 1 to 65535. When the type of the accessibility check rule is set to “icmp”, this parameter will not take effect.
interval	Optional. This parameter specifies the interval at which the accessibility check is performed, in seconds. Its value must be an integer ranging from 5 to 300. The default value is 30. If more than 400 remote site networks are configured using the “ llb rsite net ” command, the interval should be no less

than 10 seconds; if more than 800 remote site networks are configured, the interval should be no less than 15 seconds.

rsite_up	Optional. This parameter specifies the number of consecutive accessibility check successes for setting the LLB remote site as “up”. Its value must be an integer ranging from 1 to 255. The default value is 3.
rsite_down	Optional. This parameter specifies the number of consecutive accessibility check failures for setting the LLB remote site as “down”. Its value must be an integer ranging from 1 to 255. The default value is 3.

no llb rsite checker <checker_name>

This command is used to delete the specified accessibility check rule.

show llb rsite checker [checker_name]

This command is used to display the specified accessibility check rule. If the “checker_name” parameter is not specified, all accessibility check rules will be displayed.

clear llb rsite checker

This command is used to clear all accessibility check rules.

show llb rsite status [rsite_name]

This command is used to display the status of the specified LLB remote site in relation to each LLB link. If the “rsite_name” parameter is not specified, the status of all LLB remote sites in relation to each LLB link will be displayed.

For example:

AN(config)#show llb rsite status					
Remote site: Milpitas					
net: 10.3.0.10/24					
Gateway	Status	Up Time	Down	Latest Down Event	Down Time
10.8.1.1	UP	10:30:00	0		
10.8.2.1	DOWN	00:00:00	1	Tue May 28 02:28:16 2015	00:05:00
net: 10.3.1.10/24					
Gateway	Status	Up Time	Down	Latest Down Event	Down Time
10.8.1.1	UP	10:30:00		Tue May 28 02:10:16 2015	00:00:00
10.8.2.1	UP	00:00:00	0		

➤ DNS Proxy

The DNS proxy function selects a DNS proxy server for the outbound DNS query. Currently, the system supports two load balancing methods rr and wrr to select the DNS server. Besides, the system allows the administrator to configure the domain

customization policy and DNS network policy to select a DNS server for the outbound DNS query. The domain customization policy has a higher priority than the DNS network policy, and the DNS network policy has a higher priority than the load balancing method.

llb dnsproxy {on|off}

This command is used to enable or disable the DNS proxy function. The DNS proxy function provides link load balancing for outbound DNS queries. When the DNS proxy function is enabled, the system will select a specified DNS server for the DNS query. By default, this function is disabled.

llb dnsproxy server <server_name> <server_ip> [weight]

This command is used to define a DNS server for the DNS proxy function, or change the IP address and weight of an existing DNS server configured for the DNS proxy function. A maximum of 128 DNS servers can be configured for the DNS proxy function.

server_name	This parameter specifies the DNS server name which is case sensitive. Its value must be a string of 1 to 63 characters. If the parameter value contains special characters, it must be enclosed in double quotes.
server_ip	Optional. This parameter specifies the weight of the DNS server using the wrr method. Its value must be an IPv4 or IPv6 address.
weight	Optional. This parameter specifies the weight of the DNS server. Its value must be an integer ranging from 1 to 1000. The default value is 1.



Note:

- If the same “weight” is specified for every DNS server or no “weight” is specified, the rr method will be used to select the DNS server; otherwise, the wrr method will be used.
- If the local DNS server of a client is not the DNS server specified by the “**llb dnsproxy server**” command, the DNS proxy function cannot take effect for this client.

no llb dnsproxy server <server_name>

This command is used to delete the specified DNS server configured for the DNS proxy function.

show llb dnsproxy server [server_name]

This command is used to display the specified DNS server configured for the DNS proxy function. If the “server_name” parameter is not specified, all DNS servers configured for the DNS proxy function will be displayed.

clear llb dnsproxy server

This command is used to delete all DNS servers configured for the DNS proxy function.

llb dnsproxy link <link_name> <server_name>

This command is used to associate the specified LLB link with the specified DNS server configured for the DNS proxy function. When the used bandwidth on an LLB link reaches the bandwidth threshold (configured using the the “**llb link route**” command), the DNS servers associated with this LLB link will no longer be selected until the this LLB link has spare bandwidth. The system will select a DNS server from other available DNS servers configured for the DNS proxy function. A maximum of 8 LLB links can be associated with a DNS server.

link_name This parameter specifies the LLB link name.

server_name This parameter specifies the DNS server name.

no llb dnsproxy link <link_name> <server_name>

This command is used to dissociate the specified LLB link from the specified DNS server configured for the DNS proxy function.

show llb dnsproxy link [link_name]

This command is used to display the association between the specified LLB link and the DNS servers configured for the DNS proxy function. If the “link_name” parameter is not specified, the associations between all LLB links and DNS servers configured for the DNS proxy function will be displayed.

clear llb dnsproxy link

This command is used to clear the associations between all LLB links and DNS servers configured for the DNS proxy function.

llb dnsproxy network <network_ip> <netmask><server_name>

This command is used to configure a DNS network policy, which is used for selecting a DNS server for the DNS query whose source IP address belongs to a specified subnet. After this command is configured, when the source IP address of a DNS query belongs to the subnet specified by this policy, the system will select the DNS server specified by this policy. If a DNS query matches multiple DNS network policies, the system will select the DNS network policy that has the longest subnet mask.

If the parameters “network_ip” and “netmask” are set to the existing values, this command can also be used to modify the DNS server of this DNS network policy.

A maximum of 512 DNS network policies can be configured. Only one DNS network policy can be configured for one subnet.

<code>network_ip</code>	This parameter specifies the IP address of the subnet to which the DNS query belongs. Its value must be an IPv4 address.
<code>netmask</code>	This parameter specifies the subnet mask corresponding to the IP address of the subnet to which the DNS query belongs. Its value must be in dotted decimal notation or an integer ranging from 0 to 32.
<code>server_name</code>	This parameter specifies the name of an existing DNS server. This DNS server must have been configured using the “ llb dnsproxy server ” command.



Note: For the DNS query matching the DNS network policy, even if the status of the DNS server selected by this DNS network policy is “Down”, the system will still select this DNS server.

no llb dnsproxy network *<network_ip> <netmask>*

This command is used to delete the specified DNS network policy.

show llb dnsproxy network *[network_ip] [netmask]*

This command is used to display the specified DNS network policy. If the parameters “network_ip” and “netmask” are not specified, all the DNS network policies will be displayed.

clear llb dnsproxy network

This command is used to clear all the DNS network policies.

llb dnsproxy domain *<host_name> <custom_policy>*

This command is used to configure a domain customization policy for the specified domain name, or change the customization policy configured for the specified domain name. The system allows only one domain customization policy to be configured for the specified domain name. The system supports domain customization policies of three types:

- **Static domain policy:** The system selects the specified DNS server for all DNS queries of the specified domain name.
- **Bypass domain policy:** The system does not use the DNS proxy function for the DNS queries of the specified domain name.
- **Persistent domain policy:** The system persistently selects the specified DNS server for the DNS queries of the specified domain name based on the source IP address. When the DNS query matches the persistent policy of the domain for the first time, the system selects the DNS server for the DNS query based on the DNS network policy (if configured and matched) or the load balancing method, and records the mapping between the source IP address and the selected DNS server.

When subsequent DNS queries from the same IP address hit this policy, the system will select the same DNS server persistently.

host_name	This parameter specifies the domain name. Its value must be a string of 1 to 255 characters in the format of “www.xyz.com”, and every section separated by “.” cannot be longer than 63 characters. This parameter supports the wildcards “^”, “*” and “\$” to match the domain name. The details are as follows: • “^”: matches the beginning character of the domain name. • “*”: matches 0 or more characters. For example, “www.*.com” means matching any domain name beginning with “www.” and ending with “.com”. • “\$”: matches the end character of the domain name.
custom_policy	This parameter specifies the customization policy for the domain name. Its value must be: • DNS server name: indicates that the customization policy is the static domain policy. • “bypass”: indicates that the customization policy is the bypass domain policy. • “persistent”: indicates that the customization policy is the persistent domain policy.

**Note:**

- For the DNS query matching the static domain policy, even if the status of the DNS server selected by this domain customization policy is “Down”, the system will still select this DNS server.
- If the DNS server selected based on the persistent policy becomes unavailable, the system will reselect a DNS server based on this policy.

no llb dnsproxy domain <host_name>

This command is used to delete the customized policy of the specified domain name.

show llb dnsproxy domain [host_name]

This command is used to display the customized policies of the specified domain name. If the “host_name” parameter is not specified, all customized policies of all domain names will be displayed.

clear llb dnsproxy domain

This command is used to clear all customized policies of domains.

show llb dnsproxy all

This command is used to display all configurations of the DNS proxy function.

clear llb dnsproxy all

This command is used to reset the configurations of the DNS proxy function to default.

Inbound Link Load Balancing

llb dns host <host_name> <ip> [weight] [port] [link_name]

This command is used to add an A or AAAA record for the specified DNS host name. The A or AAAA record is used for LLB inbound DNS resolution. The system supports a maximum of 2048 DNS host names and a maximum of 128 resource records (including A and AAAA records) for a DNS host name.

host_name	This parameter specifies the DNS host name in the format of “www.xyz.com”. Its value must be a string of 1 to 128 characters and every section separated by “.” cannot be longer than 63 characters. This parameter supports the wildcard “*” to match 1 or more characters of the host name. The DNS host name must be enclosed in double quotes if it contains the wildcard. This parameter value cannot end with “.”.
ip	This parameter specifies the IP address of the service corresponding to the DNS domain name. Its value must be an IPv4 or IPv6 address. In the scenario using both LLB and SLB, the parameter value can be the VIP of the virtual service. The values of the parameters “host_name” and “ip” constitute the A or AAAA record for this domain name.
weight	Optional. This parameter specifies the weight of the resource record for the wrr method. Its value must be an integer ranging from 1 to 65,535. The default value is 1.
port	Optional. This parameter specifies the server port number of the service corresponding to the DNS domain name. This port number is used for health check. In the scenario using both LLB and SLB, the parameter value can be the port number of the virtual service. Its value must be an integer ranging from 0 to 65,535. The default value is 0. If this parameter is set to 0, the system will firstly try to search for the first virtual service whose IP is the same as the IP specified by the “ip” parameter; if no virtual service is matched, it will then try to search for the first real service whose IP is the same as the “ip” parameter and the port is 0 (ip/l2ip/l2mac real service); if still no real service is matched, it will try to search for the first real service

whose IP is the same as the “ip” parameter. After a real service or virtual service is found, the system will send packets to the port specified by this service to perform the health check.

link_name Optional. This parameter specifies the name of the LLB link. The default value is empty, which means the system will search for an existing LLB link.



Note: The A or AAAA record configured by this command cannot be used to reply to the DNS Canonical Name (CNAME) query of this domain name.

no llb dns host *<host_name> <ip>*

This command is used to delete a resource record for the specified DNS host.

show llb dns host *[host_name]*

This command is used to display the resource records of the specified DNS host. If the “host_name” parameter is not specified, all resource records will be displayed.

clear llb dns host

This command is used to clear the resource records of all DNS hosts.

llb dns ttl *<host_name> [seconds]*

This command is used to set the Time to Live (TTL) value for the specified DNS host. The default TTL value of the DNS host is 60 seconds.

host_name This parameter specifies the DNS host name.

seconds This parameter specifies the length of time that the resource records for the DNS host name will be cached, in seconds. Its value must be an integer ranging from 0 to 4,294,967,295. If the value is zero, it means no cache.

show llb dns ttl *<host_name>*

This command is used to display the TTL configuration of the specified DNS host.

llb dns reply *<host_name> <max_rr_count>*

This command is used to set the maximum number of resource records that will be returned in the DNS reply for the specified DNS host.

host_name This parameter specifies the DNS host name

max_rr_count This parameter specifies the maximum number of resource records that will be returned in the DNS reply. Its value

must be an integer ranging from 1 to 3.

The default value is 3.

show llb dns reply [host_name]

This command is used to display the setting of the maximum number of resource records that will be returned for the specified DNS host. If the “host_name” parameter is not specified, the settings of the maximum number of resource records that will be returned for all DNS hosts will be displayed.

llb method inbound {rr|wrr|proximity}

This command is used to define a method for inbound LLB. The system supports the following methods:

- Round Robin (rr)
- Weighted Round Robin (wrr)
- proximity

The default setting is “rr”.



Note: To use the “proximity” method for inbound load balancing, please first configure the Eroute.

Link Health Check

llb link health {on|off}

This command is used to enable or disable the link health check function. By default, this function is disabled.

The following commands are used to configure ICMP, TCP and DNS health checks. At most eight health checks can be configured for each LLB link.

llb link health checker icmp <link_name> <host> [hc_interval] [timeout] [hc_up] [hc_down]

This command is used to add an ICMP health check for the specified LLB link. The ICMP health check allows the system to send ICMP echo requests to the destination host to detect the health status of the specified link.

link_name	This parameter specifies the name of an exiting LLB link configured using the “ llb link route ” command.
host	This parameter specifies the host name or IP address of the destination host. Its value must be an IPv4 or IPv6 address. When a host name is specified, the system performs health checks on the IPv4 address of the host through an IPv4

LLB link, and performs health checks on the IPv6 address of the host through an IPv6 LLB link. When an IP address is specified, it must be enclosed in double quotes.

hc_interval	Optional. This parameter specifies the time interval of the health check, in seconds. Its value must be an integer ranging from 1 to 65,535. The default value is 10.
timeout	Optional. This parameter specifies the timeout value of the health check, in seconds. Its value must be an integer ranging from 1 to 65,535. The default value is 5.
	Note: The timeout value cannot be larger than the time interval.
hc_up	Optional. This parameter specifies the number of consecutive health check successes for setting the LLB link as “up”. Its value must be an integer ranging from 1 to 65,535. The default value is 3.
hc_down	Optional. This parameter specifies the number of consecutive health check failures for setting the LLB link as “down”. Its value must be an integer ranging from 1 to 65,535. The default value is 3.

Example:

```
AN(config)#llb link health checker icmp "link1" "10.191.2.130" 10 5 3 3
AN(config)#llb link health checker icmp "link1" "10.191.2.131" 10 5 3 3
```

no llb link health checker icmp <link_name> <host>

This command is used to delete an ICMP health check of the specified LLB link.

llb link health checker tcp <link_name> <host> <port> [hc_interval] [timeout] [hc_up] [hc_down]

This command is used to add a TCP health check for the specified LLB link. The TCP health check allows the system to establish TCP connections with the destination host to detect the health status of the specified LLB link.

link_name	This parameter specifies the name of an exiting LLB link configured using the “llb link route” command.
host	This parameter specifies the host name or IP address of the destination host. Its value must be an IPv4 or IPv6 address. When a host name is specified, the system performs health checks on the IPv4 address of the host through an IPv4 LLB link, and performs health checks on the IPv6 address of the host through an IPv6 LLB link. When an IP address

is specified, it must be enclosed in double quotes.

port	This parameter specifies the port number of the destination host, which is used as the destination port of the TCP packets. Its value must be an integer ranging from 1 to 65,535.
hc_interval	Optional. This parameter specifies the time interval of the health check, in seconds. Its value must be an integer ranging from 1 to 65,535. The default value is 10.
timeout	Optional. This parameter specifies the timeout value of the health check, in seconds. Its value must be an integer ranging from 1 to 65,535. The default value is 5.
	Note: The timeout value cannot be larger than the time interval.
hc_up	Optional. This parameter specifies the number of the number of consecutive health check successes for setting the LLB link as “up”. Its value must be an integer ranging from 1 to 65,535. The default value is 3.
hc_down	Optional. This parameter specifies the number of consecutive health check failures for setting the LLB link as “down”. Its value must be an integer ranging from 1 to 65535. The default value is 3.

Example:

```
AN(config)#llb link health checker tcp "link1" "www.xyz.com" 80 10 5 3 3
AN(config)#llb link health checker tcp "link1" "10.191.2.141" 21 10 5 3 3
```

no llb link health checker tcp <link_name> <host> <port>

This command is used to delete a TCP health check of the specified LLB link.

llb link health checker dns <link_name> <host> <host_name> [interval]
[timeout] [hc_up] [hc_down]

This command is used to add a DNS health check for the specified LLB link. The DNS health check allows the system to send DNS queries to the destination host to detect the health status of the specified link.

link_name	This parameter specifies the name of an exiting LLB link configured using the “ llb link route ” command.
host	This parameter specifies the host name or IP address of the destination host. Its value must be an IPv4 or IPv6 address. When a host name is specified, the system performs health checks on the IPv4 address of the host through an IPv4

	LLB link, and performs health checks on the IPv6 address of the host through an IPv6 LLB link. When an IP address is specified, it must be enclosed in double quotes.
host_name	This parameter specifies the domain name to be resolved. Its value must be a string of 1 to 128 characters. The system will get link status based on the resolution results.
hc_interval	Optional. This parameter specifies the time interval of the health check, in seconds. Its value must be an integer ranging from 1 to 65,535. The default value is 10.
timeout	Optional. This parameter specifies the timeout value of the health check, in seconds. Its value must be an integer ranging from 1 to 65,535. The default value is 5.
	Note: The timeout value cannot be larger than the time interval.
hc_up	Optional. This parameter specifies the number of the number of consecutive health check successes for setting the LLB link as “up”. Its value must be an integer ranging from 1 to 65,535. The default value is 3.
hc_down	Optional. This parameter specifies the number of consecutive health check failures for setting the LLB link as “down”. Its value must be an integer ranging from 1 to 65,535. The default value is 3.

Example:

```
AN(config)#llb link health checker dns "link1"10.191.2.150" www.test.com" 20 5 3 3
AN(config)#llb link health checker dns "link1" "10.191.2.151" "www.test.com" 20 5 3 3
```

no llb link health checker dns <link_name> <host> <host_name>

This command is used to delete the DNS health check of the specified LLB link.

show llb link health checker [link_name]

This command is used to display the health check configuration of the specified LLB link. If the “link_name” parameter is not specified, the health check configuration of all LLB links will be displayed.

clear llb link health checker [link_name]

This command is used to clear the health check of the specified LLB link. If the “link_name” parameter is not specified, the health check configurations of all LLB links will be cleared.

llb link health relation <link_name> <relationship>

This command is used to set the relationship among all health checks of the specified link.

link_name	This parameter specifies the link name.
relationship	This parameter specifies the relationship among all health checks of the link. Its value must be “and” or “or”, which are case-sensitive. • and: the link is marked as available only when all health checks succeed. • or: the link is marked as available when any of the health checks succeeds. The default value is “or”.

show llb link health relation [*link_name*]

This command is used to display the configuration of the “**llb link health relation**” command for the specified link. If the “link_name” parameter is not specified, the configurations of the “**llb link health relation**” command for all links will be displayed.

IP Address Group Support

ipgroup name <*ip_group_name*> [*group_type*]

This command is used to create a new IP address group. The system supports a maximum of 1024 IP address groups.

ip_group_name	This parameter specifies the name of the IP address group. Its value must be a string of 1 to 128 characters. Only letters, numbers and underscore are supported. It must start with a letter.
group_type	Optional, this parameter specifies the type of IP address group. The value must be “ipv4” or “ipv6”. The default value is “ipv4”.

no ipgroup name <*ip_group_name*>

This command is used to delete a specified IP address group and clear the IP address segments it contains. When an IP address group contains used IP addresses, the IP address group cannot be deleted.

show ipgroup name

This command is used to display all IP address groups.

clear ipgroup name

This command is used to delete all IP address groups and clear the IP address segments contained in them. IP address groups cannot be deleted when they are being used by other functions.

show statistics ipgroup [ip_group_name]

This command is used to display the statistics of the specified IP address group. If the parameter “ip_group_name” is not specified, the system will display the statistics of all IP address groups.

clear statistics ipgroup [ip_group_name]

This command is used to clear the statistics of the specified IP address group. If the parameter “ip_group_name” is not specified, the system will clear the statistics of all IP address groups.

ipgroup address <ip_group_name> <start_ip> [end_ip/mask/prefix]

This command is used to add an IP address segment to an existing IP address group. The system supports two ways to configure the IP address segment: “IP+mask/prefix length” and “start IP+end IP”.

The same address segment can be added to multiple IP address groups, but the IP address segments within the same IP address group cannot overlap.

group_name	This parameter specifies the name of an existing IP address group.
start_ip	This parameter specifies the starting address of the IP address segment added to the IP address group. It can be an IPv4 or IPv6 address.
end_ip/mask/prefix	Optional, this parameter specifies the end address of the IP address segment, or the mask of the IPv4 address or the prefix length of the IPv6 address.

The number of IP address segments supported by the system depends on the size of the system memory, as shown in the following table:

Specification	Memory					
	2G	4G	8G	16G	32G	64G
Maximum number of IP segments	10,000	20,000	40,000	80,000	160,000	160,000

no ipgroup address <ip_group_name> <start_ip> [end_ip/mask/prefix]

This command is used to delete a specified IP address segment from a specified IP address group.

show ipgroup address *[ip_group_name]*

This command is used to display all IP address segments in the specified IP address group. If the parameter “ip_group_name” is not specified, the system will display all IP address segments in all IP address groups.

clear ipgroup address *[ip_group_name]*

This command is used to clear all IP address segments in the specified IP address group. If the parameter “ip_group_name” is not specified, the system will clear all IP address segments in all IP address groups.

show ipgroup match *<ip_address>*

This command is used to query and display all IP address groups and IP address segments to which the specified IP address belongs.

LLB Statistics

llb link statistics {on|off}

This command is used to enable or disable the LLB link statistics function. By default, the function is disabled.

clear statistics llb link *[link_name]*

This command is used to clear the statistics of the specified LLB link. If the “link_name” parameter is not specified, the statistics of all LLB links will be cleared.

show llb link status *[detail]*

This command is used to display the status of LLB links and link health checks.

detail	Optional. If the parameter value “detail” is entered, the status of all the LLB links, link health checks, and related summary will be displayed, such as the number of successful health checks and the number of failed health checks.
--------	--

show llb link bandwidth

This command is used to display the bandwidth statistics of all LLB links in the inbound and outbound directions.

show llb link connection

This command is used to display the connection statistics of all LLB links, including the link name, gateway IP address and number of current connections.

show statistics eroute

This command is used to display the statistics related to Eroutes, including Eroutes, Droutes, IPflow routes and LLB link routes.

clear statistics eroute

This command is used to clear the statistics related to Eroutes, including Eroutes, Droutes, IPflow routes and LLB link routes.

show statistics ipflow <ip_address>

This command is used to display the statistics of IPflow routes for the specified IP address.

ip_address	This parameter specifies the destination or source IP address based on which IPflow route statistics are displayed. Its value must be an IPv4 or IPv6 address. If the parameter is specified as “0.0.0.0”, all IPflow route entries of IPv4 addresses will be displayed. If the parameter is specified as “::”, all IPflow route entries of IPv6 addresses will be displayed.
------------	---

show statistics rts <ip_address>

This command is used to display the statistics of RTS routes for the specified IP address.

ip_address	This parameter specifies the destination or source IP address based on which RTS route statistics are displayed. Its value must be an IPv4 or IPv6 address. If the parameter is specified as “0.0.0.0”, all RTS route entries of IPv4 addresses will be displayed. If the parameter is specified as “::”, all RTS route entries of IPv6 addresses will be displayed.
------------	--

show statistics droute

This command is used to display the statistics of Droutes.

show statistics llb dd

This command is used to display the statistics of the dd method.

Example:

```
AN(config)#show statistics llb dd

dd entry num: 3
dd sent packets num: 72
dd probing dstip num: 3
detector ipv4 llb link route num: 3
```

```

detector ipv6 llb link route num: 3
daemon detector detect times: 4
filter out RINGQ ipv4 data not into user queue: 0
filter out RINGQ ipv6 data not into user queue: 0

```

show statistics llb rsite [rsite_name]

This command is used to display the statistics of the specified LLB remote site. If the “rsite_name” parameter is not specified, the statistics of all LLB remote sites will be displayed.

For example:

```

AN(config)#show statistics llb rsite
Remote site: Milpitas
net: 10.3.0.10/24
Gateway          Status  Latest Response Time  Detect Times  UP times  Down Times
10.8.1.1          UP      1ms                   100           98         2
10.8.2.1          DOWN    timeout               100           90         10
net: 10.3.1.10/24
Gateway          Status  Latest Response Time  Detect Times  UP times  Down Times
10.8.1.1          UP      2ms                   90            89         1
10.8.2.1          UP      1ms                   90            87         3

```

clear statistics llb rsite [rsite_name]

This command is used to clear the statistics of the specified LLB remote site. If the “rsite_name” parameter is not specified, the statistics of all LLB remote sites will be cleared.

show statistics llb dnsproxy [server_name]

This command is used to display the statistics of the specified DNS server configured for the DNS proxy function. If the “server_name” parameter is not specified, the statistics of all DNS servers configured for the DNS proxy function will be displayed.

clear statistics llb dnsproxy [server_name]

This command is used to clear the statistics of the specified DNS server configured for the DNS proxy function. If the “server_name” parameter is not specified, the statistics of all DNS servers configured for the DNS proxy function will be cleared.

show statistics llb inbound [host_name]

This command is used to display the inbound LLB statistics corresponding to the specified host name, including the status and number of hits of the LLB links in the inbound direction. If the “host_name” parameter is not specified, the numbers of DNS host names, A and AAAA records, and the inbound LLB statistics corresponding to all DNS host names will be displayed.

clear statistics llb inbound

This command is used to clear the inbound LLB statistics of all DNS host names. After this command is executed, the number of hits of the LLB links in the inbound direction will be cleared.

General Configurations of LLB

llb probe tcp <source_ip> <source_port> <destination_ip> <destination_port>

This command is used to simulate a client with the specified source IP and source port sending a TCP SYN packet to the destination IP address and destination port to detect the policy matched by the specified IP address.

source_ip	This parameter specifies the source IP address.
source_port	This parameter specifies the source port.
destination_ip	This parameter specifies the destination IP address.
destination_port	This parameter specifies the destination port.

llb probe udp <source_ip> <source_port> <destination_ip>
<destination_port>

This command is used to simulate a client with the specified source IP and source port sending a UDP packet to the destination IP address and destination port to detect the policy matched by the specified IP address.

source_ip	This parameter specifies the source IP address.
source_port	This parameter specifies the source port.
destination_ip	This parameter specifies the destination IP address.
destination_port	This parameter specifies the destination port.

llb probe icmp <source_ip> <destination_ip>

This command is used to simulate a client with the specified source IP sending an ICMP Request to the destination IP address to detect the policy matched by the specified IP address.

source_ip	This parameter specifies the source IP address.
destination_ip	This parameter specifies the destination IP address.

show llb connection [filter_string]

This command is used to display the information of the specified parameter in the NAT connection. If the “filter_string” parameter is not specified, the system will

display all information of the NAT connections. The displayed contents include protocol, client source IP and port, source IP and port after NAT, destination IP and port, route type, route name, interface name, gateway and timeout.

filter_string Optional, this parameter specifies the name of the filter parameter to be displayed. Use comma to separate multiple filter parameters.

The parameter format is “keyword 1=xxx, keyword 2=xxx, ...”, for example, “source=10.8.1.1, dport=80, proto=tcp”. Keywords supported by the system include:

- source: indicates client IP address.
- local: indicates local IP address.
- destination: indicates destination IP address.
- sport: indicates client port.
- lport: indicates local port.
- dport: indicates service port.
- proto: indicates protocol type. Its value must be “tcp”, “udp”, “icmp”, “all”. “all” means all the above protocols.

The keywords above and their values are case-insensitive.

The default value is empty, which means displaying all NAT connection information.

clear llb connection [*filter_string*]

This command is used to delete the information of the specified parameter in the NAT connection. If the “filter_string” parameter is not specified, the system will delete all information of the NAT connections.

Chapter 19 Global Server Load Balancing (GSLB)

This chapter covers the commands for configuring the Smart DNS (SDNS) function to implement Global Server Load Balancing (GSLB).

Basic SDNS Commands

sdns {on|off}

This command is used to enable or disable the SDNS function. By default, this function is disabled. To use this function, you must import a license with the GSLB feature.



Note: When the SDNS function is disabled, you cannot execute the “**sdns dps**” and “**sdns iana**” commands.

show sdns status

This command is used to display the status of the SDNS function, the status of the SDNS Recursive Query function, and the usage of SDNS service IPs, pools, host names, regions, proximity rules, health check templates, and health check instances.

For example:

```
AN(config)#show sdns status
SDNS is ON
SDNS recursion is OFF
Service IPs:      10 / 20000
Pools:            4 / 16000
Host names:       2 / 16000
Regions:          10 / 100
Proximities:      100 / 200000
Monitors:         20 / 200
Monitor instances: 50 / 5000
```

show sdns all

This command is used to display all SDNS configurations.



Note: The command “show sdns all” will not display SDNS configuration files.

clear sdns all

This command is used to reset all SDNS configurations to default values.



Note: When the command “**clear sdns all**” or “**clear config secondary**” is executed to reset the SDNS configurations to default, the system will not check whether the GSLB feature is licensed. The command “clear sdns all” will not clear the SDNS configuration files.

sdns edns ecs {on|off}

This command is used to enable or disable the ECS function for SDNS domain name forwarding function. After this function is enabled, the system will insert the client source IP address in the DNS request when the SDNS domain name is forwarded. The SDNS domain name forwarding function is configured through the command “**sdns host name**”. By default, this function is disabled.

show sdns edns ecs

This command is used to display the ECS configuration of the SDNS domain name forwarding function.

show sdns query match <source_ip> <host_name> [query_type]

This command is used to display the resolution process of the specified DNS query.

source_ip	This parameter specifies the source IP address of the DNS query. Its value must be an IPv4 or IPv6 address.
host_name	This parameter specifies the host name in the DNS query. Its value must be a string of 1 to 128 characters in the format of “www.xyz.com” and every section separated by “.” cannot be longer than 63 characters. The host name does not support the wildcard “*” or any other special characters and cannot end up with the character “.”.
query_type	Optional. This parameter specifies the type of the DNS query. Its value must be “A” or “AAAA”. The default value is “A”.

For example:

```
AN(config)#show sdns query match 10.6.0.166 www.example.com
[1] Check Host Name:
"www.example.com" found.
[2] Check Region Matched:
all
[3] Check Region Policy:
Check region "all" ---> Found region policy "all" with priority 0
Matched Region Policy: "all"
Matched Pool: "pool_default"
Select member from pool "pool_default"
[4] Result:
    A Record: 10.8.6.233
```

SDNS Host Name

sdns host name <host_name> [ttl] [type]

This command is used to define an SDNS host name. A maximum of 16,000 host names can be defined in the system.

host_name This parameter specifies the SDNS host name. Its value must be a string of 1 to 128 characters in the format of “www.xyz.com” and every section separated by “.” cannot be longer than 63 characters. The value of this parameter supports wildcard “*” and special symbol “_” and “.”, and does not support any other special symbols. “.” cannot be placed in start and end positions, and “_” cannot be placed in the start position.

The wildcard “*” can be configured as a single “*” or a combination of “*” and other characters. When “*” is used, it must be placed at the beginning of the parameter value and followed by at least one other character. The whole string must be enclosed in double quotes, such as “*a”. If a domain name matches two wildcard domain names, the longest suffix match principle applies. Please note that SDNS DNSSEC and SDNS Canonical Name (CNAME) do not support wildcard domain names.

When the parameter “type” is set to “forward_only”, the SDNS domain name is a forwarding domain name. The forwarding domain name supports extensive domain name, such as “*.com”. The forwarding domain name cannot be associated with the CNAME pool.

ttl Optional. This parameter specifies the length of time that the resource records for this host name will be cached before they expire. Its value must be an integer ranging from 0 to 4,294,967,295, in seconds. The default value is 60.

type Optional. This parameter specifies the SDNS domain name type. The value must be:

- **forward_only**: indicates that when the SDNS domain name forwarding function is enabled, the system will forward DNS requests matching the specified domain name directly without local resolution. When the parameter “host_name” is specified as an extensive domain name, DNS requests to query the domain name and its subdomains will be forwarded. Note the following when the value of the parameter “type” is set to “forward_only”:

1. The parameter “ttl” does not take effect.
2. If an SDNS service pool contains IPv4 and IPv6 SDNS services, the system can use IPv4 and IPv6 addresses for forwarding domain names.
 - empty: indicates that the system resolves the SDNS domain name locally.

The default value is empty.



Note: If the data of the SDNS response exceeds 512 bytes, the SDNS response will not be returned to the client.

no sdns host name <host_name>

This command is used to delete a specified host name.

show sdns host name

This command is used to display all the defined host names.

clear sdns host name

This command is used to clear all the defined host names.

SDNS Listening IP Address

sdns listener <ip_address>

This command is used to set the IP address on which the SDNS service listens. After an SDNS listening IP is configured, the system provides the SDNS service only on the specified listening IP address. A maximum of 16 SDNS listening IP addresses are supported.

ip_address	This parameter specifies the IP address on which the SDNS service listens. Its value must be an IPv4 or IPv6 address, but cannot be a broadcast, multicast or loopback address. “0.0.0.0” and “::” are not supported.
-------------------	---

no sdns listener <ip_address>

This command is used to delete the configuration of the specified SDNS listening IP address.

show sdns listener

This command is used to display all configurations of SDNS listening IP addresses.

clear sdns listener

This command is used to clear all configurations of SDNS listening IP addresses.

SDNS Service

sdns service ip <service_name> <ip_address> [port] [dc_name] [hc_mode]

This command is used to define an SDNS service and associate it with the specified SDNS data center. Its IP address, known as SDNS service IP, is used as the resolved IP address for the specified SDNS host name. A maximum of 20,000 SDNS service IPs can be defined in the system.

service_name This parameter specifies the SDNS service name. Its value must be a string of 1 to 50 characters.

ip_address This parameter specifies the IP address of the SDNS service. Its value must be an IPv4 or IPv6 address.

port Optional. This parameter specifies the port used for health check on the SDNS service. Its value must be an integer ranging from 0 to 65,535. The default value is 0.

dc_name Optional. This parameter specifies the name of an existing SDNS data center. The default value is null, indicating that the SDNS service will not be associated with any data center.

Note that to modify the data center of the SDNS service that is associated with an SDNS link, administrators should first delete the association between the SDNS service and the SDNS link using the “**no sdns link service**” command.

hc_mode Optional. this parameter specifies the SDNS health check mode. Its value must be:

- 0: indicates the normal mode. In this mode, the system adopts the original behavior. When the local data center does not receive the status of the health check instance returned by other data centers, the system will not probe actively and the health check instance status of the data center will be directly set as as “Down”.
- 1: indicates the failover probe mode. In this mode, the system actively probes when the local data center does not receive the health check instance status returned by other data centers. Specifically, it can be divided into the following two situations:
 - If the destination IP address in the health check template is set, and the destination IP address is different from the SDNS service IP address. Then, after applying the health check template to the SDNS service, the system will create two health

check instances, the primary instance and the standby instance (viewed by using the “**show statistics sdns dc instance**” command).

- When the local data center detects the health check status of another data center, the health check status of the primary instance will be marked as the detected state, and the health check status of the standby instance will be the same as that of the primary instance.
- If the local data center cannot detect the health check status of another data center, the health check status of the primary instance will be marked as “Down”, and the health check status of the standby instance is obtained by detecting the SDNS service IP address.
- If the destination IP address of the health check template is the same as the IP address of the SDNS service, after associating the SDNS service with the health check template, only one health check instance is generated, and the local data center will not perform active detection if the status of the health check instance fails
- If the destination IP address is not set for the health check template, the system will generate only one health check instance after applying the health check template to the SDNS service. There are two specific situations:
 - When the local data center receives the status of a health check instance from another data center, it will directly use this status to identify the status of the health check instance.
 - When the local data center does not receive the status of the health check instance from another data center, the system will actively detect the health check instance status.

The default value is 0.

no sdns service ip <service_name>

This command is used to delete a specified SDNS service.

show sdns service ip

This command is used to display all the defined SDNS services.

clear sdns service ip

This command is used to clear all the defined SDNS services.

sdns service {enable|disable} <service_name>

This command is used to enable or disable a specified SDNS service. The SDNS service is enabled by default after it is defined.

service_name This parameter specifies the name of the SDNS service.

show sdns service disable

This command is used to display all the SDNS services that have been disabled.

show sdns service status

This command is used to display the status of every SDNS service, including its availability, enabling status and the number of hits.

For example:

AN(config)# show sdns service status s1 10.8.6.45 UP ENABLE HITS:2 s2 10.8.6.50 UP ENABLE HITS:5

SDNS Pool

sdns pool name <pool_name> [max_rr_count]

This command is used to define an SDNS service pool, which is a collection of SDNS services. A maximum of 16,000 SDNS service pools can be defined in the system.

In addition, when an SDNS service pool is defined through this command, the system will generate an SDNS emergency pool with the same name at the same time.

pool_name This parameter specifies the name of the SDNS service pool. Its value must be a string of 1 to 40 characters.

max_rr_count This parameter specifies the maximum number of resource records that will be returned in the DNS response message when this SDNS service pool is hit. Its value must be an integer from 1 to 3. The default value is 3.

**Note:**

- The SDNS will construct resource records by using the queried SDNS host name and the SDNS IPs chosen from the hit SDNS service pool.
- If an SDNS service pool contains both IPv4 and IPv6 SDNS services, the system will return the corresponding resource record according to the request type. For example, if the value of the parameter "**max_rr_count**" is 3, the number of the A record in the service pool is 1, and the number of

the AAAA record is 1. When the client requests the A resource, the system returns only one A resource record. When the client requests the AAAA resource, the system returns only one AAAA resource record.

no sdns pool name <pool_name>

This command is used to delete the specified SDNS service pool.

In addition, when the specified SDNS service pool is deleted through this command, the system will also delete the SDNS emergency pool with the same name.

show sdns pool name

This command is used to display all the defined SDNS service pools.

clear sdns pool name

This command is used to clear all the defined SDNS service pools.

In addition, when SDNS service pools is cleared through this command, the system will also clear the SDNS emergency pools with the same name.

sdns pool service <pool_name> <service_name>

This command is used to add an SDNS service to the specified SDNS service pool. An SDNS service can be added to multiple SDNS service pools. The system can add both IPv4 and IPv6 SDNS services. A maximum of 100 IPv4 and 100 IPv6 SDNS services can be added to an SDNS service pool.

In addition, when the SDNS service is added to the SDNS service pool through this command, the system will also add the SDNS service to the SDNS emergency pool with the same name.

pool_name	This parameter specifies the name of the SDNS service pool.
-----------	---

service_name	This parameter specifies the name of the SDNS service.
--------------	--

no sdns pool service <pool_name> <service_name>

This command is used to delete an SDNS service from the specified SDNS service pool.

In addition, when the specified SDNS service in the SDNS service pool is deleted through this command, the system will also delete the specified SDNS service in the SDNS emergency pool with the same name.

show sdns pool service [pool_name]

This command is used to display all SDNS services in the specified SDNS service pool. If the optional parameter “pool_name” is not specified, the SDNS services of all the SDNS service pools will be displayed.

clear sdns pool service <pool_name>

This command is used to clear all SDNS services from the specified SDNS service pool.

In addition, when SDNS services in the SDNS service pool are cleared through this command, the system will also clear the SDNS services in the SDNS emergency pool with the same name, but the SDNS emergency services that are not in the SDNS service pool will be not cleared from the SDNS emergency pool.

sdns pool method primary <pool_name> <method> [expression] [sorting_mode]

This command is used to specify the primary method used to choose SDNS service IPs from the SDNS service pool. When no primary method is configured for an SDNS service pool, the SDNS service pool uses the Round Robin (rr) method by default.

If a service pool contains both IPv4 and IPv6 SDNS services, the system will select the primary method for these SDNS services. For example, if the primary method is Round Robin (rr), only the A record will be checked cyclically when the A resource record is requested, and only the AAAA record will be checked cyclically when the AAAA record resource is requested.

pool_name	This parameter specifies the name of the SDNS service pool.
method	This parameter specifies the primary method of the SDNS service pool. Its value must be: • rr: indicates the Round Robin method. • wr: indicates the Weighted Round Robin method. • ipo: indicates the IP Overflow method. • hi: indicates the Hash IP method. • snmp: indicates the Simple Network Management Protocol method. • drop: indicates the Drop method.
expression	Optional. This parameter specifies the name of the SNMP expression used to compute the metric of every SDNS service IP. This parameter is available only when the “method” parameter is set to “snmp”. Its value must be the name of the SNMP expression configured using the “ sdns snmp exp weight ” command.

sorting_mode	Optional. This parameter specifies the mode that the snmp method uses to sort up SDNS service IPs. This parameter is available only when the “method” parameter is set to “snmp”. Its value must be: - asc: SDNS service IPs are sorted up in the ascending order of the metrics. - dsc: SDNS service IPs are sorted up in the descending order of the metrics. The default value is “asc”.
--------------	--

The following table describes the meaning of the SDNS service pool methods.

Method	Description
rr	If the service pool contains three service IPs, the maximum_rr_count is 1, and the rr method is used, the service IPs will be returned in the DNS responses in the sequence of “1, 2, 3, 1, 2, 3...”.
wrr	The wrr method is similar to the rr method. The difference is that every service IP in the service pool is assigned a weight. The second service IP will not be returned in the DNS responses until the number of times that the first service IP is returned reaches the weight value.
ipo	If the ipo method is used, the service IP with the highest priority will always be returned in the DNS responses.
hi	If the hi method is used, SDNS will compute the hash value based on the source IP address. For DNS queries with the same source IP address, SDNS returns the same service IP.
snmp	If the snmp method is used, SDNS will collect the data of specified OIDs from every SDNS service IP through the SNMP protocol, compute the metric of every service IP using the specified SNMP expression, and sort up service IPs in the specified sorting mode. The service IP in the first order will be returned in the DNS response. For details, refer to the section “SNMP Data Collection” in ArrayOS APV User Guide.
drop	If the drop method is used, SDNS will directly discard the DNS query and return no DNS response.

sdns pool method secondary <pool_name> <method> [expression]
[sorting_mode]

This command is used to specify the secondary method used to choose SDNS services from the SDNS service pool when no available SDNS service IP can be chosen using the primary method. By default, the SDNS service pool has no secondary method.

pool_name	This parameter specifies the name of the SDNS service pool.
method	This parameter specifies the secondary method of the SDNS service pool. Its value must be: - rr: indicates the Round Robin method. - wrr: indicates the Weighted Round Robin method.

	• ipo: indicates the IP Overflow method. • hi: indicates the Hash IP method. • snmp: indicates the Simple Network Management Protocol method. • drop: indicates the Drop method.
expression	Optional. This parameter specifies the name of the SNMP expression used to compute the metric of every SDNS service IP. This parameter is available only when the “method” parameter is set to “snmp”. Its value must be the name of the SNMP expression configured using the “ sdns snmp exp weight ” command.
sorting_mode	Optional. This parameter specifies the mode that the snmp method uses to sort up SDNS service IPs. This parameter is available only when the “method” parameter is set to “snmp”. Its value must be: • asc: SDNS service IPs are sorted up in the ascending order of the metrics. • dsc: SDNS service IPs are sorted up in the descending order of the metrics. The default value is “asc”.

no sdns pool method secondary <pool_name>

This command is used to delete the secondary method configured for the specified SDNS service pool.



Note: If the secondary method “ipo” of an SDNS service pool is deleted, the priorities of all SDNS services in this SDNS service pool will be reset to the default value 1.

sdns pool member weight <pool_name> <service_name> [weight]

This command is used to set the weight for the specified SDNS service in the SDNS service pool using the wr method.

pool_name	This parameter specifies the name of the SDNS service pool.
service_name	This parameter specifies the name of the SDNS service.
weight	Optional. This parameter specifies the weight for the SDNS service. Its value must be an integer ranging from 1 to 4,294,967,295. The default value is 1.

clear sdns pool member weight <pool_name>

This command is used to reset the weights of all SDNS services in the specified SDNS service pool using the wrd method to the default value, 1.

sdns pool member priority <pool_name> <service_name> [priority]

This command is used to set the priority for the specified SDNS service in the SDNS service pool using the ipo method.

pool_name	This parameter specifies the name of the SDNS service pool.
service_name	This parameter specifies the name of the SDNS service.
priority	Optional. This parameter specifies the priority for the SDNS service. Its value must be an integer ranging from 1 to 4,294,967,295. The default value is 1. The greater the value, the higher the priority.

clear sdns pool member priority <pool_name>

This command is used to reset the priorities of all SDNS services in the specified SDNS service pool using the ipo method to the default value, 1.

sdns pool failover <pool_name> [on|off]

This command is used to enable or disable the Auto Failover function for the specified SDNS service pool using the ipo method.

When this function is enabled for the specified SDNS service pool, the SDNS automatically switch the currently selected SDNS service to the SDNS service with second highest priority when the SDNS service with the highest priority is unavailable. By default, this function is enabled for every SDNS service pool using the ipo method.

pool_name	This parameter specifies the name of the SDNS service pool.
on off	Optional. This parameter enables or disables the auto failover function. The default value is “on”.

sdns pool preempt <pool_name> [on|off]

This command is used to enable or disable the Preemption function for the specified SDNS service pool using the ipo method.

When this function is enabled for the specified SDNS service pool, the SDNS automatically switch the currently selected SDNS service back to the SDNS service

with highest priority when this SDNS service becomes available again. By default, this function is enabled for every SDNS service pool using the ipo method.

pool_name	This parameter specifies the name of the SDNS service pool.
on off	Optional. This parameter enables or disables the preemption function. The default value is “on”.

sdns pool ipo reset <pool_name> [priority]

This command is used to manually switch the currently selected SDNS service to the SDNS service with the specified priority in the SDNS service pool. When this command is executed, the manual switch operation will be performed. The manual switch operation can be saved as configuration and will become ineffective after system reboot.

pool_name	This parameter specifies the name of the SDNS service pool.
priority	Optional. This parameter specifies the priority. Its value is an integer ranging from 0 to 4,294,967,295. The default value is 0, indicating the highest priority of all available pool members. The greater the value, the higher the priority.



Note: When both the Auto Failover and Preemption functions are disabled, the manual switch operation will become ineffective.

sdns snmp oid <oid_name> <snmp_oid>

This command is used to define an SNMP OID. A maximum of eight SNMP OIDs can be defined.

oid_name	This parameter specifies the name of the SNMP OID. Its value must be a string of 1 to 20 characters. When the parameter value contains only numbers, it must be enclosed in double quotes.
snmp_oid	This parameter specifies the SNMP OID. Its value must be a string of 2 to 895 characters.

no sdns snmp oid <oid_name>

This command is used to delete a defined SNMP OID.

show sdns snmp oid

This command is used to display all defined SNMP OIDs.

clear sdns snmp oid

This command is used to clear all defined SNMP OIDs.

show sdns snmp localoid

This command is used to display local SNMP OIDs supported by the system.

AN(config)#show sdns snmp localoid	
Oid Name	Oid
CPU usage	.1.3.6.1.4.1.7564.30.1.0
Memory usage	.1.3.6.1.4.1.7564.4.2.0
New connections	.1.3.6.1.4.1.7564.30.2.0

sdns snmp exp weight *<expression>* *<oid1_name>* [*oid1_weight*]
[*oid2_name*] [*oid2_weight*] [*oid3_name*] [*oid3_weight*]

This command is used to configure the SNMP expression used to compute the metric of the SDNS service IP. Every SNMP expression uses the data of one SNMP OID at minimum and three SNMP OIDs at maximum. The format of the SNMP expression is as follows:

Data of OID1 × Weight of OID1 + Data of OID2 × Weight of OID2 + Data of OID3 × Weight of
OID3

expression This parameter specifies the name of the SNMP expression used to compute the metric of the SDNS service IP. Its value must be a string of 1 to 20 characters. When the parameter value contains only numbers, it must be enclosed in double quotes.

oid1_name This parameter specifies the name of the first SNMP OID.

oid1_weight Optional. This parameter specifies the weight of the first SNMP OID. Its value must be an integer ranging from 0 to 65,535. The default value is 1.

oid2_name This parameter specifies the name of the second SNMP OID.

oid2_weight Optional. This parameter specifies the weight of the second SNMP OID. Its value must be an integer ranging from 0 to 65,535. The default value is 1.

oid3_name This parameter specifies the name of the third SNMP OID.

oid3_weight Optional. This parameter specifies the weight of the third SNMP OID. Its value must be an integer ranging from 0 to 65,535. The default value is 1.



Note: When an SDNS service pool uses the configured SNMP expression, you must configure the SNMP data collection template for every SNMP OID in the

expression and associate the templates with the SDNS service pool.

no sdns snmp exp weight <expression>

This command is used to delete the specified SNMP expression.

show sdns snmp exp weight [expression]

This command is used to display the specified SNMP expression. If the “expression” parameter is not specified, all SNMP expressions will be displayed.

clear sdns snmp exp weight

This command is used to clear all SNMP expressions.

sdns pool cname <pool_name> <cname>

This command is used to define an SDNS Canonical Name (CNAME) pool. The SDNS CNAME pool contains only one CNAME. When the SDNS CNAME pool is hit, the SDNS will return the CNAME to the local DNS if receiving a DNS CNAME query; the SDNS will further resolve the CNAME into IPv4 or IPv6 addresses if receiving a DNS A/AAAA query.

pool_name	This parameter specifies the name of the SDNS CNAME pool. Its value must be a string of 1 to 40 characters
cname	This parameter specifies a CNAME. Its value must be a string of 1 to 128 characters in the format of “www.xyz.com” and every section separated by “.” cannot be longer than 63 characters. The CNAME does not support the wildcard “*” or any other special characters and cannot end up with the character “.”.



Note: If the SDNS pool associated with the SDNS host name to be queried is an SDNS CNAME pool, the SDNS query will be re-executed based on the value of the CNAME associated with the SDNS CNAME pool, and the final query result is only related to the policy associated with the CNAME.

no sdns pool cname <pool_name>

This command is used to delete the specified SDNS CNAME pool.

show sdns pool cname

This command is used to display all the defined SDNS CNAME pools.

clear sdns pool cname

This command is used to clear all the defined SDNS CNAME pools.

sdns pool resort service <pool_name> <service_name>

Adds an SDNS service to a specified SDNS emergency pool. One SDNS service can be added to several SDNS emergency pools. A maximum of 100 IPv4 and 100 IPv6 SDNS services can be added to an SDNS emergency pool.

<code>pool_name</code>	This parameter specifies the name of the SDNS emergency pool. The value should be the name of the automatically generated SDNS emergency pool when the SDNS service pool is defined through the “ sdns pool name ” command.
<code>service_name</code>	This parameter specifies the name of an existing SDNS service.



Note: The SDNS emergency pool can only be associated with the SDNS domain name through the SDNS emergency policy.

no sdns pool resort service *<pool_name> <service_name>*

Deletes the SDNS emergency service from an SDNS emergency pool.

show sdns pool resort service *[pool_name]*

Displays all SDNS services in an SDNS emergency pool. If no value is not assigned to the **pool_name** parameter, the system will display all SDNS services in all SDNS emergency pools.

clear sdns pool resort service *<pool_name>*

Deletes all SDNS services from an SDNS emergency pool.

sdns pool fallback *<pool_name> <fallback_pool_name>*

This command is used to specify an SDNS fallback pool for the specified SDNS service pool. When no SDNS service IP is available in the SDNS service pool, the SDNS fallback pool will be used instead.

<code>pool_name</code>	This parameter specifies the name of the SDNS service pool.
<code>fallback_pool_name</code>	This parameter specifies the name of the SDNS fallback pool. Its value can be the name of an SDNS service or CNAME pool.



Note: Multiple SDNS service pools are allowed to form a tree structure of fallback. However, they are disallowed to form a fallback loop.

no sdns pool fallback *<pool_name>*

This command is used to delete an SDNS fallback pool configured for the specified SDNS service pool.

show sdns pool fallback

This command is used to display all configured SDNS fallback pools.

clear sdns pool fallback

This command is used to clear all configured SDNS fallback pools.

show sdns pool status [pool_name]

This command is used to display the status of the specified SDNS service or CNAME pool. If the optional parameter “pool_name” is not specified, the status of all SDNS service and CNAME pools will be displayed.

When a service pool contains both IPv4 and IPv6 SDNS services, running this command will display the status of different types of SDNS services.

Example:

```
Demo#show sdns pool status
p1 ipo none HITS:0
    A Record: Available
    1.1.1.1 UP 0
    1.1.1.2 UP 0
    1.1.1.3 DOWN 0

    AAAA Record: Not Available
    11::1 DOWN 0
    11::2 DOWN 0
    11::3 DOWN 0
```

SDNS Region**sdns region name <region_name>**

This command is used to define an SDNS region. A maximum of 100 regions can be defined in the system.

region_name This parameter specifies the name of the SDNS region. Its value is a string of 1 to 30 characters.

no sdns region <region_name>

This command is used to delete a specified SDNS region.

Before running this command, enable SDNS first (using the command **sdns on**).

show sdns region

This command is used to display all SDNS regions.

clear sdns region

This command is used to clear all SDNS regions.

Before running this command, enable SDNS first (using the command **sdns on**).

SDNS Policy

sdns policy region *<policy_name>* *<host_name>* *<pool_name>*
<region_name> [*priority*]

This command is used to define an SDNS region policy. Multiple SDNS region policies can be configured for the same SDNS host name.

policy_name	This parameter specifies the name of the SDNS region policy. Its value must be a string of 1 to 40 characters.
host_name	This parameter specifies an existing SDNS host name.
pool_name	This parameter specifies the name of an existing SDNS pool. It can be an SDNS service pool or an SDNS CNAME pool.
region_name	This parameter specifies the name of an existing SDNS region.
priority	Optional. This parameter specifies the priority of the SDNS region policy. Its value must be an integer ranging from 0 to 65,535. The greater the value, the higher the priority. The default value is 0.



Note:

- If the DNS query for the specified host name matches multiple SDNS region policies, only the region policy with the highest priority will be used.
- Do not configure region policies with the same priority for the same host name.

no sdns policy region *<policy_name>*

This command is used to delete the specified SDNS region policy.

show sdns policy region

This command is used to display all the defined SDNS region policies.

clear sdns policy region

This command is used to clear all the defined SDNS region policies.

sdns policy default *<host_name>* *<pool_name>*

This command is used to define an SDNS default policy. For every SDNS host name, only one SDNS default policy can be configured.

The SDNS default policy for the specified SDNS host name will take effect in any of the following situations:

- No SDNS region policy has been configured for or matches the specified SDNS host name.
- SDNS fails to choose any available SDNS service IP for the SDNS host name by using the matched SDNS region policy with the highest priority.

host_name This parameter specifies an existing SDNS host name.

pool_name This parameter specifies the name of an existing SDNS pool. It can be an SDNS service pool or an SDNS CNAME pool.

no sdns policy default <host_name>

This command is used to delete a specified SDNS default policy.

show sdns policy default

This command is used to display all the defined SDNS default policies.

clear sdns policy default

This command is used to clear all the defined SDNS default policies.

sdns policy resort <host_name> <pool_name>

This command is used to define an SDNS emergency policy that returns the service IP from the associated emergency pool when neither the region policy nor the default policy is hit. For each SDNS host name, only one SDNS emergency policy can be configured.

The SDNS emergency policy of the specified SDNS host name will take effect in any of the following situations:

- Neither SDNS region policy nor SDNS default policy is configured for or matched by the SDNS host name.
- The system cannot select an available IP for the specified SDNS host name using the SDNS region policy with the highest priority or the default policy.

It should be noted that when the SDNS emergency policy is hit, the system will return a service IP address from the associated emergency pool regardless of the service IP health status. If no service IP is matched, the system will look up the service IP from the SDNS fallback pool. If the system fails to match a service IP from the fallback pool, the system will return an empty DNS response.

This command can also be used to modify the configuration of an existing SDNS emergency policy.

host_name	This parameter specifies an existing SDNS host name.
pool_name	This parameter specifies the name of an existing SDNS emergency pool. It can be an SDNS emergency pool or an SDNS CNAME pool.



Note: Only the grr method can be used for emergency pools associated with SDNS emergency policies.

no sdns policy resort <host_name>

This command is used to delete the specified SDNS emergency policy.

show sdns policy resort

This command is used to display all SDNS emergency policies.

clear sdns policy resort

This command is used to clear all SDNS emergency policies.

SDNS Monitor

SDNS monitor can be used to perform health check on SDNS services and collect SNMP data from SDNS services. To use SDNS monitor, the administrator must first define the health check template or data collection template and then associate the template with an SDNS service or SDNS service pool to generate the monitor instance(s). The system will execute health check or data collection based on the monitor instances.

SDNS monitor supports the ICMP-type, TCP-type, HTTP-type and HTTPS-type health check templates and the SNMP-type data collection template. A maximum of 200 health check and data collection templates are supported. The type of the health check templates supports IPv4 and IPv6.

When a template is associated with an SDNS service, the system will generate a monitor instance for the SDNS service; while the template is associated with an SDNS service pool, the system will generate a monitor instance for every SDNS service in the service pool. The monitor instance generated based on the health check template is called the health check instance, and the monitor instance generated based on the SNMP data collection template is called the SNMP data collection instance. A maximum of 5000 monitor instances are supported and a maximum of 1024 SNMP data collection instances are supported.

sdns monitor icmp <template_name> [interval] [timeout] [max_retries]
[dst_address] [src_address] [gateway_address]

This command is used to define an ICMP-type health check template.

template_name	This parameter specifies the name of the health check template. Its value must be a string of 1 to 30 characters. The parameter value cannot contain white space and parentheses characters.
interval	Optional. This parameter specifies the interval of health checks. Its value must be an integer ranging from 3 to 86,400, in seconds. The default value is 5.
timeout	Optional. This parameter specifies the timeout value of the health check. Its value must be an integer ranging from 3 to 86,400, in seconds. The default value is 5. The timeout value should not be greater than the interval value.
max_retries	Optional. This parameter specifies the maximum number of consecutive health checks is allowed to set the status of the SDNS service as up or down. Its value must be an integer ranging from 1 to 10. The default value is 3. If the health check results are up for three consecutive times, the SDNS sets the status of the SDNS service as up. If the health check results are down for three consecutive times, the SDNS sets the status of the SDNS service as down.
dst_address	Optional. This parameter specifies the destination IP address of the ICMP health check packet. If this parameter is not specified, the SDNS service IP with which the template is associated will be used as the destination IP address.
src_address	Optional. This parameter specifies the source IP address of the ICMP health check packet. If this parameter is not specified, the system will choose an interface IP address as the source IP address.
gateway_address	Optional. This parameter specifies the IP address of the gateway from which the ICMP health check packet is transmitted. If this parameter is not specified, the system will automatically select the gateway IP address. In a multi-link environment, to ensure that the response of the ICMP health check packet can be sent back correctly, you are recommended to set the parameters “src_address” and “gateway_address”.

sdns monitor tcp <template_name> [interval] [timeout] [max_retries]
[dst_address] [dst_port] [src_address] [gateway_address]

This command is used to define a TCP-type health check template.

template_name	This parameter specifies the name of the health check template. Its value must be a string of 1 to 30 characters. The parameter value cannot contain white space and parentheses characters.
interval	Optional. This parameter specifies the interval of health checks. Its value must be an integer ranging from 3 to 86,400, in seconds. The default value is 5.
timeout	Optional. This parameter specifies the timeout value of the health check. Its value must be an integer ranging from 3 to 86,400, in seconds. The default value is 5.
max_retries	Optional. This parameter specifies the maximum number of consecutive health checks is allowed to set the status as up or down. Its value must be an integer ranging from 1 to 10. The default value is 3. If the health check results are up for three consecutive times, the SDNS sets the status of the SDNS service as up. If the health check results are down for three consecutive times, the SDNS sets the status of the SDNS service as down.
dst_address	Optional. This parameter specifies the destination IP address of the TCP health check packet. If this parameter is not specified, the SDNS service IP with which the template is associated will be used as the destination IP address.
dst_port	Optional. This parameter specifies the destination port of the TCP health check packet. Its value must be an integer ranging from 0 to 65,535. The default value is 0.
src_address	Optional. This parameter specifies the source IP address of the TCP health check packet. If this parameter is not specified, the system will choose an interface IP address as the source IP address.
gateway_address	Optional. This parameter specifies the IP address of the gateway from which the TCP health check packet is transmitted. If this parameter is not specified, the system will automatically select the gateway IP address. In a multi-link environment, to ensure that the response of the TCP health check packet can be sent back correctly, you are recommended to set the parameters “src_address” and

“gateway_address”.



Note: If the “dst_port” parameter in this command is 0, the system uses the port value configured in the command “**sdns service ip**”. The port parameters cannot be 0 at the same time in these two commands. When the port parameters are configured in both commands, the “dst_port” parameter in this command is adopted.

sdns monitor http <template_name> [request_line] [expected_response] [flag] [interval] [timeout] [max_retries] [dst_address] [dst_port] [src_address] [gateway_address]

This command is used to define an HTTP-type health check template.

template_name	This parameter specifies the name of the health check template. Its value must be a string of 1 to 30 characters and must be enclosed in double quotes. Letters, numbers, and special characters are supported. The parameter value cannot contain white space and parentheses characters.
request_line	Optional. This parameter specifies the HTTP request content, which includes HTTP method, URL path and HTTP version. Its value must be enclosed in double quotes. The default value is “HEAD / HTTP/1.0\r\n\r\n”.
expected_response	Optional. This parameter specifies the expected HTTP response. Its value must be enclosed in double quotes. The default value is “200 OK”.
flag	Optional. This parameter specifies the determined health check result when the expected HTTP response is received. Its value must be: • up: indicates that the system regards the SDNS service as Up. • down: indicates that the system regards the SDNS service as Down. The default value is “up”.
interval	Optional. This parameter specifies the interval of health checks. Its value must be an integer ranging from 3 to 86,400, in seconds. The default value is 5.
timeout	Optional. This parameter specifies the timeout value of the health check. Its value must be an integer ranging from 3 to 86,400, in seconds. The default value is 5.

The timeout value should not be greater than the interval

value.

max_retries Optional. This parameter specifies the maximum number of consecutive health checks allowed to set the status of the SDNS service as up or down. Its value must be an integer ranging from 1 to 10. The default value is 3.

If the health check results are up for three consecutive times, the system sets the status of the SDNS service as up. If the health check results are down for three consecutive times, the system sets the status of the SDNS service as down.

dst_address Optional. This parameter specifies the destination IP address of the HTTP request. If this parameter is not specified, the SDNS service IP with which the template is associated will be used as the destination IP address.

dst_port Optional. This parameter specifies the destination port of the HTTP request. Its value must be an integer ranging from 0 to 65,535. The default value is 0.

src_address Optional. This parameter specifies the source IP address of the HTTP request. If this parameter is not specified, the system will choose an interface IP address as the source IP address.

gateway_address Optional. This parameter specifies the IP address of the gateway from which the HTTP request is transmitted. If this parameter is not specified, the system will automatically select the gateway IP address. In a multi-link environment, to ensure that the HTTP response can be sent back correctly, you are recommended to set the parameters “src_address” and “gateway_address”.



Note: If the “dst_port” parameter in this command is 0, the system uses the port value configured in the command “**sdns service ip**”. The port parameters cannot be 0 at the same time in these two commands. When the port parameters are configured in both commands, the “dst_port” parameter in this command is adopted.

sdns monitor https <template_name> [request_line] [expected_response] [flag] [interval] [timeout] [max_retries] [dst_address] [dst_port] [src_address] [gateway_address]

This command is used to define an HTTPS-type health check template.

template_name This parameter specifies the name of the health check template. Its value must be a string of 1 to 30 characters and must be enclosed in double quotes. Letters, numbers,

	and special characters are supported. The parameter value cannot contain white space and parentheses characters.
request_line	Optional. This parameter specifies the HTTP request content, which includes HTTP method, URL path and HTTP version. Its value must be enclosed in double quotes. The default value is "HEAD / HTTP/1.0\r\n\r\n".
expected_response	Optional. This parameter specifies the expected HTTP response. Its value must be enclosed in double quotes. The default value is "200 OK".
flag	Optional. This parameter specifies the determined health check result when the expected HTTP response is received. Its value must be: • up: indicates that the system regards the SDNS service as Up. • down: indicates that the system regards the SDNS service as Down. The default value is "up".
interval	Optional. This parameter specifies the interval of health checks. Its value must be an integer ranging from 3 to 86,400, in seconds. The default value is 5.
timeout	Optional. This parameter specifies the timeout value of the health check. Its value must be an integer ranging from 3 to 86,400, in seconds. The default value is 5. The timeout value should not be greater than the interval value.
max_retries	Optional. This parameter specifies the maximum number of consecutive health checks allowed to set the status of the SDNS service as up or down. Its value must be an integer ranging from 1 to 10. The default value is 3. If the health check results are up for three consecutive times, the system sets the status of the SDNS service as up. If the health check results are down for three consecutive times, the system sets the status of the SDNS service as down.
dst_address	Optional. This parameter specifies the destination IP address of the HTTP request. If this parameter is not specified, the SDNS service IP with which the template is associated will be used as the destination IP address.

dst_port	Optional. This parameter specifies the destination port of the HTTP request. Its value must be an integer ranging from 0 to 65,535. The default value is 0.
src_address	Optional. This parameter specifies the source IP address of the HTTP request. If this parameter is not specified, the system will choose an interface IP address as the source IP address.
gateway_address	Optional. This parameter specifies the IP address of the gateway from which the HTTP request is transmitted. If this parameter is not specified, the system will automatically select the gateway IP address. In a multi-link environment, to ensure that the HTTP response can be sent back correctly, you are recommended to set the parameters “src_address” and “gateway_address”.



Note: If the “dst_port” parameter in this command is 0, the system uses the port value configured in the command “**sdns service ip**”. The port parameters cannot be 0 at the same time in these two commands. When the port parameters are configured in both commands, the “dst_port” parameter in this command is adopted.

sdns monitor snmp collector <template_name> <oid_name> [community]
[snmp_version] [interval] [dst_address] [dst_port] [src_address]

This command is used to define an SNMP-type data collection template for the specified SNMP OID.

template_name	This parameter specifies the name of the data collection template. Its value must be a string of 1 to 30 characters. The parameter value cannot contain white space and parentheses characters.
oid_name	This parameter specifies the name of the SNMP OID. Its value must be the name of the SNMP OID configured using the “ sdns snmp oid ” command.
community	Optional. This parameter specifies the password for the SNMP network administrator to access the system. Its value must be a string of 1 to 32 characters. The default value is “public”.
snmp_version	Optional. This parameter specifies the SNMP protocol version. Its value must be: • v1 • v2c

The default value is “v2c”.

interval	Optional. This parameter specifies the interval of data collection. Its value must be an integer ranging from 10 to 86,400, in seconds. The default value is 60.
dst_address	Optional. This parameter specifies the destination IP address of the SNMP request. If this parameter is not specified, every service IP in the SDNS service pool with which the template is associated will be used as the destination IP address.
dst_port	Optional. This parameter specifies the destination port of the SNMP request. Its value must be an integer ranging from 1 to 65,535, in seconds. The default value is 161.
src_address	Optional. This parameter specifies the source IP address of the SNMP request. If this parameter is not specified, the system will choose an interface IP address as the source IP address.

no sdns monitor <template_name>

This command is used to delete a configured health check template or data collection template.

show sdns monitor all

This command is used to display all configured health check templates and data collection templates.

clear sdns monitor

This command is used to clear all configured health check templates and data collection templates.

sdns service monitor apply <service_name> <template_name>

This command is used to associate an SDNS service with a specified health check template. When a template is associated with an SDNS service, the system will generate a monitor instance for the SDNS service. An SDNS service can be associated with a maximum of eight health check templates.

service_name	This parameter specifies the name of the SDNS service.
template_name	This parameter specifies the name of the health check template.



Note: The SNMP-type data collection template can be associated only with the SDNS service pool.

no sdns service monitor apply <service_name> <template_name>

This command is used to delete the association between the specified SDNS service and the specified health check template.

clear sdns service monitor apply [service_name]

This command is used to clear the associations between the specified SDNS service and all the health check and data collection templates. If the optional parameter “service_name” is not specified, all the associations between SDNS services and health check and data collection templates are cleared.

sdns pool monitor apply <pool_name> <template_name>

This command is used to associate an SDNS service pool with a health check template or data collection template. When a template is associated with an SDNS service pool, the system will generate a monitor instance for every member whose type is the same as that with this health check template's IP address in the service pool. In other words, if the type of the associated health check template is IPv4, the system generates instances only for IPv4 SDNS services. If the type of the associated health check template is IPv6, the system generates instances only for IPv6 SDNS services.

An SDNS service pool can be associated with a maximum of 16 health check or data collection templates (a maximum of eight IPv4 health check templates and eight IPv6 ones).

pool_name	This parameter specifies the name of the SDNS service pool.
-----------	---

template_name	This parameter specifies the name of the health check template or data collection template.
---------------	---

no sdns pool monitor apply <pool_name> <template_name>

This command is used to delete the association between the specified SDNS service pool and the specified health check template or data collection template.

clear sdns pool monitor apply [pool_name]

This command is used to clear the associations between the specified SDNS service pool and all the health check and data collection templates. If the optional parameter “pool_name” is not specified, all the associations between SDNS service pools and health check and data collection templates are cleared.

show sdns monitor instance [monitor_type]

This command is used to display the monitor instances of the specified type.

monitor_type	Optional. This parameter specifies the type of the monitor instance. Its value must be:
--------------	---

- icmp: indicates the ICMP-type health check instance.
- tcp: indicates the TCP-type health check instance.
- http: indicates the HTTP-type health check instance.
- https: indicates the HTTPS-type health check instance.
- snmp: indicates the SNMP-type data collection instance.

If this parameter is not specified, monitor instances of all types will be displayed.

sdns service health relation <service_name> [relation]

This command is used to set the relationship among the health check instances of the specified SDNS service pool.

service_name	This parameter specifies the name of the SDNS service.
relation	Optional. This parameter specifies the relationship among health check instances of the specified SDNS service. Its value must be: • and: indicates that the system will regard an SDNS service as Up only when all health check instances of the SDNS service is Up. • or: indicates that the system will regard an SDNS service as Up when any health check instance of the SDNS service is Up.

The default value is “and”.

show sdns service health relation [service_name]

This command is used to display the relationship (and/or) among health check instances of the specified SDNS service. If the optional parameter “service_name” is not specified, the relationship among health check instances of all SDNS services will be displayed.

sdns pool health relation <pool_name> [relation]

This command is used to set the relationship among the health check instances of a specified SDNS service pool.

When an SDNS service pool is associated with both IPv4 and IPv6 health check templates, the system sets the relationship among the health check instances for the IPv4 and IPv6 SDNS services in the SDNS service pool.

pool_name	This parameter specifies the name of the SDNS service
-----------	---

	pool.
relation	Optional. This parameter specifies the relationship among health check instances of the specified SDNS service pool. Its value must be: • and: indicates that the system will regard a service pool member as Up only when all health check instances of the service pool member is Up. • or: indicates that the system will regard a service pool member as Up when any health check instance of the service pool member is Up. The default value is “and”.



Note: The results of the SNMP data collection instances associated with every member in the SDNS service pool do not affect the health check result of the pool member.

show sdns pool health relation [pool_name]

This command is used to display the relationship (and/or) among health check instances of a specified SDNS pool. If the optional parameter “pool_name” is not specified, the relationship among health check instances of all SDNS service pools will be displayed.

sdns pool health mixrelation <pool_name> <expression>

This command is used to set the mixed relationship among the health check instances of each service IP in a specified SDNS service pool.

This command can be used to modify the mixed relationship in an SDNS service pool.

pool_name	This parameter specifies the name of the SDNS service pool.
expression	This parameter specifies the relation expression used to judge the health status of each service IP in the specified SDNS service pool. The expression must be enclosed in double quotes. The expression syntax should be like: (monitor group1) OR/AND (monitor group2) OR/AND (monitor group3)... One bracket in the expression represents one health check template group. The system supports a maximum of 10 groups (a maximum of five IPv4 health check template groups and five IPv6 ones). The relationship of the groups must be AND or OR like “(m1) OR (m2 OR m3) AND (m4).” Expressions such as “m1 OR m2” or “((m1 OR m2)

AND m3)” are illegal.

A maximum of 8 health check templates are allowed to be defined in each group. Different health check template groups can use the same health check template.

The health check templates in the expression should be applied to the specified service pool. Once the health check template is deleted, its related configuration in the expression will be deleted as well.

In addition, when an SDNS service pool is associated with both IPv4 and IPv6 health check templates, the mixed relationship of the SDNS service pool needs to be separated by comma (.). For example, **sdns pool health mixrelation pool_1 “(m1)OR(m2 OR m3),(m4)AND(m5 OR m6)”**, in which m1, m2, and m3 are IPv4 health check templates, and m4, m5, and m6 are IPv6 health check templates.

This parameter's value supports the following syntax:

- The relationship of the IPv4 health check template groups, and that of the IPv6 health check template groups.

Syntax: (m1)OR(m2 OR m3),(m4)AND(m5 OR m6)

- The relationship of the IPv4 health check template groups.

Syntax: (m1)OR(m2 OR m3)

- The relationship of the IPv6 health check template groups.

Syntax: (m4)OR(m5 OR m6)



Note: If the mixed relationship is defined for the service pool, the instance relationship (AND or OR) defined in “**sdns pool health relation**” command will not take effect any longer.

no sdns pool health mixrelation <pool_name>

This command is used to delete the mixed relationship of health check instances for every service IP in the specified service pool.

show sdns pool health mixrelation [pool_name]

This command is used to show the mixed relationship of health check instances for the specified service pool. If the optional parameter “pool_name” is not specified, the

relationship among health check instances of all SDNS service pools will be displayed.

SDNS Static and ipregion Proximity Rules

sdns proximity <source_ip> <netmask|prefix> <region_name>

This command is used to add a static SDNS proximity rule for the specified SDNS region. The priority of static SDNS proximity rules (including IP region proximity rules) is higher than that of dynamic proximity rules. The system supports a maximum of 250,000 SDNS proximity rules, including all the static and dynamic proximity rules.

source_ip	This parameter specifies the IP address on a subnet. Its value can be an IPv4 or IPv6 address.
netmask prefix	This parameter specifies the netmask or prefix length of the subnet. Parameters “source_ip” and “netmask prefix” determine a subnet. • “netmask” is used for an IPv4 address. Its value must be a dotted IP address or an integer. If it is an integer, its value must be an integer ranging from 0 to 32. • “prefix” is used for an IPv6 address. Its value must be an integer ranging from 0 to 128.
region_name	This parameter specifies the name of an existing SDNS region.



Note:

- SDNS uses the longest prefix match rule to choose the region that most matches the IP address of the local DNS.
- The subnet determined by parameters “source_ip” and “netmask|prefix” can be used for static SDNS proximity rules of multiple SDNS regions. However, to ensure that only one SDNS region policy with the highest priority can be hit for an SDNS host name, please configure region policies with different priorities.

no sdns proximity <source_ip> <netmask|prefix> <region_name>

This command is used to delete a specified static SDNS proximity rule.

show sdns proximity static

This command is used to display all configured static SDNS proximity rules.

clear sdns proximity

This command is used to clear all configured static SDNS proximity rules.

sdns ipregion proximity <ipregion_name> <region_name>

This command is used to associate an IP region table with a specified SDNS region. When the IP region table and the SDNS region are associated, SDNS IP region proximity rules will be generated. The system supports a maximum of 50 IP region proximity rules.

<code>ipregion_name</code>	This parameter specifies the name of an existing ipregion table.
<code>region_name</code>	This parameter specifies the name of an existing SDNS region.

no sdns ipregion proximity <ipregion_name> <region_name>

This command is used to delete the association between the specified SDNS ipregion table and SDNS region.

show sdns ipregion proximity

This command is used to display the associations between all SDNS ipregion tables and SDNS regions.

clear sdns ipregion proximity

This command is used to clear the associations between all SDNS ipregion tables and SDNS regions.

show sdns proximity ipregion

This command is used to display all the SDNS ipregion proximity rules.

show sdns proximity match <local_dns_ip>

This command is used to manually query which SDNS region a local DNS will match.

<code>local_dns_ip</code>	This parameter specifies the IP address of a local DNS. Its value must be an IPv4 or IPv6 address.
---------------------------	--

SDNS DPS

sdns dps {on|off}

This command is used to enable or disable the Dynamic Proximity System (DPS) function. By default, this function is disabled.

The DPS is comprised of DPS servers and DPS detectors. The DPS server mainly provides the following functions:

- Collect the IP address list of local DNSs and send them to DPS detectors for detection.

- Collect the proximity detection results from DPS detectors and generate dynamic proximity rules based on the proximity detection results.

The DPS detector mainly provides the following functions:

- Receive the IP address list of local DNSs from DPS servers.
- Send detection packets to local DNSs to obtain three types of proximity information: Round Trip Time (RTT), Packet Loss Rate (PLR) and hops.
- Report the proximity detection results to the DPS server.

When the DPS function is enabled, the APV appliance will become a DPS server. The DPS server send the IP address list of local DNSs only to DPS detectors configured on the same APV appliance.

sdns dps interval send *[interval]*

This command is used to set the interval at which the DPS server sends a list of collected local DNS IP addresses to DPS detectors.

interval	Optional. This parameter specifies the interval. Its value must be an integer ranging from 1 to 2,147,483,647, in seconds. The default value is 120.
----------	--

sdns dps interval query *[interval]*

This command is used to set the configured interval at which the DPS server queries the proximity detection results from the DPS detectors.

interval	Optional. This parameter specifies the interval. Its value must be an integer ranging from 1 to 2,147,483,647, in seconds. The default value is 1200.
----------	---

sdns dps detector <region_name> <ip_address> [dps_port] [detect_interval] [cache_time]

This command is used to configure the DPS detector for the specified SDNS region. Only one DPS detector is supported for every SDNS region. A maximum of 128 DPS detectors can be configured.

region_name	This parameter specifies the name of an existing SDNS region.
-------------	---

ip_address	This parameter specifies the IP address of the DPS detector. Its value must be an IPv4 or IPv6 address.
------------	---

When it is set to an interface's IP address, the APV appliance itself is used as the DPS detector for the specified the SDNS region. However, to use the APV appliance as a DPS detector, you must execute the “**sdns**

dps localdetector” command to start a local DPS detector daemon.

dps_port	Optional. This parameter specifies DPS communication port for the DPS detector to communicate with the DPS servers. Its value must be an integer ranging from 1 to 65,535. The default value is 44,544.
detect_interval	Optional. This parameter specifies the detection interval of the DPS detector. Its value must be an integer ranging from 5 to 2,147,483,647, in seconds. The default value is 900.
cache_time	Optional. This parameter specifies the length of time that the DNS IP address list received from the DPS server will be cached on the DPS detector. Its value must be an integer ranging from 1 to 2,147,483,647, in seconds. The default value is 172,800.



Note: If “ip_address” is set to “0.0.0.0” or “::” in the “sdns dps localdetector” command, the “ip_address” parameter in this command should be set to an interface’s IP address to use the local DPS detector daemon as a DPS detector.

no sdns dps detector <region_name>

This command is used to delete the DPS detector configured for the specified SDNS region.

show sdns dps detector

This command is used to display all configured DPS detectors.

clear sdns dps detector

This command is used to clear all configured DPS detectors.

sdns dps localdetector <detector_name> <ip_address> [detect_port] [dps_port] [detect_timeout] [gateway_ip] [dps_ip]

This command is used to start a local DPS detector daemon on the APV appliance. A maximum of 8 local DPS detector daemons can be started on the APV appliance.

detector_name	This parameter specifies the name of the local DPS detector daemon to be started. Its value must be a string of 1 to 30 characters.
ip_address	This parameter specifies the IP address on which the local DPS detector daemon listens. Its value must be an interface’s IP address. “0.0.0.0” indicates listening on all IPv4 addresses and “::” indicates listening on all IPv6 addresses.

<code>detect_port</code>	Optional. This parameter specifies the source port for the local DPS detector daemon to send detection packets to local DNSs. Its value must be an integer ranging from 1025 to 65,535. The default value is 53,455.
<code>dps_port</code>	Optional. This parameter specifies the listen port for the local DPS detector daemon to communicate with the DPS servers. Its value must be an integer ranging from 1025 to 65,535. The default value is 44,544.
<code>detect_timeout</code>	Optional. This parameter specifies the timeout value for detecting the local DNS. Its value must be an integer ranging from 1 to 2,147,483,647, in seconds. The default value is 30.
<code>gateway_ip</code>	Optional. This parameter specifies the IP address of the gateway that forwards the detection packets. Its value can be an IPv4 or IPv6 address. The default value is “0.0.0.0” or “::”, indicating that the APV appliance will automatically select the gateway.
<code>dps_ip</code>	Optional. This parameter specifies the IP address used by the DPS detector to communicate with the DPS server. When this parameter is specified, the IP address defined in “ip_address” will only be used to detect the local DNS server and this IP address is used to communicate with the DPS server. When this parameter is not specified, the IP address defined in “ip_address” will be responsible for both the communication with the DPS server and the detection of local DNS servers. That means, the default value of this parameter is the same as the “ip_address” parameter value. To use the local DPS detector, the “ip_address” parameter in the “sdns dps detector” command should be the same with setting of the “dps_ip” parameter in this command.

**Note:**

- The values of the parameters “detect_port” and “dps_port” in this command must be different.
- The value of the “dps_port” parameter in this command must be the same as that of the “dps_port” parameter in the corresponding “**sdns dps detector**” configuration.
- The values of the parameters “detect_port” and “dps_port” for every local DPS detector daemon must be unique among all local DPS detector daemons.

no sdns dps localdetector <detector_name>

This command is used to stop a specified local DPS detector daemon on the APV appliance.

show sdns dps localdetector

This command is used to display all started local DPS detector daemons and their status.

clear sdns dps localdetector

This command is used to stop all started local DPS detector daemons.

sdns dps method <method> [weight_of_rtt] [weight_of_plr] [weight_of_hops]

This command is used to configure the DPS method, which is used by the DPS server to calculate “metric” of the path between a local DNS and every SDNS region (DPS detector). The DPS server will find the path with the smallest metric and generate a dynamic proximity rule for the local DNS and the SDNS region.

method	This parameter specifies the name of the DPS method. Its value must be: • rtt: Metric is equal to RTT. • plr: Metric is equal to PLR. • hops: Metric is equal to the number of the hops. • mix: Metric is equal to the mixed value (weight x RTT + weight x PLR + weight x hops).
--------	---

The default value is “rtt”.

weight_of_rtt	Optional. This parameter specifies the weight of the RTT value and can be set only when the method is “mix”. Its value must be an integer ranging from 0 to 9 and the default value is 1.
---------------	---

weight_of_plr	Optional. This parameter specifies the weight of the PLR value and can be set only when the method is “mix”. Its value must be an integer ranging from 0 to 9 and the default value is 1.
---------------	---

weight_of_hops	Optional. This parameter specifies the weight of the hops value and can be set only when the method is “mix”. Its value must be an integer ranging from 0 to 9 and the default value is 1.
----------------	--



Note: The values of “weight_of_rtt”, “weight_of_plr”, and “weight_of_hops” cannot be all 0s.

show sdns dps method

This command is used to display the DPS method.

show sdns proximity dynamic

This command is used to display all dynamic proximity rules.

clear sdns dps history

This command is used to clear the DPS history data, including dynamic proximity rules and proximity results received from DPS detectors.

sdns dps expire [expire_time]

This parameter is used to specify the expiration time of dynamic proximity rules. If the dynamic proximity rules expire, they will be cleared.

interval	Optional. This parameter specifies expiration time of dynamic proximity rules. Its value must be an integer ranging from 5 to 2,147,483,647, in seconds. The default value is 1500.
----------	---

show sdns dps expire

This parameter is used to display the expiration time of dynamic proximity rules.

sdns dps write proximity file <file_name>

This command is used to save all dynamic proximity rules into a local file on the APV appliance.

file_name	This parameter specifies the name of the local file used to save the dynamic proximity rules.
-----------	---

sdns dps write proximity scp {remote_server_ip|name} <user_name> <file_path> <file_name>

This command is used to save all dynamic proximity rules into a file on the remote SCP server.

remote_server_ip name	This parameter specifies the IP address or name of the remote SCP server.
user_name	This parameter specifies the username used to access the remote SCP server. After the username is entered, you will be prompted to enter the password.
file_path	This parameter specifies the file path where the file saving the dynamic proximity rules is stored on the remote SCP server.

`file_name` This parameter specifies the name of the file used to store the dynamic proximity rules on the remote SCP server.

sdns dps write proximity tftp <remote_server_ip> <file_name>

This command is used to save all dynamic proximity rules into a file on the remote TFTP server.

`remote_server_ip` This parameter specifies the IP address of the remote TFTP server.

`file_name` This parameter specifies the name of the file used to save the dynamic proximity rules on the remote TFTP server.

show sdns dps path [/local_dns_ip]

This command is used to display the RTT, hops and PLR between the specified local DNS and every DPS detector (SDNS region).

`local_dns_ip` Optional. This parameter specifies the IP address of a local DNS. Its value must be an IPv4 or IPv6 address. “0.0.0.0” indicates all IPv4 local DNSs and “::” indicates all IPv6 local DNSs. By default, this parameter is not specified, indicating that the RTT, hops and PLR between every local DNS and every DPS detector will be displayed.

show sdns dps status

This command is used to display the status of the DPS server and local DPS detectors on the APV appliance.

For example:

```
AN(config)#show sdns dps status
```

```
SDNS DPS service is ON
```

Local Detector	Status
detect1	UP
detect2	UP

show sdns dps all

This command is used to display all SDNS DPS configurations.

clear sdns dps all

This command is used to reset all SDNS DPS configurations to default values.

DNS Resource Record

sdns record ip <record_name> <domain_name> <ip_address>

This command is used to add a AAA or AAAA resource record for the specified domain name and define a name for it.

record_name	This parameter specifies the resource record name. Its value must be a string of 1 to 50 characters.
domain_name	This parameter specifies the domain name. Its value must be a string of 1 to 128 characters, in the “xyz.com” format, and cannot end with a dot.
ip_address	This parameter specifies the IP address of the domain name. Its value must be an IPv4 or IPv6 address.

sdns record ns <record_name> <domain_name1> <domain_name2>

This command is used to add an NS resource record for the specified domain name and define a name for it. An NS resource record defines the authoritative DNS server (domain_name2) for the specified domain name (domain_name1). Therefore, the corresponding A or AAAA resource record must be available on the authoritative DNS server.

record_name	This parameter specifies the resource record name. Its value must be a string of 1 to 50 characters.
domain_name1	This parameter specifies the DNS domain name. Its value must be a string of 1 to 128 characters, in the format of “xyz.com”, and it is allowed to end the domain name with a dot or a colon.
domain_name2	This parameter specifies the domain name of the authoritative DNS server.

sdns record ptr <record_name> <ip_address> <domain_name>

This command is used to add a PTR resource record for the specified IP address and define a name for it. PTR resource records are used for reverse domain name resolution. That is, an IP address will be resolved to a domain name.

record_name	This parameter specifies the resource record name. Its value must be a string of 1 to 50 characters.
ip_address	This parameter specifies the IP address of the domain name. Its value must be an IPv4 or IPv6 address. IP addresses in abbreviated form are not supported.

domain_name	This parameter specifies the domain name. Its value must be a string of 1 to 128 characters, in the “xyz.com” format, and cannot end with a dot.
-------------	--

sdns record mx <record_name> <domain_name1> <domain_name2> <priority>

This command is used to add an MX resource record for the specified domain name and define a name for it. An MX resource record directs an email address ended with the domain name (domain_name1) to the specified mail server (domain_name2) for processing.

record_name	This parameter specifies the resource record name. Its value must be a string of 1 to 50 characters.
domain_name1	This parameter specifies the domain where the mail server resides in.
domain_name2	This parameter specifies the domain name of the mail server.
priority	This parameter specifies the priority of the MX resource record. Its value must be an integer in the range of 0 to 65535. A smaller value indicates a higher priority.

sdns record txt <record_name> <domain_name> <description1> [description2] [description3]

Adds a TXT resource record to the specified domain name and defines a name for the resource record. TXT resource records are text-based DNS records stored in DNS zone files in TXT format, used to retrieve information about domain names.

A DNS zone supports configuring several TXT resource records and doesn't support adding TXT resource records to a DNS zone of PTR type.

To make TXT resource records effective, at least one NS record and one A/AAAA record needs to be added a DNS zone, and only when the domain name of the NS/MX record in the DNS zone is the same as the A/AAAA domain name can an A/AAAA record be added to the zone. To use TXT records, SDNS (using the **sdns on** command) and the full DNS service of SDNS (using the **sdns fulldns on** command) need to be enabled first.

record_name	This parameter specifies a resource record's name. Its value must be a string with a length not greater than 50 bytes.
domain_name	This parameter specifies a domain name corresponding to a TXT resource record. Its value must be a string with a length not greater than 128 bytes. The format is “xyz.com.”

Note: This parameter must be a DNS zone name (using the command “**sdns zone name**”) or a DNS zone’s subdomain supporting multilevel subdomains’ names.

description1

This parameter specifies the content of a TXT record. Its value must be the string with a length of 1-255 bytes (spaces are also characters so their lengths are counted). The value can be numbers, uppercase and lowercase letters, and special characters. If the value contains only numbers or special characters, it must be enclosed in double quotes.

This parameter supports only the following special characters:

period (.), asterisk (*), slash (/), hyphen (-), colon (:), underscore (_), question mark (?), equal sign (=), at sign (@), comma (,), backslash (\), and space ().

Notice that if the parameter’s value is a string that contains “\number” (the number is 0-9), this setting and all subsequent settings in this DNS zone won’t take effect. After SDNS service is restarted, all settings in the DNS zone still don’t take effect. SDNS will work properly only if this setting is deleted. We suggest you avoid setting the parameter’s value to a string that contains “\number.”

description2

Optional. This parameter specifies the content of a TXT record. For further details, see the parameter “description1.” The default value is empty.

Note: When the parameter “description3” is not empty, its value cannot be empty either. That is, the command “**sdns record txt txt1 www.abc.com “d1” “” “d3”**” cannot be executed.

description3

Optional. Its value is the same as description2.

no sdns record <record_name>

This command is used to delete the specified resource record.

show sdns record

This command is used to display all resource records that have been added.

clear sdns record all

This command is used to clear all resource records that have been added.

sdns zone name <zone_name>

This command is used to create a DNS zone.

zone_name	This parameter specifies the DNS zone name. Its values must be a DNS domain name or an IP prefix, in the range of 1 to 128 characters. When a domain name is set, every section separated by “.” cannot be longer than 63 characters. DNS zones for non-PTR resource records must be defined by domain names, such as “www.a.com”, while DNS zones for PTR resource records can be defined by IP prefixes only. If an IPv4 prefix is defined, it must be in the dotted decimal format and ended with a dot, for example, “10.8.6.”. If an IPv6 prefix is defined, it must be in the hexadecimal format separated by and ended with a colon, for example, “fff:”.
-----------	--

no sdns zone name <zone_name>

This command is used to delete the specified DNS zone.

show sdns zone name

This command is used to display all DNS zones that have been created.

clear sdns zone name all

This command is used to clear all DNS zones.

sdns zone record <zone_name> <record_name>

This command is used to add the specified resource record to the specified DNS zone. Each DNS zone should contain at least an NS and an A or AAAA resource record.

zone_name	This parameter specifies the name of an existing DNS zone.
-----------	--

record_name	This parameter specifies the name of an existing DNS resource record. It must be a subdomain name of the DNS zone or an IP address with a longer netmask.
-------------	---



Note: To add an NS resource record to a PTR DNS zone, the value of the “domain_name1” parameter in the “**sdns record ns**” command must be the same as the name of the PTR DNS zone.

no sdns zone record <zone_name> <record_name>

This command is used to remove a resource record from the specified DNS zone.

show sdns zone record [zone_name]

This command is used to display the resource records contained in the specified DNS zone.

zone_name Optional. This parameter specifies the name of an existing DNS zone. When “all” is specified, resource records contained in all DNS zones will be displayed.

The default value is “all”.

clear sdns zone record <zone_name>

This command is used to clear all resource records from the specified DNS zone.

zone_name This parameter specifies the name of an existing DNS zone. When “all” is specified, this command will clear resource records from all DNS zones.

SDNS Full DNS

sdns fulldns {on|off}

This command is used to enable or disable the SDNS full DNS function. By default, this function is enabled.

When this function is disabled, the system will parse only domain names of the A, AAAA and CNAME types. For unresolvable domain names, the system will return an error code.

When this function is enabled, the system can parse domain names of all types. For unresolvable domain names, if “**sdns recursion on**” is configured, the system will send the query to the BIND server for resolution.

sdns recursion {on|off}

This command is used to enable or disable the SDNS Recursive Query function. This function takes effect only after the SDNS full DNS function is enabled. By default, this function is disabled.

SDNS IANA

sdns iana import <url>

This command is used to import an Internet Assigned Numbers Authority (IANA) file.

url This parameter specifies the HTTP or FTP URL address for the IANA file.

show sdns iana <ip_address>

This command is used to query which country or area the specified IP address belongs to.

ip_address This parameter specifies the IP address. Its value must be an IPv4 address.

SDNS DNSSEC

sdns dnssec {on|off}

This command is used to enable or disable the DNSSEC function for SDNS. By default, the function is disabled.



Note: SDNS DNSSEC function only supports the encryption of A and AAAA Resource Record, CNAME Resource Record encryption is not supported by now.

sdns dnssec keygen <host_name> <type> [algorithm] [size]

This command is used to generate a Zone Signing Key (ZSK) or a Key Signing Key (KSK) for the specified SDNS host name. ZSK is used to sign all the Resource Records, while KSK is only responsible for signing the ZSK.

host_name This parameter specifies an existing SDNS host name for which a ZSK or KSK will be generated.

type This parameter specifies the type of signing key to be generated. Its value must be:

- ZSK: Zone Signing Key
- KSK: Key Signing Key

algorithm Optional. This parameter specifies the signing algorithm used by ZSK or KSK. Its value must be:

- RSASHA1
- RSASHA256
- RSAMD5
- NSEC3RSASHA1

The default value is RSASHA1.

size Optional. This parameter specifies the key length of the generated ZSK or KSK. Its value must be an integer ranging from 512 to 2048.

The default value is 1024.

no sdns dnssec keygen [*host_name*]

This command is used to delete the DNSSEC signing keys for the specified SDNS host name. When the parameter is not specified, the signing keys of all SDNS host names will be deleted.

show sdns dnssec keygen [*host_name*]

This command is used to display all the DNSSEC signing keys of the specified SDNS host name. When the parameter is not specified, the signing keys of all SDNS host names will be displayed.

sdns dnssec keypub <*host_name*> <*type*> <*keyfile_name*> [*rollover_time*] [*expiration_time*] [*TTL*]

This command is used to publish the ZSK or KSK signing key of the specified SDNS host name onto the DNS server (APV appliance). After the signing key is published, the Resource Records of the specified SDNS host name can be signed by the signing key.

host_name	This parameter specifies an existing SDNS host name.
type	This parameter specifies the type of signing key to be generated. Its value must be: • ZSK: Zone Signing Key • KSK: Key Signing Key
keyfile_name	This parameter specifies the file name of the signing key (viewed by using the “ show sdns dnssec keygen ” command) to be published. Its value must be enclosed in double quotes.
rollover_time	Optional. This parameter specifies the period that the system renews the ZSK signing key, in days. The default ZSK refresh cycle is 25 days.
expiration_time	Optional. This parameter specifies the time span, in days, after which the system deletes an expired ZSK signing key. When the signing is expired, the system will automatically delete the expired ZSK signing key. The value of this parameter must be greater than the value of the “rollover_time” parameter. The recommended ZSK expiration time is 30 days. The default ZSK expiration time is 30 days.

TTL Optional. This parameter specifies the time span, in minutes, after which the cached signing key is expired. To avoid potential conflicts, the value of this parameter must be less than the difference between the values of “rollover_time” and “expiration_time”.

The default value is 60 minutes.

no sdns dnssec keypub [host_name]

This command is used to delete the signing keys for the specified SDNS host name. When the parameter is not specified, the signing keys of all SDNS host names will be deleted.

show sdns dnssec keypub [host_name]

This command is used to display the signing keys for the specified SDNS host name. When the parameter is not specified, the signing keys of all SDNS host names will be displayed.

sdns dnssec zonesign <host_name>

This command is used to sign the Resource Records for the specified host name. When the signing process is finished, an RRSIG record will be created for each Resource Record.

host_name This parameter specifies an existing SDNS host name.

no sdns dnssec zonesign [host_name]

This command is used to delete all the RRSIG records and signing keys for the specified SDNS host name. When the parameter is not specified, the RRSIG records and signing keys of all SDNS host names will be deleted.

show sdns dnssec zonesign [host_name]

This command is used to display the Resource Records signing information of the specified SDNS host name. When the parameter is not specified, all the SDNS host names whose Resource Records have been signed will be displayed.

show sdns dnssec ds import [host_name]

This command is used to display the imported Delegation Signer (DS) file of the specified SDNS host name. The DS file aims to verify the legitimacy of the signing key for the purpose of establishing DNSSEC trust chain. The DS file must be stored on the upper-level DNS server. If the “host_name” parameter is not specified, the imported DS files of all SDNS host names on the DNS server will be displayed.

host_name Optional. This parameter specifies an existing SDNS host

name.

show sdns dnssec ds generate [host_name]

This command is used to display the generated DS file of the specified SDNS host name. If the “host_name” parameter is not specified, the generated DS files of all SDNS host names on the DNS server will be displayed.

host_name Optional. This parameter specifies an existing SDNS host name.

sdns dnssec export ds <filename> <url>

This command is used to export the specified DS file to the specified URL address.

filename This parameter specifies the name of an DS file (viewed by using the “**show sdns dnssec keygen**” command) to be exported. Its value must be enclosed in double quotes.

URL This parameter specifies the URL address (only FTP address is supported) to which the DS file will be exported.

sdns dnssec import ds <host_name> <url>

This command is used to import the DS file from the specified URL address.

host_name This parameter specifies the host name for which the DS file is imported. Its value must be enclosed in double quotes.

URL This parameter specifies the URL address (only FTP or HTTP address is supported) in which the DS file is located.



Note: The imported DS file cannot be exported or deleted by now.

sdns dnssec export key <type> <keyfile_name> <url>

This command is used to export the specified signing key to the specified URL. DNSSEC adopts the public key encryption mechanism, so a pair of public and private keys for the specified signing key need to be exported.

type This parameter specifies the type of signing key to be generated. Its value must be:

- ZSK: Zone Signing Key
- KSK: Key Signing Key

keyfile_name	This parameter specifies the signing key (viewed by using the “show sdns dnssec keygen” command) to be exported. Its value must be enclosed in double quotes. Add the extension “.key” to the signing key name to export the public signing key, and add “.private” to export the private signing key. For example, to export the public ZSK named “Kwww.xyz.com.+005+46666+zsk+RSASHA1+1024”, you need to specify the key file name as “Kwww.xyz.com.+005+46666+zsk+RSASHA1+1024.key”.
URL	This parameter specifies the URL address (only FTP address is supported) to which the signing key will be exported. Note: The “path” of the FTP URL (ftp://user:password@host:port/path) should be a relative path.

sdns dnssec import key <type> <keyfile_name> <url>

This command is used to import the specified signing key from the specified URL address.

type	This parameter specifies the type of signing key to be generated. Its value must be: • ZSK: Zone Signing Key • KSK: Key Signing Key
keyfile_name	This parameter specifies the file to import the signing key. Its value must be enclosed in double quotes.
URL	This parameter specifies the URL address (only FTP or HTTP address is supported) in which the signing key is located. Note: The “path” of the FTP URL (ftp://user:password@host:port/path) should be a relative path.

show sdns dnssec all

This command is used to display all the configuration information of SDNS DNSSEC.

clear sdns dnssec all

This command is used to delete all the configuration information of SDNS DNSSEC.

show sdns dnssec status <host_name>

This command is used to display the DNSSEC status information of the specified SDNS host name.

Example:

```
AN(config)#show sdns dnssec status "www.abc.com"
host status flags: HOST_PUBLISH|HOST_KEY|HOST_SIGN
ttl: 60(s)
zsk key counter: 1
ksk key counter: 1
ZSK key(1):
    keyfile: Kwww.abc.com.+005+43368+zsk+RSASHA1+1024
    keyid: 43368
    type: ZSK
    algorithm: RSASHA1
    status_flags: KEY_PUBLISH|KEY_SIGN|KEY_TIMER
    ttl: 1(hour)
    ex_time: 2(days)
    ro_time: 1(days)
KSK key(0):
    keyfile: Kwww.abc.com.+005+07177+ksk+RSASHA1+1024
    keyid: 7177
    type: KSK
    algorithm: RSASHA1
    status_flags: KEY_PUBLISH|KEY_SIGN|NO
    ttl: 1(hour)
    ex_time: 30(days)
    ro_time: 25(days)
```

The “host status flags” item in the output displays the DNSSEC information of the specified host name. The parameter in the “host status flags” is as follows:

- **HOST_PUBLISH:** means that the signing keys have been published for the host name.
- **HOST_KEY:** means that DNSKEY record has been existed for the host name.
- **HOST_SIGN:** means that RRSIG signing records have been created for the host name.
-
- The parameter of “status_flags” in the signing key section is as follows:
 - **KEY_PUBLISH:** means that the signing key has been published.
 - **KEY_SIGN:** means that Resource Record has been signing by this signing key.
 - **KEY_TIMER:** means that the signing key has a refresh time span.

SDNS Data Center Synchronization

sdns dc <dc_name> <ip_address> [port] [local|remote]

This command is used to define an SDNS data center. The system supports up to 64 data centers.

dc_name	This parameter specifies the SDNS data center name. Its value must be a string of 1 to 50 characters.
ip_address	This parameter specifies the IP address of the data center, which can be IPv4 or IPv6. Note: The data center IP address cannot be the same with any SLB virtual service IP address.
port	Optional. This parameter specifies the listening port of the data center. Its value must be an integer ranging from 1025 to 65,000. The default value is 60,000.
local remote	Optional. This parameter specifies the local or remote data center. Its value must be: • local: indicates the local data center. • remote: indicates the remote data center. The default value is “remote”. Note that each data center can have only one local data center.

no sdns dc <dc_name>

This command is used to delete the specified SDNS data center.

show sdns dc name [dc_name]

This command is used to display the configuration of the specified SDNS data center. If the “dc_name” parameter is not specified, the configurations of all SDNS data centers will be displayed.

Example:

```
Demo(config)#show sdns dc name
sdns dc "dc1" 192.168.93.100 60000 "local"
sdns dc "dc2" 192.168.93.200 60000 "remote"
```

show sdns dc status [dc_name]

Displays the status of an SDNS data center (online or offline). If no value is assigned to the **dc_name** parameter, the status of all SDNS data centers will be displayed.

Example:

```
Demo(config)#show sdns dc status
dc1 192.168.93.100 60000 local|backup offline
```

dc2 192.168.93.200 60000 remote offline

clear sdns dc

This command is used clear the configurations of all SDNS data centers.

sdns sync status [interval] [timeout]

This command is used to set the interval at which a SDNS data center queries the monitor instance status from other data centers and the timeout value for the SDNS data center to receive responses from other data centers.

interval Optional. This parameter specifies the interval at which a SDNS data center queries the monitor instance status from other data centers, in seconds. Its value must be an integer ranging from 1 to 86,400. The default value is 3.

timeout Optional. This parameter specifies the timeout value for the SDNS data center to receive responses from other data centers, in seconds. Its value must be an integer ranging from 1 to 86,400. The default value is 2.

Note: The value must be equal to or smaller than the value of the “interval” parameter.

show sdns sync status

This command is used to show the interval at which a SDNS data center queries the monitor instance status from other data centers and the timeout value for the SDNS data center to receive responses from other data centers.

sdns sync config to <dc_name>

This command is used to synchronize the configurations of the local SDNS data center to the specified remote data center.

dc_name This parameter specifies the data center name. Its value must be:

- the name of the data center: synchronizes to this data center.
- all: synchronizes to all remote data centers.



Note: When the HA Active-Standby mode is configured for SDNS data centers and this command is run on the local SDNS data center, the local data center needs to be configured on the remote data center as a synchronization peer (using the command **synconfig peer**), and the IP address of the synchronization peer must be the IP address of the local data center.

sdns sync config from <dc_name>

This command is used to synchronize the configurations of the specified remote SDNS data center to the local data center.

- dc_name The name of a remote data center. The value must be:
- the name of the data center: synchronizes to this data center.
 - all: synchronizes to all remote data centers.



Note:

When the HA Active-Standby mode is configured for SDNS data centers and this command is run on the local SDNS data center, the local data center needs to be configured on the remote data center as a synchronization peer (using the command **synconfig peer**), and the IP address of the synchronization peer must be the IP address of the local data center.

show statistics sdns dc instance [dc_name]

This command is used to display the specified data center and its monitor instance statistics. If the “dc_name” parameter is not specified, all data centers and their monitor instance statistics will be displayed.

sdns sync config runtime on

This command is used to enable the SDNS runtime configuration synchronization function. When this function is enabled and the configuration of a data center changes, the system will synchronize the changed configuration of the data center to other data centers in real time. The system only synchronizes configuration changes made after this function is enabled. By default, this function is disabled.

SDNS runtime configuration synchronization between data centers is only possible when this function is enabled on both the local and remote data centers. SDNS configuration items that do not need to be synchronized are listed in a blacklist. After the SDNS runtime configuration synchronization function is enabled, all configurations except those in the blacklist will be synchronized. The following table lists all configuration items in the blacklist.

show
clear sdns all
sdns dc
no sdns dc
clear sdns dc
sdns dnssec
no sdns dnssec
clear sdns dnssec
sdns dps
no sdns dps
clear sdns dps
sdns listener
no sdns listener
clear sdns listener

```
sdns sync config
sdns on
sdns off
sdns config
no sdns config
clear sdns config
clear sdns sync log
```



Note: When the SDNS runtime configuration synchronization function is enabled, all commands can be synchronized only when the configuration is performed on one data center. If the SDNS runtime configuration synchronization is executed on multiple data centers, the configuration synchronization may cause conflicts, and the all the commands cannot be fully synchronized.

sdns sync config runtime off

This command is used to disable the SDNS runtime configuration synchronization function.

show sdns sync config

This command is used to display the settings of the SDNS runtime configuration synchronization function.

show sdns sync log [dc_name] [filter_string]

This command is used to display the SDNS runtime configuration synchronization log information.

dc_name Optional. This parameter specifies the name of the SDNS data center and its value must be a string of no more than 50 bytes.

- If this parameter is specified, the system will display the runtime configuration synchronization log information of the specified SDNS data center, and each command for which the synchronization is successful (success), not executed (failed), or failed (loss) in detail.
- If this parameter is not specified, the system will display the runtime configuration synchronization log information of all SDNS data centers, and only the number of the commands for which the synchronization is successful (success), not executed (failed), or failed (loss).

filter_string Optional. This parameter specifies the filter string used to search the SDNS runtime configuration synchronization log. The keywords “success”, “loss” and “failed” are supported.

- If this parameter is specified, the system will display

all information that matches the string of SDNS runtime configuration synchronization.

- If the keywords “success”, “loss”, or “failed” are specified, only the success synchronization commands, the loss synchronization commands, or the failed synchronization commands will be displayed.
- If this parameter is not specified as the above keywords, the commands “success” and “failed” will be displayed in the order in which the SDNS commands were configured.
- If this parameter is not specified, the system will display the statistics for each synchronization command of remote data center.



Note: After enabling the runtime configuration synchronization function, if a new data center is added, all commands after enabling the runtime configuration synchronization function will be counted to the “loss” part of the new data center.

A configuration example is as follows:

➤ SDNS Runtime Configuration Synchronization

```
Demo(config)#sdns sync config runtime on
Demo(config)#show sdns sync config
sdns sync config runtime on
Demo(config)#show sdns sync status
sdns sync status 3 2
```

➤ SDNS Runtime Configuration Synchronization Log Analysis

```
Demo(config)#show sdns sync log
dc name: dc2(192.168.93.200)
Total failed : 2
Total loss : 0
Total success: 65
Failed cli command:
no sdns service ip “resorts1”
Service name “resorts1” not found.
Failed tp execute “no sdns service ip” “resorts1”
no sdns service ip “resorts2”
Service name “resorts2” not found.
Failed tp execute “no sdns service ip” “resorts2”
dc name: dc22(182.16.8.222)
Total failed : 0
Total loss : 2
Total success: 65
Loss cli command:
no sdns service ip “resorts1”
no sdns service ip “resorts2”
Demo(config)#show sdns sync log pool
192.168.93.200:3:clear sdns pool cname
```

```
192.168.93.200:4:clear sdns pool name
192.168.93.200:33:sdns pool name "p1" 1
192.168.93.200:34:sdns pool name "return" 1
.....
```

clear sdns sync log

This command is used to clear all SDNS runtime configuration synchronization logs.

show sdns sync diff [dc_name]

Displays the difference between the SDNS configurations of the local and remote data center.

The following configurations must be done before running this command:

- Configure a host for the local data center and at least one host for the remote data center.
- Configure Challenge Code using the command **synconfig challenge**.
- If this command is run at the local data center, you need to set the IP address of the synchronized peer node at the remote data center to the IP address of the local data center.

This command doesn't display the difference between the SDNS configurations of the data center and standby device of the data center.

dc_name Optional. This parameter specifies the name of the existing remote data center. The default value is empty, which means the difference of the SDNS configurations between the local data center and all remote data centers is displayed.



Note:

When the HA Active-Standby mode is configured for SDNS data centers and this command is run on the local SDNS data center, the local data center needs to be configured on the remote data center as a synchronization peer (using the command **synconfig peer**), and the IP address of the synchronization peer must be the IP address of the local data center.

For the configuration difference that cannot be displayed using this command is managed by a blacklist. The table below shows all the configuration items in the blacklist:

no (except no sdns pool resort service)

```
show
clear
sdns dc
sdns dnssec
```



```

sdns dps
sdns listene
sdns sync config
sdns on
sdns off
sdns config

```

SDNS Link

sdns link name <link_name> <gateway_ip> <dc_name>

Creates an SDNS link for a data center. After the SDNS link is created, the system enables the SDNS link by default. The system supports creating a maximum of 1024 SDNS links for all data centers.

For existing SDNS links, this command supports modifying the IP address of the gateway associated with the SDNS link, and the IP address type must be the same before and after the modification.

link_name	This parameter specifies the name of the SDNS link, and the name of the SDNS link must be unique. The value must be a string with a length of no more than 40 bytes.
gateway_ip	This parameter specifies the gateway IP address of the SDNS link. The value must be an IPv4 or IPv6 address. For different SDNS links, the gateway IP address can be the same.
dc_name	This parameter specifies the name of a data center that already exists.

no sdns link name <link_name>

Deletes the configuration of an SDNS link.

show sdns link name [link_name]

Displays the configuration of the specified SDNS link. If no value is assigned to the **link_name** parameter, the system will display the configurations of all SDNS links.

clear sdns link name

Deletes the configurations of all SDNS links.

sdns link monitor apply <link_name> <template_name>

Creates a relationship between SDNS link and a health check template. After the health check template is associated with the SDNS link, the system generates a health check instance for the corresponding SDNS link (display through the **show statistics sdns link** or **show statistics sdns dc instance** command).

An SDNS link can be associated with up to eight health check templates.

link_name	This parameter specifies the name of the SDNS link.
template_name	This parameter specifies the name of the health check template. Health check templates for data collection types are not supported.



Note: If the health check template associated with the SDNS link is of the types that require a destination port, such as TCP, HTTP, and HTTPS, administrators need to specify the destination port when defining the template.

no sdns link monitor apply <link_name> <template_name>

Deletes the relationship between an SDNS link and the health check template.

show sdns link monitor apply [link_name]

Displays the relationship between an SDNS link and the health check template. If no value is assigned to the **link_name** parameter, the relationship between all SDNS links and health check templates will be displayed.

clear sdns link monitor apply [link_name]

Deletes the relationship between an SDNS link and the health check template. If no value is assigned to the **link_name** parameter, the relationship between all SDNS links and health check templates will be deleted.

sdns link health relation <link_name> [relation]

Sets the relationship among health check instances of an SDNS link.

link_name	This parameter specifies the name of the SDNS link.
relation	Optional. This parameter specifies the relationship among health check instances of the SDNS link. The value must be: • and: The SDNS link is marked as “Up” only when the results of all health check instances of the SDNS link are “Up”. • or: The SDNS link is marked as “Up” as long as the result of any health check instance of the SDNS link is “Up”. The default value is “and”.

show sdns link health relation [link_name]

Displays the relationship among health check instances of an SDNS link. If no value is assigned to the **link_name** parameter, the relationship among the health check instances of all SDNS links will be displayed.

sdns link health mixrelation *<link_name> <expression>*

Sets a mixed relationship for multiple health check instances of an SDNS link.

link_name	This parameter specifies the name of the SDNS link.
expression	This parameter specifies the relation expression used to detect the health status of the SDNS link. The expression must be enclosed in double quotes. The expression should be in the following format: (monitor group1) OR/AND (monitor group2) OR/AND (monitor group3).

One pair of brackets in the expression represents one health check template group. The system supports a maximum of 5 groups. The relationship among the groups must be AND or OR, for example, “(m1) OR (m2 OR m3) AND (m4)”. Each group supports defining a maximum of 8 health check templates. Each health check template must and shall only be included in one group. For instance, “m1 OR m2” and “((m1 OR m2) AND m4)” are both illegal expressions.

The health check templates in the expression should be applied to the SDNS link. If the health check template is deleted, its related configuration in the expression will be deleted as well.



Note: When a mixed relationship is configured for an SDNS link, the “and” or “or” relationship set for that SDNS link through the command “**sdns link health relation**” will no longer take effect.

no sdns link health mixrelation *<link_name>*

Deletes the mixed relationship among multiple health check instances of an SDNS link.

show sdns link health mixrelation *[link_name]*

Displays the mixed relationship among health check instances of an SDNS link. If no value is assigned to the **link_name** parameter, the mixed relationships among the health check instances of all SDNS links will be displayed.

sdns link service *<link_name> <service_name>*

Creates a relationship between an SDNS service with an SDNS link. Each SDNS service can only be associated with one SDNS link, and each SDNS link can be associated with up to 20,000 SDNS services.

<code>link_name</code>	This parameter specifies the name of an existing SDNS link.
<code>service_name</code>	This parameter specifies the name of an existing SDNS service. The IP address type of the SDNS service must be the same as the IP address type of the gateway of the SDNS link. The SDNS service must belong to the same data center as the SDNS link specified by the parameter “link_name”.

no sdns link service *<link_name> <service_name>*

Deletes the relationship between an SDNS service and an SDNS link.

show sdns link service *[link_name]*

Displays the relationship among SDNS services and an SDNS link. If no value is assigned to the **link_name** parameter, the relationship among all SDNS services and all SDNS links will be displayed.

clear sdns link service *[link_name]*

Deletes the relationship among SDNS services and an SDNS link. If no value is assigned to **link_name** parameter, the relationship among all SDNS services and SDNS links will be cleared.

sdns link enable *<link_name>*

Enables an SDNS link.

<code>link_name</code>	This parameter specifies the name of an existing SDNS link.
------------------------	---

sdns link disable *<link_name>*

Disables an SDNS link. If an SDNS link is disabled, all SDNS services associated with the link will be unavailable.

<code>link_name</code>	This parameter specifies the name of an existing SDNS link.
------------------------	---

show sdns link status *[link_name]*

Displays the status of a link, including the link name, data center, status (enabled or disabled) and availability of the link.

show statistics sdns link *[link_name]*

Displays the statistics of an SDNS link.

SDNS Statistics**sdns statistics localdns {on|off}**

This command is used to enable or disable local DNS statistics. By default, local DNS statistics is enabled.

show statistics sdns localdns all

This command is used to display the statistics of every local DNS.

For example:

AN(config)#show statistics sdns localdns all					
Local DNS	Total	Success	Unauthoritative	Fail	
Drop					
IPV4:					
10.8.6.45	150	148	2	0	0
IPV6:					

show statistics sdns localdns summary

This command is used to display the total statistics of all local DNSs.

For example:

AN(config)#show statistics sdns localdns summary					
Localdns Statistics Summary:					
Total	Success	Unauthed	Fail	Drop	
IPV4: 150	148	2	0	0	
IPV6: 0	0	0	0	0	

clear statistics sdns localdns

This command is used to clear local DNS statistics.

sdns statistics query {on|off}

This command is used to enable or disable DNS query statistics. By default, DNS query statistics is enabled.

show statistics sdns query

This command is used to display DNS query statistics.

For example:

```
AN(config)#show statistics sdns query
```

```
SDNS Query Statistics Summary:
```

```
Request:      150
Success:      148
Unauthed:     2
Fail:         0
Drop:         0
```

```
Second Statistics:
```

```
Request:      0
Success:      0
Unauthed:     0
Fail:         0
Drop:         0
Peak Request: 6
Peak Success: 6
Peak Unauthed: 2
Peak Fail:    0
Peak Drop:    0
```

```
5 Second Statistics:
```

```
Request:      0
Success:      0
Unauthed:     0
Fail:         0
Drop:         0
Peak Request: 8
Peak Success: 8
Peak Unauthed: 2
Peak Fail:    0
Peak Drop:    0
```

```
Minute Statistics:
```

```
Request:      0
Success:      0
Unauthed:     0
Fail:         0
Drop:         0
Peak Request: 37
Peak Success: 37
Peak Unauthed: 2
Peak Fail:    0
Peak Drop:    0
```

```
Hour Statistics:
```

```
Request:      150
Success:      148
Unauthed:     2
Fail:         0
Drop:         0
Peak Request: 150
Peak Success: 148
Peak Unauthed: 2
Peak Fail:    0
Peak Drop:    0
```

```
Day Statistics:
```

```
Request:      0
Success:      0
Unauthed:     0
Fail:         0
Drop:         0
```

Peak Request:	0
Peak Success:	0
Peak Unauthed:	0
Peak Fail:	0
Peak Drop:	0

clear statistics sdns query

This command is used to clear DNS query statistics.

show sdns statistics status

This command is used to display the status of local DNS statistics and DNS query statistics.

show statistics sdns service ip [service_name]

This command is used to display the statistics of the specified SDNS service. If the optional parameter “service_name” is not specified, the statistics of all SDNS services will be displayed.

For example:

```
AN(config)#show statistics sdns service ip
```

```
Service Name:    sv1
Ip Address:      10.8.6.201
Port:            0
Health:          UP
Disabled:        NO
Relation:        AND
Hits:            4
Monitor Count:   0
  Instance ID:   0
  Monitor:       h1
  Status:        INIT
  Type:          HTTP
  Address:       10.8.6.201
  Source:        0.0.0.0
  Gateway:       0.0.0.0
```

```
Service Name:    sv2
Ip Address:      10.8.6.202
Port:            0
Health:          UP
Disabled:        NO
Relation:        AND
Hits:            4
Monitor Count:   0
  Instance ID:   1
  Monitor:       h1
  Status:        UP(PROBE_SUCCESS)
  Type:          HTTP
  Address:       10.8.6.202
  Source:        0.0.0.0
  Gateway:       0.0.0.0
```

show statistics sdns pool [pool_name]

This command is used to display the statistics of the specified SDNS service pool. If the optional parameter “pool_name” is not specified, the statistics of all SDNS service pools will be displayed.

Example:

```
Demo(config)#show statistics sdns pool
Pool Name:                p1
Health Relation:          and
Max Resource Record Count: 1
Hit:                      0
Primary Method:           rr
Primary Method Success:   0
Secondary Method:         none
Secondary Method Success: 0
Pool Monitor Count:       6
    Monitor Name:         m1_v4
    Type:                  ICMP
    Monitor Name:         m2_v4
    Type:                  ICMP
    Monitor Name:         m3_v4
    Type:                  ICMP
    Monitor Name:         m1_v6
    Type:                  ICMP
    Monitor Name:         m2_v6
    Type:                  ICMP
    Monitor Name:         m3_v6
    Type:                  ICMP
Pool Member Count:        6
    Service Name:         s1_v4
    Service IP:           1.1.1.1
    Health:               UP
    Weight:               1
    Priority:              6
    Hit:                  0
    Instance ID:          0
    Status:               UP
    Instance ID:          3
    Status:               UP
    Instance ID:          6
    Status:               UP

    Service Name:         s2_v4
    Service IP:           1.1.1.2
    Health:               UP
    Weight:               1
    Priority:              10
    Hit:                  0
    Instance ID:          1
    Status:               UP
    Instance ID:          4
    Status:               UP
    Instance ID:          7
    Status:               UP
```


Service Name: s3_v4
 Service IP: 1.1.1.3
 Health: DOWN
 Weight: 1
 Priority: 1
 Hit: 0
 Instance ID: 2
 Status: DOWN
 Instance ID: 5
 Status: DOWN
 Instance ID: 8
 Status: DOWN

Service Name: s1_v6
 Service IP: 11::1
 Health: DOWN
 Weight: 1
 Priority: 1
 Hit: 0
 Instance ID: 9
 Status: DOWN
 Instance ID: 12
 Status: DOWN
 Instance ID: 15
 Status: DOWN

Service Name: s2_v6
 Service IP: 11::2
 Health: DOWN
 Weight: 1
 Priority: 1
 Hit: 0
 Instance ID: 10
 Status: DOWN
 Instance ID: 13
 Status: DOWN
 Instance ID: 16
 Status: DOWN

Service Name: s3_v6
 Service IP: 11::3
 Health: DOWN
 Weight: 1
 Priority: 1
 Hit: 0
 Instance ID: 11
 Status: DOWN
 Instance ID: 14
 Status: DOWN
 Instance ID: 17
 Status: DOWN

If it is the mixed relationship configured for the address pool, it will be displayed as follows:

Pool Name: p1

Health Mix Relation:	(m1_v4 AND m2_v4) OR (m3_v4),(m1_v6 OR m2_v6) AND (m3_v6)
Max Resource Record Count:	1
Hit:	0
Primary Method:	rr
Primary Method Success:	0
Secondary Method:	none
Secondary Method Success:	0

show statistics sdns host [host_name]

This command is used to display the statistics of the specified SDNS host name. If the optional parameter “host_name” is not specified, the statistics of all SDNS host names will be displayed.

For example:

AN(config)# show statistics sdns host	
Host Name:	www.abc.com
Host TTL:	60
Hit:	6
Default Pool:	p2
Default Policy Hit:	0
Region Policy Count:	1
Region Policy Hit:	6
Host Name:	www.xyz.com
Host TTL:	60
Hit:	2
Default Pool:	bug
Default Policy Hit:	2
Region Policy Count:	0

show statistics sdns proximity [region_name]

This command is used to display the hit statistics of the specified SDNS region. If the optional parameter “region_name” is not specified, the hit statistics of all SDNS regions will be displayed.

For example:

AN(config)# show statistics sdns proximity	
Region Name	HIT
region1	6
region2	0
Total Region Number is: 2	

show statistics sdns policy region [policy_name]

This command is used to display the hit statistics of the specified SDNS region policy by SDNS region policy. If the optional parameter “policy_name” is not specified, the hit statistics of all SDNS region policies will be displayed.

For example:

```
AN(config)#show statistics sdns policy region
Policy Name:      policy1
Host Name:        www.abc.com
Pool Name:        p1
Region Name:      region1
Priority:          0
Match Count:      6
```

show statistics sdns policy byhost [*host_name*]

This command is used to display the hit statistics of SDNS policies by SDNS host name. If the optional parameter “host_name” is not specified, the statistics of SDNS policies for all SDNS host names will be displayed.

For example:

```
AN(config)#show statistics sdns policy byhost
Policy Name:      policy1
Host Name:        www.abc.com
Pool Name:        p1
Region Name:      region1
Priority:          0
Match Count:      6
Policy Name:      Default Policy
Host Name:        www.abc.com
Default Pool:     default_pool
Match Count:      0
Host "www.abc.com" configured default policy
Policy Name:      Resort Policy
Host Name:        www.abc.com
Resort Pool:      resort_pool
Match Count:      0
Host "www.abc.com" configured resort policy
Host "www.abc.com" has 1 region policy
Host "www.xyz.com" has 0 region policy
```

show statistics sdns policy bypool [*pool_name*]

This command is used to display the hit statistics of SDNS policies by SDNS pool. If the optional parameter “pool_name” is not specified, the statistics of SDNS policies for all SDNS pools will be displayed.

For example:

```
AN(config)#show statistics sdns policy bypool
Policy Name:      policy1
Host Name:        www.abc.com
Pool Name:        p1
Region Name:      region1
Priority:          0
Match Count:      6
Pool "p1" related with 1 region policy
```

Pool "return" related with 0 region policy

```
Policy Name:      Resort Policy
Host Name:        www.abc.com
Pool Name:        resort_pool
Priority:          0
Match Count:      0
```

Pool "resort_pool" related with 0 region policy

```
Policy Name:      Default Policy
Host Name:        www.abc.com
Pool Name:        default_pool
Priority:          0
Match Count:      0
```

Pool "default_pool" related with 0 region policy

Pool "resort_return" related with 0 region policy

show statistics sdns policy byregion [region_name]

This command is used to display the hit statistics of SDNS region policies by SDNS region. If the optional parameter “region_name” is not specified, the statistics of SDNS region policies for all SDNS regions will be displayed.

For example:

AN(config)#**show statistics sdns policy byregion**

```
Policy Name:      policy1
Host Name:        www.abc.com
Pool Name:        p1
Region Name:      region1
Priority:          0
Match Count:      6
Region "region1" related with 1 region policy
```

Region "region2" related with 0 region policy

show statistics sdns policy default [host_name]

This command is used to display the hit statistics of the SDNS default policy for the specified SDNS host name. If the optional parameter “host_name” is not specified, the hit statistics of the SDNS default policies for all the SDNS host names will be displayed.

For example:

AN(config)#**show statistics sdns policy default**

Host Name	Default Pool	Match
www.abc.com	p2	0
www.xyz.com	p1	2

show statistics sdns policy resort [host_name]

This command is used to display the hit statistics of the SDNS emergency policy for the specified SDNS host name. If the optional parameter “host_name” is not specified, the hit statistics of the SDNS emergency policies for all SDNS hosts will be displayed.

For example:

AN(config)# show statistics sdns policy resort		
Host Name	Resort Pool	Match
www.abc.com	p2	0
www.xyz.com	p1	2

show statistics sdns ranking host

This command is used to display the Top10 SDNS host names in hits.

show statistics sdns ranking policy

This command is used to display the Top10 SDNS policies in hits.

show statistics sdns ranking pool

This command is used to display the Top10 SDNS service pools in hits.

show statistics sdns ranking service

This command is used to display the Top20 SDNS services in hits.

SDNS Configuration Backup

sdns config write file <file_name>

This command is used to back up the currently running SDNS configuration on a local disk. The system supports backing up at most 10 SDNS configuration files.

file_name	This parameter specifies the name of the SDNS configuration file to be backed up. The file name should not contain the suffix “.cfg”. The value must be a string with a length of no more than 255 bytes, and supports 0-9, a-z, A-Z and special characters except “.”, “\”, “/”, and “ ”. The value does not include the suffix “.cfg”.
-----------	--



Note: If there are more than 10 SDNS profiles to be backed up, the new SDNS profile will overwrite the oldest SDNS profile automatically.

sdns config write scp <remote_server_ip|name> <user_name> <config_file_name>

This command is used to back up the current SDNS configuration to a remote SCP server. The command requires to verify the password, the system will prompt for the password after entering all parameters.

<code>remote_server_ip name</code>	This parameter specifies the IP address (IPv4 or IPv6) or the name of the remote SCP server.
<code>user_name</code>	This parameter specifies the username used to access the remote SCP server.
<code>config_file_name</code>	This parameter specifies the name of the SDNS configuration file to be saved. The value must be a string containing letters, numbers, and special characters, and is not case-sensitive. If the parameter contains special characters, it must be enclosed in double quotation marks.

The following special characters are not supported: “>”, “””, “””, “[]”, “{ }”, “|”, “\$”, and “\”.

SDNS Configuration Loading

sdns config file <file_name>

This command is used to load a backup SDNS configuration file from a local disk. Before loading the backup SDNS configuration file, the system will preload the following SDNS commands, and then loading the commands in the SDNS configuration file. The preload SDNS commands are as follow:

- clear sdns zone name all
- clear sdns record all
- clear sdns pool cname
- clear sdns pool name
- clear sdns host name
- clear sdns service ip
- clear sdns service monitor apply
- clear sdns policy default
- clear sdns policy resort
- clear sdns policy region
- clear sdns proximity
- clear sdns region
- clear sdns monitor
- clear sdns snmp exp wei

<code>file_name</code>	This parameter specifies the name of the SDNS configuration file to be loaded. The file name cannot contain the extension name.
------------------------	---



Note: If SDNS runtime configuration synchronization, HA runtime configuration synchronization, or incremental synchronization are enabled when you run this command, the system will synchronize the preloaded SDNS commands with the commands in the backup SDNS configuration file (commands in the whitelist). If the SDNS function in the backup SDNS configuration file is disabled or the SDNS runtime configuration synchronization function is disabled, the SDNS runtime configuration synchronization would fail.

sdns config scp *<remote_server_ip|name>* *<user_name>*
<config_file_name>

This command is used to load the SDNS configuration file from a remote SCP server. The command requires to verify the password, the system will prompt for the password after entering all parameters .

<code>remote_server_ip name</code>	This parameter specifies the IP address (IPv4 or Ipv6) or the name of the remote SCP server.
<code>user_name</code>	This parameter specifies the username used to access the remote SCP server.
<code>config_file_name</code>	This parameter specifies the name of the SDNS configuration file to be loaded. The value must be a string containing letters, numbers, and special characters, and is not case-sensitive. If the parameter contains special characters, it must be enclosed in double quotation marks. The following special characters are not supported: “>”, “””, “[]”, “{ }”, “ ”, “\$”, and “\”.

no sdns config file *<file_name>*

This command is used to delete the specified local SDNS configuration file.

<code>file_name</code>	This parameter specifies the name of the file to be deleted, which needs to include the extension name. For example, executing the command “ no sdns config file abc.cfg ” will delete the configuration file named abc.
------------------------	---

show sdns config file *[file_name]* *[regex]*

This command is used to display the configuration information for matching filter strings in the specified local SDNS configuration file.

file_name

Optionally, this parameter specifies the name of the SDNS profile to be displayed, without the extension name in the file name.

- If this parameter is specified, the system will display the details of the SDNS profile.
- If this parameter is not specified, the system will display all loaded SDNS profiles.

regex

Optionally, this parameter specifies a string to filter configuration information in the configuration file and is available only when the parameter “file_name” is specified. This parameter supports regular expression. For example, the command “**show sdns config file new_sdns service**” lists all configuration information in the configuration file “new_sdns” that contain the “service” string.

If this parameter is not specified, the system will display all configuration information in the specified configuration file.

clear sdns config file

This command is used to clear all local SDNS configuration files.

Chapter 20 Deep Packet Inspection (DPI)

dpi {on|off}

This command is used to enable or disable the DPI feature. By default, this feature is disabled.

dpi protocol name <application_name>

This command is used to define a custom application protocol.

application_name	This parameter specifies the name of the application protocol. Its value must be a string of 1 to 40 characters. Numbers, letters, and special characters are supported. When the parameter value contains special characters, it must be enclosed in double quotes.
------------------	--

no dpi protocol name <application_name>

This command is used to delete the specified custom application protocol.

show dpi protocol name [type]

This command is used to display the specified type of application protocols.

type	This parameter specifies the type of the application protocols. Its value must be: • predefine: displays all predefined application protocols. • custom: displays all custom application protocols • empty: display both predefined and custom application protocols.
------	---

The default value is empty.

clear dpi protocol name

This command is used to clear all custom application protocols.

dpi protocol rule port <application_name> <port_number> <protocol_type>

This command is used to add a port-based application rule for the specified custom application protocol and define its transport layer protocol.

application_name	This parameter specifies the name of an existing custom
------------------	---

	application protocol.
port_number	This parameter specifies the port number of the custom application protocol. Its value must be an integer ranging from 1 to 65,535.
protocol_type	This parameter specifies the transport layer protocol of the custom application protocol. Its value must be “tcp” or “udp”.

no dpi protocol rule port *<application_name>* *<port_number>*
<protocol_type>

This command is used to delete a port-based rule for the specified custom application protocol.

show dpi protocol rule port *[application_name]* *[protocol_type]*

This command is used to display all port-based application rules of the specified protocol type for the specified custom application protocol.

application_name	Optional. This parameter specifies the name of an existing custom application protocol. Its value must be the name of an existing custom application protocol or empty, which indicates all custom application protocols.
------------------	---

The default value is empty.

protocol_type	Optional. This parameter specifies the transport layer protocol of the custom application protocol. Its value must be “tcp”, “udp” or empty (both TCP and UDP).
---------------	---

The default value is empty.

clear dpi protocol rule port *[application_name]* *[protocol_type]*

This command is used to clear all port-based application rules of the specified protocol type for the specified custom application protocol.

application_name	Optional. This parameter specifies the name of an existing custom application protocol. Its value must be the name of an existing custom application protocol or empty, which indicates all custom application protocols.
------------------	---

The default value is empty.

protocol_type	Optional. This parameter specifies the transport layer protocol of the custom application protocol. Its value must be “tcp”, “udp” or empty (both TCP and UDP).
----------------------	---

The default value is empty.

dpi protocol rule ip <application_name> <ip> {netmask|prefix}

This command is used to add an IP-based application rule for the specified custom application protocol.

application_name	This parameter specifies the name of an existing custom application protocol.
ip	This parameter specifies the IP address of the custom application protocol. Both IPv4 and IPv6 are supported.
netmask prefix	This parameter specifies the netmask or prefix for the Ip address of the application protocol: • “netmask” is used for setting the netmask of the IPv4 address. Its value must be an integer ranging from 0 to 32. • “prefix” is used for setting the prefix length of the IPv6 address. Its value must be an integer ranging from 0 to 128.

no dpi protocol rule ip <application_name> <ip> <netmask|prefix>

This command is used to delete an IP-based application rule for the specified custom application protocol.

show dpi protocol rule ip [application_name]

This command is used to display all IP-based application rules for the specified custom application protocol. If the “application_name” parameter is not specified, the IP-based application rules for all custom application protocols will be displayed.

clear dpi protocol rule ip [application_name]

This command is used to clear all IP-based application rules for the specified custom application protocol. If the “application_name” parameter is not specified, the IP-based application rules for all custom application protocols will be cleared.

dpi protocol rule host <application_name> <domain>

This command is used to add a domain-based application rule for the specified custom application protocol.

application_name	This parameter specifies the name of an existing custom application protocol.
domain	This parameter specifies the domain name of the custom application protocol. Its value must be a string of 1 to 40

characters.

no dpi protocol rule host *<application_name> <domain>*

This command is used to delete a domain-based application rule for the specified custom application protocol.

show dpi protocol rule host *[application_name]*

This command is used to display all domain-based application rules for the specified custom application protocol. If the “application_name” parameter is not specified, the domain-based application rules for all custom application protocols will be displayed.

clear dpi protocol rule host *[application_name]*

This command is used to clear all domain-based application rules for the specified custom application protocol. If the “application_name” parameter is not specified, the domain-based application rules for all custom application protocols will be cleared.

show dpi protocol rule all *[application_name]*

This command is used display all application rules configured for the specified application protocol. If the “application_name” parameter is not specified, the application rules for all custom application protocols will be displayed.

dpi import profile *<profile_name> <url>*

This command is used to import the profile from the specified URL address.

profile_name	This parameter specifies the name of the application profile. Its value must be a case-sensitive string of 1 to 40 characters. Numbers, letters, and special characters are supported. When the parameter value contains special characters, it must be enclosed in double quotes.
--------------	--

url	This parameter specifies the URL address of the application profile to be imported. SCP, TFTP, HTTP and FTP URLs are supported.
-----	---

Note: The “path” of the FTP URL (ftp://user:password@host:port/path) should be a relative path.

no dpi import profile *<profile_name>*

This command is used to delete the specified application profile.

show dpi import profile

This command is used to display all application profiles that have been imported.

clear dpi import profile

This command is used to clear all application profiles that have been imported.

dpi attach profile <application_name> <profile_name>

This command is used to attach the specified application protocol with the specified application profile.

application_name This parameter specifies the name of an existing application protocol that is predefined or self-defined.

profile_name This parameter specifies the name of an existing application profile this is imported.

no dpi attach profile <application_name>

This command is used to cancel the relationship between the specified application protocol and its profile.

show dpi attach profile [application_name]

This command is used to display the profile attached with the specified application protocol. If the “application_name” parameter is not specified, the profiles of all application protocols will be displayed.

clear dpi attach profile

This command is used to cancel the relationships between all application protocols and their profiles.

dpi update profile office365 [interval]

This command is used to configure the update interval of the Office 365 application profile. After this command is configured, the system will periodically (based on the value of the “interval” parameter) update the latest application profile of Office 365 from this address:

<https://endpoints.office.com/endpoints/worldwide?noipv6=false&ClientRequestId=b10c5ed1-bad1-445f-b386-b919946339a7>.

interval Optional. This parameter specifies the update interval of the Office 365 application profile. Its value must be an integer ranging from 0 to 8640, in hours. The value 0 indicates that the application profile of Office 365 will be updated once immediately but will not be automatically updated again in the future.

The default value is 0.

no dpi update profile office365

This command is used to delete the update interval setting of the Office 365 application profile.

show dpi update profile office365

This command is used to display the update interval setting of the Office 365 application profile.

dpi apply link route <application_name> <link_name>

This command is used to build the relationship between the specified application protocol and the specified LLB link route. Each application protocol can be associated with only one LLB link.

application_name This parameter specifies the name of an existing application protocol.

link_name This parameter specifies the name of an existing LLB link.

no dpi apply link route <application_name> [link_name]

This command is used to cancel the relationship between the specified application protocol and a specified LLB link. If the “link_name” parameter is not specified, the relationships between the specified application protocol and all LLB links will be cancelled.

show dpi apply link route [application_name]

This command is used to display the LLB link associated with the specified application protocol. If the “application_name” parameter is not specified, the LLB links associated with all application protocols will be displayed.

clear dpi apply link route [application_name]

This command is used to cancel the relationship between the specified application protocol and the LLB link. If the “application_name” parameter is not specified, the relationships between all application protocols and LLB links will be cleared.

show statistics dpi summary

This command is used to display the summary statistics of DPI.

clear statistics dpi summary

This command is used to clear the summary statistics of DPI.

show statistics dpi verbose [application_name]

This command is used to display the detailed statistics of DPI for the specified application protocol. If the “application_name” parameter is not specified, the detailed statistics of DPI for all application protocols will be displayed.

clear statistics dpi verbose [application_name]

This command is used to clear the detailed statistics of DPI for the specified application protocol. If the “application_name” parameter is not specified, the detailed statistics of DPI for all application protocols will be cleared.

dpi cache flush

Clears the DPI cache.

dpi cache timeout <timeout>

Sets the timeout of the DPI cache. If the timeout of the DPI cache is not set using this command, the default timeout will be 86,400 seconds.

timeout	This parameter specifies the timeout of the DPI cache. The unit is second. The value must be an integer ranging from 1 to 4,294,967,295.
---------	--

no dpi cache timeout

Restores the timeout of the DPI cache to the default.

show dpi cache timeout

Displays the timeout of the DPI cache.

show statistics dpi cache [filter_string]

Displays the DPI cache information matching a filter string.

filter_string	Optional. This parameter specifies a filter string. The value must be a string consisting of one or several keywords. The value’s format is “Keyword1=xxx,Keyword2=yyy,...”. There is no restriction on the order. Each item is separated by a comma and enclosed by double quotes.
---------------	---

If this parameter is specified, the system will only display the DPI cache information that matches all filter strings in the double quotes. If this parameter is not specified, the system will display all DPI cache information.

Supported keywords:

- ip: An IP address. The value is a string in IP address format.
- port: A port number. The value is an integer ranging from 0 to 65,535.
- proto: A protocol type. The value can be “http/https,” “ssh,” “telnet,” “smtp/smtps,” “pop3/pop3s,” “imap/imap,” “ftp/ftps,” “tcp-dns,” “udp-dns,” “snmp,” “ntp,” “ssl,” “unknown.”

Example:

`“ip=1.1.1.1,port=80,proto=ssl”`

In the output of the DPI cache information, the IP address is 1.1.1.1, the port is 80, and the protocol is SSL.

Chapter 21 Application Security

The Access Control List (ACL) allows you to perform administration on WebWall or firewall style of security rules. The commands in this chapter dictate those who may gain access to your system based on their location in the Internet, and the network interface used to contact the appliance.

WebWall

Access Groups

accessgroup <accesslist_id> <interface>

This command allows users to assign access list members to a specific group and a specific interface.

accesslist_id	The identification number (1-999) assigned to this group of members. This value should match the value established for the access list member created with the “ accesslist ” command.
interface	The associated interface for this access group, which can be the system interface, bond interface, VLAN interface or MNET interface.

Example:

```
AN(config)#accessgroup 250 port1
```

no accessgroup <accesslist_id> <interface>

This command allows users to remove an access group from the associated interface.

show accessgroup

This command is used to display all access groups.

clear accessgroup

This command is used to remove all the group entries created by using the “**accessgroup**” command. No TCP, UDP or ICMP packets will be allowed through the WebWall nor will users be able to access the appliance by way of the WebUI unless WebWall is disabled.

Access Rule

The following commands must be used with the “**accessgroup**” command. After an access rule is created, the administrator must associate the new access rule ID with an interface and then enable WebWall on this interface, then the access rule can take

effect. The system supports a maximum of 1024 access rules, which apply to both IPv4 and IPv6 packets.

**accesslist permit {tcp|udp} <source_ip> {source_mask|source_prefix}
<source_port> <destination_ip> {destination_mask|destination_prefix}
<destination_port> <accesslist_id>**

This command is used to create a TCP or UDP permit access rule and associate the rule with the specified access rule ID.

source_ip	This parameter specifies the source IP address of packets to be permitted. Its value must be an IPv4 or IPv6 address.
source_mask source_prefix	This parameter specifies the netmask or prefix length for the source IP address of packets to be permitted. • “source_mask” is used for an IPv4 address. It should be a dotted IP address or an integer ranging from 0 to 32. • “source_prefix” is used for an IPv6 address. It should be an integer ranging from 0 to 128.
source_port	This parameter specifies the source port number of packets to be permitted. Its value must be an integer ranging from 0 to 65,535. The value 0 indicates all port numbers.
destination_ip	This parameter specifies the destination IP address of packets to be permitted. Its value must be an IPv4 or IPv6 address.
destination_mask destination_prefix	This parameter specifies the netmask or prefix length for the destination IP address of packets to be permitted. • “destination_mask” is used for an IPv4 address. It should be a dotted IP address or an integer ranging from 0 to 32. • “destination_prefix” is used for an IPv6 address. It should be an integer ranging from 0 to 128.
destination_port	This parameter specifies the destination port number of packets to be permitted. Its value must be an integer ranging from 0 to 65,535. The value 0 indicates all port numbers.
accesslist_id	This parameter specifies the access rule ID. Its value must be an integer ranging from 1 to 999.

```
no accesslist permit {tcp|udp} <source_ip> {source_mask|source_prefix}
<source_port> <destination_ip> {destination_mask|destination_prefix}
<destination_port> <accesslist_id>
```

This command is used to delete a TCP or UDP permit access rule.

```
accesslist deny {tcp|udp} <source_ip> {source_mask|source_prefix}
<source_port> <destination_ip> {destination_mask|destination_prefix}
<destination_port> <accesslist_id>
```

This command is used to create a TCP or UDP deny access rule and associate the rule with the specified access rule ID.

source_ip	This parameter specifies the source IP address of packets to be denied. Its value must be an IPv4 or IPv6 address.
source_mask source_prefix	This parameter specifies the netmask or prefix length for the source IP address of packets to be denied. “source_mask” is used for an IPv4 address. It should be a dotted IP address or an integer ranging from 0 to 32. “source_prefix” is used for an IPv6 address. It should be an integer ranging from 0 to 128.
source_port	This parameter specifies the source port number of packets to be denied. Its value must be an integer ranging from 0 to 65,535. The value 0 indicates all port numbers.
destination_ip	This parameter specifies the destination IP address of packets to be denied. Its value must be an IPv4 or IPv6 address.
destination_mask destination_prefix	This parameter specifies the netmask or prefix length for the destination IP address of packets to be denied. “destination_mask” is used for an IPv4 address. It should be a dotted IP address or an integer ranging from 0 to 32. “destination_prefix” is used for an IPv6 address. It should be an integer ranging from 0 to 128.
destination_port	This parameter specifies the destination port number of packets to be denied. Its value must be an integer ranging from 0 to 65,535. The value 0 indicates all port numbers.
accesslist_id	This parameter specifies the access rule ID. Its value must

be an integer ranging from 1 to 999.

```
no accesslist deny {tcp|udp} <source_ip> {source_mask|source_prefix}
<source_port> <destination_ip> {destination_mask|destination_prefix}
<destination_port> <accesslist_id>
```

This command is used to delete a TCP or UDP deny access rule.

```
accesslist permit {esp|ah|icmp echoreply|icmp echorequest} <source_ip>
{source_mask|source_prefix} <destination_ip>
{destination_mask|destination_prefix} <accesslist_id>
```

This command is used to create an ESP, AH, ICMP Echo Reply or ICMP Echo Request permit access rule and associate the rule with the specified access rule ID.

source_ip	This parameter specifies the source IP address of packets to be permitted. Its value must be an IPv4 or IPv6 address.
source_mask source_prefix	This parameter specifies the netmask or prefix length for the source IP address of packets to be permitted. • “source_mask” is used for an IPv4 address. It should be a dotted IP address or an integer ranging from 0 to 32. • “source_prefix” is used for an IPv6 address. It should be an integer ranging from 0 to 128.
destination_ip	This parameter specifies the destination IP address of packets to be permitted. Its value must be an IPv4 or IPv6 address.
destination_mask destination_prefix	This parameter specifies the netmask or prefix length for the destination IP address of packets to be permitted. • “destination_mask” is used for an IPv4 address. It should be a dotted IP address or an integer ranging from 0 to 32. • “destination_prefix” is used for an IPv6 address. It should be an integer ranging from 0 to 128.
accesslist_id	This parameter specifies the access rule ID. Its value must be an integer ranging from 1 to 999.



Note:

- ICMP Echo Reply permit access rule also allows Destination Unreachable and Time Exceeded error packets with embedded Echo Request to pass through. Destination Unreachable and Time Exceeded error packets without embedded Echo Request and other ICMP error packets with embedded Echo Request are not subject to this rule.

- ICMP Echo Request permit access rule also allows Destination Unreachable and Time Exceeded error packets with embedded Echo Reply to pass through. Destination Unreachable and Time Exceeded error packets without embedded Echo Reply and other ICMP error packets with embedded Echo Reply are not subject to this rule.

no accesslist permit {esp|ah|icmp echoreply|icmp echorequest}

**<source_ip> {source_mask|source_prefix} <destination_ip>
{destination_mask|destination_prefix} <accesslist_id>**

This command is used to delete an ESP, AH, ICMP Echo Reply or ICMP Echo Request permit access rule.

accesslist deny {esp|ah|icmp echoreply|icmp echorequest} <source_ip>

**{source_mask|source_prefix} <destination_ip>
{destination_mask|destination_prefix} <accesslist_id>**

This command is used to create an ESP, AH, ICMP Echo Reply or ICMP Echo Request deny access rule and associate the rule with the specified access rule ID.

source_ip	This parameter specifies the source IP address of packets to be denied. Its value must be an IPv4 or IPv6 address.
source_mask source_prefix	This parameter specifies the netmask or prefix length for the source IP address of packets to be denied. • “source_mask” is used for an IPv4 address. It should be a dotted IP address or an integer ranging from 0 to 32. • “source_prefix” is used for an IPv6 address. It should be an integer ranging from 0 to 128.
destination_ip	This parameter specifies the destination IP address of packets to be denied. Its value must be an IPv4 or IPv6 address.
destination_mask destination_prefix	This parameter specifies the netmask or prefix length for the destination IP address of packets to be denied. • “destination_mask” is used for an IPv4 address. It should be a dotted IP address or an integer ranging from 0 to 32. • “destination_prefix” is used for an IPv6 address. It should be an integer ranging from 0 to 128.
accesslist_id	This parameter specifies the access rule ID. Its value must be an integer ranging from 1 to 999.



Note:

- ICMP Echo Reply deny access rule also disallows Destination Unreachable

and Time Exceeded error packets with embedded Echo Request to pass through. Destination Unreachable and Time Exceeded error packets without embedded Echo Request and other ICMP error packets with embedded Echo Request are not subject to this rule.

- ICMP Echo Request deny access rule also disallows Destination Unreachable and Time Exceeded error packets with embedded Echo Reply to pass through. Destination Unreachable and Time Exceeded error packets without embedded Echo Reply and other ICMP error packets with embedded Echo Reply are not subject to this rule.

no accesslist deny {esp|ah|icmp echoreply|icmp echorequest}

**<source_ip> {source_mask|source_prefix} <destination_ip>
{destination_mask|destination_prefix} <accesslist_id>**

This command is used to delete an ESP, AH, ICMP Echo Reply or ICMP Echo Request deny access rule.

show accesslist

This command is used to display all permit and deny access rules configured on the APV appliance's interfaces.

clear accesslist

This command is used to clear all permit and deny access rules.



Note: After the “**clear accesslist**” command is executed, all TCP, UDP and ICMP packets cannot pass through the webwall.

WebWall

webwall <interface> on [mode]

This command allows users to enable the WebWall function on a specified interface.

interface	This parameter specifies the name of the interface, which can be a system interface, VLAN interface, VXLAN interface or Bond interface.
mode	Optional. This parameter specifies the mode of the WebWall. • 0: Normal mode. All the packets will follow ACL rules. For security consideration, the default mode is normal mode. • 1: Ack mode. In this mode, WebWall is backward compatible in that all the ACK TCP packets will be permitted by default.

The default value is 0.

webwall <interface> off

This command allows users to disable the WebWall function on a specified interface.

By turning off the WebWall, the APV appliance will allow packets to travel freely through the system. Users should only disable the WebWall for diagnostic purposes since it will disable all access list filters. Users also have the choice to enable or disable the WebWall function for a specific interface. Users who are using the Array clustering technology should consult the clustering section of this manual before setting up the WebWall functionality. The command does not reset any of the configured parameters. The WebWall function is always off by default.

Example:

```
AN(config)#webwall port2 off
```

show statistics webwall [interface]

This command is used to display the current WebWall running information pertaining to all interfaces (with the WebWall function enabled). If an interface is specified, this command will only show the running information for this interface.



Note: Please execute the command “**ip statistics on**” before using this command.

show webwall

This command is used to display the current configuration of the WebWall.

clear statistics webwall [interface]

This command is used to clear current statistics pertaining to the WebWall on the specified interface. If no interface is specified, this command will clear the statistics for all interfaces with WebWall on.

Web Application Firewall (WAF)

waf profile name <profile_name>

This command is used to define a security profile for the Web Application Firewall (WAF). The maximum number of the Security Profile allowed on the APV appliance varies with system memories. Please see the table below for details.

profile_name	This parameter specifies the name of security profile. Its value must be a case-sensitive string of 1 to 40 characters. Numbers, lower-case and upper-case letters, and special characters including “@”, “-”, “_”, “!” are supported. Its
--------------	--

value must be enclosed in double quotes when containing any special character or containing only numbers.

System Memory	Maximum Security Profile
4 GB	16
8 GB	32
16 GB	64
32 GB	128

no waf profile name <profile_name>

This command is used to delete the specified WAF security profile.

waf profile assign <profile_name> <virtual_service>

This command is used to associate the security profile with a virtual service. A security profile can be associated with a maximum of 4000 virtual services in the system, but one virtual service can only be associated with one security profile.

profile_name This parameter specifies the name of security profile.

virtual_service This parameter specifies the name of an HTTP or HTTPS virtual service.



Note: The WAF function can be enabled on the virtual service by associating the virtual service with the security profile.

no waf profile assign <profile_name> <virtual_service>

This command is used to remove the association between the security profile and the specified virtual service.

waf profile mode {passive|active} <profile_name>

This command is used to specify the working mode for the specified security profile. The working mode of a security profile can be passive or active. In passive mode, when the attack is detected, the system will record the log, but will not block the traffic or send the error page to the client. In active mode, when the attack is detected, the system will record the log, block the traffic and send the error page to the client. By default, the WAF function works in active mode.

profile_name This parameter specifies the name of security profile.

waf profile predefine <profile_name> <ruleset_name> [action_name] [priority]

This command is used to configure the predefined ruleset for the specified security profile.

profile_name	This parameter specifies the name of security profile.
ruleset_name	This parameter specifies the name of predefined ruleset, which can be Structured Query Language injection (SQL) injection or Cross Site Scripting (XSS). Its value must be “sql” and “xss”, which is case-sensitive.
action_name	Optional. This parameter specifies the action when the attack is detected. Its value must be “pass” or “deny”. - deny: If the parameter value is “deny”, the attack will be detected, the error page will be sent to the client to indicate that the request is blocked, and the log will be recorded. Note: The “deny” action will take effect only when the specified security profile is in active mode. - pass: If the parameter value is “pass”, the attack will be detected and the log will be recorded. The default value is “deny”.
priority	Optional. This parameter specifies the priority of the predefined ruleset. Its value must be an integer ranging from 1 to 2000. The greater the value, the higher the priority is. Note: For redefined rulesets with an identical priority, the execution of the rule sets will be at random. The default value is 1000.

no waf profile predefine <profile_name> <ruleset_name>

This command is used to delete the predefined ruleset from the specified security profile.

waf profile custom <profile_name> <ruleset_name> [action_name] [priority]

This command is used to configure the custom ruleset for the specified security profile. A maximum of 1024 custom rulesets can be configured in the system.

profile_name	This parameter specifies the name of security profile.
ruleset_name	This parameter specifies the name of custom ruleset. Its value must be a string of 1 to 64 characters.
action_name	Optional. This parameter specifies the action when the attack is detected. Its value must be “pass” or “deny”. deny: If the parameter value is “deny”, the attack will be detected, the error page will be sent to the client to indicate that the request is blocked, and the log will be recorded. Note: The “deny” action will take effect only

when the specified security profile is in active mode.

- **pass:** If the parameter value is “pass”, the attack will be detected and the log will be recorded.

The default value is “deny”.

priority

Optional. This parameter specifies the priority of the predefined ruleset. Its value must be an integer ranging from 1 to 2000. The greater the value, the higher the priority is. Note: For redefined rulesets with an identical priority, the execution of the rule sets will be at random.

The default value is 1000.



Note: If the predefined ruleset and the custom ruleset are both configured for a security profile, the rulesets are executed in an ascending order of the ASCII values of the ruleset names.

no waf profile custom <profile_name> <ruleset_name>

This command is used to delete the custom ruleset from the specified security profile.

waf import <ruleset_name|data_name> <url> [type]

This command is used to import the custom ruleset file or the related data file of a custom ruleset.

ruleset_name|data_name This parameter specifies the name of the custom ruleset file or the name of the data file related to a custom ruleset. Its value must be a string of 1 to 64 characters.

url This parameter specifies the HTTP or FTP URL address of the custom ruleset file or data file related to a custom ruleset.

Note: The “path” of the FTP URL (ftp://user:password@host:port/path) should be a relative path.

type Optional. This parameter specifies the type of the imported file. Its value must be “ruleset” or “data”.

The default value is “ruleset”.



Note:

3. If a custom ruleset has a data file, the data file must be imported before the custom ruleset.
4. Most of the Modsecurity rules are supported. For details, please contact Customer Support.

no waf import <ruleset_name|data_name>

This command is used to delete the specified imported custom ruleset or data file.

clear waf import

This command is used to clear all the custom rulesets and data files

show waf ruleset [keyword]

This command is used to display the predefined or custom rulesets whose names matched with the keyword.

keyword	Optional. This parameter specifies the string used to match the ruleset name, which is case sensitive. If the parameter value begins with a numeric character, the string must be enclosed in double quotes. If this parameter is not specified, all predefined and custom rulesets in the system will be displayed.
---------	--

show waf data [keyword]

This command is used to display the custom ruleset-related data files whose names matched with the keyword.

keyword	Optional. This parameter specifies the string used to match the name of the data file related to a custom ruleset, which is case sensitive. If the parameter value begins with a numeric character, the string must be enclosed in double quotes. If this parameter is not specified, all the data files related to custom rulesets
---------	---

show statistics waf [virtual_service]

This command is used to display the related statistics of the WAF function for the specified virtual service. If the “virtual_service” parameter is not specified, all related statistics of the WAF function will be displayed.

waf log audit {on|off}

This command is used to enable or disable the audit log function of the WAF function. The audit log function will record the HTTP traffic information. If the audit log function is disabled, no audit log will be recorded. By default, this function is enabled.

waf log audit server <url> <user_name> <password>

This command is used to define a third-party remote audit log server for the WAF function. If the third-party remote audit log server is defined, audit logs will be sent to the third-party remote audit log server; otherwise, audit logs will be stored on the

local disk. Only one third-party remote audit log server can be configured. For how to install the third-party remote audit log server, please contact Customer Support.

url	This parameter specifies the URL address of the third-party remote audit log server.
user_name	This parameter specifies the user name that is used to log in to the third-party remote audit log server. Its value must be a case-sensitive string of 1 to 64 characters. Numbers, lower-case and upper-case letters, and special characters including “@”, “-”, “_”, “!” are supported. Its value must be enclosed in double quotes when containing any special character or containing only numbers.
password	This parameter specifies the password that is used to log in to the third-party remote audit log server. Its value must be a string of 1 to 32 characters. Numbers, lower-case and upper-case letters, and special characters including “@”, “-”, “_”, “!” are supported. Its value must be enclosed in double quotes when containing any special character or containing only numbers.

**Note:**

1. The size of audit logs will be no more than 2GB on the local disk. If the size is larger than 2GB, new audit logs will overwrite the earliest ones.
2. Before executing the “waf log audit server” command to define a third-party remote audit log server, make sure the server works properly. Otherwise, audit logs will be lost.
3. Audit logs cannot be stored on both the local disk and the third-party remote audit log server.

no waf log audit server

This command is used to delete the third-party remote audit log server of the WAF function.

waf update <url>

This command is used to update the predefined ruleset. Please contact Customer Support for the update address of the predefined ruleset.

url	This parameter specifies the HTTP or FTP URL address of the new predefined rulesets.
-----	--

Note: The “path” of the FTP URL (ftp://user:password@host:port/path) should be a relative path.



Note: During the update, the predefined rulesets will be deleted and updated

while the custom rulesets and the related data files will be reserved.

clear waf config

This command is used to clear all the configurations of the WAF function except predefined rulesets, custom rulesets and data files related to custom rulesets.

Advanced ACL

ACL Rule

acl rule <rule_name> <client_ip> {netmask|prefix} <acl_mode> <acl_type>
<max_limit>

This command is used to configure an ACL rule. If the IP address of a client belongs to the subnet determined by the settings of parameters “client_ip” and “netmask|prefix”, the client access will be subject to the restriction of the ACL rule.

A maximum of 200 ACL rule can be configured in the system.

rule_name	This parameter specifies the name of an ACL rule.
client_ip	This parameter specifies the IP address of a client on a subnet. The value of this parameter can be an IPv4 or IPv6 address.
netmask prefix	This parameter specifies the netmask or prefix length of the subnet. Parameters “client_ip” and “netmask prefix” determines a subnet. • “netmask” is used for an IPv4 address. It can be a dotted IP address or an integer. If it is an integer, its value should range from 0 to 32. • “prefix” is used for an IPv6 address. Its value must be an integer ranging from 0 to 128.
acl_mode	This parameter specifies the control mode of the ACL rule. The value of this parameter can be “total” and “per-ip” only, which is case sensitive. • “total” indicates that all the clients on the subnet will be controlled as a whole by the ACL rule. • “per-ip” indicates that every client of the subnet is controlled individually by the ACL rule.
acl_type	This parameter specifies the control type of the ACL rule. The value of this parameter can be “cps” and “concurrent” only, which is case sensitive. • When “acl_mode” is set to “total”: “cps” indicates that this ACL rule restricts the total connections per second

(CPS) of all clients on the subnet; “concurrent” indicates that this ACL rule restricts the total concurrent connections (CC) of all clients on the subnet.

- When “acl_mode” is set to “per-ip”: “cps” indicates that this ACL rule restricts the CPS of every client on the subnet; “concurrent” indicates that this ACL rule restricts the CC of every client on the subnet.

max_limit

This parameter specifies the maximum number of CPS or CC allowed. The value of this parameter ranges from 1 to 4294967295.

For example:

```
AN(config)#acl rule rule1 210.82.10.0 255.255.255.0 total cps 1000000
```



Note: ACL rules take effect only for the connections established after these rules are configured.

show acl rule [rule_name]

This command is used to display a specified ACL rule. If the parameter “rule_name” is not specified, the system will display all the configured ACL rules.

no acl rule <rule_name>

This command is used to delete a specified ACL rule.



Note: Before deleting an ACL rule, please make sure that this rule is not applied to any virtual service individually or globally.

clear acl rule

This command is used to delete all the ACL rules.



Note: Before deleting all the ACL rules, please make sure that no rule is applied to any virtual service individually or globally.

acl whitelist <whitelist_name> <client_ip> {netmask|prefix}

This command is used to configure an ACL whitelist. If the IP address of a client belongs to the subnet determined by the settings of parameters “client_ip” and “netmask|prefix”, the client access will not be subject to the restriction of any ACL rule.

A maximum of 400 ACL whitelists can be configured in the system.

<code>whitelist_name</code>	This parameter specifies the name of an ACL whitelist.
<code>client_ip</code>	This parameter specifies the IP address of a client on a subnet. The value of this parameter can be an IPv4 or IPv6 address.
<code>netmask prefix</code>	This parameter specifies the netmask or prefix length of the subnet. Parameters “<code>client_ip</code>” and “<code>netmask prefix</code>” determines a subnet. • “<code>netmask</code>” is used for an IPv4 address. It can be a dotted IP address or an integer. If it is an integer, its value should range from 0 to 32. • “<code>prefix</code>” is used for an IPv6 address. Its value must be an integer ranging from 0 to 128.

show acl whitelist [*whitelist_name*]

This command is used to display a specified ACL whitelist. If the parameter “`whitelist_name`” is not specified, the system will display all the configured ACL whitelists.

no acl whitelist <*whitelist_name*>

This command is used to delete a specified ACL whitelist.



Note: Before deleting an ACL whitelist, please make sure that this whitelist is not applied to any virtual service individually or globally.

clear acl whitelist

This command is used to delete all the ACL whitelists.



Note: Before deleting all the ACL whitelists, please make sure that no rule is applied to any virtual service individually or globally.

acl apply rule virtual <*rule_name*> <*vs_name*>

This command is used to apply a specified ACL rule to a virtual service or all the virtual services globally. When a client is accessing the specified virtual service, if it matches the ACL rule, it will be subject to the restriction of this rule.

<code>rule_name</code>	This parameter specifies the name of an ACL rule.
<code>vs_name</code>	This parameter specifies the name of a virtual service. If “<code>vs_name</code>” is set to “<code>global</code>”, the ACL rule is applied to all the virtual services globally.



Note: The Advanced ACL function only supports TCP-based virtual services, such as TCP and HTTP types of virtual services. Therefore, the “**acl apply rule virtual**” configuration takes effect only when the parameter “vs_name” is set to the name of a TCP-based virtual service.

show acl apply rule <rule_name>

This command is used to display the configuration of applying a specified ACL rule to virtual services individually and globally.

rule_name This parameter specifies the name of an ACL rule.

no acl apply rule virtual <rule_name> <vs_name>

This command is used to delete the configuration of applying a specified ACL rule to a specified virtual service. If “vs_name” is set to “global”, the system will delete the configuration of applying the specified ACL rule globally.

clear acl apply rule virtual <vs_name>

This command is used to delete all the configurations of applying ACL rules to a specified virtual service. If “vs_name” is set to “global”, the system will delete all the configurations of applying ACL rules globally.

acl apply whitelist virtual <whitelist_name> <vs_name>

This command is used to apply a specified ACL whitelist to a virtual service or all the virtual services globally. When a client is accessing the specified virtual service, if it matches the ACL whitelist, it will not be subject to the restriction of any ACL rule.

whitelist_name This parameter specifies the name of an ACL whitelist.

vs_name This parameter specifies the name of a virtual service.

If “vs_name” is set to “global”, the ACL whitelist is applied to all the virtual services globally.



Note: The Advanced ACL function only supports TCP-based virtual services, such as TCP and HTTP types of virtual services. Therefore, the “**acl apply whitelist virtual**” configuration takes effect only when the parameter “vs_name” is set to the name of a TCP-based virtual service.

show acl apply whitelist <whitelist_name>

This command is used to display the configuration of applying a specified ACL whitelist to virtual services individually and globally.

whitelist_name This parameter specifies the name of an ACL whitelist.

no acl apply whitelist virtual <whitelist_name> <vs_name>

This command is used to delete the configuration of applying a specified ACL whitelist to a specified virtual service. If “vs_name” is set to “global”, the system will delete the configuration of applying the specified ACL whitelist globally.

clear acl apply whitelist virtual <vs_name>

This command is used to delete all the configurations of applying ACL whitelists to a specified virtual service. If “vs_name” is set to “global”, the system will delete all the configurations of applying ACL whitelists globally.

show acl apply virtual [vs_name]

This command is used to show all the ACL rules and whitelists applied to a specified virtual service.

vs_name	Optional. This parameter specifies the name of a virtual service. If “vs_name” is set to “global”, the system will display all the ACL rules and whitelists applied globally. If this parameter is not specified, the system will display all the ACL rules and whitelists applied to virtual services individually and globally.
---------	---

clear acl all

This command is used to delete all the ACL-related configurations.

show statistics acl rule [rule_name] [vs_name]

This command is used to display the statistics on applying a specified ACL rule to a specified virtual service.

rule_name	Optional. This parameter specifies the name of an ACL rule. If the parameter “rule_name” is not specified, the system will display the statistics on applying all the ACL rules to virtual services individually and globally.
vs_name	Optional. This parameter specifies the name of a virtual service. This parameter is available only when the parameter “rule_name” has been specified. If “vs_name” is set to “global”, the system will display the statistics on applying the specified ACL rule globally. If this parameter is not specified, the system will display

the statistics on applying the specified ACL rule to virtual services individually and globally.

show statistics acl whitelist [whitelist_name] [vs_name]

This command is used to display the statistics on applying a specified ACL whitelist to a specified virtual service.

whitelist_name	Optional. This parameter specifies the name of an ACL whitelist. If the parameter “rule_name” is not specified, the system will display the statistics on applying all the ACL whitelists to virtual services individually and globally.
vs_name	Optional. This parameter specifies the name of a virtual service. This parameter is available only when the parameter “rule_name” has been specified. If “vs_name” is set to “global”, the system will display the statistics on applying the specified ACL whitelist globally. If this parameter is not specified, the system will display the statistics on applying the specified ACL whitelist to virtual services individually and globally.

show statistics acl virtual [vs_name]

This command is used to display the statistics on applying all the ACL rules and whitelists to a specified virtual service.

vs_name	Optional. This parameter specifies the name of a virtual service. If “vs_name” is set to “global”, the system will display the statistics on applying all the ACL rules and whitelists globally. If this parameter is not specified, the system will display the statistics on applying all the ACL rules and whitelists to virtual services individually and globally.
---------	--

clear statistics acl

This command is used to delete the statistics on applying all the ACL rules and whitelists to virtual services individually and globally.

HTTP ACL Rule

acl http rule <rule_name> <client_ip> {netmask|prefix} <acl_type>
<max_limit> <condition>

This command is used to configure an HTTP ACL rule. A maximum of 1000 HTTP ACL rules are supported.

rule_name	This parameter specifies the name of the HTTP ACL rule. Its value must be a string of 1 to 40 characters. Letters, numbers, and special characters are supported. When the value contains special characters, it must be enclosed in double quotes.
client_ip	This parameter specifies the client's network IP address. Its value must be an IPv4 or IPv6 address.
netmask/prefix	This parameter specifies the netmask or prefix for the client's network IP address: • “netmask” is used for setting the netmask of the IPv4 address. Its value must be in dotted decimal notation or an integer ranging from 0 to 32. • “prefix” is used for setting the prefix length of the IPv6 address. Its value must be an integer ranging from 0 to 128.
acl_type	This parameter specifies the rate limitation type of the HTTP ACL rule. Its value must be: • rps: indicates rate limitation based on the number of requests per second (RPS). • throughput: indicates rate limitation based on the download speed.
max_limit	This parameter specifies a threshold. • When the “acl_type” parameter is set to “rps”, this parameter specifies the threshold for the RPS. Its value must be an integer ranging from 1 to 20,000,000. • When the “acl_type” parameter is set to “throughput”, this parameter specifies the threshold for the download speed, in KB/s. Its value must be an integer ranging from 10 to 102,400.
condition	This parameter specifies the matching condition for the HTTP ACL rule. Its value must be a string of 1 to 300 characters. 1. When the “acl_type” parameter is set to “rps”, this parameter value must contain the following keywords: • url: indicates that RPS limitation will be implemented

upon this URL address. Its value must be in the format of “url=<regex>url_regex”. The value of “url_regex” must be a regular expression string of 1 to 256 characters.

- **method:** indicates the HTTP access method. Its value must be “ALL”, “GET”, “POST”, “HEAD”, “PUT” or “DELETE”, in the format of “method=value”, such as “method=ALL”.
- **action:** indicates the mode of processing HTTP requests when the RPS exceeds the threshold. Its value must be “errorpage” or “reset”, in the format of “action=value”, such as “action=errorpage”.
- When “action=errorpage” is set, the system will return a built-in 480 error page.
- To redirect HTTP requests, execute the “**http redirect error**” command to configure error page redirect.
- To return a customized 480 error page, execute the “**http import error**” and “**http error**” commands to import and enable the customized 480 error page.
- If both error page redirect and customized 480 error page are configured, the system preferentially returns a redirect response.

When “action=reset” is set, the system will send an RST packet to reset the connection if RPS threshold-crossing occurs.

The three keywords must be separated with spaces and enclosed in double quotes. For example, “url=<regex>abc* method=ALL action=reset”

2. When the “acl_type” parameter is set to “throughput”, this parameter value contains only the keyword “url”, in the format of “url=<regex>url_regex”. Its value must be a regular expression string of 1 to 255 characters.

Note: The value of “url” must be a Perl-based regular expression beginning with “<regex>”. If Perl meta characters need to be matched, the symbol “\” should be used before the meta character for escape. Meta characters supported by the system include “^”, “.”, “+”, “*”, “?”, “\$”, “[”, “]”, “|”, and “()”. For example, “*” means matching zero or more characters ahead of it. To match “*”, set “*”.

no acl http rule <rule_name>

This command is used to delete the specified HTTP ACL rule. Before attempting to delete an HTTP ACL rule, make sure it is dissociated from all virtual services.

show acl http rule *[rule_name]*

This command is used to display the specified HTTP ACL rule. If the “rule_name” parameter is not specified, all HTTP ACL rules are displayed.

clear acl http rule auto

This command is used to clear all HTTP ACL rules that are dynamically generated by the system. After this command is executed, the system will generate new dynamic HTTP ACL rules.

clear acl http rule manual

This command is used to clear all manually configured HTTP ACL rules.

acl http apply rule virtual *<rule_name>* *<virtual_service>*

This command is used to associate the specified HTTP ACL rule with the specified virtual service. An HTTP ACL rule takes effect only after it is associated with a virtual service. A maximum of 10,000 association configurations are supported.

rule_name This parameter specifies the name of the HTTP ACL rule.

virtual_service This parameter specifies the virtual service name.

no acl http apply rule virtual *<rule_name>* *<virtual_service>*

This command is used to dissociate the specified HTTP ACL rule from the specified virtual service.

show acl http apply rule *<rule_name>*

This command is used to display all virtual services associated with the specified HTTP ACL rule.

show acl http apply virtual *[virtual_service]*

This command is used to display the configurations of the specified virtual service and all HTTP ACL rules associated with it.

show acl http all

This command is used to display all HTTP ACL rules, virtual services associated with them, and rule matching statistics for individual virtual services.

clear acl http apply virtual *<virtual_service>*

This command is used to dissociate the specified virtual service from all HTTP ACL rules.

show statistics acl http virtual [virtual_service]

This command is used to display the statistics of HTTP ACL rules applied to the specified virtual service. If the “virtual_service” parameter is not specified, the statistics of HTTP ACL rules applied to all virtual services are displayed.

DNS ACL Rule

**acl dns rule <rule_name> <client_ip> {netmask|prefix} <acl_mode>
<dns_type> <max_limit>**

This command is used to configure a DNS ACL rule. A maximum of 1000 DNS ACL rules can be configured.

rule_name	This parameter specifies the DNS ACL rule name. Its value must be a string of 1 to 40 characters. Letters, numbers and special characters are supported. When the parameter value contains special characters, it must be enclosed in double quotes.
client_ip	This parameter specifies the clients’ network IP address. Its value must be an IPv4 or IPv6 address.
netmask/prefix	This parameter specifies the netmask or prefix for the client’s network IP address: • “netmask” is used for setting the netmask of the IPv4 address. Its value must be in dotted decimal notation or an integer ranging from 0 to 32. • “prefix” is used for setting the prefix length of the IPv6 address. Its value must be an integer ranging from 0 to 128.
acl_mode	This parameter specifies the control mode of the DNS ACL rule. Its value must be “total” or “per-ip”, which is case-sensitive. • “total”: indicates that all the clients on the subnet will be controlled as a whole by the DNS ACL rule. • “per-ip”: indicates that every client of the subnet is controlled individually by the DNS ACL rule.
dns_type	This parameter specifies the type of the DNS resource records. Its value must be “A”, “NS”, “MD”, “MF”, “CNAME”, “SOA”, “MB”, “MG”, “MR”, “NULL”, “WKS”, “PTR”, “HINFO”, “MINFO”, “MX”, “TXT”, “AAAA”, “ANY”, “OTHERS” or “ALL”.

max_limit This parameter specifies the maximum number of DNS queries per second that is allowed. Its value must be an integer ranging from 1 to 4,294,967,295.

no acl dns rule <rule_name>

This command is used to delete the specified DNS ACL rule.

show acl dns rule [rule_name]

This command is used to display the specified DNS ACL rule. If the “rule_name” parameter is not specified, all DNS ACL rules are displayed.

clear acl dns rule auto

This command is used to clear all DNS ACL rules that are dynamically generated by the system. After this command is executed, the system will generate new dynamic DNS ACL rules.

clear acl dns rule manual

This command is used to clear all manually configured DNS ACL rules.

acl dns apply rule virtual <rule_name> <virtual_service>

This command is used to associate the specified DNS ACL rule with the specified virtual service. A DNS ACL rule takes effect only after it is associated with a virtual service. A maximum of 40,000 association configurations are supported.

rule_name This parameter specifies the DNS ACL rule name.

virtual_service This parameter specifies the virtual service name. Its value must be:

- DNS virtual service name: associates the DNS ACL rule with this virtual service.
- global: associates the DNS ACL rule with all DNS virtual services.

no acl dns apply rule virtual <rule_name> <virtual_service>

This command is used to dissociate the specified DNS ACL rule from the specified virtual service.

show acl dns apply rule <rule_name>

This command is used to display all virtual services associated with the specified DNS ACL rule.

show acl dns apply virtual [virtual_service]

This command is used to display the DNS ACL rules associated with the specified virtual service. If the “virtual_service” parameter is not specified, the DNS ACL rules associated with all virtual services are displayed.

show acl dns all

This command is used to display all DNS ACL rules, virtual services associated with them, and rule matching statistics for individual virtual services.

clear acl dns apply rule virtual <virtual_service>

This command is used to dissociate the specified virtual service from all DNS ACL rules.

virtual_service This parameter specifies the virtual service name. Its value must be:

- DNS virtual service name: dissociates this virtual service rule from all DNS ACL rules.
- global: dissociates all DNS virtual services from DNS ACL rules.

show statistics acl dns virtual [virtual_service]

This command is used to display the statistics of DNS ACL rules applied to the specified virtual service. If the “virtual_service” parameter is not specified, the statistics of DNS ACL rules applied to all virtual services are displayed.

acl dns apply rule sdns <rule_name>

This command is used associate a DNS ACL rule with the SDNS module.

no acl dns apply rule sdns <rule_name>

This command is used to dissociate the specified DNS ACL rule from the SDNS module.

show acl dns apply sdns

This command is used to display all DNS ACL rules associated with the SDNS module.

clear acl dns apply rule sdns

This command is used to dissociate all DNS ACL rules from the SDNS module.

show statistics acl dns sdns

This command is used to display the statistics of DNS ACL rules applied to the SDNS module.

DDoS Attack Defense

ddos {on|off}

This command is used to enable or disable the Distributed Denial of Service (DDoS) attack defense function. To limit requests per second (RPS) or throughput as specified in the HTTP ACL rules, this feature needs to be enabled. By default, it is disabled.

topn {on|off} <protocol_type>

This command is used to enable or disable the real-time TopN traffic statistics function for the specified protocol type. After this function is enabled, the system will collect the top 10 clients and servers running IP, TCP, UDP and ICMP traffic. By default, this function is disabled.

protocol_type This parameter specifies the protocol type. Its value must be “ip”, “tcp”, “udp” or “icmp”.

show topn status <type>

This command is used to display the top 10 clients and servers running traffic of the specified protocol type.

show topn settings

This command is used to display the enabling status of the real-time TopN traffic statistics function.

clear topn settings

This command is used to restore the real-time TopN traffic statistics function to the default setting.

show ddos settings

This command is used to display the enabling status of all DDoS attack defense functions and related configurations.

clear ddos settings

This command is used to reset all DDoS attack defense functions and related configuration to the default setting.

show ddos blacklist

This command is used to display the DDoS blacklist. This command displays only the latest 2000 blacklist records. If the system is rebooted, all blacklist records will be cleared.

clear ddos blacklist

This command is used to clear all DDoS blacklist records.

show ddos record [filter_string] [start_date] [end_date]

This command is used to display the DDoS attack records matching the specified filter string within a specific period.

filter_string	Optional. This parameter specifies the filter string used to match DDoS attack records. If this parameter is not specified, all DDoS attack records within the specific period will be displayed.
start_date	Optional. This parameter specifies the start date. Its value must be in the format of “yyyymmdd”, in which the value of “yyyy” must be an integer ranging from 2000 to 2037. If this parameter is not specified, all DDoS attack records will be displayed.
end_date	Optional. This parameter specifies the end date. Its value must be in the format of “yyyymmdd”, in which the value of “yyyy” must be smaller than that of the “start_date”. If this parameter is not specified, all DDoS attack records will be displayed.

ddos record export <remote_file_path>

This command is used to export DDoS attack records to the specified URL address for backup.

remote_file_path	This parameter specifies the URL address. Its value must contain the directory for storing the DDoS attack records, such as “”. Only FTP URL addresses are supported.
------------------	---

Note: The “path” of the FTP URL (ftp://user:password@host:port/path) should be a relative path.

clear ddos record

This command is used to clear all DDoS attack records saved in the system.

TCP DDoS**ddos tcp {on|off}**

This command is used to enable or disable the TCP DDoS attack defense function. When the system DDoS attack defense is disabled, the TCP DDoS attack defense function does not take effect. By default, this function is enabled.

show statistics ddos tcp

This command is used to display TCP DDoS attack statistics.

clear statistics ddos tcp

This command is used to clear TCP DDoS attack statistics.

UDP DDoS

ddos udp {on|off}

This command is used to enable or disable the UDP DDoS attack defense function. When the system DDoS attack defense function is disabled, the UDP DDoS attack defense function does not take effect. By default, this function is enabled.

show statistics ddos udp

This command is used to display UDP DDoS attack statistics.

clear statistics ddos udp

This command is used to clear UDP DDoS attack statistics.

DNS DDoS

ddos dns {on|off}

This command is used to enable or disable the DNS DDoS attack defense function. When the system DDoS attack defense function is disabled, the DNS DDoS attack defense function does not take effect. By default, this function is enabled.

show statistics ddos dns

This command is used to display DNS DDoS attack (including TCP-based and UDP-based DNS DDoS attacks) statistics.

clear statistics ddos dns all

This command is used to clear the DDoS attack statistics of all resource record types on all DNS (including TCP-based and UDP-based DNS) virtual services.

clear statistics ddos dns virtual <virtual_service> [dns_type]

This command is used to clear the DDoS statistics of the specified resource record type on a DNS virtual service.

virtual_service This parameter specifies the DNS virtual service name.

dns_type Optional. This parameter specifies the type of the DNS resource records. Its value must be “A”, “NS”, “MD”, “MF”, “CNAME”, “SOA”, “MB”, “MG”, “MR”, “NULL”, “WKS”, “PTR”, “HINFO”, “MINFO”, “MX”,

“TXT”, “AAAA”, “ANY”, “OTHERS” or “ALL”.

The default value is “ALL”.

SSL DDoS

ddos ssl handshake timeout *[timeout]*

This command is used to set the timeout value for SSL handshakes. After a TCP connection is established, if no handshake message or application data is received within the specified time, the system will reset the TCP connection and record an SSL handshake timeout attack. If this command is not configured, the system will use 10,000 ms as the default timeout value for SSL handshakes after initial startup. Then it will generate a dynamic timeout value as the actual traffic fluctuates and use the dynamically generated timeout value for detection. If a timeout value is defined using this command, the dynamically generated timeout value does not take effect.

timeout	Optional. This parameter specifies the timeout value for SSL handshakes. Its value must be an integer ranging from 0 to 7,200,000, in milliseconds. When the value is 0, the system does not check whether SSL handshakes time out.
---------	---

The default value is 10,000.

no ddos ssl handshake timeout

This command is used to restore the timeout value for SSL handshakes to the default value. After this command is executed, the system will use the dynamically generated timeout value for detection.

ddos ssl renegotiation interval *[interval]*

This command is used to set a threshold for the interval between two SSL renegotiation requests. If the interval between two renegotiation requests is smaller than the specified threshold, the system will reset the connection and record an SSL renegotiation attack. If the parameter “interval” is not specified, the default value is 10,000 milliseconds.

interval	Optional. This parameter specifies the threshold for the interval between two SSL renegotiation requests. Its value must be an integer ranging from 1 to 7,200,000, in milliseconds.
----------	--

The default value is 10,000.

no ddos ssl renegotiation interval

This command is used to restore the threshold for the interval between two SSL renegotiation requests to the default value.

show ddos threshold ssl

This command is used to display the dynamically generated timeout value for SSL handshakes.

clear ddos threshold ssl

This command is used to clear the dynamically generated timeout value for SSL handshakes. After this command is executed, the system will regenerate a timeout value for SSL handshakes.

show statistics ddos ssl [virtual_service]

This command is used to display the statistics used for generating the dynamic SSL attack threshold.

clear statistics ddos ssl [virtual_service]

This command is used to clear the statistics used for generating the dynamic SSL attack threshold.

HTTP DDoS**ddos http verify {on|off}**

This command is used to enable or disable the HTTP verification function. After this function is enabled, the system will implement verification on suspicious clients to judge whether they are attackers. By default, this function is disabled.

ddos http acl blacklist {on|off}

This command is used to control whether to automatically add the IP addresses of clients that launch HTTP DDoS attacks to the ACL blacklist. By default, this function is disabled.

ddos http slow interval [interval]

This command is used to set a threshold for the interval between two HTTP datagrams. If the interval between two datagrams of an HTTP header or POST packet exceeds the specified threshold, the system will record a timeout event. Three times of timeout events will trigger a connection reset, and a Slowloris or Slowpost attack will be recorded. If this command is not configured, the system will use 2000 ms as the default threshold after initial startup. Then it will generate a dynamic threshold as the actual traffic fluctuates and use the dynamically generated threshold for detection. If a threshold is defined using this command, the dynamically generated threshold does not take effect.

interval	Optional. This parameter specifies the threshold for the interval between two HTTP datagrams. Its value must be an integer ranging from 0 to 7,200,000. When the value is 0, the system does not check the interval between HTTP
----------	--

datagrams.

The default value is 2000.

no ddos http slow interval

This command is used to restore the threshold for the interval between two HTTP datagrams to the default value. After this command is executed, the system will use the dynamically generated threshold for detection.

ddos http resptime <method> <interval>

This command is used to configure the system to detect the response time of HTTP servers by using the specified HTTP method, and set a threshold for the HTTP servers' response time. If the average response time of all HTTP servers exceeds the specified threshold and this fact persists for more than 30s, the HTTP servers are considered to be under a possible attack. If this command is not configured, the system will use the following default settings for detection after initial startup.

- **ddos http resptime get 100**
- **ddos http resptime post 100**
- **ddos http resptime head 100**
- **ddos http resptime put 100**
- **ddos http resptime delete 100**

Then it will generate dynamic thresholds as the actual traffic fluctuates and use the dynamically generated thresholds for detection. If a threshold is defined using this command, the dynamically generated threshold does not take effect.

method	This parameter specifies the HTTP method used for detection. Its value must be “get”, “post”, “head”, “put” or “delete”.
--------	--

interval	This parameter specifies the threshold for the response time of HTTP servers. Its value must be an integer ranging from 1 to 10000, in milliseconds.
----------	--

The default value is 100.

no ddos http resptime <method>

This command is used to restore the threshold for the HTTP servers' response time in replying the specified type of HTTP requests to the default value. After this command is executed, the system will use the dynamically generated threshold for detection.

ddos http kvincookie <count>

This command is used to set a threshold for the number of cookies carried in an HTTP request. If the number of cookies carried in an HTTP request exceeds the threshold, the HTTP server is considered to be under a possible attack. If this command is not configured, the system will use 10 as the default threshold after initial startup. Then it will generate a dynamic threshold as the actual traffic fluctuates, and use the dynamically generated threshold for detection. If a threshold is defined using this command, the dynamically generated threshold does not take effect.

count This parameter specifies the threshold for the number of cookies carried in an HTTP request. Its value must be an integer ranging from 1 to 100.

The default value is 100.

no ddos http kvincookie

This command is used to restore the threshold for the number of cookies carried in an HTTP request to the default setting. After this command is executed, the system will use the dynamically generated threshold for detection.

ddos http querycnt <count>

This command is used to set a threshold for the number of queries contained in a URL. If the number of queries contained in a URL exceeds the threshold, the HTTP server is considered to be under a possible attack. If this command is not configured, the system will use 10 as the default threshold after initial startup. Then it will generate a dynamic threshold as the actual traffic fluctuates, and use the dynamically generated threshold for detection. If a threshold is defined using this command, the dynamically generated threshold does not take effect.

count This parameter specifies the threshold for the number of queries contained in a URL. Its value must be an integer ranging from 1 to 100.

The default value is 10.

no ddos http querycnt

This command is used to restore the threshold for the number of queries contained in a URL to the default setting. After this command is executed, the system will use the dynamically generated threshold for detection.

show ddos threshold http [virtual_service]

This command is used to display the HTTP attack thresholds that the system dynamically generated for the specified virtual service, including the threshold for the HTTP servers' response time, the threshold for the number of cookies carried in an HTTP request and the threshold for the number of queries contained in a URL. If the

“virtual_service” parameter is not specified, the HTTP attack thresholds that the system dynamically generated for all virtual services are displayed.

clear ddos threshold http [virtual_service]

This command is used to clear the HTTP attack thresholds that the system dynamically generated for the specified virtual service, including the threshold for the HTTP servers’ response time, the threshold for the number of cookies carried in an HTTP request and the threshold for the number of queries contained in a URL. If the “virtual_service” parameter is not specified, the HTTP attack thresholds that the system dynamically generated for all virtual services are cleared. After this command is executed, the system will regenerate dynamic HTTP attack thresholds.

show statistics ddos http [virtual_service]

This command is used to display the HTTP DDoS attack statistics on the specified virtual service. If the “virtual_service” parameter is not specified, the HTTP DDoS attack statistics on all virtual services will be displayed.

clear statistics ddos http [virtual_service]

This command is used to clear the HTTP DDoS attack statistics on the specified virtual service. If the “virtual_service” parameter is not specified, the HTTP DDoS attack statistics on all virtual services will be cleared.

ACL Blacklist

acl blacklist {on|off}

This command is used to enable or disable the ACL blacklist function. After this function is enabled, the system will check the source IP addresses of all packets. If a source IP address matches the ACL blacklist, the system will drop the packets coming from it. By default, this function is disabled.

acl blacklist rule <ip> <netmask|prefix> [timeout]

This command is used to add the specified IP address or subnet to the ACL blacklist, and set a timeout value. When the specified time elapses, the IP address or subnet will be removed from the ACL blacklist.

ip	This parameter specifies the IP address to be added to the ACL blacklist.
netmask prefix	This parameter specifies the netmask or prefix for the IP address to be added to the ACL blacklist: • “netmask” is used for setting the netmask of the IPv4 address. Its value must be in dotted decimal notation or an integer ranging from 0 to 32. • “prefix” is used for setting the prefix length of the IPv6 address. Its value must be an integer ranging

from 0 to 128.

timeout Optional. This parameter specifies the timeout value for the IP address or subnet added to the ACL blacklist. Its value must be an integer ranging from 0 to 1440, in minutes. The value 0 indicates no timeout.

The default value is 0.

no acl blacklist rule <ip> <netmask|prefix>

This command is used to manually remove an IP address or subnet from the ACL blacklist.

show acl blacklist rule manual

This command is used to display all IP addresses and subnets added to the ACL blacklist by using the “**acl blacklist rule**” command.

clear acl blacklist rule manual

This command is used to remove all IP address and subnets added to the ACL blacklist by the “**acl blacklist rule**” command.

show acl blacklist rule auto

This command is used to display all ACL blacklists that the system’s security module has dynamically generated.

acl blacklist ipfile generate <security_module> <filename>

This command is used to save the ACL blacklists that the system dynamically generated for the specified security mode to the specified local file.

security_module This parameter specifies the security module name. Its value must be “ddos” or “waf”, which is case-sensitive.

filename This parameter specifies the name of the local file used to store the ACL blacklist. Its value must be a string or 1 to 24 characters.

clear acl blacklist rule auto

This command is used to clear all ACL blacklists that the system dynamically generated.

acl blacklist import <remote_file_path>[filename]

This command is used to import a self-defined IP list file from the specified URL address. The size of each imported IP list file cannot exceed 1000 KB.

remote_file_path	This parameter specifies the URL address where the IP list file is stored. Its value must contain the IP list file name, such as “ftp://10.8.3.28/iplist”. HTTP and FTP URLs are supported.
filename	Optional. This parameter is used to rename the imported IP list file. Its value must be a string of 1 to 24 characters. If this parameter is not specified, the IP list file name is not changed.

clear acl blacklist ipfile *[filename]*

This command is used to clear the specified IP list file. If the “filename” parameter is not specified, all IP list files will be cleared.

acl blacklist export <local_file> <remote_file_path>

This command is used to export an IP list file to the specified URL address.

local_file	This parameter specifies the name of the IP list file to be exported.
remote_file_path	This parameter specifies the URL address. Its value must contain the directory used to store the IP list file to be exported. Only FTP URLs are supported.

Note: The “path” of the FTP URL (ftp://user:password@host:port/path) should be a relative path.

acl blacklist ipfile apply <filename> *[timeout]*

This command is used to add the specified IP list to the ACL blacklist, and set a timeout value. When the specified time elapses, the IP list will be removed from the ACL blacklist. A maximum of 10 IP list files can be added to the ACL blacklist.

filename	This parameter specifies the IP list file name.
timeout	Optional. This parameter specifies the timeout value of the IP list file. Its value must be an integer ranging from 0 to 1440, in minutes. The value 0 indicates no timeout.

The default value is 0.

no acl blacklist ipfile apply <filename>

This command is used to remove an IP list file from the ACL blacklist.

show acl blacklist ipfile apply

This command is used to display all IP list files that have been added to the ACL blacklist.

clear acl blacklist apply

This command is used to remove all IP list files from the ACL blacklist.

show acl blacklist ipfile filename [filename]

This command is used to display the IP addresses contained in an IP list file. If the “filename” parameter is not specified, all IP list names that have been imported by the “acl blacklist import” command will be displayed.

show acl blacklist match <ip>

This command is used to check whether an IP address is contained in the ACL blacklist.

ip This parameter specifies the IP address to be checked.

show acl blacklist settings

This command is used to display the enabling status of the ACL blacklist and all related configurations.

show acl blacklist status

This command is used to display the timeout status of all ACL blacklists, including the ACL blacklists configured by the “**acl blacklist rule**” command and automatically generated by the system and the IP list files applied to the ACL blacklists.

clear acl blacklist all

This command is used to restore the ACL blacklist function to the default setting and clear all related configurations.

acl blacklist source tcpoption <tcp_option_kind> [offset]

This command is used to configure a blacklist match rule for checking whether the client IP address in the specified TCP option matches the blacklist.

After this command is configured, with the ACL blacklist function enabled (“**acl blacklist on**”), when receiving a TCP handshake or data packet from the client, in addition to the source IP address, the system will also match the client IP address in the specified TCP option with the ACL blacklist. When either the source IP address or client IP address in the TCP option matches the blacklist, the packet will be discarded.

tcp_option_kind This parameter specifies the kind value of the TCP option. It must be an integer ranging from 20 to 252.

offset Optional. This parameter specifies the offset of the TCP option kind's Value field, in bytes. Its value must be an integer ranging from 0 to 4.

The default value is 0.

no acl blacklist source tcpoption

This command is used to delete the configuration of the “**acl blacklist source tcpoption**” command.

show acl blacklist source tcpoption

This command is used to display the configuration of the “**acl blacklist source tcpoption**” command.

show statistics acl blacklist

This command is used to display all ACL blacklist statistics.

clear statistics acl blacklist

This command is used to clear all ACL blacklist statistics.

Chapter 22 Advanced IPv6 Configuration

DNS64 and NAT64

ipv6 dns64 on <dns_vs_name> [priority_mode]

This command is used to enable the DNS64 function for a specified DNS virtual service and configure the resource priority mode. By default, the DNS64 function is disabled.

The DNS64 function converts the DNS AAAA queries sent from IPv6 clients to DNS A queries and then converts the DNS A responses to DNS AAAA responses. This ensures that IPv6 clients can access IPv4 servers.

The DNS64 function can be enabled on only one DNS virtual service.

dns_vs_name This parameter specifies the name of a DNS virtual service.

priority_mode Optional. This parameter specifies the resource priority mode. Its value must be:

- A: preferentially sends the DNS A query to the DNS A server and selects the DNS A response.
- AAAA: preferentially sends the DNS AAAA query to the DNS AAAA server and selects the DNS AAAA response.
- Response: sends the DNS A query to the DNS A server and DNS AAAA query to DNS AAAA server and preferentially selects the resource record in the response that is received first.

The values above are case-insensitive. The default value is “AAAA”.

ipv6 dns64 off <dns_vs_name>

This command is used to disable the DNS64 function for a specified DNS virtual service.

ipv6 dns64 prefix <dns64_prefix>

This command is used to specify the IPv6 prefix used by the DNS64 function. The DNS64 function converts A records to AAAA records by adding the IPv6 prefix.

dns64_prefix This parameter specifies the IPv6 prefix used by the DNS64 function. The fixed length of this parameter is 96

bits. The default value is 64:ff9b::.

show ipv6 dns64 settings

This command is used to display all the settings related to the DNS64 function.

clear ipv6 dns64 settings

This command is used to delete all the settings related to the DNS64 function and restore the default settings related to this function.

ipv6 nat64 on

This command is used to enable the NAT64 (Network Address Translation IPv6 to IPv4) function. By default, the NAT64 function is disabled.

The NAT64 function converts the IPv6 packets sent by IPv6 clients to IPv4 packets and then converts IPv4 packets sent by IPv4 servers to IPv6 packets. This ensures that IPv6 clients can communicate with IPv4 servers normally.

ipv6 nat64 off

This command is used to disable the NAT64 function.

ipv6 nat64 ippool <ipv4_pool_name>

This command is used to configure the IPv4 address pool used by the NAT64 function globally. When the APV appliances receives IPv6 packets from IPv6 clients, the NAT64 function translates the source IPv6 addresses in the packets to the IPv4 addresses in this IPv4 address pool.

`ipv4_pool_name` This parameter specifies the name of the IPv4 address pool.

no ipv6 nat64 ippool <ipv4_pool_name>

This command is used to delete the IPv4 address pool used by the NAT64 function globally.

ipv6 nat64 prefix <nat64_prefix>

This command is used to configure the IPv6 prefix used by the NAT64 function. When the APV appliance receives IPv6 packets from IPv6 clients, the NAT64 function will cut the IPv6 prefix of the destination addresses to obtain the IPv4 addresses of real servers.

`nat64_prefix` This parameter specifies the IPv6 prefix used by the NAT64 function. The fixed length of this parameter is 96 bits.

If the DNS64 and NAT64 functions need to work together

on one APV appliance, make sure that the value of this parameter is the same as that of the parameter “dns64_prefix” in the command “**ipv6 dns64 prefix** *<dns64_prefix>*”.



Note: If the destination addresses of the IPv6 packets received by the APV appliance match not only the parameter “dns64_prefix” but also other VIPs, these packets will not hit NAT64. Therefore, NAT64 will not be used to forward these packets.

ipv6 nat64 timeout *<idle_timeout>*

This command is used to set the idle timeout period for NAT64 TCP connections.

idle_timeout	This parameter specifies the idle timeout period, in seconds, for NAT64 TCP connections. The value of this parameter ranges from 60 to 7200. The default value is 300.
--------------	--

show ipv6 nat64 connection

This command is used to display the information about the address translation on NAT64 connections.

show ipv6 nat64 settings

This command is used to display all the settings related to the NAT64 function.

clear ipv6 nat64 settings

This command is used to delete all the settings related to the NAT64 function and restore the default settings related to this function.

DNS46 and NAT46

ipv6 dnsnat46 on *<dns_vs_name>* [*priority_mode*]

This command is used to enable both the DNS46 and NAT46 functions for a specified DNS virtual service and configure the resource priority mode. By default, the DNS46 and NAT46 functions are disabled.

The DNS46 function converts the DNS A queries sent from IPv4 clients to DNS AAAA queries, and then converts the DNS AAAA responses to DNS A responses. It also creates address mapping records between the IPv6 addresses and IPv4 addresses. The APV appliance returns the translated IPv4 addresses to IPv4 clients. When IPv4 clients use these IPv4 addresses to access IPv6 servers, the NAT46 function converts IPv4 packets sent from clients to IPv6 packets based on the created mapping records. When the APV appliance receives IPv6 packets from IPv6 servers, the NAT46

function converts IPv6 packets to IPv4 packets. This ensures that IPv4 clients can communicate with IPv6 servers normally.

The DNS46 and NAT46 functions can be enabled on only one DNS virtual service.

<code>dns_vs_name</code>	This parameter specifies the name of a DNS virtual service.
<code>priority_mode</code>	Optional. This parameter specifies the resource priority mode. Its value must be: • A: preferentially sends the DNS A query to the DNS A server and selects the DNS A response. • AAAA: preferentially sends the DNS AAAA query to the DNS AAAA server and selects the DNS AAAA response. • Response: sends the DNS A query to the DNS A server and DNS AAAA query to DNS AAAA server and preferentially selects the resource record in the response that is received first.

The values above are case-insensitive. The default value is “AAAA”.

ipv6 dnsnat46 off <dns_vs_name>

This command is used to disable both the DNS46 and NAT46 functions for a specified DNS virtual service.

ipv6 dnsnat46 ipmap <ipv4_address> <netmask> [timeout]

This command is used to configure the IPv4 mapping subnet used by both the DNS46 and NAT46 functions. The IP addresses in this subnet will be used to create mappings between IPv6 addresses and IPv4 addresses.

When receiving DNS A queries from IPv4 clients, the APV appliance sends DNS A queries and DNS AAAA queries to DNS servers in sequence. If the DNS servers return only DNS AAAA records, the APV appliance will select available IPv4 addresses from this subnet as the resolving IP addresses of the mapped A records and then return the mapped A records to IPv4 clients. Meanwhile, the APV appliance will add mapping records to the mapping table to record the mappings between IPv6 addresses and IPv4 addresses. When the APV appliance receives IPv4 packets from IPv4 clients, the NAT46 function will translate the destination address in the IPv4 packets to the corresponding IPv6 addresses according to the created IP mapping records.

<code>ipv4_address</code>	This parameter specifies an IP address of the IPv4 subnet.
<code>netmask</code>	This parameter specifies the netmask of the subnet. This parameter should be a dotted IP address. The value of this

parameter ranges from 255.255.0.0 to 255.255.255.255.

Parameters “ipv4_address” and “netmask” together determine a unique IPv4 subnet.

timeout Optional. This parameter specifies the timeout period, in seconds, for the mapping records between IPv6 addresses and IPv4 addresses. The value of this parameter ranges from 1 to 86,400. The default value is 600.

no ipv6 dnsnat46 ipmap <ipv4_address> <netmask>

This command is used to delete the IPv4 mapping subnet used by both the DNS46 and NAT46 functions.

show ipv6 dnsnat46 ipmap

This command is used to display the address mapping records used by both the DNS46 and NAT46 functions.

ipv6 dnsnat46 ippool <ipv6_pool_name>

This command is used to configure the IPv6 address pool used by the NAT46 function globally. When the APV appliance receives IPv4 packets sent from IPv4 clients, the NAT46 function translates the source IPv4 addresses in the packets to IPv6 addresses in this IPv6 address pool.

ipv6_pool_name This parameter specifies the name of the IPv6 address pool.

no ipv6 dnsnat46 ippool <ipv6_pool_name>

This command is used to delete the IPv6 address pool used by the NAT46 function globally.

ipv6 dnsnat46 timeout <idle_timeout>

This command is used to set the idle timeout period for NAT46 TCP connections.

idle_timeout This parameter specifies the idle timeout period in seconds for NAT46 TCP connections. The value of this parameter ranges from 60 to 7200. The default value is 300.

show ipv6 dnsnat46 connection

This command is used to display the information about the address translation on NAT46 connections.

show ipv6 dnsnat46 settings

This command is used to display all the settings related to the DNS46 and NAT46 functions.

clear ipv6 nat64 settings

This command is used to delete all the settings related to the DNS46 and NAT46 functions and restore the default settings related to each function.

show statistics ipv6 nat64

This command is used to display the statistics on the DNS64, NAT64, DNS46 and NAT46 functions.

General Settings

ipv6 tune dnsnat priority time *[priority_time]*

This command is used to configure the priority timer for the A and AAAA priority modes. This timer indicates that if no response is returned to the query that is preferentially sent, the cached query will then be sent. The priority timer does not work in the response priority mode.

When this command is not configured, the default priority timer is 100 ms.

priority_time	This parameter specifies the priority timer, in milliseconds. Its value must be an integer ranging from 1 to 5000. The recommended value range is 100 to 500.
---------------	---

The default value is 100.

ipv6 tune dnsnat response timeout *[timeout]*

This command is used to configure the timeout value when the system waits for a response from the DNS server.

In the A and AAAA priority modes, this setting indicates the timeout value when the system waits for a response after the cached query is sent. In the response priority mode, this setting indicates the timeout value when the system waits for a response after the query is sent to both the DNS A and DNS AAAA servers. If a response is returned after the timer ends, the system will discard it.

When this command is not configured, the default response timer is 5000 ms.

timeout	This parameter specifies the response timer, in milliseconds. Its value must be an integer ranging from 100 to 10,000.
---------	--

The default value is 5000.

show ipv6 tune dnsnat

This command is used to display the configurations of the “**ipv6 tune dnsnat priority time**” and “**ipv6 tune dnsnat response timeout**” commands.

clear ipv6 tune dnsnat

This command is used to restore the configurations of the “**ipv6 tune dnsnat priority time**” and “**ipv6 tune dnsnat response timeout**” commands to default.

Chapter 23 ePolicy

This chapter covers the commands for configuring the ePolicy function.

epolicy import script <url> <script_name>

This command is used to import an event handler script.

url	This parameter specifies a URL, through which the event handler script can be imported. Currently, HTTP and FTP URLs are supported.
	Note: The “path” of the FTP URL (ftp://user:password@host:port/path) should be a relative path.
script_name	This parameter specifies the name of the event handler script saved on the local device. The filename extension of the script must be .tcl.

no epolicy import script <script_name>

This command is used to delete one specified event handler script from the disk.

script_name	This parameter specifies the name of an event handler script.
-------------	---

show epolicy import

This command is used to display all the imported event handler scripts from the disk.

clear epolicy import

This command is used to remove all the imported event handler scripts from the disk.



Note: The TCL scripts in the disk still exists after the power is off.

epolicy attach script <virtual_service> <script_name>

This command is used to associate a virtual service with an event handler script. One virtual service can be associated with a maximum of 10 event handler scripts, whereas one event handler script can be associated with multiple virtual services.

virtual_service	This parameter specifies the name of a virtual service. Its value must be an HTTP, HTTPS, TCP, TCPS, or UDP virtual service.
-----------------	--

script_name	This parameter specifies the name of an event handler script.
-------------	---

no epolicy attach script <virtual_service> <script_name>

This command is used to disassociate a virtual service from an event handler script.

virtual_service	This parameter specifies the name of a virtual service.
-----------------	---

script_name	This parameter specifies the name of an event handler script.
-------------	---

show epolicy attach [virtual_service]

This command is used to display the association of one specified or all virtual services to the event handler scripts.

virtual_service	This parameter specifies the name of a virtual service. The default value is null, that is displaying the association of all the virtual services to the event handler scripts.
-----------------	---

clear epolicy attach [virtual_service]

This command is used to remove the association of one specified or all virtual services to the event handler scripts.

virtual_service	This parameter specifies the name of a virtual service. The default value is null, that is removing the association of all virtual services to the event handler scripts.
-----------------	---



Note: The configurations not saved in memory will be lost after the power is off.

show epolicy persistence session <session_table_name>

This command is used to display a specified persistence session table created for the ePolicy module.

session_table_name	This parameter specifies the name of the persistence session table. Its value must be the name of the persistence session table specified in an ePolicy script.
--------------------	---

Chapter 24 Logging

This chapter covers the commands for configuring the Logging function.

Basic Settings

log on

This command is used to enable the system logging function. It defaults to off.

log off

This command is used to disable the system logging function.

log level <level>

This command is used to set the system log level. Once the system level is set, log messages below that level will be ignored, that is, not recorded in the system. If this command is not configured, the default system log level is info.

level	This parameter specifies the level of system logs. The values are emerg, alert, crit, err, warning, notice, info, or debug in descending order.
-------	---

For example, to let the log system record all information of the debug level and the levels higher than debug, configure the following command:

```
AN(config)#log level debug
```



Note:

- The log level of NAT logs is info. To let the log system to record NAT logs, you need to set the log level to info or debug.
- The local log level (set through the “**log localhost on**” command) has a higher priority than the system log level.

log facility <facility>

This command is used to set the desired log facility to use. The supported facilities are LOCAL0 to LOCAL7. By default, the facility is set to LOCAL0.

Example:

```
AN(config)#log facility LOCAL1
```

log language {chinese|english}

This command is used to set the language of system logs. Execution of “**log language chinese**” will set the log language to Chinese. Execution of “**log language english**” will set the log language to English. By default, system logs are displayed in English.



Note: Some functions do not support Chinese logs, including “**log nat custom**”, “**log http custom**”, “**log appvisual**”, “**log http {squid|common|combined|welf}**”, epolicy, Diameter, WAF, etc.

log source port <source_port>

This command is used to set the source port from which all log messages should be sent by the APV appliance. The value of the source port ranges from 0 to 65535, among which 0 means the source port will be randomly assigned. By default, its value is 514, which is the syslog port.

Example:

```
AN(config)#log source port 555
```

log timestamp {on|off}

This command is used to set whether to add timestamp the system logs.

log option logid {on|off}

This command is used to enable and disable the function that the log ID is added to log messages including APV appliance log buffer and log messages that will be sent to log hosts. By default, the function is disabled.

log option levelinfo {on|off}

This command is used to enable and disable the function that the level information is added to the logs that will be sent to log hosts. By default, the function is disabled.

show log buff {forward|backward} [match_str]

This command is used to display the latest 2000 logged messages stored in the buffer in the order of either first received (forward) or last received (backward). Please note that the CLI log level is debug.

match_str Optional. You can search for particular logged messages by specifying the “match_str” parameter.

Example:

```
AN(config)#show log buff backward
```

clear log buff

This command is used to clear the log buffer.

log test

This command is used to generate a test log message at the level “emerg”.

show log cli [*line_number*] [*filter_string*]

This command is used to display CLI logs.

line_number	Optional. This parameter specifies the number of lines of CLI logs to be displayed. Its value must be an integer ranging from 0 to 5000. The default value is 200, indicating that the latest 200 lines of CLI logs generated by the system will be displayed.
filter_string	Optional. This parameter specifies a filter string used to search for the CLI logs. Its value must be a string of 1 to 200 characters enclosed in double quotes.

show log restapi cli [*line_number*] [*filter_string*]

This command is used to display RESTful API CLI logs.

line_number	Optional. This parameter specifies the number of lines of CLI logs to be displayed. Its value must be an integer ranging from 0 to 5000. The default value is 200, indicating that the latest 200 lines of CLI logs generated by the system will be displayed.
filter_string	Optional. This parameter specifies a filter string used to search for the CLI logs. Its value must be a string of 1 to 200 characters enclosed in double quotes. The default value is empty, indicating that the system will display log information related to all RESTful API commands.

RFC 5424 Syslog

log rfc5424 on

This command is used to enable the RFC 5424 syslog function. When this function is enabled, the system will generate system logs in the standard format defined by RFC 5424. The standard format is **<PRI>VER TIMESTAMP HOSTNAME APPNAME PROCID MSGID STRUCTURED-DATA MSG-CONTENT**. (The PROCID and STRUCTURED-DATA fields are not supported temporarily and are displayed as “-”). For example, **<134>1 2015-06-12T15:50:13Z AN - - 100013004 - CLI: User "array" executed cmd "log rfc5424 on"** is a system log generated in the standard format defined by RFC 5424.

By default, the RFC 5424 syslog function is disabled.



Note: The configuration of “**log rfc5424 on**” takes effect only when the system logging function has been enabled by using the “**log on**” command.

log rfc5424 off

This command is used to disable the RFC 5424 syslog function.

log rfc5424 appname <app_name>

This command is used to specify the value of the APPNAME field in the header of the RFC 5424-compliant system log. If the APPNAME field is not specified, it will be displayed as “-” in the system log.

app_name	Specify the name of the device or application that generates the system logs. The value of this parameter is a string of no more than 48 characters.
----------	--

no log rfc5424 appname

This command is used to remove the settings of the APPNAME field in the header of the RFC 5424-compliant system log.

log rfc5424 msgid <system_log_id> <custom_log_id>

This command is used to specify the value of the MSGID field in the header of a specified RFC 5424-compliant system log. If the MSGID field is not specified for a system log, the internal system log ID will be used.

system_log_id	Specify the internal ID of a system log.
custom_log_id	Specify the ID that will be assigned to the MSGID field in the header of the RFC 5424-compliant system log. The value of this parameter is a string of no more than 32 characters.

no log rfc5424 msgid <system_log_id>

This command is used to remove the setting of the MSGID field in the header of a specified RFC 5424-compliant system log. The MSGID field in the log will use the internal system log ID. This setting takes effect only for the log generated after this command is executed.

system_log_id	Specify the internal ID of a system log.
---------------	--

clear log rfc5424 msgid

This command is used to clear the settings of the MSGID field in the headers of all RFC 5424-compliant system logs. The MSGID field in all logs will use the internal

system log IDs. This setting takes effect for only the logs generated after this command is executed.

Log Customization

log http {squid|common|combined|welf} [vip|novip] [host|nohost]

This command is used enable the HTTP access logging function and set the format of logging HTTP access information.

By default, this function is enabled, and the format of HTTP access logs is squid.

squid|common
|combined|welf

This parameter specifies the format in which HTTP access information is to be logged. It can be set to one of the standard formats: squid, welf, common or combined. By default, the format squid is used. To set a custom format, the “**log http custom**” command should be used.

vip|novip

This parameter specifies whether to record the virtual service IP that receives requests. Its value must be:

- vip: records the virtual service IP that receives the requests.
- novip: does not record the virtual service IP that receives the requests.

The default value is “novip”.

host|nohost

This parameter specifies whether to record the name of the request’s destination host name. Its value must be:

- host: records the destination host name.
- nohost: does not record the destination host name.

The default value is “nohost”.



Note: To record HTTP access logs with ID 100002221, the administrator must configure the logging format as welf.

To start logging the information of HTTP access, the format of log messages is set to squid, not recording virtual service IP and destination host information.

Example:

```
AN(config)#log http squid novip nohost
```

log http custom <format>

This command is used to customize the format of the log messages recording HTTP access information.

format This parameter specifies the format of the log message recording HTTP access information. Its value must be enclosed in double quotes.

You can create a customized format using the symbols listed in the following table. Any character in the format string that is not part of the symbols listed below is copied as is to the log message. Therefore, additional text can be included in the format string as required.

Symbol	Meaning
%a	Cache result
%b	Bytes returned by proxy to client
%c	Client IP address
%d	Date stamp[MM/DD/YYYY]
%e	HTTP MIME type information
%f	“PROXY LOG” tag, which can be used to distinguish with other logs
%g	Time stamp[HH:MM:SS]
%h	Host name as pulled from the Host header of a request
%i	User agent
%m	HTTP method
%n	Full date/time stamp[MM/DD/YYYY:HH:MM:SS +/-0000]
%k	Session cookie
%o	Virtual service port
%p	Virtual service IP address
%q	A single double quote
%r	HTTP return status code
%s	Real service IP address
%t	Unix time stamp
%u	Request URI Note: The system supports displaying URIs of at most 800 characters. The URI with a length of over 800 characters is displayed as “-”.
%v	Protocol version
%w	Referring page as pulled from the Referer header of a request
%E	Error log, including “error_code” and “error_string”. “error_code” is a brief description about the error and “error_string” shows where the error occurs.
%F	IP address used to connect the real service
%U	Full URL of a request Note: The system supports displaying the path of at most 800 characters. The path with a length of over 800 characters is displayed as “-”.
%R	Interval between the time receiving the request and the time returning the response. The unit of time is millisecond.
%T	Time format compatible with W3C (GMT)
%O	Port used to connect the real service
%N	Full date/time stamp [DD/MMM/YYYY:HH:MM:SS +/-0000]
%D	SSL session ID
%P	Real service port number
%H	Client source port number
%B	Time when the request is forwarded to the real service
%S	Elapsed time between a request is forwarded and its response is received. The unit of time is millisecond.

Symbol	Meaning
%C	Client connection RTT on an average
%I	Time used by the system to process a request on an average. The unit of time is millisecond.
%G	Time used by the system to process a response on an average. The unit of time is millisecond.
%J	Name of real service group
%K	Type of the HTTP connection, long connection or short connection (Keep-Alive)
%L	Hostname of the APV
%M	Name of the IP pool
%Q	Time used by the system to process a request. The unit of time is millisecond.
%W	Time used by the system to process a response. The unit of time is millisecond.
%X	Extended header X-Forwarded-For
%Z	Name of the virtual service

To start logging HTTP access information, administrators need to set the log format. The following command sets the format to “%c%d%g%k”, which lets the log system record the information of client IP address, date stamp, time stamp, session cookies and so on.

```
AN(config)#log http custom “%c %d %g %k”
```

no log http

This command is used to disable the HTTP access logging function.

log nat custom <format_string>

This command is used to enable the system to generate SLB address translation logs. It is for L4 SLB or NAT. In the SLB reverse mode, when receiving a request from the client, the APV translates the source and destination IP addresses to the VIP and a real service’s IP address. Once the connections between the client and the APV and between the APV and the real service are successfully established, the system will generate an SLB address translation log. By default, the system does not generate the SLB address translation log.

format_string Specify the content format of the generated SLB address translation logs. The value of this parameter is a string of no more than 64 characters. The configured value must be all enclosed in double quotes.

All characters in the log are displayed as their original forms except the following ones:

- %t: Display the layer 4 protocol type.
- %s: Display the source IP address before translation.
- %p: Display the source port before translation.
- %d: Display the destination IP address before translation.
- %o: Display the destination port before translation.

- %S: Display the source IP address after translation.
- %P: Display the source port after translation.
- %D: Display the destination IP address after translation.
- %O: Display the destination port after translation.
- %a: Display “SETUP”, indicating the connections have been established between the client’s IP address and the VIP and between the VIP and the real service’s IP address.
- %%: Display the character “%”.

For example:

```
AN(config)#log nat custom "%t %s %p %d %o %S %P %D %O %a"
```

no log nat custom

This command is used to disallow the system to generate SLB address translation logs.

Disabling Individual System Log

log message disable <log_id>

This command is used to disable a specified system log. The disabled system log will be added to the disabled system log list. At most 32 system logs can be disabled.

The default configuration of this command is as follows:

```
log message disable 100017001
```

```
log message disable 100017002
```

```
log message disable 100017003
```

```
log message disable 100017004
```

```
log message disable 100017005
```

```
log message disable 100017006
```

```
log message disable 100024013
```

```
log message disable 100024014
```

log_id This parameter specifies the system log ID.

Administrators can view system log IDs by clicking the  icon on the WebUI tool bar and then choosing **Log List**

Document from the drop-down list.

show log message disable [/log_id]

This command is used to display a specified system log in the disabled system log list. If the parameter “log_id” is not specified, all the system logs in the disabled system log list will be displayed.

no log message disable <log_id>

This command is used to delete a specified system log from the disabled system log list, that is to say, to enable the system log.

clear log message disable

This command is used to delete all the system logs from the disabled system log list, that is to say, to enable all the disabled system logs.

Local and Remote Log Host

log localhost on [/log_level]

This command is used to enable the local log host in the system and set the level of local logs. When the local log host is enabled and the local log level is set, log messages below that level will be ignored, that is, not recorded in the system. By default, the local log host is disabled.

After being enabled, the local log host starts to receive the system logs sent by the system on the IP address 127.0.0.1 and the UDP port 515. The local log host can save a maximum of 50,000 system logs for every log level. The system logs stored on the local host can be viewed and exported via the WebUI.

The local log level has a higher priority than the system log level (set by the “**log level**” command). When setting the local log level, administrators need to make it equal to or greater than the system log level.

The command is also used to modify the defined local log level.

log_level Optionally, this parameter specifies the log level. The values are emerg, alert, crit, err, warning, notice, info, or debug in descending order.

The default value is info.

log localhost off

This command is used to disable the local log host in the system.

log host <host_ip> [port] [protocol] [host_id] [delimiter]

This command is used to configure the remote log host, to which the log messages will be sent.

host_ip	This parameter specifies the IP address of the log host. Its value must be an IPv4 or IPv6 address.
port	Optional. This parameter specifies the service port of the log host. Its value must be an integer ranging from 0 to 65535. The default value is 514.
protocol	Optional. This parameter specifies the protocol used to send the log messages. Its value must be “udp” or “tcp”. The default value is “udp”.
host_id	Optional. This parameter specifies the ID of the log host. Its value must be an integer ranging from 1 to 15. The default value is 1.
delimiter	Optional. This parameter specifies the delimiter of the log messages. It is available only when the “protocol” parameter is set to “tcp”. Its value must be: • 0: Uses “NUL” as the delimiter. • 1: Uses “LF” as the delimiter. • 2: Uses “CR” as the delimiter. • 3: Uses “CR” and “LF” as the delimiter. The default value is 0.



Note:

- The configuration of a remote log host cannot be “**log host** 127.0.0.1 515 udp”, which is reserved for the local log host.
- Make sure the remote log host is prepared to receive log messages. You can configure the log host by configuring syslogd in the server system.

no log host <host_ip> <port> [protocol]

This command is used to remove the remote log host, where the log messages will be sent.

host_ip	This parameter specifies the IP address of the log host.
port	This parameter specifies the service port of the log host.
protocol	Optional. This parameter specifies the protocol used to

send the log messages.

The default value is “udp”.

clear log host

This command is used to clear all log hosts.

log attach <virtual_service|vlink_name> <host_id>

This command is used to associate the specified virtual service or vlink with the specified log host. This log host will receive only logs related to the specified virtual service or vlink.

With no log filter configured, a log host that is not associated with any virtual service or vlink will receive all system logs. With a log filter configured, it will receive only the filtered logs.

Each log host can be associated with up to 128 virtual services.

virtual_service|vlink_name This parameter specifies the name of an existing virtual service or vlink.

host_id This parameter specifies the ID of an existing log host.

no log attach <virtual_service|vlink_name> <host_id>

This command is used to cancel the association between the specified virtual service or vlink with the specified log host.

clear log attach <host_id>

This command is used to cancel all virtual services’ and vlinks’ association with the specified log host.

host_id This parameter specifies the ID of an existing log host. When the value 0 is set, all virtual services’ and vlinks’ association with all log hosts will be cancelled.

log filter <host_id> <filter_id> <filter_string> [module_name] [method]

This command is used to configure a log filter for a specified log host. A maximum of 20 log filters can be configured for a log host. The relationship among three filters is “OR”.

host_id This parameter specifies the ID of the log host. The ID must have been configured by the command “**log host** <host_ip> [port] [udp|tcp] [host_id]”.

filter_id	This parameter specifies the ID of the log filter. Its value must be an integer ranging from 1 to 20.
filter_string	This parameter specifies the log filter string. Its value must be a string of 1 to 40 characters. It is case insensitive and cannot be empty. When the value is set to “*”, all logs will be matched.
module_name	Optional. This parameter specifies the module name used together with “filter_string” to filter syslogs. Its value must be: • Fastlog • WebWall • uProxy • Cache • Proxy • HTTP Access Log • Filter • LLB • Conn API • Feature Control • Clustering • Health Check • SNMP • Debug • XMLRPC • Sysmon • CLI • WebUI • SLB • Compression • Port Forwarding • SSL • LDAP • IP Redundancy • SDNS • ClickTCP

- Hardware
- Diskfree
- Authentication
- Snmpinfod
- SIP
- RTSP
- Triangle
- Dynamic Route
- Radius
- Ulandmon
- IPv6
- IPregion
- PPTP
- ePolicy
- HA
- all

Multiple modules are supported in such a form as “SSL:SLB:LLB”. The default value is “all”, indicating all modules of the system.

method

Optional. This parameter specifies what types of logs will be sent.

- 0: indicates that the logs matching the log filter string but not belonging to the specified module(s) will be sent to the log host.
- 1: indicates that the logs matching the log filter string and belonging to the specified module(s) will be sent to the log host.

The default value is 1.



Note: Configuration of Chinese log filters is supported only on the WebUI.

no log filter <host_id> [filter_id]

This command is used to delete the specified filter or all filters configured for a specified log host.

host_id

Specify the ID of the log host. The ID must have been configured by the command “**log host** <host_ip> [port]

[udp|tcp] [host_id]".

filter_id Optional. This parameter specifies the ID of the filter to be deleted. Its value must be an integer ranging from 0 to 20. When the value is 0, all the filters for the specified host will be deleted.

The default value is 0.

Log Alert

log alert *<id> <expression> <email> <interval> [data|count]*

This command is used to configure the rule of sending the log alert messages for the APV appliance.

Once a log matching the “expression” setting is generated, the APV appliance will send a log alert message to the specified email address according to the pre-defined log ID, interval, and log alert message type.

id	Identify the log alert rule. The ID value ranges from 1 to 32.
expression	Specify a string to match the log messages. Simple regular expression is supported here, which must be enclosed in double quotes. Wildcards supported by this parameter include: • <code>^</code>: Matches the beginning of the string. • <code>\$</code>: Matches the end of the string. • <code>*</code>: Matches any characters, but multiple consecutive asterisks are not allowed.
email	Specify an email address for receiving the log alert messages. The email address must be enclosed in double quotes.
interval	Specify an interval to send the log alert messages. The interval ranges from 0 to 10000, in minutes, where 0 indicates sending the log message instantly whenever it is generated. If log messages are matched consecutively, the administrator can adjust the “interval” setting to reduce the frequency of sending emails.
data count	Specify the type of the log alert messages. It can be of data or count, where “data” indicates sending the whole matched log message and “count” indicates sending the count number of the matched log messages. By default, the

“data” type is used.

Example:

```
AN(config)#log alert 1 "sdns" "xyz@mailserver.com" 1 "data"
```

With the above command configured, if a log contains the string “sdns”, the APV appliance will send a log alert message with ID 1 every 1 minute to the email address xxx@mailserver.com. The following is an example of the log alert email:

```
From: AN<alert@log.domain>
To: Alert Log System Operator(s)<xyz@mailserver.com>
Subject: Log Alert ID: 1 - sdns
MIME-Version: 1.0
<134>Apr 25 21:05:01 CLI: cmd "log alert 2 "sdns" "xyz@mailserver.com"
0 "data""
<134>Apr 25 21:05:11 CLI: cmd "sdns on"
<134>Apr 25 21:05:12 CLI: cmd "sdns off"
```

The configured rules can be shown by “**show log config**”.

To make sure the specified email address can be resolved, please configure a proper DNS server by using the command “**ip nameserver**” on the APV appliance.

no log alert <id>

This command is used to delete a log alert specified by the “id” parameter.

show log alert

This command is used to show all log alerts configurations.

clear log alert

This command is used to clear all log alerts configurations.

show log config

This command is used to display the current logging configuration.

clear log config

This command is used to return the logging configuration to default settings.

SNMP Trap for System Logs

log snmp trap <log_id> <trap_oid>

This command is used to set an SNMP trap OID for the specified system log.

After a system log is bound with an SNMP trap OID, an SNMP trap message will be triggered whenever the system generates this log.

log_id	This parameter specifies the log message ID.
trap_oid	This parameter specifies the SNMP trap OID. Its value must be an integer ranging from 2 to 65535.

no log snmp trap <log_id>

This command is used to delete the SNMP trap OID setting for the specified system log.

show log snmp trap [/log_id]

This command is used to display the SNMP trap OID setting for the specified system log. If the “log_id” parameter is not specified, all SNMP trap OID settings for system logs will be displayed.

clear log snmp trap [/log_id]

This command is used to delete the SNMP trap OID setting for the specified system log. If the “log_id” parameter is not specified, all SNMP trap OID settings for system logs will be cleared.

Application Visualization

log appvisual {on|off} [/log_module]

This command is used to enable or disable the visualization logging function for the specified applicaiton module.

After application visualization is enabled, the system will collect this module’s logs that the ELK server needs to use.

By default, the function is disabled for all application modules.

log_module	Optional. This parameter specifies the application module name. Its value must be: • ssl: enables or disables visualization logging for the SSL application module. • sdns: enables or disables visualization logging for the SDNS application module. • slb-http: enables or disables visualization logging for the SLB HTTP application module. • slb-tcp: enables or disables visualization logging for the SLB TCP application module. Please note that after enabling visualization logging for the SLB TCP application module, if the health checks of TCP type is configured, the system prints logs every second, which
------------	---

may cause a surge in log data.

- **slb-ftp**: enables or disables visualization logging for the SLB FTP application module.
- **slb-rtsp**: enables and disables visualization logging for the SLB RTSP application module.
- **slb-sip**: enables and disables visualization logging for the SLB SIP application module.
- **llb**: enables or disables visualization logging for the LLB application module.
- **all**: enables or disables visualization logging for all application modules.

The default value is “all”.

show log appvisualize [log_module]

This command is used to display the enabling/disabling status of the visualization logging function for the specified application module. If the “log_module” parameter is not specified, the enabling/disabling status of the visualization logging function for all application modules will be displayed.

Prometheus

Prometheus is a third-party monitoring service that tracks APV’s performance by collecting data from various components and displaying them as time-series graphs, enabling you to monitor system resource utilization.

To monitor and retrieve APV’s data, you need to set up a Prometheus server. For more information about how to set up the server, see **25.4.3 Prometheus** in APV User Guide.

prometheus {on|off}

Enables or disables Prometheus.



Note:

- After you run **prometheus {on|off}**, you need to use **write memory** to save its setting.
- You can run **show** and **token** commands when Prometheus is on.

show prometheus status

Displays whether Prometheus is enabled.

Example:

```
AN(config)#show prometheus status
```

```
Prometheus client status: off
```

prometheus token new

Creates a Prometheus token that grants access to the services on the Prometheus server that need to retrieve data from Prometheus.

no prometheus token <token>

Revokes a token.

token The token to be revoked.

show prometheus token all [display_order]

Displays all tokens.

display_order Optional. Displays the tokens in chronological order.

- forward: Displays the tokens from oldest to newest.
- backward: Displays the tokens from newest to oldest.

The default value is **forward**.

Example:

AN(config)#show prometheus token all						
Id	User	Token	Created At	Expired At	Last Used At	Is Expired
1	array	8793b333-7c7a-41d8-8ef0-0ea3c202d1cd	2025-01-16 06:38:13	2025-01-16 06:43:13	NaN	True
2	array	2367465a-ba66-40ba-b6e6-82df6fe2f245	2025-01-16 06:39:15	2025-01-16 09:19:12	NaN	True
3	array	8ac7bd83-d337-4c69-b85c-9d51f78650fc	2025-01-16 09:19:50	2025-01-16 09:20:07	NaN	True

4	array	960d8c77-59d3-4804-8ee0-16a542259506	2025-01-17 09:22:23	2025-01-17 09:28:23	NaN	False
5	array	0d14be95-0565-4848-b432-50744818009d	2025-01-17 09:22:31	2025-01-17 09:28:31	NaN	False

show prometheus token valid *[display_order]*

Displays all valid tokens.

display_order Optional. Displays valid tokens in chronological order.

- forward: Displays valid tokens from oldest to newest.
- backward: Displays valid tokens from newest to oldest.

The default value is **forward**.

Example:

AN(config)#show prometheus token all						
Id	User	Token	Created At	Expired At	Last Used At	Is Expired
4	array	960d8c77-59d3-4804-8ee0-16a542259506	2025-01-17 09:22:23	2025-01-17 09:28:23	NaN	False
5	array	0d14be95-0565-4848-b432-50744818009d	2025-01-17 09:22:31	2025-01-17 09:28:31	NaN	False

show prometheus token expired *[display_order]*

Displays expired and revoked tokens.

display_order Optional. Displays expired and revoked tokens in chronological order.

- forward: Displays expired and revoked tokens from oldest to newest.
- backward: Displays expired and revoked tokens from newest to oldest.

The default value is **forward**.

Example:

AN(config)#show prometheus token expired						
Id	User	Token	Created At	Expired At	Last Used At	Is Expired
1	array	8793b333-7c7a-41d8-8ef0-0ea3c202d1cd	2025-01-16 06:38:13	2025-01-16 06:43:13	NaN	True
2	array	2367465a-ba66-40ba-b6e6-82df6fe2f245	2025-01-16 06:39:15	2025-01-16 09:19:12	NaN	True
3	array	8ac7bd83-d337-4c69-b85c-9d51f78650fc	2025-01-16 09:19:50	2025-01-16 09:20:07	NaN	True

prometheus token ttl <minutes>

Sets the duration of a Prometheus token.

minutes Time to Live (TTL) of a token in minutes. The range is 3–1440. The default value is 5. All tokens share the same TTL value.

Each time the token is used, its expiration time is updated to the current time plus TTL.

show prometheus token ttl

Displays Time to Live (TTL).

Example:

```
AN(config)#show prometheus token ttl
6 minute(s)
```

prometheus port <port number>

Sets the port number of Prometheus on APV.

port number The port number used by Prometheus on APV. The range is 1025–65000. The default value is 9100.



Note: After you run “**prometheus port** <port number>,” you need to use **write memory** to save its setting.

show prometheus port

Displays the port Prometheus is using on APV.

Example:

```
AN(config)#show prometheus port
Prometheus client port number: 9100
```

prometheus https {on|off}

Enables or disables Prometheus HTTPS.

- On: Prometheus HTTPS.
- Off: Prometheus HTTP.



Note:

- When you enable **prometheus https**, it checks whether the SSL private key and certificate have been loaded. If not, this feature remains disabled.
- After you run **prometheus https {on|off}**, you need to use **write memory** to save its setting.

show prometheus https status

Displays whether Prometheus HTTPS is enabled.

prometheus https ssl <key_url> <crt_url>

Imports the SSL private key and certificate of Prometheus HTTPS.

key_url The private key’s URL. The key needs to be in the PEM format.

crt_url The certificate’s URL. The certificate needs to be in the PEM format.

Example:

```
AN(config)#prometheus https ssl http://192.168.10.138:8000/server.key  
http://192.168.10.138:8000/server.crt
```

```
Downloading: 100.00% (1704/1704 bytes)  
Download completed successfully. http://192.168.10.138:8000/server.key  
Downloading: 100.00% (1383/1383 bytes)  
Download completed successfully. http://192.168.10.138:8000/server.crt  
The private key and certificate match.
```

no prometheus https ssl

Deletes the loaded SSL private key and certificate and runs **prometheus https off**.

show prometheus https ssl

Displays the SSL private key and certificate.

show prometheus log <entries>

Displays the usage status for Prometheus, such as settings, tokens, status, and users.

entries How many log entries to be displayed. The latest entry is at the top.

```
AN(config)#show prometheus log
```

```
2025-01-17 09:50:46,718:INFO: Set status = on  
2025-01-17 09:50:32,435:INFO: Set https_status = on  
2025-01-17 09:46:10,538:INFO: Set ssl_cert_path = /ca/bin/prometheus_client/server.crt  
2025-01-17 09:46:10,537:INFO: Set ssl_key_path = /ca/bin/prometheus_client/server.key  
2025-01-17 09:45:11,353:INFO: Set port = 9101  
2025-01-17 09:22:31,219:INFO: array created a new token, which is  
0d14be95-0565-4848-b432-50744818009d  
2025-01-17 09:22:23,255:INFO: array created a new token, which is  
960d8c77-59d3-4804-8ee0-16a542259506  
2025-01-16 09:58:30,017:INFO: Set token_ttl = 6  
2025-01-16 09:20:07,792:INFO: array deleted a token, which is  
8ac7bd83-d337-4c69-b85c-9d51f78650fc  
2025-01-16 09:19:50,584:INFO: array created a new token, which is  
8ac7bd83-d337-4c69-b85c-9d51f78650fc  
2025-01-16 09:19:12,276:INFO: array deleted a token, which is  
2367465a-ba66-40ba-b6e6-82df6fe2f245  
2025-01-16 06:39:27,271:INFO: Set status = off  
2025-01-16 06:39:16,098:INFO: array created a new token, which is  
2367465a-ba66-40ba-b6e6-82df6fe2f245  
2025-01-16 06:38:13,647:INFO: array created a new token, which is  
8793b333-7c7a-41d8-8ef0-0ea3c202d1cd  
2025-01-16 06:36:48,175:INFO: Set status = on  
2025-01-15 09:56:42,369:INFO: Set status = off  
2025-01-06 06:48:44,422:INFO: Set status = off  
2025-01-06 14:48:16,681:INFO: Add a user, array.
```

show prometheus metrics

Displays the metrics Prometheus supports. Currently it supports the following metrics:

- **cpu_usage**: Measures the CPU utilization across the system.
- **memory_status**: Tracks used, free, and total memory.
- **disk_status**: Monitors used, free, and total disk space.
- **network_throughput**: Measures incoming and outgoing network traffic across system interfaces.
- **start_time_seconds**: Records how long the system has been started in seconds.

Example:

```
AN(config)#show prometheus metrics
```

cpu_usage: Current CPU load(%)

memory_status: Represents the current memory usage status of the system, including total, used, available, and usage percentage. Unit in MB

disk_status: Monitor the disk usage, including total, used, free, and usage percent. Unit in GB

network_throughput: Current network throughput (Mb/s) for all network interfaces.

start_time_seconds: Start time of the APV since unix epoch in seconds.

Chapter 25 Administrative Tools

This chapter describes the commands for Administrative Tools.

Configuration Management Commands

admin aaa {on|off} [priority]

This command is used to enable or disable the external authentication function. This function allows authentication of an administrator account's login via a RADIUS, TACACS+, LDAP, and LDAPS server. By default, this function is disabled.

priority Optional. This parameter specifies the order of priority for using the local database and external authentication server when the external authentication function is enabled. Its value must be:

- 0: indicates that the local database will be used preferentially. If the username does not exist in the local database, the external authentication server will be used.
- 1: indicates that the external authentication server will be used preferentially. If authentication fails because of such reasons as the username does not exist, the password is incorrect, or the server is unreachable, the local database will be used.

The default value is 0.

admin aaa authorize {on|off}

This command is used to enable or disable the external authorization function. This function allows level-based authorization of administrator accounts via a RADIUS, TACACS+, LDAP, or LDAPS server. By default, this function is disabled.

- In RADIUS authorization, only administrator accounts whose "Service-Type" field is "Administrative" are authorized to access the Config mode. Other administrator accounts are authorized to access only the Enable mode.
- In TACACS+ authorization, only administrator accounts whose "priv-lvl" field is 15 are authorized to access the Config mode. Other administrator accounts are authorized to access only the Enable mode.
- In LDAP and LDAPS authorization, only the administrator accounts that have been added to an authorized group and assigned a specific distinguished name (DN) are allowed to access the Config mode. Accounts that are not added to this group and not assigned the DN are authorized to access only the Enable mode.

When external authorization is disabled, an administrator account will be authorized to access the Config mode as long as it passes the authentication.

LDAP(S) Example: Control Access to an Administrative Application**1. Create security groups**

Admin Group: CN=AdministratorGroup,OU=Groups,DC=example,DC=com

User Group: CN=UserGroup,OU=Groups,DC=example,DC=com

2. Add users to groups

- Administrator: An administrator, jdoe, is in AdministratorGroup:
 - Username: jdoe
 - DN: CN=John Doe,OU=Users,DC=example,DC=com
 - Add jdoe to the AdministratorGroup.
- Regular User: A regular user, asmith, is in UserGroup:
 - Username: asmith
 - DN: CN=Alice Smith,OU=Users,DC=example,DC=com
 - Add asmith to the UserGroup.
 - Do not add asmith to the AdministratorGroup.

3. Authorization checks

When a user attempts to access the application, the system checks the following:

a. jdoe:

- i. Check memberOf: The system confirms that jdoe is a member of the AdministratorGroup.
- ii. Check DN: The system verifies that jdoe's DN is valid.

Since both checks are successful, jdoe is granted access to the Config mode.

b. asmith:

- i. Check memberOf: The system finds that asmith is not a member of the AdministratorGroup.
- ii. Check DN: Check whether asmith's DN is valid or not.

Since asmith is not a member of AdministratorGroup, or if she doesn't have any valid DN, she won't gain access to the Config mode. She can gain read-only access.

admin aaa method *[method]*

This command is used to configure the external authentication and authorization method. Administrators can deploy a RADIUS, TACACS+, LDAP, or LDAPS server for external authentication and authorization. The default method is RADIUS.

method	This parameter specifies the external authentication and authorization method. Its value must be: • radius: Uses the RADIUS server for external authentication and authorization. • tac_x: Uses the TACACS+ server for external authentication and authorization. • ldap: Uses an LDAP server for external authentication and authorization. • ldaps: Uses an LDAPS server for external authentication and authorization. The default value is “radius.”
--------	--

admin aaa server <server_id> <host_name|ip_address> <port> [secret]

This command is used to configure the external authentication and authorization server. The system supports the deployment of four authentication and authorization servers with ID es01, es02, es03, and es04. If authentication or authorization via the server es01 fails, the system will try on the server es02.

The parameters of this command vary between server IDs. The following syntax shows their differences:

- RADIUS (es01) and TACACS (es02): admin aaa server <server_id> <host_name|ip_address> <port> [secret]
- LDAP (es03): admin aaa server <server_id> <host_name|ip_address> <port> [dn] [memberOf]
- LDAPS (es04): admin aaa server <server_id> settings <host_name|ip_address> <port> [dn] [memberOf] [certificate] [verification]

The syntax of es04 is used in the following ways:

- If you want to configure an LDAPS server, the syntax will be as follows:

admin aaa server <server_id> settings <host_name|ip_address> <port> [dn] [memberOf]
- If you want to add certificates, the syntax will be as follows:

admin aaa server <server_id> <certificate> [url]
- If you want to enable certificate verification, the syntax will be as follows:

admin aaa server <server_id> verifycert <verification>

server_id	This parameter specifies the ID of an authentication and authorization server. Values: • es01: An external RADIUS authentication server. • es02: An external TACACS authentication server. • es03: An external LDAP authentication server. • es04: An external LDAPS authentication server.
host_name ip_address	This parameter specifies the host name or IP address of the authentication and authorization server. If an IP address is specified, it must be enclosed in double quotes. Both IPv4 and IPv6 addresses are supported.
port	This parameter specifies the port number of the authentication and authorization server. Its value must be an integer ranging from 0 to 65,535.
dn	This parameter is used when server_id is es03 or es04. It is a distinguished name (DN). A DN uniquely identifies an entry in a directory. DNs must be enclosed in double quotes. Example: “cn=Jane,ou=Dev,dc=ArrayLab,dc=COM”“cn=Jane,ou=Dev,dc=ArrayLab,dc=COM”
memberOf	This parameter is used when server_id is es03 or es04. It is an attribute that lists a group a user belongs to. It must be enclosed in double quotes. Example: “cn=Engineer,ou=Dev,dc=ArrayLab,dc=COM”
settings	This parameter is used only when server_id is es04. It configures the LDAPS server’s settings.
certificate	This parameter is used only when server_id is es04. Values: • clientcert: Imports a PEM client certificate. You can paste the certificate manually or use url to import it.

- **clientkey**: Imports a PEM client key. You can paste the key manually or use **url** to import it.
- **rootca**: Imports a CA certificate used for client authentication. You can paste the key manually or use **url** to import it.
- **verifycert**: Enables or disables certificate verification.

verification

This parameter is used only when **server_id** is **es04** and certificates need to be verified (**certificate** is **verifycert**). It enables and disables certificate verification during a LDAPS sign-in.

Values:

- 0: Disables certificate verification.
- 1: Enables certificate verification.

If you enable certificate verification, you need to import or paste three certificates: client certificate, client key, and root certificate. Next, sign in the APV system using your LDAPS credential. If one certificate is missing, authentication can be performed but authorization cannot.

The default value is zero (0).

url

Optional. This parameter is used only when **server_id** is **es04** and **certificate** is **clientcert**, **clientkey**, or **rootca**. It is an FTP, TFTP, or HTTP URL used to import a certificate.

secret

Optional. This parameter is used when **server_id** is **es01** or **es02**. It specifies the secret of the authentication and authorization server. When this parameter is not configured, the secret is empty.

Note: The secret is masked as “*****” when it appears in the “**show admin aaa all**” command output and system logs. When displayed in the running configuration, startup configuration and configuration backup files, it is encrypted and followed by an “ENCRYPTED” flag.

Example 01: Configure two authentication and authorization servers with the IDs “es01” and “es02”:

```
AN(config)#admin aaa server es01 “10.1.31.1” 1812 radiusceret
AN(config)#admin aaa server es02 radius_host 1812 radiusceret
```

Example 02: Configure an LDAP server with the ID “es03” using an IP address or host name:

```
AN(config)#admin aaa server es03 "10.1.31.1" 653
"cn=Jane,ou=Dev,dc=ArrayLab,dc=COM" "cn=Engineer,ou=Dev,dc=ArrayLab,dc=COM"
AN(config)#admin aaa server es03 TestLDAP 653
"cn=Jane,ou=Dev,dc=ArrayLab,dc=COM" "cn=Engineer,ou=Dev,dc=ArrayLab,dc=COM"
```

Example 03: Configure an LDAPS server with the ID “es04” using an IP address or host name:

```
AN(config)#admin aaa server es04 settings "10.1.31.1" 284
"cn=Jane,ou=Dev,dc=ArrayLab,dc=COM" "cn=Engineer,ou=Dev,dc=ArrayLab,dc=COM"
AN(config)#admin aaa server es04 settings TestLDAPS 284
"cn=Jane,ou=Dev,dc=ArrayLab,dc=COM" "cn=Engineer,ou=Dev,dc=ArrayLab,dc=COM"
```

Example 04: Add a client certificate with the ID “es04.” You can use the same syntax to add a client key or a root certificate.

```
AN(config)#admin aaa server es04 clientcert
You may overwrite an existing client certificate file.
Enter the certificate file in PEM format,
use "." on a single line, without quotes
to terminate import
-----BEGIN CERTIFICATE-----
MIIF0zCCBLugAwIBAgITEgAAAA/JmDC+ouQUEwAAAAAADzANBgkqhkiG9w0BAQsFA
DBRMRIwEAYKCZImiZPyLGBGRYCSU4xGDAWBgoJkiaJk/IsZAEZFghBUIJBWUxBQjEh
MB8GA1UEAxMYQVJSQVIMQUItQVJSQVktQkxSLUFELUNBMB4XDTI0MDUzMTA4MD
E1MV0xDTI2MDUzMTA4MTE1MVowADCCASIwDQYJKoZIhvcNAQEBBQADggEPADCC
AQoCggEBAL5A/II1DJ5TGvpATHx3tcIv6a0JNcacBa3RdTzzGqicEfDSzuyv1EQ+M2hiypRIHp
pIFyDI7cYuN6qB5HK3rk77zlpq4VaGC4d/OLeiNRv3qHivTGtf7AVY
GVDCL0AGe6ZFI+uli7vLRL81026MSVAzrGYCCm5+12w/Up65uuDPJboGATw9zvzE
ATOX7cJvkm+LE55yO+yNMCmY7N4ag3G3GBxRj41/dgVqi3hsZID0tk1o0Wpw7LFf
pG6zdvLrjsFgq08Yp50WO+43WOH9tjwcarYOTS4fkCdBXm7ZVjezQ59looaLSyJd
tpQukH4jzSLU8Ffm4mRgqT4am6jAEe0CAwEAaOCaVMwggLvMD0GCSsGAQQBgjcV
BwQwMC4GJisGAQQBgjcVCIB6tV+BmoIKhdGFC4Phg0eGhcwYM4X9khiF4o1IAgFk
AgEDMDIGA1UdJQqrMCKGBysGAQUCAwUGCisGAQQBgjcUAglGCCsGAQUFBwMBBg
rBgEFBQcDAjAObgNVHQ8BAf8EBAMCBaAwQAYJKwYBBAGCNxUKBDMwMTAJBgcrB
gEFAgMFMAwGCisGAQQBgjcUAglwCgYIKwYBBQUHAWewCgYIKwYBBQUHAWIwHQY
DVR0OBByEFA51U9e9XRWgP03Nk/EUvVh6S9uNMB8GA1UdIwQYMBaAFGTgtDLTvecet7
eFwa067VYjUjn9MIHbBgNVHR8EgdMwgdAwgc2ggcqqggceGgcRsZGFwOi8vL0NOPUFSUKF
ZTEFCLUFSUKFZLUJMU1BRC1DQsxDtJ1BU1JBWS1CTFItQUQsQ049Q0RQLENOPVB1Y
mxpYyUyMETleSUyMFNlcnZpY2VzLENOPVNlcnZpY2VzLENOPUNvbmZpZ3VyYXRpb24s
REM9QVJSQVIMQUItREM9SU4/Y2VydGlmaWNhdGVsZXZvY2F0aW9uTGltZD9iYXNIP29
iamVjdENsYXNzPWNSTERpc3RyaWJldGlublBvaW50MIHKBggrBgEFBQcBAQSBvTCBujC
BtwYIKwYBBQUHMAKGgapsZGFwOi8vL0NOPUFSUKFZTEFCLUFSUKFZLUJMU1BRC1
DQsxDtJ1BSUEsQ049UHVibGljJTlwS2V5JTlwU2VydmljZXMsQ049U2VydmljZXMsQ049Q2
9uZmlndXJhdGlubixEQz1BU1JBWUxBQixEQz1JTj9jQUJlcnRpZmljYXRlP2Jhc2U/b2JqZW
N0Q2xhc3M9Y2VydGlmaWNhdGlubkF1dGhvcml0eTA9BgNVHREBAf8EMzAxghhBUIJBWS1
CTFItQUQuQVJSQVIMQUItSU6CC0FSUKFZTEFCLklOgghBUIJBWUxBQjANBgkqhkiG9w0
BAQsFAAOCAQEAx1/jvWgy5tnMlrKwkz7seN37eLJip8Jo8aAjWESYAaZjCW+a+/h8KDIDR
H1ooQPdE+FSsg/aKqOp
gh6BAKgSEDMYgr8JuRE9UvH09v05FxSvXlusUEsYBTLJfjDZ92k9ng0MKKKm6rSs
c9oQ9JSWM0UrPT4cKVx/57/EN1Xun7beaDKxT8Wq+uvGfmgXVCBU7f7TMKUsm8RB
PhV265XUtw2jlk0ETINFRr4tOfdmGjYFjd+FITCKjJCd9Ksb9fqmSgEmwUgvAorQ
```

```
BVqDQR2tBpC8TBZUQrrY4b0FQXeI2qchiG08uGVDTTb58KKhVi3MzZw4Lc2ifuvI
r//3Jz1ciA==
-----END CERTIFICATE-----
...
PEM format.
Client certificate import successful
```

Example 05: Enable certificate verification with the ID “es04.”

```
AN(config)#admin aaa server es04 verifycert 1
```

no admin aaa server <server_id> [certificate]

This command is used to delete the configurations of the specified authentication and authorization server. The **certificate** parameter is specific to **es04**.

server_id	The ID of an authentication and authorization server.
	Values:
	• es01: An external RADIUS authentication server. • es02: An external TACACS authentication server. • es03: An external LDAP authentication server. • es04: An external LDAPS authentication server.
certificate	This parameter is used only when server_id is es04 .
	Values:
	• clientcert: Deletes a PEM client certificate. • clientkey: Deletes a PEM client key. • rootca: Deletes a CA certificate used for client authentication. • settings: Deletes the LDAPS server settings. • verifycert: Disables certificate verification.

clear admin aaa all

This command is used to clear the configurations of all authentication and authorization servers.

show admin aaa all

This command is used to display the enabling status of the external authentication and authorization server and the configuration of all authentication and authorization servers.

passwd enable *[passwd_string]*

This command is used to set or change the password for entering the Enable mode of the appliance.

passwd_string	Optional. This parameter specifies the password for entering the Enable mode. The parameter value should be a case-sensitive string of 1 to 8 characters enclosed in double quotes. Numbers, letters, and special characters are supported. If this parameter is not specified, you can enter the Enable mode by pressing Enter at the command prompt.
---------------	---

user *<user_name> <password> [level]*

This command is used to create an administrator account and set its password and privilege level.

For an existing administrator account, to modify its privilege level, you need to delete it and then recreate one.

The system supports creating 100 administrative users at most.

user_name	This parameter specifies the user name of the administrator account. Its value must be a string of 1 to 32 characters. Numbers and letters are supported. Special characters are not supported, but you can use “\$” to end the parameter value.
-----------	--

password	This parameter specifies the password. When the password force mode is disabled, its value must be a case-sensitive string of 1 to 116 characters. If the parameter value contains numbers or special characters, it must be enclosed in double quotes. When the password force mode is enabled, its value must meet the password complexity requirements in the password force mode (see the “passwd forcemode” command).
----------	--

level	Optional. This parameter specifies the privilege level for the administrator account. Its value must be: • enable: administrator accounts with this privilege level are only allowed to execute CLI commands in the
-------	--

“enable” mode.

- config: administrator accounts with this privilege level are allowed to execute all CLI commands for configuring and managing the appliance.
- api: administrator accounts with this privilege level are allowed to access API-based Web service, but are not allowed to execute any CLI commands.

The default value is “config”.

**Note:**

- A maximum of 100 administrator accounts can be configured on the APV appliance. Normally, API is used to control the appliance in a centralized way. Therefore, it is recommended to create only one administrator account with the “api” privilege level although the system supports multiple ones.
- The role-based privilege management function cannot be configured for an administrator account with the “api” privilege level.

no user <user_name>

This command is used to remove the specified administrative user.

If the administrative privilege separation function is enabled, when there is only one administrative user that associates with the administrative role (role_administrator), this administrator cannot be deleted.

show users

This command is used to display the configuration of existing administrative users.

clear users

This command is used to remove all administrative users. After executing this command, the system will create the default account “array” again.

If the administrative privilege separation function is enabled, this command will create the “administrator” user of the administrative privilege separation function.

passwd user <user_name> <password>

This command is used to change the password of an existing administrator account.

user_name This parameter specifies the user name of an existing administrator account.

password This parameter specifies the new password.

When the password force mode is disabled, its value must be a case-sensitive string of 1 to 116 characters. If the

parameter value contains numbers or special characters, it must be enclosed in double quotes.

When the password force mode is enabled, its value must meet the password complexity requirements in the password force mode (see the “**passwd forcemode**” command).

passwd forcemode {on|off}

This command is used to enable or disable the password change force mode. The function is enabled by default. After the password change force mode is enabled, the system will impose the following requirements and restrictions on an administrator account (including the segment administrator account):

1. When a new administrator account is created, its password length must be in the range of 8 to 116 characters, and it must be a combination of the three of the following: uppercase, lowercase, numeric and special character. In addition, the password cannot contain the username, which is case-insensitive.
2. When you change the password of an existing administrator account, the new password must also meet requirements listed in the first item.
3. The password of a new administrator account must be changed when it is used to log in the system for the first time. The passwords of administrator accounts created before the password change force mode is enabled do not need to be changed upon next-time login, including the default administrator account whose password is changed.
4. If the password of the default administrator account is never changed or if a factory restoration of administrator account settings is performed, such as “**clear config factory**”, the password of the default administrator account also needs to be changed upon next-time login.
5. If an administrator account fails to log in 5 consecutive times, it will be locked for 5 minutes.
6. The new password of an administrator account cannot be one of the last five passwords that have been used before.
7. The expiration date of a password is 90 days. An administrator account with an expired password will be asked to change the password upon next-time login.

After the password change force mode is disabled, administrator accounts are not subject to the requirements and restrictions above.

passwd expire {on|off}

This command is used to enable or disable the password expiration function. When this function is enabled, the password will expire after 90 days of validity. When the password expires, the administrator will be forced to change the password the next time he logs in onto the system. By default, this function is enabled.

1. When “**passwd forcemode**” and “**passwd expire**” functions are both enabled, all requirements and restrictions under the “**passwd forcemode on**” command will be valid. When the “**passwd forcemode**” function is enabled and the “**passwd expire**” function is disabled, the requirement of “password will expire every 90 days” will be invalid.
2. When “**passwd forcemode**” function is disabled, all requirements and restrictions under “**passwd forcemode**” command will be invalid.

show passwd forcemode

This command is used to display the configuration of the password change force mode.

show passwd expire

This command is used to display whether the password expiration verification function is enabled.

system serialnumber

This command is used to manually generate a vAPV’s serial number. It is only supported on vAPV.

system license <key> [validate|novalidate]

This command allows users to enter a license key for the APV appliance. Without a valid license key, the ArrayOS will not auto reload configuration and ArrayOS will not run properly.

validate	The default option. After the user enters the license key and executes this command to import the key, the system will first validate the key. If validating succeeded, the system will import the license key and also save it.
novalidate	With this option used, the system will import the license key and save it, without any validation.

system reboot [interactive|noninteractive]

This command is used to reboot the APV appliance. After system rebooting succeeded, the last saved (by using the command “**write memory**”) system configurations will be used by the APV appliance.

interactive noninteractive	This is an optional parameter. If “interactive” is set, the system will prompt the administrator to input “YES” to confirm the system rebooting. If “noninteractive” is set, the system will reboot immediately without giving any prompt. The default value is “interactive”.
----------------------------	--

system restart <service_name>

This command is used to restart the specified service.

process_name	This parameter specifies the name of the service to be restarted. Its value must be “system_service”, “hc_service”, “config_service”, “ha_service” or “lb_hc_service”.
--------------	--



Note: When executing the command “system restart system_service” four times within 30 minutes, the appliance will reboot.

system shutdown [halt|poweroff]

This command is used to halt all functions of the APV appliance and disable the CLI. After executing this command, the system will ask the administrator to input “YES” to confirm. And then the system will shut down in 60 seconds.

halt poweroff	This optional parameter is used to specify the system shut down mode. And the default value is “poweroff”.
---------------	--

If the “halt” parameter is used, the system will shut down in 60 seconds. After the system is shut down, the system will boot itself automatically by re-plug in the power cord.

If the “poweroff” is used, the system will shut down in 60 seconds. And the system will boot by pressing the power button.

system dump {on|off}

This command is used to enable/disable the system dump function. It is enabled by default. With the function enabled, when system panic occurs, the system running information will be stored in the vmcore file for future reference or troubleshooting by administrators.

show system dump

This command is used to display the status of system dump function.

system console reset

This command is used to reset the system console. After executing this command, “Reset console” will prompt in the screen.

config terminal [force]

This command allows users to gain access to the commands required to configure the APV appliance. Deploying the optional parameter should force any existing Config sessions to end.

**Note:**

- Sometimes the administrator can only enter the Config mode with this CLI but cannot execute Config-mode CLIs if other existing Config sessions are not idle.
- When the multiconfig function is enabled by executing the command “**config multiconfig enable**”, the command “**config terminal force**” will no longer take effect.

config multiconfig enable

This command is used to enable the multiconfig function. When the multiconfig function is enabled, the system allows multiple administrator accounts to be simultaneously in the Config mode or one administrator accounts to simultaneously establish multiple SSH connections of the Config level authority. This function is disabled by default.

config multiconfig disable

This command is used to disable the multiconfig function. When the multiconfig function is disabled, only one administrator account or login can be in the Config mode at a time. If multiple administrator accounts are in the Config mode or multiple SSH connections of the Config level authority are established by one administrator account when the multiconfig function is disabled, only the one that executes the command will remain in the Config mode while others will be forced out of the Config mode.

show config multiconfig

This command is used to display the status of the multiconfig function.

config timeout <timeout>

This command allows users to set the timeout value of the Config mode session.

When the session inactivity duration exceeds the timeout value, the current Config session will exit to the “Enable” mode if a new session is trying to enter the Config mode. Otherwise, the current Config session will stay active.

timeout	The timeout value is measured in seconds, ranging from 30 to 36000 (10 hours). The default setting is 180 seconds (3 minutes).
---------	--

show config timeout

This command allows users to view the configured timeout setting.

clear config timeout

This command allows users to clear the configured timeout setting, thus returning it to the default setting of 180 seconds (3 minutes).

write file <file_name>

This command is used to save the current running configurations to a backup file on the local storage.

file_name	This parameter specifies the name of the configuration backup file. Its value must be a string of 1 to 182 characters. Lower-case and upper-case letters, numbers, and special characters including “@”, “-”, “_” and “!” are supported. When the parameter value contains special characters or only numbers, it must be enclosed in double quotes.
-----------	--



Note: The size of the backup file is smaller than 100 MB.

write all file <file_name> <password>

This command is used to back up all system, running and segment configurations on the APV appliance to the specified TGZ file.

file_name	This parameter specifies the name of the TGZ configuration backup file. The specified file name should not contain the extension name “.tgz”. Its value must be a string of 1 to 182 characters. Lower-case and upper-case letters, numbers, and special characters including “@”, “-”, “_” and “!” are supported, but its value cannot start with “-”. When the parameter value contains special characters or only numbers, it must be enclosed in double quotes.
-----------	---

password	This parameter specifies the password for accessing the TGZ configuration backup file. Its value must be a string of 1 to 8 characters, which supports letters, numbers and the following special characters:
----------	---

“~”, “#”, “%”, “^”, “&”, “*”, “(”, “)”, “-”, “_”, “+”, “=”, “ ”, “{”, “}”, “[”, “]”, “””, “.”, “,”, “\”, “|”, “<”, “>”, “ ”, “.”, “?”, “/” and “\$”.

When the parameter value contains special characters, it must be enclosed in double quotes.

write memory

This command allows users to save the current configuration to the file and assigns it to the boot configuration data.

show memory

This command allows users to display the memory critical information relating to the APV appliance.

Example:

The following lines describe system connection resource usage:					
ITEM	SIZE	LIMIT	USED	FREE	REQUESTS
TCP small pcb:	64,	20000,	426,	19574,	4490795
TCP pcb:	288,	20000,	1,	19999,	5219107

Each connection owns a “pcb” data structure. There are two kinds of “pcb” data structure; “small pcb” where size is 64 bytes is for TCP connections in “TIME_WAIT” state. “pcb” for all the other TCP connections has bigger size: 288 bytes. The “LIMIT” column tells the total number of data structure items. “USED” refers the number of items in use. The “Free” indicates left items that may be used. The “REQUEST” is the accumulation of total usages and is always incremented.

TCP connection is valuable system resource. When it is used up, new customer requests can’t be served. The number of total TCP connections is decided by system memory size as follows:

- 4GB: 2M (2064352) connections
- 1GB: 512K (516088) connections
- 512MB: 40000 connections
- 256MB: 20000 connections

config file <file_name>

This command allows users to change the config from terminal, or restore a config from local storage by simply supplying the saved file’s name.

file_name	This parameter specifies the name of the configuration backup file. Lower-case and upper-case letters, numbers, and special characters including “@”, “-”, “_” and “!” are supported.
-----------	---

config all file <file_name> <password>

This command is used to load the TGZ configuration backup file.

file_name	This parameter specifies the name of the TGZ configuration backup file. The specified file name should not contain the extension name “.tgz”. Lower-case and upper-case letters, numbers, and special characters including “@”, “-”, “_” and “!” are supported.
-----------	---

password	This parameter specifies the password for accessing the
----------	---

TGZ configuration backup file. Its value must be a string of 1 to 8 characters.

config memory

This command allows users to restore the configuration from the last “write memory” operation.

config net tftp <tftp_server_ip> <config_file_name>

This command allows users to load configuration data stored on a TFTP server. The IP address (in quotation marks) or the name of the TFTP server and the remote file name need to be supplied.

config net sftp <remote_server_ip> <user_name> <config_file_name>

This command is used to load configuration data stored on a remote SFTP server. To access the remote SFTP server, you need to provide its IP address, the username (you will be prompted for the server password), and the configuration file name.

`remote_server_ip` The IP address of the remote SFTP server.

`user_name` The username to access the SFTP server.

`config_file_name` The name of the configuration data file.

config net scp {remote_server_ip|name} <user_name> <config_file_path>

This command allows users to load configuration data stored on an SCP server. The IP address (in quotation marks) or the name of the SCP server, the user’s name for the remote machine being accessed (a password prompt for the remote machine will appear) and the remote file path need to be supplied.

config net http <http_url>

This command allows users to download a configuration file from a Web server. The “http_url” parameter is used to specify the URL address of the configuration file. For example, if you want to download the file “array.conf” from the Web server “www.xyz.com”, the “http_url” parameter should be “http://www.xyz.com/array.conf”.

config net ftp <ftp_ip> <ftp_username> <file_name>

This command is used to load configuration data stored on a remote FTP server.

`ftp_ip` The IP address of the remote FTP server.

`ftp_username` The username of the remote FTP server.

<code>file_name</code>	The name of the file in which the configuration data is saved.
------------------------	--

`write net tftp <tftp_ip> [file_name]`

This command is used to store the current configuration to a remote TFTP server.

<code>tftp_ip</code>	This parameter specifies the IPv4 address of the TFTP server.
----------------------	---

<code>file_name</code>	Optional. This parameter specifies the name of the remote file in which the configuration data is saved. The default value is “ca.cfg”.
------------------------	---

`write net scp {remote_server_ip|name} <user_name> <config_file_name>`

This command is used to store the current configuration to the remote SCP server with password authentication enabled. After the IP address (in quotation marks) or the name of the server, the user’s name for the remote SCP server being accessed and the remote file path are supplied, a password authentication prompt for the remote server will appear.

`write net ftp <ftp_ip> <ftp_username> <file_name>`

This command is used to store the current configuration to the specified remote FTP server.

<code>ftp_ip</code>	The IP address of the remote FTP server.
---------------------	--

<code>ftp_username</code>	The username of the remote FTP server.
---------------------------	--

<code>file_name</code>	The name of the file in which the configuration data is saved.
------------------------	--

`write net sftp <remote_server_ip> <user_name> <config_file_name>`

This command is used to store the current configuration to the remote SFTP server with password authentication enabled. After the IP address (in quotation marks) or the name of the server, the user’s name for the remote SCP server being accessed and the remote file path are supplied, a password authentication prompt for the remote server will appear.

<code>remote_server_ip</code>	The IP address of the SFTP server.
-------------------------------	------------------------------------

<code>user_name</code>	The username to access the SFTP server.
------------------------	---

<code>config_file_name</code>	The name of the local configuration data file.
-------------------------------	--

write all tftp <tftp_ip> <remote_file_name> <password>

This command is used to back up all system, running and segment configurations on the APV appliance to a remote host via the TFTP protocol.

tftp_ip	This parameter specifies the IP address of the remote host.
remote_file_name	This parameter specifies the name of the file to back up the APV configurations on the remote host.
password	This parameter specifies the password for accessing backup.conf in the configuration file saved on the remote host. Its value must be a string of 1 to 8 characters.

write all sftp <server_name> <user_name> <remote_file_path> <password>

This command is used to back up all system, running and segment configurations on the APV appliance to a remote host via the SFTP protocol.

server_name	This parameter specifies the host name or IP address of the remote host.
user_name	This parameter specifies the username to access the remote host.
remote_file_path	This parameter specifies the path to save the configuration file on the remote host.
password	This parameter specifies the password used for accessing backup.conf in the configuration file saved on the remote host. Its value must be a string of 1 to 8 characters.

config all tftp <tftp_ip> <remote_file_name> <password>

This command is used to restore the entire configurations from a remote host via the TFTP protocol.

tftp_ip	The IP address of the remote host.
remote_file_name	The name of the configuration file on the remote host.
password	The password used for accessing backup.conf in the configuration file saved on the remote host. Its value must be a string of 1 to 8 characters. If a wrong password is input, backup.conf in the configuration file cannot be decrypted and restored.

**config all sftp <server_name> <user_name> <remote_file_path>
<password>**

This command is used to restore the entire configurations from a remote host via the SFTP protocol.

server_name	The host name or IP address of the remote host.
user_name	The username to access the remote host.
remote_file_path	The path of saving the configuration file on the remote host.
password	The password for accessing backup.conf in the configuration file saved on the remote host. Its value must be a string of 1 to 8 characters. If a wrong password is input, backup.conf in the configuration file cannot be decrypted and restored.

write all scp <server_name> <user_name> <remote_file_path> <password>

This command is used to back up all system, running and segment configurations on the APV appliance to a remote host via the SCP protocol.

server_name	This parameter specifies the host name or IP address of the remote host.
user_name	This parameter specifies the user name to access the remote host.
remote_file_path	This parameter specifies the path to save the configuration file on the remote host.
password	This parameter specifies the password used for accessing backup.conf in the configuration file saved on the remote host. Its value must be a string of 1 to 8 characters.

config all scp <server_name> <user_name> <remote_file_path> <password>

This command is used to restore the entire configurations from a remote host via the SCP protocol.

server_name	The host name or IP address of the remote host.
user_name	The user name to access the remote host.
remote_file_path	The path of saving the configuration file on the remote host.
password	The password for accessing backup.conf in the configuration file saved on the remote host. Its value must be a string of 1 to 8 characters. If a wrong password is input, backup.conf in the configuration file cannot be

decrypted and restored.

no config <file_name>

This command is used to delete the specified configuration backup file.

file_name	This parameter specifies the name of the configuration backup file to be deleted. The specified file name must contain its extension name.
-----------	--

show config file [file_name] [regex]

This command is used to display the configuration items, which match the filtering string, in the specified configuration backup file.

file_name	This parameter specifies the name of the configuration backup file to be displayed. The specified file name should not contain its extension name. If this parameter is not specified, all configuration backup files will be displayed.
-----------	--

regex	This parameter specifies the string used to filter configuration items in the specified configuration backup file. It is available only when the “file_name” parameter is set. For example, the “ show config file new_lb tcp ” command will display all configuration items that contain the “tcp” string in the “new_lb” configuration backup file. If this parameter is not set, all configuration items in the specified configuration backup file will be displayed.
-------	--

clear config file

This command is used remove all user-defined configuration files.

system fallback [partition_number]

This command is used to boot the APV appliance from the other root version on the next reboot.

partition_number	Optional. This parameter specifies the number of the partition to run on the next reboot. Its value must be an integer ranging from 0 to 3, and cannot be the number of the partition currently running. If this parameter is not specified, the appliance will update and switch over to the next partition on next booting up.
------------------	--

This parameter only takes effect for software versions that support the four-partition system disk.



Note: When a software version that only supports the two-partition system disk is installed on a four-partition disk, system boot will fail. Therefore, do not install a two-partition software version on a four-partition system disk.

no system fallback

This command is used to disable the system fallback functionality.

clear config secondary

This command is used to restore all the settings on the APV appliance except the “primary” settings (restored by the command “**clear config primary**”), including settings about WebUI, NAT, FWD, SNMP, log, domain server, proxy server, cluster, HA configuration (excluding SSF peer unit configurations) and synchronization secure mode.

If the administrative privilege separation function is enabled, after executing this command, the administrative privilege separation function will be disabled. The users and roles of the administrative privilege separation function will not be deleted, but their associations will be deleted.

clear config primary

This command is used to restore the basic network settings to the default value, including settings about IP address, access list, access group, SSH IP address, Enable level password, “array” user password, etc. At the same time, all the users in the system except the “array” user will be removed.

This command cannot be executed if there are other configurations based on these basic network settings. In this situation, please execute the command “**clear config secondary**” first to delete the related configurations, and then execute the command “**clear config primary**” again.

If the administrative privilege separation function is enabled, after executing this command, the administrative privilege separation function will remain enabled. The users and roles of the administrative privilege separation function and their associations will not be deleted.

clear config all

This command allows users to restore all settings on the APV appliance.

clear config factorydefault

This command is used to reset the APV appliance box to factory default settings. Different from the existing command “**clear config all**”, this command will clean imported SSL key files so that previous user configuration influence will be totally removed. This command is an interactive command. You will be prompted the following message when executing this command: “Type ‘YES’ to revert the

configuration to factory defaults”. You need to type “YES” to continue the command execution.

show running [pattern]

This command is used to display the current configuration of your APV appliance and all active function settings for the current configuration of the APV appliance.

pattern Optional. Specify a string that is used to match the current configuration. For example “**show running tcp**” will display all file lines with the string TCP. The string contains a maximum of 100 characters.

show startup [pattern]

This command allows users to view previously saved configuration by using the command “**write memory**”. The optional parameter “pattern” calls for a string for the APV appliance to search for. For example “show running tcp” will display all file lines with the string TCP.

show version

This command is used to display the system specific data such as host name, current software version, other root versions, system CPU, available memory and total memory, latest booting time, licensed features and system up time.

show system partition

This command is used to display the software version of each partition.

show tech

This command is used to display real-time statistics of the current running system and network.

show system openport

This command is used to display the system’s all port numbers that are in open status.

debug watch

This command allows users to monitor a command string executed every few seconds so that a variance can be observed. This is an interactive command and the user will be prompted for additional information.



Note:

- It is recommended to execute this command through SSH or WebUI instead of Console.
- If you execute this command through Console, please set the refresh interval to an integer of no less than 10.

- Administrators can press Ctrl+C to stop the command at any time. Pressing Enter will stop the monitored sub-command after the output is completed.

System Management

system shutdown

This command is used to halt all functions of the APV appliance and disable the CLI. A manual reboot of the APV appliance will be necessary to reinstate CLI control. Users should wait five to ten seconds after employing this command before terminating power to the appliance.

system update <url> [immediate|deferred] [partition_number]

This command is used to import a new software version directly from Array Networks. With the optional parameter “immediate|deferred” specified, users can update the system immediately or defer the operation until the next boot up. With the parameter “immediate|deferred” specified as immediate, when the user inputs the URL supplied by Array on executing this command, the APV appliance will import the updating package from the URL, finish the updating operation, import the configurations of last version and reboot the appliance. With the parameter “immediate|deferred” specified as “deferred”, the update process and configuration synchronization will be running at the backend without interrupting the current system operations. However, the system will mark the current status as “deferred” and save all configurations occurred after the update operation. When the appliance reboots next time, the newly-updated system will take effect and the configurations will be synchronized into the new system when the command “**write all deferred**” is executed.

url	This parameter specifies the URL through which updating package can be imported. Note: The “path” of the FTP or SFTP URL (ftp://user:password@host:port/path or sftp://user:password@host:port/path) should be a relative path.
immediate deferred	Optional. If this parameter is specified as “immediate”, the system will perform update operation immediately and reboot the appliance. If this parameter is specified as “deferred”, the update process and configuration synchronization will be running at the backend without interrupting the current system operations. When the appliance reboots next time, the newly-updated system will take effect. The default value is “immediate”.
partition_number	Optional. This parameter specifies the number of the partition to be updated. Its value must be an integer ranging from 0 to 3, and cannot be the number of the partition currently running. The software version of each partition

can be viewed through the “**show version**” and “**show system partition**” commands. If this parameter is not specified, the appliance will update and switch over to the next partition on the next reboot.

This parameter only takes effect for software versions that support the four-partition system disk.



Note: When a software version that only supports the two-partition system disk is installed on a four-partition disk, system boot will fail. Therefore, do not install a two-partition software version on a four-partition system disk.

show system update deferred

This command is used to show configurations of deferred update.

system patch update <url>

This command is used to install a patch package into the system.

The patch package is provided for only a particular version, so it can be installed into the particular version only. After the installation, the system version number remains unchanged. In the “**show version**” command output, a “Patch version” line will display the patch version and installation time, such as “p1 on Tues May 23 10:48:19 2021”.

This command supports only patch upgrades but not downgrades. For example, after the system is upgraded from p1 to p2, a downgrade to p1 is not allowed.

After the installation, the system will not restart, but unsaved configurations might be lost. Therefore, saving all configurations using “**write memory**” before the installation is recommended.

url	This parameter specifies the URL address where the patch package locates. HTTP and FTP URLs are supported.
-----	--

system patch rollback

This command is used to roll back the system from the current patch version to the original version, that is, the version without any patch packages installed. For example, with patch packages p1 and p2 installed, the system is running the patch version “p2 on Tues May 25 08:32:45 2021”. After this command is executed, the system will be rolled back to the original version without any patch packages.

write all deferred

This command is used to save the current system configurations. If a deferred update operation is pending, the system will, upon running this command, save and synchronize all configurations into the new system. If no deferred update operation is

pending, the system will save the configurations with the same effect of the command “write memory”.

Management Access

WebUI Access

webui {on|off}

This command is used to enable or disable the WebUI. By default, the WebUI is disabled.

webui hostcheck {on|off}

This command is used to enable or disable HTTP Host and Referer header checking. After enabling this function, the system checks whether the Host and Referer headers in the client request are consistent with the IP address and port number of the server. By default, this function is enabled.

clear webui hostcheck

This command is used to reset the HTTP Host and Referer header checking function.

webui port <port>

This command is used to change the port number for accessing the WebUI. If this command is not configured, the default port number 8888 will be used.

port	This parameter specifies the port number for accessing the WebUI. Its value must be an integer ranging from 1025 to 65,000.
------	---

clear webui port

This command is used to reset the port number for accessing the WebUI to 8888.

webui graph {on|off}

This command is used to enable or disable the line chart statistics function of the WebUI. After this function is enabled, the appliance will collect and save statistics information and display the information in line charts. After this function is disabled, the statistics cannot be displayed in line charts.

webui graph refresh <interval>

This command is used to set the interval at which WebUI graphs are refreshed.

When this command is not configured, WebUI graphs are refreshed at the interval of 10s.

interval	This parameter specifies the interval at which WebUI graphs are refreshed, in seconds. Its value must be 10, 20, 30, 60, 180 or 300.
----------	--

webui graph clear

This command is used to clear the existing statistics information for graphs.

webui language <login_language>

This command is used to set the WebUI login language. On the first login, the WebUI language will be the same as the system default language. On subsequent logins, the WebUI will display content through the language configured by the administrator.

login_language	It can be en (English), cn (Simplified Chinese) and jp (Japanese), tw (Traditional Chinese). It defaults to “en”.
----------------	---

clear webui language

This command is used to reset the WebUI language to English.

webui ip <ip_address>

This command is used to allow users to set the WebUI IP address. After executing the command, the APV appliance will only accept the connections at the specific IP address.



Note:

- Only the IP address of system interface and the management VIP can be configured as the WebUI IP address.
- The system management traffic may be affected when other traffic is high. The administrator can configure the WebUI IP address for management traffic to avoid such a situation.

clear webui ip

This command is used to delete the WebUI IP address. After executing the command, the APV appliance will accept the connections at any IP address.

webui source <source_ip> <netmask|prefix> [action]

This command is used to add an WebUI source IP restriction rule. The action of the restriction rule can be “permit” or “deny” The “permit” rule allows the administrator’s WebUI access from the specified source IP address or segment. The “deny” rule rejects the administrator’s WebUI access from the specified source IP address or segment. A maximum of 100 WebUI source IP restriction rules can be configured. If no WebUI source IP restriction rule is configured, the system allows the administrator to access the system from any source IP address or segment via WebUI. If WebUI source IP restriction rules are configured:

- If the source IP address used for WebUI access matches a “permit” rule and does not match any “deny” rule, the system allows the administrator’s WebUI access.
- If the source IP address used for WebUI access matches a “permit” rule but also match a “deny” rule, the system will reject the administrator’s WebUI access.
- If the source IP address used for WebUI access does not match any “permit” rule, the system will directly reject the administrator’s WebUI access.

source_ip Specify the source IP address or segment accessing the system via WebUI. If the parameter value is “0.0.0.0”, it indicates all IPv4 addresses.

Note: For the IPv6 address, the parameter value cannot be “::”.

netmask|prefix Specify the netmask or prefix length of the source IP address or segment.

- “netmask” is used for an IPv4 address. It can be a dotted source IP address or an integer. If it is an integer, its value should range from 0 to 32.
- “prefix” is used for an IPv6 address. Its value is an integer ranging from 1 to 128.

action Optional. This parameter specifies the action to take when the source IP address used for WebUI access matches the rule. The parameter value can be:

- permit: Allows the specified source IP address or segment to access the system through the WebUI connection.
- deny: Rejects the specified source IP address or segment to access the system through the WebUI connection.

The default value is “permit”.



Note: If only “deny” rules are configured, administrators cannot access the system via WebUI from any subnet.

no webui source <source_ip> <netmask|prefix>

This command is used to delete the specified WebUI source IP restriction rule.

clear webui source

This command is used to clear all WebUI source IP restriction rules.

webui idletimeout <timeout>

This command is used to set the timeout value for idle WebUI connections. When a WebUI connection is idle for the specified time of period, it will automatically disconnect.

When this command is not configured, a WebUI connection will timeout after being in idle state for 15 minutes.

timeout	This parameter specifies the idle timeout value for WebUI connections, in minutes. Its value must be an integer ranging from 1 to 65535.
---------	--

clear webui idletimeout

This command is used to restore the idle timeout value for WebUI connections to default.

webui ssl settings clientauth {enable|disable}

This command is used to enable or disable the SSL client authentication function for the WebUI, which is also known as the two-way SSL authentication function. When this function is enabled, the SSL client and the SSL server will perform two-way SSL authentication during the SSL handshake when the SSL client logs into the WebUI. Before enabling this function, administrators need to use the “**webui ssl import clientca**” command to import the client CA certificate for APV WebUI. By default, this function is disabled.

webui ssl settings authmandatory {enable|disable}

This command is used to enable or disable the mandatory mode of the SSL client authentication function for the WebUI. When the mandatory mode is enabled, administrators must pass the WebUI SSL client authentication for WebUI login. Otherwise, the WebUI SSL connection request will be rejected. By default, this function is enabled. The mandatory mode can only take effect after the “**webui ssl settings clientauth enable**” command is already enabled.

webui ssl settings ciphersuites <cipher_string>

This command is used to specify the cipher suites supported by the WebUI. When this command is not configured, the default value is

“ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256:ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-SHA384:AE S128-GCM-SHA256:AES256-GCM-SHA384”.

cipher_string	This parameter specifies the cipher suites supported by the WebUI. Its value is case insensitive. To specify multiple cipher suites, separate them with colons (:). Currently, the following cipher suites are supported by the WebUI:
---------------	--

- ECDHE-ECDSA-AES128-GCM-SHA256

- ECDHE-RSA-AES128-GCM-SHA256
- ECDHE-ECDSA-AES256-GCM-SHA384
- ECDHE-RSA-AES256-GCM-SHA384
- ECDHE-ECDSA-AES128-SHA256
- ECDHE-RSA-AES128-SHA256
- ECDHE-ECDSA-AES256-SHA384
- ECDHE-RSA-AES256-SHA384
- AES128-GCM-SHA256
- AES256-GCM-SHA384
- AES128-SHA256
- AES256-SHA256

webui ssl settings protocol <version>

This command is used to specify the SSL protocol versions supported by the WebUI. When this command is not configured, the default value is “TLSv12”.

version

This parameter specifies the SSL protocol versions supported by the WebUI. To support multiple protocols, separate them with colons (:). Its value must be:

- SSLv3: indicates that the SSLv3 protocol is supported.
- TLSv1: indicates that the TLSv1.0 protocol is supported.
- TLSv11: indicates that the TLSv1.1 protocol is supported.
- TLSv12: indicates that the TLSv1.2 protocol is supported.
- ALL: indicates that all the protocols above are supported.

webui ssl import certificate [url]

This command is used to import a .pem or .pfx certificate chain file for the WebUI via copy-and-paste, from a remote FTP, TFTP or HTTP server, or from a local computer. The certificate chain file contains a certificate and the associated private key.

When importing a .pem and .pfx certificate, you'll be prompted to enter its password.

When the “url” parameter is not specified, you can import the certificate chain file by copying and pasting the contents of the certificate into the CLI. You need to enter “...” in the bottom line following the certificate to mark the end of the import.

If this command is not configured, the default certificate will be used.

url

Optional. This parameter specifies the FTP, TFTP or HTTP URL from which the `.pem` or `.pfx` certificate chain file is imported. When this parameter is not specified, you can import the certificate chain file via copy-and-paste.

You can also import a local file if your certificate is on the local computer.

Example:

- Import from a remote computer:
webui ssl import certificate ftp://10.8.6.20/cert/chain.pem
- Import from a local computer:
 - **webui ssl import certificate** file://home/test/pass1234.pfx
 - **webui ssl import certificate**
file://home/test/cert_manohar_pass1234.pem

Note: The “path” of the FTP URL (ftp://user:password@host:port/path) should be a relative path.



Note: Currently, the system does not support importing SM2 and RSAPSS certificate.

webui ssl import clientca [url]

This command is used to import a CA intermediate certificate for the WebUI SSL client authentication function via copy-and-paste or from a remote FTP, TFTP or HTTP server.

When the “url” parameter is not specified, you can import the certificate by copying and pasting the contents of the intermediate certificate into the CLI. The entering of “...” is required in the bottom line following the certificate to mark the end of the import.

url

Optional. This parameter specifies the FTP, TFTP or HTTP URL from which the CA certificate is imported. When this parameter is not specified, you can import the certificate via copy-and-paste.

Note: The “path” of the FTP URL (ftp://user:password@host:port/path) should be a relative path.

show webui ssl certificate

This command is used to display the certificate imported for the WebUI.

show webui ssl clientca

This command is used to display the client CA certificate imported for the WebUI.

clear webui ssl certificate

This command is used to restore the default certificate of the WebUI.

clear webui ssl clientca

This command is used to delete the client CA certificate imported for the WebUI.

show webui ssl settings

This command is used to display all the WebUI SSL configurations.

Example:

```
AN(config)#show webui ssl settings
webui ssl settings clientauth disable
webui ssl settings authmandatory enable
webui ssl settings ciphersuites
"ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256:ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-SHA384:AES128-GCM-SHA256:AES256-GCM-SHA384"
webui ssl settings protocol "TLSv12"
```

clear webui ssl settings

This command is used to reset all the WebUI SSL configurations to the default value.

show webui settings

This command is used to display all the WebUI configurations.

SSH Access**ssh {on|off}**

This command allows users to enable or disable SSH access to the APV appliance. The SSH feature is enabled by default.

ssh ip <ip_address>

This command is used to configure the SSH IP address. If the SSH IP is configured, the administrator can only use the specified IP address to access the APV appliance through SSH. If the SSH IP is not configured, the administrator can use any IP address of the APV appliance to access the APV appliance through SSH.

The system management traffic may be affected when other traffic is high. The administrator can configure the SSH IP address for management traffic to avoid such a situation.

ip_address This parameter specifies the SSH IP address. Its value must be an IPv4 or IPv6 address.

**Note:**

- The configurations of SSH IP address will be deleted after the administrator executes the command “**clear config primary**” command.
- The synconfig peer must be configured with the same SSH IP address. The configurations of SSH IP address cannot be synchronized after the administrator executes the command “**synconfig to**” or “**synconfig from**”.
- In cluster and HA, if a VIP is configured as the SSH IP address, the administrator should reenale the SSH IP function on the new master after the failover.
- Only the IP address of the system interface and the management VIP can be configured as the SSH IP address. The SLB VIP and the NAT VIP cannot be configured as the SSH IP address.

no ssh ip

This command is used to delete the SSH IP address configuration.

ssh port [port_number]

This command is used to set the SSH listening port. If this command is not configured, the default port 22 will be used for SSH connections to the APV appliance.

port_number Optional. This parameter specifies the SSH port number. Its value must be an integer ranging from 1 to 65535.

The default value is 22.

ssh source <source_ip> <netmask|prefix> [action]

This command is used to add an SSH source IP restriction rule. The action of the restriction rule can be “permit” or “deny”. The “permit” rule allows the administrator’s SSH access from the specified source IP address or segment. The “deny” rule rejects the administrator’s SSH access from the specified source IP address or segment. A maximum of 100 SSH source IP restriction rules can be configured. If no SSH source IP restriction rule is configured, the system allows the administrator to access the system from any source IP address or segment via SSH. If SSH source IP restriction rules are configured:

- If the source IP address used for SSH access matches a “permit” rule and does not match any “deny” rule, the system allows the administrator’s SSH access.
- If the source IP address used for SSH access matches a “permit” rule but also match a “deny” rule, the system will reject the administrator’s SSH access.

- If the source IP address used for SSH access does not match any “permit” rule, the system will directly reject the administrator’s SSH access.

source_ip	Specify the source IP address or segment accessing the system via SSH. If the parameter value is “0.0.0.0”, it indicates all IPv4 addresses; if the parameter value is “::”; it indicates all IPv6 addresses.
netmask prefix	Specify the netmask or prefix length of the source IP address or segment. • “netmask” is used for an IPv4 address. It can be a dotted source IP address or an integer. If it is an integer, its value should range from 0 to 32. • “prefix” is used for an IPv6 address. Its value is an integer ranging from 0 to 128.
action	Optional. This parameter specifies the action to take when the source IP address used for SSH access matches the rule. The parameter value can be: • permit: Allows the specified source IP address or segment to access the system through the SSH connection. • deny: Rejects the specified source IP address or segment to access the system through the SSH connection. The default value is “permit”.



Note: If only “deny” rules are configured, administrators cannot access the system via SSH from any subnet.

no ssh source <source_ip> <netmask|prefix>

This command is used to delete the specified SSH source IP restriction rule.

clear ssh source

This command is used to clear all SSH source IP restriction rules.

ssh import key <user_name> [url]

This command is used to import an SSH public key for the specified administrator account.

user_name	This parameter specifies the user name of an existing administrator account.
url	Optional. This parameter specifies the URL link used to download the SSH public key. Its value must be a URL of

the HTTP, FTP, or TFTP type. If this parameter is not specified, you must manually enter the SSH public key. The appliance supports SSH or OpenSSH public keys, the minimum length is 1024 bits and the maximum file size is 4096 bytes.

no ssh key <user_name>

This command is used to delete the SSH public key configured for the specified administrator account.

show ssh key [user_name]

This command is used to display the SSH public key configuration for the specified administrator account. If the “user_name” parameter is not specified, this command will display the SSH public key configurations for all administrator accounts.

clear ssh key

This command is used to clear all SSH public keys.

ssh auth key {on|off} <user_name>

This command is used to enable or disable SSH public key authentication for the specified administrator account. For the newly created administrator account, SSH public key authentication is disabled by default.

ssh auth passwd {on|off} <user_name>

This command is used to enable or disable SSH password authentication for the specified administrator account. For a newly created administrator account, SSH password authentication is enabled by default.

ssh cipher <cipher_string>

This command is used to configure the supported SSH ciphers.

When this command is not configured, the default supported ciphers are aes128-ctr, aes192-ctr, aes256-ctr, aes128-gcm@openssh.com, and aes256-gcm@openssh.com.

cipher_string	This parameter specifies SSH ciphers. Its value can be one or many of the following ciphers: aes128-cbc, aes192-cbc, aes256-cbc, aes128-ctr, aes192-ctr, aes256-ctr, aes128-gcm@openssh.com, and aes256-gcm@openssh.com. If multiple ciphers need to be configured, separate them with commas.
---------------	--



Note: The system no longer supports the following weak SSH ciphers: arcfour, arcfour128, arcfour256, blowfish-cbc, cast128-cbc, and chacha20-poly1305@openssh.com. Client cannot connect to the system via

these SSH weak ciphers anymore.

clear ssh cipher

This command is used to reset the configuration of SSH ciphers to default.

ssh alivetime *[interval] [max_check_times]*

This command is used to configure the lifetime of SSH connections in the idle state.

After this command is configured, the system will send detection packets at the interval specified by the “interval” parameter and wait for responses. When the number of consecutive times that the system fails to receive a response reaches the value specified by the “max_check_times” parameter, the system will terminate the SSH connection.

When this command is not configured, the system does not check whether an SSH connection is in the idle state.

interval Optional. This parameter specifies the interval at which detection packets are sent, in seconds. Its value must be an integer ranging from 1 to 86,400.

The default value is 60.

max_check_times Optional. This parameter specifies the number of times that a detection packet is sent. Its value must be an integer ranging from 1 to 30.

The default value is 3.

no ssh alivetime

This command is used to delete the configuration of the “**ssh alivetime**” command.

ssh idletimeout *[timeout]*

This command is used to set the timeout value for idle SSH connections. When a SSH connection is idle for the specified time of period, the system will disconnect this connection.

When this command is not configured, the system will disconnect a SSH connection after it is in idle state for 30 minutes.

timeout Optional. This parameter specifies the idle timeout value for SSH connections, in minutes. Its value must be an integer ranging from 1 to 65535. The default value is 30.

clear ssh idletimeout

This command is used to restore the idle timeout value for SSH connections to default.

ssh kex <kex_list>

This command is used to configure the SSH key exchange algorithm. When this command is not configured, the default SSH key exchange algorithms are curve25519-sha256,curve25519-sha256@libssh.org, ecdh-sha2-nistp256, ecdh-sha2-nistp384 and ecdh-sha2-nistp521.

kex_list This parameter specifies the SSH key exchange algorithms. Its value can be one or more of curve25519-sha256,curve25519-sha256@libssh.org,diffie-hellman-group1-sha1,diffie-hellman-group14-sha1,diffie-hellman-group14-sha256,diffie-hellman-group16-sha512,diffie-hellman-group18-sha512,diffie-hellman-group-exchange-sha1,diffie-hellman-group-exchange-sha256,ecdh-sha2-nistp256,ecdh-sha2-nistp384,ecdh-sha2-nistp521.

If multiple algorithms need to be configured, separate them with commas",".

Note: When the value contains special characters or when multiple algorithms are configured at the same time, it needs to be enclosed in double quotes.

clear ssh kex

This command is used to reset the SSH key exchange algorithm to default.

show ssh config

This command is used to display the status of the SSH function and related configurations.

RESTful API Access**restapi on [protocol] [port]**

This command is used to enable the RESTful API service. By default, this function is disabled.

protocol Optional. This parameter specifies the protocol used by the RESTful API service. Its value must be “http” or “https”.

The default value is “https”.

port Optional. This parameter specifies the port on which the RESTful API service listens. Its value must be an integer ranging from 1025 to 65,000, but should not be the same as

the one used by any other service.

The default value is 9997.

restapi off

This command is used to disable the RESTful API service.

restapi ip <ip_address>

This command is used to set the RESTful API listening IP address. After executing the command, administrators can only access the appliance's RESTful API service via the specified IP address.

ip_address	This parameter specifies the IP address on which the RESTful API service listens. Its value must be an IPv4 or IPv6 address.
------------	--



Note: It is recommended to set the IP address of the management interface as the RESTful API listening IP address for system security.

clear restapi ip

This command is used to delete the setting of the RESTful API listening IP address.

clear restapi port

This command is used to reset the RESTful API listening port to the default value 9997.

restapi source <source_ip> <netmask|prefix> [action]

This command is used to add a RESTful API source IP restriction rule. This command can also be used to modify the rule action configured for the existing “source_ip” and “netmask|prefix”. The action of the restriction rule can be “permit” or “deny”. A maximum of 100 RESTful API source IP restriction rules can be configured.

If no RESTful API source IP restriction rule is configured, the system allows the administrator to access the system from any source IP address via RESTful API. If RESTful API source IP restriction rules are configured:

- If the source IP address used for RESTful API access matches a “permit” rule and does not match any “deny” rule, the system allows the administrator's RESTful API access.
- If the source IP address used for RESTful API access matches a “permit” rule but also matches a “deny” rule, the system will reject the administrator's RESTful API access.

- If the source IP address used for RESTful API access does not match any “permit” rule, the system will directly reject the administrator’s RESTful API access.

source_ip	This parameter specifies the source IP address or segment accessing the system via RESTful API. The parameter can be an IPv4 or IPv6 address. Note: For the IPv4 address, the parameter value cannot be “0.0.0.0”. For the IPv6 address, the parameter value cannot be “::”.
netmask prefix	This parameter specifies the netmask or prefix length of the source IP address or segment. • netmask: specifies the netmask for an IPv4 address. It can be a dotted source IP address or an integer. If it is an integer, its value should range from 0 to 32. • prefix: specifies the prefix for an IPv6 address. Its value is an integer ranging from 1 to 128.
action	Optional. This parameter specifies the action to take when the source IP address used for RESTful API access matches the rule. The parameter value can be: • permit: allows the specified source IP address or segment to access the system through the RESTful API connection. • deny: rejects the specified source IP address or segment to access the system through the RESTful API connection. The default value is “permit”.



Note: If only “deny” rules are configured, administrators cannot access the system via RESTful API from any subnet.

no restapi source <source_ip> <netmask|prefix>

This command is used to delete the specified RESTful API source IP restriction rule.

clear restapi source

This command is used to clear all RESTful API source IP restriction rules.

show restapi settings

This command is used to display the configuration of the RESTful API service (excluding the SSL configuration).



Note: To access the RESTful API service, you must possess an administrator account with the API privilege level and pass HTTP basic authentication. For details about how to create an administrator account with the API privilege level, see the “**user** <user_name> <password> [level]” command.

restapi ssl settings protocol <version>

This command is used to specify the SSL protocol versions supported by the RESTful API service. When this command is not configured, the default value is “TLSv12”.

version	This parameter specifies the SSL protocol versions supported by the RESTful API service. To support multiple protocols, separate them with colons (:). Its value must be: • SSLv3: indicates that the SSLv3 protocol is supported. • TLSv1: indicates that the TLSv1.0 protocol is supported. • TLSv11: indicates that the TLSv1.1 protocol is supported. • TLSv12: indicates that the TLSv1.2 protocol is supported. • ALL: indicates that all the protocols above are supported.
---------	---

restapi ssl settings ciphersuites <cipher_string>

This command is used to specify the cipher suites supported by the RESTful API service. When this command is not configured, the default value is “ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256:ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-SHA384:AE S128-GCM-SHA256:AES256-GCM-SHA384”.

cipher_string	This parameter specifies the cipher suites supported by the RESTful API service. Its value is case insensitive. To specify multiple cipher suites, separate them with colons (:). Currently, the following cipher suites are supported by the the RESTful API service: • ECDHE-ECDSA-AES128-GCM-SHA256 • ECDHE-RSA-AES128-GCM-SHA256 • ECDHE-ECDSA-AES256-GCM-SHA384 • ECDHE-RSA-AES256-GCM-SHA384 • ECDHE-ECDSA-AES128-SHA256 • ECDHE-RSA-AES128-SHA256 • ECDHE-ECDSA-AES256-SHA384
---------------	---

- ECDHE-RSA-AES256-SHA384
- AES128-GCM-SHA256
- AES256-GCM-SHA384
- AES128-SHA256
- AES256-SHA256

restapi ssl import certificate [url]

This command is used to import a PEM-format certificate chain file for the RESTful API service via copy-and-paste or from a remote FTP, TFTP or HTTP server. A PEM-format certificate chain file contains a certificate and the associated private key.

When the “url” parameter is not specified, you can import the certificate chain file by copying and pasting the contents of the PEM-format certificate into the CLI. The entering of “...” is required in the bottom line following the certificate to mark the end of the import.

If this command is not configured, the default certificate will be used.

url	Optional. This parameter specifies the FTP, TFTP or HTTP URL from which the PEM-format certificate chain file is imported. When this parameter is not specified, you can import the certificate chain file via copy-and-paste.
-----	--



Note: Currently, the system does not support importing SM2 and RSAPSS certificate.

show restapi ssl certificate

This command is used to display the certificate chain for the RESTful API service.

clear restapi ssl certificate

This command is used to reset the certificate chain for the RESTful API service to the default.

restapi ssl import clientca [url]

This command is used to import a CA certificate for the SSL client authentication function of the RESTful API service via copy-and-paste or from a remote FTP, TFTP or HTTP server.

When the “url” parameter is not specified, you can import the certificate by copying and pasting the contents of the intermediate certificate into the CLI. The entering of “...” is required in the bottom line following the certificate to mark the end of the import.

url Optional. This parameter specifies the FTP, TFTP or HTTP URL from which the CA certificate is imported. When this parameter is not specified, you can import the certificate via copy-and-paste.

show restapi ssl clientca

This command is used to display the client CA certificate imported for the RESTful API service.

clear restapi ssl clientca

This command is used to delete the client CA certificate imported for the RESTful API service.

restapi ssl settings clientauth enable

This command is used to enable the SSL client authentication function for the RESTful API service, which is also known as the two-way SSL authentication function. When this function is enabled, the SSL client and the SSL server will perform two-way SSL authentication during the SSL handshake when the SSL client logs into the RESTful API service. Before enabling this function, administrators need to use the “**restapi ssl import clientca**” command to import the client CA certificate for the RESTful API service. By default, this function is disabled.

restapi ssl settings clientauth disable

This command is used to disable the SSL client authentication function for the RESTful API service.

restapi ssl settings authmandatory enable

This command is used to enable or disable the mandatory mode of the SSL client authentication function for the RESTful API service. When the mandatory mode is enabled, administrators must pass the SSL client authentication for the RESTful API login. Otherwise, the SSL connection request for the RESTful API service will be rejected. By default, this function is enabled. Before enabling the mandatory mode, please use the “**restapi ssl settings clientauth enable**” command to enable the SSL client authentication function first.

restapi ssl settings authmandatory disable

This command is used to disable the mandatory mode of the SSL client authentication function for the RESTful API service.

show restapi ssl settings

This command is used to display all the SSL configurations of the RESTful API service.

Example:

```

AN(config)#show restapi ssl settings
restapi ssl settings clientauth disable
restapi ssl settings authmandatory enable
restapi ssl settings ciphersuites
"ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256:ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-SHA384:AES128-GCM-SHA256:AES256-GCM-SHA384"
restapi ssl settings protocol "TLSv12"

```

clear restapi ssl settings

This command is used to reset all SSL configurations of the RESTful API service to the default value.

XML RPC Access

xmlrpc on [*https|http*]

This command is used to enable the XML RPC function, which allows the administrator to gain access and configure the system from remote locations. By default, the XML RPC function is disabled.

https|http Optional. This parameter specifies the protocol used to transmit the XML RPC messages. The default value is “https”.



Note: When the XML RPC function is enabled, the system will listen on the default XML RPC port 9999 if the port is not changed by using the command “**xmlrpc port**”.

xmlrpc off

This command is used to disable the XML RPC function.

xmlrpc ip <*ip_address*>

This command is used to configure the XML RPC IP address. Generally, it can be set to an interface IP address or virtual IP address for management. The system supports only one XML RPC IP address.

After an XML RPC IP address is configured, the administrator can access the system via XML RPC at only this IP address. If no XML RPC IP address is specified, the administrator can access the system via XML RPC at any IP address of the appliance.

ip_address This parameter specifies the XML RPC IP address. Its value must be an IPv4 or IPv6 address.

clear xmlrpc ip

This command is used to clear the XML RPC IP configuration.

xmlrpc authentication {on|off}

This command is used to enable or disable XML RPC authentication. By default, it is disabled. After XML RPC authentication is enabled, to access the APV appliance via XML RPC, the administrator must first complete the account authentication. The user name and password used for authentication is created by the “**user**” or “**segment user**” command, and it must be configured with the API privilege level.

xmlrpc port <port>

This command is used to specify the port for the XML RPC to listen on.

port	Specifies the designated port for the XML-RPC to listen on. The parameter value ranges from 1025 to 65000. The default port is 9999.
------	--

xmlrpc source <source_ip> <netmask|prefix> [action]

This command is used to add an XML RPC source IP restriction rule. The action of the restriction rule can be “permit” or “deny”. The “permit” rule allows the administrator’s XML RPC access from the specified source IP address or segment. The “deny” rule rejects the administrator’s XML RPC access from the specified source IP address or segment. A maximum of 10 XML RPC source IP restriction rules can be configured. If no XML RPC source IP restriction rule is configured, the system allows the administrator to access the system from any source IP address or segment via XML RPC. If XML RPC source IP restriction rules are configured:

- If the source IP address used for XML RPC access matches a “permit” rule and does not match any “deny” rule, the system allows the administrator’s XML RPC access.
- If the source IP address used for XML RPC access matches a “permit” rule but also match a “deny” rule, the system will reject the administrator’s XML RPC access.
- If the source IP address used for XML RPC access does not match any “permit” rule, the system will directly reject the administrator’s XML RPC access.

source_ip	Specify the source IP address or segment accessing the system via XML RPC. If the parameter value is “0.0.0.0”, it indicates all IPv4 addresses.
-----------	--

Note: For the IPv6 address, the parameter value cannot be “::”.

netmask prefix	Specify the netmask or prefix length of the source IP address or segment. • “netmask” is used for an IPv4 address. It can be a dotted source IP address or an integer. If it is an integer, its value should range from 0 to 32.
----------------	--

- “prefix” is used for an IPv6 address. Its value is an integer ranging from 1 to 128.
- action
- Optional. This parameter specifies the action to take when the source IP address used for XML RPC access matches the rule. The parameter value can be:
- permit: Allows the specified source IP address or segment to access the system through the XML RPC connection.
 - deny: Rejects the specified source IP address or segment to access the system through the XML RPC connection.
 - The default value is “permit”..



Note: If only “deny” rules are configured, administrators cannot access the system via XML RPC from any subnet.

no xmlrpc source <source_ip> <netmask|prefix>

This command is used to delete the specified XML RPC source IP restriction rule.

clear xmlrpc source

This command is used to clear all WebUI XML RPC IP restriction rules.

show xmlrpc

This command is used to display all the XML RPC configurations.

clear xmlrpc all

This command is used to reset all the XML RPC configurations.

USB Storage Access

system usbaccess {on|off}

This command is used to configure whether the system can access the USB storage. By default, the system is not allowed to access the USB storage.

show system usbaccess

This command is used to display the configuration of the “**system usbaccess**” command.

ssh regenerate keys

This command is used to regenerate host keys for SSH server in ArrayOS. After this command is executed, the SSH server will use the newly generated keys as its host

key. SSH clients must be updated with the new public keys of the SSH server to connect with the server.

pager <lines>

This command is used to enable the paging function for the terminal emulator and set the number of lines displayed by the execution of each command.

lines	This parameter specifies the number of lines displayed by the execution of each command. Its value must be an integer ranging from 0 to 255. The value 0 indicates that the paging function setting of the current window will be used.
-------	---

no pager

This command is used to disable the paging function for the terminal emulator. When the paging function is disabled, the system will display all output of a command at a time.



Note: When the paging function is disabled, it is not recommended to execute commands with a large volume of output via the console connection, such as “**show tech**” and “**show log**”. System intermittance or service interruption might occur due to the slow transmission rate of the console port.

show pager

This command is used to display the configuration of the paging function for terminal emulators.

Role-based Privilege Management

role name <role_name>

This command is used to create a self-defined role. The system allows up to 32 self-defined roles.

role_name	This parameter specifies the name of the role, which is case sensitive and should be an alphanumeric string of up to 20 characters. It cannot be the role name of the administrative privilege separation function, namely, “role_administrator”, “role_operator” and “role_auditor”.
-----------	---

no role name <role_name>

This command is used to delete the specified self-defined role.

If the administrative privilege separation function is enabled, this command will not delete the three roles of the administrative privilege separation function. If the administrative privilege separation function is disabled, this command will delete the three roles of the administrative privilege separation function.



Note: Please make sure that the role to be deleted has not been assigned to any administrator.

show role name [*role_name*]

This command is used to display the “Permit” and “Deny” rules of the specified role. If the parameter “role_name” is not specified, all role names in the system will be displayed.

If the administrative privilege separation function is enabled and the “role_name” parameter is not specified, the system will also display the roles automatically created by the administrative privilege separation function.

clear role name

This command is used to delete all self-defined roles. After executing this command, the system will delete all associations between all self-defined roles and users, and remove all self-defined users.

If the administrative privilege separation function is enabled, this command will not clear the associations between the roles and administrative users of the administrative privilege separation function. If the administrative privilege separation function is disabled, this command will clear the associations between the roles and administrative users of the administrative privilege separation function.

After the execution of this command, all the administrators will not be assigned with any roles. Then, the administrators will have the privilege to execute all the CLI commands of their access control levels.

role permit <*role_name*> <*filter_string*>

This command is used to configure a self-defined “Permit” privilege rule for the specified role. The system allows up to 256 self-defined privilege rules per role.

role_name	This parameter specifies the role name. Its value must be a self-defined role. Its value must be a string of 1 to 20 characters. Letters, numbers, and special characters are supported. When the parameter value contains special characters or only numbers, it must be enclosed in double quotes.
filter_string	This parameter specifies the rule string for the “Permit” privilege rule. Its value must be a string of up to 64 characters. The rule string must be a complete CLI command or part of the command starting from the first

letter of the command.

When the administrator with the specific role executes this command, the system will check the command against the rule string from the first letter.

- If the command does not match any “Permit” rule string, the role will be denied to execute this command.
- If the command matches any “Permit” rule string, the system will match the command with the “Deny” rule strings (configured through the “**role deny**” command) to check if the role is permitted to execute this command.



Note:

- If no “Permit” rule string has been configured for a role, the role will be denied to executing any command.
- The system does not support configuring self-defined “Permit” rules for the administrative roles predefined by the administrative privilege separation function.

no role permit *<role_name>* *<filter_string>*

This command is used to delete a specified “Permit” privilege rule.

show role permit [*role_name*] [*filter_string*]

This command is used to display the “Permit” privilege rules of the specified role.

role_name	Optional. Specify the role name. If not specified, the “Permit” privilege rules of all roles will be displayed.
filter_string	Optional. Specify the “Permit” rule string. This parameter will only work when the parameter “role_name” is specified. If this parameter is not specified, all “Permit” privilege rules of the specified role will be displayed.

clear role permit [*role_name*]

This command is used to delete all self-defined “Permit” privilege rules of the specified role. If the parameter “role_name” is not specified, the “Permit” privilege rules of all roles will be deleted.

role deny *<role_name>* *<filter_string>*

This command is used to configure a self-defined “Deny” privilege rule for the specified role. The system allows up to 256 privilege rules per role.

role_name This parameter specifies the role name. Its value must be a self-defined role. Its value must be a string of 1 to 20 characters. Letters, numbers, and special characters are supported. When the parameter value contains special characters or only numbers, it must be enclosed in double quotes.

filter_string This parameter specifies the rule string for the “Deny” privilege rule. Its value should be a string of up to 64 characters. The rule string must be a complete CLI command or part of the command starting from the first letter of the command.

When the administrator with the specific role executes this command and this command has matched a “Permit” privilege rule, the system will then check the command against the “Deny” privilege rule from the first letter.

- If the command matches any “Deny” rule string, the role will be denied to executing this command.
- If the command does not match any “Deny” rule string, the role is permitted to execute this command.



Note: If no “Permit” rule string has been configured for a role, the role will be denied to executing any command.

no role deny <role_name> <filter_string>

This command is used to delete a specified “Deny” privilege rule.

show role deny [role_name] [filter_string]

This command is used to display the “Deny” privilege rules of the specified role.

role_name Optional. Specify the role name. Its value can be a self-defined role, roles predefined by the system or roles predefined by the administrative privilege separation function.

If this parameter is specified, the system will display all “Deny” rules configured for the specified role.

If not specified, the “Deny” privilege rules of all roles will be displayed.

filter_string Optional. Specify the “Deny” rule string. This parameter will only work when the parameter “role_name” is

specified.

If this parameter is specified, the system will display the specified “Deny” rules configured for the specified role.

If this parameter is not specified, all “Deny” privilege rules of the specified role will be displayed.

clear role deny [role_name]

This command is used to delete all self-defined “Deny” privilege rules of the specified role. If the parameter “role_name” is not specified, the “Deny” privilege rules of all roles will be deleted.

role clone <old_role_name> <new_role_name>

This command is used to clone a new role based on an existing role. The “Deny” and “Permit” privilege rules of the existing role will be copied to the new role.

old_role_name Specify the existing self-defined role name.

new_role_name Specify the new role name.

role test cli <role_name> <cli_command>

This command is used to check if the specified role has the privilege to execute a given command.

role_name Specify the role name.

cli_command Specify the CLI command. The parameter must be a complete CLI command or part of the command starting from the first letter of the command, and enclosed in double quotes.

show role predefined

This command is used to display the pre-defined roles of the system and the roles of the administrative privilege separation function, and the “Deny” and “Permit” privilege rules of them. By default, the system provides two pre-defined roles: “SLB” and “NETWORK”. The administrative privilege separation function provides three predefined roles: “role_administrator”, “role_operator” and “role_auditor”.

For example:

```
AN(config)#show role predefined
role name "SLB"
role permit "SLB" "cache"
role permit "SLB" "filter"
role permit "SLB" "health"
```

```

role permit "SLB" "http"
role permit "SLB" "sip"
role permit "SLB" "slb"
...

role name "NETWORK"
role permit "NETWORK" "bond"
role permit "NETWORK" "dns"
role permit "NETWORK" "fwd"
role permit "NETWORK" "hostname"
role permit "NETWORK" "interface"
role permit "NETWORK" "ip"
role permit "NETWORK" "ipregion"
role permit "NETWORK" "ipv6"
role permit "NETWORK" "mnet"
...
role name "role_administrator"
role permit "role_administrator" "aaa"
role permit "role_administrator" "accessgroup"
role permit "role_administrator" "accesslist"
role permit "role_administrator" "acl"
role permit "role_administrator" "activationserverRemote"
role permit "role_administrator" "admin"
...
role name "role_operator"
role permit "role_operator" "aaa"
role permit "role_operator" "accessgroup"
role permit "role_operator" "accesslist"
role permit "role_operator" "acl"
role permit "role_operator" "activationserverRemote"
role permit "role_operator" "admin"
...
role name "role_auditor"
role permit "role_auditor" "show"
role permit "role_auditor" "log"
role permit "role_auditor" "no log"
role permit "role_auditor" "clear log"
role permit "role_auditor" "passwd changing"
role permit "role_auditor" "passwd user"
...

```

role user <user_name> <role_name>

This command is used to assign a specific role to a given administrator, to grant the privileges (configured using the “**role deny**” and “**role permit**” commands) of the specific role to the administrator. One administrator can be assigned up to 8 roles.

user_name	This parameter specifies the administrator name.
role_name	This parameter specifies the role name. The parameter can be configured as the pre-defined roles of the system, or the roles defined by the administrator.



Note:

- If an administrator is not assigned any role, the administrator has the privilege to execute all the commands of his access control level.
- If any role assigned to an administrator permits the execution of a command, then the administrator can execute this command; if all roles assigned to an administrator deny (or none of the roles permits) the execution of a command, the administrator cannot execute this command.
- On the WebUI, after defining a role for an administrator account, the administrator must configure a “Permit” privilege rule to allow this role to perform “**show...**” commands. Otherwise, this administrator account might encounter “500 internal error”.

no role user <user_name> <role_name>

This command is used to delete the assigned role to a given administrator. If the role to be deleted is the last role assigned to the administrator, the administrator will become a “super administrator”—who has the privilege to execute all the commands permitted by his access control level.

If the administrative privilege separation function is enabled, after executing this command, the association between the users and roles of the administrative privilege separation function will remain. If the administrative privilege separation function is disabled, after executing this command, the association between the users and roles of the administrative privilege separation function will be deleted.

show role user [user_name] [role_name]

This command is used to display the assigned roles to a given administrator.

user_name	Optional. Specify the role name. If not specified, all roles of all the administrators will be displayed.
role_name	Optional. Specify the role name. This parameter will only work when the parameter “user_name” is specified. If this parameter is not specified, all roles assigned to the given administrator will be displayed.

clear role user [user_name]

This command is used to delete all roles assigned to the given administrator.

If the administrative privilege separation function is enabled, after executing this command, the association between the users and roles of the administrative privilege separation function will remain. If the administrative privilege separation function is disabled, after executing this command, the association between the users and roles of the administrative privilege separation function will be cleared.

user_name	Optional. Specify the administrator name.
-----------	---

If not specified, all roles assigned to the all administrators will be deleted. Consequently, all administrators have the privilege to execute all the commands permitted by his access control level.

Administrative Privilege Separation

admin mode separation on

This command is used to enable the administrative privilege separation function. After this function is turned on, the system will automatically create three Config-level administrative users with three types of privileges, namely, administrator, operator and auditor. Meanwhile, the system will automatically assign the three users their respective roles, namely, role_administrator, role_operator and role_auditor, which are predefined in the system. When this function is enabled, existing Config level users will become super administrators who have the privilege to execute all the commands permitted by their access control level and have the privilege to modify the passwords of all users. Newly created users can only execute commands after they are assigned a role. This function is disabled by default.

Privileged operations of the three administrative users are as follows:

- Administrator: can change the passwords of all users except other administrators and can execute all commands except log operations.
- Operator: can change only its own password and can execute all commands except log, role and user operations.
- Auditor: can change only its own password. Auditor can execute “show” commands and commands that enable or disable the logging function, display the logging configuration, and clear the log buffer.

For more details about the privileged operations of the three administrative users, execute the commands “**show role name**”, “**show role permit**” and “**show role deny**” to check.



Note:

- After turning on the administrative privilege separation function, save configure by executing the “**write memory**” command. Otherwise, when the appliance is rebooted, the administrative privilege separation mode will be turned off and users created under the administrative privilege separation mode will be automatically deleted.
- If the three administrative users already exist when the administrative privilege separation function is enabled, their password will be reset to the default password “admin”.
- If the amount of newly created administrative users supported by the system before turning on the administrative privilege separation function is less than 3, the administrative privilege separation function cannot be

enabled.

admin mode separation off

This command is used to disable the administrative privilege separation function. When this function is disabled, users created under the administrative role separation mode cannot log into the appliance through SSH connection or WebUI. After this function is disabled, if there is no default account “Array”, the system will automatically create a default account; if there is a default account, its password will be reset to “admin”.



Note: When the administrative privilege separation function is disabled, users created when the function is enabled can still log into the appliance via API, and AAA external users can still log in.

show admin mode separation

This command is used to display the status of the administrative privilege separation function.

WebUI Two-Factor Authentication

webui twofactor on <username>

Enables the WebUI two-factor authentication for an administrator. After this feature is enabled, the specified administrator needs to pass the two-factor authentication to sign in the WebUI. By default, this feature is disabled.

username	The username of the administrator who needs to sign in the WebUI using two-factor authentication.
----------	---

webui twofactor off <username>

Disables the WebUI two-factor authentication for an administrator.

show webui twofactor user [username]

Displays the configuration of the WebUI two-factor authentication of an administrator. If the **username** parameter is not specified, the configurations of the WebUI two-factor authentication of all administrators will be displayed.

Configuration Synchronization Commands

The Configuration Synchronization feature of the APV appliance allows administrators to transfer configuration information among APV appliances within the same network.

Configuration Synchronization

synconfig secure {on|off}

This command is used to enable or disable the configuration synchronization secure mode. When using this command, please ensure that the configuration synchronization secure mode is enabled or disabled on both local unit and peer unit. By default, the configuration synchronization secure mode is enabled.

If the configuration synchronization secure mode is enabled, the administrator needs to perform the following configurations before executing the command “**synconfig from**” or “**synconfig to**”:

- Configure the same Challenge Code on both local unit and peer unit using the command “**synconfig challenge**”.
- Configure synchronization peers on both local unit and peer unit using the command “**synconfig peer**”.

If the configuration synchronization secure mode is disabled, the administrator needs to perform the following configurations before executing the command “**synconfig from**” or “**synconfig to**”:

- Configure the same Challenge Code on both local unit and peer unit using the command “**synconfig challenge**”.
- Configure synchronization peers on local unit only using the command “**synconfig peer**”.



Note: For the sake of security, it is strongly recommended to keep the configuration synchronization secure mode enabled.

show synconfig secure

This command is used to display the status of the configuration synchronization secure mode. The output “**synconfig secure off**” indicates that the configuration synchronization secure mode is disabled, and null indicates the enabled status.

synconfig challenge <code>

This command is used to configure a Challenge Code for system configuration synchronization. When performing the configuration synchronization, the administrator must configure identical Challenge Codes on both local unit and peer unit; otherwise, the configuration synchronization fails. For the sake of security, it is strongly recommended to configure a new Challenge Code.

code	Specify the challenge code. The value should contain 1 to 31 characters. The value is case-sensitive. If the challenge code begins with a numeric character, the challenge code needs to be framed in double quotes.
------	--

show synconfig challenge

This command is used to display the currently configured challenge code.



Note: The displayed challenge code is in encrypted format. The administrator must securely record the original challenge code.

no synconfig challenge

This command is used to delete the configured challenge code.

clear synconfig challenge

This command is used to clear the configured challenge code.

synconfig peer <peer_name> <peer_ip>

This command is used to define a synchronization peer. The system supports definition of a maximum of 128 synchronization peers.

peer_name	This parameter specifies the peer's name. Its value must be a string of 1 to 63 characters (not necessarily a DNS name).
-----------	--

If the parameter value contains special characters, it should be enclosed in double quotes.

peer_ip	This parameter specifies the peer's IP address. The value can be either an IPv4 or IPv6 address, which is NOT interface dependent.
---------	--

show synconfig peer

This command is used to show the details of all peer nodes currently configured.

no synconfig peer <peer_name>

This command is used to remove a specific peer entry.

peer_name	Specify the peer's name. The name should be a quoted string (not necessarily a DNS name).
-----------	---

clear synconfig peer

This command is used to clear all peer settings.

synconfig to <peer_name>

This command is used to synchronize the configuration of the local node to the peer node. After administrator executes this command, the peer node will receive the running configuration of the local node. After synchronizing, the synchronized running configuration will become the current configuration of the peer node.

When executing this command in the segment mode, administrators need to create a segment and segment administrator account with the same name on both the local node and the peer node. In addition, the segment administrator account should have the “Config” permission level on both the local node and the peer node.

peer_name	This parameter specifies the name of the peer node. If its value is set to “all”, all nodes defined by the “ synconfig peer ” command will be synchronized.
-----------	--



Note: When this command is run in a segment to synchronize segment configurations, the local unit and peer unit must configure the same segment name. Furthermore, running this command cannot synchronize the segment name.

synconfig from <peer_name> [segment_name]

This command is used to synchronize all configurations from the specified peer. This command can only synchronize the peer's saved configurations rather than the running configurations. The peer name specified here must be first defined by the command “**synconfig peer**”.

Unlike the command “**synconfig to**”, the newly synchronized configurations by using this command will NOT be saved on disk automatically. The administrator needs to run the “**write memory**” command to save the configurations into disk. Before executing “**synconfig to**” and “**synconfig from**”, the system will check the software version consistency. The software versions are considered consistent as long as the first two digits of the version numbers are the same.

When executing this command in the segment mode, the administrator needs to create a segment with the same name both on the local node and peer node.

peer_name	This parameter specifies the name of an existing peer unit.
segment_name	Optional. This parameter specifies the name of an existing segment. If this parameter is not specified, this command will synchronize all configurations from the peer unit.

The default value is empty.



Note: When this command is run in a segment to synchronize segment configurations, the local unit and peer unit must configure the same segment name. Furthermore, running this command cannot synchronize the segment name.

synconfig increment {enable|disable}

This command is used to enable or disabled the incremental synchronization function. By default, this function is disabled. After this function is enabled, the system will record subsequent configurations until this function is disabled. After this function is disabled, the system will filter the recorded configurations based on the system whitelist list, and then form a new list. The new list will be filtered according to the system blacklist to get the incremental synchronization list.

After that, the administrator can synchronize the configuration in the incremental synchronization list to the specified peer unit or all peer units by executing the “**synconfig increment to**” command.

The following table lists all configuration items in the system whitelist.

ssl
no ssl
slb
no slb
http
no http
health
no health
cache
no cache
diameter
no diameter
epolicy
no epolicy
ftp
no ftp
http2
filter
no filter
acl
no acl
system mode
ha group
no ha group
ip pool
no ip pool
sdns
no sdns

The following table lists all configuration items in the system blacklist.

ssl restore
no ssl sni crlca
no ssl sni interca
ssl activate
sdns config
no sdns config



Note: The incremental synchronization function cannot be used together with the HA runtime synconfig function.

synconfig increment to <peer_name>

This command is used to synchronize incremental configurations to the specified peer unit or all peer units. Before performing this operation, please disable the incremental synchronization function by the command “**synconfig increment disable**”.

peer_name	This parameter specifies the name of an existing peer unit. If the value is set to “all”, incremental configurations will be synchronized to all peer units.
-----------	--

show synconfig increment config

This command is used to display the enabling status of the incremental synchronization function and configurations for incremental synchronization.

Synchronization Rollback

Note: In scenarios where the segmentation function is deployed, if needing to restore segmentation configurations via the “**synconfig rollback**” commands, the administrator must first save segmentation configurations by running the “**write segment memory**” command. Otherwise, the restoration operation will fail

synconfig rollback local <peer_name>

This command is used to roll back the configuration synchronization of the local unit. After executing “**synconfig from**” on the local unit or “**synconfig to**” on the peer unit, you can use this command on the local unit to restore the local unit to its original configurations.

peer_name	This parameter specifies the peer unit’s name.
-----------	--

synconfig rollback peer <peer_name>

This command is used to roll back the configuration synchronization of the peer unit. After executing “**synconfig to**” on the local unit or “**synconfig from**” on the peer unit, you can use this command on the local unit to restore the peer unit to its original configurations.

peer_name	This parameter specifies the peer unit’s name. When the parameter value is “all”, all peer units will be restored to their original configurations.
-----------	---

show synconfig status from [peer_ip]

This command is used to display the synchronization results from other peers to the local node, including the time, remote IP, and status information.

<code>peer_ip</code>	Specify the peer's IP address. The value can be either an IPv4 or IPv6 address. If the “peer_ip” parameter is not specified, the results of synchronization from all peer nodes which the local node has been synchronized with will be displayed.
----------------------	--

Other Commands

show synconfig status history

This command is used to show the history of the last 50 synchronization events executed on the local node. The time of the command, the peer and the command will be displayed. If a command fails, the error of the command will be displayed on the next line.

clear synconfig status

This command is used to clear all configuration synchronization logs.

show synconfig diff <name>

This command is used to display the difference between the running configurations on the local node and the running configurations on the peer (including the running configurations on all partitions on the local node and the peer).

For configuration items that cannot be synchronized by the system, the system lists them in a blacklist.

The following table lists all the configuration items in the blacklist:

ip address
ip route
ip dhcp
ssh ip
ssh auth
bond
hostname
mnet
vlan
webui ip
webui twofactor
restapi ip
xmlrpc ip
ip redundant
cluster virtual priority
sdns dps localdetector
interface name
interface speed
interface mac
interface management
interface shutdown
ha ssf peer
user

SDNS Configuration Synchronization Commands

The SDNS Configuration Synchronization feature of the APV appliance allows administrators to synchronize SDNS configurations and BIND 9 zone files except SDNS member and SDNS listening IP address configurations from an APV appliance to its peers.

synconfig sdns to <peer_name>

This command is used to synchronize SDNS configurations and BIND 9 zone files to the specified remote peer.

peer_name	Specify the name of the peer to which the SDNS configurations and BIND 9 zone files are synchronized. The peer name must have been configured using the “ synconfig peer <peer_name> <peer_ip>” command. If the value of the parameter “peer_name” is “all”, SDNS configurations and BIND 9 zone files will be synchronized to all remote peers defined via the “ synconfig peer ” command.
-----------	---



Note: Before executing the “**synconfig sdns to**” command, you need to configure “**synconfig peer**” to add the synconfig peers and configure “**synconfig challenge**” to set the identical synconfig challenge code on each appliance.

SNMP Commands

Simple Network Management Protocol (SNMP) is a widely used network monitoring and control protocol. Data is passed from SNMP agents, which are hardware and/or software processes reporting activity in each network device (hub, router, etc.) to the workstation console used to oversee the network. The agents return information contained in a Management Information Base (MIB), which is a data structure that defines what is obtainable from the device and what can be controlled. The ArrayOS currently supports the SNMP GET and SET requests. For now, the SNMP SET requests can set only the hostname of the APV appliance and the current time of the system.

snmp on [default|v3]

This command is used to enable the SNMP feature and set supported SNMP versions for the SNMP agent. By default, this feature is disabled.

default v3	Optional. This parameter specifies the SNMP versions supported by the SNMP agent. Its value must be: • default: The system supports v1, v2c and v3.
------------	---

- v3: The system only supports v3.

When this parameter is not configured, v1, v2c and v3 are all supported.

snmp off

This command is used to disable the SNMP feature.



Note: The system supports showing statistics through SNMP v2c and v3. It does not support showing statistics through SNMP v1.

show snmp

This command is used to display all information related to the SNMP configuration.

Example:

```
AN(config)#show snmp
snmp off
snmp community "public"
snmp contact ""
snmp location ""
no snmp enable traps
snmp traps loglevel err
snmp ipcontrol off
```

clear snmp

This command is used to reset all SNMP configurations to default.

slb snmp oid virtual <virtual_service> <index>

This command is used to configure a static SNMP OID for the specified virtual service. The static SNMP OID of a virtual service is comprised of a predefined SNMP OID prefix and an OID index configured in the “index” parameter of this command. For example, if the predefined SNMP OID prefix of virtual service is “.1.3.6.1.4.1.7564.19.1.2.2.1.2” and the OID index is configured as “5000”, the complete SNMP OID will be “.1.3.6.1.4.1.7564.19.1.2.2.1.2.5000”. The static SNMP OID will not change once it is configured.

The static SNMP OID has a higher priority than the auto-generated dynamic SNMP OID. If a static SNMP OID is configured for a virtual service, the static SNMP OID will be used firstly by the system. If the static SNMP OID is not configured, the system will use the dynamic SNMP OID. The dynamic SNMP OID might change during usage.

virtual_service This parameter specifies an existing virtual service name.

index This parameter specifies the index of the SNMP OID, namely the last number of the SNMP OID. For appliances

running a system memory smaller than 32 GB, its value must be an integer ranging from 4001 to 8000; for appliances running a system memory of 32 GB or above, its value must be an integer ranging from 8001 to 16,000.

no slb snmp oid virtual <virtual_service>

This command is used to delete the static SNMP OID of the specified virtual service. After the static SNMP OID is deleted, the system will use the dynamic SNMP OID instead.

show slb snmp oid virtual [virtual_service]

This command is used to display the static SNMP OID of the specified virtual service. If the parameter “virtual_service” is not specified, the static SNMP OIDs of all virtual services will be displayed.

clear slb snmp oid virtual

This command is used to clear the static SNMP OIDs of all the virtual services. After the static SNMP OIDs are deleted, the system will use the dynamic SNMP OIDs instead.

slb snmp oid real <real_service> <index>

This command is used to configure a static SNMP OID for the specified real service. The static SNMP OID of a real service is comprised of a predefined SNMP OID prefix and an OID index configured in the “index” parameter of this command. For example, if the predefined SNMP OID prefix of real service is “.1.3.6.1.4.1.7564.19.2.1.1.1.2” and the OID index is configured as “4000”, the complete SNMP OID will be “.1.3.6.1.4.1.7564.19.2.1.1.1.2.4000”. The static SNMP OID will not change once it is configured.

The static SNMP OID has a higher priority than the auto-generated dynamic SNMP OID. If a static SNMP OID is configured for a real service, the static SNMP OID will be used firstly by the system. If the static SNMP OID is not configured, the system will use the dynamic SNMP OID. The dynamic SNMP OID might change during usage.

real_service	This parameter specifies an existing real service name.
index	For appliances running a system memory smaller than 32 GB, its value must be an integer ranging from 4001 to 8000; for appliances running a system memory of 32 GB or above, its value must be an integer ranging from 8001 to 16,000.

no slb snmp oid real <real_service>

This command is used to delete the static SNMP OID of the specified real service. After the static SNMP OID is deleted, the system will use the dynamic SNMP OID instead.

show slb snmp oid real [*real_service*]

This command is used to display the static SNMP OID of the specified real service. If the parameter “real_service” is not specified, the static SNMP OIDs of all real services will be displayed.

clear slb snmp oid real

This command is used to clear the static SNMP OIDs of all the real services. After the static SNMP OIDs are deleted, the system will use the dynamic SNMP OIDs instead.

snmp community <*string*>

This command is used to define the SNMP community string, which is used as a password to control the access from the Network Management System (NMS) to the SNMP agent.

When this command is not configured, the default SNMP community string is public.

The SNMP community string can be modified only when the SNMP agent is disabled.

string This parameter specifies the SNMP community string. Its values must be a string of 1 to 32 characters.



Note: For the sake of security, it is strongly recommended to modify the default SNMP community string to avoid possible system information interception.

Example:

```
AN(config)#snmp community reindeer
```

no snmp community

This command is used to reset the community default to public.

snmp contact <*contact_name*>

This command allows users to establish a contact individual should system situations require it. The “contact_name” parameter may be up to 128 characters in length enclosed in quotes. Example:

```
AN(config)#snmp contact “admin@example.com”
```

no snmp contact

This command allows users to remove the designated contact information.

snmp location <location>

This command allows users to configure the physical location of the APV appliance. The “location” string can be up to 128 characters in length.

Example:

```
AN(config)#snmp location “server room 6”
```

no snmp location

This command is used to remove the previous location entered for the APV appliance.

snmp host <host_ip> [version] [user_name|community_name] [engine_id] [auth_password] [authorization_level] [priv_password] [port] [auth_proto] [priv_proto]

This command allows users to set the SNMP host’s IP address, and its corresponding user or community string for where traps should be sent. The system supports configuring a maximum of 10 SNMP hosts.

host_ip	This parameter specifies the IP address of the host receiving the SNMP Trap message. It can be an IPv4 or IPv6 address.
version	Optional. This parameter specifies the SNMP trap version. Its value must 1, 2 or 3. The default setting is 1.
user_name community_name	Optional. This parameter specifies the trap community string for SNMP v1 and v2, and set the trap user for SNMP v3. The default is “public”. Please note that the system uses MD5 for SNMP v3 authentication.
engine_id	Optional. This parameter specifies the authoritative engine ID of a remote SNMP trap receiver for SNMP v3. Its value must be a string of no more than 32 characters, and the string length must be an even number. This parameter takes effect only when the value of the “version” parameter is set to 3.
auth_password	Optional. This parameter specifies the authentication password. Its value must be a string of 8 to 32 characters. When the parameter value is “default,” the system uses the default engine ID. This parameter takes effect only when the value of the “version” parameter is set to 3.
authorization_level	Optional. This parameter specifies the security authorization level. Its value must be:

- `authNopriv`: indicates that the private password is not required.
- `authPriv`: indicates that the private password is required.

This parameter takes effect only when the value of the “version” parameter is set to 3. The default setting is “authNopriv” which means no private password needed.

<code>priv_password</code>	Optional. This parameter specifies the private password for data encryption used in “authPriv” mode. Its value must be a string of 8 to 32 characters. This parameter takes effect only when the value of the “version” parameter is set to 3.
<code>port</code>	Optional. This parameter specifies the port number of the SNMP host to receive the trap messages. Its value must be an integer ranging from 0 to 65535. The default value is 162.
<code>auth_proto</code>	Optional. This parameter specifies the authentication algorithm used by the SNMP v3 protocol. Its value is case insensitive and must be MD4 or SHA1. This parameter takes effect only when the value of the “version” parameter is set to 3. The default value is MD5.
<code>priv_proto</code>	Optional. This parameter specifies the encryption algorithm used by the SNMP v3 protocol. Its value is case insensitive and must be AES or DES. This parameter takes effect only when the value of the “authorization_level” parameter is set to “authPriv” and the value of the “version” parameter is set to 3. The default value is DES.

no snmp host <host_ip>

This command is used to remove an SNMP host.

snmp enable traps

This command is used to enable the appliance to send generic and enterprise SNMP traps. After this function is enabled, the appliance will send an SNMP trap when any of the following conditions is met:

- Interface Down or Up
- Generation of a system log at the error or above level (Default setting. The log level can be modified via the “**snmp traps loglevel**” command.)

- License expiration within 15 days
- System startup
- System shutdown

no snmp enable traps

This command is used to disable the appliance from sending generic and enterprise SNMP traps.

snmp traps loglevel [*log_level*]

This command is used to set the system log level for triggering SNMP traps. When the system generates a log at this level or above, SNMP traps will be sent.

When this command is not configured, the default system log level is **err**. If **err** is changed to other values, such as **debug**, **info**, or **warning**, it might have impact on performance and generate unexpected result.

log_level	Optional. This parameter specifies the system log level. Its value must be emerg, alert, crit, err, warning, notice, info or debug.
-----------	---

The default value is err.

no snmp traps loglevel

This command is used to reset the system log level for triggering SNMP traps to err.

snmp ipcontrol {on|off}

This command is used to enable or disable SNMP access control based on the client's IP address. This function controls SNMP GET access requests following the View-based Access Control Model (VACM), and does not take effect for SNMPv3 GET requests. By default, this function is disabled.

snmp ippermit <source_ip> {mask|prefix}

This command is used to configure a network segment from which SNMP GET requests are allowed to be received. This command does not take effect for SNMPv3 GET requests.

source_ip	Specify the IP address of the host sending the SNMP GET request. It can be an IPv4 or IPv6 address.
-----------	---

netmask	Specify the netmask or prefix length of the host sending the SNMP GET request.
---------	--

- “netmask” is used for an IPv4 address. It can be a dotted IP address or an integer. If it is an integer, its

value should range from 0 to 32.

- “prefix” is used for an IPv6 address. Its value ranges from 0 to 128.

no snmp ippermit <source_ip> <netmask>

This command is used to remove the specified source NET from the permitted client list.

snmp v3user <user_name> <auth_password> [authNopriv|authPriv] [priv_password]

This command is used to add an SNMP v3 user to the GET request authentication database to control GET requests following User-based Security Model (USM). Note that the system uses MD5 for SNMP v3 authentication. The system supports adding a maximum of 16 SNMP v3 users.

user_name	This parameter specifies the username. Its value must be a string of 32 characters. Numbers, letters, and special characters are supported. When the parameter value contains special characters or only numbers, it must be enclosed in double quotes.
auth_password	This parameter specifies the authentication password. It should be a string with a length of 8-32 characters. When the parameter value contains special characters or only numbers, it must be enclosed in double quotes.
authNopriv authPriv	Specify the security authorization level. The default setting is “authNopriv”. A private password is needed in “authPriv” mode, but not in “authNopriv” mode.
priv_password	Set the private password for encryption in “authPriv” mode. Its length is not less than 8 characters. When the parameter value contains special characters or only numbers, it must be enclosed in double quotes.

no snmp v3user <user_name>

This command is used to remove a specified user from SNMP v3 user database in SNMP.

Troubleshooting Commands

ping {ip address|hostname} [gateway_ip] [custom_parameter]

This command is used to generate a network connectivity echo request directed toward the specified IP address or host name.

hostname	This parameter specifies the hostname or the IP address of the host.
gateway_ip	Optional. This parameter specifies the IP address of the gateway of an LLB link route. When this parameter is specified, the ping packets are sent to the host through the LLB link determined by this parameter.
custom_parameter	Optional. This parameter specifies the custom parameter of the ping packet. Its value must be in the format of “-symbol value” and be enclosed in double quotes. The “-symbol” can be “-s” (packet size), “-i” (interval), “-c” (number of packets) and so on, and the “value” should be a specific figure. For example, “-s 32, -i 10” indicates that the packet size is 32 bytes and the interval is 10 seconds. Multiple parameter values should be separated by commas. The default value is empty, indicating that no custom parameter is specified. For details of the parameter value, please refer to the Ping Tool.

For example:

```
AN(config)#ping 10.8.1.1 "-s 32, -i 10"
```

tracert {ip|hostname} [gateway_ip]

This command is used to trace the route of three packets that are sent to the specified IPv4 address or host. After this command is executed, the system will display the TTL, names and IP addresses of intermediate nodes (routers or gateways), and the round-trip time of each packet to every node.

ip hostname	Specify the hostname or the IP address of the host.
gateway_ip	Optional. This parameter specifies the IPv4 address of the gateway of an LLB link route. When this parameter is specified, the packet is sent to the specified IPv4 address or host through the LLB link determined by this parameter.

ping6 {ipv6 address|hostname} [gateway_ip]

This command is used to generate a network connectivity echo request directed toward the specified IPv6 address or host name. When parameter “hostname” is in the following format: “IPv6_link-local_address % port number” (for example, AN(config)#ping6 "fe80::225:90ff:fe39:9538%port3"), it can also ping a link-local address via specified interface.

hostname	Specify the hostname or the IP address of the host. The hostname can be in the format of “IPv6_link-local_address
----------	---

% port”, such as fe80::225:90ff:fe39:9538%port3.

gateway_ip Optional. This parameter specifies the IP address of the gateway of an LLB link route. When this parameter is specified, the ping packets are sent to the host through the LLB link determined by this parameter.

traceroute6 {ip|hostname} [gateway_ip]

This command is used to trace the route of three packets that are sent to the specified IPv6 address or host. After this command is executed, the system will display the TTL, names and IPv6 addresses of intermediate nodes (routers or gateways), and the round-trip time of each packet to every node.

ip|hostname Specify the hostname or the IPv6 address of the host. The hostname can be in the format of “IPv6_link-local_address % port”, such as fe80::225:90ff:fe39:9538%port3.

gateway_ip Optional. This parameter specifies the IPv6 address of the gateway of an LLB link route. When this parameter is specified, the packet is sent to the specified IPv6 address or host through the LLB link determined by this parameter.

nslookup {ip|hostname} [query_type]

This command is used to query the host name for the specified IP address or the reverse.

ip|hostname Specify the IP address or the hostname.

- If the parameter “ip” is specified, this command is used to query the corresponding host name. It can be an IPv4 or IPv6 address.
- If the parameter “hostname” is specified, this command is used to query the corresponding IP address. It should be an alphanumeric string of up to 63 characters.

query_type Specify the type of the resource record that will be used for DNS query. The supported types are A, NS, MD, MF, CNAME, SOA, MB, MG, MR, NULL, WKS, PTR, HINFO, MINFO, MX, TXT, AAAA, and ANY.

support <ip_address> <netmask|prefix>

This command is used to configure a network segment, within which the users are allowed to log into the APV appliance via the SSH protocol or Console for debugging and testing work.

<code>ip_address</code>	This Parameter specifies the IP address. It can be an IPv4 or IPv6 address.
<code>netmask prefix</code>	This Parameter specifies the netmask or prefix length of the IP address. • “netmask” is used for an IPv4 address. It can be a dotted IP address or an integer. If it is an integer, its value should range from 0 to 32. • “prefix” is used for an IPv6 address. Its value should range from 0 to 128.

no support `<ip_address> <netmask|prefix>`

This command is used to delete the specified network segment, within which the users are allowed to log into the APV appliance via the SSH protocol or Console for debugging and testing work.

show support

This command is used to display all the network segments, within which the users are allowed to log into the APV appliance via the SSH protocol or Console for debugging and testing work.

clear support

This command is used to delete all the network segments, within which the users are allowed to log into the APV appliance via the SSH protocol or Console for debugging and testing work.

Debug Commands

debug enable

This command is used to prepare for collecting debugging data. After this command is executed, the APV appliance will first clean up the existing files where the debugging data is written into and then create new files in the `/var/crash/sys_debug` and `/var/crash/sslkeylog` directories to store the debugging data.

debug disable

This command is used to stop collecting debugging data. After this command is executed, the APV appliance will generate a tar.gz file to store the collected debugging data (such as the `sys_debug.tar.gz`, `sys_snap.tar.gz`, `sys_log.tar.gz`, `sys_core.tar.gz`, `app_core.tar.gz` or `sslkeylog.tar.gz` file) and clean up the collected debugging data.

The following is an example of a generated `sys_debug.tar.gz` file, which only contains the debugging data collected from the moment of executing the “debug enable” command to the moment of executing the “debug disable” command.

```
/var/crash/sys_debug.tar.gz
tcpdump
ssldump
debug.tar.gz (including englog, pipe and loopback information)
```

debug corefile [*core_files_number*]

This command is used to set the number of the system core files to be collected. The value of the number ranges from 0 to 10. The default value is 0, which means do not collect any core file.



Note: Administrators must first execute this command to set the number of core files to be collected before executing the command “**debug snapshot system**” to collect core files (sys_core.tar.gz and app_core.tar.gz). If the value of the number is not specified, the system will not collect any core file.

show debug corefile

This command is used to display the configuration of the “**debug corefile**” command.

debug snapshot system

This command is used to take a snapshot for the system activities and generate the following four files to save the snapshot information by categories:

- sys_snap.tar.gz
- sys_log.tar.gz
- sys_core.tar.gz
- app_core.tar.gz

All these files will provide more comprehensive system running information for administrators to do better debugging. Administrators can collect the desired files for the system running information they need.



Note: The content displayed by the command **show system ipmi <sel> <list>** and **show system ipmi <sdr> <list>** is saved in the **system.xxx** file in the zipped **sys_snap.tar.gz** package generated by this command.

show debug file

This command is used to display all the generated tarball files in the system, including sys_snap.tar.gz, sys_log.tar.gz, sys_core.tar.gz, app_core.tar.gz, sys_debug.tar.gz and sslkeylog.tar.gz.

Example:

```
AN(config)#show debug file
File           Size      Time
sys_snap       774001    May 31 18:22:48 2010
```

sys_snap.0	775002	May 31 18:21:48 2010
sys_snap.1	785003	May 31 18:20:01 2010
sys_log	424000123	May 31 18:22:49 2010
sys_log.0	456231245	May 31 18:21:49 2010
sys_log.1	347234345	May 31 18:20:02 2010
sys_core	200000123	May 31 18:22:12 2010
app_core	142343446	May 31 18:22:12 2010
sys_debug	92000	May 31 18:22:22 2010

debug ftp <user_name> <remote_ftp_ip> [file_name]

This command is used to export the files (such as sys_snap.tar.gz, sys_log.tar.gz, sys_core.tar.gz, app_core.tar.gz, sys_debug.tar.gz or sslkeylog.tar.gz) storing the debugging data into the specified remote FTP server. A time stamp will be inserted into the name of each exported file to differentiate it from other files on the FTP server.

user_name	This parameter specifies the user name of the remote FTP server.
remote_ftp_ip	This parameter specifies the IP address of the remote FTP server. Both IPv4 and IPv6 addresses are supported. The value must be enclosed in double quotes.
file_name	Optional. This parameter specifies the name of the exported file on the FTP server, without the suffix “.tar.gz”. It defaults to “all”, which means exporting all the latest tarball files to the remote FTP server.

Note: The “path” of the FTP URL (ftp://user:password@host:port/path) should be a relative path.

debug scp {username@remote_scp_ip|host} [file_name]

This command is used to export the files (such as sys_snap.tar.gz, sys_log.tar.gz, sys_core.tar.gz, app_core.tar.gz, sys_debug.tar.gz or sslkeylog.tar.gz) storing the debugging data into the specified remote SCP server. A time stamp will be inserted into the name of each exported file to differentiate it from other files on the SCP server.

username@remote_scp_ip host	This parameter specifies the user name and the IP address or host name of the remote SCP server, which must be enclosed in double quotes, such as “test@172.16.13.12”.
file_name	Optional. This parameter specifies the name of the exported file on the remote SCP server, without the suffix “.tar.gz”. It defaults to “all”, which means exporting all the latest tarball files to the remote SCP

server.

debug monitor on [*module name*]

This command is used to enable the debugging monitor function. Once the monitor is on, it will trace the status of the APV appliance and the status information will be logged into a predefined file named as monitor.out0. If the module name is not specified, only the debugging monitor function of the system will be enabled. If the module name is specified, both the debugging monitor function of system and the specified module will be enabled, and all the status information will be logged into the predefined file named as monitor.out0.

module name	Optional. Specify the module name. The value can be SSL, SLB, LLB, GSLB, UProxy (User Proxy) and ORCH (Orchestration). The value is case insensitive. Multiple modules can be specified by repeatedly executing this command.
-------------	---

debug monitor off

This command is used to disable the debugging monitor function of the system and the specified module.

no debug monitor <*module name*>

This command is used to delete the debugging monitor function for the specified module.



Note: To execute this command, disable the debugging monitor function of the system and the specified module first.

debug monitor import ftp <*user_name*> <*remote_ftp_ip*> <*remote_file_path*>

This command allows users to import a customized script from a remote server via ftp. In the customized script, users can input the CLIs which can display the system information they want, and then import the customized script, so that they can collect the debugging information which they want. Please disable “**debug monitor**” before executing this command.

user_name	Specify the username of the remote FTP server from which the customized script file will be imported.
remote_ftp_ip	Specify the IP address of the remote FTP server from which the customized script file will be imported.

remote_file_path Specify the path in which the customized script file to be imported is located on the specified remote FTP server.

Note: The “path” of the FTP URL (ftp://user:password@host:port/path) should be a relative path.

debug monitor import scp <username@remote_address:filepath>

This command allows users to import a customized script from a remote server via SCP. On the customized script, users can input the CLIs which can display the system information they want, and then import the customized script, so that they can collect the debugging information which they want. Please disable “**debug monitor**” before executing this command.

username@remote_address:filepath It requires to be framed in double quotation marks, such as “test@172.16.13.12:/home/test”.

debug monitor export ftp <user_name> <remote_ftp_ip>

This command allows users to export the monitor result file to a remote server via ftp. Please disable “**debug monitor**” before executing this command.

debug monitor export scp <username@remote_address:filepath>

This command allows users to export the monitor result file to a remote server via SCP. Please disable “**debug monitor**” before executing this command.

username@remote address:filepath It requires to be framed in double quotation marks, such as “test@172.16.13.12:/home/test”.

show debug monitor

This command is used to display the monitor configurations, including its status and the customized scripts imported by the users.

debug trace live tcp <interface_name> [tcpdump_argument]

This command is used to trace TCP activities live. The output is displayed on the screen.

interface_name This parameter specifies the name of the interface, which can be a system interface, MNET interface, VLAN interface, VXLAN interface or Bond interface.

tcpdump_argument This parameter specifies the argument of TCPDUMP which is a packet analyzer. It specifies what TCP activities

live will be traced. Arguments c, e, f, n, N, q, T, S, t, v, x, and X are supported on APV. The format of the value is “-argument”, for example, “-T”. If this parameter is not specified, all TCP activities will be traced in real time.

debug trace tcp all [tcpdump_argument]

This command is used to trace TCP activities on all the interfaces in real time. The traced TCP activities will be stored in tcpdump and sslkeylog (if any SSL connections) files. Included in the sslkeylog file are RSA or ECC random numbers and master secret keys of clients.

tcpdump_argument	Optional. This parameter specifies the argument of TCPDUMP, which indicates the type of TCP activity to be traced. TCPDUMP is a packet analyzer. Arguments c, e, f, n, N, q, T, S, t, v, x and X are supported on the APV appliance. The format of the value is “-argument”, for example, “-T”. If this parameter is not specified, all TCP activities will be traced in real time.
------------------	---

debug trace tcp loopback [tcpdump_argument]

This command is used to trace TCP activities on loopback interfaces. The output is written into a new generated file (such as tcpdump_lo0.20090810_160302) in /var/crash/sys_debug/debug directory.

tcpdump_argument	The argument of TCPDUMP which is a packet analyzer. It specifies what TCP activities will be traced. Arguments c, e, f, n, N, q, T, S, t, v, x, and X are supported on APV. The format of the value is “-argument”, for example, “-T”. If this parameter is not specified, all TCP activities will be traced in real time.
------------------	--

debug trace tcp nic [tcpdump_argument]

This command is used to trace TCP activities on all the NICs. The output is written into a new generated file (such as tcpdump_port1.20090810_160508) in /var/crash/sys_debug/nic_trace directory.

tcpdump_argument	The argument of TCPDUMP which is a packet analyzer. It specifies what TCP activities will be traced. Arguments c, e, f, n, N, q, T, S, t, v, x, and X are supported on APV. The format of the value is “-argument”, for example, “-T”. If this parameter is not specified, all TCP activities will be traced in real time.
------------------	--

debug trace tcp pipe0 [tcpdump_argument]

This command is used to trace the TCP activities on pipe0. The output is written into a new generated file (such as tcpdump_pipe0.20090810_160410) in /var/crash/sys_debug/debug directory.

tcpdump_argument The argument of TCPDUMP which is a packet analyzer. It specifies what TCP activities will be traced. Arguments c, e, f, n, N, q, T, S, t, v, x, and X are supported on APV. The format of the value is “-argument”, for example, “-T”. If this parameter is not specified, all TCP activities will be traced in real time.

debug usage memory

This command is used to trace the usage of system mbuf and memory pool. After this command is executed, administrators can use the “**show debug usage mbuf**” and “**show debug usage mpool**” commands to view the usage of system mbuf and memory pool respectively.

To stop the trace, use the “**no debug usage memory**” command.

Example:

```
AN#show debug usage mbuf
Mbuf usage Statistics
index: 1, app: 0x201993a8
Total mbufs: 2094848
Module Name      no of mbufs (col 1) no of mbufs (col 2)
ID_0:            2094847      2094847
ID_1:             1           0
ID_21:           0           1
```

no debug usage memory

This command is used to stop tracing the usage of system mbuf and memory pool.

show debug usage mbuf

This command is used to display the mbuf usage.

show debug usage mpool

This command is used to display the memory pool usage.

show debug usage cpu

This command is used to display the usage of each CPU core. This command is available in enable and above modes.

debug trace proxy

This command is used to trace the proxy activities. The output is written into the englog file.

debug trace live proxy [src_ip] [src_port] [dst_ip] [dst_port] [and|or]

This command is used to trace the proxy activities live. The output is displayed on the screen.

src_ip	The source IP to be traced. It defaults to 0.0.0.0, which means all source IP addresses will be traced live.
src_port	The source port to be traced. It defaults to 0, which means all source ports will be traced live.
dst_ip	The destination IP to be traced. It defaults to 0.0.0.0, which means all destination IP addresses will be traced live.
dst_port	The destination port to be traced. It defaults to 0, which means all destination ports will be traced live.
and or	The relationship between the configured parameters (source ip, source port, destination ip, destination port). "and" will match exact parameters (source ip, source port, destination ip, destination port) and only show those that match. "or" will show the output that matches any one of the given parameters. The default value is "or".

debug trace live event backward <regular_expression>

This command is used to display information in the debug log buffer in reverse order of time.

debug trace live event forward <regular_expression>

This command is used to display information in the debug log buffer in forward order of time.

debug hardware diagnostics [nic_reset_threshold] [memory_error_threshold] [led_on_duration]

This command is used to configure the thresholds for hardware diagnostics. After executing this command, the administrator can execute the “show hardware diagnostics” command to check the diagnosing result.

nic_reset_threshold	Optional. This parameter specifies the NIC reset threshold. Its value ranges from 10 to 60,000. The default value is 500.
memory_error_threshold	Optional. This parameter specifies the memory error threshold. Its value ranges from 1000 to 60,000. The default value is 1000.
led_on_duration	Optional. This parameter specifies the duration that the LED is lit on. Its value ranges from 0 to 65,535, in

seconds. The default value is 300.

show hardware [*diagnostics*]

This command is used to display the hardware configuration or diagnosing result. If the “diagnostics” parameter is not specified, the hardware configuration will be displayed.

diagnostics Optional. This parameter specifies the diagnosing result to be displayed. Its value must be “diagnostics” or null.

The default value is null.

Remote Access Commands

telnet “*host port*”

This command is used to create a Telnet connection to a remote host. ArrayOS supports all standard Telnet parameters under the Unix system. For details, please refer to the technical documentation about Telnet command. This command can be used in Enable mode.

host port Specify the IP address and the port of the remote host.



Note: The parameter(s) configured for this command must be double quoted. If you need to set attributes for the parameters, you should enclose the parameters and the attribute values with single quotes, and then with double quotes. For example, telnet ““192.168.1.24 -l admin””.

Example:

```
AN#telnet ““172.16.2.182 -4””
Trying 172.16.2.182...
Connected to 172.16.2.182 -4.
Escape character is '^]'.
Trying SRA secure login:
User (root): array
Password:
[ SRA accepts you ].....succeed
```

ssh remote “*user@hostname*”

This command is used to create an SSH connection to a remote host. ArrayOS supports all standard SSH parameters under the Unix system. For details, please refer to the technical documentation about OpenSSH command. This command can be used in Enable mode.

user@hostname Specify the user name and the name or IP address of the

remote host.



Note: The parameter(s) configured for this command must be double quoted. If you need to set attributes for the parameters, you should enclose the parameters and the attribute values with double quotes. For example, `ssh remote "192.168.1.24 -p 8888"`.

Example:

```
AN#ssh remote "root@172.16.85.240"
root@172.16.85.240's password:
Linux libh-server1 2.6.32-22-generic #33-Ubuntu SMP Wed Apr 28 13:27:30 UTC 2010 i686
GNU/Linux

Welcome to Ylmf OS!
* Information: http://www.ymf.com/

0 packages can be updated.
0 updates are security updates.

Last login: Wed Apr 20 00:39:35 2011 from 10.3.46.1
root@libh-server1:~#
```

Custom Configuration Script

custom script import <url>

This command is used to import a custom configuration script file from the specified URL address. Please contact Customer Support to sign the script file before importing it.

url This parameter specifies the URL address of the script file. FTP, HTTP, SCP and TFTP URLs are supported. In the URL, the value of the script file name must be a string of 1 to 32 characters (the length does not include the URL path and the suffix of the file name). The value of the script file name only supports letters, numbers, and “.” character. The suffix of the file name must be “.signed.tar.gz”.

Note: The “path” of the FTP URL (ftp://user:password@host:port/path) should be a relative path.

custom script export <file_name> <url>

This command is used to export the specified custom configuration script file to the specified URL address.

file_name This parameter specifies the name of the script file.

- If the parameter value is the name of the script file, the value should be the same as the name of the file imported through the “**custom script import**” command (without the “.signed.tar.gz” suffix).
- If “all” is specified, all script files will be exported.

url This parameter specifies the URL address. FTP, SCP and TFTP URLs are supported.

Note: The “path” of the FTP URL (ftp://user:password@host:port/path) should be a relative path.

custom script exec <file_name> [para_string]

This command is used to execute the specified custom configuration script file and set related parameters.

file_name This parameter specifies the imported script file name. The script file is imported by the command “**custom script import**”. User can view the names of the imported script files by executing the “**show custom script**” command.

para_string This parameter specifies the relevant parameters for the script, the value must be a string of 1 to 64 characters. Only letters, numbers, “.” and “:” characters are supported.

Note: When a script file is defined to enable or disable real services in batches, if one of the real services is already enabled or disabled, the operation will be executed on the real services except this one.

no custom script <file_name>

This command is used to delete the specified custom configuration script file.

show custom script [file_name] [mode]

This command is used to display information about the specified custom configuration script file.

file_name Optional. This parameter specifies the script file name. If empty is specified (use “” to represent empty), information about all script files will be displayed.

The default value is empty.

mode Optional. This parameter specifies the display mode. Its value must be:

- `verbose`: displays script file contents.
- `summary`: displays only script file names.

The default value is “summary”



Note: When a script file is defined to enable or disable real services in batches, if one of the real services is already enabled or disabled, this command will only display this real service after the script file is executed.

clear custom script

This command is used to clear all custom configuration script files imported via the “**custom script import**” command.

clear directory var

Deletes the specific files in the `/var/crash` and `/var/crash/statmon` directory to free up disk space.

Files with the following extension will be deleted:

- `.core`
- `.rrd`
- `.out`

Bandwidth management commands

show bandwidth usage [start_date] [end_date]

This command is used to display the bandwidth usage for a maximum of 400 days. It allows users either to view the bandwidth usage of all days (since the appliance is booted up) or of a specified period. If both start and end dates are unspecified, the system will show all related messages since the date when the appliance is first booted up.

start_date Specify the start date. It defaults to null. The format is “yyyymmdd”, in which “yyyy” specifies year, “mm” specifies month, and “dd” means day. For example, “20100129” stands for Jan 29, 2010. The value of “yyyy” ranges from 2000 to 2037.

Note: The value of the parameter “start_date” should fall behind that of the parameter “end_date”.

end_date Specify the end date. It defaults to null. The parameter format and value range are identical with those of the “start_date” parameter.

show bandwidth violation

This command is used to display all messages recorded whenever the system bandwidth exceeds the licensed bandwidth limit.



Note: For the above two commands, license limit is only supported on vAPV. Therefore, the content of item “Licensed Bandwidth” in the command output of these commands is always displayed as 0 bps on a physical APV.

Login Page Management Commands

system motd {enable|disable}

This command is used to enable or disable the Message of the Day (MOTD) function. Before enabling this function, the administrator needs to import the MOTD configuration file, then the system can display the message on the CLI and WebUI login page. The administrator can customize the message to be displayed in the configuration file. By default, this function is disabled.

system motd import <url>

This command is use to import the MOTD configuration file. Only one MOTD configuration file can be imported into the system.

url This parameter specifies the HTTP or FTP URL of the MOTD configuration file to be imported.

Note: The “path” of the FTP URL (ftp://user:password@host:port/path) should be a relative path.

no system motd import

This command is used to remove the MOTD configuration file. After this command is executed, the MOTD function will become the disabled status.

show system motd config

This command is used to display the current MOTD configuration.

Network Interface Card Management Commands

system portmap

This command is used to save NIC port sorting information to the configuration file. It is applicable to physical APV appliances only.

After the NIC port sorting information is saved, the system will load the configuration file and detects the NICs at next-time system bootup.

- If a new NIC is detected or an existing NIC is replaced, the system will reorder the ports corresponding to all NICs.
- If the number of detected NIC ports is smaller than that in the configuration file but no new NIC is detected, the saved order will be used.
- If the detection finds that a position switch occurred between the same type of NICs with the same speed, the system will consider the NIC ports are not correctly ordered, and therefore reorder the ports on all NICs.

no system portmap

This command is used to clear the NIC port sorting information saved in the configuration file.

To re-save the NIC port sorting information, you need to clear the existing configuration if a new NIC is added; and you also need to reboot the system if a malfunctioned NIC exists.

show system portmap

This command is used to display the NIC port sorting information that is saved and NIC port status, including the number of NIC ports, corresponding port IDs and MAC addresses.

Chapter 26 Cloud

This chapter describes the commands for cloud operations.

Microsoft Azure

This section provides an overview of Microsoft Azure commands for APV. Commands that interact with Azure cloud resources require signing in using **cloud az login** to take effect, while commands that only modify vAPV configuration files can work without signing in.

Basic Settings

cloud az login

Signs in to Azure. You need to have an Azure account. The system will give you an authentication code to verify your account during the sign-in process.

cloud az logout

Signs out of your Azure account.

cloud az show subscription

Displays your Azure account's information and the subscribed services. You need to sign in to Azure to use this command.

High Availability (HA)

cloud az failover trigger

Manually switches the failover IP address from the primary vAPV to the secondary. You need to sign in to Azure to use this command.

cloud az failover recover

Manually restores IP address configuration to the previous status after the primary vAPV recovery. You need to sign in to Azure to use this command.

cloud az set nicconfig <subscription_id> <resource_group_name>
<from_nic_name> <to_nic_name>

Configures a set of network interface settings for HA on Azure. You can configure multiple sets of network interface settings. When you run **cloud az failover trigger** or **cloud az failover recover**, all network interfaces will be used for failover. If the network interface's name is invalid, failover will be failed.

subscription_id Azure subscription ID.

After you sign in to Azure portal, in the **Network interfaces** list, click the network interface you want to use.

On the **Overview** page, you can see the subscription ID.

resource_group_name

The name of the resource group on Azure.

After you sign in to Azure portal, in the **Network interfaces** list, click the network interface you want to use. On the **Overview** page, you can see the resource group.

from_nic_name

The name of the primary vAPV's network interface that will switch over the IP address to the secondary vAPV during a failover.

to_nic_name

The name of the secondary vAPV's network interface that will take over the IP address from the primary vAPV during a failover.

cloud az clear nicconfig

Deletes all network interfaces' settings for HA on Azure.

cloud az show nicconfig

Displays all network interfaces' settings for HA on Azure.

cloud az ha enable

Enables high availability on Azure. You need to sign in to Azure to use this command.

cloud az ha disable

Disables high availability on Azure.

cloud az show ha

Displays if HA is enabled or disabled.

User-Defined Route

cloud az set udrconfig <subscription_id> <resource_group_name>
<failover_vm_name> <target_ip_normal> <target_ip_failover>

Configures a set of user-defined route (UDR) settings on Azure. You can configure multiple sets of UDR settings.

subscription_id

Azure subscription ID.

After you sign in to Azure portal, in the **Route tables** list, click the route table you want to use. On the **Overview** page, you can see the subscription ID.

resource_group_name	The name of the resource group on Azure. After you sign in to Azure portal, in the Route tables list, click the route table you want to use. On the Overview page, you can see the resource group.
failover_vm_name	The virtual machine (VM) to be monitored for failover events. If this VM becomes unavailable, the UDR failover procedure is triggered. When the VM returns to service, the previous UDR setting is restored. The VM doesn't have to be vAPV.
target_ip_normal	The original IP address specified in the UDR when no failover occurs.
target_ip_failover	The IP address to which traffic will be updated in the UDR during a failover.

cloud az clear udrconfig

Deletes all user-defined route settings on Azure.

cloud az show udrconfig

Displays all user-defined route settings on Azure.

cloud az udr enable

Enables the user-defined route settings on Azure. You need to sign in to Azure to use this command.

cloud az udr disable

Disables the user-defined route settings on Azure.

Logging**cloud az log {on|off}**

Enables or disables Azure Logging.

cloud az show log config

Displays if Azure Logging is enabled or disabled.

cloud az show log buffer [entries]

Displays Azure's log.

entries	Determines how many log entries you want to display. The default value is zero (0), which means all log entries will
---------	--

be displayed.

cloud az show log loglevel

Displays the current log's level.

cloud az loglevel <log_level>

Sets Azure's log level. Only the messages at or above the current level are recorded.

Example: Setting level to **info** records **info**, **warning**, and **error** messages, but excludes **debug**.

log_level Specifies the system log level. Valid values are **debug**, **info**, **warning**, or **error**, from lowest to highest severity.

The default value is **info**.

Log level	Description
error (highest)	System-breaking issue requiring immediate action.
warning	Potential problem that needs attention.
info	Regular system activity and status.
debug	Low-level details for troubleshooting.

Amazon Web Services (AWS)

This section provides an overview of Amazon Web Services (AWS) commands for APV. Commands that interact with Amazon cloud resources require signing in using **cloud aws login** to take effect, while commands that only modify vAPV configuration files can work without signing in.

Basic Settings

cloud aws login <access_key> <secret_access_key> <region>

Signs in to access AWS services programmatically. The access key and secret access key are the credentials that determine the AWS services you can use.

access_key A unique identifier that is associated with your AWS account. It needs to be enclosed in double quotes. It's similar to a username and is included in API requests to identify who is making the request.

To obtain this key, you can ask your AWS administrator to generate it for you, or grant you the permission to generate

it by yourself.

`secret_access_key`

It is like a password that's used along with **access_key** to sign programmatic requests. It needs to be enclosed in double quotes. It verifies your identity to AWS and should be kept confidential.

To obtain this key, you can ask your AWS administrator to generate it for you, or grant you the permission to generate it by yourself.

`region`

Where your AWS resources are physically located. Typically it is the location of your network interfaces. It needs to be enclosed in double quotes.

Example:

```
AN(config)#cloud aws login "AKIAIOSFODNN7EXAMPLE"
"wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY" "ap-southeast-2"
```

cloud aws logout

Signs out of AWS services.

cloud aws show account

Displays the AWS account's information.

High Availability (HA)

cloud aws failover trigger

Manually switches the failover IP address from the primary vAPV to the secondary. Before you use this command, you need to use **cloud aws login** and **cloud aws set eniconfig** to sign in to your AWS services and set valid network interfaces, respectively.

cloud aws failover recover

Manually restores IP address configuration to the previous status after the primary vAPV recovery. Before you use this command, you need to use **cloud aws login** and **cloud aws set eniconfig** to sign in to your AWS services and set valid network interfaces, respectively.

cloud aws set eniconfig <eni_source_id> <eni_destination_id>

Configures a set of network interfaces for HA on AWS. You can configure multiple sets of network interfaces. When you run **cloud aws failover trigger** or **cloud aws**

failover recover, all network interfaces will be used for failover. If the ENI ID is invalid, failover will be failed.

<code>eni_source_id</code>	The ID of the primary vAPV's network interface that will switch over the IP address to the secondary vAPV during a failover. The "eni-" string must be included when you enter the ID.
<code>eni_destination_id</code>	The ID of the secondary vAPV's network interface that will take over the IP address from the primary vAPV during a failover. The "eni-" string must be included when you enter the ID.

Example:

```
AN(config)#cloud aws set eniconfig eni-0abc123def456789a eni-09876543210abcdef
```

cloud aws clear eniconfig

Deletes all network interfaces' settings for HA on AWS.

cloud aws show eniconfig

Displays all network interfaces' settings for HA on AWS.

cloud aws remove eniconfig <index>

Deletes a network interface's setting for HA on AWS.

<code>index</code>	The index of a set of network interface. It needs to be enclosed in double quotes.
--------------------	--

cloud aws ha enable

Enables high availability on AWS. You need to sign in to AWS to use this command.

cloud aws ha disable

Disables high availability on AWS.

cloud aws show ha

Displays if HA is enabled or disabled.

Logging

cloud aws log {on|off}

Enables or disables AWS Logging.

cloud aws show log config

Displays if AWS Logging is enabled or disabled.

cloud aws show log buffer *[entries]*

Displays AWS's log.

entries Determines how many log entries you want to display. The default value is zero (0), which means all log entries will be displayed.

cloud aws show log loglevel

Displays the current log's level.

cloud aws loglevel *<log_level>*

Sets AWS's log level. Messages at or above the current level are recorded.

Example: Setting level to **info** records **info**, **warning**, and **error** messages, but excludes **debug**.

log_level Specifies the system log level. Valid values are **debug**, **info**, **warning**, or **error**, from lowest to highest severity.

The default value is **info**.

Log level	Description
error (highest)	System-breaking issue requiring immediate action.
warning	Potential problem that needs attention.
info	Regular system activity and status.
debug	Low-level details for troubleshooting.

Appendix I SNMP OID List

SNMP OID List	Description
.1.3.6.1.4.1.7564	This file defines the private CA SNMP MIB extensions. Added raw CPU and IO counters. SMIv2 version converted from older MIB definitions.
.1.3.6.1.4.1.7564.3.1	Serial Number of the equipment.
.1.3.6.1.4.1.7564.3.2	A table of CPU core utilization, containing core ID and core utilization.
.1.3.6.1.4.1.7564.3.2.1	A conceptual row of the cpuCoresUtilizationTable containing information about the utilization of each CPU core.
.1.3.6.1.4.1.7564.3.2.1.1	Reference index for each cpu Core.
.1.3.6.1.4.1.7564.3.2.1.2	The ID of each CPU core.
.1.3.6.1.4.1.7564.3.2.1.3	The utilization of each CPU core.
.1.3.6.1.4.1.7564.3.3	Software version of the equipment.
.1.3.6.1.4.1.7564.3.4	Model of the equipment.
.1.3.6.1.4.1.7564.3.5	Disk size of the equipment with the unit MB
.1.3.6.1.4.1.7564.3.6	Disk used size of the equipment with the unit MB
.1.3.6.1.4.1.7564.3.7	Disk available size of the equipment with the unit MB
.1.3.6.1.4.1.7564.3.8	Disk used percentage of the equipment
.1.3.6.1.4.1.7564.3.9	Disk available percentage of the equipment
.1.3.6.1.4.1.7564.3.10	The status of ntp - unavailable (0), available (1)
.1.3.6.1.4.1.7564.3.11	Software version of the equipment of the secondary partition
.1.3.6.1.4.1.7564.3.12	System running time
.1.3.6.1.4.1.7564.3.13	Current percentage of system disk space utilization
.1.3.6.1.4.1.7564.3.14	Current percentage of data disk space utilization
.1.3.6.1.4.1.7564.3.15.0	Starting date of the license key. 83-bit license keys are not supported.
.1.3.6.1.4.1.7564.3.16.0	Expiration date of the license key.
.1.3.6.1.4.1.7564.3.17.0	Status of the license key. 0: Valid 1: Not activated 2: Expired
.1.3.6.1.4.1.7564.3.18.0	Number of CPU cores.
.1.3.6.1.4.1.7564.4.1	Current system total available memory.
.1.3.6.1.4.1.7564.4.2	Current percentage of Network memory utilization.
.1.3.6.1.4.1.7564.4.3	Currently used swap space in MB.
.1.3.6.1.4.1.7564.4.4	Current swap space usage.
.1.3.6.1.4.1.7564.4.5	Current percentage of system memory utilization
.1.3.6.1.4.1.7564.4.6	Current percentage of connection memory utilization
.1.3.6.1.4.1.7564.4.7.0	System memory usage of the device. Unit: KB.
.1.3.6.1.4.1.7564.4.8.0	Available system memory of the device. Unit: KB.
.1.3.6.1.4.1.7564.16.1.1	Current status of the reverse proxy cache - on or off.
.1.3.6.1.4.1.7564.16.1.2	Total number of requests received by the reverse proxy cache.
.1.3.6.1.4.1.7564.16.1.3	Total GET requests received by the reverse proxy cache.
.1.3.6.1.4.1.7564.16.1.4	Total HEAD requests received by the reverse proxy cache.

Appendix I SNMP OID List

.1.3.6.1.4.1.7564.16.1.5	Total PURGE requests received by the reverse proxy cache.
.1.3.6.1.4.1.7564.16.1.6	Total POST requests received by the reverse proxy cache.
.1.3.6.1.4.1.7564.16.1.7	Number of current client connections (e.g. from the browsers).
.1.3.6.1.4.1.7564.16.1.8	Number of current backend server connections.
.1.3.6.1.4.1.7564.16.1.9	Requests redirected to HTTPS.
.1.3.6.1.4.1.7564.16.1.10	Requests redirected based on regex match.
.1.3.6.1.4.1.7564.16.1.11	Requests forwarded with rewritten url.
.1.3.6.1.4.1.7564.16.1.12	Locations rewritten to HTTPS.
.1.3.6.1.4.1.7564.16.1.13	Locations rewritten based on regex match.
.1.3.6.1.4.1.7564.16.1.14	Cache skip, cache off.
.1.3.6.1.4.1.7564.16.1.15	We found the requested URL in the cache. The object was fresh and we did not have to revalidate. The object was served from our cache.
.1.3.6.1.4.1.7564.16.1.16	We got an IMS header in the request. We validated the timestamp and decided that the client's copy of this object is fresh. So we generated a 304 response and sent it out to the client.
.1.3.6.1.4.1.7564.16.1.17	Cache hit, reply with Precondition Failed.
.1.3.6.1.4.1.7564.16.1.18	The requested object was found in the cache. However, the request required revalidation (due to client generated revalidate, proxy generated revalidate or proxy generated forced miss).
.1.3.6.1.4.1.7564.16.1.19	The request does not result in a cache table search. Something in the request made us deem it non-cacheable (eg. very long URL, a 'Cache-Control: no-store' header etc.
.1.3.6.1.4.1.7564.16.1.20	Count of times the cache table was searched, no matching entry was founded and a new entry was created. However, note that sometimes, an entry is created temporarily (eg. for an IMS request resulting in a 304) and is deleted after sending it out to the client (delayed delete).
.1.3.6.1.4.1.7564.16.1.21	Cache miss, create new entry, resp noncacheable.
.1.3.6.1.4.1.7564.16.1.22	Cache hit reply using cache + cache reply with 'not modified'.
.1.3.6.1.4.1.7564.17.1	Number of HA groups.
.1.3.6.1.4.1.7564.17.15	A table of HA units.
.1.3.6.1.4.1.7564.17.15.1	An haUnitTable entry containing HA unit information.
.1.3.6.1.4.1.7564.17.15.1.1	Reference index for each HA unit.
.1.3.6.1.4.1.7564.17.15.1.2	Name of the HA unit.
.1.3.6.1.4.1.7564.17.15.1.3	The IP address type of haUnitIpAddress.
.1.3.6.1.4.1.7564.17.15.1.4	The IP address of HA unit.
.1.3.6.1.4.1.7564.17.15.1.5	The port used for the primary link to communicate with other HA units.
.1.3.6.1.4.1.7564.17.15.1.6	Number of HA secondary links.
.1.3.6.1.4.1.7564.17.15.1.7	The status of HA unit - DOWN (0), INIT (1), FFO_LOGIN (2), NETWORK LOGIN (3), SYNC HA LINK (4), SYNC CONFIG (5), SYNC_CONN (6), READY (7), UP (8), CLOSING (9), UNKNOWN (-1).

Appendix I SNMP OID List

.1.3.6.1.4.1.7564.17.25	A table of HA groups.
.1.3.6.1.4.1.7564.17.25.1	An haGroupTable entry containing HA group information.
.1.3.6.1.4.1.7564.17.25.1.1	The HA group table index.
.1.3.6.1.4.1.7564.17.25.1.2	The HA group ID.
.1.3.6.1.4.1.7564.17.25.1.3	The priority of the HA group on the local HA unit.
.1.3.6.1.4.1.7564.17.25.1.4	Enabling status of Preemption, which is used to control whether a higher-priority HA unit preempts a lower-priority HA unit.
.1.3.6.1.4.1.7564.17.25.1.5	The HA group status - disabled (0), incomplete (1), init (2), standby (3) or active (4).
.1.3.6.1.4.1.7564.17.25.1.6	Enabling status of the HA group.
.1.3.6.1.4.1.7564.17.26	A table of HA floating IP addresses.
.1.3.6.1.4.1.7564.17.26.1	An haGroupFipTable entry containing HA floating IP address information.
.1.3.6.1.4.1.7564.17.26.1.1	The index of the HA floating IP address table.
.1.3.6.1.4.1.7564.17.26.1.2	The HA group that contains this HA floating IP address.
.1.3.6.1.4.1.7564.17.26.1.3	The type of the HA floating IP address.
.1.3.6.1.4.1.7564.17.26.1.4	The floating IP addresses contained in the HA group.
.1.3.6.1.4.1.7564.18.1.1	Current maximum possible number of entries in the vrrpTable, which is 255 * (number of interfaces for which a cluster is defined). 255 is the max number of VIPs in a cluster.
.1.3.6.1.4.1.7564.18.1.2	Current number of entries in the vrrpTable.
.1.3.6.1.4.1.7564.18.1.3	A table containing clustering configuration.
.1.3.6.1.4.1.7564.18.1.3.1	An entry in the vrrpTable. Each entry represents a cluster VIP and not the cluster itself. If a cluster has n VIPs, then there will be n entries for the cluster in the vrrpTable (0 <= n <= 255). All the entries in the vrrpTable belonging to a single cluster will have the same values for all the fields except clusterVirIndex and clusterVirAddr.
.1.3.6.1.4.1.7564.18.1.3.1.1	The cluster virtual table index.
.1.3.6.1.4.1.7564.18.1.3.1.2	Cluster identifier.
.1.3.6.1.4.1.7564.18.1.3.1.3	The current state of the cluster.
.1.3.6.1.4.1.7564.18.1.3.1.4	The interface name on which the cluster is defined.
.1.3.6.1.4.1.7564.18.1.3.1.5	A virtual ip address (VIP) in the cluster.
.1.3.6.1.4.1.7564.18.1.3.1.6	Type of authentication being used. none(0) - no authentication simple-text-password(1) - use password specified in cluster virtual for authentication.
.1.3.6.1.4.1.7564.18.1.3.1.7	The password for authentication.
.1.3.6.1.4.1.7564.18.1.3.1.8	This is for controlling whether a higher priority Backup VRRP virtual preempts a low priority Master.
.1.3.6.1.4.1.7564.18.1.3.1.9	VRRP advertisement interval.
.1.3.6.1.4.1.7564.18.1.3.1.10	Priority of the local node in the cluster.
.1.3.6.1.4.1.7564.18.1.3.1.11	The IP address type of clusterVirAddress.
.1.3.6.1.4.1.7564.18.1.3.1.12	A virtual ip address (VIP) in the cluster.
.1.3.6.1.4.1.7564.19.1.1.1	Number of real services currently configured.
.1.3.6.1.4.1.7564.19.1.1.2	A table containing the configuration of real services.

Appendix I SNMP OID List

.1.3.6.1.4.1.7564.19.1.1.2.1	A rsTable entry containing the information of one real service.
.1.3.6.1.4.1.7564.19.1.1.2.1.1	Reference index for each real service.
.1.3.6.1.4.1.7564.19.1.1.2.1.2	The name of the real service.
.1.3.6.1.4.1.7564.19.1.1.2.1.3	The protocol of the real service.
.1.3.6.1.4.1.7564.19.1.1.2.1.4	The real service IP address.
.1.3.6.1.4.1.7564.19.1.1.2.1.5	The port number of the real service.
.1.3.6.1.4.1.7564.19.1.1.2.1.6	Maximum number of connections per real service.
.1.3.6.1.4.1.7564.19.1.1.2.1.8	The current status of real service - up or down..
.1.3.6.1.4.1.7564.19.1.1.2.1.9	Server Average Response Time (in microseconds)
.1.3.6.1.4.1.7564.19.1.1.2.1.10	The IP address type of rsIpAddress.
.1.3.6.1.4.1.7564.19.1.1.2.1.11	The real service IP address.
.1.3.6.1.4.1.7564.19.1.2.1	Number of virtual services currently configured.
.1.3.6.1.4.1.7564.19.1.2.2	A table containing the configuration of virtual services.
.1.3.6.1.4.1.7564.19.1.2.2.1	A vsTable entry containing the configuration of one virtual service.
.1.3.6.1.4.1.7564.19.1.2.2.1.1	Reference index for each virtual service.
.1.3.6.1.4.1.7564.19.1.2.2.1.2	Name of the virtual service.
.1.3.6.1.4.1.7564.19.1.2.2.1.3	The protocol of the virtual service.
.1.3.6.1.4.1.7564.19.1.2.2.1.4	The virtual service IP address.
.1.3.6.1.4.1.7564.19.1.2.2.1.5	The port number of the virtual service.
.1.3.6.1.4.1.7564.19.1.2.2.1.6	Maximum number of connections of the virtual service.
.1.3.6.1.4.1.7564.19.1.2.2.1.7	The IP address type of vsIpAddress.
.1.3.6.1.4.1.7564.19.1.2.2.1.8	The virtual service IP address.
.1.3.6.1.4.1.7564.19.1.3.1	Number of groups currently configured.
.1.3.6.1.4.1.7564.19.1.3.2	A table containing group member configuration.
.1.3.6.1.4.1.7564.19.1.3.2.1	A gpTable entry containing one group member configuration.
.1.3.6.1.4.1.7564.19.1.3.2.1.1	Reference index for each group member.
.1.3.6.1.4.1.7564.19.1.3.2.1.2	Name of the group.
.1.3.6.1.4.1.7564.19.1.3.2.1.3	Name of the real service.
.1.3.6.1.4.1.7564.19.1.3.2.1.4	Metric used to balance real services within the group.
.1.3.6.1.4.1.7564.19.1.4.1	Number of nodes currently configured.
.1.3.6.1.4.1.7564.19.1.4.2	A table containing node member configuration.
.1.3.6.1.4.1.7564.19.1.4.2.1	A nodeTable entry containing one node member configuration.
.1.3.6.1.4.1.7564.19.1.4.2.1.1	Reference index for each node member.
.1.3.6.1.4.1.7564.19.1.4.2.1.2	Name of the node.
.1.3.6.1.4.1.7564.19.1.4.2.1.3	Node IP address.
.1.3.6.1.4.1.7564.19.2.1.1.1.22	SSL handshake success rate of the real service.
.1.3.6.1.4.1.7564.19.2.1.1.1.23	The number of SSL handshake successes of the real service.
.1.3.6.1.4.1.7564.19.2.1.1.1.24	Total number of SSL handshakes of the real service.
.1.3.6.1.4.1.7564.19.2.1.1.1.25	The SNAT port utilization of the real service (displays zero when it is less than 1%).
.1.3.6.1.4.1.7564.19.2.2.1.1.42	SSL handshake success rate of the virtual service.
.1.3.6.1.4.1.7564.19.2.2.1.1.43	The number of SSL handshake successes of the virtual service.
.1.3.6.1.4.1.7564.19.2.2.1.1.44	Total number of SSL handshakes of the virtual service.

Appendix I SNMP OID List

.1.3.6.1.4.1.7564.19.2.3.1.1.7	The method used by a real service group.
.1.3.6.1.4.1.7564.19.2.4.1	A statistics table of the node.
.1.3.6.1.4.1.7564.19.2.4.1.1	A nodeStatsTable entry containing the statistics of one node.
.1.3.6.1.4.1.7564.19.2.4.1.1.1	Reference index for each node.
.1.3.6.1.4.1.7564.19.2.4.1.1.2	Name of the node.
.1.3.6.1.4.1.7564.19.2.4.1.1.3	Number of open connections to the node.
.1.3.6.1.4.1.7564.19.2.4.1.1.4	Number of Connection per second for the node.
.1.3.6.1.4.1.7564.19.2.4.1.1.5	Number of Request per second for the node.
.1.3.6.1.4.1.7564.19.2.4.1.1.6	Total throughput (bits/s) collected at an interval for the node.
.1.3.6.1.4.1.7564.19.2.1.1	Real service statistics table.
.1.3.6.1.4.1.7564.19.2.1.1.1	A rsStatsTable entry containing the statistics of one real service.
.1.3.6.1.4.1.7564.19.2.1.1.1.1	Reference index for each real service.
.1.3.6.1.4.1.7564.19.2.1.1.1.2	Name of the real service.
.1.3.6.1.4.1.7564.19.2.1.1.1.3	Real service IP address.
.1.3.6.1.4.1.7564.19.2.1.1.1.4	The port number of the real service.
.1.3.6.1.4.1.7564.19.2.1.1.1.5	Number of outstanding requests to the real service.
.1.3.6.1.4.1.7564.19.2.1.1.1.6	Number of open connections to the real service.
.1.3.6.1.4.1.7564.19.2.1.1.1.7	The total number of requests sent to the real service.
.1.3.6.1.4.1.7564.19.2.1.1.1.8	The health status (up or down) of the real service.
.1.3.6.1.4.1.7564.19.2.1.1.1.9	The IP address type of realAddress.
.1.3.6.1.4.1.7564.19.2.1.1.1.10	Real service IP address.
.1.3.6.1.4.1.7564.19.2.1.1.1.11	The inbound bandwidth (KB/second) of the real service.
.1.3.6.1.4.1.7564.19.2.1.1.1.12	The outbound bandwidth (KB/second) of the real service.
.1.3.6.1.4.1.7564.19.2.1.1.1.13	The number of connections established per second of the real service.
.1.3.6.1.4.1.7564.19.2.1.1.1.14	Number of Inbound bytes per second for the real service.
.1.3.6.1.4.1.7564.19.2.1.1.1.15	Number of Outbound bytes per second for the real service.
.1.3.6.1.4.1.7564.19.2.1.1.1.16	Number of Inbound packets per second for the real service.
.1.3.6.1.4.1.7564.19.2.1.1.1.17	Number of Outbound packets per second for the real service.
.1.3.6.1.4.1.7564.19.2.1.1.1.18	Number of Total inbound bytes for the real service.
.1.3.6.1.4.1.7564.19.2.1.1.1.19	Number of Total outbound bytes for the real service.
.1.3.6.1.4.1.7564.19.2.1.1.1.20	Number of Total inbound packets for the real service.
.1.3.6.1.4.1.7564.19.2.1.1.1.21	Number of Total outbound packets for the real service.
.1.3.6.1.4.1.7564.19.2.1.2	Number of the available real services whose health status is up.
.1.3.6.1.4.1.7564.19.2.1.3	The total number of connections of all real services.
.1.3.6.1.4.1.7564.19.2.2.1	A statistics table for virtual service.
.1.3.6.1.4.1.7564.19.2.2.1.1	A vsStatsTable entry containing the statistics of one virtual service.
.1.3.6.1.4.1.7564.19.2.2.1.1.1	Reference index for each virtual service.
.1.3.6.1.4.1.7564.19.2.2.1.1.2	Name of the virtual service.
.1.3.6.1.4.1.7564.19.2.2.1.1.3	IP address of the virtual service.
.1.3.6.1.4.1.7564.19.2.2.1.1.4	Port number of the virtual service.
.1.3.6.1.4.1.7564.19.2.2.1.1.5	Number of QoS URL policy hits for the virtual service.
.1.3.6.1.4.1.7564.19.2.2.1.1.6	Number of QoS Hostname policy hits for the virtual service.

Appendix I SNMP OID List

.1.3.6.1.4.1.7564.19.2.2.1.1.7	Number of Persistent Cookie policy hits for the virtual service.
.1.3.6.1.4.1.7564.19.2.2.1.1.8	Number of QoS Cookie hits for the virtual service.
.1.3.6.1.4.1.7564.19.2.2.1.1.9	Number of Default policy hits for the virtual service.
.1.3.6.1.4.1.7564.19.2.2.1.1.10	Number of Persistent URL policy hits for the virtual service.
.1.3.6.1.4.1.7564.19.2.2.1.1.11	Number of Static policy hits for the virtual service.
.1.3.6.1.4.1.7564.19.2.2.1.1.12	Number of QoS Network policy hits for the virtual service.
.1.3.6.1.4.1.7564.19.2.2.1.1.13	Number of QoS URL policy hits for the virtual service.
.1.3.6.1.4.1.7564.19.2.2.1.1.14	Number of Backup policy hits for the virtual service.
.1.3.6.1.4.1.7564.19.2.2.1.1.15	Number of Cache hits for the virtual service.
.1.3.6.1.4.1.7564.19.2.2.1.1.16	Number of Regex policy hits for the virtual service.
.1.3.6.1.4.1.7564.19.2.2.1.1.17	Number of Rewrite Cookie policy hits for the virtual service.
.1.3.6.1.4.1.7564.19.2.2.1.1.18	Number of Insert Cookie policy hits for the virtual service.
.1.3.6.1.4.1.7564.19.2.2.1.1.19	Number of open connections to the virtual service.
.1.3.6.1.4.1.7564.19.2.2.1.1.20	The IP address type of virtualAddress.
.1.3.6.1.4.1.7564.19.2.2.1.1.21	IP address of the virtual service.
.1.3.6.1.4.1.7564.19.2.2.1.1.22	Number of QoS Client Port policy hits for the virtual service.
.1.3.6.1.4.1.7564.19.2.2.1.1.23	Number of QoS Body policy hits for the virtual service.
.1.3.6.1.4.1.7564.19.2.2.1.1.24	Number of Header policy hits for the virtual service.
.1.3.6.1.4.1.7564.19.2.2.1.1.25	Number of Hash URL policy hits for the virtual service.
.1.3.6.1.4.1.7564.19.2.2.1.1.26	Number of Redirect policy hits for the virtual service.
.1.3.6.1.4.1.7564.19.2.2.1.1.27	Total number of bytes received by the virtual service.
.1.3.6.1.4.1.7564.19.2.2.1.1.28	Total number of bytes sent from the virtual service.
.1.3.6.1.4.1.7564.19.2.2.1.1.29	The total number of inbound packets received by the virtual service.
.1.3.6.1.4.1.7564.19.2.2.1.1.30	The total number of outbound packets sent by the virtual service.
.1.3.6.1.4.1.7564.19.2.2.1.1.31	The number of connections established per second of the virtual service.
.1.3.6.1.4.1.7564.19.2.2.1.1.32	The inbound throughput (byte/second) of the virtual service.
.1.3.6.1.4.1.7564.19.2.2.1.1.33	The outbound throughput (byte/second) of the virtual service.
.1.3.6.1.4.1.7564.19.2.2.1.1.34	The number of inbound packets received per second by the virtual service.
.1.3.6.1.4.1.7564.19.2.2.1.1.35	The number of outbound packets sent per second by the virtual service.
.1.3.6.1.4.1.7564.19.2.2.1.1.36	The virtual service health status.
.1.3.6.1.4.1.7564.19.2.2.1.1.37	TCP connection statistics for per virtual service.
.1.3.6.1.4.1.7564.19.2.2.1.1.38	The total number of inbound Bytes received by the virtual service.
.1.3.6.1.4.1.7564.19.2.2.1.1.39	The total number of outbound Bytes sent by the virtual service.
.1.3.6.1.4.1.7564.19.2.2.1.1.40	Throughput of the virtual service.
.1.3.6.1.4.1.7564.19.2.2.1.1.41	The number of concurrent sessions of the virtual service.
.1.3.6.1.4.1.7564.19.2.2.2	The total number of connections of all virtual services.
.1.3.6.1.4.1.7564.19.2.3.1	A statistics table of the group.
.1.3.6.1.4.1.7564.19.2.3.1.1	A gpStatsTable entry containing the statistics of one group.
.1.3.6.1.4.1.7564.19.2.3.1.1.1	Reference index for each group.

Appendix I SNMP OID List

.1.3.6.1.4.1.7564.19.2.3.1.1.2	Name of the group.
.1.3.6.1.4.1.7564.19.2.3.1.1.3	Total hits for the group.
.1.3.6.1.4.1.7564.19.2.3.1.1.4	The group service status, if services included in the group are all down, then the group is down, otherwise is up.
.1.3.6.1.4.1.7564.19.2.3.1.1.5	Total real services for the group
.1.3.6.1.4.1.7564.19.2.3.1.1.6	Total connections per second for the group
.1.3.6.1.4.1.7564.20.1.2	Number of SSL hosts currently configured.
.1.3.6.1.4.1.7564.20.2.1	Total number of open SSL connections (all SSL hosts).
.1.3.6.1.4.1.7564.20.2.2	Total number of accepted SSL connections (all SSL hosts).
.1.3.6.1.4.1.7564.20.2.3	Total number of requested SSL connections (all SSL hosts).
.1.3.6.1.4.1.7564.20.2.4	SSL host statistics table.
.1.3.6.1.4.1.7564.20.2.4.1	sslTable entry for one SSL host.
.1.3.6.1.4.1.7564.20.2.4.1.1	The SSL table index.
.1.3.6.1.4.1.7564.20.2.4.1.2	Name of the SSL host.
.1.3.6.1.4.1.7564.20.2.4.1.3	Open SSL connections for SSL hostName.
.1.3.6.1.4.1.7564.20.2.4.1.4	Number of accepted SSL connections for SSL hostName.
.1.3.6.1.4.1.7564.20.2.4.1.5	Number of requested SSL connections for SSL hostName.
.1.3.6.1.4.1.7564.20.2.4.1.6	Number of resumed SSL sessions for SSL hostName.
.1.3.6.1.4.1.7564.20.2.4.1.7	Number of resumable SSL sessions for SSL hostName.
.1.3.6.1.4.1.7564.20.2.4.1.8	Number of SSL session misses for SSL hostName.
.1.3.6.1.4.1.7564.20.2.4.1.9	Number of SSL connections established per second.
.1.3.6.1.4.1.7564.20.2.4.1.10	SSL handshake success rate of the SSL host.
.1.3.6.1.4.1.7564.20.2.4.1.11	The number of SSL handshake successes of the SSL host.
.1.3.6.1.4.1.7564.20.2.4.1.12	Total number of SSL handshakes of the SSL host.
.1.3.6.1.4.1.7564.20.2.5	Number of newly created connections per second for SSL connections.
.1.3.6.1.4.1.7564.20.2.6	Throughput of the device. (all SSL hosts)
.1.3.6.1.4.1.7564.20.2.7	Number of reusable sessions for the device. (all SSL hosts)
.1.3.6.1.4.1.7564.20.2.8	Number of concurrent sessions for the device. (all SSL hosts)
.1.3.6.1.4.1.7564.20.2.9	SSL handshake success rate of all SSL hosts.
.1.3.6.1.4.1.7564.20.2.10	The number of SSL handshake successes of all SSL hosts.
.1.3.6.1.4.1.7564.20.2.11	Total number of SSL handshakes of all SSL hosts.
.1.3.6.1.4.1.7564.22.1	Status of VIP statistics gathering - on or off.
.1.3.6.1.4.1.7564.22.2	The hostname that the VIP is representing (hostname of the appliance).
.1.3.6.1.4.1.7564.22.3	The current time in the format of MM/DD/YY HH:MM.
.1.3.6.1.4.1.7564.22.4	Total number of ip packets received on all VIPs.
.1.3.6.1.4.1.7564.22.5	Total number of ip packets sent out on all VIPs.
.1.3.6.1.4.1.7564.22.6	Total number of IP bytes received on all VIPs.
.1.3.6.1.4.1.7564.22.7	Total number of IP bytes sent out on all VIPs.
.1.3.6.1.4.1.7564.22.8	A table of VIP statistics.
.1.3.6.1.4.1.7564.22.8.1	An entry in the ipStatsTable is created for each VIP.
.1.3.6.1.4.1.7564.22.8.1.1	The VIP statistics table index.
.1.3.6.1.4.1.7564.22.8.1.2	The VIP address.
.1.3.6.1.4.1.7564.22.8.1.3	Total number of IP packets received on the VIP.

Appendix I SNMP OID List

.1.3.6.1.4.1.7564.22.8.1.4	Total number of bytes received on the VIP.
.1.3.6.1.4.1.7564.22.8.1.5	Total number of packets sent out on the VIP.
.1.3.6.1.4.1.7564.22.8.1.6	Total number of bytes sent out on the VIP.
.1.3.6.1.4.1.7564.22.8.1.7	The time statistics gathering was enabled for the VIP.
.1.3.6.1.4.1.7564.22.8.1.8	The IP address type of ipAddress.
.1.3.6.1.4.1.7564.22.8.1.9	The VIP address.
.1.3.6.1.4.1.7564.23.1	The number of network interfaces present on this system.
.1.3.6.1.4.1.7564.23.2	The total accumulated number of octets received on all the active interfaces (loopback is not included).
.1.3.6.1.4.1.7564.23.3	The total accumulated number of octets transmitted out on all the active interfaces (loopback is not included).
.1.3.6.1.4.1.7564.23.4	A table of interface statistics. The number of entries is given by the value of infNumber.
.1.3.6.1.4.1.7564.23.4.1	An infTable entry for one interface.
.1.3.6.1.4.1.7564.23.4.1.1	A unique value for each interface. Its value ranges between 1 and the value of infNumber. The value for each interface must remain constant at least from one re-initialization of the entity's network management system to the next re-initialization.
.1.3.6.1.4.1.7564.23.4.1.2	Name of the interface.
.1.3.6.1.4.1.7564.23.4.1.3	The current operational state of the interface (up or down).
.1.3.6.1.4.1.7564.23.4.1.4	The interface's IP address.
.1.3.6.1.4.1.7564.23.4.1.5	The total number of octets received on the interface, including framing characters.
.1.3.6.1.4.1.7564.23.4.1.6	The number of packets, delivered by this sub-layer to a higher (sub-)layer, which were not addressed to a multicast or broadcast address at this sub-layer.
.1.3.6.1.4.1.7564.23.4.1.7	The number of packets, delivered by this sub-layer to a higher (sub-)layer, which were addressed to a multicast or broadcast address at this sub-layer.
.1.3.6.1.4.1.7564.23.4.1.8	The number of inbound packets which were chosen to be discarded even though no errors had been detected to prevent their being deliverable to a higher-layer protocol. One possible reason for discarding such a packet could be to free up buffer space.
.1.3.6.1.4.1.7564.23.4.1.9	For packet-oriented interfaces, the number of inbound packets that contained errors preventing them from being deliverable to a higher-layer protocol. For character-oriented or fixed-length interfaces, the number of inbound transmission units that contained errors preventing them from being deliverable to a higher-layer protocol.
.1.3.6.1.4.1.7564.23.4.1.10	For packet-oriented interfaces, the number of packets received via the interface which were discarded because of an unknown or unsupported protocol. For character-oriented or fixed-length interfaces that support protocol multiplexing the number of transmission units received via the interface which were discarded because of an unknown or unsupported protocol. For any interface that does not support protocol multiplexing, this counter will always be 0.

Appendix I SNMP OID List

.1.3.6.1.4.1.7564.23.4.1.11	The total number of octets transmitted out of the interface, including framing characters.
.1.3.6.1.4.1.7564.23.4.1.12	The total number of packets that higher-level protocols requested be transmitted, and which were not addressed to a multicast or broadcast address at this sub-layer, including those that were discarded or not sent.
.1.3.6.1.4.1.7564.23.4.1.13	The total number of packets that higher-level protocols requested be transmitted, and which were addressed to a multicast or broadcast address at this sub-layer, including those that were discarded or not sent.
.1.3.6.1.4.1.7564.23.4.1.14	For packet-oriented interfaces, the number of outbound packets that could not be transmitted because of errors. For character-oriented or fixed-length interfaces, the number of outbound transmission units that could not be transmitted because of errors.
.1.3.6.1.4.1.7564.23.4.1.15	The IP address type of inflIpv4Address(should always ipv4).
.1.3.6.1.4.1.7564.23.4.1.16	The interface's IPv4 address.
.1.3.6.1.4.1.7564.23.4.1.17	The IP address type of inflIpv6Address(should always ipv6).
.1.3.6.1.4.1.7564.23.4.1.18	The interface's IPv6 address.
.1.3.6.1.4.1.7564.23.4.1.19	Inbound throughput (bit/s) collected at an interval for the interface.
.1.3.6.1.4.1.7564.23.4.1.20	Outbound throughput (bit/s) collected at an interval for the interface.
.1.3.6.1.4.1.7564.23.4.1.21	The number of packets, delivered by this sub-layer to a higher (sub-)layer, which were addressed to a multicast address at this sub-layer.
.1.3.6.1.4.1.7564.23.4.1.22	The total number of packets that higher-level protocols requested be transmitted, and which were addressed to a multicast address at this sub-layer, including those that were discarded or not sent.
.1.3.6.1.4.1.7564.23.4.1.23	The number of packets, delivered by this sub-layer to a higher (sub-)layer, which were addressed to a broadcast address at this sub-layer.
.1.3.6.1.4.1.7564.23.4.1.24	The total number of packets that higher-level protocols requested be transmitted, and which were addressed to a broadcast address at this sub-layer, including those that were discarded or not sent.
.1.3.6.1.4.1.7564.23.4.1.25	The number of outbound packets which were chosen to be discarded even though no errors had been detected to prevent their being deliverable to a higher-layer protocol. One possible reason for discarding such a packet could be to free up buffer space.
.1.3.6.1.4.1.7564.23.4.1.26	An estimate of the interface's current bandwidth in 1,000,000 bits per second. For interfaces which do not vary in bandwidth or for those where no accurate estimation can be made, this object should contain the nominal bandwidth. For a sub-layer which has no concept of bandwidth, this object should be zero.
.1.3.6.1.4.1.7564.23.4.1.27	Total throughput (bit/s) collected at an interval for the interface.

Appendix I SNMP OID List

.1.3.6.1.4.1.7564.23.4.1.28	Packets per seconds collected at an interval for the interface.
.1.3.6.1.4.1.7564.23.5	The bandwidth of the total accumulated number of octets received on all the active interfaces (loopback is not included)
.1.3.6.1.4.1.7564.23.6	The bandwidth of the total accumulated number of octets transmitted out on all the active interfaces (loopback is not included)
.1.3.6.1.4.1.7564.23.7	The system wide's packets per seconds.
.1.3.6.1.4.1.7564.23.8	The system in packets per seconds
.1.3.6.1.4.1.7564.23.9	The system out packets per seconds
.1.3.6.1.4.1.7564.23.10	Total throughput (bits/s) collected at an interval for the system
.1.3.6.1.4.1.7564.23.11	A table of bond interface statistics. The number of entries is given by the value of bond number.
.1.3.6.1.4.1.7564.23.11.1	A bondTable entry for one bond interface
.1.3.6.1.4.1.7564.23.11.1.1	A unique value for each bond interface. Its value ranges between 1 and the value of bond number. The value for each bond interface must remain constant at least from one re-initialization of the entity's network management system to the next re-initialization.
.1.3.6.1.4.1.7564.23.11.1.2	Name of the bond interface.
.1.3.6.1.4.1.7564.23.11.1.3	The current operational state of the bond interface (up or down).
.1.3.6.1.4.1.7564.23.11.1.4	The interface added to the bond interface.
.1.3.6.1.4.1.7564.23.11.1.5	The bond interface's IP address.
.1.3.6.1.4.1.7564.23.11.1.6	The IP address type of bondIpv4Address (should always be ipv4).
.1.3.6.1.4.1.7564.23.11.1.7	The bond interface's IPv4 address.
.1.3.6.1.4.1.7564.23.11.1.8	The IP address type of bondIpv6Address (should always be ipv6).
.1.3.6.1.4.1.7564.23.11.1.9	The bond interface's IPv6 address.
.1.3.6.1.4.1.7564.23.11.1.10	The total number of octets received on the bond interface, including frame characters.
.1.3.6.1.4.1.7564.23.11.1.11	The number of bond interface packets, delivered by this sub-layer to a higher (sub-)layer, which were not addressed to a multicast or broadcast address at this sub-layer.
.1.3.6.1.4.1.7564.23.11.1.12	The number of bond interface packets, delivered by this sub-layer to a higher (sub-)layer, which were addressed to a multicast or broadcast address at this sub-layer.
.1.3.6.1.4.1.7564.23.11.1.13	The number of bond interface inbound packets which were chosen to be discarded even though no errors had been detected to prevent their being deliverable to a higher-layer protocol. One possible reason for discarding such a packet is to free up buffer space.

Appendix I SNMP OID List

.1.3.6.1.4.1.7564.23.11.1.14	For packet-oriented bond interfaces, the number of inbound packets that contained errors preventing them from being deliverable to a higher-layer protocol. For character-oriented or fixed-length bond interfaces, the number of inbound transmission units that contained errors preventing them from being deliverable to a higher-layer protocol.
.1.3.6.1.4.1.7564.23.11.1.15	For packet-oriented bond interfaces, it indicates the number of packets received via the bond interface which were discarded because of an unknown or unsupported protocol. For character-oriented or fixed-length bond interfaces that support protocol multiplexing, it indicates the number of transmission units received via the bond interface which were discarded because of an unknown or unsupported protocol. For any bond interface that does not support protocol multiplexing, this counter will always be 0.
.1.3.6.1.4.1.7564.23.11.1.16	The total number of octets transmitted out of the bond interface, including frame characters.
.1.3.6.1.4.1.7564.23.11.1.17	The total number of bond interface packets that higher-level protocols requested be transmitted, and which were not addressed to a multicast or broadcast address at this sub-layer, including those that were discarded or not sent.
.1.3.6.1.4.1.7564.23.11.1.18	The total number of bond interface packets that higher-level protocols requested be transmitted, and which were addressed to a multicast or broadcast address at this sub-layer, including those that were discarded or not sent.
.1.3.6.1.4.1.7564.23.11.1.19	For packet-oriented bond interfaces, it indicates the number of outbound packets that could not be transmitted because of errors. For character-oriented or fixed-length bond interfaces, it indicates the number of outbound transmission units that could not be transmitted because of errors.
.1.3.6.1.4.1.7564.23.11.1.20	Inbound throughput (bit/s) collected at a fixed interval for the bond interface.
.1.3.6.1.4.1.7564.23.11.1.21	Outbound throughput (bit/s) collected at a fixed interval for the bond interface.
.1.3.6.1.4.1.7564.23.11.1.22	The number of bond interface packets, delivered by this sub-layer to a higher (sub-)layer, which were addressed to a multicast address at this sub-layer.
.1.3.6.1.4.1.7564.23.11.1.23	The total number of bond interface packets that higher-level protocols requested be transmitted, and which were addressed to a multicast address at this sub-layer, including those that were discarded or not sent.
.1.3.6.1.4.1.7564.23.11.1.24	The number of bond interface packets, delivered by this sub-layer to a higher (sub-)layer, which were addressed to a broadcast address at this sub-layer.
.1.3.6.1.4.1.7564.23.11.1.25	The total number of bond interface packets that higher-level protocols requested be transmitted, and which were addressed to a broadcast address at this sub-layer, including those that were discarded or not sent.
.1.3.6.1.4.1.7564.23.11.1.26	The number of bond interface outbound packets which were chosen to be discarded even though no errors had been detected to prevent their being deliverable to a higher-layer protocol. One possible reason for discarding such a packet is to free up buffer space.
.1.3.6.1.4.1.7564.23.11.1.27	Total throughput (bit/s) collected at a fixed interval for the bond interface.

Appendix I SNMP OID List

.1.3.6.1.4.1.7564.23.11.1.28	Packets per seconds collected at a fixed interval for the bond interface.
.1.3.6.1.4.1.7564.23.12	A table of VLAN interface statistics. The number of entries is given by the value of VLAN number.
.1.3.6.1.4.1.7564.23.12.1	A vlanTable entry for one interface.
.1.3.6.1.4.1.7564.23.12.1.1	A unique value for each VLAN interface. Its value ranges between 1 and the value of VLAN number. The value for each VLAN interface must remain constant at least from one re-initialization of the entity's network management system to the next re-initialization.
.1.3.6.1.4.1.7564.23.12.1.2	Name of the VLAN interface.
.1.3.6.1.4.1.7564.23.12.1.3	The current operational state of the VLAN interface (up or down).
.1.3.6.1.4.1.7564.23.12.1.4	VLAN parent interface name.
.1.3.6.1.4.1.7564.23.12.1.5	The VLAN interface's IP address.
.1.3.6.1.4.1.7564.23.12.1.6	The IP address type of vlanIpv4Address (should always be ipv4).
.1.3.6.1.4.1.7564.23.12.1.7	The VLAN interface's IPv4 address
.1.3.6.1.4.1.7564.23.12.1.8	The IP address type of vlanIpv6Address (should always be ipv6).
.1.3.6.1.4.1.7564.23.12.1.9	The VLAN interface's IPv6 address.
.1.3.6.1.4.1.7564.23.13	A table of VXLAN interface statistics. The number of entries is given by the value of VXLAN number.
.1.3.6.1.4.1.7564.23.13.1	An vxlanTable entry for one VXLAN interface.
.1.3.6.1.4.1.7564.23.13.1.1	A unique value for each vxlan interface. Its value ranges between 1 and the value of VXLAN number. The value for each VXLAN interface must remain constant at least from one re-initialization of the entity's network management system to the next re-initialization.
.1.3.6.1.4.1.7564.23.13.1.2	Name of the VXLAN interface.
.1.3.6.1.4.1.7564.23.13.1.3	The current operational state of the VXLAN interface (up or down).
.1.3.6.1.4.1.7564.23.13.1.4	VXLAN tunnel bound to VXLAN.
.1.3.6.1.4.1.7564.23.13.1.5	The VXLAN interface's IP address
.1.3.6.1.4.1.7564.23.13.1.6	The IP address type of vxlanIpv4Address (should always be IPv4).
.1.3.6.1.4.1.7564.23.13.1.7	The VXLAN interface's IPv4 address
.1.3.6.1.4.1.7564.23.13.1.8	The IP address type of vxlanIpv6Address (should always be IPv6).
.1.3.6.1.4.1.7564.23.13.1.9	The VXLAN interface's IPv6 address.
.1.3.6.1.4.1.7564.24.1.1	The number of syslog notifications that have been sent. This number may include notifications that were prevented from

Appendix I SNMP OID List

	being transmitted due to reasons such as resource limitations and/or non-connectivity. If one is receiving notifications, one can periodically poll this object to determine if any notifications were missed. If so, a poll of the logHistoryTable might be appropriate.
.1.3.6.1.4.1.7564.24.1.2	Indicates whether logMessageGenerated notifications will or will not be sent when a syslog message is generated by the device. Disabling notifications does not prevent syslog messages from being added to the logHistoryTable.
.1.3.6.1.4.1.7564.24.1.3	Indicates which syslog severity levels will be processed. Any syslog message with a severity value greater than this value will be ignored by the agent. note: severity numeric values increase as their severity decreases, e.g. error(3) is more severe than debug(7).
.1.3.6.1.4.1.7564.24.2.1	The upper limit on the number of entries that the logHistoryTable may contain. A value of 0 will prevent any history from being retained. When this table is full, the oldest entry will be deleted and a new one will be created.
.1.3.6.1.4.1.7564.24.2.2	A table of syslog messages generated by this device. All 'interesting' syslog messages (i.e. severity <= logMaxSeverity) are entered into this table.
.1.3.6.1.4.1.7564.24.2.2.1	A syslog message that was previously generated by this device. Each entry is indexed by a message index.
.1.3.6.1.4.1.7564.24.2.2.1.1	A monotonically increasing integer for the sole purpose of indexing messages. When it reaches the maximum value the agent flushes the table and wraps the value back to 1.
.1.3.6.1.4.1.7564.24.2.2.1.2	The severity of the message.
.1.3.6.1.4.1.7564.24.2.2.1.3	The text of the message. If the text of the message exceeds 255 bytes, the message will be truncated to 254 bytes and a '*' character will be appended, indicating that the message has been truncated.
.1.3.6.1.4.1.7564.24.3.1	When a syslogTrap message is generated by the device a syslogTrap notification is sent. The sending of these notifications can be enabled/disabled via the logNotifications Enabled object.
.1.3.6.1.4.1.7564.25.1	The number of times ClickTCP connections have made a direct transition to the SYN-SENT state from the CLOSED state.
.1.3.6.1.4.1.7564.25.2	The number of times ClickTCP connections have made a direct transition to the SYN-RCVD state from the LISTEN state.
.1.3.6.1.4.1.7564.25.3	The number of times ClickTCP connections have made a direct transition to the CLOSED state from either the SYN-SENT state or the SYN-RCVD state, plus the number of times TCP connections have made a direct transition to the LISTEN state from the SYN-RCVD state.
.1.3.6.1.4.1.7564.25.4	The number of times ClickTCP connections have made a direct transition to the CLOSED state from either the ESTABLISHED state or the CLOSE-WAIT state.
.1.3.6.1.4.1.7564.25.5	The number of ClickTCP connections for which the current state is either ESTABLISHED or CLOSE-WAIT.

Appendix I SNMP OID List

.1.3.6.1.4.1.7564.25.6	The total number of ClickTCP segments received, including those received in error. This count includes segments received on currently established connections.
.1.3.6.1.4.1.7564.25.7	The total number of ClickTCP segments sent, including those on current connections but excluding those containing only retransmitted octets.
.1.3.6.1.4.1.7564.25.8	The total number of segments retransmitted - that is, the number of ClickTCP segments transmitted containing one or more previously transmitted octets.
.1.3.6.1.4.1.7564.25.9	The total number of segments received in error (e.g., bad ClickTCP checksums).
.1.3.6.1.4.1.7564.25.10	The number of ClickTCP segments sent containing the RST flag.
.1.3.6.1.4.1.7564.25.11	A table containing ClickTCP connection-specific information.
.1.3.6.1.4.1.7564.25.11.1	A conceptual row of the ctcConnTable containing information about a particular current TCP connection. Each row of this table is transient, in that it ceases to exist when (or soon after) the connection makes the transition to the CLOSED state.
.1.3.6.1.4.1.7564.25.11.1.1	A unique value for each clicktcp connection.
.1.3.6.1.4.1.7564.25.11.1.2	The state of this TCP connection.
.1.3.6.1.4.1.7564.25.11.1.3	The local IP address for this TCP connection. In the case of a connection in the listen state which is willing to accept connections for any IP interface associated with the node, the value 0.0.0.0 is used.
.1.3.6.1.4.1.7564.25.11.1.4	The local port number for this TCP connection.
.1.3.6.1.4.1.7564.25.11.1.5	The remote IP address for this TCP connection.
.1.3.6.1.4.1.7564.25.11.1.6	The remote port number for this TCP connection.
.1.3.6.1.4.1.7564.25.11.1.7	The IP address type of ctcConnLocalAddress.
.1.3.6.1.4.1.7564.25.11.1.8	The local IP address for this TCP connection. In the case of a connection in the listen state which is willing to accept connections for any IP interface associated with the node, the value 0.0.0.0:: is used.
.1.3.6.1.4.1.7564.25.11.1.9	The IP address type of ctcConnRemAddress.
.1.3.6.1.4.1.7564.25.11.1.10	The remote IP address for this TCP connection.
.1.3.6.1.4.1.7564.27.1.1	The number of real services being checked.
.1.3.6.1.4.1.7564.27.1.2	Health Check statistics table.
.1.3.6.1.4.1.7564.27.1.2.1	A hcStatsTable entry containing health check statistics for one real service.
.1.3.6.1.4.1.7564.27.1.2.1.1	Reference index for each real service being checked.
.1.3.6.1.4.1.7564.27.1.2.1.2	Real service name.
.1.3.6.1.4.1.7564.27.1.2.1.3	Health Check IP address.
.1.3.6.1.4.1.7564.27.1.2.1.4	Health Check port.
.1.3.6.1.4.1.7564.27.1.2.1.5	The status (UP/DOWN) of the health check.
.1.3.6.1.4.1.7564.27.1.2.1.6	The reason why the health check is being marked UP/DOWN.
.1.3.6.1.4.1.7564.27.1.2.1.7	The number of times the health check is down.
.1.3.6.1.4.1.7564.27.1.2.1.8	The number of times the health check is up.
.1.3.6.1.4.1.7564.27.1.2.1.9	The number of connections attempted.
.1.3.6.1.4.1.7564.27.1.2.1.10	The number of successful connections.

Appendix I SNMP OID List

.1.3.6.1.4.1.7564.27.1.2.1.11	The number of connection failures.
.1.3.6.1.4.1.7564.27.1.2.1.12	The IP address type of hcAddress.
.1.3.6.1.4.1.7564.27.1.2.1.13	Health Check IP address.
.1.3.6.1.4.1.7564.27.1.2.2	A health check statistics entry containing the health check statistics for one group member.
.1.3.6.1.4.1.7564.27.1.2.2.1	Reference index for each group being checked.
.1.3.6.1.4.1.7564.27.1.2.2.2	Real service name of group health check.
.1.3.6.1.4.1.7564.27.1.2.2.3	Group health check name.
.1.3.6.1.4.1.7564.27.1.2.2.4	Group health check IP address.
.1.3.6.1.4.1.7564.27.1.2.2.5	Group health check port number.
.1.3.6.1.4.1.7564.27.1.2.2.6	Group health check status (Up/Down).
.1.3.6.1.4.1.7564.27.1.2.2.7	The reason why the group health check is marked as Up/Down.
.1.3.6.1.4.1.7564.27.1.2.2.8	The number of times the group health check is marked as Down.
.1.3.6.1.4.1.7564.27.1.2.2.9	The number of times the group health check is marked as Up.
.1.3.6.1.4.1.7564.27.1.2.2.10	The number of attempts to establish group health check connections.
.1.3.6.1.4.1.7564.27.1.2.2.11	The number of successful group health check connections.
.1.3.6.1.4.1.7564.27.1.2.2.12	The number of unsuccessful group health check connections.
.1.3.6.1.4.1.7564.27.1.2.2.13	Group health check IP address type.
.1.3.6.1.4.1.7564.27.1.2.2.14	Group health check IP address (hexadecimal value).
.1.3.6.1.4.1.7564.27.1.2.3	A health check statistics entry containing the HTTP passive health check statistics for one real service.
.1.3.6.1.4.1.7564.27.1.2.3.1	Reference index for each real server being checked.
.1.3.6.1.4.1.7564.27.1.2.3.2	Real service name of HTTP passive health check.
.1.3.6.1.4.1.7564.27.1.2.3.3	The HTTP passive health check name.
.1.3.6.1.4.1.7564.27.1.2.3.4	The HTTP passive health check IP address.
.1.3.6.1.4.1.7564.27.1.2.3.5	The HTTP passive health check port number.
.1.3.6.1.4.1.7564.27.1.2.3.6	The HTTP passive health check status (Up/Down/Nottraffic).
.1.3.6.1.4.1.7564.27.1.2.3.7	The abnormal response code of HTTP passive health check.
.1.3.6.1.4.1.7564.27.1.2.3.8	The response timeout of HTTP passive health check.
.1.3.6.1.4.1.7564.27.1.2.3.9	The statistical period of HTTP passive health check.
.1.3.6.1.4.1.7564.27.1.2.3.10	The abnormal response threshold of HTTP passive health check.
.1.3.6.1.4.1.7564.27.1.2.3.11	The URL list name of HTTP passive health check.
.1.3.6.1.4.1.7564.27.1.2.3.12	The HTTP passive health check IP address type.
.1.3.6.1.4.1.7564.27.1.2.3.13	The abnormal response number of HTTP passive health check.
.1.3.6.1.4.1.7564.27.1.2.3.14	The timeout response number of HTTP passive health check.
.1.3.6.1.4.1.7564.27.1.2.4	A health check statistics entry containing the TCP passive health check statistics for one real service.
.1.3.6.1.4.1.7564.27.1.2.4.1	Reference index for each real server being checked.
.1.3.6.1.4.1.7564.27.1.2.4.2	Real service name of TCP passive health check.
.1.3.6.1.4.1.7564.27.1.2.4.3	The TCP passive health check name.
.1.3.6.1.4.1.7564.27.1.2.4.4	The TCP passive health check IP address.
.1.3.6.1.4.1.7564.27.1.2.4.5	The TCP passive health check port number.

Appendix I SNMP OID List

.1.3.6.1.4.1.7564.27.1.2.4.6	The TCP passive health check status (Up/Down/Nottraffic).
.1.3.6.1.4.1.7564.27.1.2.4.7	The statistical period of TCP passive health check.
.1.3.6.1.4.1.7564.27.1.2.4.8	The RESET threshold of TCP passive health check.
.1.3.6.1.4.1.7564.27.1.2.4.9	The zero windows threshold of TCP passive health check.
.1.3.6.1.4.1.7564.27.1.2.4.10	The TCP passive health check IP address type.
.1.3.6.1.4.1.7564.27.1.2.4.11	The RESET packet number of TCP passive health check.
.1.3.6.1.4.1.7564.27.1.2.4.12	The ACK packet number of TCP passive health check.
.1.3.6.1.4.1.7564.27.1.2.4.13	The zero windows number of TCP passive health check.
.1.3.6.1.4.1.7564.28.1	Total number of bytes received.
.1.3.6.1.4.1.7564.28.2	Total number of bytes sent.
.1.3.6.1.4.1.7564.28.3	Number of bytes received per second.
.1.3.6.1.4.1.7564.28.4	Number of bytes sent per second.
.1.3.6.1.4.1.7564.28.5	Peak received bytes per second.
.1.3.6.1.4.1.7564.28.6	Peak sent bytes per second.
.1.3.6.1.4.1.7564.28.7	Number of currently active transactions.
.1.3.6.1.4.1.7564.30.1	Current percentage of CPU utilization.
.1.3.6.1.4.1.7564.30.2	Number of connections per second.
.1.3.6.1.4.1.7564.30.3	Number of requests per second.
.1.3.6.1.4.1.7564.30.4	Current percentage of SSL core utilization.
.1.3.6.1.4.1.7564.30.5	Current percentage of compression core utilization.
.1.3.6.1.4.1.7564.30.6	Number of established TCP connections
.1.3.6.1.4.1.7564.30.7	The outbound throughput of all VIPs
.1.3.6.1.4.1.7564.30.8	The inbound throughput of all VIPs
.1.3.6.1.4.1.7564.30.9	Current percentage of SSL AE core utilization, for H-series SSL hardware.
.1.3.6.1.4.1.7564.30.10	Current percentage of SSL SE core utilization, for H-series SSL hardware.
.1.3.6.1.4.1.7564.30.11	The percentage of CPU average utilization per 5 minutes.
.1.3.6.1.4.1.7564.31.1	Total DNS requests.
.1.3.6.1.4.1.7564.31.2	Total successful DNS resolvings.
.1.3.6.1.4.1.7564.31.3	Total failed DNS resolvings.
.1.3.6.1.4.1.7564.31.4	Total DNS requests in the last second.
.1.3.6.1.4.1.7564.31.5	Total successful DNS resolvings in the last second.
.1.3.6.1.4.1.7564.31.6	Total failed DNS resolvings in the last second.
.1.3.6.1.4.1.7564.31.7	Peak DNS requests in a second.
.1.3.6.1.4.1.7564.31.8	Peak successful DNS resolvings in a second.
.1.3.6.1.4.1.7564.31.9	Total DNS requests in the last minute.
.1.3.6.1.4.1.7564.31.10	Total successful DNS resolvings in the last minute.
.1.3.6.1.4.1.7564.31.11	Total failed DNS resolvings in the last minute.
.1.3.6.1.4.1.7564.31.12	Peak DNS requests in a minute.
.1.3.6.1.4.1.7564.31.13	Peak successful DNS resolvings in a minute.
.1.3.6.1.4.1.7564.31.14	Total DNS requests in the last hour.
.1.3.6.1.4.1.7564.31.15	Total successful DNS resolvings in the last hour.
.1.3.6.1.4.1.7564.31.16	Total failed DNS resolvings in the last hour.
.1.3.6.1.4.1.7564.31.17	Peak DNS requests in an hour.

Appendix I SNMP OID List

.1.3.6.1.4.1.7564.31.18	Peak successful DNS resolvings in an hour.
.1.3.6.1.4.1.7564.31.19	Total DNS requests in the last day.
.1.3.6.1.4.1.7564.31.20	Total successful DNS resolvings in the last day.
.1.3.6.1.4.1.7564.31.21	Total failed DNS resolvings in the last day.
.1.3.6.1.4.1.7564.31.22	Peak DNS requests in a day.
.1.3.6.1.4.1.7564.31.23	Peak successful DNS resolvings in a day.
.1.3.6.1.4.1.7564.31.24	Total DNS requests in the last 5 seconds.
.1.3.6.1.4.1.7564.31.25	Total successful DNS resolvings in the last 5 seconds.
.1.3.6.1.4.1.7564.31.26	Total failed DNS resolvings in the last 5 seconds.
.1.3.6.1.4.1.7564.31.27	Peak DNS requests in 5 seconds.
.1.3.6.1.4.1.7564.31.28	Peak successful DNS resolvings in 5 seconds.
.1.3.6.1.4.1.7564.32.1	current cpu temperature of cpu and system.
.1.3.6.1.4.1.7564.32.2	current fan speed.
.1.3.6.1.4.1.7564.32.3	current dual power supply state (0 (ok),1(error)).
.1.3.6.1.4.1.7564.32.4	Current CPU1 fan status
.1.3.6.1.4.1.7564.32.5	Current CPU2 fan status
.1.3.6.1.4.1.7564.32.6	Current System fan1 status
.1.3.6.1.4.1.7564.32.7	Current System fan2 status
.1.3.6.1.4.1.7564.32.8	Current System fan3 status
.1.3.6.1.4.1.7564.32.9	Current System fan4 status
.1.3.6.1.4.1.7564.32.11	Current status of power supply 1 (0=ok, 1=error).
.1.3.6.1.4.1.7564.32.12	Current status of power supply 2 (0=ok, 1=error).
.1.3.6.1.4.1.7564.34.2.1	The number of link routes.
.1.3.6.1.4.1.7564.34.2.2	Link route status table.
.1.3.6.1.4.1.7564.34.2.2.1	A linkStatsTable entry containing status for one link.
.1.3.6.1.4.1.7564.34.2.2.1.1	Reference index for each link being checked.
.1.3.6.1.4.1.7564.34.2.2.1.2	Link route name.
.1.3.6.1.4.1.7564.34.2.2.1.3	Link route gateway.
.1.3.6.1.4.1.7564.34.2.2.1.4	Link route status.
.1.3.6.1.4.1.7564.34.2.2.1.5	Link route response time.
.1.3.6.1.4.1.7564.34.2.2.1.6	Link route up time.
.1.3.6.1.4.1.7564.34.2.2.1.7	Link route down time.
.1.3.6.1.4.1.7564.34.2.2.1.8	Link route down count.
.1.3.6.1.4.1.7564.34.2.2.1.9	Link route down event.
.1.3.6.1.4.1.7564.35.1.1.1	Current number of configured orchestrator listener.
.1.3.6.1.4.1.7564.35.1.1.2	A table containing orchestrator listener configuration.
.1.3.6.1.4.1.7564.35.1.1.2.1	A table containing orchestrator listener configuration.
.1.3.6.1.4.1.7564.35.1.1.2.1.1	Reference index for orchestrator listener.
.1.3.6.1.4.1.7564.35.1.1.2.1.2	Name of the orchestrator listener.

Appendix I SNMP OID List

.1.3.6.1.4.1.7564.35.1.1.2.1.3	Deployment model of the orchestrator listener.
.1.3.6.1.4.1.7564.35.1.1.2.1.4	Uplink interface of orchestrator listener.
.1.3.6.1.4.1.7564.35.1.1.2.1.5	Downlink interface of orchestrator listener
.1.3.6.1.4.1.7564.35.1.2.1	Current number of configured L2 security device.
.1.3.6.1.4.1.7564.35.1.2.2	A table containing L2 security device configuration.
.1.3.6.1.4.1.7564.35.1.2.2.1	A table containing L2 security device configuration.
.1.3.6.1.4.1.7564.35.1.2.2.1.1	Reference index for L2 security device.
.1.3.6.1.4.1.7564.35.1.2.2.1.2	Name of the L2 security device.
.1.3.6.1.4.1.7564.35.1.2.2.1.3	Name of to_interface of L2 security device.
.1.3.6.1.4.1.7564.35.1.2.2.1.4	Name of from_interface of L2 security device.
.1.3.6.1.4.1.7564.35.1.3.1	Current number of configured L3 security device.
.1.3.6.1.4.1.7564.35.1.3.2	A table containing L3 security device configuration.
.1.3.6.1.4.1.7564.35.1.3.2.1	A table containing L3 security device configuration.
.1.3.6.1.4.1.7564.35.1.3.2.1.1	Reference index for L3 security device.
.1.3.6.1.4.1.7564.35.1.3.2.1.2	Name of the L3 security device.
.1.3.6.1.4.1.7564.35.1.3.2.1.3	The to_ip address of L3 security device.
.1.3.6.1.4.1.7564.35.1.3.2.1.4	The from_ip address of L3 security device
.1.3.6.1.4.1.7564.35.1.4.1	Current number of configured L4 security device.
.1.3.6.1.4.1.7564.35.1.4.2	A table containing L4 security device configuration.
.1.3.6.1.4.1.7564.35.1.4.2.1	A table containing L4 security device configuration.
.1.3.6.1.4.1.7564.35.1.4.2.1.1	Reference index for L4 security device.
.1.3.6.1.4.1.7564.35.1.4.2.1.2	The name of SLB real service associated with L4 security service.

Appendix I SNMP OID List

.1.3.6.1.4.1.7564.35.1.4.2.1.3	The from_ip address of L4 security device.
.1.3.6.1.4.1.7564.35.1.5.1	Current number of configured L2 security service.
.1.3.6.1.4.1.7564.35.1.5.2	A table containing L2 security service configuration.
.1.3.6.1.4.1.7564.35.1.5.2.1	A table containing L2 security service configuration.
.1.3.6.1.4.1.7564.35.1.5.2.1.1	Reference index for L2 security service.
.1.3.6.1.4.1.7564.35.1.5.2.1.2	Name of the L2 security service.
.1.3.6.1.4.1.7564.35.1.5.2.1.3	The action when the L2 security service is unavailable.
.1.3.6.1.4.1.7564.35.1.5.2.1.4	SLB group method for L2 security service.
.1.3.6.1.4.1.7564.35.1.5.2.1.5	Number of security device in L2 security service.
.1.3.6.1.4.1.7564.35.1.5.2.1.6	Name of the L2 security device.
.1.3.6.1.4.1.7564.35.1.6.1	Current number of configured L3 security service.
.1.3.6.1.4.1.7564.35.1.6.2	A table containing L3 security service configuration.
.1.3.6.1.4.1.7564.35.1.6.2.1	A table containing L3 security service configuration.
.1.3.6.1.4.1.7564.35.1.6.2.1.1	Reference index for L3 security service.
.1.3.6.1.4.1.7564.35.1.6.2.1.2	Name of the L3 security service.
.1.3.6.1.4.1.7564.35.1.6.2.1.3	The action when the L3 security service is unavailable.
.1.3.6.1.4.1.7564.35.1.6.2.1.4	SLB group method for L3 security service.
.1.3.6.1.4.1.7564.35.1.6.2.1.5	Number of security device in L3 security service.
.1.3.6.1.4.1.7564.35.1.6.2.1.6	Name of the L3 security device.
.1.3.6.1.4.1.7564.35.1.7.1	Current number of configured L4 security service.
.1.3.6.1.4.1.7564.35.1.7.2	A table containing L4 security service configuration.
.1.3.6.1.4.1.7564.35.1.7.2.1	A table containing L4 security service configuration.

Appendix I SNMP OID List

.1.3.6.1.4.1.7564.35.1.7.2.1.1	Reference index for L4 security service.
.1.3.6.1.4.1.7564.35.1.7.2.1.2	Name of SLB virtual service associated with L4 security service
.1.3.6.1.4.1.7564.35.1.8.1	Current number of configured TAP security service.
.1.3.6.1.4.1.7564.35.1.8.2	A table containing TAP security service configuration.
.1.3.6.1.4.1.7564.35.1.8.2.1	A table containing TAP security service configuration.
.1.3.6.1.4.1.7564.35.1.8.2.1.1	Reference index for TAP security service.
.1.3.6.1.4.1.7564.35.1.8.2.1.2	Name of the TAP security service
.1.3.6.1.4.1.7564.35.1.8.2.1.3	Name of the interface used by TAP security service to forward packets.
.1.3.6.1.4.1.7564.35.1.8.2.1.4	Destination MAC address of the TAP security service when forwarding packets.
.1.3.6.1.4.1.7564.35.1.9.1	Current number of configured security service chain.
.1.3.6.1.4.1.7564.35.1.9.2	A table containing security service chain configuration.
.1.3.6.1.4.1.7564.35.1.9.2.1	A table containing security service chain configuration.
.1.3.6.1.4.1.7564.35.1.9.2.1.1	Reference index for security service chain.
.1.3.6.1.4.1.7564.35.1.9.2.1.2	Name of the nodes of security service chain.
.1.3.6.1.4.1.7564.35.1.9.2.1.3	Number of the nodes of security service chain.
.1.3.6.1.4.1.7564.35.1.9.2.1.4	Name of the nodes of security service chain.
.1.3.6.1.4.1.7564.35.2.1.1	A table containing L2/L3/L4 security device.
.1.3.6.1.4.1.7564.35.2.1.1.1	A statistics table containing L2/L3/L4 security device
.1.3.6.1.4.1.7564.35.2.1.1.1.1	Reference index for security device.
.1.3.6.1.4.1.7564.35.2.1.1.1.2	Type of security device (L2, L3 or L4)
.1.3.6.1.4.1.7564.35.2.1.1.1.3	Name of the security device.
.1.3.6.1.4.1.7564.35.2.1.1.1.4	The health status of security device (Up or Down).

Appendix I SNMP OID List

.1.3.6.1.4.1.7564.35.2.1.1.1.5	Total number of bytes of security device to _device interface inbound direction.
.1.3.6.1.4.1.7564.35.2.1.1.1.6	The bps of security device to device interface inbound direction.
.1.3.6.1.4.1.7564.35.2.1.1.1.7	Total packets of security device to _device interface inbound direction.
.1.3.6.1.4.1.7564.35.2.1.1.1.8	The pps of security device to device interface inbound direction.
.1.3.6.1.4.1.7564.35.2.1.1.1.9	Total number of bytes of security device to _device interface outbound direction.
.1.3.6.1.4.1.7564.35.2.1.1.1.10	The bps of security device to device interface outbound direction.
.1.3.6.1.4.1.7564.35.2.1.1.1.11	Total packets of security device to _device interface outbound direction.
.1.3.6.1.4.1.7564.35.2.1.1.1.12	The pps of security device to device interface outbound direction.
.1.3.6.1.4.1.7564.35.2.1.1.1.13	Total number of bytes of security device from device interface inbound direction.
.1.3.6.1.4.1.7564.35.2.1.1.1.14	The bps of security device from device interface inbound direction.
.1.3.6.1.4.1.7564.35.2.1.1.1.15	Total packets of security device from device interface inbound direction.
.1.3.6.1.4.1.7564.35.2.1.1.1.16	The pps of security device from device interface inbound direction.
.1.3.6.1.4.1.7564.35.2.1.1.1.17	Total number of bytes of security device from device interface outbound direction
.1.3.6.1.4.1.7564.35.2.1.1.1.18	The bps of security device from device interface outbound direction.
.1.3.6.1.4.1.7564.35.2.1.1.1.19	Total packets of security device from device interface outbound direction.
.1.3.6.1.4.1.7564.35.2.1.1.1.20	The pps of security device from device interface outbound direction.
.1.3.6.1.4.1.7564.35.2.2.1	A statistics table of the L2/L3/L4security service.
.1.3.6.1.4.1.7564.35.2.2.1.1	A statistics table containing L2/L3/L4security service.
.1.3.6.1.4.1.7564.35.2.2.1.1.1	Reference index for security service.
.1.3.6.1.4.1.7564.35.2.2.1.1.2	Type of security service (L2, L3 or L4).
.1.3.6.1.4.1.7564.35.2.2.1.1.3	Name of the security service.
.1.3.6.1.4.1.7564.35.2.2.1.1.4	Health status of the security service.

Appendix I SNMP OID List

.1.3.6.1.4.1.7564.35.2.2.1.1.5	Total number of bytes of security service to device interface inbound direction.
.1.3.6.1.4.1.7564.35.2.2.1.1.6	The bps of security service to device interface inbound direction.
.1.3.6.1.4.1.7564.35.2.2.1.1.7	Total packets of security service to device interface inbound direction.
.1.3.6.1.4.1.7564.35.2.2.1.1.8	The pps of security service to device interface inbound direction.
.1.3.6.1.4.1.7564.35.2.2.1.1.9	Total number of bytes of security service to device interface outbound direction.
.1.3.6.1.4.1.7564.35.2.2.1.1.10	The bps of security service to device interface outbound direction.
.1.3.6.1.4.1.7564.35.2.2.1.1.11	Total packets of security service to device interface outbound direction.
.1.3.6.1.4.1.7564.35.2.2.1.1.12	The pps of security service to device interface outbound direction.
.1.3.6.1.4.1.7564.35.2.2.1.1.13	Total number of bytes of security service from device interface inbound direction.
.1.3.6.1.4.1.7564.35.2.2.1.1.14	The bps of security service from device interface inbound direction.
.1.3.6.1.4.1.7564.35.2.2.1.1.15	Total packets of security service from device interface inbound direction.
.1.3.6.1.4.1.7564.35.2.2.1.1.16	The pps of security service from device interface inbound direction.
.1.3.6.1.4.1.7564.35.2.2.1.1.17	Total number of bytes of security service from device interface outbound direction
.1.3.6.1.4.1.7564.35.2.2.1.1.18	The bps of security service from device interface outbound direction.
.1.3.6.1.4.1.7564.35.2.2.1.1.19	Total packets of security service from device interface outbound direction.
.1.3.6.1.4.1.7564.35.2.2.1.1.20	The pps of security service from device interface outbound direction.
.1.3.6.1.4.1.7564.35.2.2.1.1.21	The deny packets of security service to device interface.
.1.3.6.1.4.1.7564.35.2.2.1.1.22	The deny packets of security service from device interface.
.1.3.6.1.4.1.7564.35.2.3.1	A statistics table of the TAP security service.
.1.3.6.1.4.1.7564.35.2.3.1.1	A statistics table containing the TAP security service.
.1.3.6.1.4.1.7564.35.2.3.1.1.1	Reference index for the TAP security service.
.1.3.6.1.4.1.7564.35.2.3.1.1.2	Name of the TAP security service.

Appendix I SNMP OID List

.1.3.6.1.4.1.7564.35.2.3.1.1.3	Total number of bytes of TAP security service to device interface outbound direction
.1.3.6.1.4.1.7564.35.2.3.1.1.4	The bps of TAP security service to device interface outbound direction.
.1.3.6.1.4.1.7564.35.2.3.1.1.5	Total packets of TAP security service to device interface outbound direction.
.1.3.6.1.4.1.7564.35.2.3.1.1.6	The pps of TAP security service to device interface outbound direction.
.1.3.6.1.4.1.7564.35.2.4.1	A statistics table of the security service chain.
.1.3.6.1.4.1.7564.35.2.4.1.1	A statistics table containing security service chain.
.1.3.6.1.4.1.7564.35.2.4.1.1.1	Reference index for security service chain.
.1.3.6.1.4.1.7564.35.2.4.1.1.2	Name of the security service chain.
.1.3.6.1.4.1.7564.35.2.4.1.1.3	The health status of security service chain.
.1.3.6.1.4.1.7564.35.2.4.1.1.4	Total number of bytes of security service chain uplink inbound direction.
.1.3.6.1.4.1.7564.35.2.4.1.1.5	The bps of security service chain uplink inbound direction.
.1.3.6.1.4.1.7564.35.2.4.1.1.6	Total packets of security service chain uplink inbound direction.
.1.3.6.1.4.1.7564.35.2.4.1.1.7	The pps of security service chain uplink inbound direction.
.1.3.6.1.4.1.7564.35.2.4.1.1.8	Total number of bytes of security service chain uplink outbound direction.
.1.3.6.1.4.1.7564.35.2.4.1.1.9	The bps of security service chain uplink outbound direction.
.1.3.6.1.4.1.7564.35.2.4.1.1.10	Total packets of security service chain uplink outbound direction.
.1.3.6.1.4.1.7564.35.2.4.1.1.11	The pps of security service chain uplink outbound direction.
.1.3.6.1.4.1.7564.35.2.4.1.1.12	Total number of bytes of security service chain downlink inbound direction.
.1.3.6.1.4.1.7564.35.2.4.1.1.13	The bps of security service chain downlink inbound direction.
.1.3.6.1.4.1.7564.35.2.4.1.1.14	Total packets of security service chain downlink inbound direction.
.1.3.6.1.4.1.7564.35.2.4.1.1.15	The pps of security service chain downlink inbound direction.
.1.3.6.1.4.1.7564.35.2.4.1.1.16	Total number of bytes of security service chain downlink outbound direction

Appendix I SNMP OID List

.1.3.6.1.4.1.7564.35.2.4.1.1.17	The bps of security service chain downlink outbound direction.
.1.3.6.1.4.1.7564.35.2.4.1.1.18	Total packets of security service chain downlink outbound direction.
.1.3.6.1.4.1.7564.35.2.4.1.1.19	The pps of security service chain downlink outbound direction.
.1.3.6.1.4.1.7564.35.2.4.1.1.20	The deny packets of security service chain uplink direction.
.1.3.6.1.4.1.7564.35.2.4.1.1.21	The deny packets of security service chain downlink direction.
.1.3.6.1.4.1.7564.35.2.5.1	A statistics table of the deny-type security method.
.1.3.6.1.4.1.7564.35.2.5.1.1	A table containing the deny-type security method.
.1.3.6.1.4.1.7564.35.2.5.1.1.1	Reference index for the deny-type security method.
.1.3.6.1.4.1.7564.35.2.5.1.1.2	Name of the deny-type security method.
.1.3.6.1.4.1.7564.35.2.5.1.1.3	The statistics of deny packets.
.1.3.6.1.4.1.7564.36.2.1.1	The number of segments.
.1.3.6.1.4.1.7564.36.2.1.2	The table of segmentation configurations.
.1.3.6.1.4.1.7564.36.2.1.2.1	An entry containing a segment's information.
.1.3.6.1.4.1.7564.36.2.1.2.1.1	Reference index for each segment.
.1.3.6.1.4.1.7564.36.2.1.2.1.2	A segment's name.
.1.3.6.1.4.1.7564.36.2.1.2.1.3	Number of connections per second for a segment.
.1.3.6.1.4.1.7564.36.2.1.2.1.4	Number of open connections to a segment.
.1.3.6.1.4.1.7564.36.2.1.2.1.5	The outbound throughput of a segment during a certain amount time (KB/s).
.1.3.6.1.4.1.7564.36.2.1.2.1.6	The inbound throughput of a segment during a certain amount time (KB/s).
.1.3.6.1.4.1.7564.251.1	This trap is sent when the agent starts.
.1.3.6.1.4.1.7564.251.2	This trap is sent when the agent terminates.
.1.3.6.1.4.1.7564.251.3	license remaining days.
.1.3.6.1.4.1.7564.251.4	This trap is sent when one of the power supplies fails.
.1.3.6.1.4.1.7564.251.5	This trap is sent when the failed power supply is restored.
.1.3.6.1.4.1.7564.251.6	Webroot license remaining days.

Appendix I SNMP OID List

Float	A single precision floating-point number. The semantics and encoding are identical for type 'single' defined in IEEE Standard for Binary Floating-Point, ANSI/IEEE Std 754-1985. The value is restricted to the BER serialization of the following ASN.1 type: FLOATTYPE ::= [120] IMPLICIT FloatType (note: the value 120 is the sum of '30'h and '48'h) The BER serialization of the length for values of this type must use the definite length, short encoding form. For example, the BER serialization of value 123 of type FLOATTYPE is '9f780442f60000'h. (The tag is '9f78'h; the length is '04'h; and the value is '42f60000'h.) The BER serialization of value '9f780442f60000'h of data type Opaque is '44079f780442f60000'h. (The tag is '44'h; the length is '07'h; and the value is '9f780442f60000'h.
Synlogseverity	The severity of a syslog message. The enumeration values are equal to the values that syslog uses + 1. For example, with syslog, emergency=0.

Appendix II Functions and Specifications

Specification of the orchestration function:

Specification	Memory			
	<4G	≥4G	≥32G	≥128G
Number of orchestrator listener	0	4	4	4
Number of security device	0	64	128	256
Number of security device	0	16	32	64
Number of security service chain	0	32	64	128
Total number of the security service chain member	0	128	256	512
Number of orchestration rules	0	128	256	512
Total number of orchestration policy	0	64	128	256
Total number of orchestration rules associated with orchestration policies	0	512	1024	2048

Dynamic session entries using related methods:

The number of dynamic session entries that can be created by the system varies by the system memory, as shown in the following table (The actual numbers may be different depending on the memory usage):

Chapter 27 System Memory	Chapter 28 Dynamic Session Entries Using Related Methods
1 GB	512
2 GB	75*1024
4 GB	300*1024
8 GB	1024*1024
16 GB	4*1024*1024
24 GB	4*1024*1024
32 GB	8*1024*1024
64 GB	20*1024*1024
96 GB	30*1024*1024
≥128 GB	40*1024*1024



Note:

- The related methods are sipcidps, sipuidps, radpsun, persistence and diametersid.
- When the number of dynamic session entries reaches the limit, the system will no longer be able to establish new dynamic session entries to persist a session.